See all versions of this document

Vitis High-Level Synthesis User Guide

UG1399 (v2024.2) March 31, 2025

AMD Adaptive Computing is creating an environment where employees, customers, and partners feel welcome and included. To that end, we're removing non-inclusive language from our products and related collateral. We've launched an internal initiative to remove language that could exclude people or reinforce historical biases, including terms embedded in our software and IPs. You may still find examples of non-inclusive language in our older products as we work to make these changes and align with evolving industry standards. Follow this link for more information.

AMD

Table of Contents

Section I: Introduction **10**
- Navigating Content by Design Process 10
- Supported Operating Systems for Vitis HLS 11
- Obtaining a Vitis HLS License 11
- Changed Behavior 13
- Benefits of High-Level Synthesis 14
- Introduction to Vitis HLS Components 16
  - Refactoring C++ Source Code for HLS 19
- Tutorials and Examples 22

Section II: HLS Programmers Guide **23**

Chapter 1: Design Principles **24**
- Three Paradigms for Programmable Logic 25
- Combining the Three Paradigms 32
- Conclusion - A Prescription for Performance 36

Chapter 2: Abstract Parallel Programming Model for HLS **39**
- Control and Data Driven Tasks 41
- Data-driven Task-level Parallelism 42
- Control-driven Task-level Parallelism 45
- Mixing Data-Driven and Control-Driven Models 56
- Summary 58

Chapter 3: Loops Primer **60**
- Pipelining Loops 60
- Unrolling Loops 72
- Merging Loops 74
- Working with Nested Loops 76
- Working with Variable Loop Bounds 79

Chapter 4: Arrays Primer **82**

Mapping Software Arrays to Hardware Memory..... 82
Array Accesses and Performance.....83
Arrays on the Interface.....88
Initializing and Resetting Arrays..... 92
Implementing ROMs.....94
C Simulation with Arrays..... 95

Chapter 5: Functions Primer.....97
Function Inlining..... 97
Function Pipelining..... 98
Function Instantiation..... 99

Chapter 6: Data Types..... 101
Standard Types.....102
Composite Data Types.....107
Arbitrary Precision (AP) Data Types.....123
Arbitrary Precision Floating-Point Library ap_float<int W, int E>..... 129
Global Variables.....133
Pointers..... 133
Vector Data Types..... 145
Bit-Width Propagation.....146

Chapter 7: Unsupported C/C++ Constructs..... 147
System Calls..... 147
Dynamic Memory Usage..... 148
Pointer Limitations.....150
Recursive Functions..... 150
Undefined Behaviors..... 151
Virtual Functions and Pointers..... 152

Chapter 8: Interfaces of the HLS Design.....153
Defining Interfaces..... 153
Vitis HLS Memory Layout Model..... 220
Execution Modes of HLS Designs.....231
Controlling Initialization and Reset Behavior.....248

Chapter 9: Best Practices for Designing with M_AXI Interfaces. 250

Chapter 10: Optimizing Techniques and Troubleshooting Tips...253

Understanding High-Level Synthesis Scheduling and Binding....255
Optimizing Logic....262
Optimizing AXI System Performance....264
Managing Area and Hardware Resources....300
Unrolling Loops in C++ Classes....305
Limitations of Control-Driven Task-Level Parallelism....306
Limitations of Pipelining with Static Variables....311

Section III: Creating Vitis HLS Components....313

Chapter 11: Launching the Vitis Unified IDE....314
Features of the Vitis Unified IDE....316
Using the Flow Navigator....317

Chapter 12: Building and Running an HLS Component....319
Creating an HLS Component....320
Defining the HLS Config File....347
Running C Simulation....357
Running C Synthesis....377
Running C/RTL Co-Simulation....398
Packaging the RTL Design....414
Running Implementation....418
Optimizing the HLS Project....423
Component Comparison Feature....424
L1 Library Wizard Flow....425

Chapter 13: Creating HLS Components from the Command Line....429

Section IV: Vitis HLS Command Reference....433

Chapter 14: vitis, v++, and vitis-run Commands....434
hls_init.tcl....436

Chapter 15: HLS Config File Commands....437
HLS General Options....438
Array Partition Configuration....440
C-Simulation Configuration....441
Co-Simulation Configuration....442

Compile Options....445
Dataflow Configuration....447
Debug Options....450
Interface Configuration....450
Package Options....455
Operator Configuration....457
RTL Configuration....460
Schedule Setting....463
Storage Configuration....463
Implementation Configuration....464
Unroll Setting....466
HLS Optimization Directives....466

Chapter 16: HLS Pragmas....519

pragma HLS aggregate....520
pragma HLS alias....521
pragma HLS allocation....523
pragma HLS array_partition....525
pragma HLS array_reshape....527
pragma HLS array_stencil....530
pragma HLS bind_op....531
pragma HLS bind_storage....535
pragma HLS cache....539
pragma HLS dataflow....540
pragma HLS dependence....543
pragma HLS disaggregate....546
pragma HLS expression_balance....548
pragma HLS function_instantiate....549
pragma HLS inline....550
pragma HLS interface....553
pragma HLS latency....559
pragma HLS loop_flatten....561
pragma HLS loop_merge....563
pragma HLS loop_tripcount....564
pragma HLS occurrence....566
pragma HLS performance....567
pragma HLS pipeline....569
pragma HLS protocol....572

pragma HLS reset....573
pragma HLS stable....575
pragma HLS stream....576
pragma HLS top....578
pragma HLS unroll....579

Chapter 17: HLS Tcl Commands....582
Project Commands....582
Configuration Commands....609
Optimization Directives....631

Section V: Vitis HLS C Driver Reference....684

Chapter 18: AXI4-Lite Slave C Driver Reference....685
X<DUT>_Initialize....685
X<DUT>_CfgInitialize....686
X<DUT>_LookupConfig....686
X<DUT>_Release....687
X<DUT>_Start....687
X<DUT>_IsDone....688
X<DUT>_IsIdle....688
X<DUT>_IsReady....688
X<DUT>_Continue....689
X<DUT>_EnableAutoRestart....689
X<DUT>_DisableAutoRestart....690
X<DUT>_Set_ARG....690
X<DUT>_Set_ARG_vld....690
X<DUT>_Set_ARG_ack....691
X<DUT>_Get_ARG....691
X<DUT>_Get_ARG_vld....691
X<DUT>_Get_ARG_ack....692
X<DUT>_Get_ARG_BaseAddress....692
X<DUT>_Get_ARG_HighAddress....693
X<DUT>_Get_ARG_TotalBytes....693
X<DUT>_Get_ARG_BitWidth....694
X<DUT>_Get_ARG_Depth....694
X<DUT>_Write_ARG_Words....695
X<DUT>_Read_ARG_Words....695
X<DUT>_Write_ARG_Bytes....696

X<DUT>_Read_ARG_Bytes....696
X<DUT>_InterruptGlobalEnable....697
X<DUT>_InterruptGlobalDisable....697
X<DUT>_InterruptEnable....697
X<DUT>_InterruptDisable....698
X<DUT>_InterruptClear....698
X<DUT>_InterruptGetEnabled....699
X<DUT>_InterruptGetStatus....699

Section VI: Vitis HLS Libraries Reference....700

Chapter 19: C/C++ Builtin Functions....701

Chapter 20: Arbitrary Precision Data Types Library....702
Using Arbitrary Precision Data Types....702
C++ Arbitrary Precision Integer Types....705
C++ Arbitrary Precision Fixed-Point Types....727

Chapter 21: HLS Print Function....755

Chapter 22: HLS Math Library....757
HLS Math Library Accuracy....757
HLS Math Library....759
Fixed-Point Math Functions....761
Verification and Math Functions....763
Common Synthesis Errors....767

Chapter 23: HLS Stream Library....768
C Modeling and RTL Implementation....769
Using HLS Streams....770

Chapter 24: HLS Direct I/O....780
Using Direct I/O Streams....780
Mapping Direct I/O Streams to a Wire Interface....781
Mapping Direct I/O Streams to SAXI Lite Interface....783

Chapter 25: HLS Fence Library....787

Chapter 26: HLS Vector Library....790

Chapter 27: HLS Task Library **793**
Support of Scalars/Memory Interfaces in DTLP 794
Tasks and Channels 796
Tasks and Dataflow 799

Chapter 28: HLS Split/Merge Library **801**

Chapter 29: HLS Stream of Blocks Library **806**
Stream-of-Blocks Modeling Style 807
Checking for Full and Empty Blocks 810
Modeling Feedback in Dataflow Regions 812
Limitations 813

Chapter 30: HLS IP Libraries **815**
DSP Intrinsics 815
FFT IP Library 820
FIR Filter IP Library 830
DDS IP Library 838
SRL IP Library 843

Chapter 31: Working with OpenCV **846**

Section VII: Vitis HLS Migration Guide **847**

Chapter 32: Migrating from Vitis HLS to the Vitis Unified IDE **848**

Chapter 33: Migrating from Vivado HLS to Vitis HLS **850**
Key Behavioral Differences 850

Chapter 34: Migration from 2023.2 to 2024.2 (New Host Compiler) **857**

Chapter 35: Deprecated and Unsupported Features **858**

Chapter 36: Unsupported Features **863**
Pragmas 863
HLS Video Library 864
C Arbitrary Precision Types 864

Appendix A: Tcl to Config File Command Map **865**

Appendix B: Instruction/Operator Explanation..... **874**

Appendix C: Additional Resources and Legal Notices..... **876**

Finding Additional Documentation..... 876
Support Resources..... 877
References..... 877
Revision History..... 878
Please Read: Important Legal Notices..... 879

Introduction

This section contains the following chapters:

- Navigating Content by Design Process
- Supported Operating Systems for Vitis HLS
- Benefits of High-Level Synthesis
- Introduction to Vitis HLS Components
- Tutorials and Examples

Navigating Content by Design Process

AMD Adaptive Computing documentation is organized around a set of standard design processes to help you find relevant content for your current development task. You can access the AMD Versal™ adaptive SoC design processes on the Design Hubs page. You can also use the Design Flow Assistant to better understand the design flows and find content that is specific to your intended design needs. This document covers the following design processes:

- **Hardware, IP, and Platform Development:** Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the AMD Vivado™ timing, resource use, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:
  - Launching the Vitis Unified IDE
  - Running C Simulation
  - Running C Synthesis
  - Running C/RTL Co-Simulation

Supported Operating Systems for Vitis HLS

AMD supports the following operating systems on x86 and x86-64 processor architectures.

- Red Hat Enterprise Linux Workstation/Server 7/CentOS: 7.9
- Red Hat Enterprise Linux Workstation/Server: 8.5, 8.6, 8.7, 8.8
- Red Hat Enterprise Linux Workstation/Server: 9.0, 9.1, 9.2, 9.3
- Ubuntu 20.04.4 LTS, 20.04.5 LTS, 20.04.6 LTS
- Ubuntu 22.04 LTS, 22.04.1 LTS, 22.04.2 LTS, 22.04.3 LTS
- Amazon Linux 2 AL2 LTS



TIP: *Ubuntu 20.04 may require the* `libtinfo.so.5` *as described in AR# 76616.*

Windows, 64-bit for Embedded Software and HLS Development:

- Windows 10: 21H2, 22H2
- Windows 11: 21H2, 22H2
- Windows Server 2022

Obtaining a Vitis HLS License

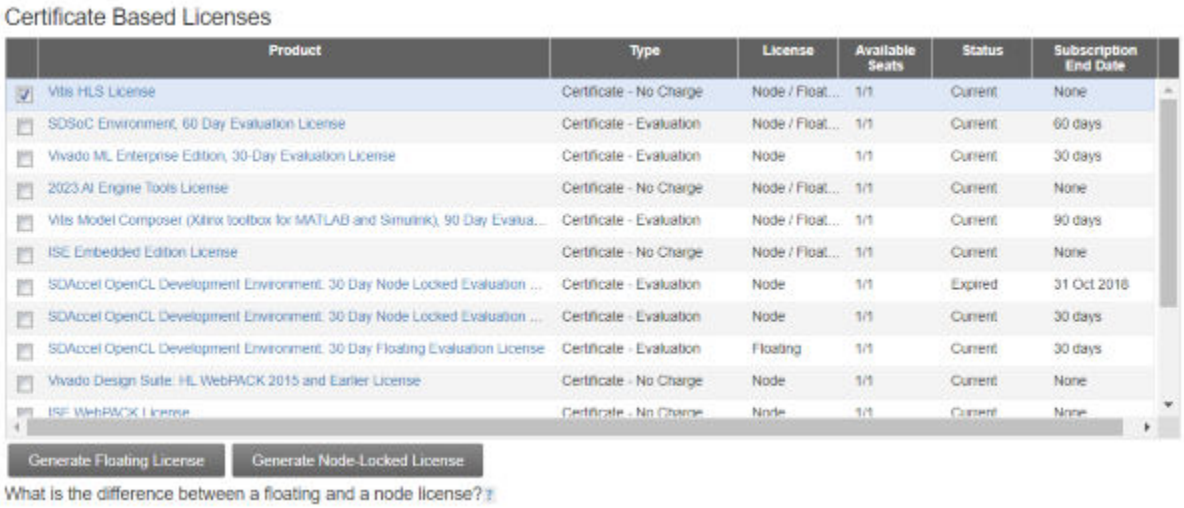
In the Vitis unified IDE, the reporting features of the HLS compiler, such as the Code Analyzer and the Dataflow viewer, require a license to be used. This means you need to obtain a license as part of the installation of the tool. You can obtain a license by using the following steps:

1. Go to https://www.xilinx.com/getlicense
2. Sign in to your account.

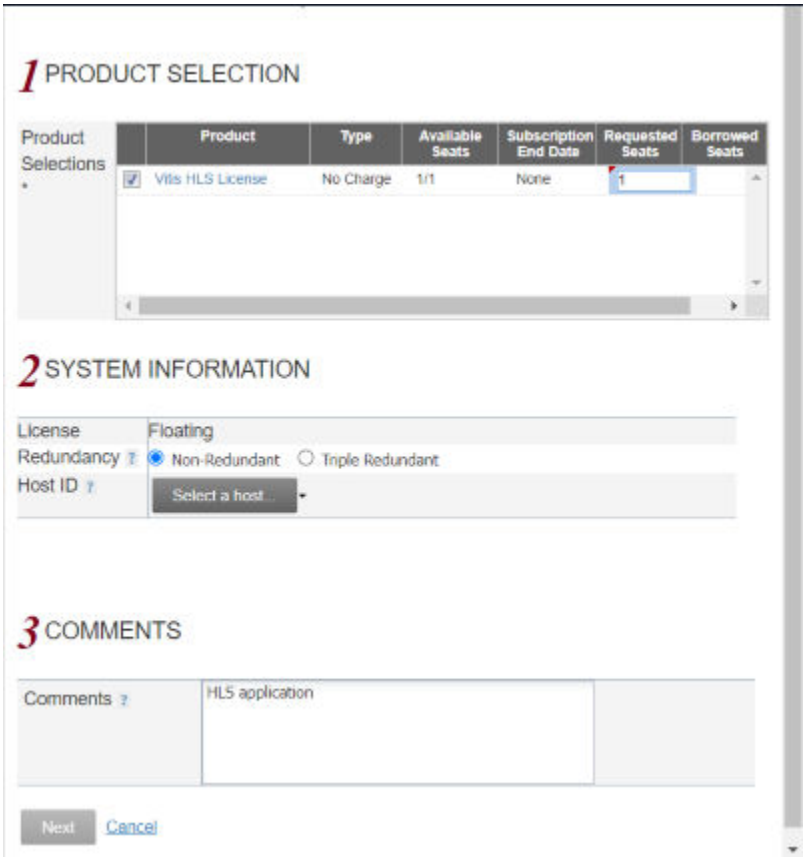
   

   TIP: *You must have an AMD account to download the installation files, and to obtain a license.*

3. This should take you to the Product Licensing - Name and Address Verification page. Enter the required information and click **Next**.
4. This should take you to the Product Licensing page. Select an account if you have multiple accounts.
5. Under Create New Licenses enable the **Vitis HLS License** checkbox and select **Generate Floating License** or **Generate Node-Locked License** as appropriate.



6. The dialog box that opens is slightly different between floating and node-locked licenses. To generate the floating license you must specify the number of Requested Seats. For node-locked licenses this number is set to 1.



7. Under System Information click **Select a host** and either choose an existing host or add a host. A host must be selected to assign the license. To Add a host use the following steps:

   a. **Host Name**: Specify a name for the host.

b. **Select OS type...**: Specify the operating system for your host.

c. **Select host type...**: Depending on the operating selected the choices of type can include:

- Dongle Flex ID
- Disk Serial Number: Can be obtained from Windows ocmmand prompt by typing `vol c:` for example
- Ethernet MAC: Can be obtained from `ipconfig /all` command as the Ethernet adapter Physical Address
- Solaris hostid

8. After specifying the Host click **Next** to move to the Review License Request dialog box.
9. Review the provided information and click **Next** to generate a license file and receive a Congratulations message. You are going to receive an Email with the required license file.

For information on using the license file refer to the Managing Licenses section in *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* (UG973). For general information related to licenses refer to the Licensing Solution Center.

Changed Behavior

The following table specifies differences between this release and prior releases that impact behavior or flow when migrating.

Table 1: **Changed Behavior Summary for Vitis Software Development Platform**

Scope	Details
Vitis Unified IDE	Metadata files for the IDE have been reorganized. A new file, `_ide/.wsdata/version.ini`, has been introduced to label the workspace version. When an older workspace or a workspace without `version.ini` is opened, the IDE will prompt the user to update the workspace. The reorganization targets to help users to add workspace to source control tool easier by keeping files can be regenerated in the `.wsdata` folder.
Vitis Unified IDE	The Python journal file `builder.py`, previously located in the logs directory, has been renamed to `workspace_journal.py` and moved to the `_ide` directory. It can now be accessed from the menu under **Vitis → Workspace Journal**.
Vitis Unified IDE	The IDE now enables users to manage application component source code compilation settings in `UserConfig.cmake`. Previously, the IDE automatically set up the build list.
Vitis Unified IDE	The process name of Vitis IDE is changed from Vitisng-ide to Vitis-ide.
Vitis Unified IDE	The user managed flow is extended from the Explorer view to the Vitis Explorer view. Vitis Explorer can handle both the GUI managed flow and the user managed flow and provide consistent flow window support.

Table 2: **Changed Behavior Summary for Vitis Accelerated Software Development**

Scope	Details
Vitis Base Platforms	The VCK190 Base Platform now initiates designs using the Vivado Customizable Example Design (CED) Vitis Extensible Platform rather than building from scratch. This update aligns the Vitis platform configuration with both CED and Vitis Base Platforms. The same approach will be applied to other base platforms in future releases.

Benefits of High-Level Synthesis

High-Level Synthesis (HLS) is an automated design process that takes an abstract behavioral specification of a digital system and generates a register-transfer level (RTL) structure that realizes the given behavior.

A typical flow using HLS has the following steps:

1. Write the algorithm at a high abstraction level using C/C++ with a target architecture in mind
2. Verify the functionality at the behavioral level
3. Use the HLS tool to generate the RTL for a given clock speed, and design constraints
4. Verify the functionality of the generated RTL
5. Explore different architectures using the same input source code

HLS can enable the path of creating high-quality RTL, rather quickly compared to manually writing error-free RTL.

The designer needs to create the macro-architecture of the algorithm in C/C++ at a high level, meaning that the design intent and how this design interacts with the outside world should be carefully thought through. The HLS tool also requires design constraints like clock period, performance constraints, etc.

Micro-architecture decisions like creating the state machine, datapath, register pipelines, etc. are not needed at a high level. These details can be left to the HLS tool and optimized RTL can be generated by providing design constraints.

Using the defined macro-architecture of the C/C++ algorithm, designers can also vary constraints to generate multiple RTL solutions to explore trade-offs between performance and area. So a single algorithm can lead to multiple implementations, allowing designers to choose an implementation that best meets the needs of the overall application.

Improve Productivity

With HLS, the designer works at a high level of abstraction, meaning fewer lines of code need to be written as input to HLS. Due to less time spent on writing the C++ code and quicker turnaround, user errors may be significantly reduced, thus increasing overall design productivity. The designers can focus on creating efficient designs at a higher level of abstraction rather than worrying about mechanical RTL implementation details.

HLS not only enables high design productivity but also verification productivity. With HLS, the test bench is also generated or created at a high level, meaning the original design intent can be verified very quickly. The designer can explore quick turnarounds of verified algorithms as the flow is still within the C/C++ domain. Once the algorithm is verified in C/C++, the same test bench can be used for generated RTL by the HLS tool. Nevertheless, the generated RTL can be integrated with the existing RTL verification flow for more comprehensive verification coverage.

The design and verification benefits of using HLS are summarized here:

- Developing and validating algorithms at the C-level for the purpose of designing at an abstract level from the hardware implementation details.
- Using C-simulation to validate the design, and iterate more quickly than with traditional RTL design.
- Creating multiple design solutions from the C source code and pragmas to explore the design space, and find an optimal solution.

Enable Re-Use

The designs created for High-level synthesis are generic and unaware of implementation. These sources are not tied to any technology node or any given clock period like a given RTL. With few updates of input constraints and without any source code changes, multiple architectures can be explored. A similar practice with RTL is not pragmatic. The designers create RTL for a given clock period and any change for a derivative product, however small, leads to a new, complex project. Working at a higher level with HLS, designers don't need to worry about the micro-architecture and can rely on the HLS tool to regenerate new RTL automatically.

Introduction to Vitis HLS Components

An HLS component is synthesized from a C or C++ function into RTL code for implementation in the programmable logic (PL) region of a Versal Adaptive SoC , Zynq MPSoC, or AMD FPGA device. The HLS component is tightly integrated with both the Vivado Design Suite for synthesis, place, and route, and the Vitis core development kit for heterogeneous system-level design and application acceleration.

The HLS component can be used to develop and export:

- Vivado IP to be integrated into hardware designs using the Vivado Design Suite, and used with provided software drivers for application development in embedded systems
- Vitis kernels for use in the Vitis application acceleration development flow for use with AI Engine graph applications in heterogeneous compute systems, or for use in Data Center acceleration



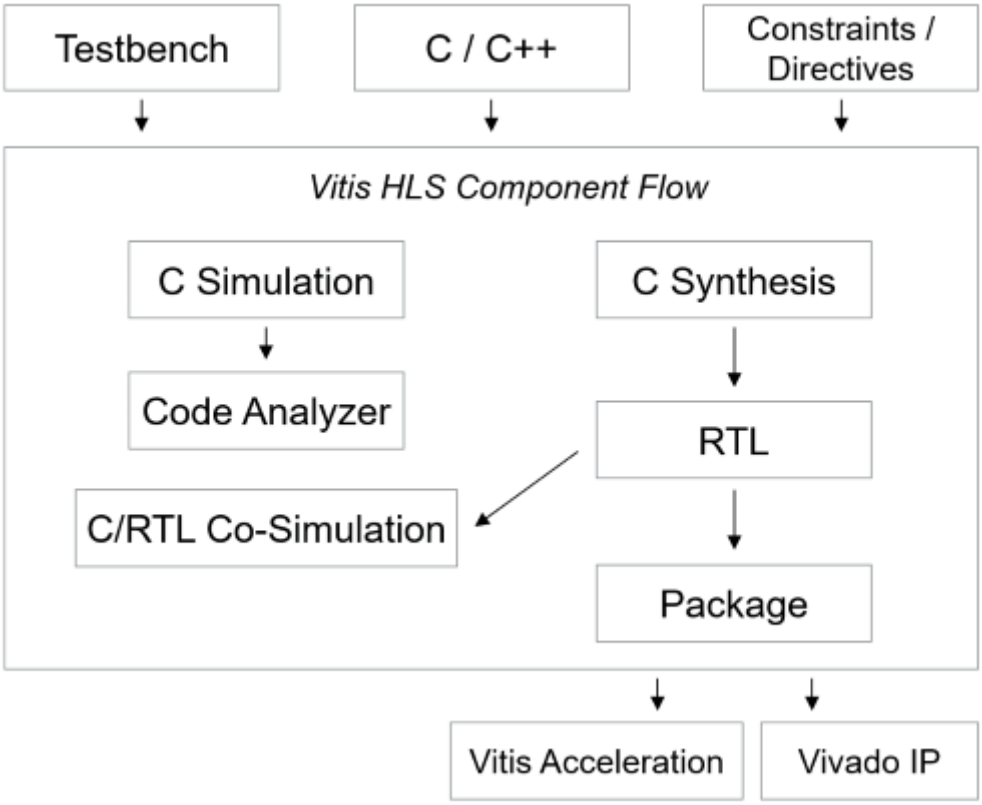
TIP: *The Vitis kernel (`.xo`) has specific requirements and limitations as described in Interfaces for Vitis Kernel Flow; while Vivado IP have few restrictions and offer greater design flexibility as described in Interfaces for Vivado IP Flow.*

The HLS component tool flow provides tools to simulate, analyze, implement, and optimize the C/C++ code in programmable logic and to achieve low latency and high throughput. The inference of required pragmas to produce the right interface for your function arguments and to pipeline loops and functions within your code is the foundation of the HLS component flow whether accomplished from the command-line, Makefile, or the Vitis unified IDE.

Here are the steps for the development of the HLS component from a C++ function:

1. Architect the algorithm based on the Design Principles.
2. (C-Simulation) Verify the functionality of the C/C++ code with the C/C++ test bench.
3. (Code Analyzer) Analyze the performance, parallelism, and legality of the C/C++ code.
4. (C-Synthesis) Generate the RTL using the `v++` compiler.
5. (C/RTL Co-Simulation) Verify the RTL code generated using the C/C++ test bench.
6. (Package) Review the HLS synthesis reports and implementation timing reports.
7. Re-run previous steps until performance goals are met.

Figure 1: **HLS Component Development Flow**



The tool implements the HLS component based on the target flow, default tool configuration, design constraints, and any optimization pragmas or directives you specify. You can use optimization directives to modify and control the implementation of the internal logic and I/O ports, overriding the default behaviors of the tool.

Here are some key concepts related to coding and synthesizing the C++ functions in your HLS component with details covered in forthcoming sections:

- **Hardware Interfaces:** The arguments of the top-level function in an HLS component are synthesized into interfaces and ports that group multiple signals to define the communication protocol between the hardware design and components external to the design. The `v++` compiler defines interfaces automatically, using industry standards to specify the protocol used. The default interface protocols differ based on whether the HLS component is targeted for packaging as a Vivado IP or a Vitis kernel. The default assignments of the interfaces can be overridden by using the INTERFACE pragma or directive.
- **Controlling the Execution of the HLS Component:** The execution mode of an HLS component is specified by the block-level control protocol. The HLS component can have control signals to start/stop the execution or it can be only driven when the data is available. As a designer, you do need to be aware of how your HLS design can be executed, as described in Execution Modes of HLS Designs.

- **Task-Level Parallelism:**

  To achieve high performance on the generated hardware, the synthesizer tool must infer parallelism from sequential code and exploit it to achieve greater performance. The Design Principles section introduces the three main paradigms that need to be understood for writing good software for FPGA platforms. The HLS component offers two types of task-level parallelism (TLP): either by specifying the DATAFLOW pragma or explicitly creating parallelism using `hls::task` object as described in Abstract Parallel Programming Model for HLS.

- **Memory Architecture:**

  - The memory architecture is fixed in the CPU but the developer can create their own to optimize the memory accesses for running applications on programmable logic.
  - In a C++ program, arrays are fundamental data structures used to save or manipulate data. In hardware, these arrays are implemented as memory or registers after synthesis. The memory can be implemented as local storage or global memory which is often DDR memory or HBM memory. Access to global memory has higher latency costs and can take many cycles while access to local memory is often quick and only takes one or more cycles.
  - Often memory is allocated/deallocated dynamically in a C++ program but this can not be synthesized in hardware. So the designer needs to be aware of the exact amount of memory required for the algorithm.
  - Memory accesses should be optimized to reduce the overhead of global memory accesses. The tool may perform memory burst access or coalescing (for example, combining multiple accesses into one, by expanding the data width) as directed by pragmas or when a particular access pattern is detected. Burst access hides the memory access latency and improves the memory bandwidth.

- **Micro Level Optimization:**

  - In C++ programs, there is a frequent need to implement repetitive algorithms that process blocks of data — for example, signal or image processing. Typically, the C/C++ source code tends to include several loops or several nested loops. The `v++` compiler can unroll, or pipeline a loop or nested loops by inserting pragmas at appropriate levels in the source code. For more information, refer to the Loops Primer.
  - Once the algorithm is architected based on the design principles, inferring parallelism, you still need the right combination of micro-level HLS pragmas like PIPELINE, UNROLL, ARRAY_PARTITION, etc. The PERFORMANCE pragma or directive lets you define a single top-level performance goal for a given body of loop or nested loops. The tool automatically infers the necessary lower-level pragmas to meet the goal. With the PERFORMANCE pragma, fewer pragmas are needed to achieve good QoR and is an intuitive way to drive the tool.

Refactoring C++ Source Code for HLS

The following is a simple program that includes a `compute()` function written in C++ for execution on the CPU. The execution of this program is sequential on the CPU. This example needs to be refactored to achieve optimal performance when running on programmable logic.

```
#include <vector>
#include <iostream>
#include <ap_int.h>
#include "hls_vector.h"

#define  totalNumWords 512
unsigned char data_t;

int main(int, char**) {
    // initialize input vector arrays on CPU
    for (int i = 0; i < totalNumWords; i++) {
      in[i] = i;
    }
    compute(data_t in[totalNumWords], data_t Out[totalNumWords]);
    check_results();
}

void compute (data_t in[totalNumWords ], data_t Out[totalNumWords ]) {
  data_t tmp1[totalNumWords], tmp2[totalNumWords];
  A: for (int i = 0; i < totalNumWords ; ++i) {
    tmp1[i] = in[i] * 3;
    tmp2[i] = in[i] * 3;
  }
  B: for (int i = 0; i < totalNumWords ; ++i) {
    tmp1[i] = tmp1[i] + 25;
  }
  C: for (int i = 0; i < totalNumWords ; ++i) {
    tmp2[i] = tmp2[i] * 2;
 }
  D: for (int i = 0; i <  totalNumWords ; ++i) {
     out[i] = tmp1[i] + tmp2[i] * 2;
   }
}
```

This program runs sequentially on an FPGA, producing correct results without any performance gain. To achieve higher performance on an FPGA, the program must be refactored to enable parallelism within the hardware. Examples of parallelism can include:

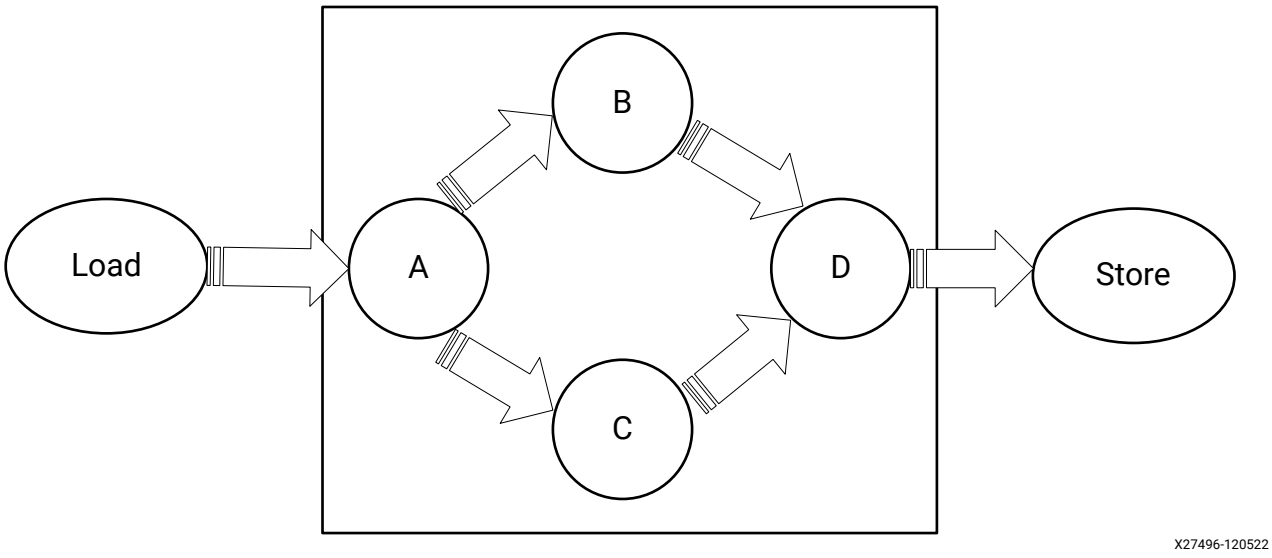
- The compute function can start before all the data is transferred to it
- Multiple compute functions can run in an overlapping fashion, for example a "for" loop can start the next iteration before the previous iteration has completed
- The operations within a "for" loop can run concurrently on multiple words and doesn't need to be executed on a per-word basis

Re-Architecting the Hardware Module

From the prior example it is the `compute()` function that needs to be re-architected for FPGA-based acceleration.

The `compute()` function Loop A multiplies an input value with 3 and creates two separate paths, B and C. Loop B and C perform operations and feed the data to D. This is a simple representation of a realistic case where you have several tasks to be performed one after another and these tasks are connected to each other as a network like the one shown below.

Figure 2: **Module Architecture**



The key takeaways for re-architecting the hardware module are:

- Task-level parallelism is implemented at the function level. To implement task-level parallelism loops are pushed into separate functions. The original `compute()` function is split into multiple sub-functions. As a rule of thumb, sequential functions can be made to execute concurrently, and sequential loops can be pipelined.
- Instruction-level parallelism is implemented by reading 16 32-bit words from memory (or 512-bits of data). Computations can be performed on all these words in parallel. The `hls::vector` class is a C++ template class for executing vector operations on multiple samples concurrently.
- The `compute()` function needs to be re-architected into load-compute-store sub-functions, as shown in the example below. The load and store functions encapsulate the data accesses and isolate the computations performed by the various compute functions.
- Additionally, there are compiler directives starting with `#pragma` that can transform the sequential code into parallel execution.

TIP: *This is the using_fifos example found in the Vitis-HLS Introductory Examples on GitHub.*

```
#include "diamond.h"
#define NUM_WORDS 16
extern "C" {

void diamond(vecOf16Words* vecIn, vecOf16Words* vecOut, int size)
{
  hls::stream<vecOf16Words> c0, c1, c2, c3, c4, c5;
  assert(size % 16 == 0);

  #pragma HLS dataflow
  load(vecIn, c0, size);
  compute_A(c0, c1, c2, size);
  compute_B(c1, c3, size);
  compute_C(c2, c4, size);
  compute_D(c3, c4,c5, size);
  store(c5, vecOut, size);
}
}

void load(vecOf16Words *in, hls::stream<vecOf16Words >& out, int size)
{
Loop0:
  for (int i = 0; i < size; i++)
  {
    #pragma HLS PERFORMANCE target_ti=32
    #pragma HLS LOOP_TRIPCOUNT max=32
    out.write(in[i]);
  }
}

void compute_A(hls::stream<vecOf16Words >& in, hls::stream<vecOf16Words >&
out1, hls::stream<vecOf16Words >& out2, int size)
{
Loop0:
  for (int i = 0; i < size; i++)
  {
    #pragma HLS PERFORMANCE target_ti=32
    #pragma HLS LOOP_TRIPCOUNT max=32
    vecOf16Words t = in.read();
    out1.write(t * 3);
    out2.write(t * 3);
  }
}
void compute_B(hls::stream<vecOf16Words >& in, hls::stream<vecOf16Words >&
out, int size)
{
Loop0:
  for (int i = 0; i < size; i++)
  {
    #pragma HLS PERFORMANCE target_ti=32
    #pragma HLS LOOP_TRIPCOUNT max=32
    out.write(in.read() + 25);
  }
}


void compute_C(hls::stream<vecOf16Words >& in, hls::stream<vecOf16Words >&
out, int size)
{
Loop0:
```

```
  for (data_t i = 0; i < size; i++)
  {
    #pragma HLS PERFORMANCE target_ti=32
    #pragma HLS LOOP_TRIPCOUNT max=32
    out.write(in.read() * 2);
  }
}
void compute_D(hls::stream<vecOf16Words >& in1, hls::stream<vecOf16Words >&
in2, hls::stream<vecOf16Words >& out, int size)
{
Loop0:
  for (data_t i = 0; i < size; i++)
  {
    #pragma HLS PERFORMANCE target_ti=32
    #pragma HLS LOOP_TRIPCOUNT max=32
    out.write(in1.read() + in2.read());
  }
}

void store(hls::stream<vecOf16Words >& in, vecOf16Words *out, int size)
{
Loop0:
  for (int i = 0; i < size; i++)
  {
    #pragma HLS PERFORMANCE target_ti=32
    #pragma HLS LOOP_TRIPCOUNT max=32
    out[i] = in.read();
  }
}
```

Tutorials and Examples

To get started with Vitis HLS, you can find tutorials and example applications at the following locations:

- **Vitis HLS Introductory Examples (https://github.com/Xilinx/Vitis-HLS-Introductory-Examples):** Small code examples to demonstrate good design practices, coding guidelines, design patterns for common applications, and most importantly, optimization techniques to maximize application performance. All examples include a `README` file, an `hls_config.cfg` file, and a `run_hls.tcl` script to help you use the example code.
- **Vitis Accel Examples Repository (https://github.com/Xilinx/Vitis_Accel_Examples):** Contains examples to showcase various features of the Vitis tools and platforms. This repository illustrates specific scenarios related to host code and kernel programming for the Vitis application acceleration development flow, by providing small working examples. The kernel code in these examples can be directly compiled as HLS components.
- **Vitis Application Acceleration Development Flow Tutorials (https://github.com/Xilinx/Vitis-Tutorials):** Provides tutorials that can be worked through to teach specific concepts regarding the tool flow and application development, including the use of the Vitis unified IDE to create an HLS component for use in the Vitis bottom up design flow.

HLS Programmers Guide

Introduction

This Programmers Guide is intended to provide real world design techniques, and details of hardware design which helps you get the most out of the AMD Vitis™ HLS tool. This guide provides details on programming techniques you should apply when writing C/C++ code for high-level synthesis into RTL code, and a checklist of best practices to follow when creating IP that uses the AXI4 interfaces. Finally, it details various optimization techniques that can improve the performance of your code, improving both the fit and function of the resulting hardware design.

This section contains the following chapters:

- Design Principles
- Abstract Parallel Programming Model for HLS
- Loops Primer
- Arrays Primer
- Functions Primer
- Data Types
- Unsupported C/C++ Constructs
- Interfaces of the HLS Design
- Best Practices for Designing with M_AXI Interfaces
- Optimizing Techniques and Troubleshooting Tips

Chapter 1

Design Principles

Introduction

You might be working with the HLS tool to take advantage of productivity gains from writing C/C++ code to generate RTL for hardware; or you might be looking to accelerate portions of a C/C++ algorithm to run on custom hardware implemented in programmable logic. This chapter is intended to help you understand the process of synthesizing hardware from a software algorithm written in C/C++. This document introduces the fundamental concepts used to design and create good synthesizable software in such a way that it can be successfully converted to hardware using high-level synthesis (HLS) tools. The discussion in this document is tool-agnostic and the concepts introduced are common to most HLS tools. For experienced designers, reviewing this material can provide a useful reinforcement of the importance of these concepts; help you understand how to approach HLS, and in particular how to structure HLS code to achieve high-performance designs.

Throughput and Performance

C/C++ functions implemented as custom hardware in programmable logic can run at a significantly faster rate than what is achievable on traditional CPU/GPU architectures, and achieve higher processing rates and/or performance. First establish what these terms mean in the context of hardware acceleration. *Throughput* is defined as the number of specific actions executed per unit of time or results produced per unit of time. This is measured in units of whatever is being produced (cars, motorcycles, I/O samples, memory words, iterations) per unit of time. For example, the term "memory bandwidth" is sometimes used to specify the throughput of the memory systems. Similarly, *performance* is defined with higher throughput but higher throughput with low power consumption. Lower power consumption is as important as higher throughput in today's world.

Architecture Matters

In order to better understand how custom hardware can accelerate portions of your program, you first need to understand how your program runs on a traditional computer. The von Neumann architecture is the basis of almost all computing done today even though it was designed more than 7 decades ago. This architecture was deemed optimal for a large class of applications and has tended to be very flexible and programmable. However, as application demands started to stress the system, CPUs began supporting the execution of multiple processes. Multithreading and/or Multiprocessing can include multiple *system processes* (For example: executing two or more programs at the same time), or it can consist of one process

that has multiple *threads* within it. Multi-threaded programming using a shared memory system became very popular as it allowed the software developer to design applications with parallelism in mind but with a fixed CPU architecture. But when multi-threading and the ever-increasing CPU speeds could no longer handle the data processing rates, multiple CPU cores and hyperthreading were used to improve throughput as shown in the figure on the right.

This general purpose flexibility comes at a cost in terms of power and peak throughput. In today's world of ubiquitous smart phones, gaming, and online video conferencing, the nature of the data being processed has changed. To achieve higher throughput, you must move the workload closer to memory, and/or into specialized functional units. So the new challenge is to design a new programmable architecture in such a way that you can maintain enough programmability while achieving higher performance and lower power costs.

A field-programmable gate array (FPGA) provides for this kind of programmability and offers enough memory bandwidth to make this a high-performance and lower power cost solution. Unlike a CPU that executes a program, an FPGA can be configured into a custom hardware circuit that responds to inputs in the same way that a dedicated piece of hardware would behave. Reconfigurable devices such as FPGAs contain computing elements of extremely flexible granularities, ranging from elementary logic gates to complete arithmetic-logic units such as digital signal processing (DSP) blocks. At higher granularities, user-specified composable units of logic called kernels, can be strategically placed on the FPGA to perform various roles. This characteristic of reconfigurable FPGA devices allows the creation of custom macro-architectures and gives FPGAs a big advantage over traditional CPUs/GPUs in using application-specific parallelism. Computation can be spatially mapped to the device, enabling much higher operational throughput than processor-centric platforms. Today's latest FPGA devices can also contain processor cores (Arm-based) and other hardened IP blocks that can be used without having to program them into the programmable fabric.

Three Paradigms for Programmable Logic

While FPGAs can be programmed using lower-level Hardware Description Languages (HDLs) such as Verilog or VHDL, there are now several High-Level Synthesis (HLS) tools that can take an algorithmic description written in a higher-level language like C/C++ and convert it into lower-level hardware description languages such as Verilog or VHDL. This can be processed by downstream tools to program the FPGA device.

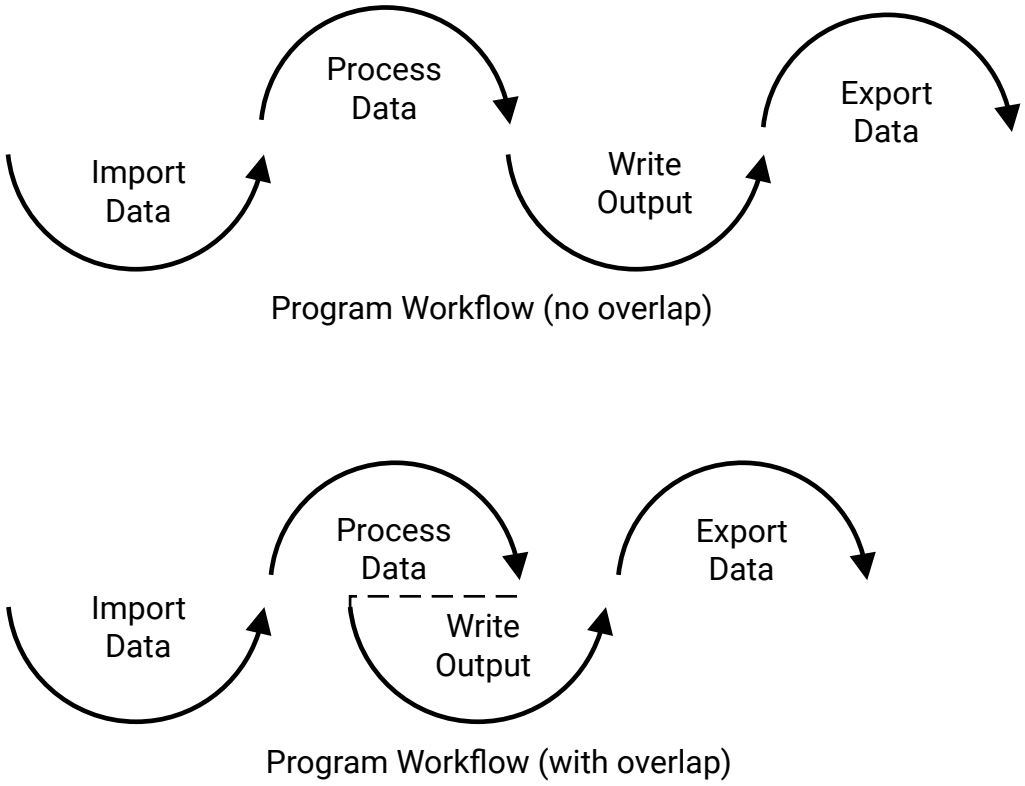
The main benefit of this type of flow is that you can retain the advantages of the programming language like C/C++ to write efficient code that can be translated into hardware. Additionally, writing good code is the software designer's forte and is easier than learning a new hardware description language. However, achieving acceptable quality of results (QoR), requires additional work such as rewriting the software to help the HLS tool achieve the desired performance goals. The next few sections discuss how you can first identify some macro-level architectural optimizations to structure your program and focus on some fine-grained micro-level architectural optimizations to boost your performance goals.

Producer-Consumer Paradigm

Consider how software designers write a multithreaded program - there is usually a master thread that performs some initialization steps and forks off a number of child threads to do some parallel computation and when all the parallel computation is done, the main thread collates the results and writes to the output. The programmer has to figure out what parts can be forked off for parallel computation and what parts need to be executed sequentially. This fork/join type of parallelism applies as well to FPGAs as it does to CPUs, but a key pattern for throughput on FPGAs is the producer-consumer paradigm. You need to apply the producer-consumer paradigm to a sequential program and convert it to extract functionality that can be executed in parallel to improve performance.

You can better understand this decomposition process with the help of a simple problem statement. Assume that you have a datasheet from which you can import items into a list. You can then process each item in the list. The processing of each item takes around 2 seconds. After processing, you write the result in another datasheet and this action takes an additional 1 second per item. So if you have a total of 100 items in the input Excel sheet then it takes a total of 300 seconds to generate output. The goal is to decompose this problem in such a way that you can identify tasks that can potentially execute in parallel and therefore increase the throughput of the system.

Figure 3: **Program Workflows**



The first step is to understand the program workflow and identify the independent tasks or functions. The four-step workflow is something like the Program Workflow (no overlap) shown in the above diagram. In the example, the "Write Output" (step 3) task is independent of the "Process Data" (step 2) processing task. Although step 3 depends on the output of step 2, as soon as any of the items are processed in Step 2, you can immediately write that item to the output file. You don't have to wait for all the data to be processed before starting to write data to the output file. This type of interleaving/overlapping the execution of tasks is a very common principle. This is illustrated in the above diagram (For example: the program workflow with overlap). As can be seen, the work gets done faster than with no overlap. You can now recognize that step 2 is the producer, and step 3 is the consumer. The producer-consumer pattern has a limited impact on performance on a CPU. You can interleave the execution of the steps of each thread but this requires careful analysis to exploit the underlying multi-threading and L1 cache architecture and therefore a time consuming activity. On FPGAs however, due to the custom architecture, the producer and consumer threads can be executed simultaneously with little or no overhead leading to a considerable improvement in throughput.

The simplest case to first consider is the single producer and single consumer, who communicate via a finite-size buffer. If the buffer is full, the producer has a choice of either blocking/stalling or discarding the data. Once the consumer removes an item from the buffer, it notifies the producer, who starts to fill the buffer again. In the same way, the consumer can stall if it finds the buffer empty. Once the producer puts data into the buffer, it wakes up the sleeping consumer. The solution can be achieved by means of inter-process communication, typically using monitors or semaphores. An inadequate solution could result in a deadlock where both processes are stalled waiting to be woken up. However, in the case of a single producer and consumer, the communication pattern strongly maps to a first-in-first-out (FIFO) or a Ping-Pong buffer (PIPO) implementation. This type of channel provides highly efficient data communication without relying on semaphores, mutexes, or monitors for data transfer. The use of such locking primitives can be expensive in terms of performance and difficult to use and debug. PIPOs and FIFOs are popular choices because they avoid the need for end-to-end atomic synchronization.

This type of macro-level architectural optimization, where the communication is encapsulated by a buffer, frees the programmer from worrying about memory models and other non-deterministic behavior (like race conditions etc). The type of network that is achieved in this type of design is purely a "dataflow network" that accepts a stream of data on the input side and essentially does some processing on this stream of data and sends it out as a stream of data. The complexities of a parallel program are abstracted away. The "Import Data" (Step 1) and "Export Data" (Step 4) also have a role to play in maximizing the available parallelism. In order to allow computation to successfully overlap with I/O, it is important to encapsulate reading from inputs as the first step and writing to outputs as the last step. This allows for a maximal overlap of I/O with computation. Reading or writing to input/output ports in the middle of the computation step limits the available concurrency in the design. It is another thing to keep in mind while designing the workflow of your design.

Finally, the performance of such a "dataflow network" relies on the designer being able to continually feed data to the network such that data keeps streaming through the system. Having interruptions in the dataflow can result in lower performance. A good analogy for this is video streaming applications like online gaming where the real-time high definition (HD) video is constantly streamed through the system and the frame processing rate is constantly monitored to ensure that it meets the expected quality of results. Any slowdown in the frame processing rate can be immediately seen by the gamers on their screens. Now imagine being able to support consistent frame rates for a whole bunch of gamers all the while consuming much less power than with traditional CPU or GPU architectures - this is the sweet spot for hardware acceleration. Keeping the data flowing between the producer and consumer is of paramount importance. Next, you can delve a little deeper into this *streaming* paradigm that was introduced in this section.

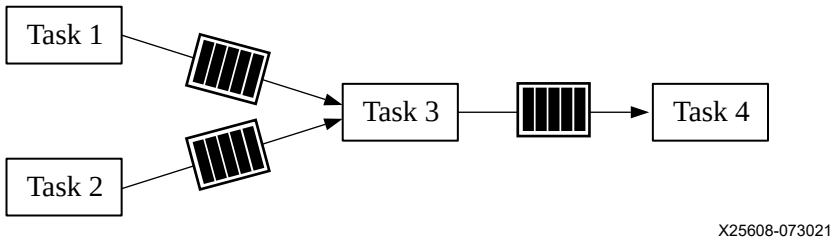
Streaming Data Paradigm

A *stream* is an important abstraction: it represents an unbounded, continuously updating data set, where unbounded means "of unknown or of unlimited size." A stream can be a sequence of data (scalars or buffers) flowing unidirectionally between a source (producer) process and a destination (consumer) process. The streaming paradigm forces you to think in terms of data access patterns (or sequences). In software, random memory accesses to data are virtually free (ignoring the caching costs), but in hardware, it is really advantageous to make sequential accesses, which can be converted into streams. Decomposing your algorithm into producer-consumer relationships that communicate by streaming data through the network has several advantages. It lets the programmer define the algorithm in a sequential manner and the parallelism is extracted through other means (such as by the compiler). Complexities like synchronization between the tasks etc are abstracted away. It allows the producer and the consumer tasks to process data simultaneously, which is key for achieving higher throughput. Another benefit is cleaner and simpler code.

As was mentioned before, in the case of the producer and consumer paradigm, the data transfer pattern strongly maps to a FIFO or a PIPO buffer implementation. A FIFO buffer is simply a queue with a predetermined size/depth where the first element that gets inserted into the queue also becomes the first element that can be popped from the queue. The main advantage of using a FIFO buffer is that the consumer process can start accessing the data inside the FIFO buffer as soon as the producer inserts the data into the buffer. The only issue with using FIFO buffers is that due to varying rates of production/consumption between the producers and consumers, it is possible for improperly sized FIFO buffers to cause a deadlock. This typically happens in a design that has several producers and consumers. A Ping Pong Buffer is a double buffer that is used to speed up a process that can overlap the I/O operation with the data processing operation. One buffer is used to hold a block of data so that a consumer process sees a complete (but old) version of the data, while in the other buffer a producer process is creating a new (partial) version of data. When the new block of data is complete and valid, the consumer and the producer processes alternates access to the two buffers. As a result, the usage of a ping-pong buffer increases the overall throughput of a device and helps to prevent eventual bottlenecks. The key advantage of PIPOs is that the tool automatically matches the rate of production vs the rate of consumption and creates a channel of communication that is both high performance and is deadlock free. It is important to note here that regardless of whether FIFOs/PIPOs are used, the key characteristic is the same: the producer sends or streams a block of data to the consumer. A block of data can be a single value or a group of N values. The bigger the block size, the more memory resources are needed.

The following is a simple sum application to illustrate the classic streaming/dataflow network. In this case, the goal of the application is to pair-wise add a stream of random numbers then print them. The first two tasks (Task 1 and 2) provide a stream of random numbers to add. These are sent over a FIFO channel to the sum task (Task 3) which reads the values from the FIFO channels. The sum task then sends the output to the print task (Task 4) to publish the result. The FIFO channels provide asynchronous buffering between these independent threads of execution.

Figure 4: **Streaming/Dataflow Network**



The streams that connect each “task” are usually implemented as FIFO queues. The FIFO abstracts away the parallel behavior from the programmer, leaving them to reason about a “snapshot” of time when the task is active (scheduled). FIFOs make parallelization easier to implement. This largely results from the reduced variable space that programmers must contend with when implementing parallelization frameworks or fault-tolerant solutions. The FIFO between two independent kernels (see example above) exhibits classic queueing behavior. With purely streaming systems, these can be modeled using queueing or network flow models. Another big advantage of this dataflow type network and streaming optimization is that it can be applied at different levels of granularity. A programmer can design such a network inside each task and for a system of tasks or kernel. In fact, you can have a streaming network that instantiates and connects multiple streaming networks or tasks, hierarchically. Another optimization that allows for finer-grained parallelism is *pipelining*.

Pipelining Paradigm

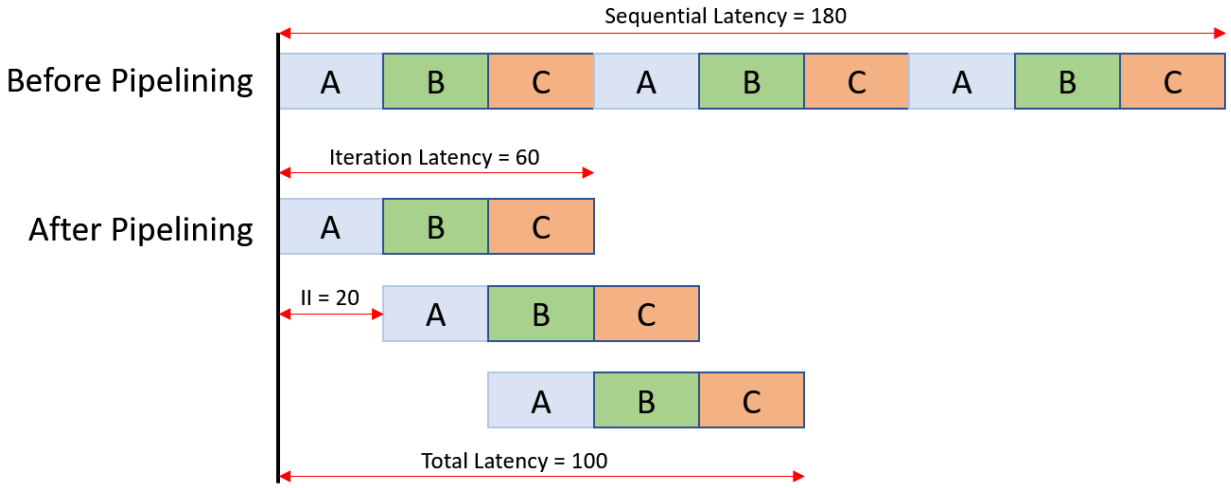
Pipelining is a commonly used concept that you can encounter in everyday life. A good example is the production line of a car factory, where each specific task such as installing the engine, installing the doors, and installing the wheels, is often done by a separate and unique workstation. The stations carry out their tasks in parallel, each on a different car. Once a car has had one task performed, it moves to the next station. Variations in the time needed to complete the tasks can be accommodated by *buffering* (holding one or more cars in a space between the stations) and/or by *stalling* (temporarily halting the upstream stations) until the next station becomes available.

Suppose that assembling one car requires three tasks A, B, and C that takes 20, 10, and 30 minutes, respectively. Then, if all three tasks were performed by a single station, the factory would output one car every 60 minutes. By using a pipeline of three stations, the factory would output the first car in 60 minutes, and then a new one every 30 minutes. As this example shows, pipelining does not decrease the *latency*, that is, the total time for one item to go through the whole system. It does however increase the system's throughput, that is, the rate at which new items are processed after the first one.

The throughput of a pipeline cannot be better than that of its slowest element, the programmer should try to divide the work and resources among the stages so that they all take the same time to complete their tasks. In the car assembly example above, if the three tasks A, B, and C took 20 minutes each, instead of 20, 10, and 30 minutes, the latency would still be 60 minutes, but a new car would then be finished every 20 minutes, instead of 30. The diagram below shows a hypothetical manufacturing line tasked with the production of three cars. Assuming each of the tasks A, B, and C takes 20 minutes, a sequential production line would take 180 minutes to produce three cars. A pipelined production line would take only 100 minutes to produce three cars.

The time taken to produce the first car is 60 minutes and is called the *iteration latency* of the pipeline. After the first car is produced, the next two cars only take 20 minutes each and this is known as the *initiation interval* (II) of the pipeline. The overall time taken to produce the three cars is 100 minutes and is referred to as the *total latency* of the pipeline, for example, total latency = iteration latency + II * (number of items - 1). Therefore, improving II improves total latency, but not the iteration latency. From the programmer's point of view, the pipelining paradigm can be applied to functions and loops in the design. After an initial setup cost, the ideal throughput goal can be to achieve an II of 1 - for example, after the initial setup delay, the output is available at every cycle of the pipeline. In the example above, after an initial setup delay of 60 minutes, a car is then available every 20 minutes.

Figure 5: **Pipelining**



Pipelining is a classical micro-level architectural optimization that can be applied to multiple levels of abstraction. Task-level pipelining with the producer-consumer paradigm was covered earlier. This same concept applies to the instruction-level. This is in fact key to keeping the producer-consumer pipelines (and streams) filled and busy. The producer-consumer pipeline can only be efficient if each task produces/consumes data at a high rate, and hence the need for the instruction-level pipelining (ILP).

Due to the way pipelining uses the same resources to execute the same function over time, it is considered a static optimization because it requires complete knowledge about the latency of each task. Due to this, the low level instruction pipelining technique cannot be applied to dataflow type networks where the latency of the tasks can be unknown as it is a function of the input data. The next section details how to leverage the three basic paradigms that are introduced to model different types of task parallelism.

Combining the Three Paradigms

Functions and loops are the main focus of most optimizations in the user's program. Today's optimization tools typically operate at the function/procedure level. Each function can be converted into a specific hardware component. Each such hardware component is like a class definition and many objects (or instances) of this component can be created and instantiated in the eventual hardware design. Each hardware component can in turn be composed of many smaller predefined components that typically implement basic functions such as add, sub, and multiply. Functions call other functions although recursion is not supported. Functions that are small and called less often can be also inlined into their callers, showing how software functions can be inlined. In this case, the resources needed to implement the function are subsumed into the caller function's component which can potentially allow for better sharing of common resources. Constructing your design as a set of communicating functions lends to inferring parallelism when executing these functions.

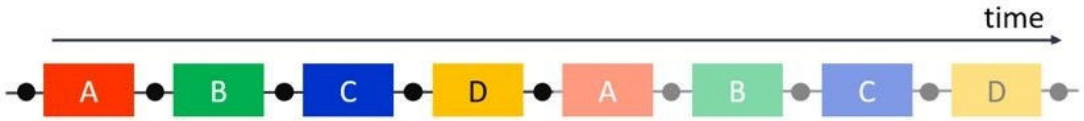
Loops are one of the most important constructs in your program. Because the body of a loop is iterated over a number of times, this property can be easily exploited to achieve better parallelism. There are several transformations (such as pipelining and unrolling) that can be made to loops and loop nests to achieve efficient parallel execution. These transformations enable both memory-system optimizations, mapping to multi-core and SIMD execution resources. Many programs in science and engineering applications are expressed as operations over large data structures. These can be simple element-wise operations on arrays or matrices or more complex loop nests with loop-carried dependencies - for example, data dependencies across the iterations of the loop. Such data dependencies impact the parallelism achievable in the loop. In many such cases, the code must be restructured such that loop iterations can be executed efficiently and in parallel on modern parallel platforms.

The following diagrams illustrate different overlapping executions for a simple example of 4 consecutive tasks (for example, C/C++ functions) A, B, C, and D, where A produces data for B and C, in two different arrays, and D consumes data from two different arrays produced by B and C. Assume that this "diamond" communication pattern is to be run twice (two invocations) and that these two runs are independent.

```
void diamond(data_t vecIn[N], data_t vecOut[N])
{
   data_t c1[N], c2[N], c3[N], c4[N];
   #pragma HLS dataflow
   A(vecIn, c1, c2);
   B(c1, c3);
   C(c2, c4);
   D(c3, c4, vecOut);
}
```

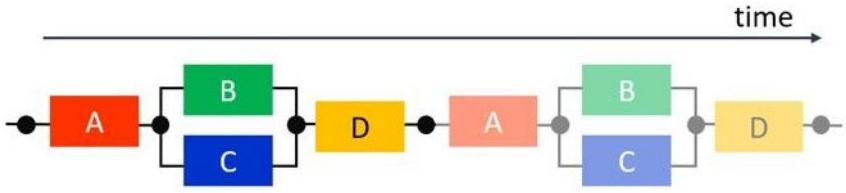
The code example above shows the C/C++ source snippet for how these functions are invoked. The tasks B and C have no mutual data dependencies. A fully-sequential execution corresponds to the following figure where the black circles represent some form of synchronization used to implement the serialization.

Figure 6: **Sequential Execution - Two Runs**



In the diamond example, B and C are fully-independent. They do not communicate nor do they access any shared memory resource, and so if no sharing of computation resource is required, they can be executed in parallel. This leads to the diagram in the following figure, with a form of fork-join parallelism within a run. B and C are executed in parallel after A ends while D waits for both B and C, but the next run is still executed in series.

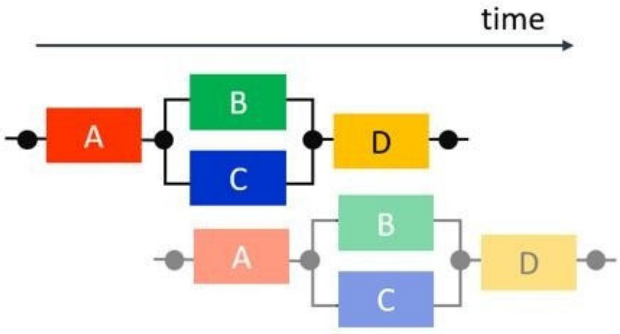
Figure 7: **Task Parallelism within a Run**



Such an execution can be summarized as (A; (B || C); D); (A; (B || C); D) where ";" represents serialization and "||" represents full parallelism. This form of nested fork-join parallelism corresponds to a subclass of dependent tasks, namely series-parallel task graphs. More generally, any DAG (directed acyclic graph) of dependent tasks can be implemented with separate fork-and-join-type synchronization. Also, it is important to know that this is exactly how a multithreaded program would run on a CPU with multiple threads and using shared memory.

On FPGAs, you can explore what other forms of parallelism are available. The previous execution pattern exploited task-level parallelism within an invocation. What about overlapping successive runs? If they are truly independent, but if each function (for example, A, B, C, or D) reuses the same computation hardware as for its previous run, you might still want to execute, for example, the second invocation of A in parallel with the first invocations of B and C. This is a form of task-level pipelining across invocations, leading to a diagram as depicted in the following figure. The throughput is now improved because it is limited by the maximum latency among all tasks, rather than by the sum of their latencies. The latency of each run is unchanged but the overall latency for multiple runs is reduced.

Figure 8: **Task Parallelism with Pipelining**

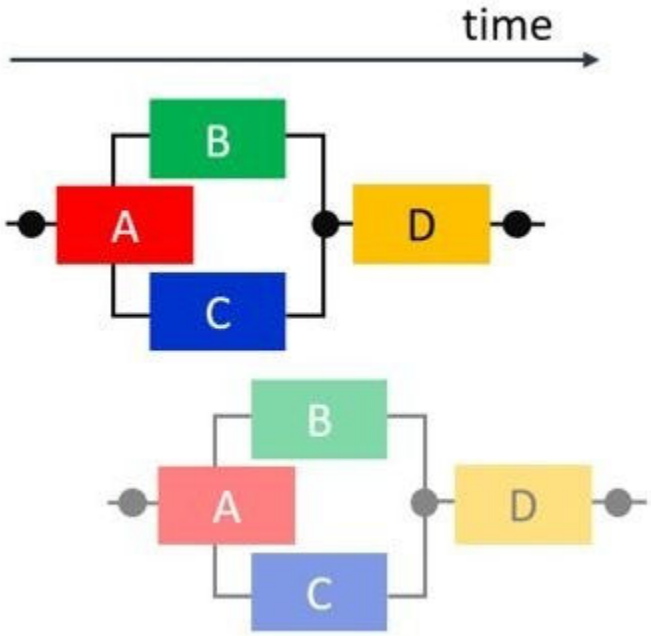


Now, however, when the first run of B reads from the memory where A placed its first result, the second run of A is possibly already writing in the same memory. To avoid overwriting the data before it is consumed, you can rely on a form of memory expansion, namely double buffering or PIPOs to allow for this interleaving. This is represented by the black circles between the tasks.

An efficient technique to improve throughput and reuse computational resources is to pipeline operators, loops, and/or functions. If each task can now overlap with itself, you can achieve simultaneously task parallelism within a run and task pipelining across runs, both of which are examples of macro-level parallelism. Pipelining within the tasks is an example of micro-level parallelism. The overall throughput of a run is further improved because it now depends on the minimum throughput among the tasks, rather than their maximum latency. Finally, depending on how the communicated data are synchronized, only after all are produced (PIPOs) or in a more element-wise manner (FIFOs), some additional overlapping within a run can be expected. For example, in the following figure, both B and C start earlier and are executed in a pipelined fashion with respect to A, while D is assumed to still have to wait for the completion of B and C. This

last type of overlap within a run can be achieved if A communicates to B and C through FIFO streaming accesses (represented as lines without circles). Similarily, D can also be overlapped with B and C, if the channels are FIFOs instead of PIPOs. However, unlike all previous execution patterns, using FIFOs can lead to deadlocks and so these streaming FIFOs need to be sized correctly.

Figure 9: **Task Parallelism and Pipelining within a Run, Pipelining of Runs, and Pipelining within a Task**



In summary, the three paradigms presented in the earlier section show how parallelism can be achieved in your design without needing the complexities of multi-threading and/or parallel programming languages. The producer-consumer paradigm coupled with streaming channels allows for the composition of small to large scale systems easily. As mentioned before, streaming interfaces allow for easy coupling of parallel tasks or even hierarchical dataflow networks. This is in part due to the flexibility in the programming language (C/C++) to support such specifications and the tools to implement them on the heterogeneous computing platform available on today's FPGA devices.

Conclusion - A Prescription for Performance

The design concepts presented in this document have one main central principle - a model of parallel computation that favors encapsulation of state and sequential execution within modular units or tasks to facilitate a simpler programming model for parallel programming. Tasks are then connected together with streams (for synchronization and communication). A stream can be different types of channels such as FIFOs or PIPOs. The state/logic compartmentalization makes it much easier for tools (such as a compiler and a scheduler) to figure out where to run which pieces of an application and when. The second reason why stream-based processing is becoming popular is that it breaks the traditional multi-threading based “fork/join” view on parallel execution. By enabling task-level pipelining and instruction-level pipelining, the runtime can do many more concurrent actions than what is possible today with the fork/join model. This extra parallelism is critical to taking advantage of the hardware available on today's FPGA devices. In the same vein as enabling pipeline parallelism, streaming also enables designers to build parallel applications without having to worry about locks, race conditions, etc. that make parallel programming hard in the first place.

Finally, the following checklist of high-level actions is recommended as a prescription for achieving performance on reconfigurable FPGA platforms:

- Software written for CPUs and software written for FPGAs is fundamentally different. You cannot write code that is portable between CPU and FPGA platforms without sacrificing performance. Therefore, embrace and do not resist the fact that you have to write significantly different software for FPGAs.
- Right from the start of your project, establish a flow that can functionally verify the source code changes that are being made. Testing the software against a reference model or using golden vectors are common practices.
- Focus first on the macro-architecture of your design. Consider modeling your solution using the producer-consumer paradigm.
- Once you have identified the macro-architecture of your design, draw the desired activity timeline where the horizontal axis represents time, and show when you expect each function to execute relative to each other over multiple iterations (or invocations). This gives you a sense of the expected parallelism in the design and can then be used to compare with the final achieved results. Often the HLS GUIs can be used to visualize this achieved parallelism.
- Only start coding or refactoring your program once you have the macro-architecture and the activity timeline well established
- As a general rule, the HLS compiler only infer task-level parallelism from function calls. Therefore, sequential code blocks (such as loops) which need to run concurrently in hardware should be put into dedicated functions.

- Decompose/partition the original algorithm into smaller components that talk to each other via streams. This gives you some ideas of how the data flows in your design.
  - Smaller modular components have the advantage that they can be replicated when needed to improve parallelism.
  - Avoid having communication channels with very wide bit-widths. Decomposing such wide channels into several smaller ones help implementation on FPGA devices.
  - Large functions (written by hand or generated by inlining smaller functions) can have non-trivial control paths that can be hard for tools to process. Smaller functions with simpler control paths aid implementation on FPGA devices.
  - Aim to have a single loop nest (with either fixed loop bounds that can be inferred by HLS tool, or by providing loop trip count information by hand to the HLS tool) within each function. This greatly facilitates the measurement and optimization of throughput. While this might not be applicable for all designs, it is a good approach for a large majority of cases.
- Throughput - Having an overall vision about what rates of processing are required during each phase of your design is important. Knowing this influences how you write your application for FPGAs.
  - Think about the critical path (for example, critical task level paths such as ABD or ACD) in your design and study what part of this critical path is potentially a bottleneck. Look at how individual tasks are pipelined and if different branches of a path are unaligned in terms of throughput by simulating the design. HLS GUI tools and/or the simulation waveform viewer can then be used to visualize such throughput issues.
  - Stream-based communication allows consumers to start processing as soon as producers start producing which allows for overlapped execution (which in turn increases parallelism and throughput).
  - In order to keep the producer and consumer tasks running constantly without any hiccups, optimize the execution of each task to run as fast as possible using techniques such as pipelining and the appropriate sizing of streams.
- Think about the granularity (and overhead) of the streaming channels with respect to synchronization. The usage of PIPO channels allows you to overlap task execution without the fear of deadlock while explicit manual streaming FIFO channels allow you to start the overlapped execution sooner (than PIPOs) but require careful adjustment of FIFO sizes to avoid deadlocks.
- Learn about synthesizable C/C++ coding styles.
- Use the reports generated by the HLS compiler to guide the optimization process.

Keep the above checklist nearby so that you can refer to it from time to time. It summarizes the whole design activity needed to build a design that meets your performance goals.

Another important aspect of your design to consider next is the interface of your accelerated function or kernel. The interface of your kernel to the outside world is an important element of your eventual system design. Your kernel might need to plug into a bigger design, or to communicate with other kernels in a large system of kernels, or to communicate with memory or devices outside of the system. Best Practices for Designing with M_AXI Interfaces provides another checklist of items to consider when designing the external interfaces of your acceleration kernel.

Abstract Parallel Programming Model for HLS

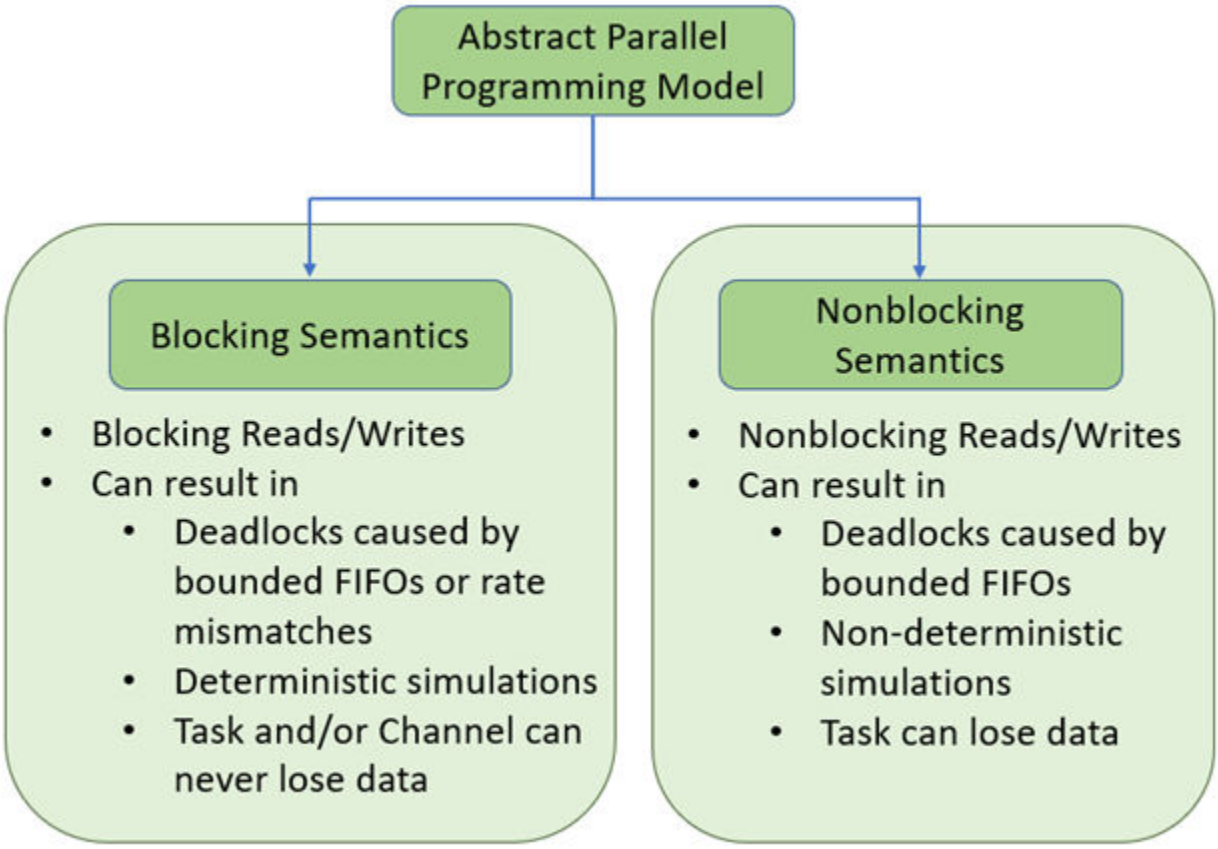
In order to achieve high performance hardware, the HLS tool must infer parallelism from sequential code and exploit it to achieve greater performance. This is not an easy problem to solve. In addition, good software design often uses well-defined rules and practices such as runtime type information (RTTI), recursion, and dynamic memory allocation. Many of these techniques have no direct equivalence in hardware and present challenges for the HLS tool. This generally means that off-the-shelf software cannot be efficiently converted into hardware. At a bare minimum, such software needs to be examined for non-synthesizable constructs and the code needs to be refactored to make it synthesizable. Even if a software program can be automatically converted (or synthesized) into hardware, to assist the tool you need to understand the best practices for writing good software for execution on the FPGA device.

The Design Principles section introduced the three main paradigms that need to be understood for writing good software for FPGA platforms: producer-consumer, streaming data, and pipelining. The underlying parallel programming model that these paradigms work on is as follows:

- The design/program needs to be constructed as a collection of tasks that communicate by sending messages to each other through communication links (aka channels).
- Tasks can be structured as control-driven, waiting for some signal to start execution, or data-driven in which the presence of data on the channel drives the execution of the task.
- A task consists of an executable unit that has some local storage/memory and a collection of input/output (I/O) ports.
- The local memory contains private data, for example, the data to which the task has exclusive access.
- Access to this private memory is called local data access - like data stored in block RAM/ URAM. This type of access is fast. The only way that a task can send copies of its local data to other tasks is through its output ports, and conversely, it can only receive data through its input ports.
- An I/O port is an abstraction; it corresponds to a channel that the task uses for sending or receiving data and it is connected by the caller of the module, or at the system integration level if it is a top port.
- Data sent or received through a channel is called non-local data access. A channel is a data queue that connects one task's output port to another task's input port.

- A channel is assumed to be reliable and has the following behaviors:
  - Data written at the output of the producer are read at the input port of the consumer in the same order for FIFOs. Data can be read/written in random order for PIPOs.
  - No data values are lost.
- Both blocking and non-blocking read and write semantics are supported for channels, as described in HLS Stream Library.

Figure 10: **Blocking/Non-Blocking Semantics**



When blocking semantics are used in the model, a read to an empty channel results in the blocking of the reading process. Similarly, a write to a full channel results in the blocking of the writing process. The resulting process/channel network exhibits deterministic behavior that does not depend on the timing of computation nor on communication delays. These style of models have proven convenient for modeling embedded systems, high-performance computing systems, signal processing systems, stream processing systems, dataflow programming languages, and other computational tasks.

The blocking style of modeling can result in deadlocks due to insufficient sizing of the channel queue (when the channels are FIFOs) and/or due to differing rates of production between producers and consumers. If non-blocking semantics are used in the model, a read to an empty channel results in the reading of uninitialized data or in the re-reading of the last data item. Similarly, a write to a full queue can result in that data being lost. To avoid such loss of data, the design must first check the status of the queue before performing the read/write. But this causes the simulation of such models to be non-deterministic because it relies on decisions made based on the runtime status of the channel. This makes verifying the results of this model much more challenging.

Both blocking and non-blocking semantics are supported by theVitis HLS abstract parallel programming model.

Control and Data Driven Tasks

Using this abstract model as the basis, two types of task-level parallelism (TLP) models can be used to structure and design your application. TLP can be data-driven or control-driven, or can mix control-driven and data-driven tasks in a single design. The main differences between these two models are:

- If your application is purely data-driven, does not require any interaction with external memory and the functions can execute in parallel with no data dependencies, then the data-driven TLP model is the best fit. You can design a purely data-driven model that is always running, requires no control, and reacts only to data. For additional details refer to Data-driven Task-level Parallelism.
- If your application requires some interaction with external memory, and there are data dependencies between the tasks that execute in parallel, then the control-driven TLP model is the best fit. Vitis HLS infers the parallelism between tasks and creates the right channels (as defined by you) such that these functions can be overlapped during execution. The control-driven TLP model is also known as the dataflow optimization in Vitis HLS as described in Control-driven Task-level Parallelism.

The next few sections describe these major modeling options that are available to use. You can use any of these models to write your source code using C++ to optimize the execution of the program on parallel hardware.

Data-driven Task-level Parallelism

Data-driven task-level parallelism uses a task-channel modeling style that requires you to statically instantiate and connect tasks and channels explicitly. The tasks are not controlled by any function call/return semantics but rather are always running waiting for data on their input stream. Tasks in this modeling style typically only have stream type inputs and outputs. However, you can introduce a pointer to arrays and scalar I/O in the TLP design as desscribed in Support of Scalars/Memory Interfaces in DTLP.

Data-driven TLP models are tasks that execute when there is data to be processed. In Vitis HLS C-simulation used to be limited to seeing only the sequential semantics and behavior. With the data-driven model it is possible during simulation to see the concurrent nature of parallel tasks and their interactions via the FIFO channels.

Implementing data-driven TLP in the Vitis HLS tool uses simple classes for modeling tasks (`hls::task`) and channels (`hls::stream`/`hls::stream_of_blocks`).

IMPORTANT! *While Vitis HLS supports `hls::tasks` for a top-level function, you cannot use `hls::stream_of_blocks` for interfaces in top-level functions.*

Consider the simple task-channel example shown below:

```
#include "test.h"

void splitter(hls::stream<int> &in, hls::stream<int> &odds_buf,
hls::stream<int> &evens_buf) {
    int data = in.read();
    if (data % 2 == 0)
        evens_buf.write(data);
    else
        odds_buf.write(data);
}

void odds(hls::stream<int> &in, hls::stream<int> &out) {
    out.write(in.read() + 1);
}

void evens(hls::stream<int> &in, hls::stream<int> &out) {
    out.write(in.read() + 2);
}

void odds_and_evens(hls::stream<int> &in, hls::stream<int> &out1,
hls::stream<int> &out2) {
    hls_thread_local hls::stream<int> s1; // channel connecting t1 and
t2
    hls_thread_local hls::stream<int> s2; // channel connecting t1 and t3

    // t1 infinitely runs function splitter, with input in and outputs s1
and s2
    hls_thread_local hls::task t1(splitter, in, s1, s2);
```

```
    // t2 infinitely runs function odds, with input s1 and output out1
    hls_thread_local hls::task t2(odds, s1, out1);
    // t3 infinitely runs function evens, with input s2 and output out2
    hls_thread_local hls::task t3(evens, s2, out2);
}
```

The special `hls::task` C++ class is:

- A new object declaration in your source code that requires a special qualifier. The `hls_thread_local` qualifier is required to keep the object (and the underlying thread) alive across multiple calls of the instantiating function (`odds_and_evens` in the example).

  The `hls_thread_local` qualifier is only required to ensure that the C simulation of the data-driven TLP model exhibits the same behavior as the RTL simulation. In the RTL, these functions are already in always running mode once started. In order to ensure the same behavior during C Simulation, the `hls_thread_local` qualifier is required to ensure that each task is started only once and keeps the same state even when called multiple times. Without the `hls_thread_local` qualifier, each new invocation of the function would result in a new state.
- Task objects implicitly manage a thread that runs a function infinitely, passing to it a set of arguments that must be either `hls::stream` or `hls::stream_of_blocks`.

  TIP: *No other types of arguments are supported. In particular, even constant values cannot be passed as function arguments. If constants need to be passed to the task's body, define the function as a templated function and pass the constant as a template argument to this templated function.*
- The supplied function (`splitter/odds/evens` in the example above) is called the task body, and it has an implicit infinite loop wrapped around it to ensure that the task keeps running and waiting on input.
- The supplied function can contain pipelined loops but they need to be flushable pipelines (FLP) to prevent deadlock. The tool automatically selects the right pipeline style to use for a given pipelined loop or function.

IMPORTANT! *An `hls::task` should not be treated as a function call - instead a `hls::task` needs to be thought of as a persistent instance statically bound to channels. Due to this, it is your responsibility to ensure that multiple invocations to any function that contains `hls::tasks` be uniquified or these calls use the same `hls::tasks` and channels.*

Channels are modeled by the special templatized `hls::stream` (or `hls::stream_of_blocks`) C++ class. Such channels have the following attributes:

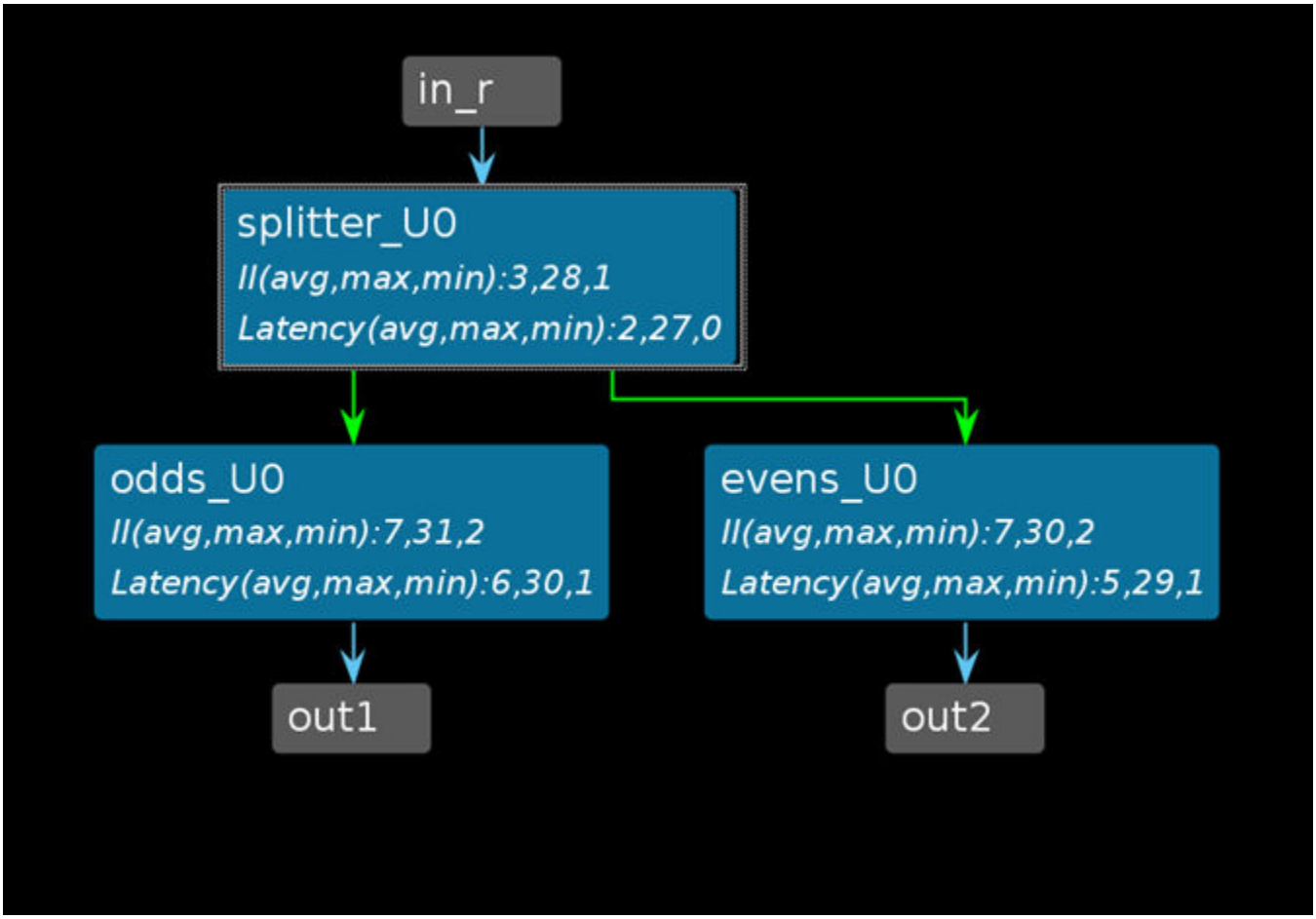
- In the data-driven TLP model, an `hls::stream<type,depth>` object behaves like a FIFO with a specified depth. Such streams have a default depth of 2 which can be overridden by the user.
- The streams are read from and written to sequentially. That implies that once a data item is read from an `hls::stream<>` that same data item cannot be read again.

> **TIP:** *Accesses to different streams are not ordered (for example the order of a write to a stream and a read from a different stream can be changed by the scheduler).*

- Streams can be defined either locally or globally. Streams defined in the global scope follow the same rules as any other global variables.
- The `hls_thread_local` qualifier is also required for streams (`s1` and `s2` in the example below) to keep the same streams alive across multiple calls of the instantiating function (`odds_and_evens` in the code example below).

The following diagram shows the graphical representation in Vitis HLS of the code example above. In this diagram, the green colored arrows are FIFO channels while the blue arrows indicate the inputs and outputs of the instantiating function (`odds_and_evens`). Tasks are shown as blue rectangular boxes.

Figure 11: **Dataflow Diagram of `hls::task` Example**



Given a read of an empty stream is a blocking read, deadlocks can occur due to:

- The design itself, where the production and consumption rates by processes are unbalanced.
  - During C simulation, deadlocks can occur only due to a cycle of processes, or a chain of processes starting from a top-level input, that are attempting to read from empty channels.
  - Deadlocks can occur during both C/RTL Co-simulation and when running in hardware (HW) due to cycles of processes trying to write to full channels and/or reading from empty channels.
- The test bench, which is providing less data than those that are needed to produce all the outputs that the test bench is expecting when checking the computation results.

Due to this, a deadlock detector is automatically instantiated when the design contains an `hls::task`. The deadlock detector detects deadlocks and stops the C simulation. Further debugging is performed using a C debugger such as `gdb` and looking at where the simulated `hls::tasks` are all blocked trying to read from an empty channel. This is easy to do using the Vitis HLS GUI as shown in the handling_deadlock example for debugging deadlocks.

In summary, the `hls::task` model is recommended if your design requires a completely data-driven, pure streaming type of behavior, with no sort of control. This type of model is also useful in modeling feedback and dynamic multi-rate designs. Feedback in the design occurs when there is a cyclical dependency between tasks. Dynamic multi-rate models, where the producer writes data or consumer reads data at a rate that is data dependent, can only be handled by the data-driven TLP. The simple_data_driven design on GitHub is an example of this.



TIP: *Static multi-rate designs, in which the producer writes data or consumer reads data at rates that are data-independent, can be managed by both data-driven and control-driven TLP. For example, the producer writes two values in a stream for each call, the consumer reads one value per call.*

Control-driven Task-level Parallelism

Control-driven TLP is useful to model parallelism while relying on the sequential semantics of C++, rather than on continuously running threads. Examples include functions that can be executed in a concurrent pipelined fashion, possibly within loops, or with arguments that are not channels but C++ scalar and array variables, both referring to on-chip and to off-chip memories. For this kind of model, Vitis HLS introduces parallelism where possible while preserving the behavior obtained from the original C++ sequential execution. The control-driven TLP (or dataflow) model provides:

- A subsequent function can start before the previous finishes
- A function can be restarted before it finishes
- Two or more sequential functions can be started simultaneously

While using the dataflow model, Vitis HLS implements the sequential semantics of the C++ code by automatically inserting synchronization and communication mechanisms between the tasks.

The dataflow model takes this series of sequential functions and creates a task-level pipeline architecture of concurrent processes. The tool does this by inferring the parallel tasks and channels. The designer specifies the region to model in the dataflow style (for example, a function body or a loop body) by specifying the DATAFLOW pragma or directive as shown below. The tool scans the loop/function body, extracts the parallel tasks as parallel processes, and establishes communication channels between these processes. The designer can additionally guide the tool to select the type of channels - for example, FIFO (`hls::stream` or `#pragma HLS STREAM`) or PIPO or `hls::stream_of_blocks`. The dataflow model is a powerful method for improving design throughput and latency.

In order to understand how Vitis HLS transforms your C++ code into the dataflow model, refer to the simple_fifos example shown below. The example applies the dataflow model to the top-level `diamond` function using the DATAFLOW pragma as shown.

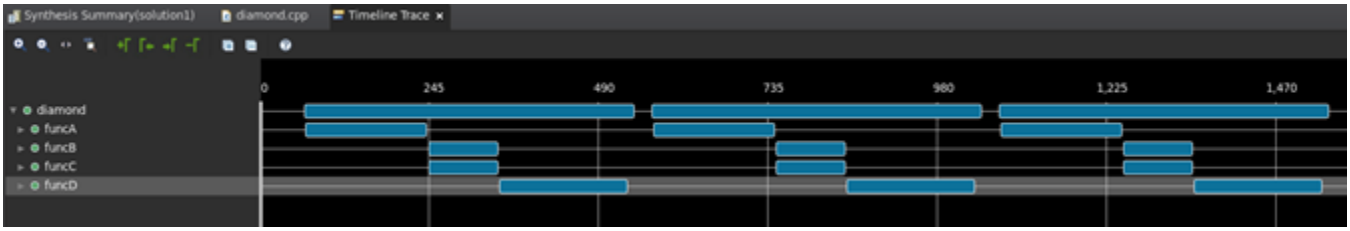
```
#include "diamond.h"

void diamond(data_t vecIn[N], data_t vecOut[N])
{
  data_t c1[N], c2[N], c3[N], c4[N];
#pragma HLS dataflow
  funcA(vecIn, c1, c2);
  funcB(c1, c3);
  funcC(c2, c4);
  funcD(c3, c4, vecOut);
}
```

In the above example, there are four functions: `funcA`, `funcB`, `funcC`, and `funcD`. `funcB` and `funcC` do not have any data dependencies between them and therefore can be executed in parallel. `funcA` reads from the non-local memory (`vecIn`) and needs to be executed first. Similarly, `funcD` writes to the non-local memory (`vecOut`) and therefore has to be executed last.

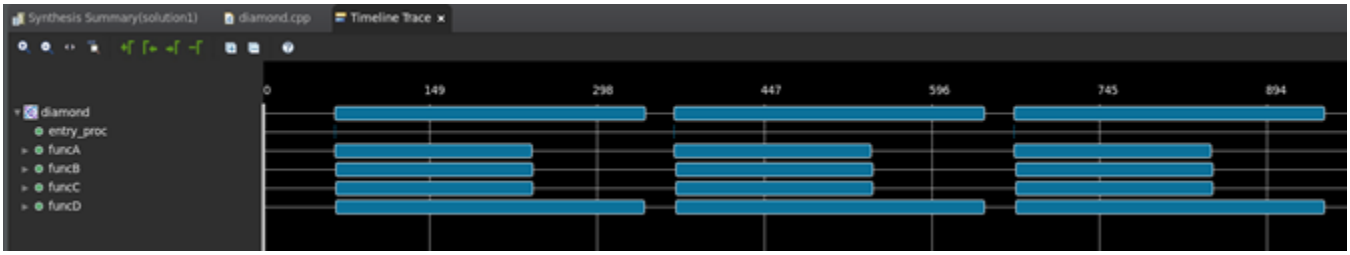
The following waveform shows the execution profile of this design without the dataflow model. There are three calls to the function diamond from the test bench. `funcA`, `funcB`, `funcC`, and `funcD` are executed in sequential order. Each call to diamond, therefore, takes 475 cycles in total as shown in the following figure.

Figure 12: **Diamond Example without Dataflow**



In the following figure, when the dataflow model is applied and the designer selected to use FIFOs for channels, all the functions are started immediately by the controller and are stalled waiting on input. As soon as the input arrives, it is processed and sent out. Due to this type of overlap, each call to diamond now only takes 275 cycles in total as shown below. Refer to Combining the Three Paradigms for a more detailed discussion of the types of parallelism that can be achieved for this example.

Figure 13: **Diamond Example with Dataflow**



This type of parallelism cannot be achieved without incurring an increase in hardware utilization. When a particular region, such as a function body or a loop body, is identified as a region to apply the dataflow model, Vitis HLS analyzes the function or loop body and creates individual channels from C++ variables (such as scalars, arrays, or user-defined channels such as `hls::streams` or `hls::stream_of_blocks`) that model the flow of data in the dataflow region. These channels can be simple FIFOs for scalar variables, or ping-pong (PIPO) buffers for non-scalar variables like arrays (or stream of blocks when you need a combination of FIFO and PIPO behavior with explicit locking of the blocks).

Each of these channels can contain additional signals to indicate when the channel is full or empty. By having individual FIFOs and/or PIPO buffers, Vitis HLS frees each task to execute independently and the throughput is only limited by the availability of the input and output buffers. This allows for better overlapping of task execution than a normal pipelined implementation, but does so at the cost of additional FIFO or block RAM registers for the ping-pong buffer.

TIP: *This overlapped execution optimization is only visible after you run the cosimulation of the design - it is not observable statically (though can be easily imagined in the Dataflow Viewer after C Synthesis).*

The dataflow model is not limited to a chain of processes but can be used on any directed acyclic graph (DAG) structure, or cyclic structure when using `hls::streams`. It can produce two different forms of overlapping: within an iteration if processes are connected with FIFOs, and across different iterations when connected with PIPOs and FIFOs. This potentially improves performance over a statically pipelined solution. It replaces the strict, centrally-controlled pipeline stall philosophy with a distributed handshaking architecture using FIFOs and/or PIPOs. The replacement of the centralized control structure with a distributed one also benefits the fanout of control signals, for example register enables, which is distributed among the control structures of individual processes. Refer to the Task Level Parallelism/Control-Driven examples on GitHub for more examples of these concepts.

Canonical Forms

Vitis HLS transforms the region to apply the DATAFLOW optimization. For more predictibility of the resulting dataflow network, AMD recommends writing the code inside this region (referred to as the canonical region) using canonical forms. There are two main canonical forms for the dataflow optimization:

1. The canonical form for a function where sub-functions are not inlined. These subfunctions can themselves be dataflow in function regions or dataflow inside loop regions. Note also that variable initialization (including those performed automatically by constructors) or passing expressions by value to processes are not part of canonical form. For standard data types that have default constructors that cannot be redefined (For example: std::complex), this can be achieved by using no_ctor (For example: std::complex<float> arr[SIZE] __attribute__((no_ctor));). Vitis HLS does its best to implement the resulting dataflow but, if the code is not in canonical form, you should always check the GUI dataflow viewer and the cosimulation timeline trace to ensure that the dataflow happens as expected and the achieved performance is as expected.

```
void dataflow(Input0, Input1, Output0, Output1)
{
 #pragma HLS dataflow
 UserDataType C0, C1, C2;        // UserDataType can be scalars or arrays
 func1(Input0, Input1, C0, C1); // read Input0, read Input1, write C0,
write C1
 func2(C0, C1, C2);              // read C0, read C1, write C2
 func3(C2, Output0, Output1);    // read C2, write Output0, write Output1
}
```

2. Dataflow inside a loop body enclosed in a function without any other code but the loop. For the `for` loop (where no function inside is inlined), the integral loop variable should have:
   a. The initial value is declared in the loop header and set to 0.
   b. The loop bound is a non-negative numerical constant or scalar argument of the function that encloses the loop.
   c. Increment by 1.
   d. Dataflow pragma needs to be inside the loop as shown below.

```
void dataflow(Input0, Input1, Output0, Output1)
{
    for (int i = 0; i < N; i++)
    {
    #pragma HLS dataflow
        UserDataType C0, C1, C2;        // UserDataType can be scalars or
arrays
        func1(Input0, Input1, C0, C1); // read Input0, read Input1,
write C0, write C1
        func2(C0, C1, C2);              // read C0, read C1, write C2
        func3(C2, Output0, Output1);    // read C2, write Output0, write
Output1
    }
}
```

Canonical Body

Inside the canonical region, the canonical body should follow these guidelines:

1. Use a local, non-static scalar or an array variable. A local variable is declared inside the function body (for dataflow in a function) or loop body (for dataflow inside a loop). Refer to Limitations of Control-Driven Task-Level Parallelism for additional limitations on arrays.
2. A sequence of function calls that pass data forward (with no feedback unless using `hls::stream/hls::stream_of_blocks`), from a function to one that is lexically later, under the following conditions:
    a. Variables (except scalar) can have only one reading process and one writing process.
    b. Use write before read (producer before consumer) if you are using local non-scalar variables, which then become channels. For scalar variables, both write before read and read before write are allowed.
    c. Function return type must be void.
    d. No loop-carried dependencies are allowed among different processes via variables except when FIFOs are used. Forward loop-carried dependencies are supported for arrays transformed to streams and both forward and backward dependencies are supported for `hls::streams`.
    e. No control whatsoever is supported inside a dataflow region, except inside function calls (that define processes).
        - For canonical dataflow, there should be no conditionals, no loops, no return or goto statements, and no C++ exceptions such as throw.

Dataflow Checking

The Vitis Unified IDE and `v++` command has a dataflow checker which, when enabled, checks the code to see if it is in the recommended canonical form. Otherwise, it emits an error/warning message to the user. By default, this checker is set to `warning`. You can set the checker to `error` or disable it by selecting `off` in the `syn.dataflow.strict_mode` configuration command as described in Dataflow Configuration.

Configuring Dataflow Memory Channels

Vitis HLS implements channels between the tasks as either PIPO or FIFO buffers, depending on the user's choice:

- For scalars, Vitis HLS automatically infers FIFOs as the channel type.

- If the parameter (of a producer or consumer) is an array, the user has a choice of implementing the channel as a PIPO or a FIFO based on the following considerations:
  - If the data is always accessed in sequential order, the user can choose to implement this memory channel as PIPO/FIFO. Choosing PIPOs comes with the advantage that PIPOs can never deadlock but they require more memory to use. Choosing FIFOs offers the advantage of lesser memory requirements but this comes with the risk of deadlock if the FIFO sizes are not correct.
  - If the data is accessed in an arbitrary manner, the memory channel must be implemented as a PIPO (with a default size that is twice the size of the original array).

    TIP: *A PIPO ensures that the channel always has the capacity to hold all samples produced in one iteration, without a loss.*

Specifying the size of the FIFO channels overrides the default value that is computed by the tool to attempt to optimize the throughput. If any function in the design can produce or consume samples at a greater rate than the specified size of the FIFO, the FIFOs might become empty (or full). In this case, the design halts operation, because it is unable to read (or write). This might lead to a stalled, deadlock state.

TIP: *If a deadlocked situation is created, you only see this when executing C/RTL co-simulation or when the block is used in a complete system.*

When setting the depth of the FIFOs, AMD recommends initially setting the depth as the maximum number of data values transferred (for example, the size of the array passed between tasks), confirming the design passes C/RTL co-simulation, and then reducing the size of the FIFOs and confirming C/RTL co-simulation still completes without issues. If RTL co-simulation fails, the size of the FIFO is likely too small to prevent stalling or a deadlock situation. The Vitis HLS GUI now supports an automatic way of determining the right FIFO size to use. Additionally, the Vitis HLS IDE can display a histogram of the size of each FIFO/PIPO buffer over time, after RTL co-simulation has been run. This can be useful to help determine the best depth for each buffer.

Specifying Arrays as PIPOs or FIFOs

All arrays are implemented by default as ping-pong to enable random access. These buffers can also be re-sized if needed. For example, in some circumstances, such as when a task is being bypassed, performance degradation is possible. To mitigate this effect on performance, you can give more slack to the producer and consumer by increasing the size of these buffers by using the STREAM pragma or directive as shown below.

```
void top ( ... ) {
#pragma HLS dataflow
  int A[1024];
#pragma HLS stream type=pipo variable=A depth=3

  producer(A, B, …);  // producer writes A and B
  middle(B, C, ...);  // middle reads B and writes C
  consumer(A, C, …);  // consumer reads A and C
```

Arrays on the interface are defined as `m_axi` or `ap_memory` by default. However, arrays can be specified as streaming if the INTERFACE pragma or directive defines the array as `ap_fifo` or `axis`.

Inside the design, an array must be specified as streaming using the STREAM pragma or directive if a FIFO is desired for the implementation.



TIP: *When the STREAM directive is applied to an array, the resulting FIFO implemented in the hardware contains as many elements as the array. However, the `depth` option can be used to specify the size of the FIFO as described in syn.directive.stream.*

The STREAM pragma or directive can also be used to override the default implementation specified by the `syn.dataflow.fifo_depth` configuration command for any arrays in a DATAFLOW region, as described in Dataflow Configuration. In general, the use of the STREAM pragma or directive for a given array is preferable to the global option.

- If the `syn.dataflow.default_channel` is set as pingpong, any array can still be implemented as a FIFO by applying the STREAM directive to the array.

  TIP: *To use a FIFO implementation, the array must be accessed in sequential order (and not in random access order).*

- If the `syn.dataflow.default_channel` is set to FIFO, any array can still be implemented as a ping-pong implementation by applying the `type=pipo` to the STREAM pragma or directive for the array.



IMPORTANT! *To preserve sequential accesses, and thus the correctness of streaming, it might be necessary to prevent compiler optimizations (dead code elimination particularly) by using the `volatile` qualifier.*

When an array in a DATAFLOW region is specified as streaming and implemented as a FIFO, the FIFO is typically not required to hold the same number of elements as the original array. The tasks in a DATAFLOW region consume each data sample as soon as it becomes available. The depth of the FIFO can be specified using the `syn.dataflow.fifo_depth` option or the STREAM pragma or directive with the `depth` option. This can be used to set the size of the FIFO to ensure the flow of data never stalls.

If the `type=pipo` option is selected, the `depth` option sets the depth (number of blocks) of the PIPO.

Specifying Arrays as Stream-of-Blocks

The `hls::stream_of_blocks` type provides a user-synchronized stream that supports streaming blocks of data for process-level interfaces in a dataflow context, where each block is an array or multidimensional array. The intended use of stream-of-blocks is to replace array-based communication between a pair of processes within a dataflow region. Refer to the using_stream_of_blocks example on GitHub.

Currently, Vitis HLS implements arrays written by a producer process and read by a consumer process in a dataflow region by mapping them to ping pong buffers (PIPOs). The buffer exchange for a PIPO buffer occurs at the return of the producer function and the calling of the consumer function in C++.

Stream-of-Blocks Modeling Style

On the other hand, for a stream-of-blocks the communication between the producer and the consumer is modeled as a stream of array-like objects, providing several advantages over array transfer through PIPO.

The use of stream-of-blocks in your code requires the following include file:

```
#include "hls_streamofblocks.h"
```

The stream-of-blocks object template is:

```
hls::stream_of_blocks<block_type, depth> v
```

Where:

- `<block_type>` specifies the datatype of the array or multidimensional array held by the stream-of-blocks
- `<depth>` is an optional argument that provides depth control like `hls::stream` or PIPOs, and specifies the total number of blocks, including the one acquired by the producer and the one acquired by the consumer at any given time. The default value is 2
- `v` specifies the variable name for the stream-of-blocks object

Use the following steps to access a block in a stream of blocks:

1. The producer or consumer process that wants to access the stream first needs to acquire access to it, using a `hls::write_lock` or `hls::read_lock` object.
2. After the producer has acquired the lock it can start writing (or reading) the acquired block. Once the block has been fully initialized, it can be released by the producer, when the `write_lock` object goes out of scope.

   Note: The producer process with a `write_lock` can also read the block as long as it only reads from already written locations, because the newly acquired buffer must be assumed to contain uninitialized data. The ability to write and read the block is unique to the producer process, and is not supported for the consumer.
3. Then the block is queued in the stream-of-blocks in a FIFO fashion, and when the consumer acquires a `read_lock` object, the block can be read by the consumer process.

The main difference between `hls::stream_of_blocks` and the PIPO mechanism seen in the prior examples is that the block becomes available to the consumer as soon as the `write_lock` goes out of scope, rather than only at the return of the producer process. Therefore the amount of storage is much less with stream-of-blocks than with PIPOs: namely 2N instead of 2xMxN.

The producer acquires the block by constructing an `hls::write_lock` object called `b`, and passing it the reference to the stream-of-blocks object, called `s`. The `write_lock` object provides an overloaded array access operator, letting it be accessed as an array to access underlying storage in random order as shown in the example below.

The acquisition of the lock is performed by constructing the `write_lock`/`read_lock` object, and the release occurs automatically when that object is destructed as it goes out of scope. This approach uses the common *Resource Acquisition Is Initialization* (RAII) style of locking and unlocking.

```
#include "hls_streamofblocks.h"
typedef int buf[N];
void producer (hls::stream_of_blocks<buf> &s, ...) {
  for (int i = 0; i < M; i++) {
    // Allocation of hls::write_lock acquires the block for the producer
    hls::write_lock<buf> b(s);
    for (int j = 0; j < N; j++)
      b[f(j)] = ...;
    // Deallocation of hls::write_lock releases the block for the consumer
  }
}

void consumer(hls::stream_of_blocks<buf> &s, ...) {
  for (int i = 0; i < M; i++) {
    // Allocation of hls::read_lock acquires the block for the consumer
    hls::read_lock<buf> b(s);
    for (int j = 0; j < N; j++)
       ... = b[g(j)] ...;
    // Deallocation of hls::write_lock releases the block to be reused by
the producer
  }
}
```

```
void top(...) {
#pragma HLS dataflow
  hls::stream_of_blocks<buf> s;

  producer(b, ...);
  consumer(b, ...);
}
```

The key features of this approach include:

- The expected performance of the outer loop in the producer above is to achieve an overall Initiation Interval (II) of 1
- A locked block can be used as though it were private to the producer or the consumer process until it is released.
- The initial state of the array object for the producer is undefined, whereas it contains the values written by the producer for the consumer.
- The principal advantage of stream-of-blocks is to provide overlapped execution of multiple iterations of the consumer and the producer to increase throughput.

Resource Usage

The resource cost when increasing the depth beyond the default value of 2 is similar to the resource cost of PIPOs. Namely, each increment of 1 requires enough memory for a block, for example, in the example above N * 32-bit words.

The stream of blocks object can be bound to a specific RAM type, by placing the `BIND_STORAGE` pragma where the stream-of-blocks is declared, for example in the top-level function. The stream of blocks uses 2-port block RAM (`type=RAM_2P`) by default.

Specifying Compiler-Created FIFO Depth

Start Propagation FIFOs

The compiler might automatically create a start FIFO to propagate the `ap_start/ap_ready` handshake to an internal process. Such FIFOs can sometimes be a bottleneck for performance, in which case you can increase the default size (which can be incorrectly estimated by the tool) with the `syn.dataflow.start_fifo_depth` configuration command.

If an unbounded slack between producer and consumer is needed, and internal processes can run forever, fully and safely driven by their inputs or outputs (FIFOs or PIPOs), these start FIFOs can be removed locally for a given dataflow region, at your own risk, with the following pragma or directive:

```
#pragma HLS DATAFLOW disable_start_propagation
```

Scalar Propagation FIFOs

The compiler automatically propagates some scalars from C/C++ code through scalar FIFOs between processes. Such FIFOs can sometimes be a bottleneck for performance or cause deadlocks, in which case you can set the size with the `syn.dataflow.scalar_fifo_depth` configuration command.



TIP: *The default depth is set with `syn.dataflow.fifo_depth`.*

Stable Arrays

The `stable` pragma can be used to mark input or output variables of a dataflow region. Its effect is to remove their corresponding task-level synchronizations, assuming that the user guarantees this removal is indeed correct.

```
void dataflow_region(int A[...], ...
#pragma HLS stable variable=A
#pragma HLS dataflow
    proc1(...);
    proc2(A, ...);
```

Without the `stable` pragma, and assuming that `A` is read by `proc2`, then `proc2` would be part of the initial synchronization for the dataflow region where it is located. This means that `proc1` would not restart until `proc2` is also ready to start again, which would prevent dataflow iterations to be overlapped and induce a possible loss of performance. The `stable` pragma indicates that this synchronization is not necessary to preserve correctness.

With the `stable` pragma, the compiler assumes that:

- If `A` is read by `proc2`, then the memory locations that are read are still accessible and can not be overwritten by any other process or calling context, while the `dataflow_region` is being executed.
- If `A` is written by `proc2`, then the memory locations written can not be read, before their definition, by any other process or calling context, while `dataflow_region` is being executed.

A typical scenario is when the caller updates or reads these variables only when the dataflow region has not started yet or has completed execution.

In summary, the Dataflow optimization is a powerful optimization that can significantly improve the throughput of your design. As there is reliance on the HLS tool to do the inference of the available parallelism in your design, it requires the designer's help to ensure that the code is written in such a way that the inference is straightforward for the HLS tool. Finally, there can be situations where the designer might see the need to deploy both the Dataflow model and the Task-Channel model in the same design. The next section describes this hybrid combination model that can lead to some interesting designs.

Mixing Data-Driven and Control-Driven Models

The following table highlights factors of the HLS design that can help you determine when to apply control-driven task-level parallelism (TLP) or data-driven TLP.

Table 3: **Control-Driven and Data-Driven TLP**

Control-Driven TLP	Data-Driven TLP
HLS Design requires control signals to start/stop the process	HLS Design uses a completely data-driven approach that does not require control signals to start/stop the process
Design requires non-local memory access	Design uses pure streaming data transfers
Design requires interaction with an external software application	Designs with data-dependent multi-rate behavior: • Producer writes data or consumer reads data at a rate that is data dependent • Easier to model for designs that require feedback between processes
Designs with multiple processes running the same number of executions	Task-level parallelism is observable in C simulation and RTL simulation.
Requires RTL simulation to model the effect of the parallelism	

As the preceding table indicates, the two forms of task-level parallelism presented have different use cases and advantages. However, sometimes it is not possible to design an entire application that is purely data-driven TLP, while some portion of the design can still be constructed as a purely streaming design. In this case a mixed control-driven/data-driven model can be useful to create the application. Consider the following mixed_control_and_data_driven example from GitHub.

```
void dut(int in[N], int out[N], int n) {
#pragma HLS dataflow
  hls_thread_local hls::split::round_robin<int, NP> split1;
  hls_thread_local hls::merge::round_robin<int, NP> merge1;

  read_in(in, n, split1.in);

  // Task-Channels
  hls_thread_local hls::task t[NP];
  for (int i=0; i<NP; i++) {
#pragma HLS unroll
    t[i](worker, split1.out[i], merge1.in[i]);
  }

  write_out(merge1.out, out, n);
}
```

In the above example, there are two distinct regions - a dataflow region that has the functions read_in/write_out in which the sequential semantics is preserved - for example, `read_in` can be executed before `write_out` and a task-channel region that contains the dynamic instantiation of 4 tasks (since NP = 4 in this example) along with some special type of channels called a `split` or a `merge` channel. A split channel is one that has a single input but has multiple outputs - in this case, the split channel has 4 outputs as described in HLS Split/Merge Library. Similarly, a merge channel has multiple inputs but only one output.

In addition, to the ports, these channels also support an internal job scheduler. In the above example, both the merge and the split channels have selected a round-robin scheduler that assigns the incoming data to each of the 4 tasks, one by one starting with `worker_U0`. If a load balancing scheduler had been chosen then the incoming data has been assigned to the first available worker task (and this would lead to a non-deterministic simulation since this order might be different each time you run the simulation). Because this is a pure task-channel region, the 4 tasks are executed in parallel as soon as there is data in their incoming stream. Refer to the merge_split example on GitHub for more examples of these concepts.

It is important to note that, although the code above can give the impression that each task is "*called*" in the loop, and connected to a potentially different pair of channels every time the loop body is executed, in reality, this usage implies a static instantiation, i.e.:

- each `t[i](...)` call must be executed exactly once per execution of `dut()`.
- the loop over `i` must be fully unrolled, to infer a corresponding set of 4 instances in RTL.
- The `dut()` function must be called exactly once by the test bench.
- Each split output or merge input must be bound to exactly one `hls::task` instance.

While it is true that for `hls::task` objects the order of specification does not matter, for the control-driven dataflow network, Vitis HLS must be able to see that there is a chain of processes, such as from `read_in` to `write_out`. To define this chain of processes, Vitis HLS uses the calling order, which for `hls::tasks` is also the declaration order. This means that the model must define an explicit order from the `read_in` function to the `hls::task` region and then finally to the `write_out` function in the dataflow region as shown in the example above.

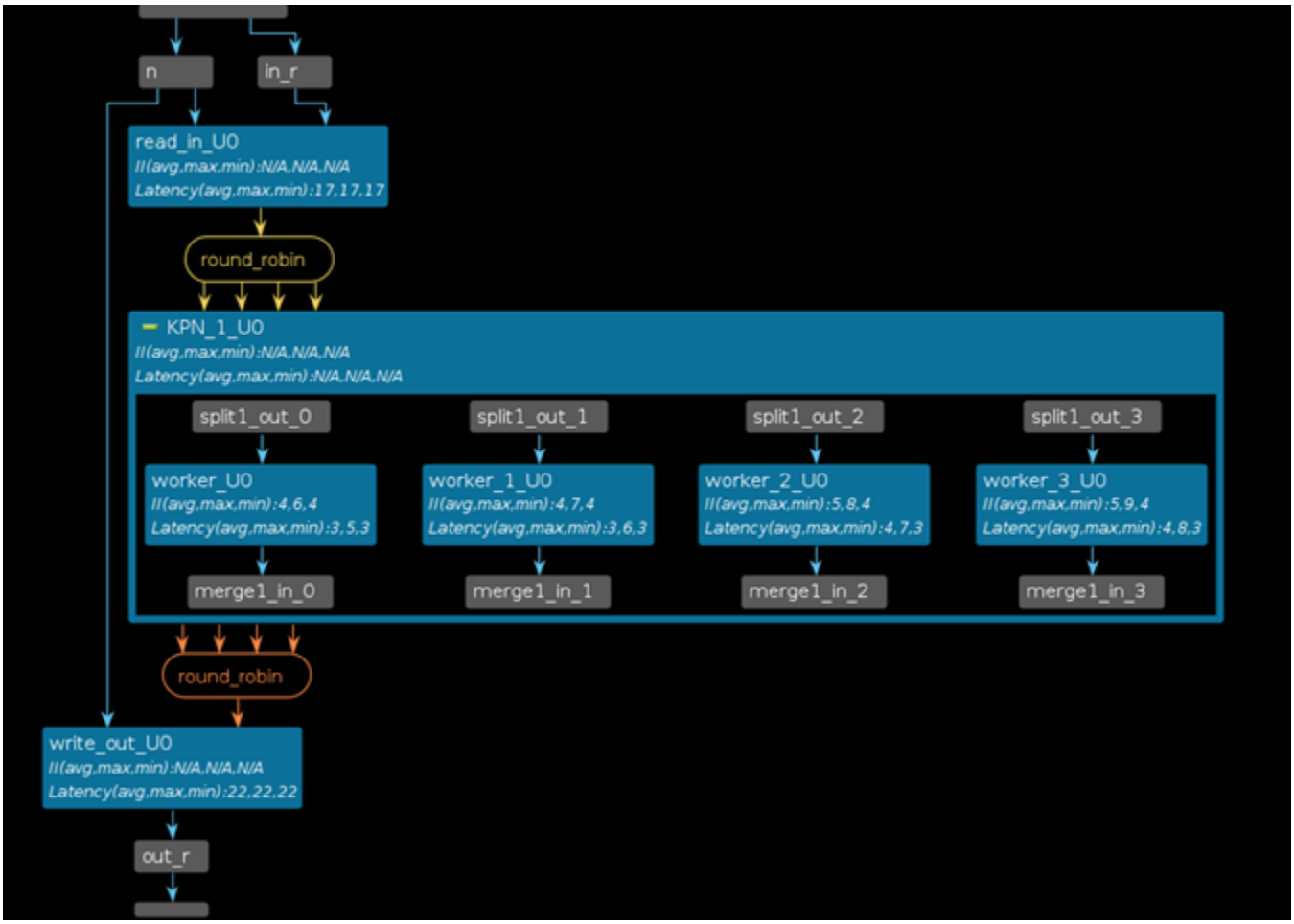
Generally:

- If a control-based process (i.e. regular dataflow) produces a stream for an `hls::task`, then it must be called before the declaration of the tasks in the code
- If a control-based process consumes a stream from an `hls::task`, then it must be called after the declaration of the tasks in the code

Violation of the above rules can cause unexpected outcomes because each of the NP `hls::task` instances is statically bound to the channels that are used in the first invocation of `t[i](...)`.

The following diagram shows the graph of this mixed task-channel and dataflow example:

Figure 14: **Mixed Task-Channel and Dataflow**



Summary

HLS designs that are purely data-driven and that do not require any interaction with the software application can be modeled using data-driven TLP models. Examples of such designs are:

- Simple rule-based “firewall” with “rules” compiled into the kernel
- Fast-Fourier Transforms with configuration data compiled into the kernel
- FIR filters with coefficients compiled into the kernel

If the design requires data transfer to/from external memory, then the control-driven TLP model can used. Examples of such designs include:

- Network router where the routing table must be updated entirely for kernel execution

- Load-balancer that uses a hash map to send data to a server, that must update server list, server map, and corresponding IP addresses simultaneously

However, most designs can be a mixed control-driven and data-driven model, requiring some access to external memory, and enabling streaming between parallel and pipelined tasks within the HLS design.

In summary, this chapter described some modeling choices to consider when designing your application written in C/C++. So far, this discussion talked about structuring your algorithm at a high level to make use of these special models such as the task-channel or dataflow optimizations. Another key aspect to achieving good throughput is to also consider instruction-level parallelism. Instruction-level parallelism in HLS refers to the ability to efficiently parallelize the operations inside loops, functions, and even arrays. The next few sections walk you through these lower-level optimizations that work hand-in-hand with the macro-level optimizations.

Loops Primer

When writing code intended for high-level synthesis (HLS), there is a frequent need to implement repetitive algorithms that process blocks of data — for example, signal, or image processing. Typically, the C/C++ source code tends to include several loops or several nested loops.

When it comes to optimizing performance, loops are one of the best places to start exploring optimization. Each iteration of the loop takes at least one clock cycle to execute in hardware. Thinking from the hardware perspective, there is an implicit *wait until clock* for the loop body. The next iteration of a loop only starts when the previous iteration is finished. To improve performance loops can generally be either *pipelined* or *unrolled* to take advantage of the highly distributed and parallel FPGA architecture, as explained in the following sections.

Pipelining Loops

Pipelining loops permits starting the next iteration of a loop before the previous iteration finishes, enabling portions of the loop to overlap in execution. By default, every iteration of a loop only starts when the previous iteration has finished. In the loop example below, a single iteration of the loop adds two variables and stores the result in a third variable. Assume that in hardware this loop takes three cycles to finish one iteration. Also, assume that the loop variable `len` is 20, that is, the `vadd` loop runs for 20 iterations in the kernel. Therefore, it requires a total of 60 clock cycles (20 iterations * 3 cycles) to complete all the operations of this loop.

```
vadd: for(int i = 0; i < len; i++) {
   c[i] = a[i] + b[i];
}
```



TIP: *It is good practice to always label a loop as shown in the example above (`vadd:...`). This practice helps with debugging the design in Vitis HLS. Sometimes the unused labels generate warnings during compilation, which can be safely ignored.*

Pipelining the loop allows subsequent iterations of the loop to overlap and run concurrently. Pipelining a loop can be enabled by adding the PIPELINE pragma or directive inside the body of the loop as shown below:

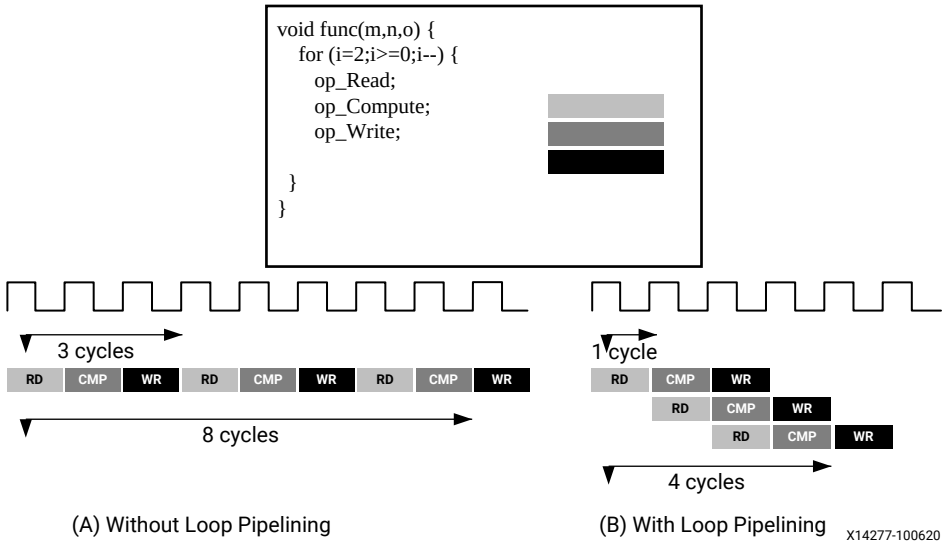
```
vadd: for(int i = 0; i < len; i++) {
#pragma HLS PIPELINE
c[i] = a[i] + b[i];
}
```

TIP: *Vitis HLS automatically pipelines loops with more than 64 iterations. This feature can be changed or disabled using the* `syn.compile.pipeline_loops` *configuration command as described in Compile Options.*

The number of cycles it takes to start the next iteration of a loop is called the Initiation Interval (II) of the pipelined loop. So II = 2 means the next iteration of a loop starts two cycles after the current iteration. An II = 1 is the ideal case, where each iteration of the loop starts in the very next cycle. When you use pragma HLS pipeline, you can specify the II for the compiler to achieve. If a target II is not specified, the compiler can try to achieve II=1 by default.

The following figure illustrates the difference in execution between pipelined and non-pipelined loops. In this figure, (A) shows the default sequential operation where there are three clock cycles between each input read (II = 3), and it requires eight clock cycles before the last output write is performed.

Figure 15: **Loop Pipelining**



In the pipelined version of the loop shown in (B), a new input sample is read every cycle (II = 1) and the final output is written after only four clock cycles: substantially improving both the II and latency while using the same hardware resources.

IMPORTANT! *Pipelining a loop causes any loops nested inside the pipelined loop to get automatically unrolled.*

If there are data dependencies inside a loop, it might not be possible to achieve II = 1, and a larger initiation interval might be the result. Loop dependencies are data dependencies that can constrain the optimization of loops, typically pipelining. They can be within a single iteration of a loop and or between different iterations of a loop. The easiest way to understand loop dependencies is to examine an extreme example. In the following example, the result of the loop is used as the loop continuation or exit condition. Each iteration of the loop must finish before the next can start.

```
Minim_Loop: while (a != b) {
if (a > b)a -= b;
else b -= a;
}
```

The `Minim_Loop` loop in the example above cannot be pipelined because the next iteration of the loop cannot begin until the previous iteration ends. Not all loop dependencies are as extreme as this, but the example highlights that some operations cannot begin until some other operation has been completed. The solution is to try to ensure that the initial operation is performed as early as possible.

Loop dependencies can occur with any and all types of data. They are particularly common when using arrays.

Automatic Loop Pipelining

The `syn.compile.pipeline_loops` configuration command enables loops to be pipelined automatically based on the iteration count. All loops with a trip count greater than the specified limit are automatically pipelined. The default limit is 64.

Given the following example code:

```
loop1: for (i = 0; i < 5; i++) {
   // do something 5 times ...
}
```

If the `syn.compile.pipeline_loops` command is set to 4, the `for` loop (`loop1`) is pipelined. This is equivalent to adding the PIPELINE pragma as shown in the following code snippet:

```
loop1: for (i = 0; i < 5; i++) {
#pragma HLS PIPELINE II=1
   // do something 5 times ...
}
```

However, for nested loops, the tool starts at the innermost loop, and determines if it can be pipelined, and then moves up the loop nest. In the example below, the `syn.compile.pipeline_loops` threshold is still 4, and the inner most loop (`loop1`) is marked to be pipelined. Then the tool evaluates the parent loop of the loop pair (`loop2`) and finds that it can also be pipelined. In this case, the tool unrolls `loop1` into `loop2`, and marks `loop2` to be pipelined. Then it moves up to the parent loop (`loop3`) and repeats the analysis.

```
loop3: for (y = 0; y < 480; y++) {
   loop2: for (x = 0; x < 640; x++) {
      loop1: for (i = 0; i < 5; i++) {
         // do something 5 times ...
      }
   }
}
```

If there are loops in the design for which you do not want to use automatic pipelining, apply the PIPELINE directive with the `off` option to that loop. The `off` option prevents automatic loop pipelining.

IMPORTANT! *The tool applies the `syn.compile.pipeline_loops` command after performing all user-specified directives. For example, if a user-specified UNROLL directive to a loop is applied, the loop is first unrolled and automatic loop pipelining can not be applied as the loop is eliminated.*

If the `pipeline_loops` threshold is set to 0, will turn off all auto-pipelining (unless a pragma is added to a loop which overwrites that setting.) And the examples shown always auto-pipeline regardless of the threshold setting. When the tool does/does not auto-pipeline.

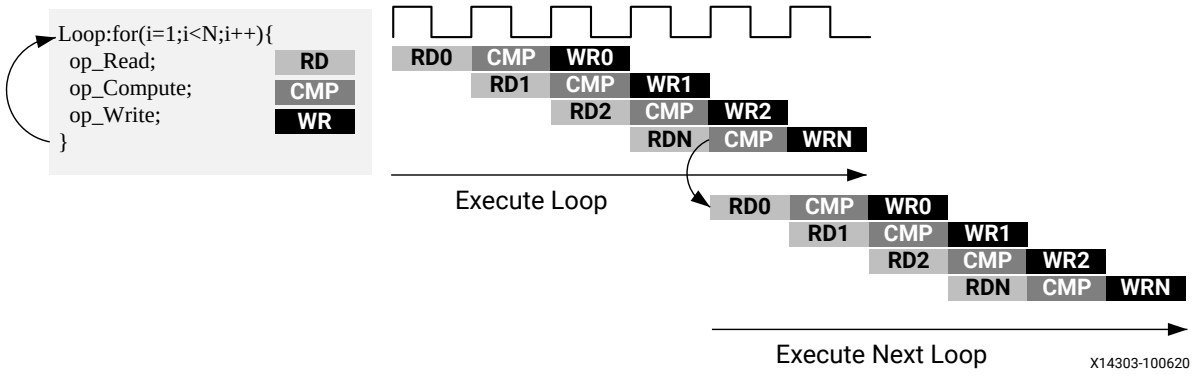
- If a loop is innermost with an unknown trip count or tripcount >= threshold (or > I don't remember) or no loop above, it gets pipeline unless there is pipeline off.
- If a loop has a known tripcount < threshold and a loop above, check if it can be pipelined from above: for that, the loop above must only contain subloops that will also be unrolled (For example, with tripcount < threshold). If yes, the pipeline will be placed in this loop above.
- Actually, the same happens for the loop above, the cumulative tripcount is what counts. For example, the number of iterations that were unrolled, multiplied by the current tripcount, and this goes up until the threshold is reached.

Rewinding Pipelined Loops for Performance

The PIPELINE pragma has an option called `rewind`. This option enables overlap of the execution of successive calls to the pipelined loop when this loop is the outermost construct of the top function, or of a dataflow region and the dataflow region is executed multiple times.

The following figure shows the operation when the `rewind` option is used when pipelining a loop. At the end of the loop iteration count, the loop starts to execute again. While it generally re-executes immediately, a delay is possible and is shown and described in the GUI.

Figure 16: **Loop Pipelining with Rewind Option**



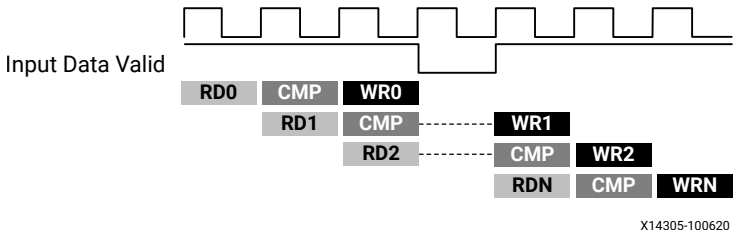
***Note*:** If a loop is used around a DATAFLOW region, Vitis HLS automatically implements it to allow successive executions to overlap.

Flushing Pipelines and Pipeline Types

Flushing Pipelines

Pipelines continue to execute as long as data is available at the input of the pipeline. If there is no data available to process, the pipeline stalls. This is shown in the following figure, where the `Input Data Valid` signal goes low to indicate there is no more valid input data. Once the signal goes high, indicating there is new data available to process, the pipeline continues operation.

Figure 17: **Loop Pipelining with Stall**



In some cases, it is desirable to have a pipeline that can be “emptied” or “flushed.” The `flush` option is provided to perform this. When a pipeline is “flushed” the pipeline stops reading new inputs when none are available (as determined by a data `valid` signal at the start of the pipeline) but continues processing, shutting down each successive pipeline stage, until the final input has been processed through to the output of the pipeline.

As described below, the tool automatically selects the right pipeline style to use for a given pipelined loop or function. However, you can override this default behavior by using the `syn.compile.pipeline_style` configuration command to specify the default pipeline style.

You can also specify the pipeline style for functions or loops with the PIPELINE pragma or directive. This option applies to the specific scope of the pragma or directive and does not change the global default assigned by `syn.compile.pipeline_style`.

Both `stp` and `flp` types of pipelines use the standard pipeline logic where the hardware pipeline created use various kinds of blocking signals to stall the pipeline. These blocking signals often become the driver of a high-fanout net, especially on pipelines that are deep in the number of physical stages and work on significant data sizes. Such high fanout nets, when they are created, are the prime cause of timing closure issues which cannot be fixed in RTL/Logic Synthesis or during Place-and-Route. To solve this issue, a new type of pipeline implementation called *free-running pipeline* (or `frp`) was created. The free-running pipeline is the most efficient architecture for handling a pipeline that operates with blocking signals. This is because

- It completely eliminates the blocking signal connections to the register enables
- It is a fully flushable pipeline, which allows bubbling invalid transactions
- Unlike the previous architectures which distribute fanouts (across flops), this reduces the fanouts
- It does not rely on the synthesis and/or place and route optimizations such as flop cloning
- This helps PnR by creating a structure where the wire length is reduced along with the reduction of the high fanouts

But there is a cost associated with this fanout reduction:

- the size of the FIFO buffers required for the blocking output ports causes additional resource usage.
- the mux delay at those blocking output ports
- the potential performance hit due to early validation of forward-pressure triggers



IMPORTANT! *The free-running pipeline (`frp`) can only be called from within a DATAFLOW region. The `frp` style cannot be applied to a loop that is called in a sequential or a pipelined region.*

Pipeline Types

The three types of pipelines available in the tool are summarized in the following table. The tool automatically selects the right pipeline style to use for a given pipelined loop or function. If the pipeline is used with `hls::tasks`, the flushing pipeline (FLP) style is automatically selected to avoid deadlocks. If the pipeline control requires high fanout, and meets other free-running requirements, the tool selects the free-running pipeline (FRP) style to limit the high fanout. Finally, if neither of the above cases apply, then the standard pipeline (STP) style is selected.

Table 4: **Pipelining Types**

Name	Stalled Pipeline	Free-Running/ Flushable Pipeline	Flushable Pipeline
Use cases	• When there is no timing issue due to high fanout on pipeline control • When flushable is not required (such as no performance or deadlock issue due to stall)	• When you need better timing due to fanout to register enables from pipeline control • When flushable is required for better performance or avoiding deadlock • Can only be called from a dataflow region.	When flushable is required for better performance or avoiding deadlock.
Pragma/Directive	`#pragma HLS pipeline style=stp`	`#pragma HLS pipeline style=frp`	`#pragma HLS pipeline style=flp`
Global Setting	`syn.compile.pipeline_style=stp` (default)	`syn.compile.pipeline_style=frp`	`syn.compile.pipeline_style=flp`
Disadvantages	• Not flushable, hence it can: ◦ Cause more deadlocks in dataflow ◦ Prevent already computed outputs from being delivered, if the inputs to the next iterations are missing • Timing issues due to high fanout on pipeline controls	• Moderate resource increase due to FIFOs added on outputs • Requires at least one blocking I/O (stream or ap_hs). • Not all pipelining scenarios and I/O types are supported.	• Can have larger II • Greater resource usage due to less sharing when II>1
Advantages	• Default pipeline. No usage constraints. • Typically the lowest overall resource usage.	• Better timing due to ◦ Less fanout ◦ Simpler pipeline control logic • Flushable	• Flushable • Avoids deadlock

IMPORTANT! *Flushing pipelines (`flp`) are compatible with the rewind option specified in the PIPELINE pragma or directive when the `syn.compile.enable_auto_rewind` configuration command is also enabled.*

Managing Pipeline Dependencies

Vitis HLS constructs a hardware datapath that corresponds to the C/C++ source code. When there is no pipeline directive, the execution is always sequential and so there are no dependencies that the tool needs to take into account. But when features of the design has been pipelined, the tool needs to ensure that any possible dependencies are respected in the hardware that Vitis HLS generates.

Typical cases of *data dependencies* or *memory dependencies* are when a read or a write occurs after a previous read or write.

- A read-after-write (RAW), also called a true dependency, is when an instruction (and data it reads/uses) depends on the result of a previous operation.
  - I1: t = a * b;
  - I2: c = t + 1;

  The read in statement I2 depends on the write of t in statement I1. If the instructions are reordered, it uses the previous value of t.
- A write-after-read (WAR), also called an anti-dependence, is when an instruction cannot update a register or memory (by a write) before a previous instruction has read the data.
  - I1: b = t + a;
  - I2: t = 3;

  The write in statement I2 cannot execute before statement I1, otherwise the result of b is invalid.
- A write-after-write (WAW) is a dependence when a register or memory must be written in specific order otherwise other instructions might be corrupted.
  - I1: t = a * b;
  - I2: c = t + 1;
  - I3: t = 1;

  The write in statement I3 must happen after the write in statement I1. Otherwise, the statement I2 result is incorrect.
- A read-after-read has no dependency as instructions can be freely reordered if the variable is not declared as volatile. If it is, then the order of instructions has to be maintained.

For example, when a pipeline is generated, the tool needs to take care that a register or memory location read at a later stage has not been modified by a previous write. This is a true dependency or read-after-write (RAW) dependency. A specific example is:

```
int top(int a, int b) {
 int t,c;
I1: t = a * b;
I2: c = t + 1;
 return c;
}
```

Statement `I2` cannot be evaluated before statement `I1` completes because there is a dependency on variable `t`. In hardware, if the multiplication takes 3 clock cycles, then `I2` is delayed for that amount of time. If the above function is pipelined, then Vitis HLS detects this as a true dependency and schedules the operations accordingly. It uses data forwarding optimization to remove the RAW dependency, so that the function can operate at II =1.

Memory dependencies arise when the example applies to an array and not the variables.

```
int top(int a) {
 int r=1,rnext,m,i,out;
 static int mem[256];
L1: for(i=0;i<=254;i++) {
#pragma HLS PIPELINE II=1
I1:     m = r * a; mem[i+1] = m;    // line 7
I2:     rnext = mem[i]; r = rnext; // line 8
 }
 return r;
}
```

In the above example, scheduling of loop `L1` leads to a scheduling warning message:

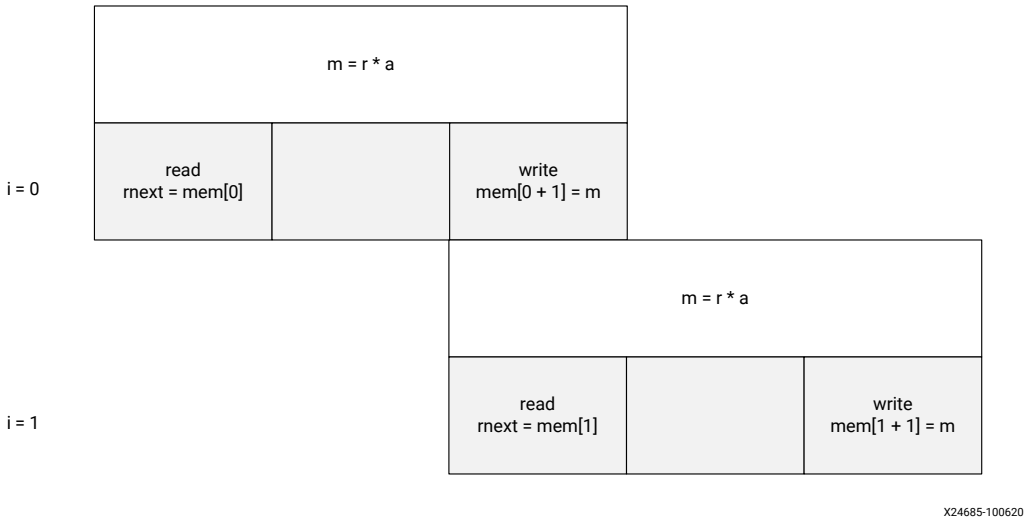
```
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 1, distance = 1) between 'store' operation (top.cpp:7) of variable
'm', top.cpp:7
on array 'mem' and 'load' operation ('rnext', top.cpp:8) on array 'mem'.
INFO: [SCHED 204-61] Pipelining result: Target II: 1, Final II: 2, Depth: 3.
```

There are no issues within the same iteration of the loop as you write an index and read another one. The two instructions could execute at the same time, concurrently. However, observe the read and writes over a few iterations:

```
// Iteration for i=0
I1:     m = r * a; mem[1] = m;      // line 7
I2:     rnext = mem[0]; r = rnext; // line 8
// Iteration for i=1
I1:     m = r * a; mem[2] = m;      // line 7
I2:     rnext = mem[1]; r = rnext; // line 8
// Iteration for i=2
I1:     m = r * a; mem[3] = m;      // line 7
I2:     rnext = mem[2]; r = rnext; // line 8
```

When considering two successive iterations, the multiplication result `m` (with a latency = 2) from statement `I1` is written to a location that is read by statement `I2` of the next iteration of the loop into `rnext`. In this situation, there is a RAW dependence as the next loop iteration cannot start reading `mem[i]` before the previous computation's write completes.

Figure 18: **Dependency Example**



If the clock frequency is increased, then the multiplier needs more pipeline stages and increased latency. This forces II to increase as well.

Consider the following code, where the operations are swapped, changing the functionality:

```
int top(int a) {
 int r,m,i;
 static int mem[256];
L1: for(i=0;i<=254;i++) {
#pragma HLS PIPELINE II=1
I1:     r = mem[i];              // line 7
I2:     m = r * a , mem[i+1]=m; // line 8
 }
 return r;
}
```

The scheduling warning is:

```
INFO: [SCHED 204-61] Pipelining loop 'L1'.
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 1,
distance = 1)
 between 'store' operation (top.cpp:8) of variable 'm', top.cpp:8 on array
'mem'
and 'load' operation ('r', top.cpp:7) on array 'mem'.
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 2,
distance = 1)
 between 'store' operation (top.cpp:8) of variable 'm', top.cpp:8 on array
'mem'
and 'load' operation ('r', top.cpp:7) on array 'mem'.
```

```
WARNING: [SCHED 204-68] Unable to enforce a carried dependency constraint
(II = 3,
distance = 1)
 between 'store' operation (top.cpp:8) of variable 'm', top.cpp:8 on array
'mem'
and 'load' operation ('r', top.cpp:7) on array 'mem'.
INFO: [SCHED 204-61] Pipelining result: Target II: 1, Final II: 4, Depth: 4.
```

Observe the continued read and writes over a few iterations:

```
Iteration with i=0
I1:     r = mem[0];           // line 7
I2:     m = r * a , mem[1]=m; // line 8
Iteration with i=1
I1:     r = mem[1];           // line 7
I2:     m = r * a , mem[2]=m; // line 8
Iteration with i=2
I1:     r = mem[2];           // line 7
I2:     m = r * a , mem[3]=m; // line 8
```

A longer II is needed because the RAW dependence is via reading `r` from `mem[i]`, performing multiplication, and writing to `mem[i+1]`.

Removing False Dependencies to Improve Loop Pipelining

False dependencies are dependencies that arise when the compiler is too conservative. These dependencies do not exist in the real code, but cannot be determined by the compiler. These dependencies can prevent loop pipelining.

The following example illustrates false dependencies. In this example, the read and write accesses are to two different addresses in the same loop iteration. Both of these addresses are dependent on the input data, and can point to any individual element of the `hist` array. Because of this, Vitis HLS assumes that both of these accesses can access the same location. As a result, it schedules the read and write operations to the array in alternating cycles, resulting in a loop II of 2. However, the code shows that `hist[old]` and `hist[val]` can never access the same location because they are in the else branch of the conditional `if(old == val)`.

```
void histogram(int in[INPUT SIZE], int hist[VALUE SIZE]) f
  int acc = 0;
  int i, val;
  int old = in[0];
  for(i = 0; i < INPUT SIZE; i++)
  {
    #pragma HLS PIPELINE II=1
    val = in[i];
    if(old == val)
    {
        acc = acc + 1;
    }
    else
    {
        hist[old] = acc;
        acc = hist[val] + 1;
    }
```

```
    old = val;
  }

  hist[old] = acc;
```

To overcome this deficiency, you can use the DEPENDENCE directive to provide Vitis HLS with additional information about the dependencies.

```
void histogram(int in[INPUT SIZE], int hist[VALUE SIZE]) {
  int acc = 0;
  int i, val;
  int old = in[0];
  #pragma HLS DEPENDENCE variable=hist type=intra direction=RAW
dependent=false
  for(i = 0; i < INPUT SIZE; i++)
  {
    #pragma HLS PIPELINE II=1
    val = in[i];
    if(old == val)
    {
        acc = acc + 1;
    }
    else
    {
        hist[old] = acc;
        acc = hist[val] + 1;
    }

    old = val;
  }

  hist[old] = acc;
```

IMPORTANT! *Specifying a FALSE dependency, when in fact the dependency is not FALSE, can result in incorrect hardware. Be sure dependencies are correct (TRUE or FALSE) before specifying them.*

When specifying dependencies there are two main types:

- Inter - Specifies the dependency is between different iterations of the same loop.If this is specified as FALSE it allows Vitis HLS to perform operations in parallel if the pipelined or loop is unrolled or partially unrolled and prevents such concurrent operation when specified as TRUE.
- Intra - Specifies dependence within the same iteration of a loop, for example an array being accessed at the start and end of the same iteration. When intra dependencies are specified as FALSE, Vitis HLS can move operations freely within the loop, increasing their mobility and potentially improving performance or area. When the dependency is specified as TRUE, the operations must be performed in the order specified.

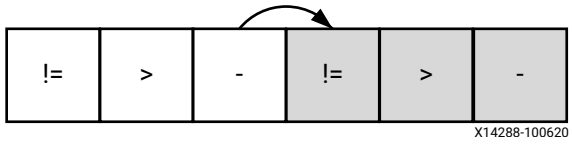
Scalar Dependencies

Some scalar dependencies are much harder to resolve and often require changes to the source code. A scalar data dependency could look like the following:

```
while (a != b) {
   if (a > b) a -= b;
   else b -= a;
 }
```

The next iteration of this loop cannot start until the current iteration has calculated the updated the values of `a` and `b`, as shown in the following figure.

Figure 19: **Scalar Dependency**



If the result of the previous loop iteration must be available before the current iteration can begin, loop pipelining is not possible. If Vitis HLS cannot pipeline with the specified initiation interval, it increases the initiation internal. If it cannot pipeline at all, as shown by the above example, it halts pipelining and proceeds to output a non-pipelined design.

Unrolling Loops

A loop is executed for the number of iterations specified by the loop induction variable. The number of iterations might also be impacted by logic inside the loop body (for example, `break` conditions or modifications to a loop exit variable). You can unroll loops to create multiple copies of the loop body in the RTL design, which allows some or all loop iterations to occur in parallel. Using the UNROLL pragma you can unroll loops to increase data access and throughput.

By default, HLS loops are kept rolled. This means that each iteration of the loop uses the same hardware. Unrolling the loop means that each iteration of the loop has its own hardware to perform the loop function. This means that the performance for unrolled loops can be significantly better than for rolled loops. However, the added performance comes at the expense of added area and resource utilization.

Consider the basic_loops_primer example from GitHub, as shown below:

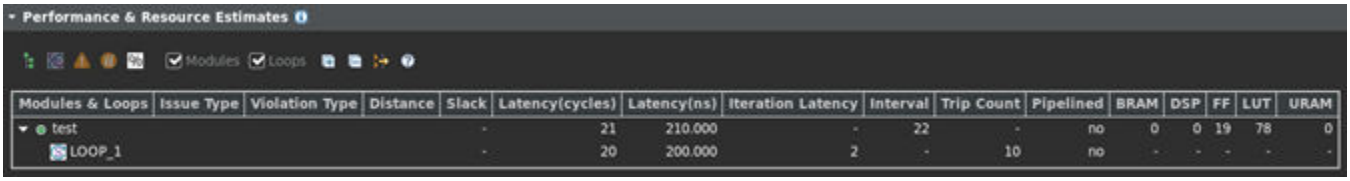
```
#include "test.h"

dout_t test(din_t A[N]) {
  dout_t out_accum=0;
  dsel_t x;

  LOOP_1:for (x=0; x<N; x++) {
      out_accum += A[x];
  }
  return out_accum;
}
```

With no optimization, the Synthesis Summary report in the figure below shows that the implementation is sequential. This can be confirmed by looking at the trip count for LOOP_1, which reports the number of iterations as 10 and the Latency as 200. The latency is the time before the loop can accept new input values.

Figure 20: **Performance & Resource Estimates**



Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
test			-		21	210.000	-	22	-	no	0	0	19	78	0
LOOP_1			-		20	200.000	2	-	10	no	-	-	-	-	-

To get optimal throughput, the latency needs to be as short as possible. To increase performance, assuming the loop bounds are static, the loop can be fully unrolled using the UNROLL pragma to create parallel implementations of the loop body. After the LOOP_1 is fully unrolled a significant reduction in the latency (50 ns) is shown in the figure below. Unrolling loops implies a trade-off by achieving higher performance but at the cost of using extra resources (as seen below in the increase of FFs and LUTs). Fully unrolling the loop will also cause the loop itself to disappear and be replaced by the parallel implementations of the loop body which will use up the extra resources as shown below.

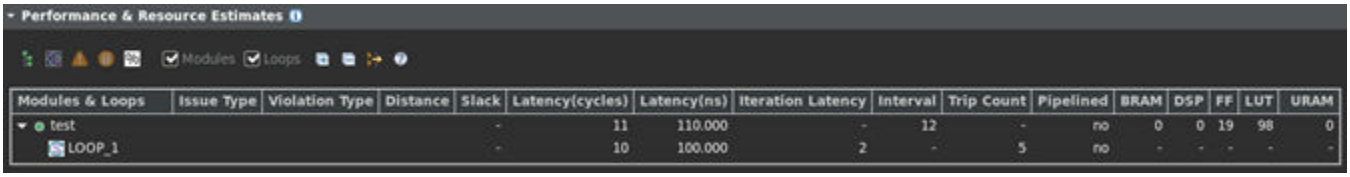
Figure 21: **Performance & Resource Estimates**



Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
test			-		5	50.000	-	6	-	no	0	0	51	261	0

Of course, there will be cases where it is not possible to unroll the loop completely due to the increase in resources and the available resources of the platform. In this situation, partially unrolled loops can be the preferred solution offering some improvement of performance while not requiring as many resources. To partially unroll a loop you will define an unroll factor for the pragma or directive. Unrolling the same loop with a factor of 2 (which implies that the loop body is duplicated and the trip count is reduced by half to 5) can be an acceptable solution for this constrained case as shown below.

Figure 22: **Performance & Resource Estimates**



Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
test			-	-	11	110.000	-	12	-	no	0	0	19	98	0
LOOP_1			-	-	10	100.000	2	-	5	no	-	-	-	-	-

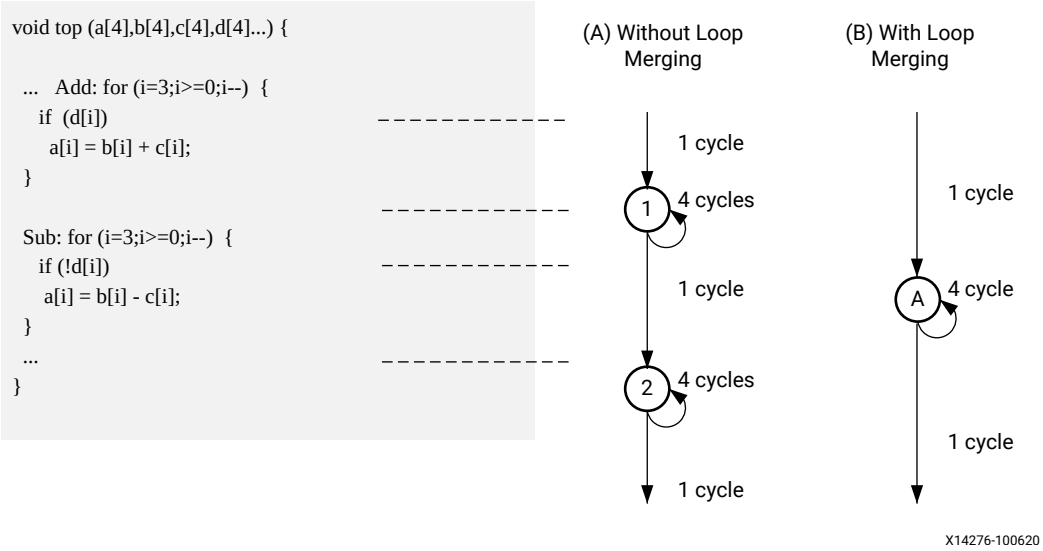
Additionally, when you partially unroll the loop, the HLS tool will implement an exit check in the loop in case the trip count is not perfectly divisible by the unroll factor. The exit check is skipped if the trip count is perfectly divisible by the unroll factor.

Merging Loops

All rolled loops imply and create at least one state in the design FSM. When there are multiple sequential loops it can create additional unnecessary clock cycles and prevent further optimizations.

The following figure shows a simple example where a seemingly intuitive coding style has a negative impact on the performance of the RTL design.

Figure 23: **Loop Directives**



In the preceding figure, (A) shows how, by default, each rolled loop in the design creates at least one state in the FSM. Moving between those states costs clock cycles: assuming each loop iteration requires one clock cycle, it takes a total of 11 cycles to execute both loops:

- 1 clock cycle to enter the ADD loop.
- 4 clock cycles to execute the add loop.
- 1 clock cycle to exit ADD and enter SUB.
- 4 clock cycles to execute the SUB loop.
- 1 clock cycle to exit the SUB loop.
- For a total of 11 clock cycles.

In this simple example, it is obvious that an else branch in the ADD loop would also solve the issue but in a more complex example it may be less obvious and the more intuitive coding style may have greater advantages.

The LOOP_MERGE optimization directive is used to automatically merge loops. The loop merge optimization directive will seek to merge all loops within the scope it is placed. In the above example, merging the loops creates a control structure similar to that shown in (B) in the preceding figure, which requires only 6 clocks to complete.

Merging loops allows the logic within the loops to be optimized together. The loop merging transformation has limitations and may not always succeed. However, it is still possible to manually merge the loops by refactoring the code. In the example above, using a dual-port block RAM allows the add and subtraction operations to be performed in parallel.

Working with Nested Loops

To get the best performance (lowest latency) when working with nested loops, it becomes crucial to create perfectly nested loops. In a perfect nested loop, the loop bounds are constant and only the innermost loop contains any functionality (as shown below)

```
Perfect_nested_loop_1: for (int i = 0; i < N; ++i) {
    Perfect_nested_loop_2: for (int j = 0; j < M; ++j) {
        // Perfect Nested Loop Code goes here and no where else
    }
}

Imperfect_nested_loop_1: for (int i = 0; i < N; ++i) {
    // Imperfect Nested Loop Code contains code here
    Imperfect_nested_loop_2: for (int j = 0; j < M; ++j) {
        // Imperfect Nested Loop Code goes here
    }
    // Imperfect Nested Loop Code may contain code here as well
}
```

- **Perfect loop nest:** Only the innermost loop has loop body content, there is no logic specified between the loop statements and all the loop bounds are constant
- **Semi-perfect loop nest:** Only the innermost loop has loop body content, there is no logic specified between the loop statements but the outermost loop bound can be a variable.
- **Imperfect loop nest:** The inner loop has variable bounds or the loop body is not exclusively inside the inner loop. In this case designers should try to restructure the code or unroll the loops in the loop body to create a perfect loop nest.

It also requires additional clock cycles to move between rolled nested loops. It requires one clock cycle to move from an outer loop to an inner loop or from an inner loop to an outer loop. In the small example shown here, this implies 200 extra clock cycles to execute the loop `Outer`.

```
void foo_top { a, b, c, d} {
    ...
    Outer: while(j<100)
        Inner: while(i<6) // 1 cycle to enter inner
            ...
            LOOP_BODY
            ...
        } // 1 cycle to exit inner
    }
 ...
}
```

The LOOP_FLATTEN pragma or directive is used to allow labeled perfect and semi-perfect nested loops to be flattened, removing the need to re-code for optimal hardware performance and reducing the number of cycles it takes to perform the operations in the loop. When the LOOP_FLATTEN optimization is applied to a set of nested loops, it should be applied to the innermost loop that contains the loop body. Loop flattening can also be performed either by applying it to individual loops or applying it to all loops in a function by applying the directive at the function level.

When pipelining nested loops, the optimal balance between area and performance is typically found by pipelining the innermost loop. This also results in the fastest runtime. The following code example, pipelined_loop available on GitHub, demonstrates the trade-offs when pipelining loops and functions.

```
#include "loop_pipeline.h"

dout_t loop_pipeline(din_t A[N]) {
    int i,j;
    static dout_t acc;

    LOOP_I:for(i=0; i < 20; i++){
        LOOP_J: for(j=0; j < 20; j++){
            acc += A[j] * i;
        }
    }
    return acc;
}
```

In the above example, if the innermost (`LOOP_J`) is pipelined, there is one copy of `LOOP_J` in hardware (a single multiplier). Vitis HLS automatically flattens the loops when possible, as in this case, and effectively creates a new single loop (now called `LOOP_I_LOOP_J`) with 20*20 iterations. Only one multiplier operation and one array access need to be scheduled, then the loop iterations can be scheduled as a single loop-body entity (20x20 loop iterations).



TIP: *When a loop or function is pipelined, any loop in the hierarchy below the loop or function being pipelined must be unrolled.*

If the outer loop (`LOOP_I`) is pipelined, inner-loop (`LOOP_J`) is unrolled creating 20 copies of the loop body: 20 multipliers and 1 array accesses must now be scheduled. Then each iteration of `LOOP_I` can be scheduled as a single entity.

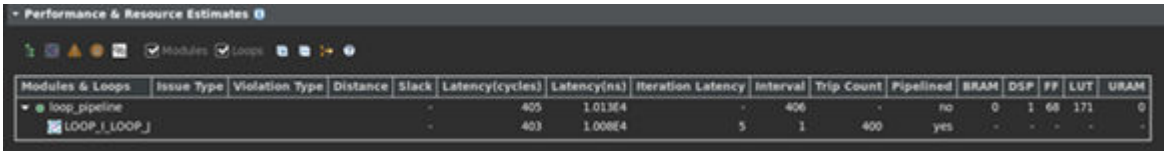
If the top-level function is pipelined, both loops must be unrolled: 400 multipliers and 20 array accesses must now be scheduled. It is very unlikely that Vitis HLS will produce a design with 400 multiplications because, in most designs, data dependencies often prevent maximal parallelism, for example, even if a dual-port RAM is used for `A`, the design can only access two values of `A` in any clock cycle. Otherwise, the array must be partitioned into 400 registers, which then can all be read in one clock cycle, with a very significant HW cost.

The concept to appreciate when selecting at which level of the hierarchy to pipeline is to understand that pipelining the innermost loop gives the smallest hardware with generally acceptable throughput for most applications. Pipelining the upper levels of the hierarchy unrolls all sub-loops and can create many more operations to schedule (which could impact compile time and memory capacity), but typically gives the highest performance design in terms of throughput and latency. The data access bandwidth must be matched to the requirements of the operations that are expected to be executed in parallel. This implies that you might need to partition array A in order to make this work.

To summarize the above options:

- Pipeline `LOOP_J`: Latency is approximately 400 cycles (20x20) and requires less than 250 LUTs and registers (the I/O control and FSM are always present).

Figure 24: **Performance & Resource Estimates**



- Pipeline `LOOP_I`: Latency is 13 cycles but requires a few hundred LUTs and registers. About twice the logic as the first option, minus any logic optimizations that can be made.

Figure 25: **Performance & Resource Estimates**



- Pipeline function `loop_pipeline`: Latency is now only 3 cycles (due to 20 parallel register accesses) but requires almost twice the logic as the second option (and about 4 times the logic of the first option), minus any optimizations that can be made.

Figure 26: **Performance & Resource Estimates**



Vitis HLS cannot flatten imperfect loop nests. This will result in additional clock cycles to enter and exit the loops. When the design contains nested loops, analyze the results to ensure that as many nested loops as possible have been flattened: review the log file or look in the synthesis report for cases (as shown above) where the loop labels have been merged (`LOOP_I` and `LOOP_J` are now reported as `LOOP_I_LOOP_J`).

Working with Variable Loop Bounds

Some of the optimizations that Vitis HLS can apply are prevented when the loop has variable bounds. In the following code example, variable_bound_loops on GitHub, the loop bounds are determined by the variable `width`, which is driven from a top-level input. In this case, the loop is considered to have a variable bound, because Vitis HLS cannot know when the loop will complete.

```
#include "ap_int.h"
#define N 32

typedef ap_int<8> din_t;
typedef ap_int<13> dout_t;
typedef ap_uint<5> dsel_t;

dout_t code028(din_t A[N], dsel_t width) {

 dout_t out_accum=0;
 dsel_t x;

 LOOP_X:for (x=0;x<width; x++) {
 out_accum += A[x];
 }

 return out_accum;
}
```

Attempting to optimize the design in the example above reveals the issues created by variable loop bounds. The first issue with variable loop bounds is that they prevent Vitis HLS from determining the latency of the loop. Vitis HLS can determine the latency to complete one iteration of the loop, but because it cannot statically determine the exact variable `width`, it does not know how many iterations are performed and thus cannot report the loop latency (the number of cycles to completely execute all iterations of the loop).

When variable loop bounds are present, Vitis HLS reports the latency as a question mark (`?`) instead of using exact values. The following shows the result after the synthesis of the previous example:

```
+ Summary of overall latency (clock cycles):
 * Best-case latency:    ?
 * Worst-case latency:   ?
+ Summary of loop latency (clock cycles):
 + LOOP_X:
 * Trip count: ?
 * Latency:    ?
```

The way to overcome this issue is to use the LOOP_TRIPCOUNT pragma or directive to specify a minimum and/or maximum iteration count for the loop. The tripcount is the number of loop iterations. If a maximum tripcount of 32 is applied to `LOOP_X` in the first example, the report is updated to the following:

```
+ Summary of overall latency (clock cycles):
 * Best-case latency:    2
 * Worst-case latency:   34
+ Summary of loop latency (clock cycles):
 + LOOP_X:
 * Trip count: 0 ~ 32
 * Latency:    0 ~ 32
```

The user-provided values for the LOOP_TRIPCOUNT directive are used only for reporting, or to support the PERFORMANCE pragma or directive. The specified tripcount value allows Vitis HLS to determine latency values in the report, allowing values from different solutions to be compared. To have this same loop-bound information used for synthesis, the C/C++ code must be updated by using asserts, which impact synthesis (however, they must be used carefully since the assert condition is assumed to be true).

The next steps in optimizing the first example for a lower initiation interval are:

- Unroll the loop and allow the accumulations to occur in parallel.
- Partition the array input, or the parallel accumulations are limited by a single memory port.

If these code transformations are applied, the output from Vitis HLS highlights the most significant issue with variable bound loops:

```
WARNING: [HLS 200-936] Cannot unroll loop 'LOOP_X' (loop_var.cpp:22) in
function 'loop_var': cannot completely unroll a loop with a variable trip
count.
```

Because variable bounds loops cannot be fully unrolled, they not only prevent the unroll directive from being applied, they also prevent pipelining the levels above the loop.

IMPORTANT! *When a loop or function is pipelined, Vitis HLS unrolls all loops in the hierarchy below the function or loop. If there is a loop with variable bounds in this hierarchy, it prevents pipelining.*

The solution to loops with variable bounds is to make the number of loop iteration a fixed value with conditional executions inside the loop. The code from the variable loop bounds example can be rewritten as shown in the following code example. Here, the loop bounds are explicitly set to the maximum value of variable `width` and the loop body is conditionally executed:

```
#include "ap_int.h"
#define N 32

typedef ap_int<8> din_t;
typedef ap_int<13> dout_t;
typedef ap_uint<5> dsel_t;

dout_t loop_max_bounds(din_t A[N], dsel_t width) {
```

```
 dout_t out_accum=0;
 dsel_t x;

 LOOP_X:for (x=0; x<N; x++) {
 if (x<width) {
  out_accum += A[x];
 }
 }

 return out_accum;
}
```

The for-loop (`LOOP_X`) in the example above can be fully unrolled. Because the loop has fixed upper bounds, Vitis HLS knows how much hardware to create. There are `N(32)` copies of the loop body in the RTL design. Each copy of the loop body has conditional logic associated with it and is executed depending on the value of variable `width`. Refer to Vitis-HLS-Introductory-Examples/Modeling/variable_bound_loops for an example.

Arrays Primer

Mapping Software Arrays to Hardware Memory

Arrays are a fundamental data structure in any C++ software program. Software programmers view arrays as simply a container and allocate/deallocate arrays on demand - often dynamically. This type of dynamic memory allocation for arrays is not supported when the same program needs to be synthesized for hardware. For synthesizing arrays to hardware, knowing the exact amount of memory (statically) required for your algorithm becomes necessary. In addition, the memory architecture on FPGAs (also called "local memory") has very different trade-offs when compared to global memory which is often the DDR memory or HBM memory banks. Access to global memory has high latency costs and can take many cycles while access to local memory is often quick and only takes one or more cycles.

When an HLS design has been suitably pipelined and/or unrolled, the memory access pattern becomes established. The HLS compiler allows users to map arrays to various types of resources - where the array elements are available in parallel with or without handshaking signals. Both internal arrays and arrays in the top-level function's interface can be mapped to registers or memories. If the array is in the top-level interface, the tool automatically creates the address, data, and control signals required to interface to external memory. If the array is internal to the design, the tool not only creates the necessary address, data, and control signals to access the memory but also instantiates the memory model (which is then inferred as memory by the downstream RTL synthesis tool).

Arrays are typically implemented as memory (RAM, ROM, or shift registers) after synthesis. Arrays can also be fully partitioned into individual registers to create a fully parallel implementation provided the platform has enough registers to support this step. The initialization_and_reset example available on GitHub demonstrates different implementations of memory.

Arrays on the top-level function interface are synthesized as RTL ports that access external memory. Internal to the design, smaller arrays will be synthesized as a shift register. Larger arrays will be synthesized into block RAM (BRAM), LUTRAM, or UltraRAM (URAM) depending on the optimization settings (*see BIND_STORAGE directive/pragma*).

Consider the following example in which the HLS compiler infers a shift register when encountering the following code:

```
int A[N]; // This will be replaced by a shift register

for(...) {
  // The loop below is the shift operation
  for (int i = 0; i < N-1; ++i)
    A[i] = A[i+1];
  A[N-1] = ...;

  // This is an access to the shift register
  ... A[x] ...
}
```

Shift registers can perform a one-shift operation per cycle, and also allows random read access per cycle anywhere in the shift register, and thus is more flexible than a FIFO.

Cases in which arrays can create issues in the RTL include:

- When implemented as a memory (BRAM/LUTRAM/URAM), the number of memory ports can limit access to the data leading to II violations in pipelined loops
- Mutually exclusive accesses might not be correctly inferred by the HLS compiler
- Some care must be taken to ensure arrays that only require read accesses are implemented as ROMs in the RTL

The HLS compiler supports arrays of pointers. Each pointer can point only to a scalar or an array of scalars.



TIP: *Arrays must be sized. This is required even for function arguments (the size is ignored by the C++ compiler, but it is used by the tool), for example:* `Array[10];`. *However, unsized arrays are not supported, for example:* `Array[];`.

Array Accesses and Performance

In a previous section, optimization concepts such as loop unrolling and pipelining were introduced as a means for exploring parallelism. However, this was done without considering how array access patterns may prevent such optimizations when the arrays are mapped to memories instead of registers. Arrays mapped to memories can become the bottleneck in a design's performance. Vitis HLS provides a number of optimizations, such as array reshaping and array partitioning, that can remove these memory bottlenecks. Whenever possible, these automatic memory optimizations should be used, minimizing the number of code modifications. However, there may be situations where explicitly coding the memory architecture is either required to meet performance or may allow designers to achieve an even better quality of results. In these cases, it is essential that array accesses are coded in such a way as to not limit performance. This means analyzing array access patterns and organizing the

memories in a design so that the desired throughput and area can be achieved. The following code example shows a case in which access to an array can limit performance in the final RTL design. In this example, there are three accesses to the array `mem[N]` to create a summed result. Refer to `Vitis-HLS-Introductory-Examples/Interface/Memory/memory_bottleneck` for the full version of this example.

```
#include "array_mem_bottleneck.h"

dout_t array_mem_bottleneck(din_t mem[N]) {

 dout_t sum=0;
 int i;

 SUM_LOOP:for(i=2;i<N;++i)
   sum += mem[i] + mem[i-1] + mem[i-2];

 return sum;
}
```

During synthesis, the array is implemented as a RAM. If the RAM is specified as a single-port RAM it is impossible to pipeline loop `SUM_LOOP` to process a new loop iteration every clock cycle.

Trying to pipeline `SUM_LOOP` with an initiation interval of 1 results in the following message (after failing to achieve a throughput of 1, Vitis HLS relaxes the constraint):

```
INFO: [SCHED 61] Pipelining loop 'SUM_LOOP'.
WARNING: [SCHED 69] Unable to schedule 'load' operation ('mem_load_2',
bottleneck.c:62) on array 'mem' due to limited memory ports.
INFO: [SCHED 61] Pipelining result: Target II: 1, Final II: 2, Depth: 3.
```

The issue here is that the single-port RAM has only a single data port: only one read (or one write) can be performed in each clock cycle.

- SUM_LOOP Cycle1: read `mem[i]`;
- SUM_LOOP Cycle2: read `mem[i-1]`, sum values;
- SUM_LOOP Cycle3: read `mem[i-2]`, sum values;

A dual-port RAM could be used, but this allows only two accesses per clock cycle. Three reads are required to calculate the value of sum, and so three accesses per clock cycle are required to pipeline the loop with an new iteration every clock cycle.

The code in the example above can be rewritten as shown in the following code example to allow the code to be pipelined with a throughput of 1. In the following code example, by performing pre-reads and manually pipelining the data accesses, there is only one array read specified in each iteration of the loop. This ensures that only a single-port RAM is required to achieve the performance.

```
#include "array_mem_perform.h"

dout_t array_mem_perform(din_t mem[N]) {

 din_t tmp0, tmp1, tmp2;
 dout_t sum=0;
 int i;

 tmp0 = mem[0];
 tmp1 = mem[1];
 SUM_LOOP:for (i = 2; i < N; i++) {
 tmp2 = mem[i];
 sum += tmp2 + tmp1 + tmp0;
 tmp0 = tmp1;
 tmp1 = tmp2;
 }

 return sum;
}
```

Such changes to the source code as shown above are not always required. The more typical case is to use optimization directives/pragmas to achieve the same result. Vitis HLS includes optimization directives for changing how arrays are implemented and accessed. There are two main classes of optimization:

- Array Partition splits apart the original array into smaller arrays or into individual registers.
- Array Reshape reorganizes the array into a different memory arrangement to increase parallelism but without splitting apart the original array.

Array Partitioning

Arrays can be partitioned into blocks or into their individual elements. In some cases, Vitis HLS partitions arrays into individual elements. This is controllable using the configuration settings for auto-partitioning. When an array is partitioned into multiple blocks, the single array is implemented as multiple RTL RAM blocks. When partitioned into elements, each element is implemented as a register in the RTL. In both cases, partitioning allows more elements to be accessed in parallel and can help with performance; the design trade-off is between performance and the number of RAMs or registers required to achieve it.

A common issue when pipelining functions is the following message:

```
INFO: [SCHED 204-61] Pipelining loop 'SUM_LOOP'.
WARNING: [SCHED 204-69] Unable to schedule 'load' operation ('mem_load_2',
bottleneck.c:62) on array 'mem' due to limited memory ports.
WARNING: [SCHED 204-69] The resource limit of core:RAM:mem:p0 is 1, current
assignments:
```

```
WARNING: [SCHED 204-69]     'load' operation ('mem_load', bottleneck.c:62)
on array
'mem',
WARNING: [SCHED 204-69] The resource limit of core:RAM:mem:p1 is 1, current
assignments:
WARNING: [SCHED 204-69]     'load' operation ('mem_load_1',
bottleneck.c:62) on array
'mem',
INFO: [SCHED 204-61] Pipelining result: Target II: 1, Final II: 2, Depth: 3.
```

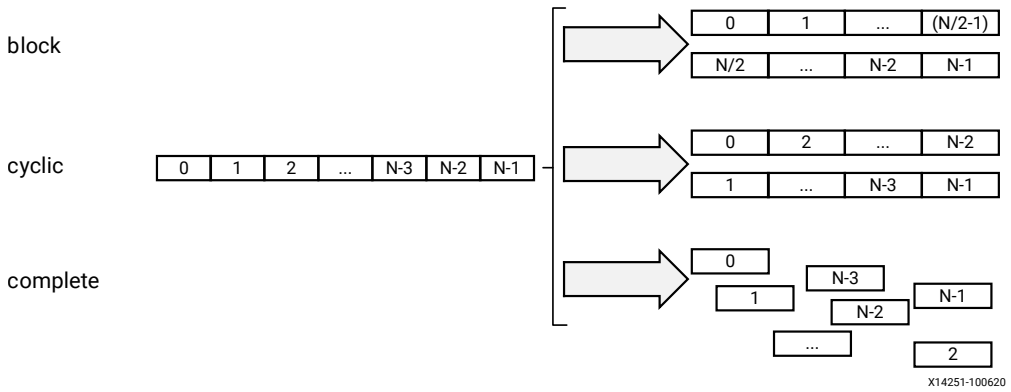
In this example, Vitis HLS states it cannot reach the specified initiation interval (II) of 1 because it cannot schedule a `load` (read) operation (`mem_load_2`) onto the memory because of limited memory ports. The above message notes that the resource limit for "`core:RAM:mem:p0 is 1`" which is used by the operation `mem_load` on line 62. The second port of the block RAM also only has 1 resource, which is also used by operation `mem_load_1`. Due to this memory port contention, Vitis HLS reports a final II of 2 instead of the desired 1.

This issue is typically caused by arrays. Arrays that are not interfaces to the top-level function are implemented as block RAM which has a maximum of two data ports. This can limit the throughput of a read/write (or load/store) intensive algorithm. The bandwidth can be improved by splitting the array (a single block RAM resource) into multiple smaller arrays (multiple block RAMs), effectively increasing the number of ports.

Arrays are partitioned using the ARRAY_PARTITION directive. Vitis HLS provides three types of array partitioning, as shown in the following figure. The three styles of partitioning are:

- **`block`:** The original array is split into equally sized blocks of consecutive elements of the original array.
- **`cyclic`:** The original array is split into equally sized blocks interleaving the elements of the original array.
- **`complete`:** The default operation is to split the array into its individual elements. This corresponds to resolving a memory into registers.

Figure 27: **Array Partitioning**



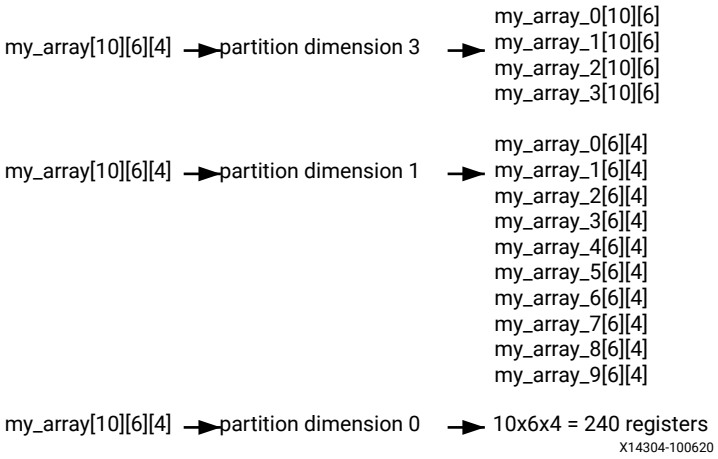
For `block` and `cyclic` partitioning the `factor` option specifies the number of arrays that are created. In the preceding figure, a factor of 2 is used, that is, the array is divided into two smaller arrays. If the number of elements in the array is not an integer multiple of the factor, the final array has fewer elements.

When partitioning multi-dimensional arrays, the `dimension` option is used to specify which dimension is partitioned. The following figure shows how the `dimension` option is used to partition the following example code:

```
void foo (...) {
 int  my_array[10][6][4];
   ...
}
```

The examples in the figure demonstrate how partitioning `dimension` 3 results in 4 separate arrays and partitioning `dimension` 1 results in 10 separate arrays. If zero is specified as the `dimension`, all dimensions are partitioned.

Figure 28: **Partitioning Array Dimensions**



Automatic Array Partitioning

The `syn.array_partition.complete_threshold` configuration command defines how arrays are automatically partitioned based on the number of elements, as explained in Array Partition Configuration.

Array Reshaping

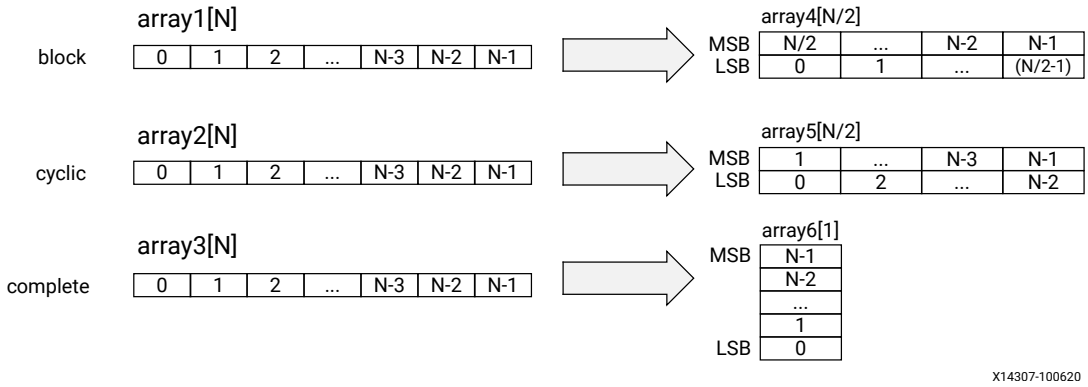
The ARRAY_RESHAPE directive reforms the array with a vertical mode of remapping, and is used to reduce the number of block RAM consumed while providing parallel access to the data.

Given the following example code:

```
void foo (...) {
int  array1[N];
int  array2[N];
int  array3[N];
#pragma HLS ARRAY_RESHAPE variable=array1 type=block factor=2 dim=1
#pragma HLS ARRAY_RESHAPE variable=array2 type=cyclic factor=2 dim=1
#pragma HLS ARRAY_RESHAPE variable=array3 type=complete dim=1
...
}
```

The ARRAY_RESHAPE directive transforms the arrays into the form shown in the following figure.

Figure 29: **Array Reshaping**



The ARRAY_RESHAPE directive allows more data to be accessed in a single clock cycle. In cases where more data can be accessed in a single clock cycle, Vitis HLS might automatically unroll any loops consuming this data, if doing so will improve the throughput. The loop can be fully or partially unrolled to create enough hardware to consume the additional data in a single clock cycle. This feature is controlled using the `syn.unroll.tripcount_threshold` configuration command as described Unroll Setting.

In the following example, any loops with a tripcount of less than 16 will be automatically unrolled if doing so improves the throughput.

```
syn.unroll.tripcount_threshold=16
```

Arrays on the Interface

When you use an array as an argument to the top-level function, Vitis HLS assumes one of the following:

- Memory is off-chip and Vitis HLS synthesizes M_AXI ports on the interface to access the memory as the default for the Vitis Kernel flow.
- Memory is standard block RAM with a latency of 1 as the default behavior in the Vivado IP flow. The data is ready one clock cycle after the address is supplied.

To configure how Vitis HLS creates these ports:

- Specify the interface as a M_AXI, BRAM, or FIFO interface using the INTERFACE pragma or directive.
- Specify the RAM as a single or dual-port RAM using the `storage_type` option of the INTERFACE pragma or directive.
- Specify the RAM latency using the `latency` option of the INTERFACE pragma or directive.
- Use array optimization directives, ARRAY_PARTITION, or ARRAY_RESHAPE, to reconfigure the structure of the array and therefore, the number of I/O ports.



TIP: *Because access to the data is limited through a memory (RAM or FIFO) port, arrays on the interface can create a performance bottleneck. Typically, you can overcome these bottlenecks using optimization directives.*

Arrays must be sized when used in synthesizable code. If, for example, the declaration `d_i[4]` in Array Interfaces is changed to `d_i[]`, Vitis HLS issues a message that the design cannot be synthesized:

```
@E [SYNCHK-61] array_RAM.c:52: unsupported memory access on variable 'd_i'
which is (or contains) an array with unknown size at compile time.
```

Array Interfaces

The INTERFACE pragma or directive lets you explicitly define which type of RAM or ROM is used with the `storage_type=<value>` option. This defines which ports are created (single-port or dual-port). If no `storage_type` is specified the tool uses:

- A single-port RAM by default.
- A dual-port RAM if it reduces the initiation interval or reduces latency.

The ARRAY_PARTITION and ARRAY_RESHAPE pragmas can re-configure arrays on the interface. Arrays can be partitioned into multiple smaller arrays, each implemented with its own interface. This includes the ability to completely partition the array into a set of scalars. On the function interface, this results in a unique port for every element in the array. This provides maximum parallel access, but creates many more ports and might introduce routing issues during hardware implementation.

By default, the array arguments in the function shown in the following code example are synthesized into a single-port RAM interface.

```
#include "array_RAM.h"

void array_RAM (dout_t d_o[4], din_t d_i[4], didx_t idx[4]) {
 int i;

 For_Loop: for (i=0;i<4;i++) {
 d_o[i] = d_i[idx[i]];
 }

}
```

A single-port RAM interface is used because the `for-loop` ensures that only one element can be read and written in each clock cycle. There is no advantage in using a dual-port RAM interface. If the for-loop is unrolled, Vitis HLS uses a dual-port RAM. Doing so allows multiple elements to be read at the same time and improves the initiation interval. The type of RAM interface can be explicitly set by applying the INTERFACE pragma or directive, and setting the `storage_type`.

Issues related to arrays on the interface are typically related to throughput. For example, if the arrays in the example above are partitioned into individual elements, and the `for-loop` is unrolled, all four elements in each array are accessed simultaneously.

You can also use the INTERFACE pragma or directive to specify the latency of the RAM, using the `latency=<value>` option. This lets the tool model external SRAMs with a latency greater than 1 at the interface.

FIFO Interfaces

Vitis HLS allows array arguments to be implemented as FIFO ports in the RTL. If a FIFO ports is to be used, you must ensure that the accesses to and from the array are sequential.

Note: If the accesses at the interface are not sequential, there is an RTL simulation mismatch.

The following code example shows a case in which the tool cannot determine whether the accesses are sequential. In this example, both `d_i` and `d_o` are specified to be implemented with a FIFO interface during synthesis. In this case, you must ensure the access is sequential or you will be introducing errors into your system.

```
#include "array_FIFO.h"

void array_FIFO (dout_t d_o[4], din_t d_i[4], didx_t idx[4]) {
 int i;
#pragma HLS INTERFACE mode=ap_fifo port=d_i
#pragma HLS INTERFACE mode=ap_fifo port=d_o
 For_Loop: for (i=0;i<4;i++) {
 d_o[i] = d_i[idx[i]];
 }
}
```

In this case, the values of variable `idx` would determine whether or not a FIFO interface can be successfully created for argument `d_i[]`.

- If the values of the elements of `idx` are sequential, then a FIFO interface could be created
- If random values are used for `idx`, a FIFO interface fails in Co-simulation when implemented in RTL, and can also fail during runtime

However, because these conditions cannot be validated at compile time, Vitis HLS issues a message during synthesis and creates a FIFO interface:

```
@W [XFORM-124] Array 'd_i': may have improper streaming access(es).
```

In addition, the `idx` array is never read, because its elements are assumed to have sequential values starting from 0, due to the presence of the `ap_fifo` INTERFACE pragma.

***Note*:** FIFO ports cannot be synthesized for arrays that are both read from and written to in the same loop or function. Separate input and output arrays (as in the example above) must be created.

The following general rules apply to arrays that are implemented with a FIFO interface:

- The array must be only read or written in the loop or function. This can be transformed into a point-to-point connection that matches the characteristics of FIFO links.
- The array reads must be in the same order as the array writes. Because random access is not supported for FIFO channels, the array must be used in the program following first in, first out semantics.

The following conditions apply when the data type of an array is a struct, and the array is sequentially accessed (i.e. the array is specified with the `axis` or `ap_fifo` interface or is marked with STREAM pragma or directive):

- You cannot access struct members directly from I/O arguments that use array-to-stream, or are as streaming interfaces. You can make a local copy of the struct to read/write member elements.
- You must ensure sequential order access, as shown below

```
struct A {
  short foo;
  int bar;
};

void dut(A in[N], A out[out], bool flag) {
  #pragma HLS interface ap_fifo port=in,out
  for (unsigned i=0; i<N; i++) {
    A tmp = in[i];
    if (flag)
      tmp.bar += 5;
    out[i] = tmp;
  }
}

Bad example 1:
void dut(A in[N], A out[out], bool flag) {
```

```
  #pragma HLS interface ap_fifo port=in,out
  for (unsigned i=0; i<N; i++) {
    out[i] = in[i];
    if (flag)
      out[i].bar += 5;
  }
}

Bad example 2:
void dut(A in[N], A out[out], bool flag) {
  #pragma HLS interface ap_fifo port=in,out
  for (unsigned i=0; i<N; i++) {
    out[i].foo = in[i].foo;
    if (flag)
      out[i].bar = in[i].bar + 5;
    else
      out[i].bar = in[i].bar;
  }
}
```

Memory Mapped Interfaces

As mentioned earlier, Vitis HLS allows the user to specify M_AXI interfaces for arrays in the interface. Since this memory is off-chip and not local, access to these memories can be expensive in terms of cycles. To optimize how these accesses are made, Vitis HLS performs an automatic *burst* optimization, to efficiently read/write to these external memories. Bursting is an optimization that tries to intelligently aggregate the memory accesses to the DDR to maximize the throughput bandwidth and/or minimize the latency. Bursting is one of many possible optimizations to the kernel. Bursting typically gives you a 4-5x improvement. Bursting is useful when you have contention on the DDR ports from multiple competing kernels.

For more information on this bursting optimization and details on how to write code to infer more bursts, please review the Optimizing AXI System Performance section.

Initializing and Resetting Arrays

RECOMMENDED: *Although not a requirement, AMD recommends specifying arrays that are to be implemented as memories with the* `static` *qualifier. This not only ensures that Vitis HLS implements the array with a memory in the RTL; it also allows the default initialization behavior of the static types to be used.*

In the following code, an array is initialized with a set of values. Each time the function is executed, array `coeff` is assigned these values. After synthesis, each time the design executes the RAM that implements `coeff` is loaded with these values. For a single-port RAM this would take eight clock cycles. For an array of 1024, it would of course take 1024 clock cycles, during which time no operations depending on `coeff` could occur.

```
int coeff[8] = {-2, 8, -4, 10, 14, 10, -4, 8, -2};
```

The following code uses the `static` qualifier to define array `coeff`. The array is initialized with the specified values at start of execution. Each time the function is executed, array `coeff` remembers its values from the previous execution. A static array behaves in C/C++ code as a memory does in RTL.

```
static int coeff[8] = {-2, 8, -4, 10, 14, 10, -4, 8, -2};
```

In addition, if the variable has the `static` qualifier, Vitis HLS initializes the variable in the RTL design and in the FPGA bitstream. This removes the need for multiple clock cycles to initialize the memory and ensures that initializing large memories is not an operational overhead. Refer to the initialization_and_reset example available on GitHub for examples.

The RTL configuration command `syn.rtl.reset` can specify if static variables return to their initial state after a reset is applied. This is not the default. When `syn.rtl.reset=state` or `all` are used, it forces all arrays implemented as block RAM to be returned to their initialized state after reset. This can result in two very undesirable conditions in the RTL design:

- Unlike a power-up initialization (or power-on reset), an explicit reset requires the RTL design to iterate through each address in the block RAM to set the value: this can take many clock cycles if N is large, and requires more area resources to implement the reset.
- A reset is added to every array in the design.

To prevent adding reset logic onto every such block RAM, and incurring the cycle overhead to reset all elements in the RAM, specify the default `syn.rtl.reset=control` reset mode and use the RESET pragma or directive to identify individual static or global variables to be reset.

Alternatively, you can use the `syn.rtl.reset=state` reset mode, and use the RESET directive `off` option to select which individual static or global variables to not reset.

Finally, depending on the hardware device or platform of your choice (UltraScale+ or Versal, etc), there can be differences in how block RAMs and URAMs are initialized and/or reset. In general, Vitis HLS supports two types of reset: one is when the device is powered on (and also termed as power-up initialization or power-on reset), and the second is when a hardware RESET signal is asserted during device execution. The following shows the differences in behavior for the different memory resources:

- Initialization Behavior: Applies to all block RAMs on all platforms and only to Versal URAMs. This is the behavior during power-on initialization (or power-on reset).
- Maintaining an “initial value array” and “runtime array” if the array is read/written. This applies to both block RAMs and URAMs and this corresponds to the hardware “RESET” signal during device execution.

TIP: *URAMs do not support a read-first output write_mode (unlike block RAMs) when a read and a write to the same address is mapped to the same memory port. Block RAM supports the following write-modes:* `write thru`, `read first`, `no change`. *URAM only supports* `no change`. *Vitis HLS will issue the following warning message when it cannot schedule memory operations in the same cycle on a URAM port:*

```
Usage of URAM can potentially cause worse II as Vitis HLS does not
exploit
read-first mode for URAMs. Consider using block RAMs instead.
```

Implementing ROMs

The `const` qualifier is recommended when arrays are only read, because Vitis HLS cannot always infer that a ROM should be used by analysis of the design. The general rule for the automatic inference of a ROM is that a local (non-global), `static` array is fully written to before being read, and never written again. The following practices in the code can help infer a ROM:

- Initialize the array as early as possible in the function that uses it.
- Group writes together.
- Do not interleave `array(ROM)` initialization writes with non-initialization code.
- Do not store different values to the same array element (group all writes together in the code).
- Element value computation must not depend on any non-constant (at compile-time) design variables, other than the initialization loop counter variable.

If complex assignments are used to initialize a ROM (for example, functions from the `math.h` library), placing the array initialization into a separate function allows a ROM to be inferred. In the following example, array `sin_table[256]` is inferred as a memory and implemented as a ROM after RTL synthesis.

```
#include "array_ROM_math_init.h"
#include <math.h>

void init_sin_table(din1_t sin_table[256])
{
 int i;
 for (i = 0; i < 256; i++) {
 dint_t real_val = sin(M_PI * (dint_t)(i - 128) / 256.0);
 sin_table[i] = (din1_t)(32768.0 * real_val);
 }
}

dout_t array_ROM_math_init(din1_t inval, din2_t idx)
{
 short sin_table[256];
 init_sin_table(sin_table);
 return (int)inval * (int)sin_table[idx];
}
```

TIP: *Because the `sin()` function results in constant values, no core is required in the RTL design to implement the `sin()` function.*

C Simulation with Arrays

Arrays can introduce issues during C/C++ simulation, even before the synthesis step is performed. If you specify a very large array, it might cause the C/C++ simulation to run out of memory and fail, as shown in the following example:

```
#include "ap_int.h"

  int i, acc;
  // Use an arbitrary precision type
  ap_int<32>  la0[10000000], la1[10000000];

  for (i=0 ; i < 10000000; i++) {
      acc = acc + la0[i] + la1[i];
  }
```

The simulation might fail by running out of memory, because the array is placed on the stack that exists in memory rather than the heap that is managed by the OS and can use local disk space to grow. Certain issues might make this issue more likely:

- On PCs, the available memory is often less than large Linux boxes and there might be less memory available.
- Using arbitrary precision types as shown in the example above could make this issue worse as they require more memory to model than standard C/C++ types.
- Using the more complex fixed-point arbitrary precision types found in C++ might make the issue of designs running out of memory even more likely as types require even more memory.

The standard way to improve memory resources in C/C++ code development is to increase the size of the stack using the linker options such as the following option which explicitly sets the stack size `syn.csimflags -z stack-size=10485760`.

However, the machine might not have enough available memory, and increasing the stack size will not help. In this case a solution is to use dynamic memory allocation for simulation but a fixed-sized array for synthesis, as shown in the next example. This means that the memory required for this is allocated on the heap, managed by the OS, and can use local disk space to grow.

```
#include "ap_int.h"

  int i, acc;
#ifdef __SYNTHESIS__
  // Use an arbitrary precision type & array for synthesis
  ap_int<32>  la0[10000000], la1[10000000];
#else
  // Use an arbitrary precision type & dynamic memory for simulation
```

```
 ap_int<32> *la0 = malloc(10000000  * sizeof(ap_int<32>));
 ap_int<32> *la1 = malloc(10000000  * sizeof(ap_int<32>));
#endif
  for (i=0 ; i < 10000000; i++) {
      acc = acc + la0[i] + la1[i];
  }
```

However, this is not an ideal solution because the simulated code and the synthesized code are not the same. But this might be the only way to complete simulation. If you take this approach be sure that the C/C++ test bench covers all aspects of accessing the array. The RTL simulation performed by `cosim_design` will verify that the memory accesses are correct in the synthesized code.

Note: Only use the `__SYNTHESIS__` macro on the code to be synthesized. Do *not* use this macro in the test bench, because it has no significance in the C/C++ simulation or C/C++ RTL co-simulation. Refer to `Vitis-HLS-Introductory-Examples/Pipelining/Functions/hier_func` for the full version of this example.

Chapter 5

Functions Primer

The top-level function becomes the top-level module of the RTL design after synthesis. All sub-functions that are not in-lined are synthesized into separate modules in the RTL design. Arguments of the top-level function are implemented as interface ports in the hardware as described in Interfaces of the HLS Design. Global variables used by the kernel cannot be accessed from the outside. Any variable that is accessed by both the test bench (or other compiled kernels or host) and the kernel itself should be defined as an argument of the top-level function.

IMPORTANT! *The top-level function cannot be a static function.*

After synthesis, each function in the design has its own synthesis report and HDL file (Verilog and VHDL).

Function Inlining

Function inlining removes the function hierarchy by disolving the function logic into the calling function. A function is inlined using the INLINE directive. Inlining a function can improve the area by allowing the components within the function to be better shared or optimized with the logic in the calling function. This type of function inlining is also performed automatically by Vitis HLS for small functions.

Inlining allows function sharing to be better controlled. For functions to be shared they must be used within the same level of hierarchy. In this code example, function `foo_top` calls `foo` twice and function `foo_sub`.

```
foo_sub (p, q) {
 int q1 = q + 10;
 foo(p1,q); // foo_3
 ...
}
void foo_top { a, b, c, d} {
 ...
 foo(a,b); //foo_1
 foo(a,c); //foo_2
 foo_sub(a,d);
 ...
}
```

Inlining function `foo_sub` and using the ALLOCATION directive to specify that only one instance of the function `foo` is used, results in a design that only has one instance of function `foo`: one-third the area of the example above. Using this strategy allows for more fine-grained control of what user-defined resources can be shared and is a useful feature to when area utilization is a consideration.

```
foo_sub (p, q) {
#pragma HLS INLINE
 int q1 = q + 10;
 foo(p1,q); // foo_3
 ...
}
void foo_top { a, b, c, d} {
#pragma HLS ALLOCATION instances=foo limit=1 function
 ...
 foo(a,b); //foo_1
 foo(a,c); //foo_2
 foo_sub(a,d);
 ...
}
```

The INLINE directive optionally allows all functions below the specified function to be recursively inlined by using the `recursive` option. If the `recursive` option is used on the top-level function, all function hierarchy in the design is removed.

The INLINE `off` option can optionally be applied to functions to prevent them from being inlined. This option may be used to prevent Vitis HLS from automatically inlining a function.

The INLINE directive is a powerful way to substantially modify the structure of the code without actually performing any modifications to the source code and provides a very powerful method for architectural exploration.

Function Pipelining

Function pipelining is handled similarly to loop pipelining as described in Pipelining Loops. Vitis HLS treats the function body as if it were the same as a loop body being called multiple times - except in this case, it is the function that is called multiple times and the tools pipelines the execution of these calls. So similar to loops, when a function is pipelined, all the loops in the function body and in the hierarchy below are automatically unrolled. This is a requirement for pipelining to proceed. If a loop has variable bounds and it cannot be unrolled then this will prevent the function from being pipelined.

Function Instantiation

Function instantiation is an optimization technique that has the area benefits of maintaining the function hierarchy but provides an additional powerful option: performing targeted local optimizations on specific instances of a function. This can simplify the control logic around the function call and potentially improve latency and throughput.

The FUNCTION_INSTANTIATE pragma or directive exploits the fact that some inputs to a function may be a constant value when the function is called and uses this to both simplify the surrounding control structures and produce smaller more optimized function blocks. This is best explained by example as shown in the following code.

```
char func(char inval, char incr) {
#pragma HLS INLINE OFF
#pragma HLS FUNCTION_INSTANTIATE variable=incr
 return inval + incr;
}

void top(char inval1, char inval2, char inval3,
 char *outval1, char *outval2, char *outval3)
{
 *outval1 = func(inval1,   0);
 *outval2 = func(inval2,   1);
 *outval3 = func(inval3, 100);
}
```

TIP: *The Vitis HLS tool automatically decomposes (or inlines) small functions into higher-level calling functions. Using the INLINE pragma with the `OFF` option can be used to prevent this automatic inlining.*

It is clear that function `func` has been written to perform three exclusive operations (depending on the value of `incr`). Each instance of function `func` is implemented in an identical manner. While this is great for function reuse and area optimization, it also means that the control logic inside the function must be more complex to account for the two exclusive operations. Refer to `Vitis-HLS-Introductory-Examples/Pipelining/Functions/function_instantiate` for the full version of this example.

The FUNCTION_INSTANTIATE optimization allows each instance to be independently optimized, reducing the functionality and area. After FUNCTION_INSTANTIATE optimization, the code above can effectively be transformed to have two separate functions, each optimized for different possible values of mode, as shown:

```
void func1() {
  // code segment 1
}

void func2() {
  // code segment 2
}
```

If the function is used at different levels of hierarchy such that function sharing is difficult without extensive inlining or code modifications, function instantiation can provide the best means of improving area: many small locally optimized copies are better than many large copies that cannot be shared.

Data Types

IMPORTANT! *Due to library dependencies that are only compiled for 64-bit systems, Vitis HLS does not support 32-bit builds. Due to this, usage of* `-m32` *flag is not allowed and will cause an error.*

The data types used in a C/C++ function compiled into an executable impact the accuracy of the result and the memory requirements, and can impact the performance. A 32-bit integer int data type can hold more data and therefore provide more precision than an 8-bit char type, but it requires more storage. Similarly, when the C/C++ function is to be synthesized to an RTL implementation, the types impact the precision, the area, and the performance of the RTL design. The data types used for variables determine the size of the operators required and therefore the area and performance of the RTL.

Vitis HLS supports the synthesis of all standard C/C++ types, including exact-width integer types.

- `(unsigned) char`, `(unsigned) short`, `(unsigned) int`
- `(unsigned) long`, `(unsigned) long long`
- `(unsigned) intN_t` (where `N` is 8, 16, 32, and 64, as defined in `stdint.h`)
- `float`, `double`

Exact-width integers types are useful for ensuring designs are portable across all types of system.

The C/C++ standard dictates that type `(unsigned)long` is implemented as 64 bits on 64-bit operating systems. Synthesis matches this behavior and produces different sized operators, and therefore different RTL designs, depending on the type of operating system on which Vitis HLS is run. On Windows OS, Microsoft defines type long as 32-bit, regardless of the OS.

- Use data type `(unsigned)int` or `(unsigned)int32_t` instead of type `(unsigned)long` for 32-bit.
- Use data type `(unsigned)long long` or `(unsigned)int64_t` instead of type `(unsigned)long` for 64-bit.

AMD highly recommends defining the data types for all variables in a common header file, which can be included in all source files.

- During the course of a typical Vitis HLS project, some of the data types might be refined, for example to reduce their size and allow a more efficient hardware implementation.

- One of the benefits of working at a higher level of abstraction is the ability to quickly create new design implementations. The same files typically are used in later projects but might use different (smaller or larger or more accurate) data types.

Both of these tasks are more easily achieved when the data types can be changed in a single location: the alternative is to edit multiple files.

IMPORTANT! *When using macros in header files, always use unique names. For example, if a macro named `_TYPES_H` is defined in your header file, it is likely that such a common name might be defined in other system files, and it might enable or disable some other code causing unforeseen side effects.*

TIP: *The `std::complex<long double>` data type is not supported in Vitis HLS and should not be used.*

Standard Types

The following code example shows some basic arithmetic operations being performed.

```
#include "types_standard.h"

void types_standard(din_A  inA, din_B  inB, din_C  inC, din_D  inD,
 dout_1 *out1, dout_2 *out2, dout_3 *out3, dout_4 *out4
) {

 // Basic arithmetic operations
 *out1 = inA * inB;
 *out2 = inB + inA;
 *out3 = inC / inA;
 *out4 = inD % inA;

}
```

The data types in the example above are defined in the header file `types_standard.h` shown in the following code example. They show how the following types can be used:

- Standard signed types
- Unsigned types
- Exact-width integer types (with the inclusion of header file `stdint.h`)

```
#include <stdio.h>
#include <stdint.h>

#define N 9

typedef char din_A;
typedef short din_B;
typedef int din_C;
typedef long long din_D;
```

```
typedef int dout_1;
typedef unsigned char dout_2;
typedef int32_t dout_3;
typedef int64_t dout_4;

void types_standard(din_A inA,din_B inB,din_C inC,din_D inD,dout_1

*out1,dout_2 *out2,dout_3 *out3,dout_4 *out4);
```

These different types result in the following operator and port sizes after synthesis:

- The multiplier used to calculate result `out1` is a 24-bit multiplier. An 8-bit `char` type multiplied by a 16-bit `short` type requires a 24-bit multiplier. The result is sign-extended to 32-bit to match the output port width.
- The adder used for `out2` is 8-bit. Because the output is an 8-bit `unsigned char` type, only the bottom 8-bits of `inB` (a 16-bit `short`) are added to 8-bit `char` type `inA`.
- For output `out3` (32-bit exact width type), 8-bit `char` type `inA` is sign-extended to 32-bit value and a 32-bit division operation is performed with the 32-bit (`int` type) `inC` input.
- A 64-bit modulus operation is performed using the 64-bit `long long` type `inD` and 8-bit `char` type `inA` sign-extended to 64-bit, to create a 64-bit output result `out4`.

As the result of `out1` indicates, Vitis HLS uses the smallest operator it can and extends the result to match the required output bit-width. For result `out2`, even though one of the inputs is 16-bit, an 8-bit adder can be used because only an 8-bit output is required. As the results for `out3` and `out4` show, if all bits are required, a full sized operator is synthesized.

Floats and Doubles

Vitis HLS supports `float` and `double` types for synthesis. Both data types are synthesized with IEEE-754 standard partial compliance (see *Floating-Point Operator LogiCORE IP Product Guide* (PG060)).

- Single-precision 32-bit
  - 24-bit fraction
  - 8-bit exponent
- Double-precision 64-bit
  - 53-bit fraction
  - 11-bit exponent

RECOMMENDED: *When using floating-point data types, AMD highly recommends that you review Floating-Point Design with Vivado HLS (XAPP599). Also refer to Vitis-HLS-Introductory-Examples/Modeling/using_float_and_double on GitHub for an example of using floating and double data types.*

In addition to using floats and doubles for standard arithmetic operations (such as +, -, *) floats and doubles are commonly used with the `math.h` (and `cmath.h` for C++). This section discusses support for standard operators.

The following code example shows the header file used with Standard Types updated to define the data types to be `double` and `float` types.

```
#include <stdio.h>
#include <stdint.h>
#include <math.h>

#define N 9

typedef double din_A;
typedef double din_B;
typedef double din_C;
typedef float din_D;

typedef double dout_1;
typedef double dout_2;
typedef double dout_3;
typedef float dout_4;

void types_float_double(din_A inA,din_B inB,din_C inC,din_D inD,dout_1
*out1,dout_2 *out2,dout_3 *out3,dout_4 *out4);
```

This updated header file is used with the following code example where a `sqrtf()` function is used.

```
#include "types_float_double.h"

void types_float_double(
 din_A  inA,
 din_B  inB,
 din_C  inC,
 din_D  inD,
 dout_1 *out1,
 dout_2 *out2,
 dout_3 *out3,
 dout_4 *out4
 ) {

 // Basic arithmetic & math.h sqrtf()
 *out1 = inA * inB;
 *out2 = inB + inA;
 *out3 = inC / inA;
 *out4 = sqrtf(inD);

}
```

When the example above is synthesized, it results in 64-bit double-precision multiplier, adder, and divider operators. These operators are implemented by the appropriate floating-point AMD IP catalog cores.

The square-root function used `sqrtf()` is implemented using a 32-bit single-precision floating-point core.

If the double-precision square-root function `sqrt()` was used, it would result in additional logic to cast to and from the 32-bit single-precision float types used for inD and `out4: sqrt()` is a double-precision (`double`) function, while `sqrtf()` is a single precision (`float`) function.

In C functions, be careful when mixing float and double types as float-to-double and double-to-float conversion units are inferred in the hardware.

```
float foo_f    = 3.1459;
float var_f = sqrt(foo_f);
```

The above code results in the following hardware:

```
wire(foo_t)
-> Float-to-Double Converter unit
-> Double-Precision Square Root unit
-> Double-to-Float Converter unit
-> wire (var_f)
```

Using a `sqrtf()` function:

- Removes the need for the type converters in hardware
- Saves area
- Improves timing

When synthesizing float and double types, Vitis HLS maintains the order of operations performed in the C code to ensure that the results are the same as the C simulation. Due to saturation and truncation, the following are not guaranteed to be the same in single and double precision operations:

```
        A=B*C; A=B*F;
        D=E*F; D=E*C;
        O1=A*D O2=A*D;
```

With `float` and `double` types, `O1` and `O2` are not guaranteed to be the same.



TIP: *In some cases (design dependent), optimizations such as unrolling or partial unrolling of loops, might not be able to take full advantage of parallel computations as Vitis HLS maintains the strict order of the operations when synthesizing float and double types. This restriction can be overridden using* `config_compile -unsafe_math_optimizations`.

For C++ designs, Vitis HLS provides a bit-approximate implementation of the most commonly used math functions.

Floating-Point Accumulator and MAC

Floating point accumulators (`facc`), multiply and accumulate (`fmacc`), and multiply and add (`fmadd`) can be enabled using the `syn.op` configuration commands as described in Operator Configuration. The following is an example:

```
syn.op=op:mul impl:dsp
syn.op=op:add impl:fabric latency:6
syn.op=op:fmacc precision:high
syn.op=op:hdiv latency:5
```

The tool supports different levels of precision for these operators that tradeoff between performance, area, and precision on both Versal and non-Versal devices.

- Low-precision accumulation is suitable for high-throughput low-precision floating point accumulation and multiply-accumulation, this mode is only available in non-Versal devices.
  - It uses an integer accumulator with a pre-scaler and a post-scaler (to convert input and output to single-precision or double-precision floating point).
    - It uses a 60 bit and 100 bit accumulator for single and double precision inputs respectively.
    - It can cause cosim mismatches due to insufficient precision with respect to C++ simulation
  - It can always be pipelined with an II=1 without source code changes
  - It uses approximately 3X the resources of standard-precision floating point accumulation, which achieves an II that is typically between 3 and 5, depending on clock frequency and target device.

  Using low-precision, accumulation for floats and doubles is supported with an initiation interval (II) of 1 on all devices. This means that the following code can be pipelined with an II of 1 without any additional coding:

```
float foo(float A[10], float B[10]) {
 float sum = 0.0;
 for (int i = 0; i < 10; i++) {
 sum += A[i] * B[i];
 }
 return sum;
}
```

- Standard-precision accumulation and multiply-add is suitable for most uses of floating-point, and is available on Versal and non-Versal devices.
  - It always uses a true floating-point accumulator
  - It can be pipelined with an II=1 on Versal devices.
  - It can be pipelined with an II that is typically between 3 and 5 (depending on clock frequency and target device) on non-Versal devices. The standard precision mode is more efficient on Versal devices than on non-Versal devices.

- High-precision fused multiply-add is suitable for high-precision applications and is available on Versal devices.
  - It uses one extra bit of precision
  - It always uses a single fused multiply-add, with a single rounding at the end, although it uses more resources than the unfused multiply-add
  - It can cause cosim mismatches due to the extra precision with respect to C++ simulation

Composite Data Types

HLS supports composite data types for synthesis:

- Structs
- Enumerated Types
- Unions

Structs

Structs in the code, for instance internal and global variables, are disaggregated by default. They are decomposed into separate objects for each of their member elements. The number and type of elements created are determined by the contents of the struct itself. Arrays of structs are implemented as multiple arrays, with a separate array for each member of the struct.

IMPORTANT! *Structs used as arguments to the top-level function are aggregated by default as described in Structs in the Interface.*

Alternatively, you can use the AGGREGATE pragma or directive to collect all the elements of a struct into a single wide vector. This allows all members of the struct to be read and written to simultaneously. The aggregated struct will be padded as needed to align the elements on a 4-byte boundary, as discussed in Struct Padding and Alignment. The member elements of the struct are placed into the vector in the order they appear in the C/C++ code: the first element of the struct is aligned on the LSB of the vector and the final element of the struct is aligned with the MSB of the vector. Any arrays in the struct are partitioned into individual array elements and placed in the vector from lowest to highest, in order.

TIP: *You should take care when using the AGGREGATE pragma on structs with large arrays. If an array has 4096 elements of type* `int`*, this will result in a vector (and port) of width 4096 * 32 = 131072 bits. While Vitis HLS can create this RTL design, it is unlikely that the Vivado tool will be able to route this during implementation.*

The single wide-vector created by using the AGGREGATE directive allows more data to be accessed in a single clock cycle. When data can be accessed in a single clock cycle, Vitis HLS automatically unrolls any loops consuming this data, if doing so improves the throughput. The loop can be fully or partially unrolled to create enough hardware to consume the additional data in a single clock cycle. This feature is controlled using the `syn.unroll.tripcount_threshold` configuration command as described in Unroll Setting.

In the following example, any loops with a tripcount of less than 16 will be automatically unrolled if doing so improves the throughput:

```
syn.unroll.tripcount_threshold=16
```

If a struct contains arrays, the AGGREGATE directive performs a similar operation as ARRAY_RESHAPE and combines the reshaped array with the other elements in the struct. However, a struct cannot be optimized with AGGREGATE and then partitioned or reshaped. The AGGREGATE, ARRAY_PARTITION, and ARRAY_RESHAPE directives are mutually exclusive.

Structs in the Interface

Structs in the interface are kept aggregated by Vitis HLS by default; combining all of the elements of a struct into a single wide vector. This allows all members of the struct to be read and written-to simultaneously. You can disaggregate structs in the interface by using the DISAGGREGATE pragma or directive. When a struct contains one or more `hls::stream` objects Vitis HLS will automatically disaggregate the struct as described below in *Structs in the Interface with `hls::stream` Elements*.



IMPORTANT! *Disaggregating a struct in the interface is not supported in the Vitis kernel flow because the Vitis tool cannot map a single C-argument to multiple RTL ports. When disaggregating a struct in the interface, either manually or automatically, Vitis HLS will build and export the Vitis kernel output (`.xo`), but that output will result in an error when used with the `v++` command. To support the Vitis Kernel flow you must manually break the struct into its constituent elements, and define any `hls::stream` objects as using an AXIS interface.*

As part of aggregation, the elements of the struct are also aligned on a 4 byte alignment for the Vitis kernel flow, and on 1 byte alignment for the Vivado IP flow. This alignment might require the addition of bit padding to keep or make things aligned, as discussed in Struct Padding and Alignment. By default the aggregated struct is padded rather than packed, but in the Vivado IP flow you can pack it using the `compact=bit` option of the AGGREGATE pragma or directive. However, any port that gets defined as an AXI4 interface (`m_axi`, `s_axilite`, or `axis`) cannot use `compact=bit`.

The member elements of the struct are placed into the vector in the order they appear in the C/C++ code: the first element of the struct is aligned on the LSB of the vector and the final element of the struct is aligned with the MSB of the vector. This allows more data to be accessed in a single clock cycle. Any arrays in the struct are partitioned into individual array elements and placed in the vector from lowest to highest, in order.

In the following example, `struct data_t` is defined in the header file shown. The struct has two data members:

- An unsigned vector `varA` of type `short` (16-bit).
- An array `varB` of four `unsigned char` types (8-bit).

```
typedef struct {
   unsigned short varA;
   unsigned char varB[4];
   } data_t;

data_t struct_port(data_t i_val, data_t *i_pt, data_t *o_pt);
```

Aggregating the struct on the interface results in a single 48-bit port containing 16 bits of `varA`, and 4x8 bits of `varB`.

TIP: *The maximum bit-width of any port or bus created by data packing is 8192 bits, or 4096 bits for `axis` streaming interfaces.*

There are no limitations in the size or complexity of structs that can be synthesized by Vitis HLS. There can be as many array dimensions and as many members in a struct as required. The only limitation with the implementation of structs occurs when arrays are to be implemented as streaming (such as a FIFO interface). In this case, follow the same general rules that apply to arrays on the interface (FIFO Interfaces).

Structs on the Interface with hls::stream Elements

User-defined structs on the interface containing `hls::stream` elements are automatically disaggregated by Vitis HLS. This disaggregated struct is supported in the Vivado IP flow, and the exported IP will work as expected. However, this disaggregated struct is not supported for the Vitis Kernel flow, and the exported kernel (`.xo`) will cause an error when used with the `v++ --link` command. To support the Vitis Kernel flow you must manually break the struct into its constituent elements, and define the `hls::stream` object as using an AXIS interface.

If you have a struct that is disaggregated automatically, Vitis HLS applies any INTERFACE pragmas to the individual elements of the disaggregated struct. If there is only one INTERFACE pragma specified for the struct, it is applied to each element of the struct. If you provide an INTERFACE pragma for each element of the disaggregated struct, it is applied as expected.

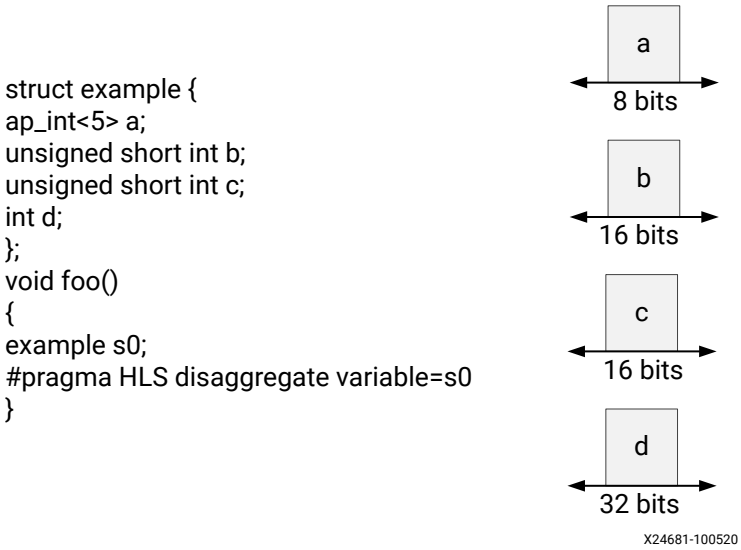
Struct Padding and Alignment

Structs in Vitis HLS can have different types of padding and alignment depending on the use of `__attributes__` or `#pragmas`. These features are described below.

- **Disaggregate:** By default, structs in the code as internal variables are disaggregated into individual elements. The number and type of elements created are determined by the contents of the struct itself. Vitis HLS will decide whether a struct will be disaggregated or not based on certain optimization criteria.

TIP: *You can use the AGGREGATE pragma or directive to prevent the default disaggregation of structs in the code.*

Figure 30: **Disaggregated Struct**



- **Aggregate:** Aggregating structs on the interface is the default behavior of the tool, as discussed in Structs in the Interface. Vitis HLS joins the elements of the struct, aggregating the struct into a single data unit. This is done in accordance with the AGGREGATE pragma or directive, although you do not need to specify the pragma as this is the default for structs on the interface. The aggregate process may also involve bit padding for elements of the struct, to align the byte structures on a default 4-byte alignment, or specified alignment.

  TIP: *The tool can issue a warning when bits are added to pad the struct, by specifying* `-Wpadded` *as a compiler flag.*

- **Aligned:** By default, Vitis HLS will align struct on a 4-byte alignment, padding elements of the struct to align it to a 32-bit width. However, you can use the `__attribute__((aligned(X)))` to add padding between elements of the struct, to align it on "X" byte boundaries.

  IMPORTANT! *Note that "X" can only be defined as a power of 2.*

  The `__attribute__((aligned))` does not change the sizes of variables it is applied to, but may change the memory layout of structures by inserting padding between elements of the struct. As a result the size of the structure will change.

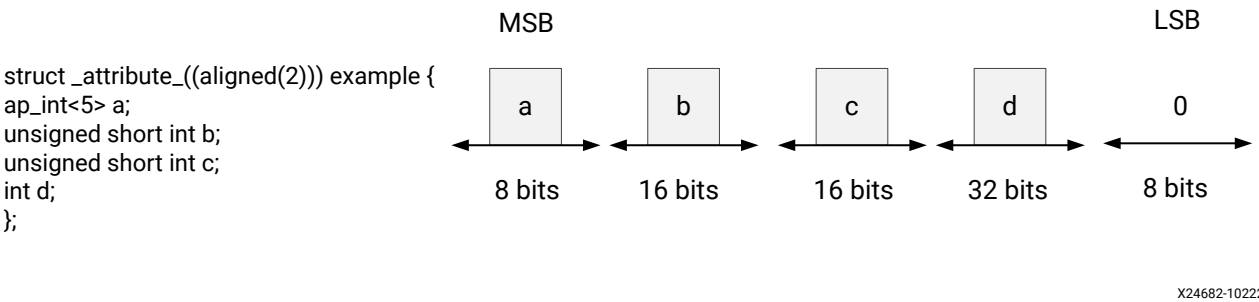
  Data types in struct with custom data widths, such as `ap_int`, are allocated with sizes which are powers of 2. Vitis HLS adds padding bits for aligning the size of the data type to a power of 2.

Vitis HLS will also pad the `bool` data type to align it to 8 bits.

In the following example, the size of `varA` in the struct will be padded to 8 bits instead of 5.

```
struct example  {
ap_int<5> varA;
unsigned short int varB;
unsigned short int varC;
int d;
};
```

Figure 31: **Aligned Struct Implementation**



The padding used depends on the order and size of elements of your struct. In the following code example, the struct alignment is 4 bytes, and Vitis HLS will add 2 bytes of padding after the first element, `varA`, and another 2 bytes of padding after the third element, `varC`. The total size of the struct will be 96-bits.

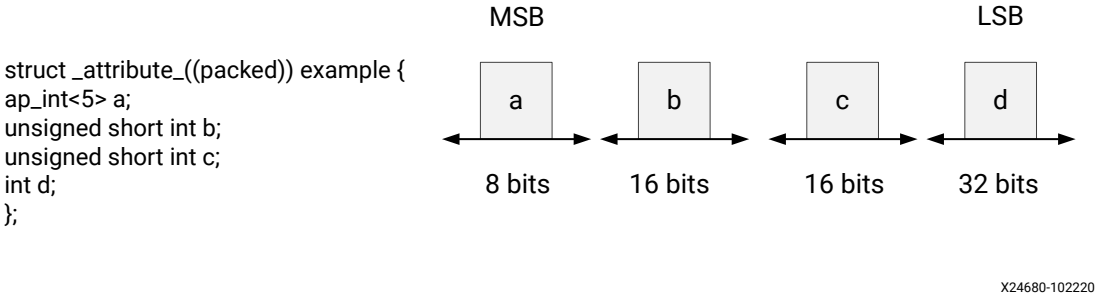
```
struct data_t {
  short varA;
  int varB;
  short varC;
};
```

However, if you rewrite the struct as follows, there will be no need for padding, and the total size of the struct will be 64-bits.

```
struct data_t {
  short varA;
  short varC;
  int varB;
};
```

- **Packed:** Specified with `__attribute__(packed(X))`, Vitis HLS packs the elements of the struct so that the size of the struct is based on the actual size of each element of the struct. In the following example, this means the size of the struct is 72 bits:

Figure 32: **Packed Struct Implementation**



TIP: *This can also be achieved using the* `compact=bit` *option of the AGGREGATE pragma or directive.*

C++ Classes and Templates

C++ classes are fully supported for synthesis with Vitis HLS. The top-level for synthesis must be a function. A class cannot be the top-level for synthesis. To synthesize a class member function, instantiate the class itself into function. Do not simply instantiate the top-level class into the test bench. The following code example shows how class `CFir` (defined in the header file discussed next) is instantiated in the top-level function `cpp_FIR` and used to implement an `FIR` filter.

```
#include "cpp_FIR.h"

// Top-level function with class instantiated
data_t cpp_FIR(data_t x)
 {
 static CFir<coef_t, data_t, acc_t> fir1;

 cout << fir1;

 return fir1(x);
 }
```

IMPORTANT! *Classes and class member functions cannot be the top-level for synthesis. Instantiate the class in a top-level function.*

Before examining the class used to implement the design in the C++ FIR Filter example above, it is worth noting Vitis HLS ignores the standard output stream `cout` during synthesis. When synthesized, Vitis HLS issues the following warnings:

```
INFO [SYNCHK-101] Discarding unsynthesizable system call:
'std::ostream::operator<<' (cpp_FIR.h:108)
INFO [SYNCHK-101] Discarding unsynthesizable system call:
'std::ostream::operator<<' (cpp_FIR.h:108)
INFO [SYNCHK-101] Discarding unsynthesizable system call: 'std::operator<<
<std::char_traits<char> >' (cpp_FIR.h:110)
```

The following code example shows the header file `cpp_FIR.h`, including the definition of class `CFir` and its associated member functions. In this example the operator member functions `()` and `<<` are overloaded operators, which are respectively used to execute the main algorithm and used with `cout` to format the data for display during C/C++ simulation.

```
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

#define N 85

typedef int coef_t;
typedef int data_t;
typedef int acc_t;

// Class CFir definition
template<class coef_T, class data_T, class acc_T>
class CFir {
 protected:
   static const coef_T c[N];
   data_T shift_reg[N-1];
 private:
 public:
   data_T operator()(data_T x);
   template<class coef_TT, class data_TT, class acc_TT>
   friend ostream&
   operator<<(ostream& o, const CFir<coef_TT, data_TT, acc_TT> &f);
};

// Load FIR coefficients
template<class coef_T, class data_T, class acc_T>
const coef_T CFir<coef_T, data_T, acc_T>::c[N] = {
 #include "cpp_FIR.h"
};

// FIR main algorithm
template<class coef_T, class data_T, class acc_T>
data_T CFir<coef_T, data_T, acc_T>::operator()(data_T x) {
 int i;
 acc_t acc = 0;
 data_t m;

 loop: for (i = N-1; i >= 0; i--) {
   if (i == 0) {
      m = x;
      shift_reg[0] = x;
   } else {
      m = shift_reg[i-1];
      if (i != (N-1))
      shift_reg[i] = shift_reg[i - 1];
   }
   acc += m * c[i];
 }
 return acc;
}

// Operator for displaying results
template<class coef_T, class data_T, class acc_T>
ostream& operator<<(ostream& o, const CFir<coef_T, data_T, acc_T> &f) {
```

```
 for (int i = 0; i < (sizeof(f.shift_reg)/sizeof(data_T)); i++) {
    o << shift_reg[ << i << ]=  << f.shift_reg[i] << endl;
 }
 o << _______________ << endl;
 return o;
}

data_t cpp_FIR(data_t x);
```

The test bench in the C++ FIR Filter example is shown in the following code example and demonstrates how top-level function `cpp_FIR` is called and validated. This example highlights some of the important attributes of a good test bench for Vitis HLS synthesis:

- The output results are checked against known good values.
- The test bench returns 0 if the results are confirmed to be correct.

```
#include "cpp_FIR.h"

int main() {
 ofstream result;
 data_t output;
 int retval=0;


 // Open a file to saves the results
 result.open(result.dat);

 // Apply stimuli, call the top-level function and saves the results
 for (int i = 0; i <= 250; i++)
 {
    output = cpp_FIR(i);

    result << setw(10) << i;
    result << setw(20) << output;
    result << endl;

 }
 result.close();

 // Compare the results file with the golden results
 retval = system(diff --brief -w result.dat result.golden.dat);
 if (retval != 0) {
    printf(Test failed  !!!\n);
    retval=1;
 } else {
    printf(Test passed !\n);
 }

 // Return 0 if the test
 return retval;
}
```

C++ Test Bench for `cpp_FIR`

To apply directives to objects defined in a class:

1. Open the file where the class is defined (typically a header file).
2. Apply the directive using the **Directives** tab.

As with functions, all instances of a class have the same optimizations applied to them.

Global Variables and Classes

AMD does not recommend using global variables in classes. They can prevent some optimizations from occurring. In the following code example, a class is used to create the component for a filter (class `polyd_cell` is used as a component that performs shift, multiply and accumulate operations).

```
typedef long long acc_t;
typedef int mult_t;
typedef char data_t;
typedef char coef_t;

#define TAPS 3
#define PHASES 4
#define DATA_SAMPLES 256
#define CELL_SAMPLES 12

// Use k on line 73 static int k;

template <typename T0, typename T1, typename T2, typename T3, int N>
class polyd_cell {
private:
public:
     T0 areg;
     T0 breg;
     T2 mreg;
     T1 preg;
 T0 shift[N];
     int k;   //line 73
 T0 shift_output;
     void exec(T1 *pcout, T0 *dataOut, T1 pcin, T3 coeff, T0 data, int col)
     {
     Function_label0:;

     if (col==0) {
   SHIFT:for (k = N-1; k >= 0; --k) {
      if (k > 0)
        shift[k] = shift[k-1];
      else
        shift[k] = data;
     }
     *dataOut = shift_output;
     shift_output = shift[N-1];
     }
     *pcout = (shift[4*col]* coeff) + pcin;

     }
};

// Top-level function with class instantiated
void cpp_class_data (
 acc_t *dataOut,
 coef_t coeff1[PHASES][TAPS],
 coef_t  coeff2[PHASES][TAPS],
 data_t  dataIn[DATA_SAMPLES],
 int  row
) {
```

```
 acc_t pcin0 = 0;
 acc_t pcout0, pcout1;
 data_t dout0, dout1;
 int col;
 static acc_t accum=0;
 static int sample_count = 0;
 static polyd_cell<data_t, acc_t, mult_t, coef_t, CELL_SAMPLES>
polyd_cell0;
 static polyd_cell<data_t, acc_t, mult_t, coef_t, CELL_SAMPLES>
polyd_cell1;

 COL:for (col = 0; col <= TAPS-1; ++col) {

 polyd_cell0.exec(&pcout0,&dout0,pcin0,coeff1[row]
[col],dataIn[sample_count],

col);

 polyd_cell1.exec(&pcout1,&dout1,pcout0,coeff2[row][col],dout0,col);


    if ((row==0) && (col==2)) {
        *dataOut = accum;
        accum = pcout1;
    } else {
        accum = pcout1 + accum;
 }

 }
 sample_count++;
}
```

Within class `polyd_cell` there is a loop `SHIFT` used to shift data. If the loop index `k` used in loop `SHIFT` was removed and replaced with the global index for `k` (shown earlier in the example, but commented `static int k`), Vitis HLS is unable to pipeline any loop or function in which class `polyd_cell` was used. Vitis HLS would issue the following message:

```
@W [XFORM-503] Cannot unroll loop 'SHIFT' in function 'polyd_cell<char,
long long,
int, char, 12>::exec' completely: variable loop bound.
```

Using local non-global variables for loop indexing ensures that Vitis HLS can perform all optimizations.

Templates

Vitis HLS supports the use of templates in C++ for synthesis. Vitis HLS does not support templates for the top-level function. Refer to Vitis-HLS-Introductory-Examples/Modeling/using_C++_templates on GitHub for an example of these concepts.



IMPORTANT! *The top-level function cannot be a template.*

Using Templates to Create Unique Instances

A static variable in a template function is duplicated for each different value of the template arguments.

Different C++ template values passed to a function creates unique instances of the function for each template value. Vitis HLS synthesizes these copies independently within their own context. This can be beneficial as the tool can provide specific optimizations for each unique instance, producing a straightforward implementation of the function.

```
template<int NC, int K>
void startK(int* dout) {
 static int acc=0;
 acc  += K;
 *dout = acc;
}

void foo(int* dout) {
 startK<0,1> (dout);
}

void goo(int* dout) {
 startK<1,1> (dout);
}

int main() {
 int dout0,dout1;
 for (int i=0;i<10;i++) {
 foo(&dout0);
 goo(&dout1);
   cout <<"dout0/1 = "<<dout0<<" / "<<dout1<<endl;
 }
    return 0;
}
```

Using Templates for Recursion

Templates can also be used to implement a form of recursion that is not supported in standard C synthesis (Recursive Functions).

The following code example shows a case in which a templatized `struct` is used to implement a tail-recursion Fibonacci algorithm. The key to performing synthesis is that a termination class is used to implement the final call in the recursion, where a template size of one is used.

```
//Tail recursive call
template<data_t N> struct fibon_s {
  template<typename T>
    static T fibon_f(T a, T b) {
       return fibon_s<N-1>::fibon_f(b, (a+b));
 }
};

// Termination condition
template<> struct fibon_s<1> {
  template<typename T>
    static T fibon_f(T a, T b) {
```

```
        return b;
 }
};

void cpp_template(data_t a, data_t b, data_t &dout){
 dout = fibon_s<FIB_N>::fibon_f(a,b);
}
```

Enumerated Types

The header file in the following code example defines some `enum` types and uses them in a `struct`. The `struct` is used in turn in another `struct`. This allows an intuitive description of a complex type to be captured.

The following code example shows how a complex define (`MAD_NSBSAMPLES`) statement can be specified and synthesized.

```
#include <stdio.h>

enum mad_layer {
 MAD_LAYER_I   = 1,
 MAD_LAYER_II  = 2,
 MAD_LAYER_III = 3
};

enum mad_mode {
 MAD_MODE_SINGLE_CHANNEL = 0,
 MAD_MODE_DUAL_CHANNEL = 1,
 MAD_MODE_JOINT_STEREO = 2,
 MAD_MODE_STEREO = 3
};

enum mad_emphasis {
 MAD_EMPHASIS_NONE = 0,
 MAD_EMPHASIS_50_15_US = 1,
 MAD_EMPHASIS_CCITT_J_17 = 3
};

typedef   signed int mad_fixed_t;

typedef struct mad_header {
 enum mad_layer layer;
       enum mad_mode mode;
 int mode_extension;
 enum mad_emphasis emphasis;

 unsigned long long bitrate;
 unsigned int samplerate;

 unsigned short crc_check;
 unsigned short crc_target;

 int flags;
 int private_bits;

} header_t;

typedef struct mad_frame {
 header_t header;
```

```
 int options;
 mad_fixed_t sbsample[2][36][32];
} frame_t;

# define MAD_NSBSAMPLES(header)  \
 ((header)->layer == MAD_LAYER_I ? 12 :  \
 (((header)->layer == MAD_LAYER_III &&  \
 ((header)->flags & 17)) ? 18 : 36))


void types_composite(frame_t *frame);
```

The `struct` and `enum` types defined in the previous example are used in the following example. If the `enum` is used in an argument to the top-level function, it is synthesized as a 32-bit value to comply with the standard C/C++ compilation behavior. If the enum types are internal to the design, Vitis HLS optimizes them down to the only the required number of bits.

The following code example shows how `printf` statements are ignored during synthesis.

```
#include "types_composite.h"

void types_composite(frame_t *frame)
{
 if (frame->header.mode != MAD_MODE_SINGLE_CHANNEL) {
   unsigned int ns, s, sb;
   mad_fixed_t left, right;

   ns = MAD_NSBSAMPLES(&frame->header);
   printf("Samples from header %d \n", ns);

 for (s = 0; s < ns; ++s) {
 for (sb = 0; sb < 32; ++sb) {
 left  = frame->sbsample[0][s][sb];
 right = frame->sbsample[1][s][sb];
 frame->sbsample[0][s][sb] = (left + right) / 2;
 }
 }
 frame->header.mode = MAD_MODE_SINGLE_CHANNEL;
 }
}
```

Unions

In the following code example, a union is created with a `double` and a `struct`. Unlike C/C++ compilation, synthesis does not guarantee using the same memory (in the case of synthesis, registers) for all fields in the `union`. Vitis HLS perform the optimization that provides the most optimal hardware.

```
#include "types_union.h"

dout_t types_union(din_t N, dinfp_t F)
{
 union {
    struct {int a; int b; } intval;
    double fpval;
 } intfp;
```

```
 unsigned long long one, exp;

 // Set a floating-point value in union intfp
 intfp.fpval = F;

 // Slice out lower bits and add to shifted input
 one = intfp.intval.a;
 exp = (N & 0x7FF);

 return ((exp << 52) + one) & (0x7fffffffffffffLL);
}
```

Vitis HLS does *not* support the following:

- Unions on the top-level function interface.
- Pointer reinterpretation for synthesis. Therefore, a union cannot hold pointers to different types or to arrays of different types.
- Access to a union through another variable. Using the same union as the previous example, the following is not supported:

```
for (int i = 0; i < 6; ++i)
if (i<3)
 A[i] = intfp.intval.a + B[i];
 else
 A[i] = intfp.intval.b + B[i];
}
```

- However, it can be explicitly re-coded as:

```
 A[0] = intfp.intval.a + B[0];
 A[1] = intfp.intval.a + B[1];
 A[2] = intfp.intval.a + B[2];
 A[3] = intfp.intval.b + B[3];
 A[4] = intfp.intval.b + B[4];
 A[5] = intfp.intval.b + B[5];
```

The synthesis of unions does not support casting between native C/C++ types and user-defined types.

Often with Vitis HLS designs, unions are used to convert the raw bits from one data type to another data type. Generally, this raw bit conversion is needed when using floating point values at the top-level port interface. For one example, see below:

```
typedef float T;
unsigned int value; // the "input" of the conversion
T myhalfvalue; // the "output" of the conversion
union
{
  unsigned int as_uint32;
  T as_floatingpoint;
} my_converter;
my_converter.as_uint32 = value;
myhalfvalue = my_converter. as_floatingpoint;
```

This type of code is fine for float C/C++ data types and with modification, it is also fine for double data types. Changing the `typedef` and the `int` to `short` will not work for half data types, however, because half is a class and cannot be used in a union. Instead, the following code can be used:

```
typedef half T;
short value;
T myhalfvalue = static_cast<T>(value);
```

Similarly, the conversion the other way around uses `value=static_cast<ap_uint<16>>(myhalfvalue)` or `static_cast< unsigned short >(myhalfvalue)`.

```
ap_fixed<16,4> afix = 1.5;
ap_fixed<20,6> bfix = 1.25;
half ahlf = afix.to_half();
half bhlf = bfix.to_half();
```

Another method is to use the helper class `fp_struct<half>` to make conversions using the methods `data()` or `to_int()`. Use the header file `hls/utils/x_hls_utils.h`.

Type Qualifiers

The type qualifiers can directly impact the hardware created by high-level synthesis. In general, the qualifiers influence the synthesis results in a predictable manner, as discussed below. Vitis HLS is limited only by the interpretation of the qualifier as it affects functional behavior and can perform optimizations to create a more optimal hardware design. Examples of this are shown after an overview of each qualifier.

Volatile

The `volatile` qualifier impacts how many reads or writes are performed in the RTL when pointers are accessed multiple times on function interfaces. Although the `volatile` qualifier impacts this behavior in all functions in the hierarchy, the impact of the `volatile` qualifier is primarily discussed in the section on top-level interfaces.

***Note*:** Accesses to/from volatile variables is preserved. This means:

- no burst access
- no port widening
- no dead code elimination

Tip: Arbitrary precision types do not support the volatile qualifier for arithmetic operations. Any arbitrary precision data types using the volatile qualifier must be assigned to a non-volatile data type before being used in arithmetic expression.

Statics

Static types in a function hold their value between function calls. The equivalent behavior in a hardware design is a registered variable (a flip-flop or memory). If a variable is required to be a static type for the C/C++ function to execute correctly, it will certainly be a register in the final RTL design. The value must be maintained across invocations of the function and design.

It is *not* true that *only* `static` types result in a register after synthesis. Vitis HLS determines which variables are required to be implemented as registers in the RTL design. For example, if a variable assignment must be held over multiple cycles, Vitis HLS creates a register to hold the value, even if the original variable in the C/C++ function was *not* a static type.

Vitis HLS obeys the initialization behavior of statics and assigns the value to zero (or any explicitly initialized value) to the register during initialization. This means that the `static` variable is initialized in the RTL code and in the FPGA bitstream. It does not mean that the variable is re-initialized each time the reset signal is.

See RTL Configuration to determine how static initialization values are implemented with regard to the system reset.

Const

A `const` type specifies that the value of the variable is never updated. The variable is read but never written to and therefore must be initialized. For most `const` variables, this typically means that they are reduced to constants in the RTL design. Vitis HLS performs constant propagation and removes any unnecessary hardware).

In the case of arrays, the `const` variable is implemented as a ROM in the final RTL design (in the absence of any auto-partitioning performed by Vitis HLS on small arrays). Arrays specified with the `const` qualifier are (like statics) initialized in the RTL and in the FPGA bitstream. There is no need to reset them, because they are never written to.

ROM Optimization

The following shows a code example in which Vitis HLS implements a ROM even though the array is not specified with a `static` or `const` qualifier. This demonstrates how Vitis HLS analyzes the design, and determines the most optimal implementation. The qualifiers guide the tool, but do not dictate the final RTL.

```
#include "array_ROM.h"

dout_t array_ROM(din1_t inval, din2_t idx)
{
 din1_t lookup_table[256];
 dint_t i;

 for (i = 0; i < 256; i++) {
```

```
 lookup_table[i] = 256 * (i - 128);
 }

 return (dout_t)inval * (dout_t)lookup_table[idx];
}
```

In this example, the tool is able to determine that the implementation is best served by having the variable `lookup_table` as a memory element in the final RTL.

Arbitrary Precision (AP) Data Types

C/C++-based native data types are based-on on 8-bit boundaries (8, 16, 32, 64 bits). However, RTL buses (corresponding to hardware) support arbitrary data lengths. Using the standard C/C++ data types can result in inefficient hardware implementation. For example, the basic multiplication unit in an AMD device is the DSP library cell. Multiplying "ints" (32-bit) would require more than one DSP cell while using arbitrary precision types could use only one cell per multiplication.

Arbitrary precision (AP) data types allow your code to use variables with smaller bit-widths, and for the C/C++ simulation to validate the functionality remains identical or acceptable. The smaller bit-widths result in hardware operators which are in turn smaller and run faster. This allows more logic to be placed in the FPGA, and for the logic to execute at higher clock frequencies.

AP data types are provided for C++ and allow you to model data types of any width from 1 to 1024-bit. You must specify the use of AP libraries by including them in your C++ source code as explained in Arbitrary Precision Data Types Library.

TIP: *Arbitrary precision types are only required on the function boundaries, because Vitis HLS optimizes the internal logic and removes data bits and logic that do not fanout to the output ports.*

AP Example

For example, a design with a filter function for a communications protocol requires 10-bit input data and 18-bit output data to satisfy the data transmission requirements. Using standard C/C++ data types, the input data must be at least 16-bits and the output data must be at least 32-bits. In the final hardware, this creates a datapath between the input and output that is wider than necessary, uses more resources, has longer delays (for example, a 32-bit by 32-bit multiplication takes longer than an 18-bit by 18-bit multiplication), and requires more clock cycles to complete.

Using arbitrary precision data types in this design, you can specify the exact bit-sizes needed in your code prior to synthesis, simulate the updated code, and verify the results prior to synthesis. Refer to Vitis-HLS-Introductory-Examples/Modeling on GitHub for examples of using arbitrary precision and fixed point ap data types.

Advantages of AP Data Types

IMPORTANT! *One disadvantage of AP data types is that arrays are not automatically initialized with a value of 0. You must manually initialize the array if desired.*

The following code performs some basic arithmetic operations:

```
#include "types.h"

void apint_arith(dinA_t  inA, dinB_t  inB, dinC_t  inC, dinD_t  inD,
          dout1_t *out1, dout2_t *out2, dout3_t *out3, dout4_t *out4
  ) {

 // Basic arithmetic operations
 *out1 = inA * inB;
 *out2 = inB + inA;
 *out3 = inC / inA;
 *out4 = inD % inA;
}
```

The data types `dinA_t`, `dinB_t`, etc. are defined in the header file `types.h`. It is highly recommended to use a project wide header file such as `types.h` as this allows for the easy migration from standard C/C++ types to arbitrary precision types and helps in refining the arbitrary precision types to the optimal size.

If the data types in the above example are defined as:

```
typedef char dinA_t;
typedef short dinB_t;
typedef int dinC_t;
typedef long long dinD_t;
typedef int dout1_t;
typedef unsigned int dout2_t;
typedef int32_t dout3_t;
typedef int64_t dout4_t;
```

The design gives the following results after synthesis:

```
+ Timing (ns):
    * Summary:
    +---------+-------+----------+------------+
    |  Clock  | Target| Estimated| Uncertainty|
    +---------+-------+----------+------------+
    |default  |   4.00|      3.85|        0.50|
    +---------+-------+----------+------------+

+ Latency (clock cycles):
    * Summary:
    +-----+-----+-----+-----+---------+
    |  Latency  |  Interval | Pipeline|
    | min | max | min | max |   Type  |
    +-----+-----+-----+-----+---------+
    |   66|   66|   67|   67|   none  |
    +-----+-----+-----+-----+---------+
* Summary:
+-----------------+---------+-------+--------+--------+
|       Name      | BRAM_18K| DSP48E|   FF   |   LUT  |
```

```
+-----------------+---------+-------+--------+--------+
|Expression       |        -|      -|       0|      17|
|FIFO             |        -|      -|       -|       -|
|Instance         |        -|      1|   17920|   17152|
|Memory           |        -|      -|       -|       -|
|Multiplexer      |        -|      -|       -|       -|
|Register         |        -|      -|       7|       -|
+-----------------+---------+-------+--------+--------+
|Total            |        0|      1|   17927|   17169|
+-----------------+---------+-------+--------+--------+
|Available        |      650|    600|  202800|  101400|
+-----------------+---------+-------+--------+--------+
|Utilization (%)  |        0|   ~0  |       8|      16|
+-----------------+---------+-------+--------+--------+
```

However, if the width of the data is not required to be implemented using standard C/C++ types but in some width which is smaller, but still greater than the next smallest standard C/C++ type, such as the following:

```
typedef int6 dinA_t;
typedef int12 dinB_t;
typedef int22 dinC_t;
typedef int33 dinD_t;
typedef int18 dout1_t;
typedef uint13 dout2_t;
typedef int22 dout3_t;
typedef int6 dout4_t;
```

The synthesis results show an improvement to the maximum clock frequency, the latency and a significant reduction in area of 75%.

```
+ Timing (ns):
    * Summary:
    +---------+-------+----------+------------+
    |  Clock  | Target| Estimated| Uncertainty|
    +---------+-------+----------+------------+
    |default  |   4.00|      3.49|        0.50|
    +---------+-------+----------+------------+

+ Latency (clock cycles):
    * Summary:
    +-----+-----+-----+-----+---------+
    |  Latency  |  Interval | Pipeline|
    | min | max | min | max |   Type  |
    +-----+-----+-----+-----+---------+
    |   35|   35|   36|   36|   none  |
    +-----+-----+-----+-----+---------+
* Summary:
+-----------------+---------+-------+--------+--------+
|       Name      | BRAM_18K| DSP48E|   FF   |   LUT  |
+-----------------+---------+-------+--------+--------+
|Expression       |        -|      -|       0|      13|
|FIFO             |        -|      -|       -|       -|
|Instance         |        -|      1|    4764|    4560|
|Memory           |        -|      -|       -|       -|
|Multiplexer      |        -|      -|       -|       -|
|Register         |        -|      -|       6|       -|
+-----------------+---------+-------+--------+--------+
|Total            |        0|      1|    4770|    4573|
```

```
+-----------------+---------+-------+--------+--------+
|Available        |      650|    600|  202800|  101400|
+-----------------+---------+-------+--------+--------+
|Utilization (%)  |        0|   ~0  |       2|       4|
+-----------------+---------+-------+--------+--------+
```

The large difference in latency between both design is due to the division and remainder operations which take multiple cycles to complete. Using AP data types, rather than force fitting the design into standard C/C++ data types, results in a higher quality hardware implementation: the same accuracy with better performance with fewer resources.

Overview of Arbitrary Precision Integer Data Types

Vitis HLS provides integer and fixed-point arbitrary precision data types for C++.

Table 5: **Arbitrary Precision Data Types**

Language	Integer Data Type	Required Header
C++	ap_[u]int<W> (1024 bits) Can be extended to 4K bits wide as described below.	#include “ap_int.h”
C++	ap_[u]fixed<W,I,Q,O,N>	#include “ap_fixed.h”

For the C++ language `ap_[u]int` data types the header file `ap_int.h` defines the arbitrary precision integer data type. To use arbitrary precision integer data types in a C++ function:

- Add header file `ap_int.h` to the source code.
- Change the bit types to `ap_int<N>` or `ap_uint<N>`, where N is a bit-size from 1 to 1024.

The following example shows how the header file is added and two variables implemented to use 9-bit integer and 10-bit unsigned integer types:

```
#include "ap_int.h"

void foo_top () {

ap_int<9>  var1;           // 9-bit
ap_uint<10>  var2;           // 10-bit unsigned
```

The default maximum width allowed for `ap_[u]int` data types is 1024 bits. This default may be overridden by defining the macro `AP_INT_MAX_W` with a positive integer value less than or equal to 4096 before inclusion of the `ap_int.h` header file.

IMPORTANT! *Setting the value of `AP_INT_MAX_W` too high can cause slow software compile and runtimes.*

The following is an example of overriding `AP_INT_MAX_W`:

```
#define AP_INT_MAX_W 4096 // Must be defined before next line
#include "ap_int.h"

ap_int<4096> very_wide_var;
```

Overview of Arbitrary Precision Fixed-Point Data Types

Fixed-point data types model the data as an integer and fraction bits in two's complement with the format `ap_fixed<W,I,[Q,O,N]>` as explained in the following table. In the following example, the Vitis HLS `ap_fixed` type is used to define an 18-bit variable with 6 bits (including the sign bit) specified as representing the numbers above the binary point, and 12 bits implied to represent the fractional value after the decimal point. The variable is specified as signed and the quantization mode is set to round to plus infinity. Because the overflow mode is not specified, the default wrap-around mode is used for overflow.

```
#include <ap_fixed.h>
...
ap_fixed<18,6,AP_RND> t1 = 1.5; // internally represented as
0b00'0001.1000'0000'0000 (0x01800)
ap_fixed<18.6,AP_RND> t2 = -1.5; // 0b11'1110.1000'0000'0000 (0x3e800)
...
```



TIP: *The Integer value (`I`) of `ap_fixed` specifies the number of integer bits to the left of the binary point, including the sign bit.*

When performing calculations where the variables have different number of bits or different precision, the binary point is automatically aligned. For example, when performing division with fixed-point type variables of different sizes, the fraction of the quotient is no greater than that of the dividend. To preserve the fractional part of the quotient you can cast the result to the new variable width before assignment.

The behavior of the C++ simulations performed using fixed-point matches the resulting hardware. This allows you to analyze the bit-accurate, quantization, and overflow behaviors using fast C-level simulation.

Fixed-point types are a useful replacement for floating point types which require many clock cycle to complete. Unless the entire range of the floating-point type is required, the same accuracy can often be implemented with a fixed-point type resulting in the same accuracy with smaller and faster hardware.

A summary of the `ap_fixed` type identifiers is provided in the following table.

Table 6: **Fixed-Point Identifier Summary**

Identifier	Description	
W	Word length in bits	

I	The number of bits used to represent the integer value, that is, the number of integer bits to the *left* of the binary point, including the sign bit.
	When I is negative, as shown in the example below, it represents the number of *implicit* sign bits (for signed representation), or the number of *implicit* zero bits (for unsigned representation) to the *right* of the binary point. For example,
	`ap_fixed<2, 0> a = -0.5;  // a can be -0.5,`
	`ap_ufixed<1, 0> x = 0.5;  // 1-bit representation. x can be 0 or 0.5`
	`ap_ufixed<1, -1> y = 0.25;  // 1-bit representation. y can be 0 or 0.25`
	`const ap_fixed<1, -7> z = 1.0/256;  // 1-bit representation for z = 2^-8`	

Q	Quantization mode: This dictates the behavior when greater precision is generated than can be defined by smallest fractional bit in the variable used to store the result.	
	ap_fixed Types	Description
	AP_RND	Round to plus infinity
	AP_RND_ZERO	Round to zero
	AP_RND_MIN_INF	Round to minus infinity
	AP_RND_INF	Round to infinity
	AP_RND_CONV	Convergent rounding
	AP_TRN	Truncation to minus infinity (default)
	AP_TRN_ZERO	Truncation to zero
O	Overflow mode: This dictates the behavior when the result of an operation exceeds the maximum (or minimum in the case of negative numbers) possible value that can be stored in the variable used to store the result.	
	ap_fixed Types	Description
	AP_SAT[1]	Saturation
	AP_SAT_ZERO[1]	Saturation to zero
	AP_SAT_SYM[1]	Symmetrical saturation
	AP_WRAP	Wrap around (default)
	AP_WRAP_SM	Sign magnitude wrap around
N	This defines the number of saturation bits in overflow wrap modes.	

Notes:

1. Using the AP_SAT* modes can result in higher resource usage as extra logic will be needed to perform saturation and this extra cost can be as high as 20% additional LUT usage.
2. Fixed-point math functions from the `hls_math` library do not support the `ap_[u]fixed` template parameters Q,O, and N, for quantization mode, overflow mode, and the number of saturation bits, respectively. The quantization and overflow modes are only effective when an `ap_[u]fixed` variable is on the left hand of assignment or being initialized, but not during the calculation.

The default maximum width allowed for `ap_[u]fixed` data types is 1024 bits. This default may be overridden by defining the macro `AP_INT_MAX_W` with a positive integer value less than or equal to 4096 before inclusion of the `ap_int.h` header file.

IMPORTANT! *ROM Synthesis can take a long time when using `ap_[u]fixed`. Changing it to `int` results in a quicker synthesis. For example:*

```
static ap_fixed<32,0> a[32][depth] =
```

Can be changed to:

```
static int a[32][depth] =
```

Arbitrary Precision Floating-Point Library ap_float<int W, int E>

The arbitrary precision floating-point library has been added to provide extra flexibility so that designers can investigate which precision best fits their design and perform trade-offs between resource costs and performance versus accuracy against standard floating-point formats.

The arbitrary precision floating-point library can match additional floating-point formats such as bfloat16, tf32, etc.

The header file needed is `ap_float.h` and must be add using `#include <ap_float.h>` to the source code. This library is provided as a templatized library `ap_float<int W, int E>` where W is the total bitwidth of the variable and E is the bitwidth of the exponent. The rest of the internal bit fields are the same as the IEEE 754 floating-point standard with a 1-bit sign and the remaining W-E-1 bits to hold the fraction bits of the significant. The significant has W-E bits: an implicit bit set to "1" followed by W-E-1 fraction bits.

Using the `ap_float<>` library, the normal 32-bit IEEE 754 floating point number type will be `ap_float<32,8>`; other common types are given in the following table.

Table 7: **C++ Common Floating-Point Data Types with ap_float<>**

C++ data type	ap_float<W,E> equivalent
float	ap_float<32,8>
double	ap_float<64,11>
half	ap_float<16,5>
bfloat16	ap_float<16,8>

***Note*:** Designs do not need to change as the standard C++ float and double floating-point datatypes are supported natively by the compiler.

The following restrictions apply because they are inherited from the AMD Floating-Point LogiCORE IP (see *Floating-Point Operator LogiCORE IP Product Guide* (PG060)) which the ap_float<> library uses when the RTL is generated:

1. Total bitwidth W can be up to 80 bits.
2. Exponent bitwidth E from 4 to 16 bits.
3. Mantissa bitwidth from 4 to 64 bits.
4. Sub-normals are not supported: sub-normal results and arguments are rounded to 0.
5. Round to Nearest ties to Even is the default and only supported rounding mode.
6. All NaNs are Quiet NaNs (Signaling NaNs are not supported).

Sub-normals are not supported, the significant will always have its implicit bit set to 1.

Supported Devices

All Versal and UltraScale+ FPGA Devices are supported.

Supported Functions

All common C++ floating-point arithmetic operators are supported:

- binary +,-,*,/,
- unary +=,-=,*=,/=,
- comparators,
- specific HLS functions like hls::sqrt, hls::fma, hls::abs
- accumulation is supported using operator+= or by using the specific ap_float_acc<> helper library discussed below.

No automatic conversion from ap_float<> to float so other functions will cause a compilation error. This is a design choice and is discussed below.

A simple example is shown:

```
#include <ap_float.h>

typedef ap_float<18, 6> my_float;

float apf_simple_example( float f_input) {
    #pragma HLS pipeline
    my_float pi   = M_PI;

    // Implicit conversion from float to ap_float<> is allowed
    my_float r    = f_input;

    my_float area = pi*r*r;
    if ( area>1.0f ) { area=1.0f; }

    // Explicit conversion from ap_float<>
    // Otherwise => error: no viable conversion from 'ap_float<18, 6>' to
```

```
'float'
    float ret_value=(float)area;

    return ret_value;
}
```

Dealing with Unsupported Functions

There is no automatic conversion from ap_float<> to any float datatypes: this is a library's design decision to force designers to think about the desired precision and associated costs when making a type conversion.

As such, the use of ap_float<> with any unsupported functions in the library will trigger a compilation error like in the example below.

```
ap_float<16,8> k;
... = hls::log(k); // log is not expecting ap_float<>
// will give
ERROR: [HLS 207-3339] no matching function for 'log'
INFO: [HLS 207-4378] candidate function not viable: no known conversion
from 'ap_float<16, 8>' for 1st argument
```

When faced with this example the design can explicitly convert ap_float<> to any of float, ap_fixed<> or ap_int<>/int datatypes:

```
... = hls::log((float)k);          // using float
... = hls::log((ap_fixed<20,5>)k); // using ap_fixed
... = hls::log((int)k);            // using int or ap_int
```

Conversions of ap_float<> To and From Other Data Types

Conversions between one ap_float<> and another ap_float<> of different precision, ie different W or E template parameters, will instantiate a floating-point IP for rounding and/or saturation logic. It will be free only when the parameters are the same or in the situation of C floating-point data types when the equivalent parameters are used, for example, float to and from ap_float<32,8>.

Conversions between one ap_float<> and integer types or ap_fixed<> will instantiate a floating-point IP for quantization.

Accumulation of ap_float<> Using ap_float_acc<> Helper Class

The helper class ap_float_acc<> is provided to perform an accumulation using an ap_fixed<> integer underlying datatype to enable II=1 for the accumulation loop. The pseudo-code of the class declaration is shown below.

```
template <...>
class ap_float_acc< ap_float<W, E>, ap_fixed<WFX, IFX> > {
   ap_float<W, E> accumulate(const ap_float<W, E> &value, bool last);
}
```

The first template argument defines the input data type ap_float<W,E> of the values to be accumulated.

The second template argument defines the underlying datatype ap_fixed<WFX, IFX> of the internal accumulator which performs the actual accumulation.

The member function accumulate(value,last) is used to accumulate the input "value" into its internal accumulator, the input "last" is used to clear the internal accumulator after the last accumulation. The return value is the accumulator value.

```
using apf_t = ap_float<18, 5>;                // using is a C++ syntax
using acc_datatype_t = ap_fixed<32, 16>;

... my_function(...) {
   ap_float_acc<apf_t, acc_datatype_t> acc;  // instance of accumulator
class

   apf_t res;
   for (unsigned int i = 0; i < n; i++) {
      apf_t value = ...;                      // value to accumulate
      res=acc.accumulate( /*input:ap_float*/ value, /*last:bool*/ i == n-1);
   }
}
```

Using ap_float_acc<> Restrictions

An array of **ap_float_acc<>** instances must be fully partitioned by using the array_partition pragma.

The **ap_float_acc<>** helper class does not support crossing hierarchies and cannot be copied:

- Declare the ap_float_acc instance in the same function as the accumulate member function will be used; this also applies to the dataflow process.
- If the accumulate function is used in a pipelined loop it should only be used in that loop (as the compiler extracts pipelined loops into their own function).

Global Variables

Global variables can be freely used in the code and are fully synthesizable. However, global variables can not be inferred as arguments to the top-level function, but must instead be explicitly specified as arguments for ports in the RTL design.

The following code example shows the default synthesis behavior of global variables. It uses three global variables (`idx`, `Ain` and `Aout`). Although this example uses arrays, Vitis HLS supports all types of global variables.

- Values are read from array `Ain`.
- Array `Aint` is used to transform and pass values from `Ain` to `Aout`.
- The outputs are written to array `Aout`.



IMPORTANT! *Access to the global variables `Ain` and `Aout` must be explicitly listed in the argument list.*

```
#include "top.h"

void top(const int idx, const int Ain[N], int Aout[Nhalf]) {

    int Aint[N];

    // Move elements in the input array

    ILOOP: for (int i = 0; i < N; i++) {

        int iadj = (i + idx) % N;

        Aint[i] = Ain[i] + Ain[iadj];

    } // end ILOOP

    // sum the 1st and 2nd halves
    OLOOP: for (int i = 0; i < Nhalf; i++) {

        Aout[i] = (Aint[i] + Aint[Nhalf + i]);

    } // end OLOOP

} // end top()
```

Pointers

Pointers are used extensively in C/C++ code and are supported for synthesis, but it is generally recommended to avoid the use of pointers in your code. This is especially true when using pointers in the following cases:

- When pointers are accessed (read or written) multiple times in the same function.
- When using arrays of pointers, each pointer must point to a scalar or a scalar array (not another pointer).
- Pointer casting is supported only when casting between standard C/C++ types, as shown.

***Note*:** Pointer to pointer is not supported.

The following code example shows synthesis support for pointers that point to multiple objects.

```
#include "pointer_multi.h"

dout_t pointer_multi (sel_t sel, din_t pos) {
 static const dout_t a[8] = {1, 2, 3, 4, 5, 6, 7, 8};
 static const dout_t b[8] = {8, 7, 6, 5, 4, 3, 2, 1};

 dout_t* ptr;
 if (sel)
 ptr = a;
 else
 ptr = b;

 return ptr[pos];
}
```

Vitis HLS supports pointers to pointers for synthesis but does not support them on the top-level interface, that is, as argument to the top-level function. If you use a pointer to pointer in multiple functions, Vitis HLS inlines all functions that use the pointer to pointer. Inlining multiple functions can increase runtime.

```
#include "pointer_double.h"

data_t sub(data_t ptr[10], data_t size, data_t**flagPtr)
{
 data_t x, i;

 x = 0;
 // Sum x if AND of local index and pointer to pointer index is true
 for(i=0; i<size; ++i)
   if (**flagPtr & i)
        x += *(ptr+i);
 return x;
}

data_t pointer_double(data_t pos, data_t x, data_t* flag)
{
 data_t array[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
 data_t* ptrFlag;
 data_t i;

 ptrFlag = flag;

 // Write x into index position pos
 if (pos >=0 & pos < 10)
```

```
 *(array+pos) = x;

 // Pass same index (as pos) as pointer to another function
 return sub(array, 10, &ptrFlag);
}
```

Arrays of pointers can also be synthesized. See the following code example in which an array of pointers is used to store the start location of the second dimension of a global array. The pointers in an array of pointers can point only to a scalar or to an array of scalars. They cannot point to other pointers.

```
#include "pointer_array.h"

data_t A[N][10];

data_t pointer_array(data_t B[N*10]) {
 data_t i,j;
 data_t sum1;

 // Array of pointers
 data_t* PtrA[N];

 // Store global array locations in temp pointer array
 for (i=0; i<N; ++i)
    PtrA[i] = &(A[i][0]);

 // Copy input array using pointers
 for(i=0; i<N; ++i)
    for(j=0; j<10; ++j)
       *(PtrA[i]+j) = B[i*10 + j];

  // Sum input array
 sum1 = 0;
 for(i=0; i<N; ++i)
    for(j=0; j<10; ++j)
       sum1 += *(PtrA[i] + j);

 return sum1;
}
```

Pointer casting is supported for synthesis if native C/C++ types are used. In the following code example, type `int` is cast to type `char`.

```
#define N 1024

typedef int data_t;
typedef char dint_t;

data_t pointer_cast_native (data_t index, data_t A[N]) {
 dint_t* ptr;
 data_t i =0, result = 0;
 ptr = (dint_t*)(&A[index]);

 // Sum from the indexed value as a different type
 for (i = 0; i < 4*(N/10); ++i) {
```

```
    result += *ptr;
    ptr+=1;
  }
  return result;
}
```

Vitis HLS does not support pointer casting between general types. For example, if a `struct` composite type of signed values is created, the pointer cannot be cast to assign unsigned values.

```
struct {
 short first;
 short second;
} pair;

// Not supported for synthesis
*(unsigned*)(&pair) = -1U;
```

In such cases, the values must be assigned using the native types.

```
struct {
 short first;
 short second;
} pair;

// Assigned value
pair.first = -1U;
pair.second = -1U;
```

Pointers on the Interface

Pointers can be used as arguments to the top-level function. It is important to understand how pointers are implemented during synthesis, because they can sometimes cause issues in achieving the desired RTL interface and design after synthesis. Refer to Vitis-HLS-Introductory-Examples/Modeling/Pointers on GitHub for examples of some of the following concepts.

Basic Pointers

A function with basic pointers on the top-level interface, such as shown in the following code example, produces no issues for Vitis HLS. The pointer can be synthesized to either a simple wire interface or an interface protocol using handshakes.



TIP: *To be synthesized as a FIFO interface, a pointer must be read-only or write-only.*

```
#include "pointer_basic.h"

void pointer_basic (dio_t *d) {
 static dio_t acc = 0;

 acc += *d;
 *d  = acc;
}
```

The pointer on the interface is read or written only once per function call. The test bench is shown in the following code example.

```
#include "pointer_basic.h"

int main () {
 dio_t d;
 int i, retval=0;
 FILE *fp;

 // Save the results to a file
 fp=fopen(result.dat,w);
 printf( Din Dout\n, i, d);

 // Create input data
 // Call the function to operate on the data
 for (i=0;i<4;i++) {
    d = i;
    pointer_basic(&d);
    fprintf(fp, %d \n, d);
    printf(  %d   %d\n, i, d);
 }
 fclose(fp);

 // Compare the results file with the golden results
 retval = system(diff --brief -w result.dat result.golden.dat);
 if (retval != 0) {
    printf(Test failed!!!\n);
    retval=1;
 } else {
    printf(Test passed!\n);
 }

 // Return 0 if the test
 return retval;
}
```

C and RTL simulation verify the correct operation (although not all possible cases) with this simple data set:

```
Din Dout
  0   0
  1   1
  2   3
  3   6
Test passed!
```

Pointer Arithmetic

Introducing pointer arithmetic limits the possible interfaces that can be synthesized in RTL. The following code example shows the same code, but in this instance simple pointer arithmetic is used to accumulate the data values (starting from the second value).

```
#include "pointer_arith.h"

void pointer_arith (dio_t *d) {
 static int acc = 0;
 int i;
```

```
 for (i=0;i<4;i++) {
   acc += *(d+i+1);
   *(d+i) = acc;
 }
}
```

The following code example shows the test bench that supports this example. Because the loop to perform the accumulations is now inside function `pointer_arith`, the test bench populates the address space specified by array `d[5]` with the appropriate values.

```
#include "pointer_arith.h"

int main () {
 dio_t d[5], ref[5];
 int i, retval=0;
 FILE        *fp;

 // Create input data
 for (i=0;i<5;i++) {
    d[i] = i;
    ref[i] = i;
 }

 // Call the function to operate on the data
 pointer_arith(d);

 // Save the results to a file
 fp=fopen(result.dat,w);
 printf( Din Dout\n, i, d);
 for (i=0;i<4;i++) {
    fprintf(fp, %d \n, d[i]);
    printf(  %d   %d\n, ref[i], d[i]);
 }
 fclose(fp);

 // Compare the results file with the golden results
 retval = system(diff --brief -w result.dat result.golden.dat);
 if (retval != 0) {
    printf(Test failed!!!\n);
    retval=1;
 } else {
    printf(Test passed!\n);
 }

 // Return 0 if the test
 return retval;
}
```

When simulated, this results in the following output:

```
Din Dout
  0   1
  1   3
  2   6
  3   10
Test passed!
```

The pointer arithmetic can access the pointer data out of sequence. On the other hand, wire, handshake, or FIFO interfaces can only access data in order:

- A wire interface reads data when the design is ready to consume the data or write the data when the data is ready.
- Handshake and FIFO interfaces read and write when the control signals permit the operation to proceed.

In both cases, the data must arrive (and is written) in order, starting from element zero. In the Interface with Pointer Arithmetic example, the code starts reading from index 1 (`i` starts at 0, 0+1=1). This is the second element from array `d[5]` in the test bench.

When this is implemented in hardware, some form of data indexing is required. Vitis HLS does not support this with wire, handshake, or FIFO interfaces.

Alternatively, the code must be modified with an array on the interface instead of a pointer, as in the following example. This can be implemented in synthesis with a RAM (`ap_memory`) interface. This interface can index the data with an address and can perform out-of-order, or non-sequential, accesses.

Wire, handshake, or FIFO interfaces can be used only on streaming data. It cannot be used with pointer arithmetic (unless it indexes the data starting at zero and then proceeds sequentially).

```
#include "array_arith.h"

void array_arith (dio_t d[5]) {
 static int acc = 0;
 int i;

 for (i=0;i<4;i++) {
    acc += d[i+1];
    d[i] = acc;
 }
}
```

Multi-Access Pointers on the Interface

IMPORTANT! *Although multi-access pointers are supported on the interface, it is strongly recommended that you implement the required behavior using the `hls::stream` class instead of multi-access pointers to avoid some of the difficulties discussed below. Details on the `hls::stream` class can be found in HLS Stream Library.*

Designs that use pointers in the argument list of the top-level function (on the interface) need special consideration when multiple accesses are performed using pointers. Multiple accesses occur when a pointer is *read from* or *written to* multiple times in the same function.

Using pointers which are accessed multiple times can introduce unexpected behavior after synthesis. In the following "bad" example pointer `d_i` is read four times and pointer `d_o` is written to twice: the pointers perform multiple accesses.

```
#include "pointer_stream_bad.h"

void pointer_stream_bad ( dout_t *d_o,  din_t *d_i) {
 din_t acc = 0;

 acc += *d_i;
 acc += *d_i;
 *d_o = acc;
 acc += *d_i;
 acc += *d_i;
 *d_o = acc;
}
```

After synthesis this code will result in an RTL design which reads the input port once and writes to the output port once. As with any standard C/C++ compiler, Vitis HLS will optimize away the redundant pointer accesses. The test bench to verify this design is shown in the following code example:

```
#include "pointer_stream_bad.h"
int main () {
din_t d_i;
dout_t d_o;
int retval=0;
FILE *fp;

// Open a file for the output results
fp=fopen(result.dat,w);

// Call the function to operate on the data
for (d_i=0;d_i<4;d_i++) {
   pointer_stream_bad(&d_o,&d_i);
   fprintf(fp, %d %d\n, d_i, d_o);
}
fclose(fp);

// Compare the results file with the golden results
retval = system(diff --brief -w result.dat result.golden.dat);
if (retval != 0) {
   printf(Test failed !!!\n);
   retval=1;
} else {
   printf(Test passed !\n);
}

// Return 0 if the test
return retval;
}
```

To implement the code as written, with the “anticipated” 4 reads on `d_i` and 2 writes to the `d_o`, the pointers must be specified as `volatile` as shown in the "pointer_stream_better" example.

```
#include "pointer_stream_better.h"

void pointer_stream_better ( volatile dout_t *d_o,  volatile din_t *d_i) {
 din_t acc = 0;

 acc += *d_i;
 acc += *d_i;
 *d_o = acc;
 acc += *d_i;
 acc += *d_i;
 *d_o = acc;
}
```

To support multi-access pointers on the interface you should take the following steps:

- Validate the C/C++ before synthesis to confirm the intent and that the C/C++ model is correct.
- The pointer argument must have the number of accesses on the port interface specified when verifying the RTL using co-simulation within Vitis HLS.

Understanding Volatile Data

The code in Multi-Access Pointers on the Interface is written with *intent* that input pointer `d_i` and output pointer `d_o` are implemented in RTL as FIFO (or handshake) interfaces to ensure that:

- Upstream producer modules supply new data each time a read is performed on RTL port `d_i`.
- Downstream consumer modules accept new data each time there is a write to RTL port `d_o`.

When this code is compiled by standard C/C++ compilers, the multiple accesses to each pointer is reduced to a single access. As far as the compiler is concerned, there is no indication that the data on `d_i` changes during the execution of the function and only the final write to `d_o` is relevant. The other writes are overwritten by the time the function completes.

Vitis HLS matches the behavior of the `gcc` compiler and optimizes these reads and writes into a single read operation and a single write operation. When the RTL is examined, there is only a single read and write operation on each port.

The fundamental issue with this design is that the test bench and design do not adequately model how you expect the RTL ports to be implemented:

- You expect RTL ports that read and write multiple times during a transaction (and can stream the data in and out).

- The test bench supplies only a single input value and returns only a single output value. A C/C++ simulation of Multi-Access Pointers on the Interface shows the following results, which demonstrates that each input is being accumulated four times. The same value is being read once and accumulated each time. It is not four separate reads.

```
Din Dout
0   0
1   4
2   8
3   12
```

To make this design read and write to the RTL ports multiple times, use a `volatile` qualifier as shown in Multi-Access Pointers on the Interface. The `volatile` qualifier tells the C/C++ compiler and Vitis HLS to make no assumptions about the pointer accesses, and to not optimize them away. That is, the data is volatile and might change.

The `volatile` qualifier:

- Prevents pointer access optimizations.
- Results in an RTL design that performs the expected four reads on input port `d_i` and two writes to output port `d_o`.

Even if the `volatile` keyword is used, the coding style of accessing a pointer multiple times still has an issue in that the function and test bench do not adequately model multiple distinct reads and writes. In this case, four reads are performed, but the same data is read four times. There are two separate writes, each with the correct data, but the test bench captures data only for the final write.



TIP: *In order to see the intermediate accesses, use `cosim_design -trace_level` to create a trace file during RTL simulation and view the trace file in the appropriate viewer.*

The Multi-Access volatile pointer interface can be implemented with wire interfaces. If a FIFO interface is specified, Vitis HLS creates an RTL test bench to stream new data on each read. Because no new data is available from the test bench, the RTL fails to verify. The test bench does not correctly model the reads and writes.

Modeling Streaming Data Interfaces

Unlike software, the concurrent nature of hardware systems allows them to take advantage of streaming data. Data is continuously supplied to the design and the design continuously outputs data. An RTL design can accept new data before the design has finished processing the existing data.

As Understanding Volatile Data shows, modeling streaming data in software is non-trivial, especially when writing software to model an existing hardware implementation (where the concurrent/streaming nature already exists and needs to be modeled).

There are several possible approaches:

- Add the `volatile` qualifier as shown in the Multi-Access Volatile Pointer Interface example. The test bench does not model unique reads and writes, and RTL simulation using the original C/C++ test bench might fail, but viewing the trace file waveforms shows that the correct reads and writes are being performed.
- Modify the code to model explicit unique reads and writes. See the following example.
- Modify the code to using a streaming data type. A streaming data type allows hardware using streaming data to be accurately modeled.

The following code example has been updated to ensure that it reads four unique values from the test bench and write two unique values. Because the pointer accesses are sequential and start at location zero, a streaming interface type can be used during synthesis.

```
#include "pointer_stream_good.h"

void pointer_stream_good ( volatile dout_t *d_o,  volatile din_t *d_i) {
 din_t acc = 0;

 acc += *d_i;
 acc += *(d_i+1);
 *d_o = acc;
 acc += *(d_i+2);
 acc += *(d_i+3);
 *(d_o+1) = acc;
}
```

The test bench is updated to model the fact that the function reads four unique values in each transaction. This new test bench models only a single transaction. To model multiple transactions, the input data set must be increased and the function called multiple times.

```
#include "pointer_stream_good.h"

int main () {
 din_t d_i[4];
 dout_t d_o[4];
     int i, retval=0;
     FILE        *fp;

 // Create input data
 for (i=0;i<4;i++) {
     d_i[i] = i;
 }

 // Call the function to operate on the data
 pointer_stream_good(d_o,d_i);

 // Save the results to a file
 fp=fopen(result.dat,w);
 for (i=0;i<4;i++) {
     if (i<2)
 fprintf(fp, %d %d\n, d_i[i], d_o[i]);
     else
 fprintf(fp, %d \n, d_i[i]);
 }
 fclose(fp);

 // Compare the results file with the golden results
```

```
 retval = system(diff --brief -w result.dat result.golden.dat);
 if (retval != 0) {
     printf(Test failed  !!!\n);
     retval=1;
 } else {
     printf(Test passed !\n);
 }

 // Return 0 if the test
 return retval;
}
```

The test bench validates the algorithm with the following results, showing that:

- There are two outputs from a single transaction.
- The outputs are an accumulation of the first two input reads, plus an accumulation of the next two input reads and the previous accumulation.

```
Din Dout
0   1
1   6
2
3
```

- The final issue to be aware of when pointers are accessed multiple time at the function interface is RTL simulation modeling.

Multi-Access Pointers and RTL Simulation

When pointers on the interface are accessed multiple times, to read or write, Vitis HLS cannot determine from the function interface how many reads or writes are performed. Neither of the arguments in the function interface informs Vitis HLS how many values are read or written.

```
void pointer_stream_good (volatile dout_t *d_o, volatile din_t *d_i)
```

Unless the code informs Vitis HLS how many values are required (for example, the maximum size of an array), the tool assumes a single value and models C/RTL co-simulation for only a single input and a single output. If the RTL ports are actually reading or writing multiple values, the RTL co-simulation stalls. RTL co-simulation models the external producer and consumer blocks that are connected to the RTL design through the port interface. If it requires more than a single value, the RTL design stalls when trying to read or write more than one value because there is currently no value to read, or no space to write.

When multi-access pointers are used at the interface, Vitis HLS must be informed of the required number of reads or writes on the interface. Manually specify the INTERFACE pragma or directive for the pointer interface, and set the `depth` option to the required depth.

For example, argument `d_i` in the code sample above requires a FIFO depth of four. This ensures RTL co-simulation provides enough values to correctly verify the RTL.

Vector Data Types

The vector data type is provided to easily model and synthesize single instruction multiple data (SIMD) type operations. Many operators are overloaded to provide SIMD behavior for vector types. The AMD Vitis™ HLS library provides the reference implementation for the `hls::vector<T, N>` type which represent a single-instruction multiple-data (SIMD) vector, as defined below.

- `T`: The type of elements that the vector holds, can be a user-defined type which must provide common arithmetic operations.
- `N`: The number of elements that the vector holds, must be a positive integer.
- The best performance is achieved when both the bit-width of `T` and `N` are integer powers of 2.

Vitis HLS provides a template type `hls::vector` that can be used to define SIMD operands. All the operation performed using this type are mapped to hardware during synthesis that will execute these operations in parallel. These operations can be carried out in a loop which can be pipelined with II=1. The following example shows how an eight element vector of integers is defined and used:

```
typedef  hls::vector<int, 8> t_int8Vec;
t_int8Vec intVectorA, intVectorB;
.
.
.
void processVecStream(hls::stream<t_int8Vec>
&inVecStream1,hls::stream<t_int8Vec> &inVecStream2, hls::stream<int8Vec>
&outVecStream)
{
    for(int i=0;i<32;i++)
    {
        #pragma HLS pipeline II=1
        t_int8Vec aVec = inVecStream1.read();
        t_int8Vec bBec = inVecStream2.read();
        //performs a vector operation on 8 integers in parallel
        t_int8Vec cVec = aVec * bVec;
        outVecStream.write(cVec);
    }
}
```

Refer to HLS Vector Library for additional information. Refer to Vitis-HLS-Introductory-Examples/Modeling/using_vectors on GitHub for an example.

Bit-Width Propagation

The primary impact of a coding style on functions is on the function arguments and interface. If the arguments to a function are sized accurately, Vitis HLS can propagate this information through the design. There is no need to create arbitrary precision types for every variable. In the following example, two integers are multiplied, but only the lower 24 bits are used for the result.

```
#include "ap_int.h"

ap_int<24> foo(int x, int y) {
 int tmp;

 tmp = (x * y);
 return tmp
}
```

When this code is synthesized, the result is a 32-bit multiplier with the output truncated to 24-bit.

If the inputs are correctly sized to 12-bit types (int12) as shown in the following code example, the final RTL uses a 24-bit multiplier.

```
#include "ap_int.h"
typedef ap_int<12> din_t;
typedef ap_int<24> dout_t;

dout_t func_sized(din_t x, din_t y) {
 int tmp;

 tmp = (x * y);
 return tmp
}
```

Using arbitrary precision types for the two function inputs is enough to ensure Vitis HLS creates a design using a 24-bit multiplier. The 12-bit types are propagated through the design. AMD recommends that you correctly size the arguments of all functions in the hierarchy so that there is no need to size local variables.

In general, when variables are driven directly from the function interface, especially from the top-level function interface, variables can prevent some optimizations from taking place. A typical case of this is when an input is used as the upper limit for a loop index.

Unsupported C/C++ Constructs

TIP: *There are other unsupported C/C++ constructs. This list is not intended to be exhaustive, but rather to provide details related to some of the unsupported constructs.*

While the HLS compiler supports a wide range of the C/C++ languages, some constructs are not synthesizable, or can result in errors further down the design flow. This section discusses areas in which coding changes must be made for the function to be synthesized and implemented in a device.

To be synthesized:

- The function and its calls must contain the entire functionality of the design.
- None of the functionality can be performed by system calls to the operating system.
- The C/C++ constructs must be of a fixed or bounded size.
- The implementation of those constructs must be unambiguous.

System Calls

System calls cannot be synthesized because they are actions that relate to performing some task upon the operating system in which the C/C++ program is running.

Vitis HLS ignores commonly-used system calls that display only data and that have no impact on the execution of the algorithm, such as `printf()` and `fprintf(stdout,)`. In general, calls to the system cannot be synthesized and should be removed from the function before synthesis. Other examples of such calls are `getc()`, `time()`, `sleep()`, all of which make calls to the operating system.

Vitis HLS defines the macro `__SYNTHESIS__` when synthesis is performed. This allows the `__SYNTHESIS__` macro to exclude non-synthesizable code from the design.

Note: Only use the `__SYNTHESIS__` macro in the code to be synthesized. Do *not* use this macro in the test bench, because it is not obeyed by C/C++ simulation or C/C++ RTL co-simulation.

CAUTION! *You must not define or undefine the `__SYNTHESIS__` macro in code or with compiler options, otherwise compilation might fail.*

In the following code example, the intermediate results from a sub-function are saved to a file on the hard drive. The macro `__SYNTHESIS__` is used to ensure the non-synthesizable files writes are ignored during synthesis.

```
#include "hier_func4.h"

int shift_func(dint_t *in1, dint_t *in2, dout_t *outA, dout_t *outB)
{
 *outA = *in1 >> 1;
 *outB = *in2 >> 2;
}

void hier_func4(din_t A, din_t B, dout_t *C, dout_t *D)
{
 dint_t apb, amb;

 sumsub_func(&A,&B,&apb,&amb);
#ifndef __SYNTHESIS__
 FILE *fp1; // The following code is ignored for synthesis
 char filename[255];
 sprintf(filename,Out_apb_%03d.dat,apb);
 fp1=fopen(filename,w);
 fprintf(fp1, %d \n, apb);
 fclose(fp1);
#endif
 shift_func(&apb,&amb,C,D);
}
```

The `__SYNTHESIS__` macro is a convenient way to exclude non-synthesizable code without removing the code itself from the function. Using such a macro does mean that the code for simulation and the code for synthesis are now different.

> **CAUTION!** *If the `__SYNTHESIS__` macro is used to change the functionality of the C/C++ code, it can result in different results between C/C++ simulation and C/C++ synthesis. Errors in such code are inherently difficult to debug. Do not use the `__SYNTHESIS__` macro to change functionality.*

Dynamic Memory Usage

Any system calls that manage memory allocation within the system, for example, `malloc()`, `alloc()`, and `free()`, are using resources that exist in the memory of the operating system and are created and released during runtime. To be able to synthesize a hardware implementation the design must be fully self-contained, specifying all required resources.

Memory allocation system calls must be removed from the design code before synthesis. Because dynamic memory operations are used to define the functionality of the design, they must be transformed into equivalent bounded representations. The following code example shows how a design using `malloc()` can be transformed into a synthesizable version and highlights two useful coding style techniques:

- The design does not use the `__SYNTHESIS__` macro.

The user-defined macro `NO_SYNTH` is used to select between the synthesizable and non-synthesizable versions. This ensures that the same code is simulated in C/C++ and synthesized in Vitis HLS.

- The pointers in the original design using `malloc()` do not need to be rewritten to work with fixed sized elements.

  Fixed sized resources can be created and the existing pointer can simply be made to point to the fixed sized resource. This technique can prevent manual recoding of the existing design.

```
#include "malloc_removed.h"
#include <stdlib.h>
//#define NO_SYNTH

dout_t malloc_removed(din_t din[N], dsel_t width) {

#ifdef NO_SYNTH
 long long *out_accum = malloc (sizeof(long long));
 int* array_local = malloc (64 * sizeof(int));
#else
 long long _out_accum;
 long long *out_accum = &_out_accum;
 int _array_local[64];
 int* array_local = &_array_local[0];
#endif
 int i,j;

 LOOP_SHIFT:for (i=0;i<N-1; i++) {
 if (i<width)
 *(array_local+i)=din[i];
 else
 *(array_local[i])=din[i]>>2;
 }

 *out_accum=0;
 LOOP_ACCUM:for (j=0;j<N-1; j++) {
 *out_accum += *(array_local+j);
 }

 return *out_accum;
}
```

Because the coding changes here impact the functionality of the design, AMD does not recommend using the `__SYNTHESIS__` macro. AMD recommends performing the following steps:

1. Add the user-defined macro `NO_SYNTH` to the code and modify the code.
2. Enable macro `NO_SYNTH`, execute the C/C++ simulation, and save the results.
3. Disable the macro `NO_SYNTH`, and execute the C/C++ simulation to verify that the results are identical.
4. Perform synthesis with the user-defined macro disabled.

This methodology ensures that the updated code is validated with C/C++ simulation and that the identical code is then synthesized. As with restrictions on dynamic memory usage in C/C++, Vitis HLS does not support (for synthesis) C/C++ objects that are dynamically created or destroyed.

Pointer Limitations

General Pointer Casting

Vitis HLS does not support general pointer casting, but supports pointer casting between native C/C++ types.

Pointer Arrays

Vitis HLS supports pointer arrays for synthesis, provided that each pointer points to a scalar or an array of scalars. Arrays of pointers cannot point to additional pointers.

Function Pointers

Function pointers are not supported.

***Note*:** Pointer to pointer is not supported.

Recursive Functions

Recursive functions cannot be synthesized. This applies to functions that can form multiple recursions:

```
unsigned foo (unsigned n)
{
    if (n == 0 || n == 1) return 1;
    return (foo(n-2) + foo(n-1));
}
```

Vitis HLS also does not support tail recursion, in which there is a finite number of function calls.

```
unsigned foo (unsigned m, unsigned n)
{
    if (m == 0) return n;
    if (n == 0) return m;
    return foo(n, m%n);
}
```

In C++, templates can implement tail recursion and can then be used for synthesizable tail-recursive designs.

CAUTION! *Virtual Functions are not supported.*

Standard Template Libraries

Many of the C++ Standard Template Libraries (STLs) contain function recursion and use dynamic memory allocation. For this reason, the STLs cannot be synthesized by Vitis HLS. The solution for STLs is to create a local function with identical functionality that does not feature recursion, dynamic memory allocation, or the dynamic creation and destruction of objects.

***Note*:** Standard data types, such as `std::complex`, are supported for synthesis. However, the `std::complex<long double>` data type is not supported in Vitis HLS and should not be used.

Undefined Behaviors

The C/C++ undefined behaviors may lead to a different behavior in simulation and synthesis. An example of this behavior is shown below:

```
for (int i=0; i<N; i++) {
  int val; // uninitialized value
  if (i==0) val=0;
  else if (cond) val=1;
  // val may have indeterminate value here
  A[i] = val; // undefined behavior
  val++; // dead code
}
```

In the above example you should not expect that A[i] gets the value of val from the previous loop iteration if neither `i==0`, nor `(cond)` are true. You should even not expect that the increment (`val++`) will happen. The same is true for scalars values obtained after complete partition.

For such C/C++ undefined behavior situations, the behavior between GCC and Vitis HLS when compiling code is likely to be different, which will lead to a mismatch during RTL/Co-simulation. This is because in GCC, compiled for CPU, `val` is often left in the same register or in the same stack location across loop iterations, and therefore the behavior is that the value of `val` is retained between loop iterations.

The solution is either to initialize val at each iteration (if this is the expected behavior) or to move the declaration of `val` above the loop, as high as necessary, so that its lifetime scope matches the intent reuse. You should not expect that the compiler will infer a specific defined RTL behavior from an undefined C/C++ behavior.

Virtual Functions and Pointers

Not supported.

Chapter 8

Interfaces of the HLS Design

After the C++ code is complete, and synthesis is converting it to an RTL design, there are elements of the hardware implementation that are also in your control. Specifically, you need to consider the inputs to and the outputs from the HLS design, the layout of memory and managing data alignment, and the execution models of the HLS design. This section discusses the following topics:

- Defining Interfaces
- Vitis HLS Memory Layout Model
- Execution Modes of HLS Designs
- Controlling Initialization and Reset Behavior

Defining Interfaces

Introduction to Interface Synthesis

The arguments of the top-level function in an HLS component are synthesized into interfaces and ports that group multiple signals to define the communication protocol between the HLS component and elements external to the design. The tool defines interfaces automatically, using industry standards to specify the protocol used. The type of interfaces that the tool chooses depends on the data type and direction of the parameters of the top-level function, the target flow for the HLS component, the default interface configuration settings as described in Interface Configuration, and any specified INTERFACE pragmas or directives.



TIP: *Interfaces can be manually assigned using the INTERFACE pragma or directive. Refer to Adding Pragmas and Directives for more information.*

The target flows supported for an HLS component include the following as described in Target Flow Overview:

- The AMD Vivado™ IP flow which is the default flow for the tool
- The Vitis Kernel flow, which is the bottom-up design flow for the Vitis Application Acceleration Development flow

You can specify the target flow when creatingan HLS component as described in Section III: Creating Vitis HLS Components.

The interface defines three elements of the kernel:

1. The interface defines channels for data to flow into or out of the HLS design. Data can flow from a variety of sources external to the kernel or IP, such as a host application, an external camera or sensor, or from another kernel or IP implemented on the AMD device. The default channels for Vitis kernels are AXI adapters as described in Interfaces for Vitis Kernel Flow.
2. The interface defines the port protocol that is used to control the flow of data through the data channel, defining when the data is valid and can be read or can be written, as defined in Port-Level Protocols for Vivado IP Flow.

   TIP: *These port protocols can be customized in the Vivado IP flow, but are set and cannot be changed in the Vitis kernel flow, in most cases.*
3. The interface also defines the execution control scheme for the HLS design, specifying the operation of the kernel or IP as pipelined or sequential, as defined in Block-Level Control Protocols.

As described in Best Practices for Designing with M_AXI Interfaces the choice and configuration of interfaces is a key to the success of your design. However, the tool tries to simplify the process by selecting default interfaces for the target flows as described above.

After synthesis completes you can review the mapping of the software arguments of your C/C++ code to hardware ports or interfaces in the *SW I/O Information* section of the Synthesis Summary report.

Interfaces for Vitis Kernel Flow

The Vitis kernel flow provides support for compiled kernel objects (`.xo`) for software control from a host application and by the Xilinx Runtime (XRT). As described in PL Kernel Properties in the *Data Center Acceleration using Vitis* (UG1700), this flow has very specific interface requirements that Vitis HLS must meet.

Vitis HLS supports memory, stream, and register interface paradigms where each paradigm follows a certain interface protocol and uses the adapter to communicate with the external world.

- Memory Paradigm (`m_axi`): the data is accessed by the kernel through memory such as DDR, HBM, PLRAM/BRAM/URAM
- Stream Paradigm (`axis`): the data is streamed into the kernel from another streaming source, such as video processor or another kernel, and can also be streamed out of the kernel.
- Register Paradigm (`s_axilite`): The data is accessed by the kernel through register interfaces and accessed by software as register reads/writes.

TIP: *The AXI protocol requires an active-Low reset. The tool will define this reset with a warning if the* `syn.rtl.reset_level` *is active-High, which is the default setting.*

The Vitis kernel flow implements the following interfaces by default:

C-argument type	Paradigm	Interface protocol (I/O/Inout)
Scalar(pass by value)	Register	AXI4-Lite (`s_axilite`)
Array	Memory	AXI4 Memory Mapped (`m_axi`)
Pointer to array	Memory	`m_axi`
Pointer to scalar	Register	`s_axilite`
Reference	Register	`s_axilite`
hls::stream	Stream	AXI4-Stream (`axis`)

As you can see from the table above, a pointer to an array is implemented as an `m_axi` interface for data transfer, while a pointer to a scalar is implemented using the `s_axilite` interface. A scalar value passed as a constant does not need read access, while a pointer to a scalar value needs both read/write access. The `s_axilite` interface implements an additional internal protocol depending upon the C argument type. This internal implementation can be controlled using Port-Level Protocols for Vivado IP Flow. However, you should not modify the default port protocols in the Vitis kernel flow unless necessary.

***Note*:** Vitis HLS will not automatically infer the default interfaces for the member elements of a struct/class when the elements require different interface types. For example, when one element of a struct requires a stream interface while another member element requires an `s_axilite` interface. You must explicitly define an INTERFACE pragma for each element of the struct instead of relying on the default interface assignment. If no INTERFACE pragma or directive is defined Vitis HLS will issue the following error message:

```
ERROR: [HLS 214-312] Vitis mode requires explicit INTERFACE
pragmas for structs in the interface. Please add one INTERFACE pragma for
each struct
member field for argument 'd' of function 'dut(A&)' (example.cpp:19:0)
```

The default execution mode for Vitis kernel flow is pipelined execution, which enables overlapping execution of a kernel to improve throughput. This is specified by the `ap_ctrl_chain` block control protocol on the `s_axilite` interface.

TIP: *The Vitis environment supports kernels with all of the supported block control protocols as described in Block-Level Control Protocols.*

The `vadd` function in the following code provides an example of interface synthesis.

```
#define VDATA_SIZE 16

typedef struct v_datatype { unsigned int data[VDATA_SIZE]; } v_dt;

extern "C" {
void vadd(const v_dt* in1, // Read-Only Vector 1
          const v_dt* in2, // Read-Only Vector 2
```

```
          v_dt* out_r, // Output Result for Addition
          const unsigned int size // Size in integer
) {

   unsigned int vSize = ((size - 1) / VDATA_SIZE) + 1;

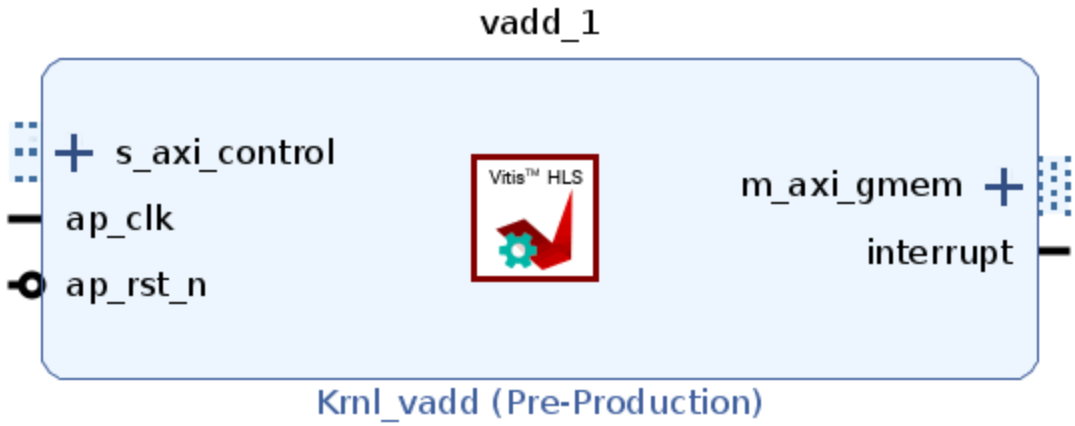
   // Auto-pipeline is going to apply pipeline to this loop
   vadd1:
   for (int i = 0; i < vSize; i++) {
      vadd2:
      for (int k = 0; k < VDATA_SIZE; k++) {
         out_r[i].data[k] = in1[i].data[k] + in2[i].data[k];
      }
   }
}
}
```

The `vadd` function includes:

- Two pointer inputs: `in1` and `in2`
- A pointer output: `out_r` that the results are written to
- A scalar value `size`

With the default interface synthesis settings used by Vitis HLS for the Vitis kernel flow, the design is synthesized into an RTL block with the ports and interfaces shown in the following figure.

Figure 33: **RTL Ports After Default Interface Synthesis**



The tool creates three types of interface ports on the RTL design to handle the flow of both data and control.

- Clock, Reset, and Interrupt ports: `ap_clk` and `ap_rst_n` and `interrupt` are added to the kernel.

- AXI4-Lite interface: `s_axi_control` interface which contains the scalar arguments like `size`, and manages address offsets for the m_axi interface, and defines the block control protocol.
- AXI4 memory mapped interface: `m_axi_gmem` interface which contains the pointer arguments: `in1`, `in2`, and `out_r`.

Details of M_AXI Interfaces for Vitis

AXI4 memory-mapped (`m_axi`) interfaces allow kernels to read and write data in global memory (DDR, HBM, PLRAM), Memory-mapped interfaces are a convenient way of sharing data across different elements of the accelerated application, such as between the host and kernel, or between kernels on the accelerator card. The main advantages for `m_axi` interfaces are listed below:

- The interface has independent read and write channels
- It supports burst-based accesses
- It provides a queue for outstanding transactions
- **Understanding Burst Access:** AXI4 memory-mapped interfaces support high throughput bursts of up to 4K bytes with just a single address phase. With burst mode transfers, Vitis HLS reads or writes data using a single base address followed by multiple sequential data samples, which makes this mode capable of higher data throughput. Burst mode of operation is possible when you use a pipelined `for` loop to copy memory. Refer to Controlling AXI4 Burst Behavior or AXI Burst Transfers for more information.
- **Automatic Port Widening and Port Width Alignment:**

  As discussed in Automatic Port Width Resizing, Vitis HLS has the ability to automatically widen a port width to facilitate data transfers and improve burst access, if a burst access can be seen by the tool. Therefore all the preconditions needed for bursting, as described in AXI Burst Transfers, are also needed for port resizing.

  In the Vitis Kernel flow automatic port width resizing is enabled by default with the following configuration commands (notice that one command is specified as bits and the other is specified as bytes):

  ```
  syn.interface.m_axi_max_widen_bitwidth=512
  syn.interface.m_axi_alignment_byte_size=64
  ```

- **Rules for Offset:**

  > **IMPORTANT!** *In the Vitis kernel flow the default mode of operation is syn.interface.m_axi_offset=direct and syn.interface.default_slave_interface=s_axilite and should not be changed.*

  The correct specification of the offset will let the HLS kernel correctly integrate into the Vitis system. Refer to Offset and Modes of Operation for more information.

- **Bundle Interfaces - Performance vs. Resource Utilization:**

  By default, Vitis HLS groups function arguments with compatible options into a single `m_axi` interface adapter as described in M_AXI Bundles. Bundling ports into a single interface helps save device resources by eliminating AXI4 logic, which can be necessary when working in congested designs.

  However, a single interface bundle can limit the performance of the kernel because all the memory transfers have to go through a single interface. The `m_axi` interface has independent READ and WRITE channels, so a single interface can read and write simultaneously, though only at one location. Using multiple bundles lets you increase the bandwidth and throughput of the kernel by creating multiple interfaces to connect to memory banks.

Details of S_AXILITE Interfaces for Vitis

In C++, a function starts to process data when the function is called from a parent function. The function call is pushed onto the stack when called, and removed from the stack when processing is complete to return control to the calling function. This process ensures the parent knows the status of the child.

Since the host and kernel occupy two separate compute spaces in the Vitis kernel flow, the "stack" is managed by the Xilinx Runtime (XRT), and communication is managed through the `s_axilite` interface. The kernel is software controlled through XRT by reading and writing the control registers of an `s_axilite` interface as described in S_AXILITE Control Register Map. The interface provides the following features:

- **Control Protocols:** The block control protocol defines control registers in the `s_axilite` interface that let you set control signals to manage execution and operation of the kernel.
- **Scalar Arguments:** Scalar inputs on a kernel are typical, and can be thought of as programming constants or parameters. The host application transfers these values through the `s_axilite` interface.
- **Pointers to Scalar Arguments:** Vitis HLS lets you read to or write from a pointer to a scalar value when assigned to an `s_axilite` interface. Pointers are assigned by default to `m_axi` interfaces, so this requires you to manually assign the pointer to the `s_axilite` using the INTERFACE pragma or directive:

  ```
  int top(int *a, int *b) {
  #pragma HLS interface s_axilite port=a
  ```

- **Rules for Offset:**

  ***Note*:** The Vitis kernel flow determines the required offsets. Do not specify the `offset` option in that flow.

- **Rules for Bundle:**

The Vitis kernel flow supports only a single `s_axilite` interface, which means that all `s_axilite` interfaces must be bundled together.

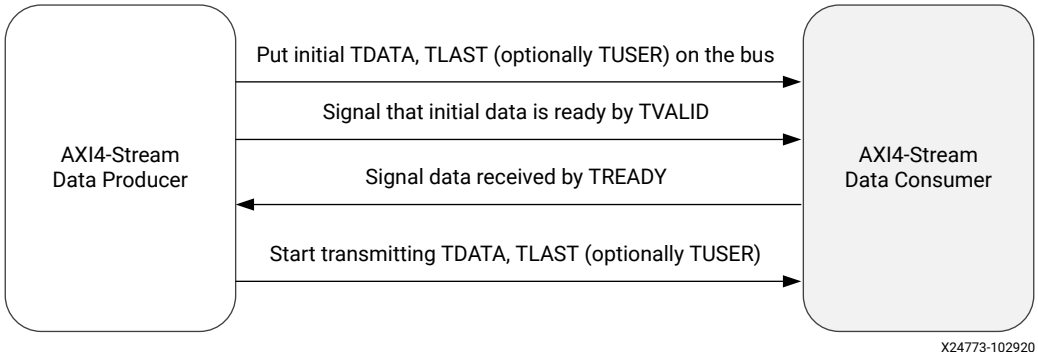
- When no bundle is specified the tool automatically creates a default bundle named `Control`.
- If for some reason you want to manually specify the bundle name, you must apply the same bundle to all `s_axilite` interfaces to create a single bundle.

Details of AXIS Interfaces for Vitis

The AXI4-Stream protocol (AXIS) defines a single uni-directional channel for streaming data in a sequential manner. The AXI4-Stream interfaces can burst an unlimited amount of data, which significantly improves performance. Unlike the AXI4 memory-mapped interface which needs an address to read/write the memory, the AXIS interface simply passes data to another AXIS interface without needing an address, and so uses fewer device resources. Combined, these features make the streaming interface a light-weight high performance interface.

The AXI4-Stream works on an industry-standard `ready/valid` handshake between a producer and consumer, as shown in the figure below. The data transfer is started once the producer sends the `TVALID` signal, and the consumer responds by sending the `TREADY` signal. This handshake of data and control should continue until either `TREADY` or `TVALID` are set low, or the producer asserts the `TLAST` signal indicating it is the last data packet of the transfer.

Figure 34: **AXI4-Stream Handshake**



IMPORTANT! *The AXIS interface can only be assigned to the top-level arguments (ports) of a kernel or IP, and cannot be assigned to the arguments of functions internal to the design. Streaming channels used inside the HLS design should use `hls::stream` and not an AXIS interface.*

You should define the streaming data type using `hls::stream<T_data_type>`, and use the `ap_axis` struct type to implement the AXIS interface. As explained in AXI4-Stream Interfaces the `ap_axis` struct lets you choose the implementation of the interface as with or without side-channels:

- AXI4-Stream Interfaces without Side-Channels implements the AXIS interface as a very light-weight interface using fewer resources

- AXI4-Stream Interfaces with Side-Channels implements a full featured interface providing greater control

> **TIP:** *You should not define your own struct for modeling the AXIS signals (side channels, TLAST, TVALID). Instead you can overload the TDATA signal for implementing your data type .*

Interfaces for Vivado IP Flow

The Vivado IP flow supports a wide variety of I/O protocols and handshakes due to the requirement of supporting FPGA design for a wide variety of applications. This flow supports a traditional system design flow where multiple IP are integrated into a system. IP can be generated through Vitis HLS. In this IP flow there are two modes of control for execution of the system:

- Software Control: The system is controlled through a software application running on an embedded Arm processor or external x86 processor, using drivers to access elements of the hardware design, and reading and writing registers in the hardware to control the execution of IP in the system.
- Self Synchronous: In this mode the IP exposes signals which are used for starting and stopping the kernel. These signals are driven by other IP or other elements of the system design that handles the execution of the IP.

The Vivado IP flow supports memory, stream, and register interface paradigms where each paradigm supports different interface protocols to communicate with the external world, as shown in the following table. Note that while the Vitis kernel flow supports only the AXI4 interface adapters, this flow supports a number of different interface types.

Table 8: **Interface Types**

Paradigm	Description	Interface Types
Memory	Data is accessed by the kernel through memory such as DDR, HBM, PLRAM/BRAM/URAM	`ap_memory`, AXI4 Memory Mapped (`m_axi`)
Stream	Data is streamed into the kernel from another streaming source, such as video processor or another kernel, and can also be streamed out of the kernel.	`ap_fifo`, AXI4-Stream (`axis`)
Register	Data is accessed by the kernel through register interfaces performed by register reads and writes.	`ap_none`, `ap_hs`, `ap_ack`, `ap_ovld`, `ap_vld`, and AXI4-Lite adapter (`s_axilite`).

The default interfaces are defined by the C-argument type in the top-level function, and the default paradigm, as shown in the following table.

Table 9: **Default Interfaces**

C-Argument Type	Supported Paradigms	Default Paradigm	Default Interface Protocol		
			Input	Output	Inout
Scalar variable (pass by value)	Register	Register	ap_none	N/A	N/A
Array	Memory, Stream	Memory	ap_memory	ap_memory	ap_memory
Pointer	Memory, Stream, Register	Register	ap_none	ap_vld	ap_ovld
Reference	Register	Register	ap_none	ap_vld	ap_vld
`hls::stream`	Stream	Stream	ap_fifo	ap_fifo	N/A

The default execution mode for Vivado IP flow is sequential execution, which requires the HLS IP to complete one iteration before starting the next. This is specified by the `ap_ctrl_hs` block control protocol. The control protocol can be changed as specified in Block-Level Control Protocols.

The `vadd` function in the following code provides an example of interface synthesis in the Vivado IP flow.

```
#define VDATA_SIZE 16

typedef struct v_datatype { unsigned int data[VDATA_SIZE]; } v_dt;

extern "C" {
void vadd(const v_dt* in1, // Read-Only Vector 1
          const v_dt* in2, // Read-Only Vector 2
          v_dt* out_r, // Output Result for Addition
          const unsigned int size // Size in integer
) {

   unsigned int vSize = ((size - 1) / VDATA_SIZE) + 1;

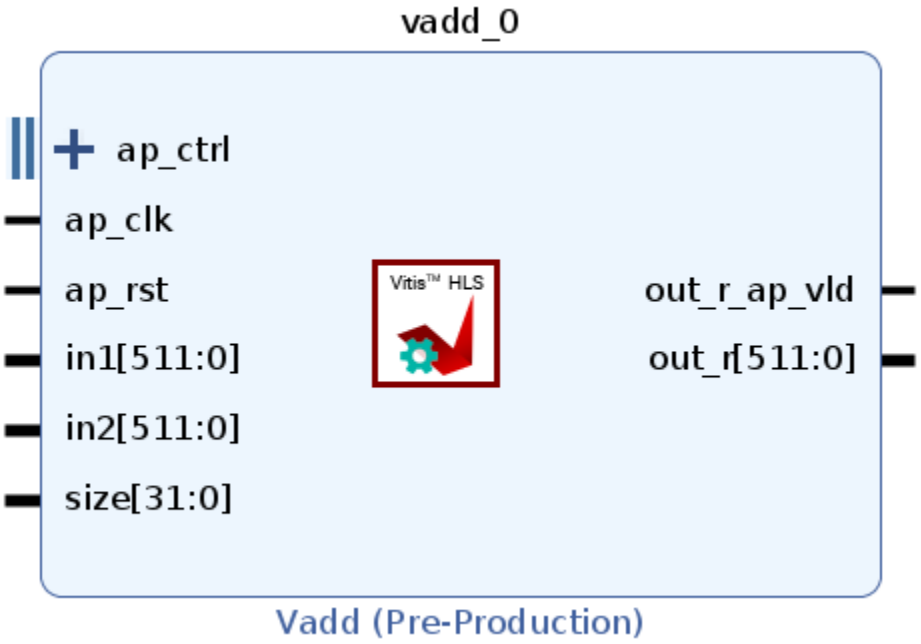
   // Auto-pipeline is going to apply pipeline to this loop
   vadd1:
   for (int i = 0; i < vSize; i++) {
      vadd2:
      for (int k = 0; k < VDATA_SIZE; k++) {
         out_r[i].data[k] = in1[i].data[k] + in2[i].data[k];
      }
   }
}
}
```

The `vadd` function includes:

- Two pointer inputs: `in1` and `in2`
- A pointer: `out_r` that the results are written to
- A scalar value `size`

With the default interface synthesis settings used for the Vivado IP flow, the design is synthesized into an RTL block with the ports and interfaces shown in the following figure.

Figure 35: **RTL Ports After Default Interface Synthesis**



In the default Vivado IP flow the tool creates three types of interface ports on the RTL design to handle the flow of both data and control.

- Clock and Reset ports: `ap_clk` and `ap_rst` are added to the kernel.
- Block-level control protocol: The `ap_ctrl` interface is implemented as an `s_axilite` interface.
- Port-level interface protocols: These are created for each argument in the top-level function and the function return (if the function returns a value). As explained in the table above most of the arguments use a port protocol of `ap_none`, and so have no control signals. In the `vadd` example above these ports include: `in1`, `in2`, and `size`. However, the `out_r_o` output port uses the `ap_vld` protocol and so is associated with the `out_r_o_ap_vld` signal.

AP_Memory in the Vivado IP Flow

The `ap_memory` is the default interface for the memory paradigm described in the tables above. In the Vivado IP flow it is used for communicating with memory resources such as BRAM and URAM. The `ap_memory` protocol also follows the address and data phase. The protocol initially requests to read/write the resource and waits until it receives an acknowledgment of the resource availability. It then initiates the data transfer phase of read/write.

An important consideration for `ap_memory` is that it can only perform a single beat data transfer to a single address, which is different from `m_axi` which can do burst accesses. This makes the `ap_memory` a lightweight protocol, compared to the others.

- Memory Resources: By default Vitis HLS implements a protocol to communicate with a single-port RAM resource. You can control the implementation of the protocol by specifying the `storage_type` as part of the INTERFACE pragma or directive. The `storage_type` lets you explicitly define which type of RAM is used, and which RAM ports are created (single-port or dual-port). If no `storage_type` is specified Vitis HLS uses:
  - A single-port RAM by default.
  - A dual-port RAM if it reduces the initiation interval or latency.

M_AXI Interfaces in the Vivado IP Flow

AXI4 memory-mapped (`m_axi`) interfaces allow an IP to read and write data in global memory (DDR, HBM, PLRAM), Memory-mapped interfaces are a convenient way of sharing data across multiple IP. The main advantages for `m_axi` interfaces are listed below:

- The interface has independent read and write channels
- It supports burst-based accesses
- It provides a queue for outstanding transactions
- **Understanding Burst Access:** AXI4 memory-mapped interfaces support high throughput bursts of up to 4K bytes with just a single address phase. With burst mode transfers, Vitis HLS reads or writes data using a single base address followed by multiple sequential data samples, which makes this mode capable of higher data throughput. Burst mode of operation is possible when you use the C `memcpy` function or a pipelined `for` loop. Refer to Controlling AXI4 Burst Behavior or AXI Burst Transfers for more information.
- **Automatic Port Widening and Port Width Alignment:**

  As discussed in Automatic Port Width Resizing, Vitis HLS has the ability to automatically widen a port width to facilitate data transfers and improve burst access when all the preconditions needed for bursting are present. In the Vivado IP flow the following configuration settings disable automatic port width resizing by default. To enable this feature you must change these configuration options (notice that one command is specified as bits and the other is specified as bytes):

  ```
  syn.interface.m_axi_max_widen_bitwidth=0
  syn.interface.m_axi_alignment_byte_size=0
  ```

- **Specifying Alignment for Vivado IP mode:**

  The alignment for an `m_axi` port allows the port to read and write memory according to the specified alignment. Choosing the correct alignment is important as it will impact performance in the best case, and can impact functionality in the worst case.

Aligned memory access means that the pointer (or the start address of the data) is a multiple of a type-specific value called the alignment. The alignment is the natural address multiple where the type must be or should be stored (for example for performance reasons) on a Memory. For example, standard 32-bit architecture stores words of 32 bits, each of 4 bytes in the memory. The data is aligned to one-word or 4-byte boundary.

The alignment should be consistent in the system. The alignment is determined when the IP is operating in AXI4 master mode and should be specified, like the 32-bit architecture with 4-byte alignment. When the IP is operating in slave mode the alignment should match the alignment of the master.

- **Rules for Offset:**

  The default for `m_axi` offset is *offset=direct* and *syn.interface.default_slave_interface=s_axilite*. However, in the Vivado IP flow you can change it as described in Offset and Modes of Operation.

- **Bundle Interfaces - Performance vs. Resource Utilization:**

  By default, Vitis HLS groups function arguments with compatible options into a single `m_axi` interface adapter as described in M_AXI Bundles. Bundling ports into a single interface helps save device resources by eliminating AXI4 logic, which can be necessary when working in congested designs.

  However, a single interface bundle can limit the performance of the IP because all the memory transfers have to go through a single interface. The `m_axi` interface has independent READ and WRITE channels, so a single interface can read and write simultaneously, though only at one location. Using multiple bundles lets you increase performance by creating multiple interfaces to connect to memory banks.

S_AXILITE in the Vivado IP Flow

In the Vivado IP flow, the default execution control is managed by register reads and writes through an `s_axilite` interface using the default `ap_ctrl_hs` control protocol. The IP is software controlled by reading and writing the control registers of an `s_axilite` interface as described in S_AXILITE Control Register Map.

The `s_axilite` interface provides the following features:

- **Control Protocols:** The block control protocol as specified in Block-Level Control Protocols.
- **Scalar Arguments:** Scalar arguments from the top-level function can be mapped to an `s_axilite` interface which creates a register for the value as described in S_AXILITE Control Register Map. The software can perform reads/writes to this register space.
- **Rules for Offset:** The Vivado IP flow defines the size, or range of addresses assigned to a port based on the data type of the associated C-argument in the top-level function. However, the tool also lets you manually define the offset size as described in S_AXILITE Offset Option.

- **Rules for Bundle:** In the Vivado IP flow you can specify multiple bundles using the `s_axilite` interface, and this will create a separate interface adapter for each bundle you have defined. However, there are some rules related to using multiple bundles that you should be familiar with as explained in S_AXILITE Bundle Rules.

AP_FIFO in the Vivado IP Flow

In the Vivado IP flow, the `ap_fifo` interface protocol is the default interface for the streaming paradigm on the interface for communication with a memory resource FIFO, and can also be used as a communication channel between different functions inside the IP. This protocol should only be used if the data is accessed sequentially, and AMD *strongly recommends* using the `hls::stream<data type>` which implements a FIFO.

TIP: *The `<data type>` should not be the same as the `T_data_type`, which should only be used on the interface.*

AXIS Interfaces in the Vivado IP Flow

The AXI4-Stream protocol (`axis`) is an alternative for streaming interfaces, and defines a single uni-directional channel for streaming data in a sequential manner. Unlike the `m_axi` protocol, the AXI4-Stream interfaces can burst an unlimited amount of data, which significantly improves performance. Unlike the AXI4 memory-mapped interface which needs an address to read/write the memory, the `axis` interface simply passes data to another `axis` interface without needing an address, and so uses fewer device resources. Combined, these features make the streaming interface a light-weight high performance interface as described in AXI4-Stream Interfaces.

AXI Adapter Interface Protocols

IMPORTANT! *As discussed in Interfaces for Vitis Kernel Flow, the AXI4 adapter interfaces are the default interfaces used by Vitis HLS for the Vitis Application Acceleration Development flow, though they are also supported in the Vivado IP flow. The AXI4-Stream Accelerator Adapter is a soft AMD LogiCORE™ Intellectual Property (IP) core used as a infrastructure block for connecting hardware accelerators to embedded CPUs.*

The AXI4 interfaces supported by Vitis HLS include the AXI4-Stream interface (`axis`), AXI4-Lite (`s_axilite`), and AXI4 master (`m_axi`) interfaces. For a complete description of the AXI4 interfaces, including timing and ports, see the *Vivado Design Suite: AXI Reference Guide* (UG1037). As described in the following sections, the AXI4 interfaces implement an adapter to manage communication according to the protocol. None of the other available Vitis HLS interfaces implement such an adapter.

- **m_axi:** Specify on arrays and pointers (and references in C++) only. The `m_axi` mode specifies an AXI4 Memory Mapped interface.

  TIP: *You can group bundle arguments into a single `m_axi` interface.*

- **s_axilite:** Specify this protocol on any type of argument except streams. The `s_axilite` mode specifies an AXI4-Lite slave interface.

  TIP: *You can bundle multiple arguments into a single `s_axilite` interface.*

- **axis:** Specify this protocol on input arguments or output arguments only, not on input/output arguments. The `axis` mode specifies an AXI4-Stream interface.

TIP: *The AXI protocol requires an active-Low reset. If your design uses AXI interfaces the tool will define this reset level with a warning if the `syn.rtl.reset_level` is active-High, which is the default setting.*

AXI4 Master Interface

AXI4 memory-mapped (`m_axi`) interfaces allow kernels to read and write data in global memory (DDR, HBM, PLRAM). Memory-mapped interfaces are a convenient way of sharing data across different elements of the accelerated application, such as between the host and kernel, or between kernels on the accelerator card. Refer to Vitis-HLS-Introductory-Examples/Interface/Memory on GitHub for examples of some of these concepts.

The main advantages for `m_axi` interfaces are listed below:

- The interface has a separate and independent read and write channels
- It supports burst-based accesses with potential performance of ~17 GBps
- It provides support for outstanding transactions
- It can include cache to improve performance as indicated by the CACHE pragma or directive.

In the Vitis Kernel flow the `m_axi` interface is assigned by default to pointer and array arguments. In this flow it supports the following default features:

- Pointer and array arguments are automatically mapped to the `m_axi` interface
- The default mode of operation is `offset=slave` in the Vitis flow and should not be changed
- All pointer and array arguments are mapped to a single interface bundle to conserve device resources, and ports share read and write access across the time it is active
- The default alignment in the Vitis flow is set to 64 bytes
- The maximum read/write burst length is set to 16 by default

While not used by default in the Vivado IP flow, when the `m_axi` interface is specified it has the following default features:

- The default operation mode is *offset=off* but you can change it as described in Offset and Modes of Operation

- Assigned pointer and array arguments are mapped to a single interface bundle to conserve device resources, and share the interface across the time it is active
- The default alignment in Vivado IP flow is set to 1 byte
- The maximum read/write burst length is set to 16 by default

In both the Vivado IP flow and Vitis kernel flow, global default values for the `m_axi` interface are defined by the `syn.interface.xxx` commands as described in Interface Configuration. The INTERFACE pragma or directive can be used to modify default values as needed for specific interfaces. Some customization can help improve design performance as described in Optimizing AXI System Performance. You can also specify the CACHE pragma or directive as described in pragma HLS cache to improve performance of the `m_axi` interface.

You can use an AXI4 master interface on array or pointer/reference arguments, which Vitis HLS implements in one of the following modes:

- Individual data transfers
- Burst mode data transfers

With individual data transfers, Vitis HLS reads or writes a single element of data for each address. The following example shows a single read and single write operation. In this example, Vitis HLS generates an address on the AXI interface to read a single data value and an address to write a single data value. The interface transfers one data value per address.

```
void bus (int *d) {
 static int acc = 0;

 acc += *d;
 *d  = acc;
}
```

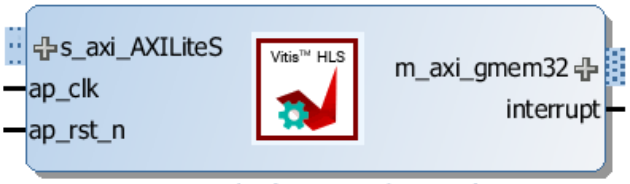
With burst mode transfers, Vitis HLS reads or writes data using a single base address followed by multiple sequential data samples, which makes this mode capable of higher data throughput. Burst mode of operation is possible when using a pipelined `for` loop. Refer to AXI Burst Transfers for more information.

IMPORTANT! *The use of `memcpy` is discouraged because it can not be inlined or pipelined. Its use can also be problematic because it changes the type of the argument into `char`, which can lead to errors if `array_partition`, `array_reshape`, or struct `disaggregate` is used. Instead you are recommended to write your own version of `memcpy` with explicit arrays and loops to provide better control.*

When the prior example is synthesized, it results in the interfaces shown in the following figure.

***Note*:** In this figure, the AXI4 interfaces are collapsed.

Figure 36: **AXI4 Interface**



When using a `for` loop to implement burst reads or writes, follow these requirements:

- Pipeline the loop
- Access addresses in increasing order
- Do not place accesses inside a conditional statement
- For nested loops, do not manually flatten loops (using LOOP_FLATTEN pragma or directive), because this inhibits the burst operation

***Note*:** Only one read and one write is allowed in a `for` loop unless the ports are bundled in different AXI ports.

Offset and Modes of Operation

IMPORTANT! *In the Vitis kernel flow the default mode of operation is offset=slave and should not be changed. In the Vivado IP flow the default is also offset=slave, but can be changed.*

The AXI4 Master interface has a read/write address channel that can be used to read/write specific addresses. By default the `m_axi` interface starts all read and write operations from the address `0x00000000`. For example, given the following code, the design reads data from addresses `0x00000000` to `0x000000C7` (50 32-bit words, gives 200 bytes), which represents 50 address values. The design then writes data back to the same addresses.

```
#include <stdio.h>
#include <string.h>

void example(int *a){

#pragma HLS INTERFACE mode=m_axi port=a depth=50

  int i;
  int buff[50];

  //The use of memcpy is discouraged because multiple calls of memcpy
cannot be
  //pipelined and will be scheduled sequentially. Use a dedicated for loop
instead
  //memcpy(buff,(const int*)a,50*sizeof(int));
  //Burst read input
  for(i=0; i<50; i++){
    buff[i] = a[i];
  }

  // Compute value
  for(i=0; i < 50; i++){
```

```
    buff[i] = buff[i] + 100;
  }

  //Burst write output
  for(i=0; i<50; i++){
    a[i] = buff[i];
  }
}
```

The HLS compiler provides the capability to let the base address be configured statically in the Vivado IP for instance, or dynamically by the software application or another IP during runtime.

The `m_axi` interface can be both a master initiating transactions, and also a slave interface that receives the data and sends acknowledgment. Depending on the mode specified with the `offset` option of the INTERFACE pragma, an HLS IP can use multiple approaches to set the base address.



TIP: *The `syn.interface.m_axi_offset` configuration command provides a global setting for the offset, that can be overridden for specific `m_axi` interfaces using the INTERFACE pragma `offset` option.*

- *Master Mode*: When acting as a master interface with different `offset` options, the `m_axi` interface start address can be either hard-coded or set at runtime.
  - `offset=off`: Sets the base address of the `m_axi` interface to 0x00000000 and it cannot be changed in the Vivado IP integrator. One disadvantage with this approach is that you cannot change the base address during runtime. See Customizing AXI4 Master Interfaces in IP Integrator for setting the base address.

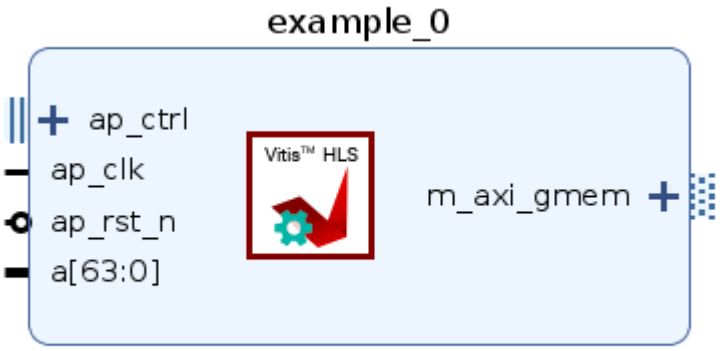
    The following shows the INTERFACE pragma setting `offset=off` for the function.

    ```
    void example(int *a){
    #pragma HLS INTERFACE m_axi depth=50 port=a offset=off
    ...
    }
    ```

  - `offset=direct`: Vitis HLS generates a port on the IP for setting the address. Note the addition of the `a` port as shown in the figure below. This lets you update the address at runtime, so you can have one `m_axi` interface reading and writing different locations. For example, an HLS module that reads data from an ADC into RAM, and an HLS module that processes that data. Because you can change the address on the module, while one HLS module is processing the initial dataset the other module can be reading more data into different address.

    ```
    void example(int *a){
    #pragma HLS INTERFACE m_axi depth=50 port=a offset=direct
    ...
    }
    ```

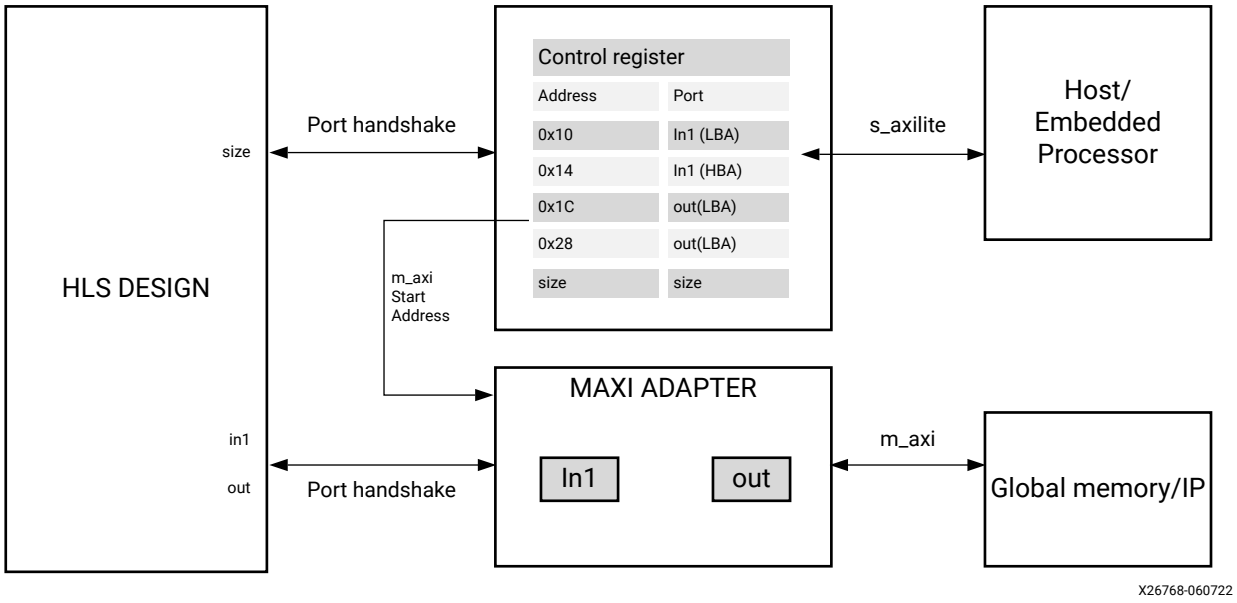
Figure 37: **offset=direct**



- *Slave Mode*: The slave mode for an interface is set with `offset=slave`. In this mode the IP will be controlled by the host application, or the micro-controller through the `s_axilite` interface. This is the default for both the Vitis kernel flow, and the Vivado IP flow. Here is the flow of operation:
  1. initially, the Host/CPU will start the IP or kernel using the block-level control protocol which is mapped to the `s_axilite` adapter.
  2. The host will send address offsets for the `m_axi` interfaces through the `s_axilite` adapter.
  3. The `m_axi` adapter will read the start address from the `s_axilite` adapter and store it in a queue.
  4. The HLS design starts to read the data from the global memory.

As shown in the following figure, the HLS design will have both the `s_axilite` adapter for the base address, and the `m_axi` to perform read and write transfer to the global memory.

Figure 38: **AXI Adapters in Slave Mode**



Offset Rules

The following are rules associated with the `offset` option:

- Fully Specified Offset: When the user explicitly sets the offset value the tool uses the specified settings. The user can also set different offset values for different `m_axi` interfaces in the design, and the tool will use the specified offsets.

```
#pragma HLS INTERFACE s_axilite port=return
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=out offset=direct
#pragma HLS INTERFACE mode=m_axi bundle=BUS_B port=in1 offset=slave
#pragma HLS INTERFACE mode=m_axi bundle=BUS_C port=in2 offset=off
```

- No Offset Specified: If there are no offsets specified in the INTERFACE pragma, the tool will defer to the setting specified by `syn.interface.m_axi_offset`.

  Note: If the global `syn.interface.m_axi_offset` setting is specified, and the design has an `s_axilite` interface, the global setting is ignored and `offset=slave` is assumed.

```
void top(int *a) {
#pragma HLS interface mode=m_axi port=a
#pragma HLS interface mode=s_axilite port=a
}
```

M_AXI Bundles

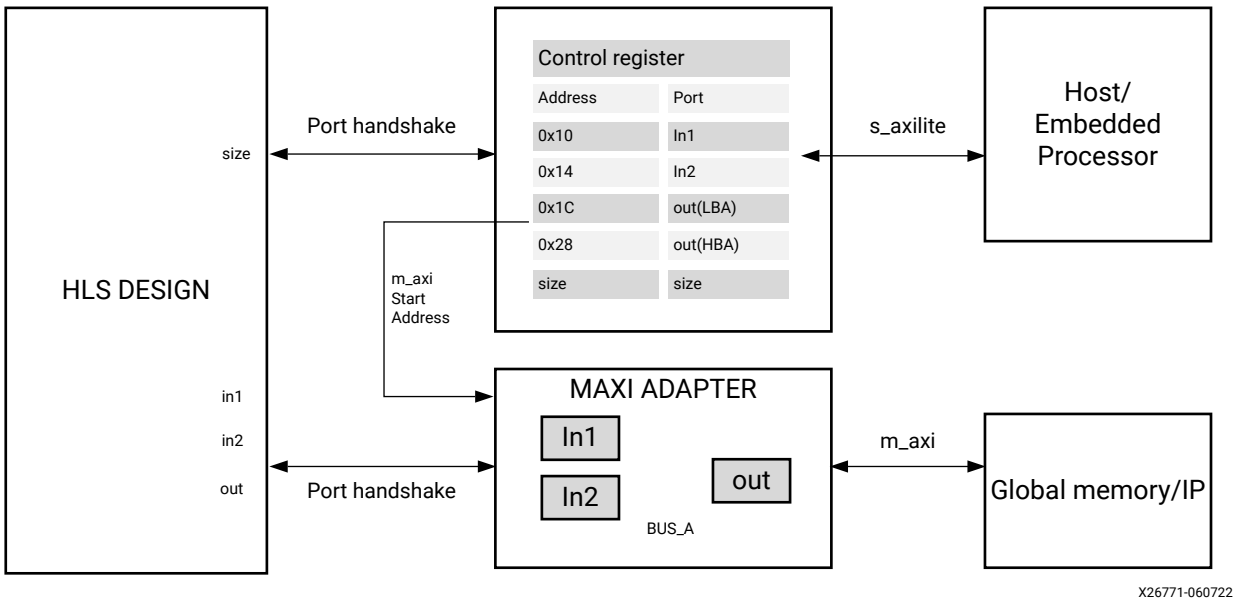
The Vitis Unified IDE and `v++` command groups function arguments with compatible options into a single `m_axi` interface adapter. Bundling ports into a single interface helps save FPGA resources by eliminating AXI logic, but it can limit the performance of the kernel because all the memory transfers have to go through a single interface. The `m_axi` interface has independent READ and WRITE channels, so a single interface can read and write simultaneously, though only at one location. Using multiple bundles the bandwidth and throughput of the kernel can be increased by creating multiple interfaces to connect to multiple memory banks.

In the following example all the pointer arguments are grouped into a single `m_axi` adapter using the interface option `bundle=BUS_A`, and adds a single `s_axilite` adapter for the `m_axi` offsets, the scalar argument `size`, and the function return.

```
extern "C" {
void vadd(const unsigned int *in1, // Read-Only Vector 1
          const unsigned int *in2, // Read-Only Vector 2
          unsigned int *out,     // Output Result
          int size                 // Size in integer
          ) {

#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=out
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=in1
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=in2
#pragma HLS INTERFACE mode=s_axilite port=in1
#pragma HLS INTERFACE mode=s_axilite port=in2
#pragma HLS INTERFACE mode=s_axilite port=out
#pragma HLS INTERFACE mode=s_axilite port=size
#pragma HLS INTERFACE mode=s_axilite port=return
```

Figure 39: **MAXI and S_AXILITE**

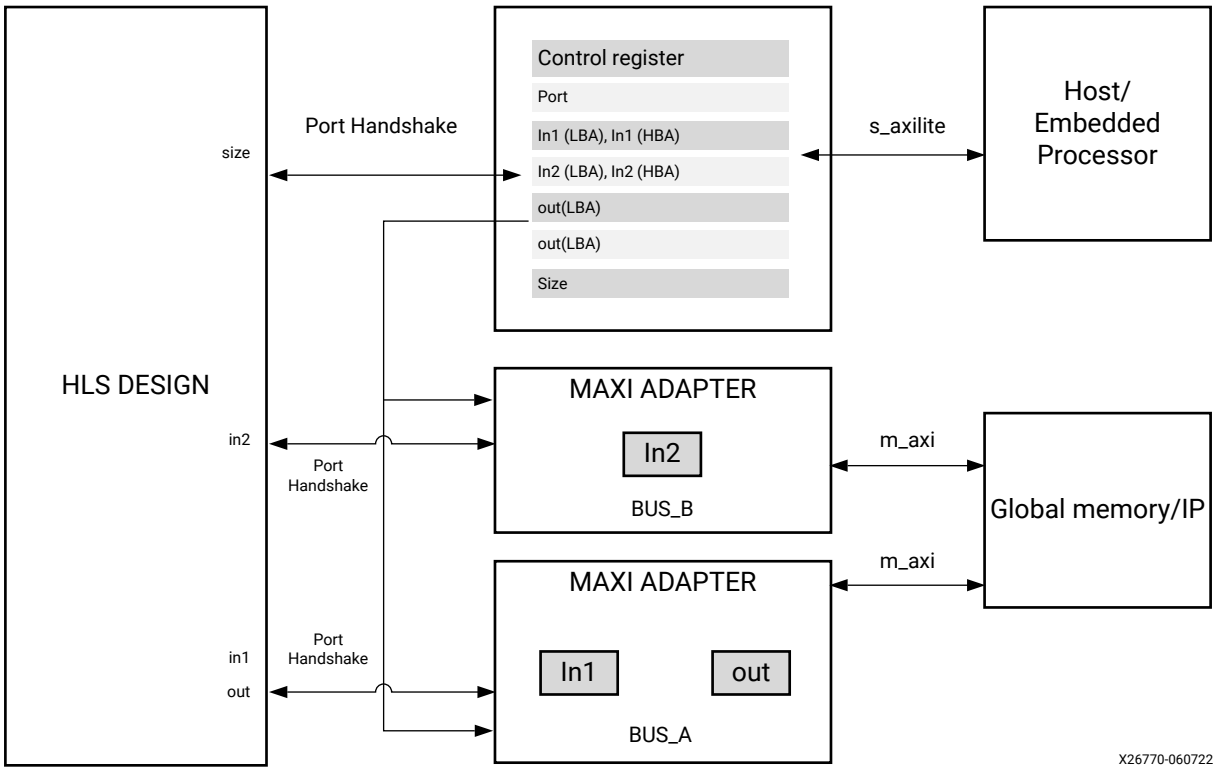


You can also choose to bundle function arguments into separate interface adapters as shown in the following code. Here the argument `in2` is grouped into a separate interface adapter with `bundle=BUS_B`. This creates a new `m_axi` interface adapter for port `in2`.

```
extern "C" {
void vadd(const unsigned int *in1, // Read-Only Vector 1
          const unsigned int *in2, // Read-Only Vector 2
          unsigned int *out,     // Output Result
          int size                 // Size in integer
          ) {

#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=out
#pragma HLS INTERFACE mode=m_axi bundle=BUS_A port=in1
#pragma HLS INTERFACE mode=m_axi bundle=BUS_B port=in2
#pragma HLS INTERFACE mode=s_axilite port=in1
#pragma HLS INTERFACE mode=s_axilite port=in2
#pragma HLS INTERFACE mode=s_axilite port=out
#pragma HLS INTERFACE mode=s_axilite port=size
#pragma HLS INTERFACE mode=s_axilite port=return
```

Figure 40: **2 MAXI Bundles**



Bundle Rules

The global configuration command `syn.interface.m_axi_auto_max_ports=false` will limit the number of interface bundles to the minimum required. It will allow the tool to group compatible ports into a single `m_axi` interface. The default setting for this command is disabled (false), but you can enable it to maximize bandwidth by creating a separate `m_axi` adapter for each port.

With `m_axi_auto_max_ports` disabled, the following are some rules for how the tool handles bundles under different circumstances:

1. **Default Bundle Name**: The tool groups all interface ports with no bundle name into a single `m_axi` interface port using the tool default name `bundle=<default>`, and names the RTL port `m_axi_<default>`. The following pragmas:

   ```
   #pragma HLS INTERFACE mode=m_axi port=a depth=50
   #pragma HLS INTERFACE mode=m_axi port=a depth=50
   #pragma HLS INTERFACE mode=m_axi port=a depth=50
   ```

   Result in the following messages:

   ```
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to
   'm_axi'.
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to
   'm_axi'.
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to
   'm_axi'.
   ```

2. User-Specified Bundle Names: The tool groups all interface ports with the same user-specified `bundle=<string>` into the same `m_axi` interface port, and names the RTL port the value specified by `m_axi_<string>`. Ports without `bundle` assignments are grouped into the default bundle as described above. The following pragmas:

   ```
   #pragma HLS INTERFACE mode=m_axi port=a depth=50 bundle=BUS_A
   #pragma HLS INTERFACE mode=m_axi port=b depth=50
   #pragma HLS INTERFACE mode=m_axi port=c depth=50
   ```

   Result in the following messages:

   ```
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/BUS_A' to
   'm_axi'.
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to
   'm_axi'.
   INFO: [RTGEN 206-500] Setting interface mode on port 'example/gmem' to
   'm_axi'.
   ```

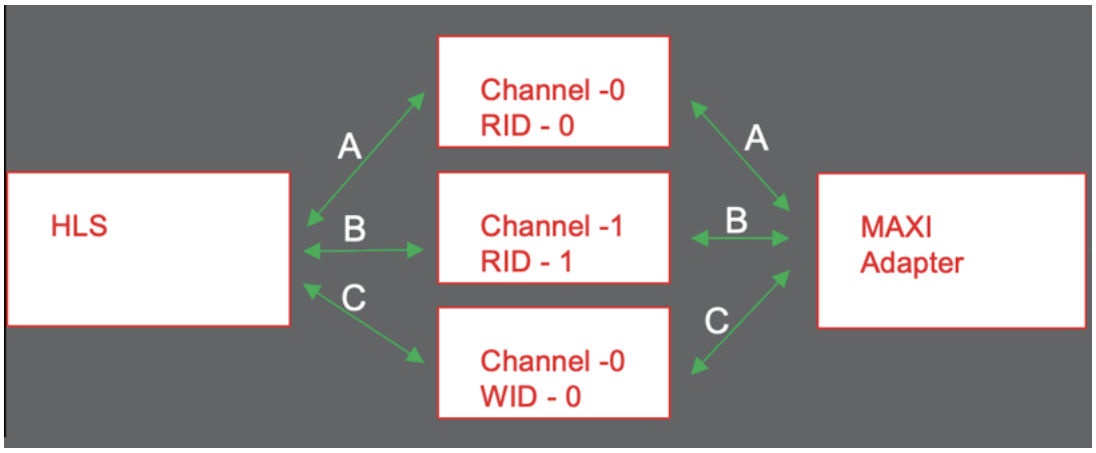


IMPORTANT! *If you bundle incompatible interfaces the tool issues a message and ignores the bundle assignment.*

M_AXI Channels

M_AXI channels implement a separate channel for each pointer argument mapped to a single AXI interface, rather than requiring a separate adapter.

Figure 41: **Maxi Adapter**



This enables the following benefits:

- The kernel uses fewer M_AXI adapters and consumes fewer hardware resources.
- Multiple pointer arguments mapped to a single AXI interface can be used inside a dataflow region.
- Using a unique AXI ID for each pointer argument enables burst interleaving which can result in higher utilization of the AXI bus bandwidth.

There are two methods to enable this feature in your design:

- Enable globally on `m_axi` interfaces using the `syn.interface.m_axi_auto_id_channel=true` configuration command as described in Interface Configuration. The HLS tool automatically adds channels to the `m_axi` adapter when this is enabled.
- Enable on a specific `m_axi` interface using the `channel` option of the INTERFACE pragma or directive as described in syn.directive.interface.

M_AXI Resources

The AXI Master Adapter converts the customized AXI commands from the HLS scheduler to standard AXI AMBA® protocol and sends them to the external memory. The MAXI adapter uses resources such as FIFO to store the requests/Data and ack. Here is the summary of the modules and the resource they consume:

- Write Module: The bus write modules performs the write operations.
  - `FIFO_wreq`: This FIFO module stores the future write requests. When the AW channel is available a new write request to global memory will be popped out of this FIFO.
  - `buff_wdata`: This FIFO stores the future write data that needs to be sent to the global memory. When the W channel is available and AXI protocol conditions are met, the write data of size= burst_length will be popped out of this FIFO and sent to the global memory.

    - `FIFO_resp`: This module is responsible for controlling the number of pipelined write responses sent to the module.
- Read Module: These modules perform the read operations. It uses the following resources
    - `FIFO_rreq`: This FIFO module stores the future write requests. When the AR channel is free a read request to global memory will be popped out of this FIFO.
    - `buff_rdata`: This FIFO stores the read data that are received from the global memory.

The device resource consumption of the M_AXI adapter is a sum of all the write modules (size of the `FIFO_wreq module`, `buff_wdata`, and size of FIFO_ resp) and the sum of all read modules. In general, the size of the FIFO is calculated as = Width * Depth. When you refer to a 1KB FIFO storage it can be one of the configurations such as 32*32, 8*64 etc, which are selected according to the design specification. Similarly, the adapter FIFO storage can be globally configured for the design using the following configuration commands:

- `syn.interface.m_axi_latency`
- `syn.interface.m_axi_max_read_burst_length`/ `syn.interface.m_axi_max_write_burst_length`
- `syn.interface.m_axi_num_read_outstanding`/ `syn.interface.m_axi_num_write_outstanding`
- `syn.interface.m_axi_addr64`



TIP: *You can also use similar options on the INTERFACE pragma or directive to configure specific m_axi interfaces.*

These configuration options control the width and depth of the FIFO as shown below.

- Size of the `FIFO_wreq/rreq module` = (width(`syn.interface.m_axi_addr64`)) * Depth(`syn.interface.m_axi_latency`)). This FIFO will be implemented as a shift register by the Vivado tool.
- Size of the `buff_wdata/buff_rdata` module = (width (port width after HLS synthesis) * Depth (`syn.interface.m_axi_num_write_outstanding` * `syn.interface.m_axi_max_write_burst_length`)).



TIP: *This FIFO by default will be implemented as block RAM, but it can be implemented in LUTRAM or URAM as determined by `syn.interface.m_axi_buffer_impl`.*

- Size of the `FIFO_resp` module = depth (`syn.interface.m_axi_num_write_outstanding`).

Controlling AXI4 Burst Behavior

An optimal AXI4 interface is one in which the design never stalls while waiting to access the bus, and after bus access is granted, the bus never stalls while waiting for the design to read/write. To create the optimal AXI4 interface, the following options are provided in the INTERFACE pragma or directive to specify the behavior of burst access and optimize the efficiency of the AXI4 interface. Refer to AXI Burst Transfers for more information on burst transfers.

In some cases, where autmatic burst access has failed, an efficient solution is to re-write the code or use manual burst as described in Using Manual Burst. If that does not work, then another solution might be to use cache memory in the AXI4 interface using the CACHE pragma or directive.



IMPORTANT! *A `volatile` qualifier on the variable prevents burst access to reado or write it.*

Some of these options use internal storage to buffer data and can have an impact on area and resources:

- `latency`: Specifies the expected latency of the AXI4 interface, allowing the design to initiate a bus request a number of cycles (latency) before the read or write is expected. If this figure is too low, the design will be ready too soon and can stall waiting for the bus. If this figure is too high, bus access might be granted but the bus can stall waiting on the design to start the access.
- `max_read_burst_length`: Specifies the maximum number of data values read during a burst transfer.
- `num_read_outstanding`: Specifies how many read requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, a FIFO of size: `num_read_outstanding*max_read_burst_length*word_size`.
- `max_write_burst_length`: Specifies the maximum number of data values written during a burst transfer.
- `num_write_outstanding`: Specifies how many write requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, a FIFO of size: `num_write_outstanding*max_write_burst_length*word_size`

The following example can be used to help explain these options:

```
 #pragma HLS interface mode=m_axi port=input offset=slave bundle=gmem0
depth=1024*1024*16/(512/8)
 latency=100
 num_read_outstanding=32
 num_write_outstanding=32
 max_read_burst_length=16
 max_write_burst_length=16
```

The interface is specified as having a latency of 100. Vitis HLS seeks to schedule the request for burst access 100 clock cycles before the design is ready to access the AXI4 bus. To further improve bus efficiency, the options `num_write_outstanding` and `num_read_outstanding` ensure the design contains enough buffering to store up to 32 read and write bursts. This allows the design to continue processing until the bus requests are serviced. Finally, the options `max_read_burst_length` and `max_write_burst_length` ensure the maximum burst size is 16 and that the AXI4 interface does not hold the bus for longer than this.

These options allow the behavior of the AXI4 interface to be optimized for the system in which it will operate. The efficiency of the operation does depend on these values being set accurately.

Automatic Port Width Resizing

In the Vitis tool flow Vitis HLS provides the ability to automatically re-size `m_axi` interface ports to 512-bits to improve burst access. However, automatic port width resizing only supports standard C data types and power of two size `struct`, where the pointer is aligned to the expected widened byte size. If the tool cannot automatically widen the port, you can manually change the port width by using Vector or Arbitrary Precision (AP) as the data type of the port.

IMPORTANT! *Structs on the interface prevent automatic widening of the port. You must break the struct into individual elements to enable this feature.*

Vitis HLS controls automatic port width resizing using the following two commands:

- `syn.interface.m_axi_max_widen_bitwidth=<N>`: Directs the tool to automatically widen bursts on M-AXI interfaces up to the specified bitwidth. The value of <N> must be a power-of-two between 0 and 1024.
- `syn.interface.m_axi_alignment_byte_size=<N>`: Note that burst widening also requires strong alignment properties. Assume pointers that are mapped to `m_axi` interfaces are at least aligned to the provided width in bytes (power of two). This can help automatic burst widening.

In the Vitis Kernel flow automatic port width resizing is enabled by default with the following:

```
syn.interface.m_axi_max_widen_bitwidth=512
syn.interface.m_axi_alignment_byte_size=64
```

In the Vivado IP flow this feature is disabled by default:

```
syn.interface.m_axi_max_widen_bitwidth=0
syn.interface.m_axi_alignment_byte_size=1
```

Automatic port width resizing will only re-size the port if a burst access can be seen by the tool. Therefore all the preconditions needed for bursting, as described in AXI Burst Transfers, are also needed for port resizing. These conditions include:

- Must be a monotonically increasing order of access (both in terms of the memory location being accessed as well as in time). You cannot access a memory location that is in between two previously accessed memory locations- aka no overlap.
- The access pattern from the global memory should be in sequential order, and with the following additional requirements:
  - The sequential accesses need to be on a primitive type or non vector power of two size aggregate type
  - The start of the sequential accesses needs to be aligned to the widen word size
  - The length of the sequential accesses needs to be divisible by the widen factor

The following code example is used in the calculations that follow:

```
vadd_pipeline:
  for (int i = 0; i < iterations; i++) {
#pragma HLS LOOP_TRIPCOUNT min = c_len/c_n max = c_len/c_n

  // Pipelining loops that access only one variable is the ideal way to
  // increase the global memory bandwidth.
  read_a:
    for (int x = 0; x < N; ++x) {
#pragma HLS LOOP_TRIPCOUNT min = c_n max = c_n
#pragma HLS PIPELINE II = 1
      result[x] = a[i * N + x];
    }

  read_b:
    for (int x = 0; x < N; ++x) {
#pragma HLS LOOP_TRIPCOUNT min = c_n max = c_n
#pragma HLS PIPELINE II = 1
      result[x] += b[i * N + x];
    }

  write_c:
    for (int x = 0; x < N; ++x) {
#pragma HLS LOOP_TRIPCOUNT min = c_n max = c_n
#pragma HLS PIPELINE II = 1
      c[i * N + x] = result[x];
    }
  }
}
}
```

The width of the automatic optimization for the code above is performed in three steps:

1. The tool checks for the number of access patterns in the read_a loop. There is one access during one loop iteration, so the optimization determines the interface bit-width as 32= 32 *1 (bitwidth of the int variable * accesses).
2. The tool tries to reach the default max specified by the `config_interface -m_axi_max_widen_bitwidth 512`, using the following expression terms:

```
length = (ceil((loop-bound of index inner loops) *
(loop-bound of index - outer loops)) * #(of access-patterns))
```

- In the above code, the tool supports imperfect loop nest. If a and b are bundled to the same port, the tool will not extend the bursts on a and b due to conflicting. If a and b are bundled to different ports, the tool will extend the bursts on a and b to the outer loop. Therefore the formula will be shortened to:

  ```
  length = (ceil((loop-bound of index inner loops)) * #(of access-patterns))
  ```

  or: length = ceil(128) *32 = 4096

3. Is the calculated length a power of 2? If Yes, then the length will be capped to the width specified by `syn.interface.m_axi_max_widen_bitwidth`.

There are some pros and cons to using the automatic port width resizing which you should consider when using this feature. This feature improves the access throughput, instead of the data type size. It also adds more resources as it needs to buffer the huge vector and might need to shift the data accordingly to the data path size.

Creating an AXI4 Interface with 32-bit Address

By default, Vitis HLS implements the AXI4 port with a 64-bit address bus. However, some devices such as the Zynq 7000 have a 32 bit address bus. In this case you can implement the AXI4 interface with a 32-bit address bus by setting the `syn.interface.m_axi_addr64=0` configuration command to disable the 64-bit address bus.

> **IMPORTANT!** *When you disable the `syn.interface.m_axi_addr64` option, the HLS tool implements all AXI4 interfaces in the design with a 32-bit address bus.*

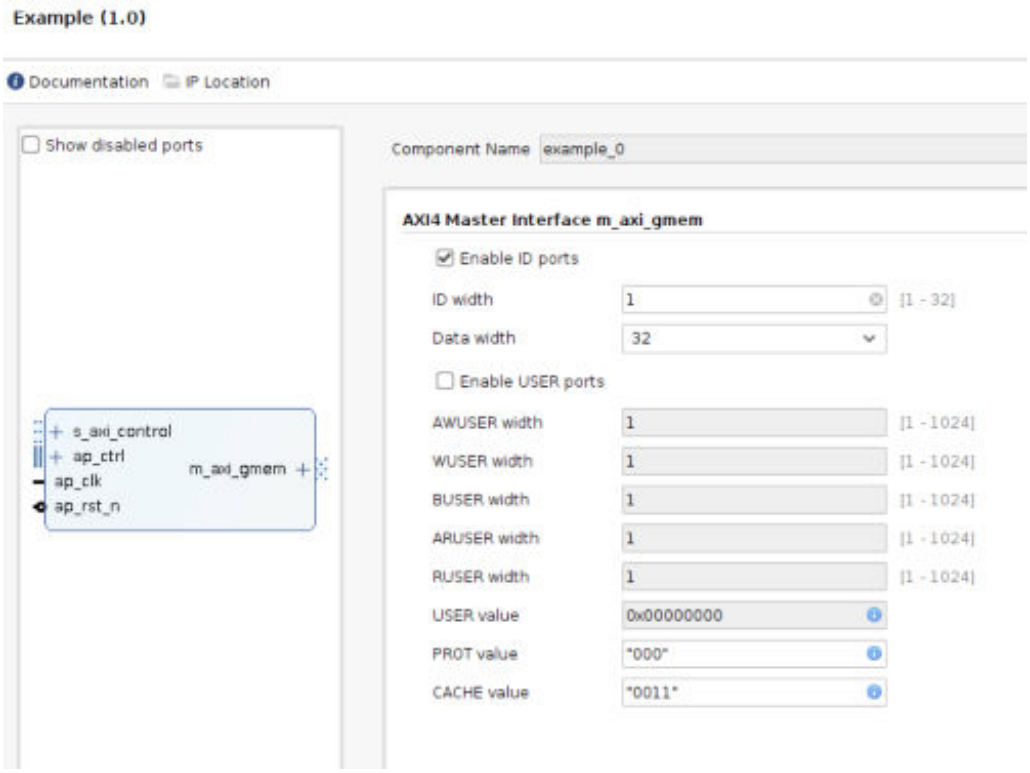
Vitis HLS

Customizing AXI4 Master Interfaces in IP Integrator

When you incorporate an HLS RTL design that uses an AXI4 master interface into a design in the Vivado IP integrator, you can customize the block. From the block diagram in IP integrator, select the HLS block, right-click, and select **Customize Block** to customize any of the settings provided. A complete description of the AXI4 parameters are provided in the *Vivado Design Suite: AXI Reference Guide* (UG1037).

The following figure shows the Re-Customize IP dialog box for the design shown below. This design includes an AXI4-Lite port.

Figure 42: **Customizing AXI4 Master Interfaces in IP Integrator**



AXI4-Lite Interface

Overview

An HLS IP or kernel can be controlled by a host application, or embedded processor using the Slave AXI4-Lite interface (`s_axilite`) which acts as a system bus for communication between the processor and the kernel. Using the `s_axilite` interface the host or an embedded processor can start and stop the kernel, and read or write data to it. When Vitis HLS synthesizes the design the `s_axilite` interface is implemented as an adapter that captures the data that was communicated from the host in registers on the adapter. Refer to Vitis-HLS-Introductory-Examples/Interface/Register on GitHub for examples of some of these concepts.

The AXI4-Lite interface performs several functions within a Vivado IP or Vitis kernel:

- It maps a block-level control mechanism which can be used to start and stop the kernel.
- It provides a channel for passing scalar arguments, pointers to scalar values, function return values, and address offsets for `m_axi` interfaces from the host to the IP or kernel
- For the Vitis Kernel flow:
  - The tool will automatically infer the `s_axilite` interface pragma to provide offsets to pointer arguments assigned to `m_axi` interfaces, scalar values, and function return type.

    - Vitis HLS lets you read to or write from a pointer to a scalar value when assigned to an `s_axilite` interface. Pointers are assigned by default to `m_axi` interfaces, so this requires you to manually assign the pointer to the `s_axilite` using the INTERFACE pragma or directive:

      ```
      int top(int *a, int *b) {
      #pragma HLS interface s_axilite port=a
      ```

    - *Bundle*: Do not specify the `bundle` option for the `s_axilite` adapter in the Vitis Kernel flow. The tool will create a single `s_axilite` interface that will serve for the whole design.

      IMPORTANT! *HLS will return an error if multiple bundles are specified for the Vitis Kernel flow.*

    - *Offset*: The tool will automatically choose the offsets for the interface. Do not specify any offsets in this flow.

- For the Vivado IP flow:
    - This flow will not use the `s_axilite` interface by default.
    - To use the `s_axilite` as a communication channel for scalar arguments, pointers to scalar values, offset to `m_axi` pointer address, and function return type, you must manually specify the INTERFACE pragma or directive.
    - *Bundle*: This flow supports multiple `s_axilite` interfaces, specified by bundle. Refer to S_AXILITE Bundle Rules for more information.
    - *Offset*: By default the tool will place the arguments in a sequential order starting from 0x10 in the control register map. Refer to S_AXILITE Offset Option for additional details.

S_AXILITE Example

The following example shows how Vitis HLS implements multiple arguments, including the function return, as an `s_axilite` interface. Because each pragma uses the same name for the `bundle` option, each of the ports is grouped into a single interface.

```
void example(char *a, char *b, char *c)
{
#pragma HLS INTERFACE mode=s_axilite port=return bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=a bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=b bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=c bundle=BUS_A
#pragma HLS INTERFACE mode=ap_vld port=b

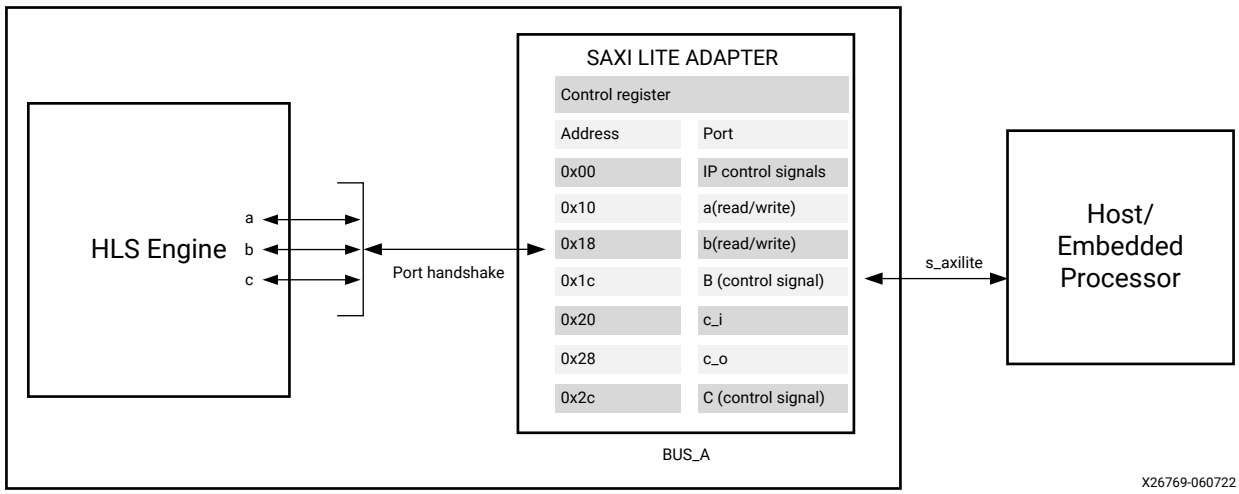
  *c += *a + *b;
}
```

TIP: *If you do not specify the `bundle` option, Vitis HLS groups all arguments into a single `s_axilite` bundle and automatically names the port.*

The synthesized example will be part of a system that has three important elements as shown in the figure below:

1. Host application running on an x86 or embedded processor interacting with the IP or kernel
2. SAXI Lite Adapter: The INTERFACE pragma implements an `s_axilite` adapter. The adapter has two primary functions: implementing the interface protocol to communicate with the host, and providing a Control Register Map to the IP or kernel.
3. The HLS engine or function that implements the design logic

Figure 43: **S_AXILITE Adapter**



By default, Vitis HLS automatically assigns the address for each port that is grouped into an `s_axilite` interface. The size, or range of addresses assigned to a port is dependent on the argument data type and the port protocol used, as described below. You can also explicitly define the address using the `offset` option as discussed in S_AXILITE Offset Option.

- Port a: By default, is implemented as `ap_none`. 1-word for the data signal is assigned and only 3 bits are used for addressing as the argument data type is `char`. Remaining bits are unused.
- Port b: is implemented as `ap_vld` defined by the INTERFACE pragma in the example. The corresponding control register is of size 2 bytes (16-bits) and is divided into two sections as follows:
  - (0x1c) Control signal : 1-word for the control signal is assigned.
  - (0x18) Data signal: 1-word for the data signal address is assigned and only 3 bits are used as the argument data type is `char`. Remaining bits are unused.
- Port c: By default, is implemented as `ap_ovld` as an output. The corresponding control register is of size 4 bytes (32 bits) and is divided into three sections:
  - (0x20) Data signal of `c_i`: 1-word for the input data signal is assigned, and only 3 bits are used for addressing as the argument data type is `char`, the rest are not used.
  - (0x24) Reserved Space
  - (0x28) Data signal of `c_o`: 1-word for the output data signal is assigned.

- (0x2c) Control signal of `c_o` : 1-word for control signal `ap_ovld` is assigned and only 3 bits are used for addressing as the argument data type is `char`. Remaining bits are unused.

In operation the host application will initially start the kernel by writing into the Control address space (0x00). The host/CPU completes the initial setup by writing into the other address spaces which are associated with the various function arguments as defined in the example.

The control signal for port b is asserted and only then can the kernel read ports a and b (port a is `ap_none` and does not have a control signal). Until that time the design is stalled and waiting for the *valid* register to be set for port b. Each time port b is read by the HLS engine the input *valid* register is cleared and the register resets to logic 0.

After the HLS engine finishes its computation, the output value on port C is stored in the control register and the corresponding *valid* bit is set for the host to read. After the host reads the data, the HLS engine will write the `ap_done` bit in the Control register (0x00) to mark the end of the IP computation.

Vitis HLS reports the assigned addresses in the S_AXILITE Control Register Map, and also provides them in C Driver Files to aid in your software development. Using the `s_axilite` interface, you can exploit the C driver files for use with code running on an embedded or x86 processor using provided C application program interface (API) functions, to let you control the hardware from your software.

S_AXILITE Control Register Map

The Vitis unified IDE and `v++` command automatically generates a Control Register Map for controlling the Vivado IP or Vitis kernel, and the ports grouped into `s_axilite` interface. The register map, which is added to the generated RTL files, can be divided into two sections:

1. Block-level control signals
2. Function arguments mapped into the `s_axilite` interface

In the Vitis kernel flow, the default block protocol is `ap_ctrl_chain` and is assigned to the `s_axilite` interface. The default settings should not be changed.

However, in the Vivado IP flow the default block control protocol is `ap_ctrl_hs` and is assigned to its own interface as seen in Interfaces for Vivado IP Flow. If you are using an `s_axilite` interface in your IP, you can also assign the block control protocol to that interface using the following INTERFACE pragma, as an example:

```
#pragma HLS INTERFACE mode=s_axilite port=return
```

To change the block control protocol you can also use the INTERFACE pragma or directive as follows:

```
#pragma HLS INTERFACE mode=ap_ctrl_chain port=return
```

In the Control Register Map of the `s_axilite` interface, Vitis HLS reserves addresses `0x00` through `0x18` for the block-level protocol, interrupt, mailbox and auto-restart controls. The latter are present only when counted auto-restart and the mailbox are enabled, as shown below:

Table 10: **Addresses**

Address	Description
0x00	Control signals
0x04	Global Interrupt Enable Register
0x08	IP Interrupt Enable Register (Read/Write)
0x0c	IP Interrupt Status Register (Read/TOW)
0x10	Auto-restart counter (Write; present only with counted autorestart)
0x14	Input mailbox write (Read/Write; present only when the input mailbox is enabled)
0x18	Output mailbox read (Read/Write; present only when the output mailbox is enabled)

The Control signals (0X00) contains `ap_start`, `ap_done`, `ap_ready`, and `ap_idle`; and in the case of `ap_ctrl_chain` the block protocol also contains `ap_continue`. These are the *block-level interface* signals which are accessed through the `s_axilite` adapter.

To start the block operation the`ap_start` bit in the Control register must be set to 1. The HLS engine will then proceed and read any inputs grouped into the AXI4-Lite slave interface from the register in the interface.

When the block completes the operation, the`ap_done`, `ap_idle` and `ap_ready` registers will be set by the hardware output ports and the results for any output ports grouped into the `s_axilite` interface read from the appropriate register.

For function arguments, the tool automatically assigns the address for each argument or port that is assigned to the `s_axilite` interface. The tool will assign each port an offset starting from `0x10`, the lower addresses being reserved for control signals. The size, or range of addresses assigned to a port is dependent on the argument data type and the port protocol used.

Because the variables grouped into an AXI4-Lite interface are function arguments which do not have a default value in the C code, none of the argument registers in the `s_axilite` interface can be assigned a default value. The registers can be implemented with a reset using the `config_rtl` command, but they cannot be assigned any other default value.

The Control Register Map generated by Vitis HLS for the `ap_ctrl_chain` block protocol is provided below:

```
...
//------------------------Address Info-------------------
// 0x00 : Control signals
//        bit 0  - ap_start (Read/Write/COH)
//        bit 1  - ap_done (Read)
//        bit 2  - ap_idle (Read)
//        bit 3  - ap_ready (Read/COR)
//        bit 4  - ap_continue (Read/Write/SC)
//        bit 7  - auto_restart (Read/Write)
//        bit 9  - interrupt (Read)
//        others - reserved
// 0x04 : Global Interrupt Enable Register
//        bit 0  - Global Interrupt Enable (Read/Write)
//        others - reserved
// 0x08 : IP Interrupt Enable Register (Read/Write)
//        bit 0 - enable ap_done interrupt (Read/Write)
//        bit 1 - enable ap_ready interrupt (Read/Write)
//        others - reserved
// 0x0c : IP Interrupt Status Register (Read/TOW)
//        bit 0 - ap_done (Read/TOW)
//        bit 1 - ap_ready (Read/TOW)
//        others - reserved
// 0x10 : Data signal of a
//        bit 7~0 - a[7:0] (Read/Write)
//        others  - reserved
// 0x14 : reserved
// 0x18 : Data signal of b
//        bit 7~0 - b[7:0] (Read/Write)
//        others  - reserved
// 0x1c : reserved
// 0x20 : Data signal of c_i
//        bit 7~0 - c_i[7:0] (Read/Write)
//        others  - reserved
// 0x24 : reserved
// 0x28 : Data signal of c_o
//        bit 7~0 - c_o[7:0] (Read)
//        others  - reserved
// 0x2c : reserved
// (SC = Self Clear, COR = Clear on Read, TOW = Toggle on Write, COH =
Clear on Handshake)
...
```

S_AXILITE and Port-Level Protocols

Port-level I/O protocols sequence data into and out of the HLS engine from the `s_axilite` adapter as seen in S_AXILITE Example. In the Vivado IP flow, you can assign port-level I/O protocols to the individual ports and signals bundled into an `s_axilite` interface. In the Vitis kernel flow, changing the default port-level I/O protocols is not recommended unless necessary. The tool assigns a default port protocol to a port depending on the type and direction of the argument associated with it. The port can contain one or more of the following:

- Data signal for the argument
- Valid signal (`ap_vld`/`ap_ovld`) to indicate when the data can be read

- Acknowledge signal (`ap_ack`) to indicate when the data has been read

The default port protocol assignments for various argument types are as follows:

Table 11: **Supported Argument Types**

Argument Type	Default	Supported
scalar	`ap_none`	`ap_ack` and `ap_vld` can also be used
Pointers/References		
Inputs	`ap_none`	`ap_ack` and `ap_vld`
Outputs	`ap_vld`	`ap_none`, `ap_ack`, and `ap_ovld` can also be used
Inouts	`ap_ovld`	`ap_none`, `ap_ack`, and `ap_vld` are also supported

IMPORTANT! *Arrays default to `ap_memory`.*

The S_AXILITE Example groups port `b` into the `s_axilite` interface and specifies port `b` as using the `ap_vld` protocol with INTERFACE pragmas. As a result, the `s_axilite` adapter contains a register for the port `b` data, and a register for the port `b` input valid signal.

If the input valid register is not set to logic 1, the data in the `b` data register is not considered valid, and the design stalls and waits for the valid register to be set. Each time port `b` is read, Vitis HLS automatically clears the input valid register and resets the register to logic 0.

RECOMMENDED: *To simplify the operation of your design, AMD recommends that you use the default port protocols associated with the `s_axilite` interface.*

S_AXILITE Bundle Rules

In the S_AXILITE Example all the function arguments are grouped into a single `s_axilite` interface adapter specified by the `bundle=BUS_A` option in the INTERFACE pragma. The `bundle` option simply lets you group ports together into one interface.

In the Vitis kernel flow there should only be a single interface bundle, commonly named `s_axi_control` by the tool. So you should not specify the `bundle` option in that flow, or you will probably encounter an error during synthesis. However, in the Vivado IP flow you can specify multiple bundles using the `s_axilite` interface, and this will create a separate interface adapter for each bundle you have defined. The following example shows this:

```
void example(char *a, char *b, char *c)
{
#pragma HLS INTERFACE mode=s_axilite port=a bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=b bundle=BUS_A
#pragma HLS INTERFACE mode=s_axilite port=c bundle=OUT
#pragma HLS INTERFACE mode=s_axilite port=return bundle=BUS_A
#pragma HLS INTERFACE mode=ap_vld port=b
  *c += *a + *b;
}
```

After synthesis completes, the Synthesis Summary report provides feedback regarding the number of `s_axilite` adapters generated. The SW-to-HW Mapping section of the report contains the HW info showing the control register offset and the address range for each port.

However, there are some rules related to using bundles with the `s_axilite` interface.

1. Default Bundle Names: This rule explicitly groups all interface ports with no bundle name into the same AXI4-Lite interface port, uses the tool default bundle name, and names the RTL port `s_axi_<default>`, typically `s_axi_control`.

   In this example all ports are mapped to the default bundle:

   ```
   void top(char *a, char *b, char *c)
   {
   #pragma HLS INTERFACE mode=s_axilite port=a
   #pragma HLS INTERFACE mode=s_axilite port=b
   #pragma HLS INTERFACE mode=s_axilite port=c
        *c += *a + *b;
   }
   ```

2. User-Specified Bundle Names: This rule explicitly groups all interface ports with the same `bundle` name into the same AXI4-Lite interface port, and names the RTL port the value specified by `s_axi_<string>`.

   The following example results in interfaces named `s_axi_BUS_A`, `s_axi_BUS_B`, and `s_axi_OUT`:

   ```
   void example(char *a, char *b, char *c)
   {
   #pragma HLS INTERFACE mode=s_axilite port=a bundle=BUS_A
   #pragma HLS INTERFACE mode=s_axilite port=b bundle=BUS_B
   #pragma HLS INTERFACE mode=s_axilite port=c bundle=OUT
   #pragma HLS INTERFACE mode=s_axilite port=return bundle=OUT
   #pragma HLS INTERFACE mode=ap_vld port=b
        *c += *a + *b;
   }
   ```

3. Partially Specified Bundle Names: If you specify `bundle` names for some arguments, but leave other arguments unassigned, then the tool will bundle the arguments as follows:

- Group all ports into the specified bundles as indicated by the INTERFACE pragmas.
- Group any ports without bundle assignments into a default named bundle. The default name can either be the standard tool default, or an alternative default name if the tool default has already been specified by the user.

In the following example the user has specified `bundle=control`, which is the tool default name. In this case, port c will be assigned to `s_axi_control` as specified by the user, and the remaining ports will be bundled under `s_axi_control_r`, which is an alternative default name used by the tool.

```
void top(char *a, char *b, char *c) {
#pragma HLS INTERFACE mode=s_axilite port=a
#pragma HLS INTERFACE mode=s_axilite port=b
#pragma HLS INTERFACE mode=s_axilite port=c bundle=control
}
```

S_AXILITE Offset Option

***Note*:** The Vitis kernel flow determines the required offsets. Do not specify the `offset` option in that flow.

In the Vivado IP flow, Vitis HLS defines the size, or range of addresses assigned to a port in the S_AXILITE Control Register Map depending on the argument data type and the port protocol used. However, the INTERFACE pragma also contains an `offset` option that lets you specify the address offset in the AXI4-Lite interface.

When specifying the offset for your argument, you must consider the size of your data and reserve some extra for the port control protocol. The range of addresses you reserve should be based on a 32-bit word. You should reserve enough 32-bit words to fit your argument data type, and add reserve one additional word for the control protocol, even for `ap_none`.



TIP: *In the case of the `ap_memory` protocol for arrays, you do not need to reserve the extra word for the control protocol. In this case, simply reserve enough 32-bit words to fit your argument data type.*

For example, to reserve enough space for a double you need to reserve two 32-bit words for the 64-bit data type, and then reserve an additional 32-bit word for the control protocol. So you need to reserve a total of three 32-bit words, or 96 bits. If your argument offset starts at 0x020, then the next available offset would begin at 0x02c, in order to reserve the required address range for your argument.

If you make a mistake in setting the offset of your arguments, by not reserving enough address range to fit your data type and the control protocol, Vitis HLS will recognize the error, will warn you of the issue, and will recover by moving your misplaced argument register to the end of the Control Register Map. This will allow your build to proceed, but may not work with your host application or driver if they were written to your specified offset.

C Driver Files

When an AXI4-Lite slave interface is implemented, a set of C driver files are automatically created. These C driver files provide a set of APIs that can be integrated into any software running on a CPU and used to communicate with the device via the AXI4-Lite slave interface.

The C driver files are created when the design is packaged as IP in the IP catalog.

Driver files are created for standalone and Linux modes. In standalone mode the drivers are used in the same way as any other AMD standalone drivers. In Linux mode, copy all the C files (`.c`) and header files (`.h`) files into the software project.

The driver files and API functions derive their name from the top-level function for synthesis. In the above example, the top-level function is called "example". If the top-level function was named "DUT" the name "example" would be replaced by "DUT" in the following description. The driver files are created in the packaged IP (located in the `impl` directory inside the `solution`).

Table 12: **C Driver Files for a Design Named Example**

File Path	Usage Mode	Description
`data/example.mdd`	Standalone	Driver definition file.
`data/example.tcl`	Standalone	Used by SDK to integrate the software into an SDK project.
`src/xexample_hw.h`	Both	Defines address offsets for all internal registers.
`src/xexample.h`	Both	API definitions
`src/xexample.c`	Both	Standard API implementations
`src/xexample_sinit.c`	Standalone	Initialization API implementations
`src/xexample_linux.c`	Linux	Initialization API implementations
`src/Makefile`	Standalone	Makefile

In file `xexample.h`, two structs are defined.

- **XExample_Config:** This is used to hold the configuration information (base address of each AXI4-Lite slave interface) of the IP instance.
- **XExample:** This is used to hold the IP instance pointer. Most APIs take this instance pointer as the first argument.

The standard API implementations are provided in files `xexample.c`, `xexample_sinit.c`, `xexample_linux.c`, and provide functions to perform the following operations.

- Initialize the device
- Control the device and query its status
- Read/write to the registers
- Set up, monitor, and control the interrupts

Refer to Section V: Vitis HLS C Driver Reference for a description of the API functions provided in the C driver files.

IMPORTANT! *The C driver APIs always use an unsigned 32-bit type (U32). You might be required to cast the data in the C code into the expected type.*

C Driver Files and Float Types

C driver files always use a data 32-bit unsigned integer (U32) for data transfers. In the following example, the function uses float type arguments `a` and `r1`. It sets the value of `a` and returns the value of `r1`:

```
float calculate(float a, float *r1)
{
#pragma HLS INTERFACE mode=ap_vld register port=r1
#pragma HLS INTERFACE mode=s_axilite port=a
#pragma HLS INTERFACE mode=s_axilite port=r1
#pragma HLS INTERFACE mode=s_axilite port=return

 *r1 = 0.5f*a;
 return (a>0);
}
```

After synthesis, Vitis HLS groups all ports into the default AXI4-Lite interface and creates C driver files. However, as shown in the following example, the driver files use type U32:

```
// API to set the value of A
void XCalculate_SetA(XCalculate *InstancePtr, u32 Data) {
    Xil_AssertVoid(InstancePtr != NULL);
    Xil_AssertVoid(InstancePtr->IsReady == XIL_COMPONENT_IS_READY);
    XCalculate_WriteReg(InstancePtr->Hls_periph_bus_BaseAddress,
XCALCULATE_HLS_PERIPH_BUS_ADDR_A_DATA, Data);
}

// API to get the value of R1
u32 XCalculate_GetR1(XCalculate *InstancePtr) {
    u32 Data;

    Xil_AssertNonvoid(InstancePtr != NULL);
    Xil_AssertNonvoid(InstancePtr->IsReady == XIL_COMPONENT_IS_READY);

    Data = XCalculate_ReadReg(InstancePtr->Hls_periph_bus_BaseAddress,
XCALCULATE_HLS_PERIPH_BUS_ADDR_R1_DATA);
    return Data;
}
```

If these functions work directly with float types, the write and read values are not consistent with expected float type. When using these functions in software, you can use the following casts in the code:

```
float a=3.0f,r1;
u32 ua,ur1;

// cast float “a” to type U32
XCalculate_SetA(&calculate,*((u32*)&a));
ur1=XCalculate_GetR1(&calculate);

// cast return type U32 to float type for “r1”
r1=*((float*)&ur1);
```

Controlling Hardware



TIP: *The example provided below demonstrates the `ap_ctrl_hs` block control protocol, which is the default for the Vivado IP flow. Refer to Block-Level Control Protocols for more information and a description of the `ap_ctrl_chain` protocol which is the default for the Vitis kernel flow.*

In this example, the hardware header file `xexample_hw.h` provides a complete list of the memory mapped locations for the ports grouped into the AXI4-Lite slave interface, as described in S_AXILITE Control Register Map.

```
// 0x00 : Control signals
//        bit 0  - ap_start (Read/Write/SC)
//        bit 1  - ap_done (Read/COR)
//        bit 2  - ap_idle (Read)
//        bit 3  - ap_ready (Read)
//        bit 7  - auto_restart (Read/Write)
//        others - reserved
// 0x04 : Global Interrupt Enable Register
//        bit 0  - Global Interrupt Enable (Read/Write)
//        others - reserved
// 0x08 : IP Interrupt Enable Register (Read/Write)
//        bit 0  - Channel 0 (ap_done)
//        bit 1  - Channel 1 (ap_ready)
// 0x0c : IP Interrupt Status Register (Read/TOW)
//        bit 0  - Channel 0 (ap_done)
//        others - reserved
// 0x10 : Data signal of a
//        bit 7~0 - a[7:0] (Read/Write)
//        others  - reserved
// 0x14 : reserved
// 0x18 : Data signal of b
//        bit 7~0 - b[7:0] (Read/Write)
//        others  - reserved
// 0x1c : reserved
// 0x20 : Data signal of c_i
//        bit 7~0 - c_i[7:0] (Read/Write)
//        others  - reserved
// 0x24 : reserved
// 0x28 : Data signal of c_o
//        bit 7~0 - c_o[7:0] (Read)
//        others  - reserved
// 0x2c : Control signal of c_o
```

```
//        bit 0  - c_o_ap_vld (Read/COR)
//        others - reserved
// (SC = Self Clear, COR = Clear on Read, TOW = Toggle on Write, COH =
Clear on
Handshake)
```

To correctly program the registers in the `s_axilite` interface, you must understand how the hardware ports operate with the default port protocols, or the custom protocols as described in S_AXILITE and Port-Level Protocols.

For example, to start the block operation the `ap_start` register must be set to 1. The device will then proceed and read any inputs grouped into the AXI4-Lite slave interface from the register in the interface. When the block completes operation, the `ap_done`, `ap_idle` and `ap_ready` registers will be set by the hardware output ports and the results for any output ports grouped into the AXI4-Lite slave interface read from the appropriate register.

The implementation of function argument `c` in the example highlights the importance of some understanding how the hardware ports operate. Function argument `c` is both read and written to, and is therefore implemented as separate input and output ports `c_i` and `c_o`, as explained in S_AXILITE Example.

The first recommended flow for programing the `s_axilite` interface is for a one-time execution of the function:

- Use the interrupt function standard API implementations provided in the C Driver Files to determine how you want the interrupt to operate.
- Load the register values for the block input ports. In the above example this is performed using API functions `XExample_Set_a`, `XExample_Set_b`, and `XExample_Set_c_i`.
- Set the `ap_start` bit to 1 using `XExample_Start` to start executing the function. This register is self-clearing as noted in the header file above. After one transaction, the block will suspend operation.
- Allow the function to execute. Address any interrupts which are generated.
- Read the output registers. In the above example this is performed using API functions `XExample_Get_c_o_vld`, to confirm the data is valid, and `XExample_Get_c_o`.

  Note: The registers in the `s_axilite` interface obey the same I/O protocol as the ports. In this case, the output valid is set to logic 1 to indicate if the data is valid.
- Repeat for the next transaction.

The second recommended flow is for continuous execution of the block. In this mode, which is described in much more detail in the next section, the input ports included in the AXI4-Lite interface should only be ports which perform configuration. The block will typically run much faster than a CPU. If the block must wait for inputs, the block will spend most of its time waiting:

- Use the interrupt function to determine how you wish the interrupt to operate.

- Load the register values for the block input ports. In the above example this is performed using API functions `XExample_Set_a`, `XExample_Set_a` and `XExample_Set_c_i`.
- Set the auto-start function using API `XExample_EnableAutoRestart`.
- Allow the function to execute. The individual port I/O protocols will synchronize the data being processed through the block.
- Address any interrupts which are generated. The output registers could be accessed during this operation but the data may change often.
- Use the API function `XExample_DisableAutoRestart` to prevent any more executions.
- Read the output registers. In the above example this is performed using API functions `XExample_Get_c_o` and `XExample_Set_c_o_vld`.

Controlling Software

The API functions can be used in the software running on the CPU to control the hardware block. An overview of the process is:

- Create an instance of the hardware
- Look Up the device configuration
- Initialize the device
- Set the input parameters of the HLS block
- Start the device and read the results

An example application is shown below.

```
#include "xexample.h"    // Device driver for HLS HW block
#include "xparameters.h"

// HLS HW instance
XExample HlsExample;
XExample_Config *ExamplePtr

int main() {
 int res_hw;

// Look Up the device configuration
 ExamplePtr = XExample_LookupConfig(XPAR_XEXAMPLE_0_DEVICE_ID);
 if (!ExamplePtr) {
 print("ERROR: Lookup of accelerator configuration failed.\n\r");
 return XST_FAILURE;
 }

// Initialize the Device
 status = XExample_CfgInitialize(&HlsExample, ExamplePtr);
 if (status != XST_SUCCESS) {
 print("ERROR: Could not initialize accelerator.\n\r");
 exit(-1);
 }

//Set the input parameters of the HLS block
 XExample_Set_a(&HlsExample, 42);
```

```
 XExample_Set_b(&HlsExample, 12);
 XExample_Set_c_i(&HlsExample, 1);

// Start the device and read the results
 XExample_Start(&HlsExample);
 do {
 res_hw = XExample_Get_c_o(&HlsExample);
 } while (XExample_Get_c_o(&HlsExample) == 0); // wait for valid data output
 print("Detected HLS peripheral complete. Result received.\n\r");
}
```

Control Clock and Reset in AXI4-Lite Interfaces

***Note*:** If you instantiate the slave AXI4-Lite register file in a bus fabric that uses a different clock frequency, Vivado IP integrator will automatically generate a clock domain crossing (CDC) slice that performs the same function as the control clock described below, making use of the option unnecessary.

By default, Vitis HLS uses the same clock for the AXI4-Lite interface and the synthesized design. Vitis HLS connects all registers in the AXI4-Lite interface to the clock used for the synthesized logic (`ap_clk`).

Optionally, you can use the INTERFACE directive `clock` option to specify a separate clock for each AXI4-Lite port. When connecting the clock to the AXI4-Lite interface, you must use the following protocols:

- AXI4-Lite interface clock must be synchronous to the clock used for the synthesized logic (`ap_clk`). That is, both clocks must be derived from the same master generator clock.
- AXI4-Lite interface clock frequency must be equal to or less than the frequency of the clock used for the synthesized logic (`ap_clk`).

If you use the `clock` option with the INTERFACE directive, you only need to specify the `clock` option on one function argument in each bundle. Vitis HLS implements all other function arguments in the bundle with the same clock and reset. Vitis HLS names the generated reset signal with the prefix `ap_rst_` followed by the clock name. The generated reset signal is active-Low independent of the `config_rtl` command.

The following example shows how Vitis HLS groups function arguments `a` and `b` into an AXI4-Lite port with a clock named `AXI_clk1` and an associated reset port.

```
// Default AXI-Lite interface implemented with independent clock called
AXI_clk1
#pragma HLS interface mode=s_axilite port=a clock=AXI_clk1
#pragma HLS interface mode=s_axilite port=b
```

In the following example, Vitis HLS groups function arguments `c` and `d` into AXI4-Lite port `CTRL1` with a separate clock called `AXI_clk2` and an associated reset port.

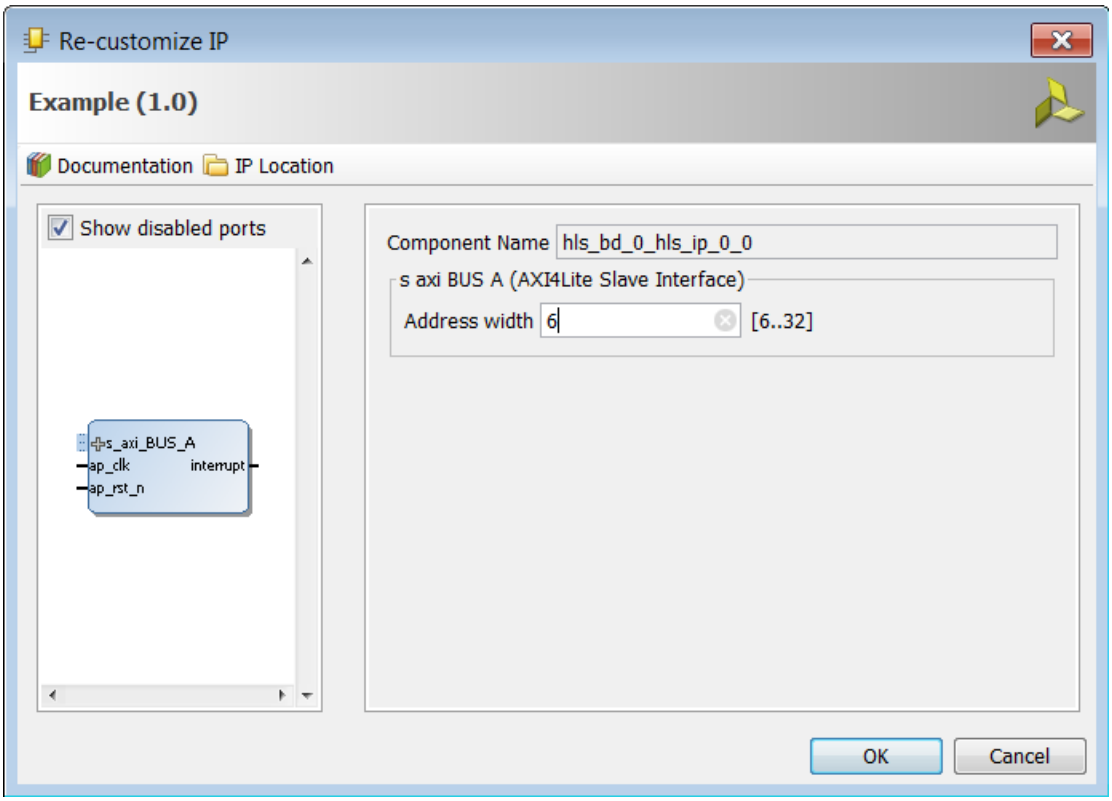
```
// CTRL1 AXI-Lite bundle implemented with a separate clock (called AXI_clk2)
#pragma HLS interface mode=s_axilite port=c bundle=CTRL1 clock=AXI_clk2
#pragma HLS interface mode=s_axilite port=d bundle=CTRL1
```

Customizing AXI4-Lite Slave Interfaces in IP Integrator

When an HLS RTL design using an AXI4-Lite slave interface is incorporated into a design in Vivado IP integrator, you can customize the block. From the block diagram in IP integrator, select the HLS block, right-click with the mouse button and select **Customize Block**.

The address width is by default configured to the minimum required size. Modify this to connect to blocks with address sizes less than 32-bit.

Figure 44: **Customizing AXI4-Lite Slave Interfaces in IP Integrator**



AXI4-Stream Interfaces

IMPORTANT! *`hls::axis` (and `ap_axiu`/`ap_axis`) cannot be used on internal functions or variables as the AXI4-Stream protocol is only supported on the interfaces of top-level functions. For internal functions or variables you must use `hls::stream` objects as described in HLS Stream Library.*

An AXI4-Stream interface can be applied to any input argument and any array or pointer output argument. Because an AXI4-Stream interface transfers data in a sequential streaming manner, it cannot be used with arguments that are both read and written. In terms of data layout, the data type of the AXI4-Stream is aligned to the next byte. For example, if the size of the data type is 12 bits, it will be extended to 16 bits. Depending on whether a signed/unsigned interface is selected, the extended bits are either sign-extended or zero-extended.

If the stream data type is an user-defined struct, the default procedure is to keep the struct aggregated and align the struct to the size of the largest data element to the nearest byte. The only exception to this rule is if the struct contains a `hls::stream` object. In this special case, the struct will be disaggregated and an axi stream will be created for each member element of the struct.



TIP: *The maximum supported port width is 4096 bits, even for aggregated structs or reshaped arrays.*

The following code examples show how the packed alignment depends on your struct type. If the struct contains only char type, as shown in the following example, then it will be packed with alignment of one byte. Total size of the struct will be two bytes:

```
struct A {
  char foo;
  char bar;
};
```

However, if the struct has elements with different data types, as shown below, then it will be packed and aligned to the size of the largest data element, or four bytes in this example. Element `bar` will be padded with three bytes resulting in a total size of eight bytes for the struct:

```
struct A {
  int foo;
  char bar;
};
```

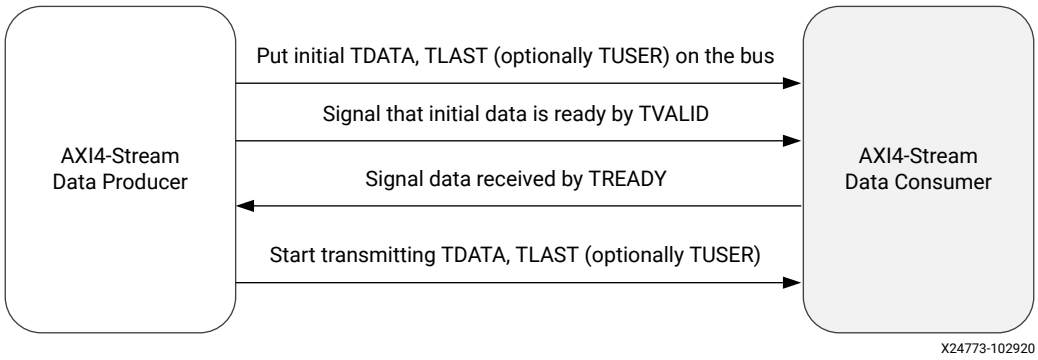


IMPORTANT! *Structs contained in AXI4-Stream interfaces (axis) are aggregated by default, and the stream itself cannot be disaggregated. If separate streams for member elements of the struct are desired then this must be manually coded as separate elements, resulting in a separate axis interface for each element. Refer to Vitis-HLS-Introductory-Examples/Interface/Aggregation_Disaggregation/disaggregation_of_axis_port on GitHub for an example.*

How AXI4-Stream Works

AXI4-Stream is a protocol designed for transporting arbitrary unidirectional data. In an AXI4-Stream, `TDATA` width of bits is transferred per clock cycle. The transfer is started once the producer sends the `TVALID` signal and the consumer responds by sending the `TREADY` signal (once it has consumed the initial `TDATA`). At this point, the producer will start sending `TDATA` and `TLAST` (`TUSER` if needed to carry additional user-defined sideband data). `TLAST` signals the last byte of the stream. So the consumer keeps consuming the incoming `TDATA` until `TLAST` is asserted.

Figure 45: **AXI4-Stream Handshake**



AXI4-Stream has additional optional features like sending positional data with `TKEEP` and `TSTRB` ports which makes it possible to multiplex both the data position and data itself on the `TDATA` signal. Using the `TID` and `TDIST` signals, you can route streams as these fields roughly corresponds to stream identifier and stream destination identifier. Refer to *Vivado Design Suite: AXI Reference Guide* (UG1037) or the *AMBA AXI4-Stream Protocol Specification* (ARM IHI 0051A) for more information.

How AXI4-Stream is Implemented

If your design requires a streaming interface begin by defining and using a streaming data structure like `hls::stream` in Vitis HLS. This simple object encapsulates the requirements of streaming and its streaming interface is by default implemented in the RTL as a FIFO interface (ap_fifo) but can be optionally, implemented as a handshake interface (ap_hs) or an AXI4-Stream interface (axis). Refer to Vitis-HLS-Introductory-Examples/Interface/Streaming on GitHub for different examples of streaming interfaces.

If a AXI4-Stream interface (axis) is specified via the interface pragma mode option, the interface implementation will mimic the style of an AXIS interface by defining the TDATA, TVALID and TREADY signals.

If a more formal AXIS implementation is desired, then Vitis HLS requires the usage of a special data type (`hls::axis` defined in `ap_axi_sdata.h`) to encapsulate the requirements of the AXI4-Stream protocol and implement the special RTL signals needed for this interface.

The AXI4-Stream interface is implemented as a struct type in Vitis HLS and has the following signature (defined in `ap_axi_sdata.h`):

```
template <typename T, size_t WUser, size_t WId, size_t WDest> struct axis
{ .. };
```

Where:

- **`T`:** The data type to be streamed.

> **TIP:** *This can support any data type, including* `ap_fixed`.

- **`WUser`:** Width of the TUSER signal
- **`WId`:** Width of the TID signal
- **`WDest`:** Width of the TDest signal

When the stream data type (`T`) are simple integer types, there are two predefined types of AXI4-Stream implementations available:

- A signed implementation of the AXI4-Stream class (or more simply `ap_axis<Wdata, WUser, WId, WDest>`)

```
hls::axis<ap_int<WData>, WUser, WId, WDest>
```

- An unsigned implementation of the AXI4-Stream class (or more simply `ap_axiu<WData, WUser, WId, WDest>`)

```
hls::axis<ap_uint<WData>, WUser, WId, WDest>
```

The value specified for the `WUser`, `WId`, and `WDest` template parameters controls the usage of side-channel signals in the AXI4-Stream interface.

When the `hls::axis` class is used, the generated RTL will typically contain the actual data signal `TDATA`, and the following additional signals: `TVALID`, `TREADY`, `TKEEP`, `TSTRB`, `TLAST`, `TUSER`, `TID`, and `TDEST`.

`TVALID`, `TREADY`, and `TLAST` are necessary control signals for the AXI4-Stream protocol. `TKEEP`, `TSTRB`, `TUSER`, `TID`, and `TDEST` signals are optional special signals that can be used to pass around additional bookkeeping data.

> **TIP:** *If `WUser`, `WId`, and `WDest` are set to 0, the generated RTL will not include the optional `TUSER`, `TID`, and `TDEST` signals in the interface.*

Registered AXI4-Stream Interfaces

As a default, AXI4-Stream interfaces are always implemented as registered interfaces to ensure that no combinational feedback paths are created when multiple HLS IP blocks with AXI4-Stream interfaces are integrated into a larger design. For AXI4-Stream interfaces, four types of register modes are provided to control how the interface registers are implemented:

- **Forward:** Only the `TDATA` and `TVALID` signals are registered.
- **Reverse:** Only the `TREADY` signal is registered.
- **Both:** All signals (`TDATA`, `TREADY`, and `TVALID`) are registered. This is the default.
- **Off:** None of the port signals are registered.

The AXI4-Stream side-channel signals are considered to be data signals and are registered whenever `TDATA` is registered.

RECOMMENDED: *When connecting HLS generated IP blocks with AXI4-Stream interfaces at least one interface should be implemented as a registered interface or the blocks should be connected via an AXI4-Stream Register Slice.*

There are two basic methods to use an AXI4-Stream in your design:

- Use an AXI4-Stream without side-channels.
- Use an AXI4-Stream with side-channels.

This second use model provides additional functionality, allowing the optional side-channels which are part of the AXI4-Stream standard, to be used directly in your C/C++ code.

AXI4-Stream Interfaces without Side-Channels

An AXI4-Stream is used without side-channels when the function argument, `ap_axis` or `ap_axiu` data type, does not contain any AXI4 side-channel elements (that is, when the `WUser`, `WId`, and `WDest` parameters are set to 0). In the following example, both interfaces are implemented using an AXI4-Stream:

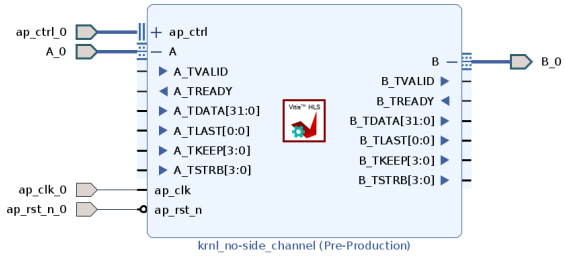
```
#include "ap_axi_sdata.h"
#include "hls_stream.h"

typedef ap_axiu<32, 0, 0, 0> trans_pkt;

void example(hls::stream< trans_pkt > &A, hls::stream< trans_pkt > &B)
{
#pragma HLS INTERFACE mode=axis port=A
#pragma HLS INTERFACE mode=axis port=B
    trans_pkt tmp;
    A.read(tmp);
    tmp.data += 5;
    B.write(tmp);
}
```

After synthesis, both arguments are implemented with a data port (`TDATA`) and the standard AXI4-Stream protocol ports, `TVALID`, `TREADY`, `TKEEP`, `TLAST`, and `TSTRB`, as shown in the following figure.

Figure 46: **AXI4-Stream Interfaces without Side-Channels**



TIP: *If you specify an `hls::stream` object with a data type other than `ap_axis` or `ap_axiu`, the tool will infer an AXI4-Stream interface without the `TLAST` signal, or any of the side-channel signals. This implementation of the AXI4-Stream interface consumes fewer device resources, but offers no visibility into when the stream is ending.*

Multiple variables can be combined into the same AXI4-Stream interface by using a struct, which is aggregated by Vitis HLS by default. Aggregating the elements of a struct into a single wide-vector, allows all elements of the struct to be implemented in the same AXI4-Stream interface.

AXI4-Stream Interfaces with Side-Channels

The following example shows how the side-channels can be used directly in the C/C++ code and implemented on the interface. The code uses `#include "ap_axi_sdata.h"` to provide an API to handle the side-channels of the AXI4-Stream interface. In the following example a signed 32-bit data type is used:

```
#include "ap_axi_sdata.h"
#include "ap_int.h"
#include "hls_stream.h"

#define DWIDTH 32

typedef ap_axiu<DWIDTH, 1, 1, 1> trans_pkt;

extern "C"{
    void krnl_stream_vmult(hls::stream<trans_pkt> &A,
                           hls::stream<trans_pkt> &B) {
#pragma HLS INTERFACE mode=axis port=A
#pragma HLS INTERFACE mode=axis port=B
#pragma HLS INTERFACE mode=s_axilite port=return bundle=control
        bool eos = false;

        vmult: do {
#pragma HLS PIPELINE II=1
            trans_pkt t2 = A.read();

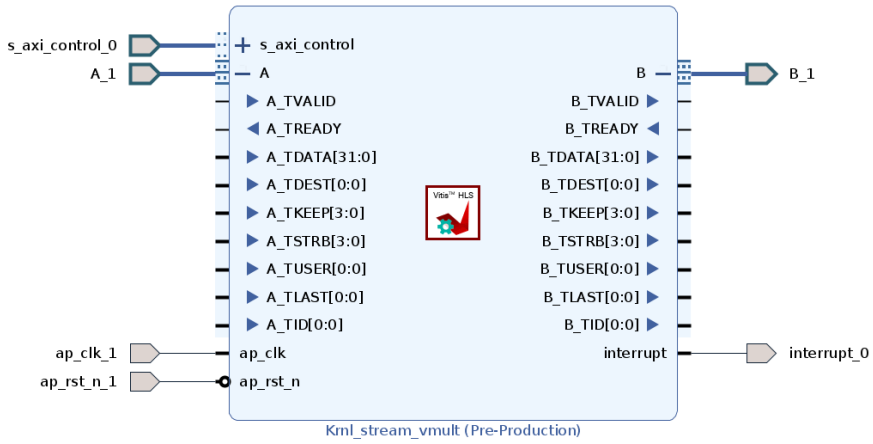
            // Packet for Output
            trans_pkt t_out;

            // Reading data from input packet
            ap_uint<DWIDTH> in2 = t2.data;
            ap_uint<DWIDTH> tmpOut = in2 * 5;

            // Setting data and configuration to output packet
            t_out.data = tmpOut;
            t_out.last = t2.last;
            t_out.keep = -1; //Enabling all bytes
            // Writing packet to output stream
            B.write(t_out);
            if (t2.last) {
               eos = true;
            }
        } while (eos == false);
    }
}
```

After synthesis, both the A and B arguments are implemented with data ports, the standard AXI4-Stream protocol ports, `TVALID` and `TREADY` and all of the optional ports described in the struct.

Figure 47: **AXI4-Stream Interfaces with Side-Channels**



Coding Style for Array to Stream

While arrays can be converted to streams, it can often lead to coding and synthesis issues as arrays can be accessed in random order while a stream requires a sequential access pattern where every element is read in order. To avoid such issues, any time a streaming interface is required, it is highly recommended to use the `hls::stream` object as described in Using HLS Streams. Usage of this construct will enforce streaming semantics in the source code.

However, to convert an array to a stream you should perform all the operations on temp variables. Read the input stream, process the temp variable, and write the output stream, as shown in the example below. This approach lets you preserve the sequential reading and writing of the stream of data, rather than attempting multiple or random reads or writes.

```
struct A {
  short varA;
  int varB;
};

void dut(A in[N], A out[N], bool flag) {
#pragma HLS interface mode=axis port=in,out
   for (unsigned i=0; i<N; i++) {
      A tmp = in[i];
      if (flag)
         tmp.varB = tmp.varA + 5;
   out[i] = tmp;
   }
}
```

If this coding style is not adhered to, it will lead to functional failures of the stream processing.

The recommended method is to define the arguments as `hls::stream` objects as shown below:

```
void dut(hls::stream<A> &in, hls::stream<A> &out, bool flag) {
#pragma HLS interface mode=axis port=in,out

for (unsigned i=0; i<N; i++) {
   A tmp = in.read();
   if (flag)
      tmp.varB = tmp.varA + 5;
   out.write(tmp);
  }
}
```

Customizing AXI4-Stream Interfaces

An instance of an AXI4-Stream without side channels (for example `hls::stream<ap_axiu<32, 0, 0, 0>>`) can result in the following RTL signals on the interface:

- `TVALID`
- `TREADY`
- `TDATA[ ]`
- `TLAST[ ]`
- `TKEEP[ ]`
- `TSTRB[ ]`

Some of these signals (`TLAST`, `TKEEP`, and `TSTRB`) can be unnecessary or unwanted in your design. In addition, there are use cases where `TDATA` is not required as the modeler uses `TUSER` to move data. To support these additional use cases the `hls::axis<T>` definition supports the following options:

```
template <typename T,
          std::size_t WUser = 0,
          std::size_t WId = 0,
          std::size_t WDest = 0,
          uint8_t EnableSignals = (AXIS_ENABLE_KEEP |
                                   AXIS_ENABLE_LAST |
                                   AXIS_ENABLE_STRB),
          bool StrictEnablement = false>
struct axis {
  // members
  T data;
  ap_uint<bytesof<T>> strb;
  ap_uint<bytesof<T>> keep;
  ap_uint<WUser>      user;
  ap_uint<WId>        id;
  ap_uint<WDest>      dest;
  ap_uint<1>          last;
```

```
  // get_* methods
  …
  // set_* methods
  …
};
```

The `EnableSignals` and `StrictEnablement` fields let you specify which of the three signals (`TKEEP`, `TLAST`, and `TSTRB`) should appear in your design using the specification of special macros. The complete list of macros is shown below:

```
// Enablement for axis signals
#define AXIS_ENABLE_DATA 0b00000001
#define AXIS_ENABLE_DEST 0b00000010
#define AXIS_ENABLE_ID   0b00000100
#define AXIS_ENABLE_KEEP 0b00001000
#define AXIS_ENABLE_LAST 0b00010000
#define AXIS_ENABLE_STRB 0b00100000
#define AXIS_ENABLE_USER 0b01000000

// Disablement mask for DATA axis signals
#define AXIS_DISABLE_DATA (0b11111111 ^ AXIS_ENABLE_DATA) & \
                          (0b11111111 ^ AXIS_ENABLE_KEEP) & \
                          (0b11111111 ^ AXIS_ENABLE_STRB)

// Enablement/disablement of all axis signals
#define AXIS_ENABLE_ALL  0b01111111
#define AXIS_DISABLE_ALL 0b00000000
```



TIP: *These macros also enable some error checking to identify cases where your specification is incomplete.*

In addition to letting you enable or disable specific side-channel signals, the set of macros was expanded to allow the specification of `all` or `none` of the signals. This allowed for the definition of the special `hls::axis_data<>` and `hls::axis_user<>` helper classes that make it easier to pick and choose the exact signal specification. The definitions of these special helper classes are shown below:

```
// Struct: axis_data (alternative to axis)
//   DATA signal always enabled
//   All other signals are optional, disabled by default
template <typename TData,
          uint8_t EnableSignals = AXIS_ENABLE_DATA,
          std::size_t WUser = 0,
          std::size_t WId = 0,
          std::size_t WDest = 0,
          bool StrictEnablement = true>
using axis_data = axis<TData, WUser, WId, WDest,
                       (EnableSignals | AXIS_ENABLE_DATA),
                       StrictEnablement>;
```

```
// Struct: axis_user (alternative to axis)
//   USER signal always enabled
//   DATA signal always disabled
//   All other signals are optional, disabled by default
template <std::size_t WUser,
          uint8_t EnableSignals = AXIS_ENABLE_USER,
          std::size_t WId = 0,
```

```
          std::size_t WDest = 0,
          bool StrictEnablement = true>
using axis_user = axis<void, WUser, WId, WDest,
                       (EnableSignals & AXIS_DISABLE_DATA),
                       StrictEnablement>;
```

The following use cases illustrate the various options letting you customize the signals associated with the AXI4-Stream interface:

```
// Allow user to control existence of TKEEP, TSTRB and TLAST by template
parameter `EnableSideChannels`

// Enable only TKEEP
using data_t = hls::axis<int, 0, 0, 0, AXIS_ENABLE_KEEP>; Or using data_t =
hls::axis_data<int, AXIS_ENABLE_KEEP>

// Enable two channels TKEEP and TLAST
using data_t = hls::axis_data<int, AXIS_ENABLE_KEEP|AXIS_ENABLE_LAST>;

// Disable all side-channels
using data_t = hls::axis_data<int, AXIS_DISABLE_ALL>;

// Enable all side-channels
using data_t = hls::axis_data<int, AXIS_ENABLE_ALL, 4, 2, 3>;

// Error case: All signals enabled but zero size specified for WUser/WId/
WDest
using data_t = hls::axis_data<int, AXIS_ENABLE_ALL>;

// Allow user to control existence of TDATA by specifying template
parameter `T` with `void` type
// and only use TUSER channel
using data_t = hls::axis<void, 5, 0, 0, AXIS_DISABLE_ALL>; Or using data_t
= hls::axis_user<5>

// Use default enabling of TDATA, TKEEP, TSTRB and TLAST
using data_t = hls::axis<int>;
```

Port-Level Protocols for Vivado IP Flow

IMPORTANT! *The port-level protocols described here can be used in the Vivado IP flow as explained in Interfaces for Vivado IP Flow. These protocols are not supported in the Vitis kernel flow.*

By default input pointers and pass-by-value arguments are implemented as simple wire ports with no associated handshaking signal. For example, in the `vadd` function discussed in Interfaces for Vivado IP Flow, the input ports are implemented without an I/O protocol, only a data port. If the port has no I/O protocol, (by default or by design) the input data must be held stable until it is read.

By default output pointers are implemented with an associated output valid signal to indicate when the output data is valid. In the `vadd` function example, the output port is implemented with an associated output valid port (`out_r_o_ap_vld`) which indicates when the data on the port is valid and can be read. If there is no I/O protocol associated with the output port, it is difficult to know when to read the data.

TIP: *It is always a good idea to use an I/O protocol on an output.*

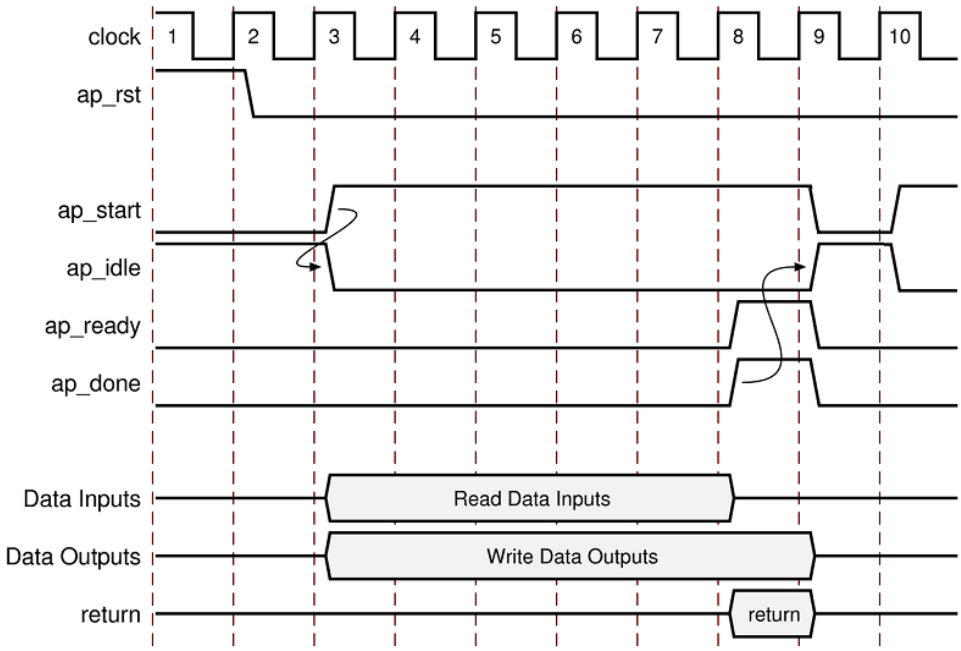
Function arguments which are both read from and written to are split into separate input and output ports. In the `vadd` function example, the `out_r` argument is implemented as both an input port `out_r_i`, and an output port `out_r_o` with associated I/O protocol port `out_r_o_ap_vld`.

If the function has a return value, an output port `ap_return` is implemented to provide the return value. When the RTL design completes one transaction, this is equivalent to one execution of the C/C++ function, the block-level protocols indicate the function is complete with the `ap_done` signal. This also indicates the data on port `ap_return` is valid and can be read.

***Note*:** The return value of the top-level function cannot be a pointer.

For the example code shown the timing behavior is shown in the following figure (assuming that the target technology and clock frequency allow a single addition per clock cycle).

Figure 48: **RTL Port Timing with Default Synthesis**



- The design starts when `ap_start` is asserted High.
- The `ap_idle` signal is asserted Low to indicate the design is operating.
- The input data is read at any clock after the first cycle. Vitis HLS schedules when the reads occur. The `ap_ready` signal is asserted High when all inputs have been read.
- When output `sum` is calculated, the associated output handshake (`sum_o_ap_vld`) indicates that the data is valid.

- When the function completes, `ap_done` is asserted. This also indicates that the data on `ap_return` is valid.
- Port `ap_idle` is asserted High to indicate that the design is waiting start again.

Port-Level I/O: No Protocol

The `ap_none` specifies that no I/O protocol be added to the port. When this is specified the argument is implemented as a data port with no other associated signals. The `ap_none` mode is the default for scalar inputs.

ap_none

The `ap_none` port-level I/O protocol is the simplest interface type and has no other signals associated with it. Neither the input nor output data signals have associated control ports that indicate when data is read or written. The only ports in the RTL design are those specified in the source code.

An `ap_none` interface does not require additional hardware overhead. However, the `ap_none` interface does requires the following:

- Producer blocks to do one of the following:
  - Provide data to the input port at the correct time, typically before the design starts.
  - Hold data for the length of a transaction until the design raises the ap_ready signal.
- Consumer blocks to read output ports when the design is done, and before it is started again.

***Note*:** The `ap_none` interface cannot be used with array arguments.

Port-Level I/O: Wire Handshakes

Interface mode `ap_hs` includes a two-way handshake signal with the data port. The handshake is an industry standard valid and acknowledge handshake. Mode `ap_vld` is the same but only has a valid port and `ap_ack` only has a acknowledge port.

Mode `ap_ovld` is for use with in-out arguments. When the in-out is split into separate input and output ports, mode `ap_none` is applied to the input port and `ap_vld` applied to the output port. This is the default for pointer arguments that are both read and written.

The `ap_hs` mode can be applied to arrays that are read or written in sequential order. If Vitis HLS can determine the read or write accesses are not sequential, it will halt synthesis with an error. If the access order cannot be determined, Vitis HLS will issue a warning.

ap_hs (ap_ack, ap_vld, and ap_ovld)

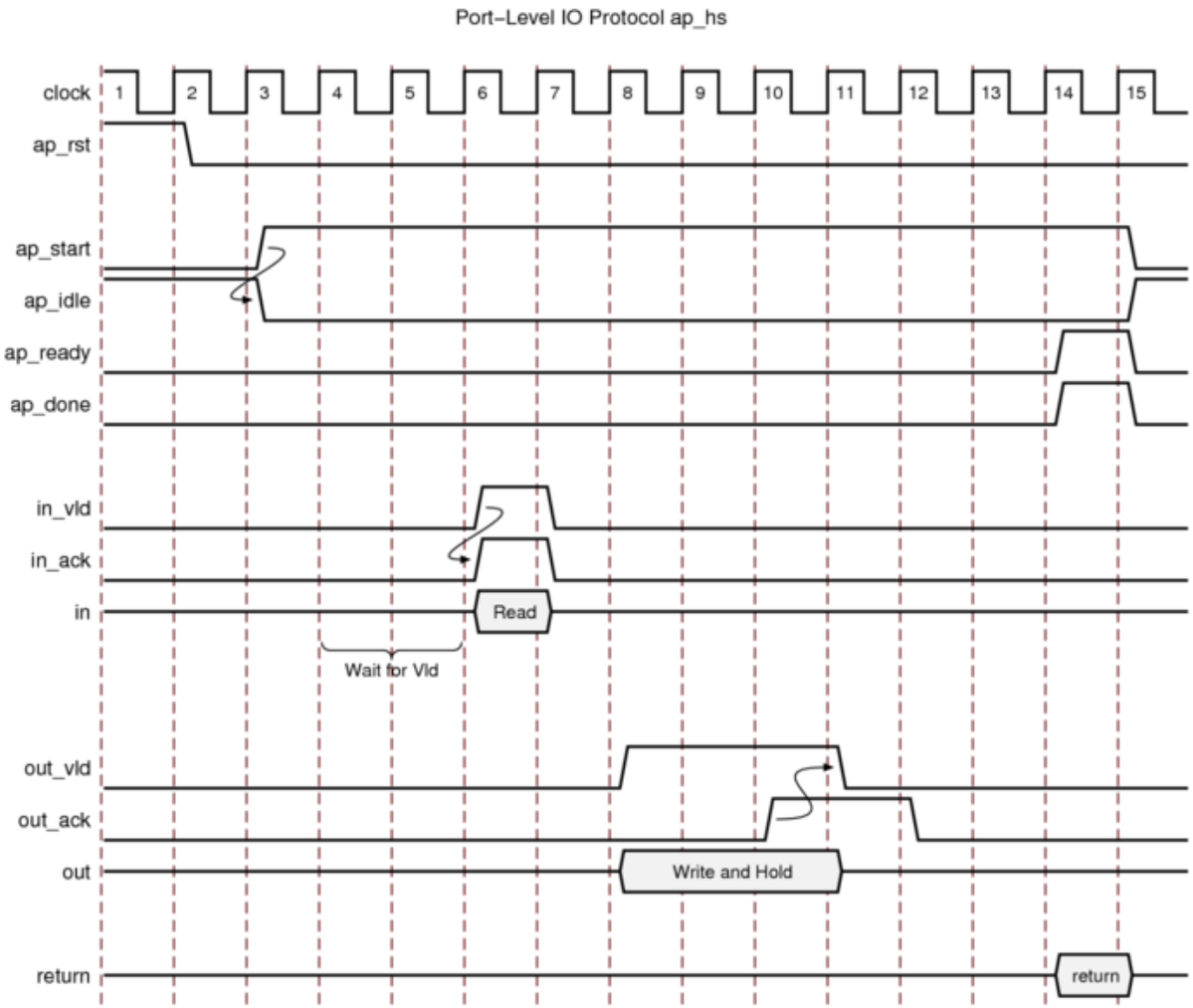
The `ap_hs` port-level I/O protocol provides the greatest flexibility in the development process, allowing both bottom-up and top-down design flows. Two-way handshakes safely perform all intra-block communication, and manual intervention or assumptions are not required for correct operation. The `ap_hs` port-level I/O protocol provides the following signals:

- Data port
- Valid signal to indicate when the data signal is valid and can be read
- Acknowledge signal to indicate when the data has been read

The following figure shows how an `ap_hs` interface behaves for both an input and output port. In this example, the input port is named `in`, and the output port is named `out`.

***Note*:** The control signals names are based on the original port name. For example, the `valid` port for data input `in` is named `in_vld`.

Figure 49: **Behavior of ap_hs Interface**



For inputs, the following occurs:

- After start is applied, the block begins normal operation.
- If the design is ready for input data but the input `valid` is Low, the design stalls and waits for the input `valid` to be asserted to indicate a new input value is present.

  ***Note*:** The preceding figure shows this behavior. In this example, the design is ready to read data input `in` on clock cycle 4 and stalls waiting for the input `valid` before reading the data.
- When the input `valid` is asserted High, an output acknowledge is asserted High to indicate the data was read.

For outputs, the following occurs:

- After start is applied, the block begins normal operation.

- When an output port is written to, its associated output `valid` signal is simultaneously asserted to indicate valid data is present on the port.
- If the associated input acknowledge is Low, the design stalls and waits for the input acknowledge to be asserted.
- When the input acknowledge is asserted, indicating the data has been read, the output `valid` is deasserted on the next clock edge.

ap_ack

The `ap_ack` port-level I/O protocol is a subset of the `ap_hs` interface type. The `ap_ack` port-level I/O protocol provides the following signals:

- Data port
- Acknowledge signal to indicate when data is consumed
  - For input arguments, the design generates an output acknowledge that is active-High in the cycle the input is read.
  - For output arguments, Vitis HLS implements an input acknowledge port to confirm the output was read.

  ***Note*:** After a write operation, the design stalls and waits until the input acknowledge is asserted High, which indicates the output was read by a consumer block. However, there is no associated output port to indicate when the data can be consumed.



CAUTION! *You cannot use C/RTL co-simulation to verify designs that use `ap_ack` on an output port.*

ap_vld

The `ap_vld` is a subset of the `ap_hs` interface type. The `ap_vld` port-level I/O protocol provides the following signals:

- Data port
- Valid signal to indicate when the data signal is valid and can be read
  - For input arguments, the design reads the data port as soon as the `valid` is active. Even if the design is not ready to read new data, the design samples the data port and holds the data internally until needed.
  - For output arguments, Vitis HLS implements an output `valid` port to indicate when the data on the output port is valid.

ap_ovld

The `ap_ovld` is a subset of the `ap_hs` interface type. The `ap_ovld` port-level I/O protocol provides the following signals:

- Data port

- Valid signal to indicate when the data signal is valid and can be read
  - For input arguments and the input half of inout arguments, the design defaults to type `ap_none`.
  - For output arguments and the output half of inout arguments, the design implements type `ap_vld`.

Port-Level I/O: Memory Interface Protocol

Array arguments are implemented by default as an `ap_memory` interface using word-addressing. This is a standard block RAM interface with data, address, chip-enable, and write-enable ports.

An `ap_memory` interface can be implemented as a single-port of dual-port interface. If Vitis HLS can determine that using a dual-port interface will reduce the initial interval, it will automatically implement a dual-port interface. The BIND_STORAGE pragma or directive can be used to specify the memory resource and if this directive is specified on the array with a single-port block RAM, a single-port interface will be implemented. Conversely, if a dual-port interface is specified using the BIND_STORAGE pragma and Vitis HLS determines this interface provides no benefit it will automatically implement a single-port interface.

If the array is accessed in a sequential manner an `ap_fifo` interface can be used. As with the `ap_hs` interface, Vitis HLS will halt if it determines the data access is not sequential, report a warning if it cannot determine if the access is sequential or issue no message if it determines the access is sequential. The `ap_fifo` interface can only be used for reading or writing, not both.

ap_memory

The `ap_memory` interface port-level I/O protocol implements arrays for arguments. This type of port-level I/O protocol connects to memory elements (for example, RAMs and ROMs) when the implementation requires random accesses to the memory address locations.

***Note*:** If you only need sequential access to the memory element, use the `ap_fifo` interface instead. The `ap_fifo` interface reduces the hardware overhead.

The `ap_memory` interface uses word-based addressing, and generates a chip enabled control signal.

Since the v2024.1 release, the `ap_memory` interface appears as a single, grouped port. In IP integrator, you can use a single connection to create connections to all ports.

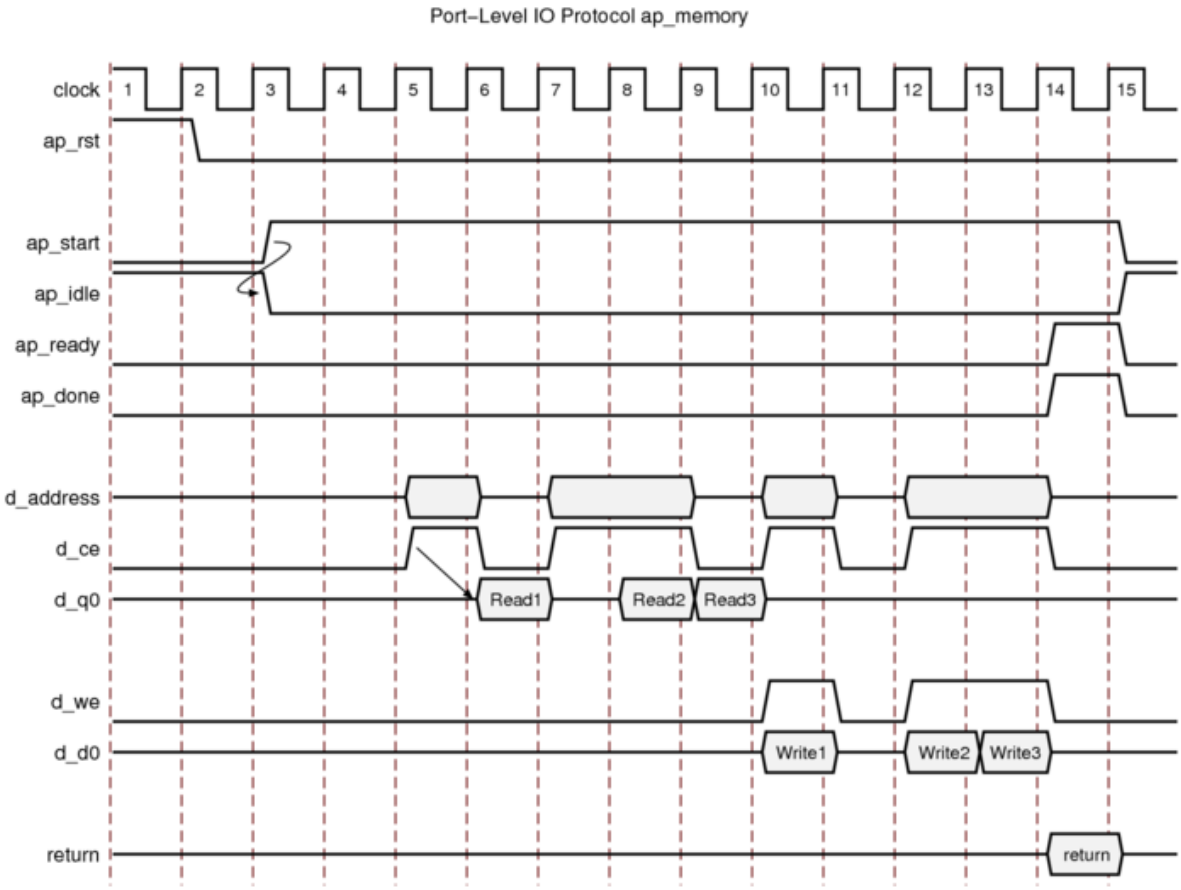
When using a memory interface, specify the implementation using the BIND_STORAGE pragma. If no target is specified for the arrays, Vitis HLS determines whether to use a single or dual-port RAM interface.



TIP: *Before running synthesis, ensure array arguments are targeted to the correct memory type using the BIND_STORAGE pragma. Re-synthesizing with different memories can result in a different schedule and RTL.*

The following figure shows an array named `d` specified as a single-port block RAM. The port names are based on the C/C++ function argument. For example, if the C/C++ argument is `d`, the chip-enable is `d_ce`, and the input data is `d_q0` based on the `output/q` port of the block RAM.

Figure 50: **Behavior of ap_memory Interface**



After reset, the following occurs:

- After start is applied, the block begins normal operation.
- Reads are performed by applying an address on the output address ports while asserting the output signal `d_ce`.

  Note: For a default block RAM, the design expects the input data `d_q0` to be available in the next clock cycle. You can use the BIND_STORAGE pragma to indicate the RAM has a longer read latency.
- Write operations are performed by asserting output ports `d_ce` and `d_we` while simultaneously applying the address and output data `d_d0`.

ap_fifo

When an output port is written to, its associated output `valid` signal interface is the most hardware-efficient approach when the design requires access to a memory element and the access is always performed in a sequential manner, that is, no random access is required. The `ap_fifo` port-level I/O protocol supports the following:

- Allows the port to be connected to a FIFO
- Enables complete, two-way `empty-full` communication
- Works for arrays, pointers, and pass-by-reference argument types

***Note*:** Functions that can use an `ap_fifo` interface often use pointers and might access the same variable multiple times. To understand the importance of the `volatile` qualifier when using this coding style, see Multi-Access Pointers on the Interface.

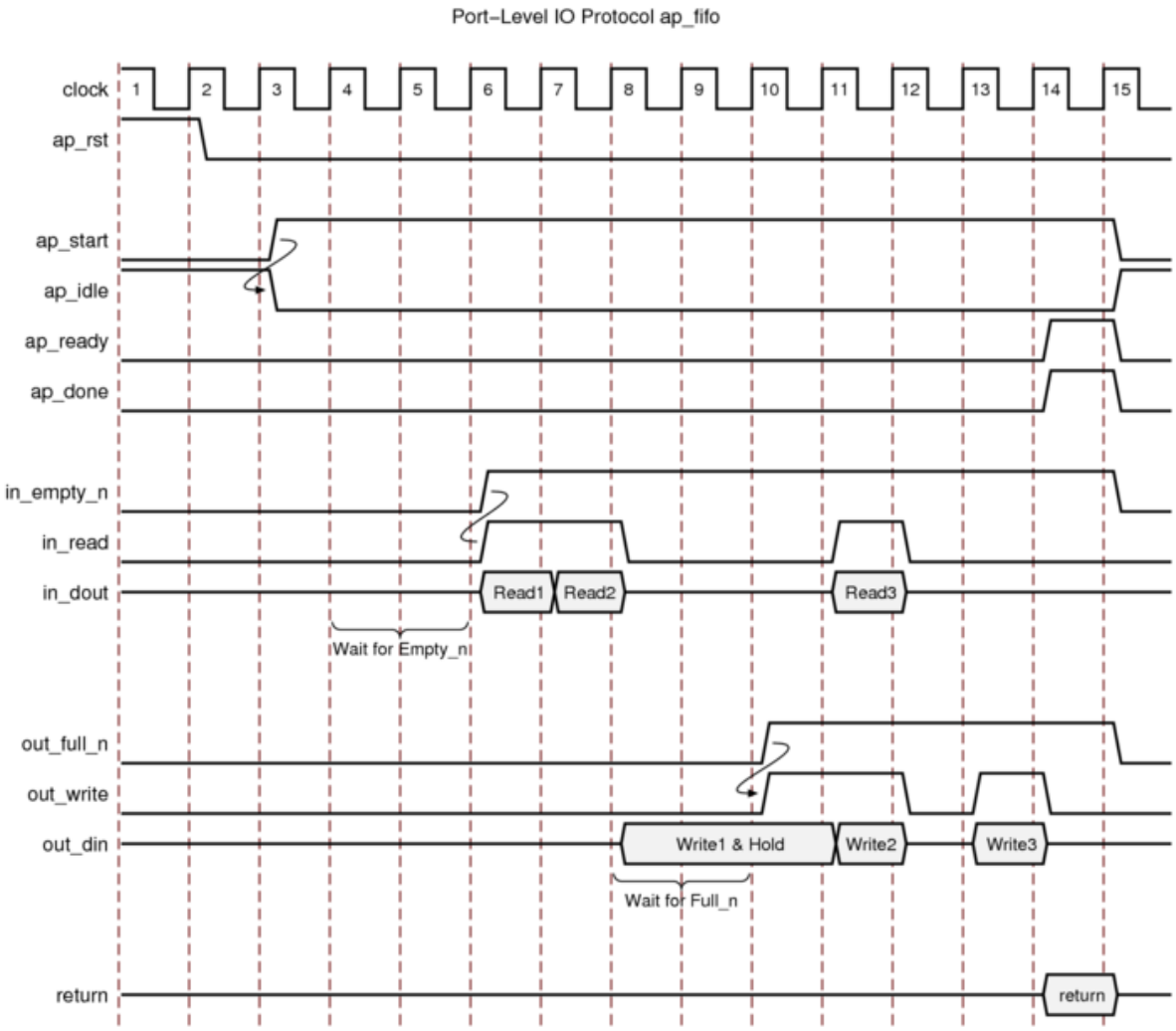
In the following example, `in1` is a pointer that accesses the current address, then two addresses above the current address, and finally one address below.

```
void foo(int* in1, ...) {
 int data1, data2, data3;
       ...
 data1= *in1;
 data2= *(in1+2);
 data3= *(in1-1);
 ...
}
```

If `in1` is specified as an `ap_fifo` interface, Vitis HLS checks the accesses, determines the accesses are not in sequential order, issues an error, and halts. To read from non-sequential address locations, use the `ap_memory` interface.

You cannot specify an `ap_fifo` interface on an argument that is both read from and written to. You can only specify an `ap_fifo` interface on an input or an output argument. A design with input argument `in` and output argument `out` specified as `ap_fifo` interfaces behaves as shown in the following figure.

Figure 51: **Behavior of ap_fifo Interface**



For inputs, the following occurs:

- After ap_start is applied, the block begins normal operation.
- If the input port is ready to be read but the FIFO is empty as indicated by input port `in_empty_n` Low, the design stalls and waits for data to become available.
- When the FIFO contains data as indicated by input port `in_empty_n` High, an output acknowledge `in_read` is asserted High to indicate the data was read in this cycle.

For outputs, the following occurs:

- After start is applied, the block begins normal operation.
- If an output port is ready to be written to but the FIFO is full as indicated by `out_full_n` Low, the data is placed on the output port but the design stalls and waits for the space to become available in the FIFO.

- When space becomes available in the FIFO as indicated by `out_full_n` High, the output acknowledge signal `out_write` is asserted to indicate the output data is `valid`.
- If the top-level function or the top-level loop is pipelined using the `-rewind` option, Vitis HLS creates an additional output port with the suffix `_lwr`. When the last write to the FIFO interface completes, the `_lwr` port goes active-High.

Programming Model for Multi-Port Access in HBM

High Bandwidth Memory (or HBM) provides high bandwidth by splitting arrays into different banks/pseudo-channels in the design. This is a common practice in partitioning an array into different memory regions in high-performance computing. The software application interacting with the hardware allocates a single buffer, which will be spread across the pseudo-channels.

Enabling Dependence Analysis on Different Arguments

Vitis HLS considers different pointers to be independent channels, and removes any dependency analysis. But the software application allocates a single buffer for both pointers, and this lets the tool maintain dependency analysis through `pragma HLS ALIAS`. The ALIAS pragma informs data dependence analysis about the pointer distance. Refer to the ALIAS pragma or directive for more information.

In the following code example, the kernel `arg0` is allocated in `bank0` and kernel `arg1` is allocated in `bank1`. The pointer distance should be specified in the `distance` option of the ALIAS pragma as shown below:

```
//Assume that the host code looks like this:
int *buf = clCreateBuffer(ctxt, CL_MEM_READ_ONLY, 2*bank_size, ...);
clSetKernelArg(kernel, 0, 0x20000000, buf);            // bank0
clSetKernelArg(kernel, 1, 0x20000000, buf+bank_size); // bank1

//The ALIAS pragma informs data dependence analysis about the pointer
distance
void kernel(int *bank0, int *bank1, ...)
{
#pragma HLS alias ports=bank0,bank1 distance=bank_size
```

The ALIAS pragma can be specified using one of the following forms:

- Constant distance:

```
#pragma HLS alias ports=arr0,arr1,arr2,arr3 distance=1024
```

- Variable distance:

```
#pragma HLS alias ports=arr0,arr1,arr2,arr3 offset=0,512,1024,2048
```

Constraints:

- The depths of all the ports in the interface pragma must be the same

- All ports must be assigned to different bundles, bound to different HBM controllers
- The number of ports specified in the second form must be the same as the number of offsets specified, one offset per port. `#pragma HLS interface offset=off` is not supported
- Each port can only be used in one ALIAS pragma

Improving HBM Bandwidth with Multiple M_AXI Interfaces

Vitis HLS supports features on top-level array or pointer arguments to improve the parallelism and throughput achievable from C++ code with a few code changes.

Improvements to the ARRAY_PARTITION Pragma

The ARRAY_PARTITION pragma or directive can be applied to arrays on top-level arguments, at the interface of the top-level function. Used alongside the INTERFACE pragma, this combination lets Vitis HLS create multiple M_AXI interfaces for a single software argument to use with HBM Pseudo Channels. Use ARRAY_PARTITION `factor=N` to create N point-to-point connections between the M_AXI interfaces and HBM Pseudo Channels (or HBM_PC) to achieve up to N times the throughput of a single interface.

For the micro-architecture, an explicit parallelization of the array accesses and associated datapath is needed to match the increase of M_AXI interfaces. This translates to a similar parallelization from a coding perspective.

Array partitioning types can be complete, cyclic, or block, though for this application only cyclic or block are used. Each type can lead to a different micro-architecture.

- Cyclic partitions can be used within a loop: consecutive accesses of the array will connect to different M_AXI and their matching HBM_PC, and you can partially unroll the loop with the same factor.

```
constexpr int partitions = 4;
constexpr int NWORDS=32<<20;
void example(int a[NWORDS]  , int b[1]) {
    #pragma HLS INTERFACE m_axi port=a depth=NWORDS bundle=gmem
max_widen_bitwidth=512
    #pragma HLS ARRAY_PARTITION cyclic variable=a factor=partitions
    #pragma HLS INTERFACE m_axi port=b depth=1 bundle=outmem
    int tot=0;
accessloop:
    for (int i=0; i < NWORDS; ++i) {
        #pragma HLS PIPELINE II=1
        #pragma HLS UNROLL factor=partitions
        tot+= a[i] * i; // computation on a[i]
    }
    b[0]=tot;
}
```

The preceding example shows the use of a sized array rather than a pointer, and the depth option is provided with the INTERFACE pragma. Both methods are equivalent and at least one is needed for C-RTL co-simulation, to size the RTL buffers correctly. The sizing information on the arguments and interfaces is not needed when used with the Vitis target flow as pointers are sized to 64 bits automatically. Additionally, the cyclic partitioning is size-independent.

- Block partitions can be used when the design includes several instances of the same compute function, and each compute function will access a different block of the array via different M_AXI and their matching HBM_PC.

```
constexpr int partitions = 4;
constexpr int NWORDS=32<<20;
void my_pe(int a[NWORDS/partitions], int &b, int offset) { // PE
processes a fraction of the data
    // offset is needed to adjust the coefficient to multiply the array
value
    int tot=0;
accessloop:
    for (int i=0; i < NWORDS/partitions; ++i) {
        #pragma HLS PIPELINE II=1
        tot += a[i] * (i+offset); // computation on a[i]
    }
    b=tot;
}

void example(int a[NWORDS  ], int b[1]) {
    #pragma HLS INTERFACE m_axi port=a depth=NWORDS bundle=gmem
max_widen_bitwidth=MAXWBW
    #pragma HLS ARRAY_PARTITION block variable=a factor=partitions
    #pragma HLS INTERFACE m_axi port=b depth=1 bundle=outmem

    int partial_totals[partitions], tot;
    #pragma HLS ARRAY_PARTITION complete variable=partial_totals
    #pragma HLS DATAFLOW
instanceloop_unrolled:
    for (int i=0; i < partitions; ++i) {
        #pragma HLS UNROLL
        // each instance accesses a different block
        my_pe( &a[NWORDS*i/partitions], partial_totals[i], NWORDS*i/
partitions);
    }

reduce:
    for (int i=tot=0; i < partitions; ++i) {
        #pragma HLS PIPELINE off
        tot+=partial_totals[i];
    }
    b[0]=tot;
}
```

In the above example, the array sizing information is needed because the block partitioning is dependent on the size of the original array.

The M_AXI interfaces generated will have their bundle names derived from the original bundle name, with decimal suffix added. For example, pragma HLS INTERFACE bundle=gmem with a partition factor=4 will generate 4 M_AXI named m_axi_gmem_0, m_axi_gmem_1, m_axi_gmem_2 and m_axi_gmem_3. You will need to update the connectivity settings for the system to integrate all the M_AXI interfaces. Refer to *Mapping Kernel Ports to Memory* in *Embedded Design Development Using Vitis* (UG1701) for additional information.

In the examples above, the function signature used by the software application to call the `example()` module will change because of the use of `ARAY_PARTITION factor=4`. The function signature changes from `example(int a[NWORDS], int b[1])` to the following:

```
void example(
    int a_0[NWORDS/4],
    int a_1[NWORDS/4],
    int a_2[NWORDS/4],
    int a_3[NWORDS/4],
    int b[1]);
```

Generated Helper Functions for Host Side Array Partitioning

The software application will have to partition the buffer containing the array into the same cyclic or block partitioning used by the HLS design as described above. To help with this task, Vitis HLS generates helper functions to split host array buffers into multiple partitions, and the reverse helper functions to recombine several partitions into one array. Those helper functions are:

- `<top_name>_Set_<array_name>` is used to partition an array into several partitions. For example, with a `factor=4` and `int` datatype:

  ```
  void <DUT>_Set_<ARG>(int *dst[4], int *src, unsigned long long
  num_elements);
  ```

- `<top_name>_Get_<array_name>` is used to recombine several partitions into a single array. For example with a `factor=4` and `int` datatype:

  ```
  void <DUT>_Get_<ARG>(int *dst, int *src[4], unsigned long long
  num_elements)
  ```

  TIP: *The order of the arguments follows the convention:* `destination(s)`, `source(s)`, `number of elements`.

- Generated helper functions are available as assembly files: `./project/solution/impl/ip/drivers/example_v1_0/src/hbm_helper_x86_64.s & hbm_helper_aarch64.s`
- The generated helper functions are also available as compiled files: `./project/solution/impl/ip/drivers/example_v1_0/src/hbm_helper_x86_64.o & hbm_helper_aarch64.o`

***Note*:** Header files are not generated automatically, so you need to include function declarations in the software application. The following example shows a `factor=4` and `int` datatype:

```
extern "C" void example_Set_a(int **, int *, long);
extern "C" void example_Get_a(int *, int **, long);
```

Example usage:

```
int *host_in=new int[NWORDS];
for (size_t i = 0; i < NWORDS; ++i) {
    host_in[i] = ... // initialize the host input memory
}

xrt::bo bo_Inputs[partitions];
int* bufIn_map[partitions];
for (int i = 0; i < partitions; i++) {
    // buffer object matching kernel arguments/memory banks
    bo_Inputs[i] = xrt::bo(device,NWORDS*sizeof(int)/partitions,
krnl.group_id(i));
    // Map buffer object data to user pointer for manipulation
    bufIn_map[i] = bo_Inputs[i].template map<int*>();
}

// partition the input data into the mapped partitions
example_Set_a(bufIn_map, host_in, NWORDS);

xrt::run run(krnl); // run object for that kernel

for (int i = 0; i < partitions; i++) {
    // sync updated bo contents to board
    bo_Inputs[i].sync(XCL_BO_SYNC_BO_TO_DEVICE);
}
for (int i = 0; i < partitions; i++) {
    // set arguments according to function signature
    run.set_arg(i,bo_Inputs[i]);
}
...
run.start();
run.wait();
```

Managing Interfaces with SSI Technology Devices

Certain AMD devices use stacked silicon interconnect (SSI) technology. In these devices, the total available resources are divided over multiple super logic regions (SLRs). The connections between SLRs use super long line (SLL) routes. SLL routes incur delays costs that are typically greater than standard FPGA routing. To ensure designs operate at maximum performance, use the following guidelines:

- Register all signals that cross between SLRs at both the SLR output and SLR input.
- You do not need to register a signal if it enters or exits an SLR via an I/O buffer.
- Ensure that the logic created by Vitis HLS fits within a single SLR.

***Note*:** When you select an SSI technology device as the target technology, the utilization report includes details on both the SLR usage and the total device usage.

If the logic is contained within a single SLR device, the HLS tool provides a `syn.rtl.register_all_io` configuration command. If the option is enabled, all inputs and outputs are registered. If disabled, none of the inputs or outputs are registered.

Vitis HLS Memory Layout Model

The Vitis application acceleration development flow provides a framework for developing and delivering FPGA accelerated applications using standard programming languages for both software and hardware components. The software component, or host program, is developed using C/C++ to run on x86 or embedded processors, with OpenCL/ Native XRT API calls to manage run time interactions with the accelerator. The hardware component, or kernel (that runs on the actual FPGA card/platform), can be developed using C/C++, OpenCL C, or RTL. The Vitis software platform promotes concurrent development and test of the hardware and software elements of a heterogeneous application. Due to this, the software program that runs on the host computer needs to communicate with the acceleration kernel that runs on the FPGA hardware model using well-defined interfaces and protocols. As a result, it becomes important to define the exact memory model that is used so that the data that is being read/ written can be correctly processed. The memory model defines the way data is arranged and accessed in computer memory. It consists of two separate but related issues: data alignment and data structure padding. In addition, the Vitis HLS compiler supports the specification of special attributes (and pragmas) to change the default data alignment and data structure padding rules.

Data Alignment

Software programmers are conditioned to think of memory as a simple array of bytes and the basic data types are composed of one or more blocks of memory. However, the computer's processor does not read from and write to memory in single byte-sized chunks. Instead, today's modern CPUs access memory in 2, 4, 8, 16, or even 32-byte chunks at a time - although 32 bit and 64 bit instruction set architecture (ISA) architectures are the most common. Due to how the memory is organized in your system, the addresses of these chunks should be multiples of their sizes. If an address satisfies this requirement, then it is said to be *aligned*. The difference between how high-level programmers think of memory and how modern processors actually work with memory is pretty important in terms of application correctness and performance. For example, if you don't understand the address alignment issues in your software, the following situations are all possible:

- your software will run slower
- your application will lock up/hang
- your operating system can crash
- your software will silently fail, yielding incorrect results

The C++ language provides a set of fundamental types of various sizes. To make manipulating variables of these types fast, the generated object code will try to use CPU instructions that read/write the whole data type at once. This in turn means that the variables of these types should be placed in memory in a way that makes their addresses suitably aligned. As a result, besides size, each fundamental type has another property: its alignment requirement. It may seem that the fundamental type's alignment is the same as its size. This is not generally the case since the most suitable CPU instruction for a particular type may only be able to access a part of its data at a time. For example, a 32-bit x86 GNU/Linux machine may only be able to read at most 4 bytes at a time so a 64-bit `long long` type will have a size of 8 and an alignment of 4. The following table shows the size and alignment (in bytes) for the basic native data types in C/C++ for both 32-bit and 64-bit x86-64 GNU/Linux machines.

Table 13: **Data Types**

Type	32-bit x86 GNU/Linux		64-bit x86 GNU/Linux	
	Size	Alignment	Size	Alignment
bool	1	1	1	1
char	1	1	1	1
short int	2	2	2	2
int	4	4	4	4
long int	4	4	8	8
long long int	8	4	8	8
float	4	4	4	4
double	8	4	8	8
long double	12	4	16	16
void*	4	4	8	8

Given the above arrangement, why does a programmer need to change the alignment? There are several reasons but the main reason will be to trade-off between memory requirements and performance. When you are sending data back and forth from the host computer and the accelerator, every byte that is transmitted has a cost. Fortunately, the GCC C/C++ compiler provides the language extension `__attribute__ ((aligned(X)))` in order to change the default alignment for the variable, structures/classes, or a structure field, measured in bytes. For example, the following declaration causes the compiler to allocate the global variable x on a 16-byte boundary.

```
int x __attribute__ ((aligned (16))) = 0;
```

The `__attribute__((aligned (X)))` does not change the sizes of variables it is applied to, but may change the memory layout of structures by inserting padding between elements of the struct. As a result, the size of the structure will change. If you don't specify the alignment factor in an aligned attribute, the compiler automatically sets the alignment for the declared variable or field to the largest alignment used for any data type on the target machine you are compiling

for. Doing this can often make copy operations more efficient because the compiler can use whatever instructions copy the biggest chunks of memory when performing copies to or from the variables or fields that you have aligned this way. The `aligned` attribute can only increase the alignment and can never decrease it. The C++ function offsetof can be used to determine the alignment of each member element in a structure.

Data Structure Padding

As shown in the table in Data Alignment, the native data types have a well-defined alignment structure but what about user-defined data types? The C++ compiler also needs to make sure that all the member variables in a struct or class are properly aligned. For this, the compiler may insert *padding* bytes between member variables. In addition, to make sure that each element in an array of a user-defined type is aligned, the compiler may add some extra padding after the last data member. Consider the following example:

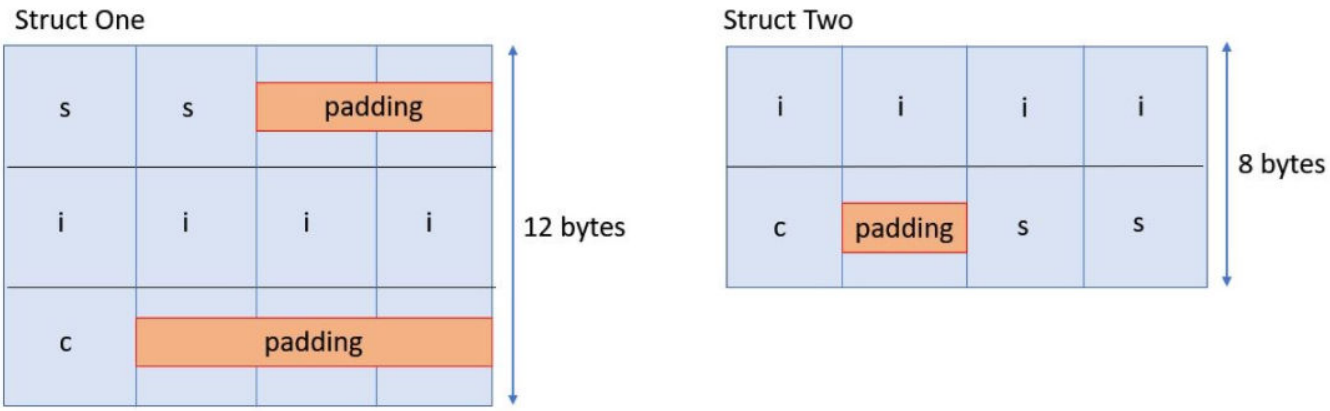
```
struct One
{
    short int s;
    int i;
    char c;
}
```

```
struct Two
{
   int i;
   char c;
   short int s;
}
```

The GCC compiler always assumes that an instance of `struct One` will start at an address aligned to the most strict alignment requirement of all of the struct's members, which is `int` in this case. This is actually how the alignment requirements of user-defined types are calculated. Assuming the memory are on x86-64 alignment with `short int` having the alignment of 2 and `int` having an alignment of 4, to make the `i` data member of `struct One` suitably aligned, the compiler needs to insert two extra bytes of padding between `s` and `i` to create alignment, as shown in the figure below. Similarly, to align data member `c`, the compiler needs to insert three bytes after `c`.

In the case of `struct One`, the compiler will infer a total size of 12 bytes based on the arrangement of the elements of the struct. However, if the elements of the struct are reordered (as shown in `struct Two`), the compiler is now able to infer the smaller size of 8 bytes.

Figure 52: **Padding of Structs**



By default, the C/C++ compiler will lay out members of a struct in the order in which they are declared, with possible padding bytes inserted between members, or after the last member, to ensure that each member is aligned properly. However, the GCC C/C++ compiler provides a language extension, `__attribute__((packed))` which tells the compiler not to insert padding but rather allow the struct members to be misaligned. For example, if the system normally requires all `int` objects to have 4-byte alignment, the usage of `__attribute__((packed))` can cause `int` struct members to be allocated at odd offsets.

Usage of `__attribute__((packed))` must be carefully considered because accessing unaligned memory can cause the compiler to insert code to read the memory byte by byte instead of reading multiple chunks of memory at one time.

Vitis HLS Alignment Rules and Semantics

Given the behavior of the GCC compiler described previously, this section will detail how Vitis HLS uses `aligned` and `packed` attributes to create efficient hardware. First, you need to understand the Aggregate and Disaggregate features in Vitis HLS. Structures or class objects in the code, for instance internal and global variables, are disaggregated by default. Disaggregation implies that the structure/class is decomposed into separate objects, one for each struct/class member. The number and type of elements created are determined by the contents of the struct itself. Arrays of structs are implemented as multiple arrays, with a separate array for each member of the struct.

However, structs used as arguments to the top-level function are kept aggregated by default. Aggregation implies that all the elements of a struct are collected into a single wide vector. This allows all members of the struct to be read and written simultaneously. The member elements of the struct are placed into the vector in the order in which they appear in the C/C++ code: the first element of the struct is aligned on the LSB of the vector and the final element of the struct is aligned with the MSB of the vector. Any arrays in the struct are partitioned into individual array elements and placed in the vector from lowest to the highest order.

Table 15: **Interface Arguments and Internal Variables**

	Behavior without `AGGREGATE` pragma		**Behavior with `AGGREGATE` pragma (compact=auto or not specified)**	
	Interface Argument	**Internal Variable**	**Interface Argument**	**Internal variable**
AXI protocol interface (`m_axi`/`s_axilite`/`axis`)	aggregate compact=**none**	N/A	compact=**none**	N/A
Struct/Class containing `hls::stream` object	**Automatically disaggregate the struct/class**	**Automatically disaggregate the struct/class**	N/A	N/A
other interface protocols	aggregate compact=**bit**	**Automatically disaggregated**	compact=**bit**	compact=**bit**

The goal of the default aggregation behavior in Vitis HLS is to use an x86_64-gnu-linux memory layout at the top level hardware interface while optimizing the internal hardware for better quality of results (QoR). The above table shows the default behavior of Vitis HLS. Two modes are shown in the table: the default mode where the `AGGREGATE` pragma is not specified by the user, and the case where the `AGGREGATE` pragma is specified by the user.

In the case of AXI4 interfaces (`m_axi`/`s_axilite`/`axis`), a structure is padded by default according to the order of elements of the struct as explained in Data Structure Padding. This aggregates the structure to a size that is the closest power of 2, and so some padding may be applied in this case. This in effect infers the `compact=none` option on the AGGREGATE pragma.

In the case of other interface protocols, the struct is packed at the bit-level, so the aggregated vector is only the size of the various elements of the struct, This in effect infers the `compact=bit` option on the AGGREGATE pragma.

The only exception to the above rules is when using `hls::stream` in the interface indirectly (i.e. the `hls::stream` object is specified inside a struct/class that is then used as the type of an interface port). The struct containing the `hls::stream` object is always disaggregated into its individual member elements.

Examples of Aggregation

Aggregate Memory Mapped Interface

This is an example of the AGGREGATE pragma or directive for an `m_axi` interface. The following is the aggregation_of_m_axi_ports example available on GitHub.

```
struct A {
  char foo;     // 1 byte
  short bar;    // 2 bytes
};

int dut(A* arr) {
```

```
#pragma HLS interface m_axi port=arr depth=10
#pragma HLS aggregate variable=arr compact=auto
  int sum = 0;
  for (unsigned i=0; i<10; i++) {
    auto tmp = arr[i];
    sum += tmp.foo + tmp.bar;
  }
  return sum;
}
```

For the above example, the size of the `m_axi` interface port `arr` is 3 bytes (or 24 bits) but due to the `AGGREGATE compact=auto` pragma, the size of the port will be aligned to 4 bytes (or 32 bits) as this is the closest power of 2. Vitis HLS will issue the following message in the log file:

```
INFO: [HLS 214-241] Aggregating maxi variable 'arr' with compact=none mode
in 32-bits (example.cpp:19:0)
```

TIP: *The message above is only issued if the `AGGREGATE` pragma is specified. But even without the pragma, the tool will automatically aggregate and pad the interface port `arr` to 4 bytes as the default behavior for an AXI interface port.*

Aggregate Structs on the Interface

This is an example of the AGGREGATE pragma or directive for an `ap_fifo` interface. The following is the aggregation_of_struct example available on GitHub.

```
struct A {
  int myArr[3];      // 4 bytes per element (12 bytes total)
  ap_int<23> length; // 23 bits
};

int dut(A arr[N]) {
#pragma HLS interface ap_fifo port=arr
#pragma HLS aggregate variable=arr compact=auto
  int sum = 0;
  for (unsigned i=0; i<10; i++) {
    auto tmp = arr[i];
    sum += tmp.myArr[0] + tmp.myArr[1] + tmp.myArr[2] + tmp.length;
  }
  return sum;
}
```

For `ap_fifo` interface, the struct will packed at the bit-level with or without aggregate pragma.

In the above example, the `AGGREGATE` pragma will create a port of size 119 bits for port `arr`. The array `myArr` will take 12 bytes (or 96 bits) and the element `length` will take 23 bits for a total of 119 bits. Vitis HLS will issue the following message in the log file:

```
INFO: [HLS 214-241] Aggregating fifo (array-to-stream) variable 'arr' with
compact=bit mode
        in 119-bits (example.cpp:19:0)
```

Aggregate Nested Struct Port

This is an example of the AGGREGATE pragma or directive in the Vivado IP flow. The following is the aggregation_of_nested_structs example available on GitHub.

```
#define N 8

struct T {
   int m;   // 4 bytes
   int n;   // 4 bytes
   bool o;  // 1 byte
};

struct S {
   int p;   // 4 bytes
   T q;     // 9 bytes
};
  void top(S a[N], S b[N], S c[N]) {
#pragma HLS interface bram port=c
#pragma HLS interface ap_memory port=a
#pragma HLS aggregate variable=a compact=byte
#pragma HLS aggregate variable=b compact=bit
#pragma HLS aggregate variable=c compact=byte
    for (int i=0; i<N; i++) {
        c[i].q.m = a[i].q.m + b[i].q.m;
        c[i].q.n = a[i].q.n - b[i].q.n;
        c[i].q.o = a[i].q.o || b[i].q.o;
        c[i].p = a[i].q.n;
    }
}
```

In the above example, the aggregation algorithm will create a port of size 104 bits for ports `a`, and `c` as the `compact=byte` option was specified in the aggregate pragma but the `compact=bit` default option is used for port `b` and its packed size will be 97 bits. The nested structures `S` and `T` are aggregated to encompass three 32 bit member variables (p, m, and n) and one bit/byte member variable (`o`).



TIP: *This example uses the Vivado IP flow to illustrate the aggregation behavior. In the Vitis kernel flow, port `b` will be automatically inferred as an `m_axi` port and will not allow the `compact=bit` setting.*

Vitis HLS will issue the following messages in the log file:

```
INFO: [HLS 214-241] Aggregating bram variable 'b' with compact=bit mode in
97-bits (example.cpp:19:0)
INFO: [HLS 214-241] Aggregating bram variable 'a' with compact=byte mode in
104-bits (example.cpp:19:0)
INFO: [HLS 214-241] Aggregating bram variable 'c' with compact=byte mode in
104-bits (example.cpp:19:0)
```

Examples of Disaggregation

Disaggregate AXIS Interface

This is an example of the DISAGGREGATE pragma or directive for an `axis` interface. The following is the disaggregation_of_axis_port example available on GitHub.

Table 16: **Disaggregated Struct on AXIS Interface**

HLS Source Code	Synthesized IP Module

`#define N 10`
 	
`struct A {`
`  char c;`
`  int i;`
`};`
 	
`void dut(A in[N], A out[N]) {`
`#pragma HLS interface axis port=in`
`#pragma HLS interface axis port=out`
`#pragma HLS disaggregate variable=in`
`#pragma HLS disaggregate variable=out`
`  int sum = 0;`
`  for (unsigned i=0; i<N; i++) {`
`  out[i].c = in[i].c;`
`  out[i].i = in[i].i;`
`  }`
`}`
	`module dut (`
	`ap_local_block,`
	`ap_local_deadlock,`
	`ap_clk,`
	`ap_rst_n,`
	`ap_start,`
	`ap_done,`
	`ap_idle,`
	`ap_ready,`
	`in_c_TVALID,`
	`in_i_TVALID,`
	`out_c_TREADY,`
	`out_i_TREADY,`
	`in_c_TDATA,`
	`in_c_TREADY,`
	`in_i_TDATA,`
	`in_i_TREADY,`
	`out_c_TDATA,`
	`out_c_TVALID,`
	`out_i_TDATA,`
	`out_i_TVALID`
	`);`

In the above disaggregation example, the struct arguments `in` and `out` are mapped to AXIS interfaces, and then disaggregated. This results in Vitis HLS creating two AXI streams for each argument: `in_c`, `in_i`, `out_c` and `out_i`. Each member of the struct `A` becomes a separate stream.

The RTL interface of the generated module is shown on the right above where the member elements `c` and `i` are individual AXI stream ports, each with its own TVALID, TREADY and TDATA signals.

Vitis HLS will issue the following messages in the log file:

```
INFO: [HLS 214-210] Disaggregating variable 'in' (example.cpp:19:0)
INFO: [HLS 214-210] Disaggregating variable 'out' (example.cpp:19:0)
```

Disaggregate HLS::STREAM

This is an example of the DISAGGREGATE pragma or directive when used with the `hls::stream` type.

Table 17: **Disaggregated Struct of HLS::STREAM**

HLS Source Code	Synthesized IP Module

`#define N 1024`
 	
`struct A {,`
`  hls::stream<int> s_in;`
`  long arr[N];`
`};`
 	
`long dut(struct A &d) {`
`  long sum = 0;`
`  while(!d.s_in.empty())`
`  sum += d.s_in.read();`
`  for (unsigned i=0; i<N; i++)`
`  sum += d.arr[i];`
`  return sum;`
`}`
	`module dut (`
	`ap_local_block,`
	`ap_local_deadlock,`
	`ap_clk,`
	`ap_rst,`
	`ap_start,`
	`ap_done,`
	`ap_idle,`
	`ap_ready,`
	`d_s_in_dout,`
	`d_s_in_empty_n,`
	`d_s_in_read,`
	`d_arr_ce0,`
	`d_arr_q0,`
	`ap_return`
	`);`

Using an `hls::stream` object inside a structure that is used in the interface will cause the struct port to be automatically disaggregated by the Vitis HLS compiler. As shown in the above example, the generated RTL interface will contain separate RTL ports for the `hls::stream` object `s_in` (named `d_s_in_*`) and separate RTL ports for the array `arr` (named `d_arr_*`).

Vitis HLS will issue the following messages in the log file:

```
INFO: [HLS 214-210] Disaggregating variable 'd'
INFO: [HLS 214-241] Aggregating fifo (hls::stream) variable 'd_s_in' with
compact=bit mode in 32-bits
```

Impact of Struct Size on Pipelining

The size of a struct used in a function interface can adversely impact pipelining of loops in that function that have access to the interface in the loop body. Consider the following code example which has two M_AXI interfaces:

```
struct A { /* Total size = 192 bits (32 x 6) or 24 bytes */
    int s_1;
    int s_2;
    int s_3;
    int s_4;
    int s_5;
    int s_6;
};

void read(A *a_in, A buf_out[NUM]) {
READ:
    for (int i = 0; i < NUM; i++)
    {
        buf_out[i] = a_in[i];
    }
}

void compute(A buf_in[NUM], A buf_out[NUM], int size) {
COMPUTE:
```

```
    for (int j = 0; j < NUM; j++)
    {
        buf_out[j].s_1 = buf_in[j].s_1 + size;
        buf_out[j].s_2 = buf_in[j].s_2;
        buf_out[j].s_3 = buf_in[j].s_3;
        buf_out[j].s_4 = buf_in[j].s_4;
        buf_out[j].s_5 = buf_in[j].s_5;
        buf_out[j].s_6 = buf_in[j].s_6 % 2;
    }
}

void write(A buf_in[NUM], A *a_out) {
    WRITE:
    for (int k = 0; k < NUM; k++)
    {
        a_out[k] = buf_in[k];
    }
}

void dut(A *a_in, A *a_out, int size)
{
#pragma HLS INTERFACE m_axi port=a_in bundle=gmem0
#pragma HLS INTERFACE m_axi port=a_out bundle=gmem1
    A buffer_in[NUM];
    A buffer_out[NUM];

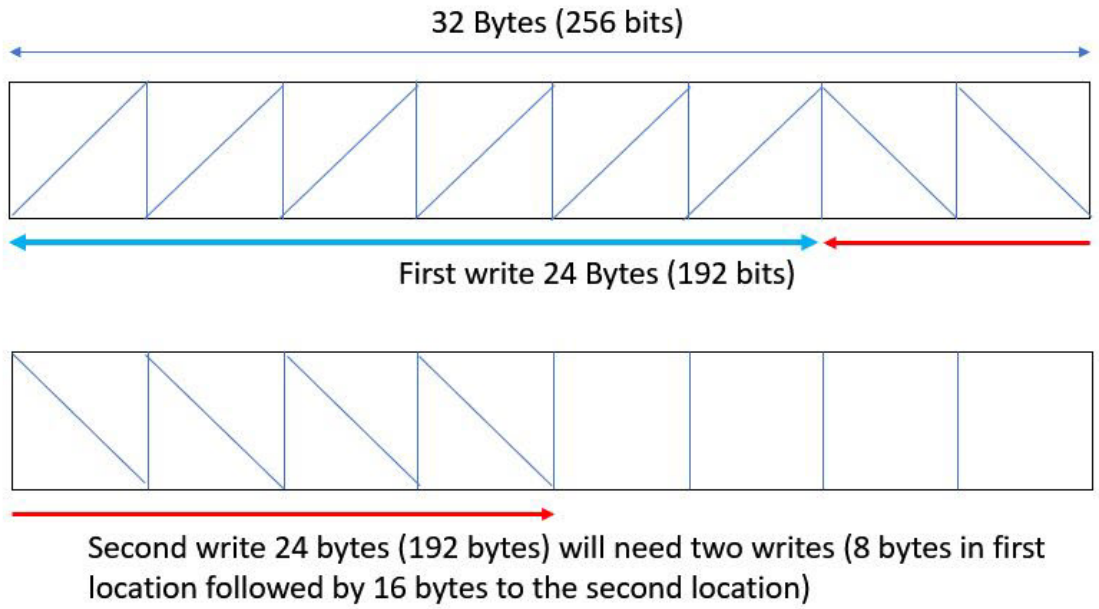
#pragma HLS dataflow
    read(a_in, buffer_in);
    compute(buffer_in, buffer_out, size);
    write(buffer_out, a_out);
}
```

In the above example, the size of struct `A` is 192 bits, which is not a power of 2. As stated earlier in the document, all AXI4 interfaces are by default sized to a power of 2. Vitis HLS will automatically size the two M_AXI interfaces (`a_in` and `a_out`) to be of size 256 - the closest power of 2 to the size of 192 bits (and report in the log file as shown below).

```
INFO: [HLS 214-241] Aggregating maxi variable 'a_out' with compact=none
mode in
256-bits (example.cpp:49:0)
INFO: [HLS 214-241] Aggregating maxi variable 'a_in' with compact=none mode
in 256-bits
(example.cpp:49:0)
```

This will imply that when writing the struct data out, the first write will write 24 bytes to the first buffer in one cycle but the second write will have to write 8 bytes to the remaining 8 bytes in the first buffer and then write 16 bytes into a second buffer resulting in two writes - as shown in the figure below.

Figure 53: **Misaligned Write Cycles**



This will cause the II of the WRITE loop in function `write()` to have an II violation since it needs II=2 instead of II=1. Similar behavior will happen when reading and therefore the `read()` function will also have an II violation since it needs II=2. Vitis HLS will issue the following warning for the II violation in function `read()` and `write()`:

```
WARNING: [HLS 200-880] The II Violation in module 'read_r' (loop 'READ'):
Unable
to enforce a carried dependence constraint (II = 1, distance = 1, offset =
1) between
bus read operation ('gmem0_addr_read_1', example.cpp:23) on port 'gmem0'
(example.cpp:23)
and bus read operation ('gmem0_addr_read', example.cpp:23) on port 'gmem0'
(example.cpp:23).

WARNING: [HLS 200-880] The II Violation in module 'write_Pipeline_WRITE'
(loop 'WRITE'):
Unable to enforce a carried dependence constraint (II = 1, distance = 1,
offset = 1)
between bus write operation ('gmem1_addr_write_ln44', example.cpp:44) on
port 'gmem1'
(example.cpp:44) and bus write operation ('gmem1_addr_write_ln44',
example.cpp:44) on
port 'gmem1' (example.cpp:44).
```

The way to fix such II issues is to pad struct `A` with 8 additional bytes such that you are always writing 256 bits (32 bytes) at a time or by using the other alternatives shown in the table below. This will allow the scheduler to schedule the reads/writes in the READ/WRITE loop with II=1.

Table 18: **Struct Alignment**

Code Block	Description

`struct A {`
`  int`
`s_1;`
 	
`  int`
`s_2;`
 	
`  int s_3;`
`  int s_4;`
`  int s_5;`
`  int s_6;`
`  int pad_1;`
`  int pad_2;`
`};`
	Defines the total size of the struct as 256 bits (32 x 8) or 32 bytes, by adding required padding elements.

`struct A {`
`  int`
`s_1;`
 	
`  int`
`s_2;`
 	
`  int s_3;`
`  int s_4;`
`  int s_5;`
`  int s_6;`
`} __attribute__ ((aligned(32)));`
	Uses the standard `__aligned__` attribute.

`struct alignas(32) A {`
`  int`
`s_1;`
 	
`  int`
`s_2;`
 	
`  int s_3;`
`  int s_4;`
`  int s_5;`
`  int s_6;`
`}`
	Uses the C++ standard `alignas` type specifier to specify custom alignment of variables and user defined types.

Execution Modes of HLS Designs

The execution mode of the HLS design refers to the way the design works as a block (or module) both with regard to itself and the functions within the block, or in relationship with other blocks modules, or outside software that addresses the block. These modes are determined by block control protocols assigned to the HLS design as described in Block-Level Control Protocols, and by the internal structure of the HLS design as described in Abstract Parallel Programming Model for HLS.

Execution modes of kernels include:

- **Overlapping Mode:** Let the next execution of a new transaction begin before the current transaction is complete. Function pipelined, loop rewind or dataflow execution allows overlapping block runs to begin processing additional data as soon as the design is ready.

  Control Protocols: Use the `ap_ctrl_chain` or `ap_ctrl_hs` with backpressure support. Backpressure is a mechanism that allows downstream modules to control the flow of data. If a downstream module is not ready to receive new data, it can assert a signal to stall the upstream modules temporarily. This prevents data loss and ensures synchronization across different stages of the pipeline.

- **Sequential Mode:** Requires the current transaction to complete before a new transaction can be started.

  Control Protocols: Can be implemented using `ap_ctrl_hs` or `ap_ctrl_chain` without backpressure support. By eliminating backpressure, the design assumes that data is consumed at a consistent rate, and there is no need for flow control between stages.

- **Continuously Running Mode:** Continuously running kernels run indefinitely until explicitly reset or run for a certain number of times, making them fundamentally different from classical software acceleration kernels. In contrast to traditional software kernels that are called, execute, and return upon completion, continuously running kernel continuously executes, mimicking a `while(1)` loop. These kernels are especially useful in hardware contexts where data streams in without predefined boundaries or known completion points.

  Continuously running kernel are commonly used in applications where data is continuously fed into the system, and the kernel must process it without interruption. Examples include:

  - **Networking**: Handling incoming data packets from Ethernet ports without the host system knowing the total number of packets in advance. For instance, a kernel might process packets from `TVALID` (valid data) to `TLAST` (last packet), with continuous execution.
  - **Firewall**: A simple rule-based firewall where the rules are written once by the host at reset, and the kernel processes packets continuously. The host can read packet drop counters at any time, but these counters are the result of a single kernel execution.
  - **Network Routing**: A network router that updates its routing table during execution but doesn't stop to process incoming packets continuously.
  - **Load Balancing**: A load balancer that routes data based on a hash map, updating server lists and IP addresses while continuously handling traffic.
  - **Fintech**: Financial applications where data packets from Ethernet ports are processed continuously, and the host loosely monitors the execution status without interrupting the ongoing operations.

  Control Protocols: Continuously Running mode can be modeled using both the scenarios.

  - Control Driven Task-level Parallelism (CDTLP) in auto-restart mode using `ap_ctrl_chain` block level protocol.

  - Data-driven Task-level Parallelism (DTLP) using `ap_ctrl_none` block level protocol.
- **Auto-Restarting:** Auto-restart mode lets the HLS design automatically restart the module when it is ready to begin processing additional data. This mode supports both overlapping and sequential execution. Auto-restarting is the approach for data-driven TLP designs, but can also be implemented in control-driven TLP designs as described in Auto-Restarting Mode.

Block-Level Control Protocols

The execution mode of a Vitis kernel or Vivado IP is defined by the block-level control protocol and the structure of sub-functions within the HLS design. For control-driven TLP the `ap_ctrl_chain` and `ap_ctrl_hs` protocols support both sequential or pipelined execution. For data-driven TLP `ap_ctrl_none` is the required control protocol.

The `ap_ctrl_chain` control protocol is the default for the Vitis kernel flow as explained in Interfaces for Vitis Kernel Flow. The `ap_ctrl_hs` block-level control protocol is the default for the Vivado IP flow as described in Interfaces for Vivado IP Flow. However, you should use `ap_ctrl_chain` when chaining HLS designs together to better support pipelined execution.

You can specify the block-level control protocol on the function return using the INTERFACE pragma or directive. If the C/C++ code does not return a value, you can still specify the control protocol on the function return. If the C/C++ code uses a function return, Vitis HLS creates an output port `ap_return` for the return value.

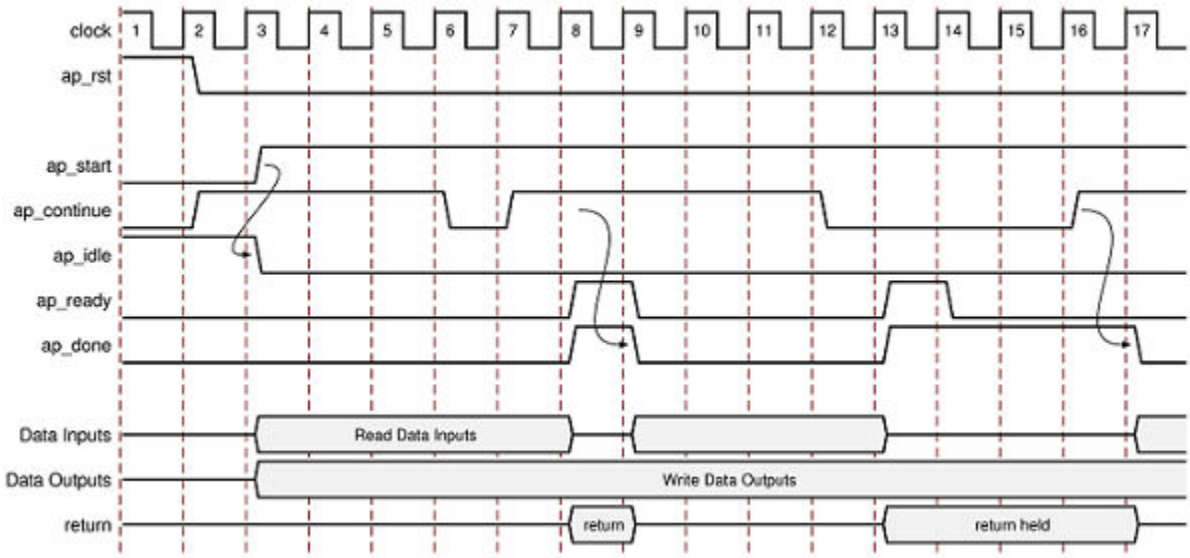


TIP: *When the function return is specified as an AXI4-Lite interface (`s_axilite`) all the ports in the control protocol are bundled into the `s_axilite` interface. This is a common practice for software-controllable kernels or IP when an application or software driver is used to configure and control when the block starts and stops operation. This is a requirement of XRT and the Vitis kernel flow.*

ap_ctrl_chain

The following figure shows the behavior of the block-level handshake signals created by the `ap_ctrl_chain` control protocol for a sequential execution. In the following figure, the first transaction of the HLS design completes, and the second transaction starts immediately because `ap_continue` is High when `ap_done` is High. However, the design halts at the end of the second transaction until `ap_continue` is asserted High.

Figure 54: **Behavior of ap_ctrl_chain Interface**



The timing diagram displays the following behavior after reset occurs:

1. The block waits for `ap_start` to go High before it begins operation.
2. Output `ap_idle` goes Low immediately to indicate the design is no longer idle.
3. The `ap_start` signal must remain High until `ap_ready` goes High. Once `ap_ready` goes High:
   - If `ap_start` remains High the design will start the next transaction.
   - If `ap_start` is taken Low, the design will complete the current transaction and halt operation.
4. Data can be read on the input ports.
5. Data can be written to the output ports.

   ***Note*:** The input and output ports can also specify a port-level I/O protocol that is independent of the control protocol. For details, see Port-Level Protocols for Vivado IP Flow.
6. Output `ap_done` goes High when the block completes operation.

   ***Note*:** If there is an `ap_return` port, the data on this port is valid when `ap_done` is High. Therefore, the `ap_done` signal also indicates when the data on output `ap_return` is valid.
7. The `ap_ctrl_chain` control protocol provides an active-High `ap_continue` signal that indicates when the downstream block that consumes the output data is ready for new data inputs. This allows the downstream block to provide back-pressure to prevent the flow of data.
   - If the `ap_continue` signal is High when `ap_done` is High, the design continues operating.

- If the downstream block is not able to consume new data inputs, the `ap_continue` signal is Low. If the `ap_continue` signal is Low when `ap_done` is High, the design stops operating, the `ap_done` signal remains High waiting for `ap_continue` to go High.

8. When the design is ready to accept new inputs, the `ap_ready` signal is pulsed High for one clock cycle. The `ap_ready` port of a downstream block can directly drive the `ap_continue` port. Following is additional information about the `ap_ready` signal:
   - The `ap_ready` signal is inactive until the design starts operation.
   - In non-pipelined designs, the `ap_ready` signal is asserted at the same time as `ap_done`.
   - In pipelined designs, the `ap_ready` signal might go High at any cycle after `ap_start` is sampled High. This depends on how the design is pipelined.
   - If `ap_start` remains high after `ap_ready` goes high, the next transaction starts immediately.
   - When `ap_start` goes low right after `ap_ready` goes high, the design keeps executing until `ap_done` is high and then stops operation, unless `ap_start` goes high again in the meantime, which starts a new transaction.
9. The `ap_idle` signal indicates when the design is idle and not operating. Following is additional information about the `ap_idle` signal:
   - If the `ap_start` signal is Low when `ap_ready` is High, the design stops operation, and the `ap_idle` signal goes High one cycle after `ap_done`.
   - If the `ap_start` signal is High when `ap_ready` is High, the design continues to operate, and the `ap_idle` signal remains Low.

ap_ctrl_hs

The `ap_ctrl_hs` control protocol has the same signals as `ap_ctrl_chain`, but sets the `ap_continue` signal to 1 so it remains high. This control protocol supports sequential and pipelined execution modes, but does not offer back-pressure from downstream design modules to control the flow of data.

ap_ctrl_none

`ap_ctrl_none` also has the same signals as `ap_ctrl_chain`, but the handshake signal ports (`ap_start`, `ap_idle`, `ap_ready`, and `ap_done`) are set high and optimized away.

An `ap_ctrl_none` region, or one or more `hls::tasks`, can be instantiated in one of two ways:

- Either with only `ap_ctrl_none` regions all the way to the top, including the top (i.e. no sequential FSM or non-`ap_ctrl_none` dataflow anywhere above, even due to a coding mistake),

- Or in a dataflow region where:
  - its input streams are produced by one or more non-`ap_ctrl_none` processes or regions that are called before it
  - its output streams are consumed by one or more non-`ap_ctrl_none` processes or regions that are called after it

The former enables the elimination of the `ap_start/ap_ready/ap_done/ap_continue` handshakes all the way to the top. The latter enables the generation of the `ap_start/ap_ready` handshake for the dataflow region above by the predecessor processes or regions, and of the `ap_done/ap_continue` handshake by the successor processes or regions.

IMPORTANT! *If you use the `ap_ctrl_none` control protocol in your design, you must meet at least one of the conditions for C/RTL co-simulation as described in Co-simulation Requirements for Interface Synthesis to verify the RTL design. If at least one of these conditions is not met, C/RTL co-simulation halts with the following message:*

```
@E [SIM-345] Cosim only supports the following 'ap_ctrl_none' designs:
(1)
combinational designs; (2) pipelined design with task interval of 1;
(3) designs with
array streaming or hls_stream ports.
@E [SIM-4] *** C/RTL co-simulation finished: FAIL ***
```

Continuously Running Mode

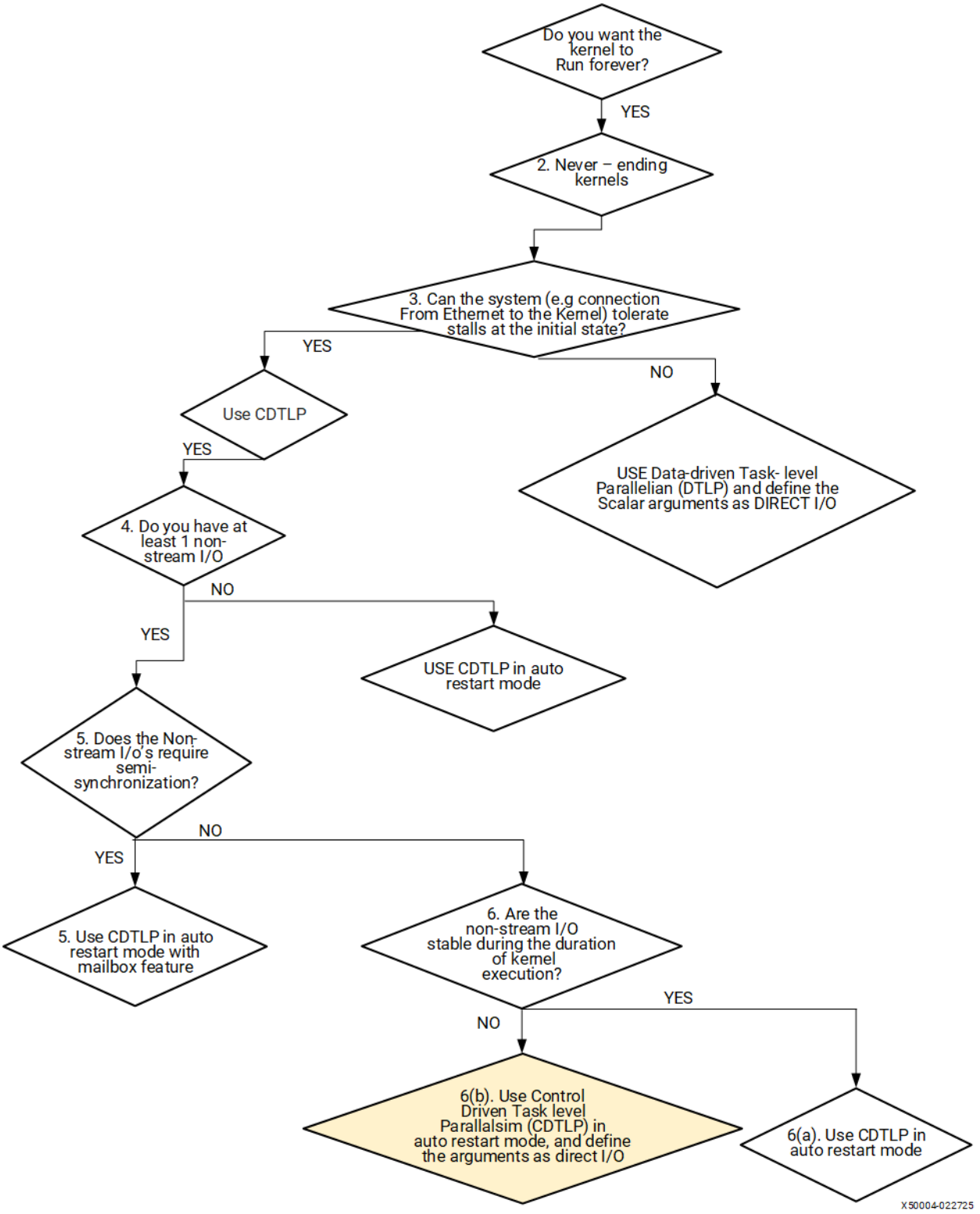
The continuously running kernels are kernels that will run forever, until they are reset. The notion of a never-ending kernel is typical of an HW design context. These kernels are closely related to HW FSMs, modeled as processes in VHDL or Verilog. They are very different from classical SW acceleration kernels, which are meant to "be called" and "return" when done in a manner akin to function calls in SW. The behavior of a never-ending kernel differs from that of an SW acceleration kernel. Consider the below example; as a classical SW acceleration kernel, it is meant to "be called" and "return" when done. But a never-ending kernel will re-execute until reset without explicitly being started by the host; it is similar to a while (1) loop around the function.

The use case is in the field of video/networking/fintech where the kernel receives data continuously without any interrupt from the Ethernet ports/DAC etc. This use case needs Continuously running kernels because the host does not know in advance the number of packets (where the number of packets is counted from the first TVALID packet to the last TLAST packet) the kernel should process. This problem is solved when the kernel is modeled with a Continuously running mode, allowing the kernel to run forever without knowing the number of packets in advance.

Some other examples are:

- Simple rule-based “firewall”, with “rules” written by the host code at reset time, and where the host code can read a set of counters of dropped packets at any time, but where all counter values must come from a single kernel execution.
- Network router where the routing table must be updated entirely for kernel execution.
- A load balancer that uses a hash map to send data to a server must update the server list, server map, and corresponding IP addresses simultaneously.
- Fintech Example where the host loosely monitors the execution status of the kernel running on the accelerator card processing requests coming in over the Ethernet ports.

Figure 55: **Kernel Flow Chart**



1. **Can the system tolerate a stall at the initial state**

There are two ways to model the Continuously running kernel, but the decision of option-1/2 will depend on the system designer.

- **Option 1:** Data-driven Task-level Parallelism (DTLP).
- **Option 2:** Control and Data-Driven Task-level Parallelism (CDTLP) in auto restart mode.

Option 1: In Data-driven Task-level Parallelism, the kernel will start as soon as the `xclbin` is loaded onto the device; there won't be any overhead for starting the kernel. This choice is best suited for designs that **cannot tolerate or prefer not to for any stalls on the output side of Ethernet/DAC**.

Option 2: In Control and Driven Task-level Parallelism (CDTLP), the host must set the kernel in auto restart mode. This would involve some startup overhead compared to option -1, but the kernel designer needs to do this only once for the entirety of the application run. Similarly, the kernel designer is responsible for stopping the kernel when the application has finished.

In summary, the decision to use options 1 and 2 depends on **how the kernel starts and its pros and cons.** Once the modeling style is decided, the kernel designer must decide whether they have at least one non-stream I/O.

2. **What is a non-stream I/O**

   Vitis HLS supports memory, stream, and register paradigms, where each paradigm follows a certain interface protocol to interact with the external world. The different paradigms description is documented at Interfaces for Vitis Kernel Flow.

   - **Non-Stream I/O:**
     - Non-stream I/O falls under the following Interface paradigms:
       - Memory Paradigm
       - Register Paradigm
   - **Stream I/O:**
     - Stream I/O falls under the following Interface paradigms:
       - Stream Paradigm

3. Need for Semi-synchronization for non-stream I/O

   Semi-synchronization exchanges I/O data to/from auto restart kernel in an asynchronous, non-blocking way. In an SW acceleration kernel, the I/O is fully synchronized. This means the kernel waits for all the inputs to be available before starting the kernel. In a Semi-synchronized I/O, from the second function start, the kernel will use the old values and does not block the kernel execution for the new updated values. Semi-synchronized I/O can be used in video application that streams continuously. A continuously streamed design

might require changing the requirements asynchronously, non-blocking. For example, In the middle of the application run, a video application might want to increase the output image's resolution from HD to 4K. Using the Semi-synchronization feature, the application can change the input parameters without blocking the kernel execution. In HLS, this feature is implemented as a *Mailbox* feature. Refer to Mailbox Semantics.

Once the kernel design has decided on semi-synchronization, the kernel designer needs to be aware of the behavior of I/O ports.

4. The behavior of I/O ports: Stable/Direct I/O

   Depending on the behavior of the input(s), there are two cases when an input (non-stream) argument can change with respect to a kernel execution:

   - Case (a): The I/O never changes during the whole execution of the kernel.
   - Case (b): Can change at any time, regardless of whether the kernel is executing or idle.

   Case (a): The I/O never changes during the whole execution of the kernel.

   - The inputs that only change when the kernel is idle are, by default, treated as stable by the compiler. This means the compiler assumes these inputs will remain constant during the active execution of the kernel. In the example below, the argument "a" is sampled during the kernel's idle state and retains the same value for the entire duration of the kernel's execution.

     ```
     void func(hls::stream<int> in, int a,hls::stream<int>out)
     {
     #pragma HLS INTERFACE mode=ap_ctrl_chain
         do {
             out.write(in.read()+a);
          }
     }
     ```

   Case (b): Can change at any time, regardless of whether the kernel is executing or idle.

   - Kernels whose arguments are modeled using hls::direct I/O class can change the value at any point in the kernel execution. The kernel designer is not particular about the timing of the change if the kernel can see the updated values at some point in future time. In the example below, the argument "reset_myCounter" is sampled whenever there is valid input during the kernel's execution.

     ```
     void krnl_stream_vdatamover(hls::stream<pkt> &in,
                           hls::stream<pkt> &out,
                                     int mem[DATA],
                           hls::ap_vld<int> &reset_value,
                           hls::ap_vld<int> &reset_myCounter
                           )


     {for(int i=0;i<DATA;i++)
     {
     ...
       if(reset_myCounter.valid())


         {
     ```

```
        int reset = reset_myCounter.read();
    }
..
}
}
```

Auto-Restarting Mode

Vitis HLS designs can be control-driven modules or data-driven modules, as described in Abstract Parallel Programming Model for HLS. This means that the execution of the HLS design is managed through control signals, and managed by an external software application, or by software drivers, or is driven by the presence of data at the inputs.

Data-driven modules are auto-restarting designs by the mere fact of additional data to be processed. The HLS design finishes execution of one transaction and begins execution of the next transaction when data is available. Execution stalls when no data is present, and restarts when data is present. This section is not concerned with these data-driven designs.

For control-driven modules though, the managing software application or a connected HLS design must trigger the `ap_start` signal of the module to begin processing, as described in Block-Level Control Protocols. While these designs are not auto-restarting by default, they can be implemented using auto-restarting mechanisms as described here. This auto-restarting mode is useful for control-drivel TLP when the HLS design uses streaming I/O for data input or output, but also has control signals allowing it to be managed by a software application. In this case the software application starts the kernel one time and the kernel runs for a specified number of transactions, or restarts continuously until it is reset or explicitly stopped by the software application.

This control-driven auto-restarting design is useful for designs with streaming data coming from and going to the I/O pins (Ethernet, SerDes) of the AMD device, or data streamed from or to connected HLS design modules. In other words, the HLS design is primarily data-driven, but not exclusively.

Working with Auto-Restarting Designs

Auto-restarting designs can run continuously after being started once by the software application. They can run continuously until reset and restarted, or they can be programmed to run for a predetermined number of iterations without the software application explicitly calling them multiple times in a mode called counted auto-restart. This functionality is similar to a `while(running)` loop in software code, where the `running` variable is controlled by the software application.

The control of the design is managed in hardware so that once the block is started by the software application it is automatically restarted until the iteration count is exceeded, or until explicitly stopped by the host code. In addition, the application can query the status of the design to check specific register states or to provide new parameters to be used at the next opportunity.

Auto-restarting designs make use of several unique features:

1. The auto-restart bit in the control register is used to continuously restart the design, or restart it for a specified number of iterations, without explicit software calls for each execution.
2. A mailbox feature of the Xilinx Runtime (XRT) to enable the software application to occasionally synchronize with the design to set new operating parameters, or check the status of the current run, as described in Using the Mailbox.
3. The `software_reset` feature letting the software application reset the design to stop execution.

Supported Interfaces

Auto-restarting HLS design require the `ap_ctrl_chain` or `ap_ctrl_hs` protocol specified by Vitis HLS, where `ap_ctrl_chain` is the recommended protocol.

- The design should specify the `mode=ap_ctrl_chain` on the INTERFACE pragma or directive.
- Auto-restarting designs support streaming interfaces (`axis`) and both scalar arguments (`s_axilite`) and memory mapped (`m_axi`) arguments which can be both read and written.

Auto-restart designs can be used in a couple of different scenarios:

1. Auto-restart kernel with streaming interfaces (`axis`) only, that does not require the kernel to interact with the host application at all. Examples of this would be a Fast-Fourier Transform (FFT) with configuration data compiled into the kernel, or FIR filters with coefficients compiled into the kernel.
2. Auto-restart kernel using scalars and memory mapped (`m_axi`) arguments, either with `axis` interfaces or without. The scalar and memory mapped arguments require the mailbox to update the kernel parameters when needed. Examples of this would include a simple rule-based firewall with rules written by the host application at reset, with a counter of dropped packets that can be read by the host code but where all values must come from a single kernel execution. Or, a load balancer that uses a hash map to send data to a server, must update the server list, server map, and corresponding IP addresses simultaneously.

Enabling Auto-Restart

To enable a continuously restarting HLS design specify the following in a configuration file:

```
syn.interface.s_axilite_auto_restart_counter=1
```

To enable auto-restart with the mailbox features in the HLS design use the following configuration commands:

```
syn.interface.s_axilite_mailbox=both
syn.interface.s_axilite_auto_restart_counter=1
```

You can enable the software reset feature for auto-restart using the following option:

```
syn.interface.s_axilite_sw_reset=1
```

TIP: *Resetting the kernel stops all the incoming traffic on the ports and flushes out any outstanding MAXI transactions. Refer to Using Auto-Restart with the Mailbox for more information on using reset.*

Using the Mailbox

The main advantage of auto-restarting HLS designs is that they run semi-autonomously operating as data-driven without the need for frequent interaction with the software application or for software control. But auto-restarting kernels also offer semi-synchronization through a feature known as the mailbox, which provides the ability to exchange data with the software application in an asynchronous, non-blocking, and safe way.

Once started by the software application the HLS design is automatically restarted until explicitly stopped. The software application can also query the status of the design to determine when it actually has finished executing after being instructed to do so. The application and auto-restarting design use the following communications protocol:

- For passing argument values from the software application to the design, the mailbox implements a set of double-buffered `s_axilite` mapped registers to ensure non-blocking communication and consistent passing of inputs by the software application and passing of outputs by the design.
- Whenever the software application writes to an input argument, it changes the software-side copy. The HLS design running in hardware does not see that change. After the software application requests a mailbox write, the next time the design automatically restarts the copy of the registers that are seen by the design is updated with the latest inputs. Hence the software application can write any number of arguments in any order, and the HLS design does not see these updates until the software application requests a mailbox write, and the design restarts.

  TIP: *If some arguments are arrays mapped to `s_axilite` register files, then the entire array must be written between successive mailbox writes because it is implemented as a ping-pong buffer.*

- The same process occurs on the output side when the HLS design writes to the design-side register and requests a mailbox read. The next time the design is done, the values of the `s_axilite` mapped output arguments are updated, and the software application can read them as needed.

Hence the software application has the following criteria:

- Not in charge of providing input data at every execution of the HLS design and collecting output data at its finish, as it would be in the case of control-driven designs.
- Involved in occasionally setting and updating some design input parameters (for example, routing tables, etc.) and checking the status of the design execution. This semi-synchronization operation is typically done without a fixed communication rate between the software application and the HLS design.
- When the software application has a new set of parameters to send to the HLS design, it does so without caring where the design is in its execution. When the software application needs to check the status of the HLS design, it does so without caring where design is in its execution. It is satisfied only that the parameter update and status check is performed consistently for the hardware.

Mailbox Semantics

The mailbox features can be used for both input and output, and would need to be specified for all `s_axilite` I/Os of an HLS component. It is enabled with a global option for the interfaces using the following configuration commands:

```
syn.interface.s_axilite_mailbox=both
syn.interface.s_axilite_auto_restart_counter=1
```

After setting up the configuration options, the mailbox implements a pair of registers called `HW copy` and `SW copy`. The input mailbox and output mailbox has an independent pair of registers. Communication from the software application with the HLS component includes:

- **Input Mailbox:**
  - The application writes some or all elements to the `SW copy` register of the mailbox.
  - The application notifies the mailbox that `SW copy` is updated.
  - When the HLS design restarts, the `SW copy` register is copied to the `HW copy` register.
  - The application is notified that the `HW copy` has been updated, and can change the `SW copy` register again as needed.



TIP: *Multiple reads by the HLS design can occur without an update from the software application.*

- **Output Mailbox:**
  - The application notifies the mailbox that it wants to read an updated copy of the mailbox. The HLS design writes some or all the elements to the `HW copy` register of the mailbox at the end of the current execution.
  - When the design is done, `HW copy` is copied to the `SW copy` register.
  - The application is notified that `SW copy` is updated and can read it at any time.



TIP: *Multiple writes by the hardware can occur without any software request to update.*

Examples of Auto-Restarting Designs

The following sections describe the auto-restarting HLS design examples.

Using Auto-Restart with the Mailbox

The mailbox feature provides the ability to have semi-synchronization with a software application. The mailbox is a non-blocking mechanism that updates the HLS design parameters. Any updates provided through the mailbox will be picked up the next time the design starts.

This example design uses scalar values which will be programmed from the software application, and the design will pick them at the next software call. The scalars, `adder1` and `adder2`, will be asynchronously updated from the software application.

Configure the HLS compiler for auto-restarting mode and enable the mailbox feature using the following configuration commands:

```
syn.interface.s_axilite_mailbox=both
syn.interface.s_axilite_auto_restart_counter=1
syn.interface.s_axilite_sw_reset=1
```

This informs the tool to create the IP or kernel using the autorestart mechanisms. The example HLS design code follows:

```
#define DWIDTH 32
11
12 typedef ap_axiu<DWIDTH, 0, 0, 0> pkt;
13
14 extern "C" {
15 void krnl_stream_vdatamover(hls::stream<pkt> &in,
16      ¦    ¦    ¦    ¦    ¦ hls::stream<pkt> &out,
17      ¦    ¦    ¦    ¦    ¦ int adder1,
18      ¦    ¦    ¦    ¦    ¦ int adder2
19      ¦    ¦    ¦    ¦    ¦ ) {
20
21 #pragma HLS interface ap_ctrl_chain port=return
22 #pragma HLS INTERFACE s_axilite port=adder2
25 #pragma  HLS port=adder1 stable
   #pragma  HLS port=adder2 stable
27 bool eos = false;
28 vdatamover:
29   do {
30     // Reading a and b streaming into packets
31     pkt t1 = in.read();
32
33     // Packet for output
34     pkt t_out;
35
36     // Reading data from input packet
37     ap_uint<DWIDTH> in1 = t1.data;
38
39     // Vadd operation
40     ap_uint<DWIDTH> tmpOut = in1+adder1+adder2;
41
42     // Setting data and configuration to output packet
43     t_out.data = tmpOut;
44     t_out.last = t1.last;
```

```
45      t_out.keep = -1; // Enabling all bytes
46
47      // Writing packet to output stream
48      out.write(t_out);
49
50      if (t1.last) {
51      ¦ eos = true;
52      }
53    } while (eos == false);
54
```

Create a mailbox to update the scalars values `adder1` and `adder2`.

Update the design parameters from the software application using the `set_arg` and `write` methods as shown below. The auto-restarting HLS design will not stop itself because there is no start and stop for a streaming interface. The module must be explicitly stopped or reset. The application code can explicitly stop the design from running using the `abort()` method.

```
// add(in1, in2, nullptr, data_size)
  xrt::kernel add(device, uuid, "krnl_stream_vadd");
  xrt::bo in1(device, data_size_bytes, add.group_id(0));
  auto in1_data = in1.map<int*>();
  xrt::bo in2(device, data_size_bytes, add.group_id(1));
  auto in2_data = in2.map<int*>();

  // mult(in3, nullptr, out, data_size)
  xrt::kernel mult(device, uuid, "krnl_stream_vmult");
  xrt::bo in3(device, data_size_bytes, mult.group_id(0));
  auto in3_data = in3.map<int*>();
  xrt::bo out(device, data_size_bytes, mult.group_id(2));
  auto out_data = out.map<int*>();


  xrt::kernel incr(device, uuid, "krnl_stream_vdatamover");
  int adder1 = 20;  // arbitrarily chosen to be different from 0
  int adder2 = 10;  // arbitrarily chosen to be different from 0

  // create run objects for re-use in loop
  xrt::run add_run(add);
  xrt::run mult_run(mult);
  std::cout <<"performing never-ending mode with infinite auto
restart"<<std::endl;
  auto incr_run = incr(xrt::autostart{0}, nullptr, nullptr, adder1, adder2);

// create mailbox to programatically update the incr scalar adder
  xrt::mailbox incr_mbox(incr_run);

  // computed expected result
  std::vector<int> sw_out_data(data_size);

  std::cout << " for loop started" <<std::endl;
  bool error = false;   // indicates error in any of the iterations
  for (unsigned int cnt = 0; cnt < iter; ++cnt) {


    // Create the test data and software result
    for(size_t i = 0; i < data_size; ++i) {
      in1_data[i] = static_cast<int>(i);
      in2_data[i] = 2 * static_cast<int>(i);
      in3_data[i] = static_cast<int>(i);
      out_data[i] = 0;
```

```
      sw_out_data[i] = (in1_data[i] + in2_data[i] + adder1 + adder2) *
in3_data[i];
    }

    // sync test data to kernel
    in1.sync(XCL_BO_SYNC_BO_TO_DEVICE);
    in2.sync(XCL_BO_SYNC_BO_TO_DEVICE);
    in3.sync(XCL_BO_SYNC_BO_TO_DEVICE);

    // start the pipeline
    add_run(in1, in2, nullptr, data_size);
    mult_run(in3, nullptr, out, data_size);

    // wait for the pipeline to finish
    add_run.wait();
    mult_run.wait();

    // prepare for next iteration, update the mailbox with the next
    // value of 'adder'.
    incr_mbox.set_arg(2, ++adder1); // update the mailbox
    incr_mbox.set_arg(3, --adder2); // update the mailbox


    // write the mailbox content to hw, the write will not be picked
    // up until the next iteration of the pipeline (incr).
    incr_mbox.write();  // requests sync of mailbox to hw


    // sync result from device to host
    out.sync(XCL_BO_SYNC_BO_FROM_DEVICE);

    // compare with expected scalar adders
    for (size_t i = 0 ; i < data_size; i++) {
      if (out_data[i] != sw_out_data[i]) {
        std::cout << "error in iteration = " << cnt
                  << " expected output = " << sw_out_data[i]
                  << " observed output = " << out_data[i]
                  << " adder1 = " << adder1 - 1
                  << " adder2 = " << adder2 + 1 << '\n';
        throw std::runtime_error("result mismatch");
      }
    }
  }
        incr_run.abort();
}
```

Using Counted Auto-Restart

This example uses the same code as the prior example, but in this case the HLS design is configured to run and restart for three iterations and then stop. The only required change is to specify the number of iterations from the software application as shown in the code example below.

TIP: *In the counted auto-restart case, there is no need to use the* `abort()`*, because the software application will know when to stop.*

```
143   xrt::kernel incr(device, uuid, "increment");
144   int adder1 = 20;  // arbitrarily chosen to be different from 0
145   int adder2 = 10;  // arbitrarily chosen to be different from 0
151   // start the incr kernel in auto restart mode with default adders
152   // since it is a streaming kernel it will be stalled waiting for
153   // input
154   auto incr_run = incr(xrt::autostart{3}, nullptr, nullptr, adder1);
```

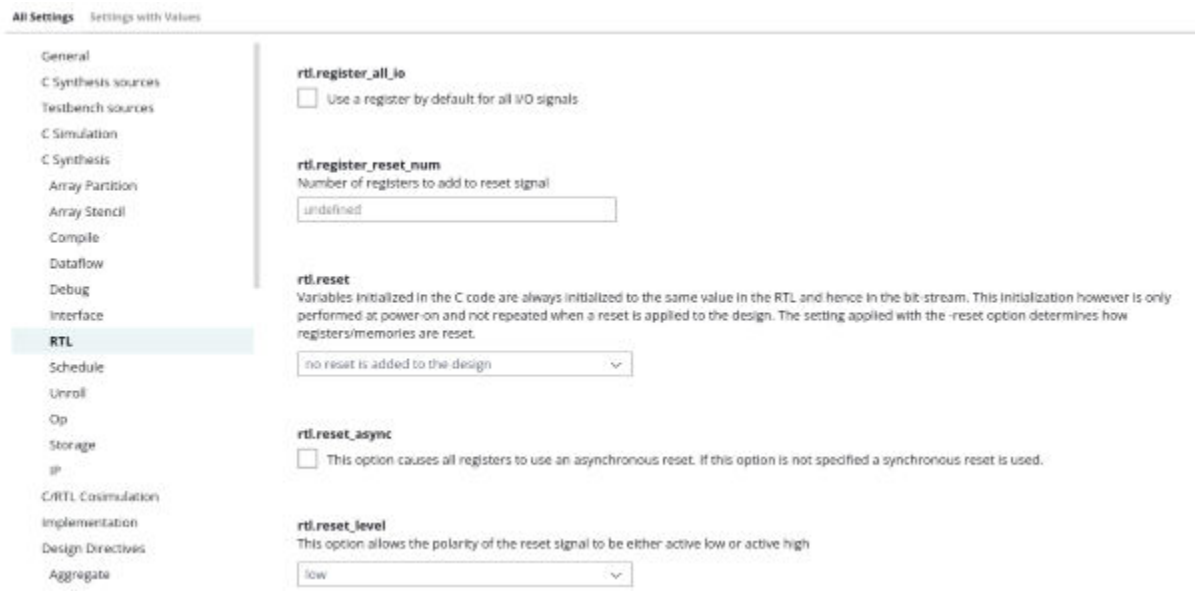
Controlling Initialization and Reset Behavior

The reset port is used in a device to return the registers and block RAM connected to the reset port to an initial value any time the reset signal is applied. Typically the most important aspect of RTL configuration is selecting the reset behavior.

***Note*:** When discussing reset behavior it is important to understand the difference between initialization and reset. Refer to Initialization Behavior for more information.

The presence and behavior of the RTL reset port is controlled using configuration commands as described in RTL Configuration, as shown in the following figure.

Figure 56: **RTL Configurations**



The reset settings include the ability to set the polarity of the reset and whether the reset is synchronous or asynchronous but more importantly it controls, through the **reset** option, which registers are reset when the reset signal is applied.

IMPORTANT! *When AXI4 interfaces are used on a design the reset polarity is automatically changed to active-Low irrespective of the setting in the* `config_rtl` *configuration. This is required by the AXI4 standard.*

The **reset** option has four settings:

- **none:** No reset is added to the design.
- **control:** This is the default and ensures all control registers are reset. Control registers are those used in state machines and to generate I/O protocol signals. This setting ensures the design can immediately start its operation state.
- **state:** This option adds a reset to control registers (as in the **control** setting) plus any registers or memories derived from static and global variables in the C/C++ code. This setting ensures static and global variable initialized in the C/C++ code are reset to their initialized value after the reset is applied.
- **all:** This adds a reset to all registers and memories in the design.

Finer grain control over reset is provided through the RESET pragma or directive as described in syn.directive.reset. Static and global variables can have a reset added through the RESET directive. Variables can also be removed from those being reset by using the RESET directive's `off` option.

IMPORTANT! *It is important when using the reset* `state` *or* `all` *options to consider the effect on resetting arrays.*

Initialization Behavior

In C/C++, variables defined with the static qualifier and those defined in the global scope are initialized to zero, by default. These variables can optionally be assigned a specific initial value. For these initialized variables, the value in the C/C++ code is assigned at compile time (at time zero) and never again. In both cases, the initial value is implemented in the RTL.

- During RTL simulation the variables are initialized with the same values as the C/C++ code.
- The variables are also initialized in the bitstream used to program the FPGA. When the device powers up, the variables will start in their initialized state.

In the RTL, although the variables start with the same initial value as the C/C++ code, there is no way to force the variable to return to this initial state. To restore the initial state, variables must be implemented with a reset signal.

IMPORTANT! *Top-level function arguments can be implemented in an AXI4-Lite interface. Because there is no way to provide an initial value in C/C++ for function arguments, these variable cannot be initialized in the RTL as doing so would create an RTL design with different functional behavior from the C/C++ code which would fail to verify during C/RTL co-simulation.*

Best Practices for Designing with M_AXI Interfaces

For designers implementing an AMD Vitis™ kernel there are various trade-offs available when working with the device memory (PLRAM, HBM and DDR) available on FPGA devices. The following is a checklist of best practices to use when designing AXI4 memory mapped interfaces for your application.

With throughput as the chief optimization goal, it is clear that accelerating the compute part of your application using the macro and micro-architecture optimizations is the first step but the time taken for transferring data to/from the kernel can also influence the application architecture with respect to throughput goals. Due to the high overhead for data transfer, it becomes important to think about overlapping the computation with the communication (data movement) that is present in your application.

For your given application:

- Decompose the kernel algorithm by building a pipeline of producer-consumer tasks, modeled using a Load, Compute, Store (LCS) coding pattern
  - All external I/O accesses must be in the Load and Store tasks.
  - There should be multiple Load or Store tasks if the kernel needs to read or write from different ports in parallel.
  - The Compute task(s) should only have scalars, array, streams or stream of blocks arguments.
  - Ensure that all these tasks (specified as functions) can be executed in overlapped fashion (enables task-level parallelism by the compiler).
  - Compute tasks can be further split up into smaller compute tasks which can result in further optimizations such as pipelining. The same rules as LCS apply for these smaller compute functions as well.
  - Always use local memory to pass data to/from the Compute tasks.
- Load and Store blocks are responsible for moving data between global memory and the Compute blocks as efficiently as possible.
  - On one end, they must read or write data through the streaming interface according to the (temporal) sequential order mandated by the Compute task inside the kernel

    - On the other end, they must read or write data through the memory-mapped interface according to the (spatial) arrangement order set by the software application
- Changing your mindset about data accesses is key to building a proper HW design with HLS
    - In SW, it is common to think about how the data is "accessed" (the algorithm *pulls* the data it needs).
    - In HW, it is more efficient in think of how data "flows" through the algorithm (the data is *pushed* to the algorithm)
    - In SW, you reason about array indices and "where" data is accessed
    - In HW, you reason about streams and "when" data is accessed
- Global memories have long access times (DRAM, HBM) and their bandwidth is limited (DRAM). To reduce the overhead of accessing global memory, the interface function needs to
    - Access sufficiently large contiguous blocks of data (to benefit from AXI Burst Transfers)
    - Accessing data sequentially leads to larger bursts (and higher data throughput efficiency) as compared to accessing random and/or out-of-order data (where burst analysis will fail)
    - Avoid redundant accesses (to preserve bandwidth)
- In many cases, the sequential order of data in and out of the Compute tasks is different from the arrangement order of data in global memory.
    - In this situation, optimizing the interface functions requires creating internal *caching* structures that gather enough data and organize it appropriately to minimize the overhead of global memory accesses while being able to satisfy the sequential order expected by the streaming interface.
    - In order to simplify the data movement logic, the developer can also consider different ways of storing the data in memory. For instance, accessing data in DRAM in a column-major fashion can be very inefficient. Rather than implementing a dedicated data-mover in the kernel, it could be better to transpose the data in SW and store in row-major order instead which will greatly simply HW access patterns.
- Maximize the port width of the interface, i.e., the bit-width of each AXI port by setting it to 512 bits (64 bytes).
    - Use `hls::vector` or `ap_(u)int<512>` as the data type of the port to infer maximal burst lengths. Usage of structs in the interface can result in poor burst performance.
    - Accessing the global memory is expensive and so accessing larger word sizes is more efficient.
    - Imagine the interface ports to be like pipes feeding data to your kernel. The wider the pipe, the more data that can be accessed and processed, and sent back.

    - Transfer large blocks of data from the global device memory. One large transfer is more efficient than several smaller transfers. The bandwidth is limited by the PCIe performance. Run the DMA test to measure PCIe® transfer effective max throughput. It is usually in the range of 10-17 GBps for reading and writing, respectively.
        - Memory resources include PLRAM (small size but fast access with the lowest latency), HBM (moderate size and access speed with some latency), and DRAM (large size but slow access with high latency).
        - Given the asynchronous nature of reads, distributed RAMs are ideal for fast buffers. You can use the read value immediately, rather than waiting for the next clock cycle. You can also use distributed RAM to create small ROMs. However, distributed ram is not suited for large memories, and you'll get better performance (and lower power consumption) for memories larger than about 128 bits using block RAM or UltraRAM.
- Decide on the optimal number of concurrent ports, for example, the number of concurrent AXI (memory-mapped) ports
    - If the Load task needs to get multiple input data sets to feed to the Compute task, it can choose to use multiple interface ports to access this data in parallel.
    - However, the data needs to be stored in different memory banks or the accesses will be sequentialized. There is a maximum of 4 DDR memory banks on FPGAs while there are 32 HBM channels.
    - When multiple processes are accessing the same memory port or memory bank, an arbiter will make these concurrent accesses to the same memory port or bank sequential.
- Setting the right burst length i.e., the maximum burst access length (in terms of the number of elements) for each AXI port.
    - Set the burst length equivalent to the maximum 4k bytes transfer. For example, using AXI data width of 512-bit (64 bytes), the burst length should be set to 64.
    - Transferring data in bursts hides the memory access latency and improves bandwidth usage and efficiency of the memory controller
    - Write application code in such a way to infer the maximal length bursts for both reads and writes to/from global memory
- Setting the number of outstanding memory requests that an AXI port can sustain before stalling
    - Setting a reasonable number of outstanding requests allows the system to submit multiple memory requests before stalling - this pipelining of requests allows the system to hide some of the memory latency at the cost of additional BRAM/URAM resources.

Chapter 10

Optimizing Techniques and Troubleshooting Tips

This section outlines the various optimization techniques you can use to direct AMD Vitis™ HLS to produce a micro-architecture that satisfies the desired performance and area goals. Using Vitis HLS, you can apply different optimization directives to the design, including:

- Pipelining tasks, allowing the next execution of the task to begin before the current execution is complete.
- Specifying a target latency for the completion of functions, loops, and regions.
- Specifying a limit on the number of resources used.
- Overriding the inherent or implied dependencies in the code to permit specific operations. For example, if it is acceptable to discard or ignore the initial data values, such as in a video stream, allow a memory read before write if it results in better performance.
- Specifying the I/O protocol to ensure function arguments can be connected to other hardware blocks with the same I/O protocol.

  ***Note*:** Vitis HLS automatically determines the I/O protocol used by any sub-functions. You *cannot* control these ports except to specify whether the port is registered.

It helps to understand the process used to synthesize RTL hardware description from C/C++ source code. The Understanding High-Level Synthesis Scheduling and Binding describes some of the important details of this process to help you better understand how you can optimize for it.

You can add optimization directives directly into the source code as compiler pragmas using various HLS pragmas, or you can use directives as configuration commands as discussed in Adding Pragmas and Directives. The following table lists the optimization directives provided by the HLS compiler as either pragma or configuration commands.

Table 19: **Vitis HLS Optimization Directives**

Directive	Description
AGGREGATE	The AGGREGATE pragma is used for grouping all the elements of a struct into a single wide vector to allow all members of the struct to be read and written to simultaneously.
ALIAS	The ALIAS pragma enables data dependence analysis in Vitis HLS by defining the distance between multiple pointers accessing the same DRAM buffer.
ALLOCATION	Specify a limit for the number of operations, implementations, or functions used. This can force the sharing or hardware resources and may increase latency.

Table 19: **Vitis HLS Optimization Directives** *(cont'd)*

Directive	Description
ARRAY PARTITION	Partitions large arrays into multiple smaller arrays or into individual registers, to improve access to data and remove block RAM bottlenecks.
ARRAY_RESHAPE	Reshape an array from one with many elements to one with greater word-width. Useful for improving block RAM accesses without using more block RAM.
BIND_OP	Define a specific RTL implementation for an operation.
BIND_STORAGE	Define a specific implementation for a storage element, or memory, in the RTL.
DATAFLOW	Enables task level pipelining, allowing functions and loops to execute concurrently. Used to optimize throughput and/or latency.
DEPENDENCE	Used to provide additional information that can overcome loop-carried dependencies and allow loops to be pipelined (or pipelined with lower intervals).
DISAGGREGATE	Break a struct down into its individual elements.
EXPRESSION_BALANCE	Allows automatic expression balancing to be turned off.
INLINE	Inlines a function, removing function hierarchy at this level. Used to enable logic optimization across function boundaries and improve latency/interval by reducing function call overhead.
INTERFACE	Specifies how RTL ports are created from the function description.
LATENCY	Allows a minimum and maximum latency constraint to be specified.
LOOP_FLATTEN	Allows nested loops to be collapsed into a single loop with improved latency.
LOOP_MERGE	Merge consecutive loops to reduce overall latency, increase sharing and improve logic optimization.
LOOP_TRIPCOUNT	Used for loops which have variables bounds. Provides an estimate for the loop iteration count. This has no impact on synthesis, only on reporting.
OCCURRENCE	Used when pipelining functions or loops, to specify that the code in a location is executed at a lesser rate than the code in the enclosing function or loop.
PERFORMANCE	Specify the desired transaction interval for a loop and let the tool to determine the best way to achieve the result.
PIPELINE	Reduces the initiation interval by allowing the overlapped execution of operations within a loop or function.
PROTOCOL	This commands specifies a region of code, a protocol region, in which no clock operations will be inserted by Vitis HLS unless explicitly specified in the code.
RESET	This directive is used to add or remove reset on a specific state variable (global or static).
STABLE	Indicates that a variable input or output of a dataflow region can be ignored when generating the synchronizations at entry and exit of the dataflow region.
STREAM	Specifies that a specific array is to be implemented as a FIFO or RAM memory channel during dataflow optimization. When using hls::stream, the STREAM optimization directive is used to override the configuration of the hls::stream.
TOP	The top-level function for synthesis is specified in the project settings. This directive may be used to specify any function as the top-level for synthesis. This then allows different solutions within the same project to be specified as the top-level function for synthesis without needing to create a new project.
UNROLL	Unroll for-loops to create multiple instances of the loop body and its instructions that can then be scheduled independently.

In addition to the optimization directives, Vitis HLS provides a number of configuration commands that can influence the performance of synthesis results. Details on using configurations commands can be found in HLS Config File Commands. The following table reflects some of these commands.

Table 20: **Vitis HLS Configurations**

GUI Directive	Description
Array Partition Configuration	Determines how arrays are partitioned, including global arrays and if the partitioning impacts array ports.
Compile Options	Controls synthesis specific optimizations such as the automatic loop pipelining and floating point math optimizations.
Dataflow Configuration	Specifies the default memory channel and FIFO depth in dataflow optimization.
Interface Configuration	Controls I/O ports not associated with the top-level function arguments and allows unused ports to be eliminated from the final RTL.
Operator Configuration	Configures the default latency and implementation of specified operations.
RTL Configuration	Provides control over the output RTL including file and module naming, and reset controls.
Schedule Setting	Determines the effort level to use during the synthesis scheduling phase and the verbosity of the output messages
Storage Configuration	Configures the default latency and implementation of specified storage types.
Unroll Setting	Configures the default tripcount threshold for unrolling loops.

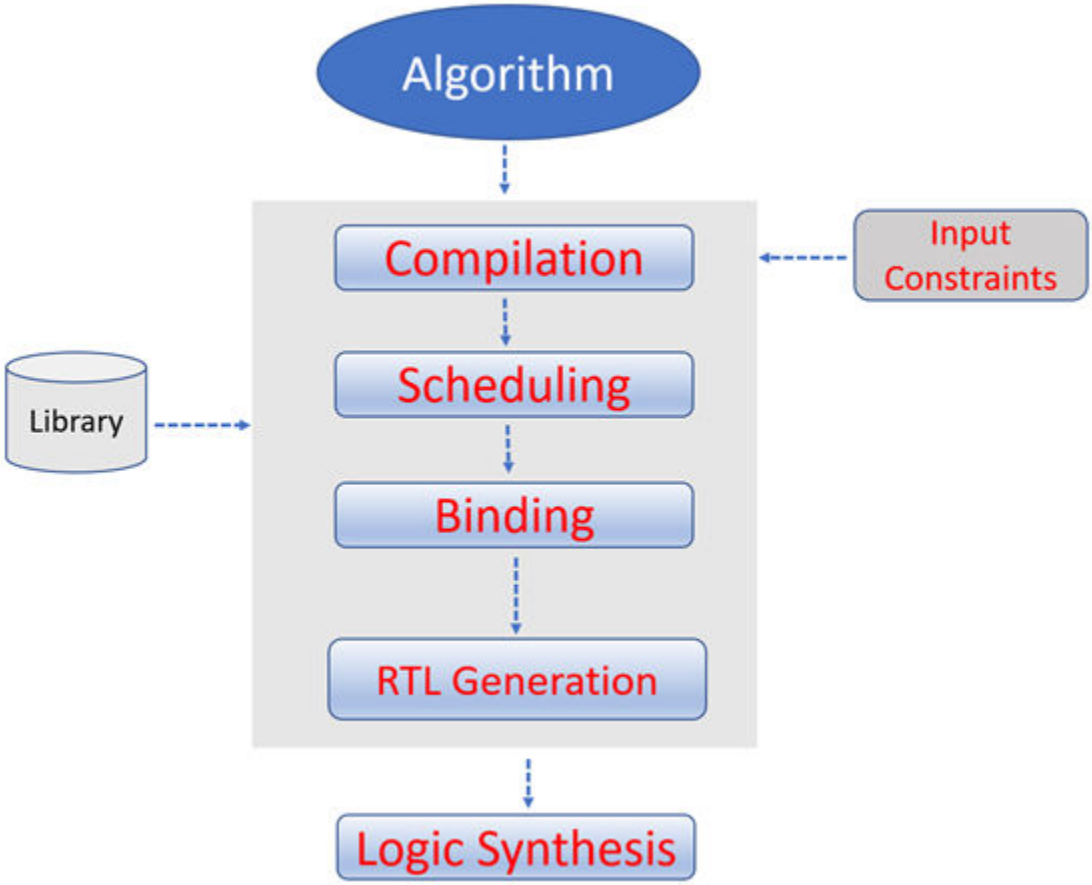
Understanding High-Level Synthesis Scheduling and Binding

High-Level Synthesis tools transform an untimed high-level specification into a fully timed implementation. During this transformation, a custom architecture is implemented to meet the specification requirements. The architecture generated contains the data path, control logic, memory interfaces, and how the RTL communicates with the external world. A data path consists of a set of storage elements such as (registers, register files, or memories), a set of functional units (such as ALUs, multipliers, shifters, and other custom functions), and interconnect elements (such as tristate drivers, multiplexers, and buses). Each component can take one or more clock cycles to execute, can be pipelined, and can have input or output registers. In addition, the entire data path and controller can be pipelined in several stages.

The designers should invest the early part of the project in redefining the architecture of the algorithm to meet the performance while keeping the algorithm at a higher level. For any specific HLS tool, there are design principles and best practices that are required to be followed to generate the optimized RTL that meets the expected performance.

The HLS Tool executes the following tasks as shown in the diagram below.

Figure 57: **HLS Steps**



1. Compile the algorithm written to meet specifications: This step includes several code optimizations such as dead-code elimination, constant folding, reporting unsupported constructs, etc.
2. Schedule the operations for given clock cycles:
   a. The "Schedule" phase determines which operations occur during each clock cycle based on:
      - When an operation's dependencies have been satisfied or are available.
      - The length of the clock cycle or clock frequency.
      - The time it takes for the operation to complete, as defined by the target device. More operations can be completed in a single clock cycle for longer clock periods. Some operations might need to be implemented as multi-cycle resources. HLS automatically schedules operations over more clock cycles
      - The available resources.
      - Incorporation of any user-specified optimization directives.

   b. During the "Schedule" phase, the tool determines what operator will execute in a given cycle and how many of these components are needed. The next step determines what operation binds to what resource.

3. Bind the operations to the functional components and variables to the storage elements
   a. The binding task assigns hardware resources to implement each scheduled operation and maps operators (such as addition, multiplication, and shift) to specific RTL implementations. For example, a mult operation can be implemented in RTL as a combinational or pipelined multiplier.
   b. The binding task assigns memories, registers, or combinations of these to the array variables inside the function to meet the desired performance.
   c. If multiple operations use the same resource, this step can perform the resource sharing if not used in the same cycle.
4. Control logic extraction creates a finite state machine (FSM) that sequences the operations in the RTL design according to the defined schedule.
5. Creates the logic to communicate with the external world: The RTL generated will be communicating with the external world like streaming data from the external port or start/stop logic or accessing external memory.
6. Finally, generate the RTL architecture

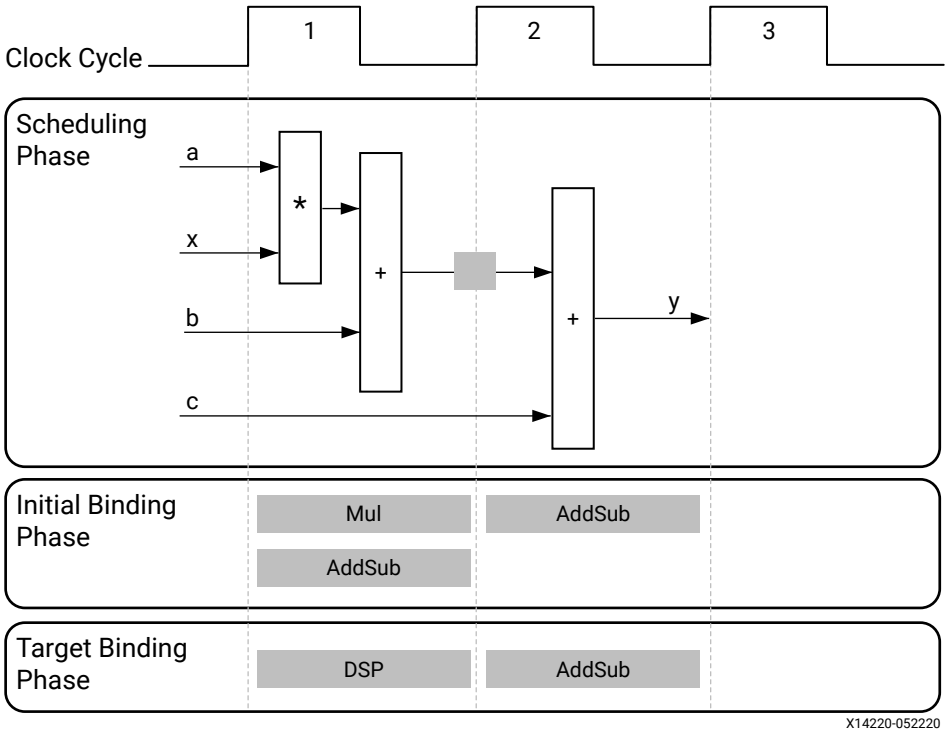
The next section walks through conceptually, how an HLS tool in general schedules the operators based on input constraints like a clock cycle and binds them to available hardware resources.

Scheduling and Binding Example

The following figure shows an example of the scheduling and binding phases for this code example:

```
int foo(char x, char a, char b, char c) {
 char y;
 y = x*a+b+c;
 return y;
}
```

Figure 58: **Scheduling and Binding Example**



In the scheduling phase of this example, high-level synthesis schedules the following operations to occur during each clock cycle:

- First clock cycle: Multiplication and the first addition
- Second clock cycle: Second addition, if the result of the first addition is available in the second clock cycle, and output generation

***Note*:** In the preceding figure, the square between the first and second clock cycles indicates when an internal register stores a variable. In this example, high-level synthesis only requires that the output of the addition is registered across a clock cycle. The first cycle reads `x`, `a`, and `b` data ports. The second cycle reads data port `c` and generates output `y`.

In the final hardware implementation, high-level synthesis implements the arguments to the top-level function as input and output (I/O) ports. In this example, the arguments are simple data ports. Because each input variable is a `char` type, the input data ports are all 8-bits wide. The function `return` is a 32-bit `int` data type, and the output data port is 32-bits wide.

IMPORTANT! *The advantage of implementing the C code in the hardware is that all operations finish in a shorter number of clock cycles. In this example, the operations complete in only two clock cycles. In a central processing unit (CPU), even this simple code example takes more clock cycles to complete.*

In the initial binding phase of this example, high-level synthesis implements the multiplier operation using a combinational multiplier (Mul) and implements both add operations using a combinational adder/subtractor (AddSub).

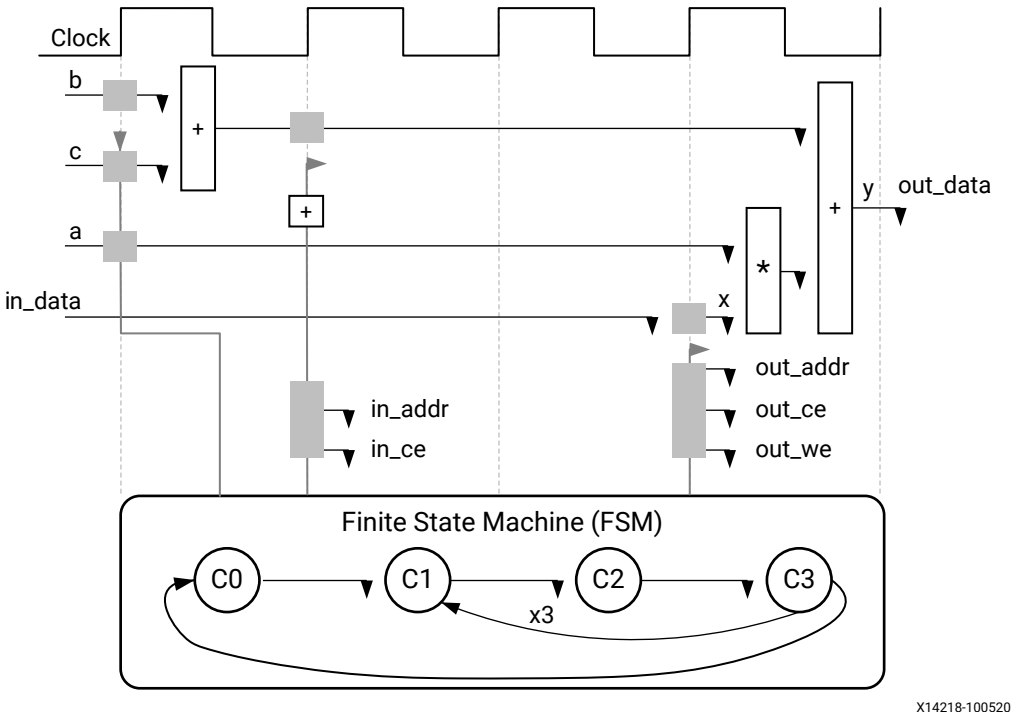
In the target binding phase, high-level synthesis implements both the multiplier and one of the addition operations using a DSP module resource. Some applications use many binary multipliers and accumulators that are best implemented in dedicated DSP resources. The DSP module is a computational block available in the FPGA architecture that provides the ideal balance of high-performance and efficient implementation.

Extracting Control Logic and Implementing I/O Ports Example

The following figure shows the extraction of control logic and implementation of I/O ports for this code example:

```
void foo(int in[3], char a, char b, char c, int out[3]) {
  int x,y;
  for(int i = 0; i < 3; i++) {
    x = in[i];
    y = a*x + b + c;
    out[i] = y;
  }
}
```

Figure 59: **Control Logic Extraction and I/O Port Implementation Example**



This code example performs the same operations as the previous example. However, it performs the operations inside a for-loop, and two of the function arguments are arrays. The resulting design executes the logic inside the for-loop three times when the code is scheduled. High-level synthesis automatically extracts the control logic from the C code and creates an FSM in the RTL design to sequence these operations. Top-level function arguments become ports in the final RTL design. The scalar variable of type `char` maps into a standard 8-bit data bus port. Array arguments, such as `in` and `out`, contain an entire collection of data.

In high-level synthesis, arrays are synthesized into block RAM by default, but other options are possible, such as FIFOs, distributed RAM, and individual registers. When using arrays as arguments in the top-level function, high-level synthesis assumes that the block RAM is outside the top-level function and automatically creates ports to access a block RAM outside the design, such as data ports, address ports, and any required chip-enable or write-enable signals.

The FSM controls when the registers store data and controls the state of any I/O control signals. The FSM starts in the state `C0`. On the next clock, it enters state `C1`, then state `C2`, and then state `C3`. It returns to state `C1` (and `C2`, `C3`) a total of three times before returning to state `C0`.

***Note*:** This closely resembles the control structure in the C code for-loop. The full sequence of states are: `C0`,`{C1, C2, C3}`,`{C1, C2, C3}`,`{C1, C2, C3}`, and return to `C0`.

The design requires the addition of `b` and `c` only one time. High-level synthesis moves the operation outside the for-loop and into state `C0`. Each time the design enters state `C3`, it reuses the result of the addition.

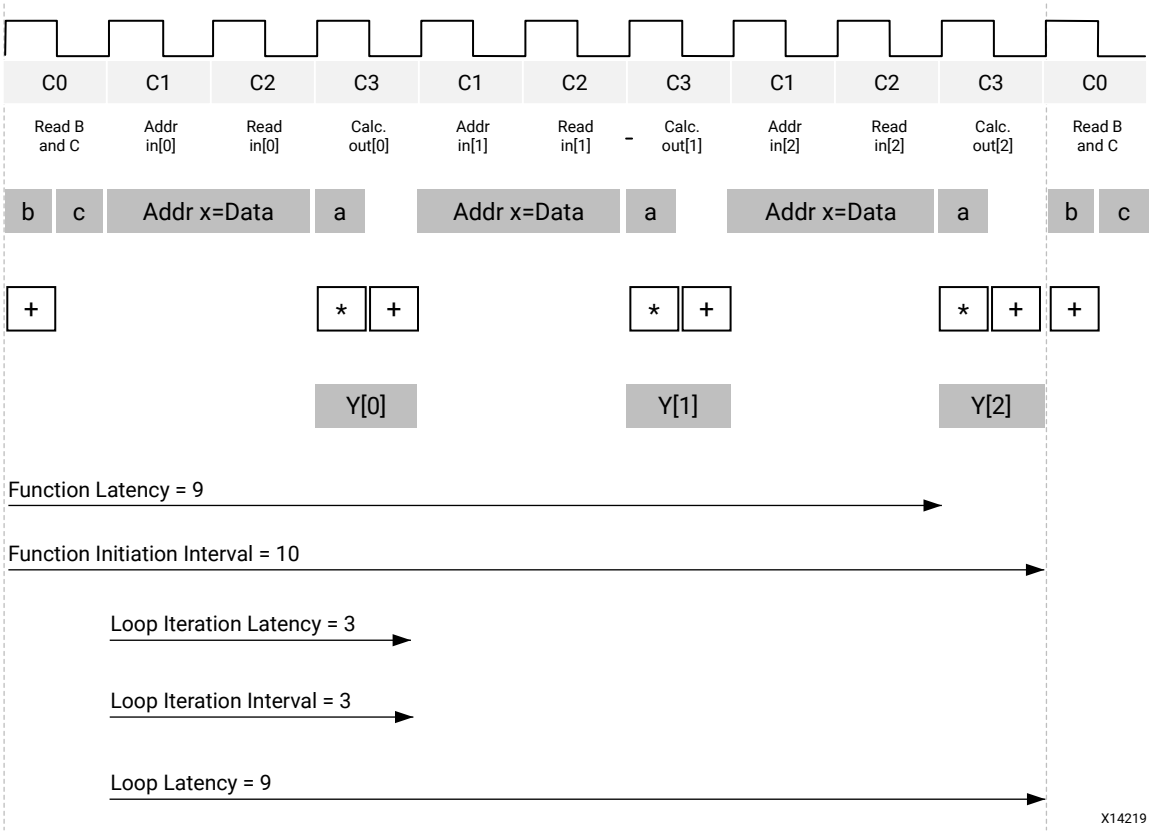
The design reads the data from `in` and stores the data in `x`. The FSM generates the address for the first element in state `C1`. In addition, in state `C1`, an adder increments to keep track of how many times the design must iterate around states `C1`, `C2`, and `C3`. In state `C2`, the block RAM returns the data for in and stores it as variable `x`.

High-level synthesis reads the data from port `a` with other values to perform the calculation and generates the first `y` output. The FSM ensures that the correct address and control signals are generated to store this value outside the block. The design then returns to state `C1` to read the next value from the array/block RAM `in`. This process continues until all outputs are written. The design then returns to state `C0` to read the next values of `b` and `c` to start the process again.

Performance Metrics Example

The following figure shows the complete cycle-by-cycle execution for the code in the previous example, including the states for each clock cycle, read operations, computation operations, and write operations.

Figure 60: **Latency and Initiation Interval Example**



The following are performance metrics for this example:

- Latency: It takes the function 9 clock cycles to output all values.

  ***Note*:** When the output is an array, the latency is measured to the last array value output.

- Initiation Interval (II): The II is 10, which means it takes 10 clock cycles before the function can initiate a new set of input reads and start to process the next set of input data.

  ***Note*:** The time to perform one complete execution of a function is referred to as one *transaction*. In this example, it takes 11 clock cycles before the function can accept data for the next transaction.

- Loop iteration latency: The latency of each loop iteration is 3 clock cycles.
- Loop II: The interval is 3.
- Loop latency: The latency is 9 clock cycles.

Optimizing Logic

Inferring Shift Registers

Vitis HLS will now infer a shift register when encountering the following code:

```
int A[N]; // This will be replaced by a shift register

for(...) {
  // The loop below is the shift operation
  for (int i = 0; i < N-1; ++i)
    A[i] = A[i+1];
  A[N] = ...;

  // This is an access to the shift register
  ... A[x] ...
}
```

Shift registers can perform a one shift operation per cycle, and also allows a random read access per cycle anywhere in the shift register, thus it is more flexible than a FIFO.

Optimizing Logic Expressions

During synthesis several optimizations, such as strength reduction and bit-width minimization are performed. Included in the list of automatic optimizations is expression balancing.

Expression balancing rearranges operators to construct a balanced tree and reduce latency.

- For integer operations expression balancing is on by default but may be disabled using the EXPRESSION_BALANCE pragma or directive.
- For floating-point operations, expression balancing is off by default but may be enabled using using the `config_compile -unsafe_math_optimizations` command, as discussed below.

Given the highly sequential code using assignment operators such as `+=` and `*=` in the following example (or resulting from loop unrolling):

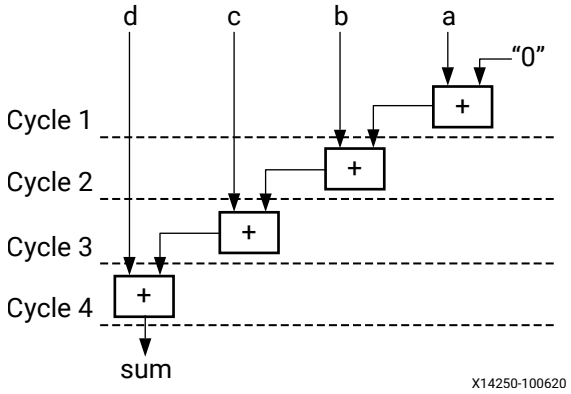
```
data_t foo_top (data_t a, data_t b, data_t c, data_t d)
{
 data_t sum;

 sum = 0;
 sum += a;
 sum += b;
 sum += c;
 sum += d;
 return sum;

}
```

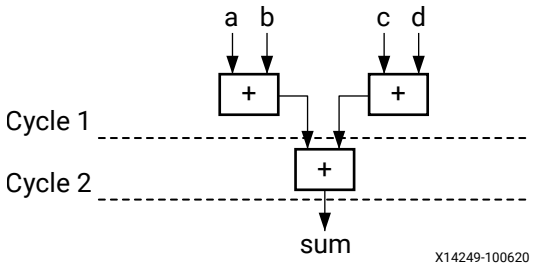
Without expression balancing, and assuming each addition requires one clock cycle, the complete computation for `sum` requires four clock cycles shown in the following figure.

Figure 61: **Adder Tree**



However additions `a+b` and `c+d` can be executed in parallel allowing the latency to be reduced. After balancing the computation completes in two clock cycles as shown in the following figure. Expression balancing prohibits sharing and results in increased area.

Figure 62: **Adder Tree After Balancing**



For integers, you can disable expression balancing using the EXPRESSION_BALANCE optimization directive with the `off` option. By default, the HLS tool does not perform the EXPRESSION_BALANCE optimization for operations of type `float` or `double`. When synthesizing `float` and `double` types, the tool maintains the order of operations performed in the C/C++ code to ensure that the results are the same as the C/C++ simulation. For example, in the following code example, all variables are of type `float` or `double`. The values of `O1` and `O2` are not the same even though they appear to perform the same basic calculation.

```
A=B*C; A=B*F;
D=E*F; D=E*C;
O1=A*D O2=A*D;
```

This behavior is a function of the saturation and rounding in the C/C++ standard when performing operation with types `float` or `double`. Therefore, the HLS compiler always maintains the exact order of operations when variables of type `float` or `double` are present and does not perform expression balancing by default.

You can enable expression balancing for specific operations, or you can configure the tool to enable expression balancing with `float` and `double` types using the `syn.compile.unsafe_math_optimizations=1` configuration command.

With this setting enabled, the tool might change the order of operations to produce a more optimal design. However, the results of C/RTL co-simulation might differ from the C/C++ simulation.

The `unsafe_math_optimizations` feature also enables the `syn.compile.no_signed_zeros` optimization. The `no_signed_zeros` optimization ensures that the following expressions used with `float` and `double` types are identical:

```
x - 0.0 = x;
x + 0.0 = x;
0.0 - x = -x;
x - x = 0.0;
x*0.0 = 0.0;
```

Without the `no_signed_zeros` optimization the expressions above would not be equivalent due to rounding. The optimization may be optionally used without expression balancing by selecting only the `syn.compile.no_signed_zeros` configuration command.



TIP: *When the `unsafe_math_optimizations` and `no_signed_zero` optimizations are used, the RTL implementation will have different results than the C/C++ simulation. The test bench should be capable of ignoring minor differences in the result: check for a range, do not perform an exact comparison.*

Optimizing AXI System Performance

Introduction

A Vitis accelerated system includes a global memory subsystem that is used to share data between the kernels and the host application. Global memory available on the host system, outside of the AMD device, provides very large amounts of storage space but at the cost of longer access time compared to local memory on the AMD device. One of the measurements of the performance of a system/application is throughput, which is defined as the number of bytes transferred in a given time frame. Therefore, inefficient data transfers from/to the global memory will have a long memory access time which can adversely affect system performance and kernel execution time.

Development of accelerated applications in Vitis HLS should include two phases: kernel development, and improving system performance. Design Principles suggested a kernel development approach implementing a cache-like Load-Compute-Store structure where the load-store functions read/write data to the global memory. Improving system performance involves implementing an efficient load and store design that can improve the kernel execution time. This chapter describes the features and metrics that can impact and improve the throughput of the load-store (LS) functions. Refer to Vitis-HLS-Introductory-Examples/Interface/Memory on GitHub for examples of some of the following concepts.

AXI Burst Transfers

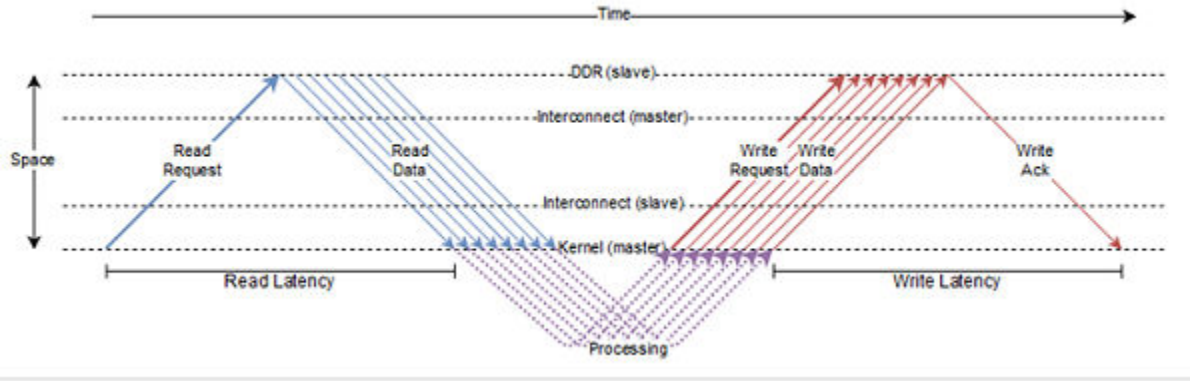
Overview of Burst Transfers

Bursting is an optimization that tries to intelligently aggregate your memory accesses to the DDR to maximize the throughput bandwidth and/or minimize the latency. Bursting is one of many possible optimizations to the kernel. Bursting typically gives you a 4-5x improvement while other optimizations, like access widening or ensuring there are no dependencies through the DDR memory, can provide even bigger performance improvements. Typically, bursting is useful when you have contention on the DDR ports from multiple competing kernels.

The Vitis HLS tool supports automatic burst inference, and supports a number of features such as automatic port-widening to facilitate automatic burst access. In some cases, where autmatic burst access has failed, an efficient solution is to re-write the code or use manual burst as described in Using Manual Burst. If that does not work, then another solution might be to use cache memory in the AXI4 interface using the CACHE pragma or directive.

The burst feature of the AXI4 protocol improves the throughput of the load-store functions by reading/writing chunks of data to or from the global memory in a single request. The larger the size of the data, the higher the throughput. This metric is calculated as follows ((#of bytes transferred)* (kernel frequency)/(Time)). The maximum kernel interface bitwidth is 512 bits, and if the kernel is compiled to run at 300 MHz then it can theoretically achieve (512* 300 MHz)/1 sec = ~17 GBps for a DDR memory.

Figure 63: **AXI Protocol**



The preceding figure shows how the AXI protocol works. The HLS kernel sends out a read request for a burst of length 8 and then sends a write request burst of length 8. The read latency is defined as the time taken between the sending of the read request burst to when the data from the first read request in the burst is received by the kernel. Similarly, the write latency is defined as the time taken between when data for the last write in the write burst is sent and the time the write acknowledgment is received by the kernel. Read requests are usually sent at the first available opportunity while write requests get queued until the data for each write in the burst becomes available.

TIP: *This behavior is determined by the* `syn.interface.m_axi_conservative_mode` *option as described in Interface Configuration, which is enabled as the default behavior.*

To understand the underlying semantics of burst transfers consider the following code snippet:

```
for(size_t i = 0; i < size; i++) {
    out[f(i)] = in[f(i)]);
}
```

Vitis HLS performs automatic burst optimization, which intelligently aggregates the memory accesses inside the loops/functions from the user code and performs read/write to the global memory of a particular size. These read/writes are converted into a read request, write request, and write response to the global memory. Depending on the memory access pattern Vitis HLS automatically inserts these read and write requests either outside the loop bound or inside the loop body. Depending on the placement of these requests, Vitis HLS defines two types of burst requests: sequential burst and pipelined burst.

Burst Semantics

For a given kernel, the HLS compiler implements the burst analysis optimization as a multi-pass optimization, but on a per function basis. Bursting is only done for a function and bursting across functions is not supported. The burst optimizations are reported in the Synthesis Summary report, and missed burst opportunities are also reported to help you improve burst optimization.

At first, the HLS compiler looks for memory accesses in the basic blocks of the function, such as memory accesses in a sequential set of statements inside the function. Assuming the preconditions of bursting are met, each burst inferred in these basic blocks is referred to as *sequential* burst. The compiler will automatically scan the basic block to build the longest sequence of accesses into a single sequential burst.

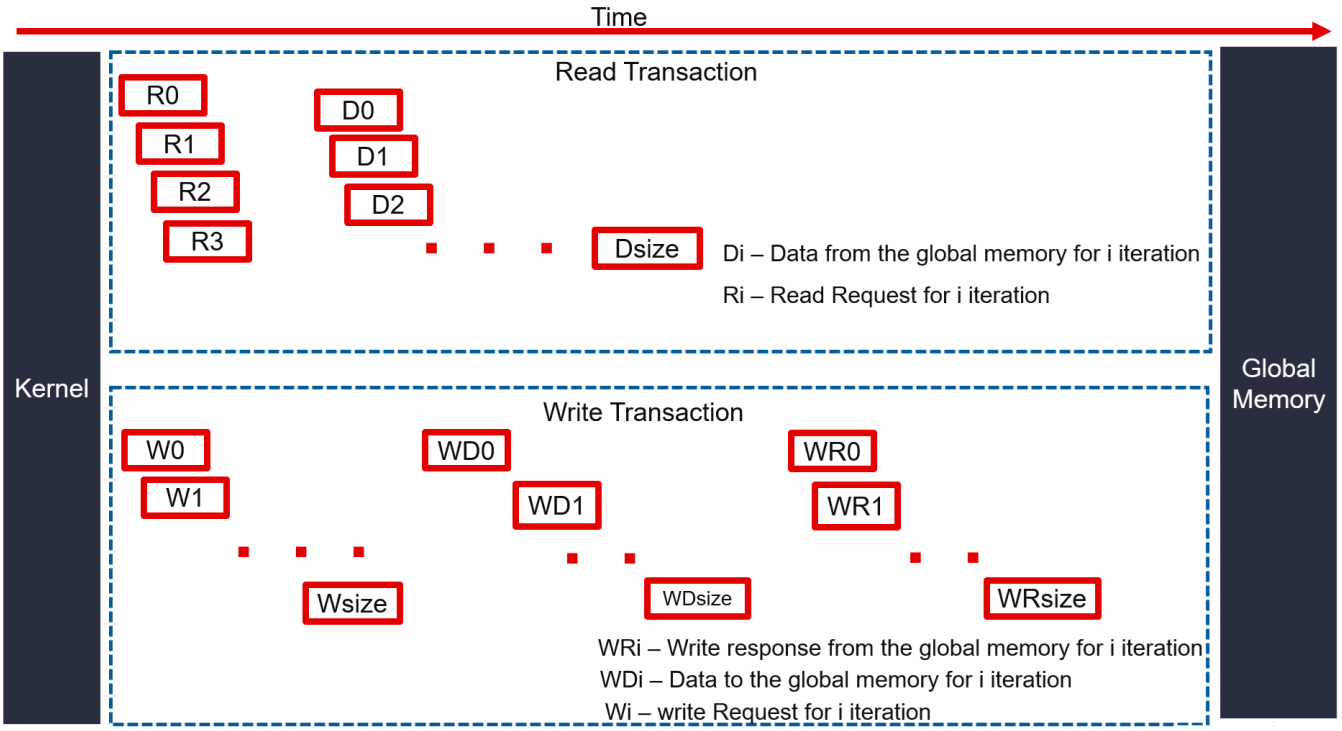
The compiler then looks at loops and tries to infer what are known as *pipeline* bursts. A pipeline burst is the sequence of reads/writes across the iterations of a loop. The compiler tries to infer the length of the burst by analyzing the loop induction variable and the trip count of the loop. If the analysis is successful, the compiler can chain the sequences of reads/writes in each iteration of the loop into one long pipeline burst. The compiler today automatically infers a pipeline or a sequential burst, but there is no way to request a specific type of burst. The code needs to be written so as to cause the tool to infer the pipeline or sequential burst.

Pipeline Burst

Pipeline burst improves the throughput of the functions by reading or writing large amounts, or the maximum amount of data in a single request. The advantage of the pipeline burst is that the future requests (i+1) do not have to wait for the current request (i) to finish because the read request, write request, and write response are outside the loop body and performs the requests as soon as possible, as shown in the following code example. This significantly improves the throughput of the functions as it takes less time to read/write the whole loop bound.

```
rb = ReadReq(i, size);
wb = WriteReq(i, size);
for(size_t i = 0; i < size; i++) {
  Write(wb, i) = f(Read(rb, i));
}
WriteResp(wb);
```

Figure 64: **Pipeline Burst**



If the compiler can successfully deduce the burst length from the induction variable (`i`) and the trip count of the loop (`size`), it will infer one big pipeline burst and will move the `ReadReq`, `WriteReq` and `WriteResp` calls outside the loop, as shown in the Pipeline Burst code example. So, the read requests for all loop iterations are combined into one read request and all the write requests are combined into one write request. All read requests are typically sent out immediately while write requests are only sent out after the data becomes available.

In this case, the read and write requests for each loop iteration are combined into one read or write request. However, if any of the preconditions of bursting are not met, as described in Preconditions and Limitations of Burst Transfer, the compiler might not infer a pipeline burst but will instead try and infer a sequential burst where the `ReadReq`, `WriteReg` and `WriteResp` are alongside the read/write accesses being burst optimized, as shown in the Sequential Burst code example.

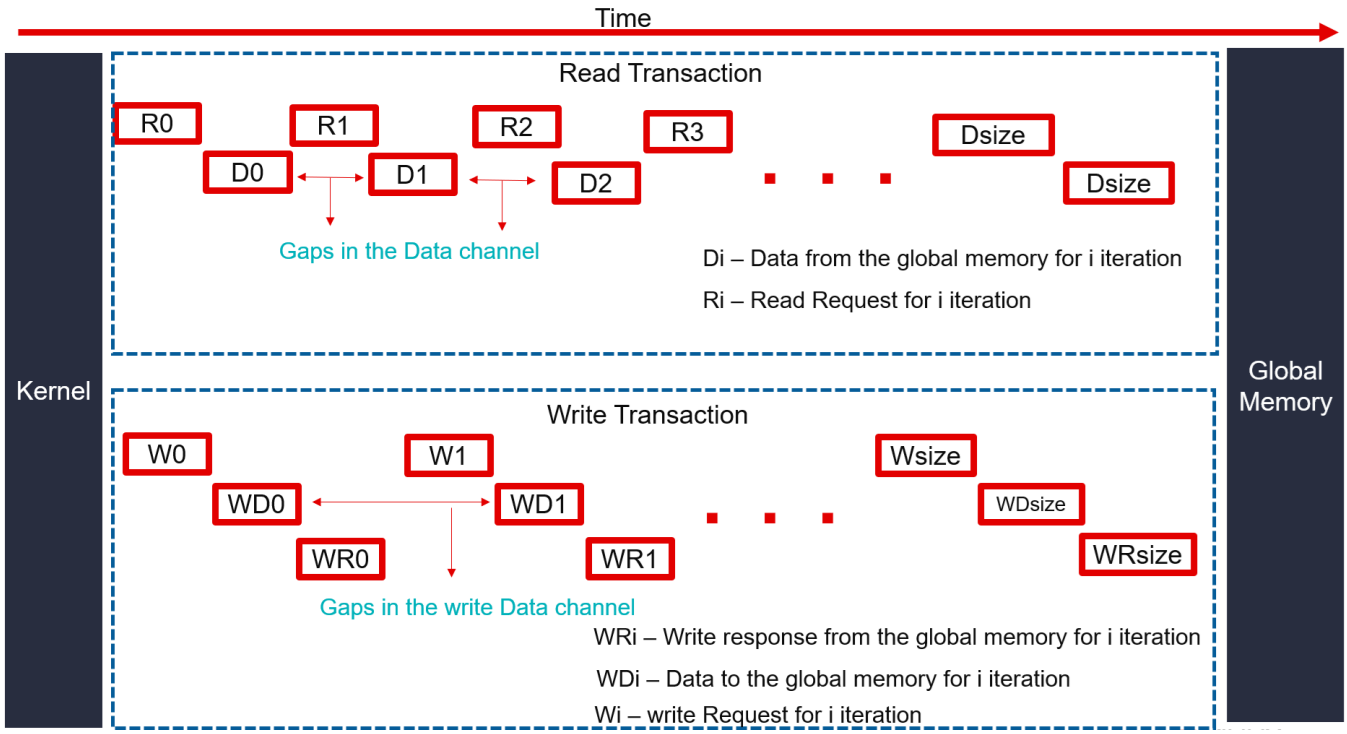
Sequential Burst

A sequential burst consists of smaller data sizes where the read requests, write requests, and write responses are inside a loop body as shown in the following code example.

```
for(size_t i = 0; i < size; i++) {
    rb = ReadReq(i, 1);
    wb = WriteReq(i, 1);
    Write(wb, i) = f(Read(rb, i));
    WriteResp(wb);
}
```

The drawback of sequential burst is that a future request (i+1) depends on the current request (i) finishing because it is waiting for the read request, write request, and write response to complete. This will create gaps between requests as shown in the following figure.

Figure 65: **Sequential Burst**



A sequential burst is not as effective as pipeline burst because it is reading or writing a small data size multiple times to compensate for the loop bounds. Although this will have a significant impact on the throughput, sequential burst is still better than no burst. Vitis HLS uses this burst technique if your code does not adhere to the Preconditions and Limitations of Burst Transfer.

TIP: *The size of burst requests can be further partitioned into multiple requests of user-specified size, which is controlled using the* `max_read_burst_length` *and* `max_write_burst_length` *of the INTERFACE pragma or directive, as discussed in Options for Controlling AXI4 Burst Behavior.*

Preconditions and Limitations of Burst Transfer

Bursting Preconditions

Bursting is about aggregating successive memory access requests. Here are the set of preconditions that these successive accesses must meet for the bursting optimization to launch successfully:

- Must be all reads, or all writes – bursting reads and writes is not possible.

- Must be a monotonically increasing order of access (both in terms of the memory location being accessed as well as in time). You cannot access a memory location that is in between two previously accessed memory locations.
- Must be consecutive in memory – one next to another with no gaps or overlap and in forward order.
- The number of read/write accesses (or burst length) must be determinable before the request is sent out. This means that even if the burst length is computed at runtime, it must be computed before the read/write request is sent out.
- If bundling two arrays to the same M-AXI port, bursting will be done only for one array, at most, in each direction at any given time.
- If same direction accesses on the same channel of the same bundle in the same region, there will be no burst for all these accesses.
- There must be no dependency issues from the time a burst request is initiated and finished.



TIP: *The `volatile` qualifier prevents burst access to or from the variable.*

Outer Loop Burst Failure Due to Overlapping Memory Accesses

Outer loop burst inference will fail in the following example because both iteration 0 and iteration 1 of the loop L1 access the same element in arrays a and b. Burst inference is an all or nothing type of optimization - the tool will not infer a partial burst. It is a greedy algorithm that tries to maximize the length of the burst. The auto-burst inference will try to infer a burst in a bottom up fashion - from the inner loop to the outer loop, and will stop when one of the preconditions is not met. In the example below the burst inference will stop when it sees that element 8 is being read again, and so an inner loop burst of length 9 will be inferred in this case.

```
L1: for (int i = 0; i < 8; ++i)
    L2: for (int j = 0; j < 9; ++j)
            b[i*8 + j] = a[i*8 + j];

itr 0: |0 1 2 3 4 5 6 7 8|
itr 1: |                 8 9 10 11 12 13 14 15 16|
```

Usage of ap_int/ap_uint Types as Loop Induction Variables

Because the burst inference depends on the loop induction variable and the trip count, using non-native types can hinder the optimization from firing. It is recommended to always use unsigned integer type for the loop induction variable.

Must Enter Loop at Least Once

In some cases, the compiler can fail to infer that the max value of the loop induction variable can never be zero – that is, if it cannot prove that the loop will always be entered. In such cases, an assert statement will help the compiler infer this.

```
assert (N > 0);
L1: for(int a = 0; a < N; ++a) { … }
```

Inter or Intra Loop Dependencies on Arrays

If you write to an array location and then read from it in the same iteration or the next, this type of array dependency can be hard for the optimization to decipher. Basically, the optimization will fail for these cases because it cannot guarantee that the write will happen before the read.

Conditional Access to Memory

If the memory accesses are being made conditionally, it can cause the burst inferencing algorithm to fail as it cannot reason through the conditional statements. In some cases, the compiler will simplify the conditional and even remove it but it is generally recommended to not use conditional statements around the memory accesses.

M-AXI Accesses Made from Inside a Function Called from a Loop

Cross-functional array access analysis is not a strong suit for compiler transformations such as burst inferencing. In such cases, users can inline the function using the INLINE pragma or directive to avoid burst failures.

```
void my_function(hls::stream<T> &out_pkt, int *din, int input_idx) {
    T v;
    v.data = din[input_idx];
    out_pkt.write(v);
}

void my_kernel(hls::stream<T> &out_pkt,
               int            *din,
               int            num_512_bytes,
               int            num_times) {
#pragma HLS INTERFACE mode=m_axi port = din offset=slave bundle=gmem0
#pragma HLS INTERFACE mode=axis port=out_pkt
#pragma HLS INTERFACE mode=s_axilite port=din bundle=control
#pragma HLS INTERFACE mode=s_axilite port=num_512_bytes bundle=control
#pragma HLS INTERFACE mode=s_axilite port=num_times bundle=control
#pragma HLS INTERFACE mode=s_axilite port=return bundle=control

unsigned int idx = 0;
L0: for (int i = 0; i < ntimes; ++i) {
    L1: for (int j = 0; j < num_512_bytes; ++j) {
#pragma HLS PIPELINE
        my_function(out_pkt, din, idx++);
    }
}
```

Burst inferencing will fail because the memory accesses are being made from a called function. For the burst inferencing to work, it is recommended that users inline any such functions that are making accesses to the M-AXI memory.

An additional reason the burst inferencing will fail in this example is that the memory accessed through `din` in `my_function`, is defined by a variable (`idx`) which is not a function of the loop induction variables `i` and `j`, and therefore might not be sequential or monotonic. Instead of passing `idx`, use `(i*num_512_bytes+j)`.

Pipelined Burst Inference on a Dataflow Loop

Burst inference is not supported on a loop that has the DATAFLOW pragma or directive. However, each process/task inside the dataflow loop can have bursts. Also, sharing of M-AXI ports is not supported inside a dataflow region because the tasks can execute in parallel.

Options for Controlling AXI4 Burst Behavior

An optimal AXI4 interface is one in which the design never stalls while waiting to access the bus, and after bus access is granted, the bus never stalls while waiting for the design to read/write. There are many elements of the design that affect the system performance and burst transfer, such as the following:

- Latency
- Port Width
- Multiple Ports
- Specified Burst Length
- Number of Outstanding Reads/Writes

Latency

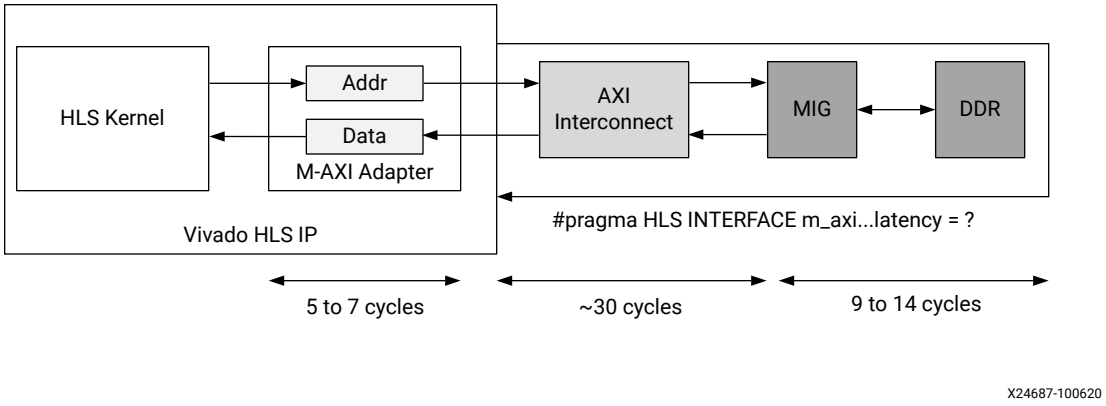
The read latency is defined as the time taken between sending the burst read request to when the kernel receives the data from the first read request in the burst. Similarly, the write latency is defined as the time it takes between when data for the last write in the burst is sent and the time the write response is received by the kernel. These latencies can be non-deterministic because they depend on system characteristics such as congestion on the DDR memory access. Because of this the HLS compiler can not accurately determine the memory read/write latency during synthesis, and so uses a default latency of 64 kernel cycles to schedule the requests and operations as below.

- It schedules the read/write requests and waits for the data, in parallel perform memory-independent operations, such as working on streams or compute
- Wait to schedule new read/write requests

TIP: *The default tool latency can be changed using the LATENCY pragma or directive. The default latency of a specific interface can be changed using the latency option of the INTERFACE pragma or directive.*

To help you understand the various latencies that are possible in the system, the following figure shows what happens when an HLS kernel sends a burst to the DDR memory.

Figure 66: **Burst Transaction Diagram**



When your design makes a read/write request, the request is sent to the DDR memory through several specialized helper modules. First, the `m_axi` adapter serves as a buffer for the requests created by the HLS kernel. The adapter contains logic to cut large bursts into smaller ones (which it needs to do to prevent hogging the channel or if the request crosses the 4 KB boundary, see *Vivado Design Suite: AXI Reference Guide* (UG1037)).

By default the adapter stalls the sending of burst requests until all data is available (depending on the maximum outstanding requests parameter) so that it can safely buffer the entirety of the data for each kernel. This is done to reduce deadlock due to concurrent requests (read or write) on the memory subsystem, but can increase write latency. You can disable this hold of write requests by setting `syn.interface.m_axi_conservative_mode=false` as described in Interface Configuration, but increase the risk of deadlock.

Getting through the adapter will cost a few cycles of latency, typically 5 to 7 cycles. Then, the request goes to the AXI interconnect that routes the kernel's request to the MIG and then eventually to the DDR memory. Getting through the interconnect is expensive in latency and can take around 30 cycles. Finally, getting to the DDR and back can cost anywhere from 9 to 14 cycles. These are not precise measurements of latency but rather estimates provided to show the relative latency cost of these specialized modules. For more precise measurements, you can test and observe these latencies using the Application Timeline report for your specific system, as described in AXI Performance Case Study.

TIP: *For information about the Application Timeline report, see Timeline Trace in the Data Center Acceleration using Vitis (UG1700).*

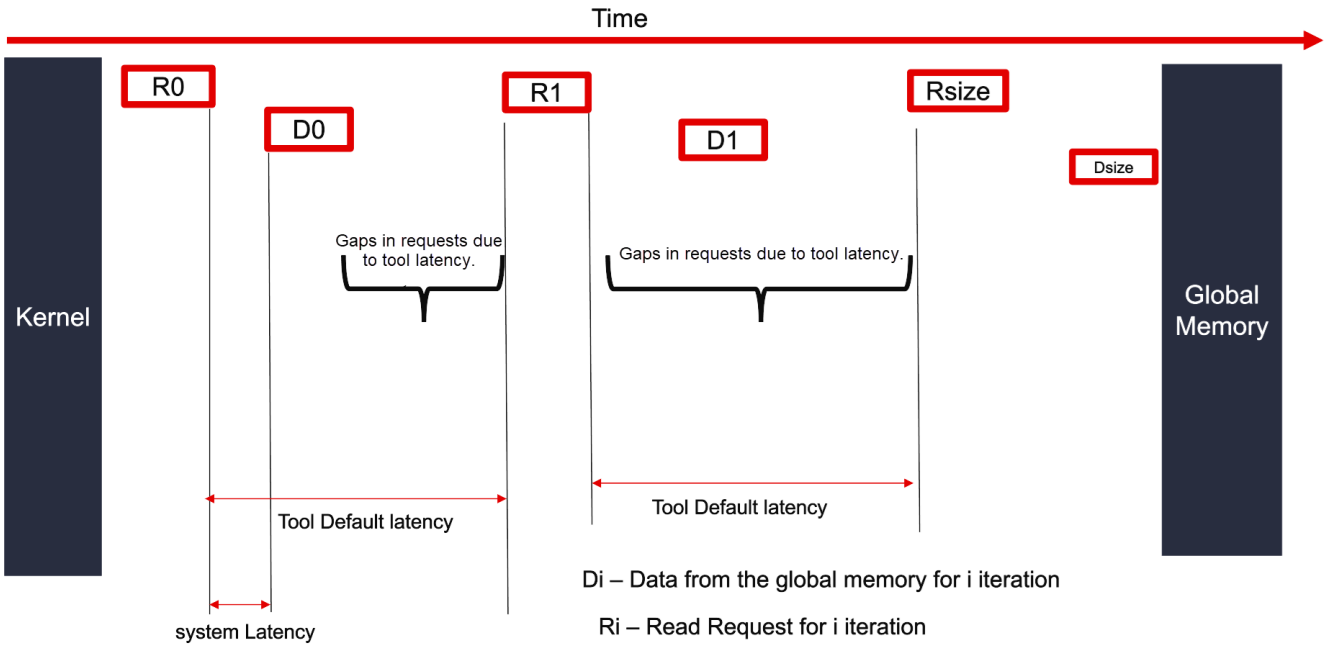
Another way to view the latencies in the system is as follows: the interconnect has an average II of 2 while the DDR memory controller has an average II of 4-5 cycles on requests (while on the data they are both II=1). The interconnect arbitration strategy is based on the size of read/write requests, and so data requested with longer burst lengths get prioritized over requests with shorter bursts (thus leading to a bigger channel bandwidth being allocated to longer bursts in case of contention). Of course, a large burst request has the side-effect of preventing anyone else from accessing the DDR memory, and therefore there must be a compromise between burst length and reducing DDR port contention. Fortunately, the large latencies help prevent some of this port contention, and effective pipelining of the requests can significantly improve the bandwidth throughput available in the system.

Latency does not affect loops/functions with pipelined bursts because the burst requests the maximum size in a single request.

Latency effects loops/functions with sequential burst in two possible ways:

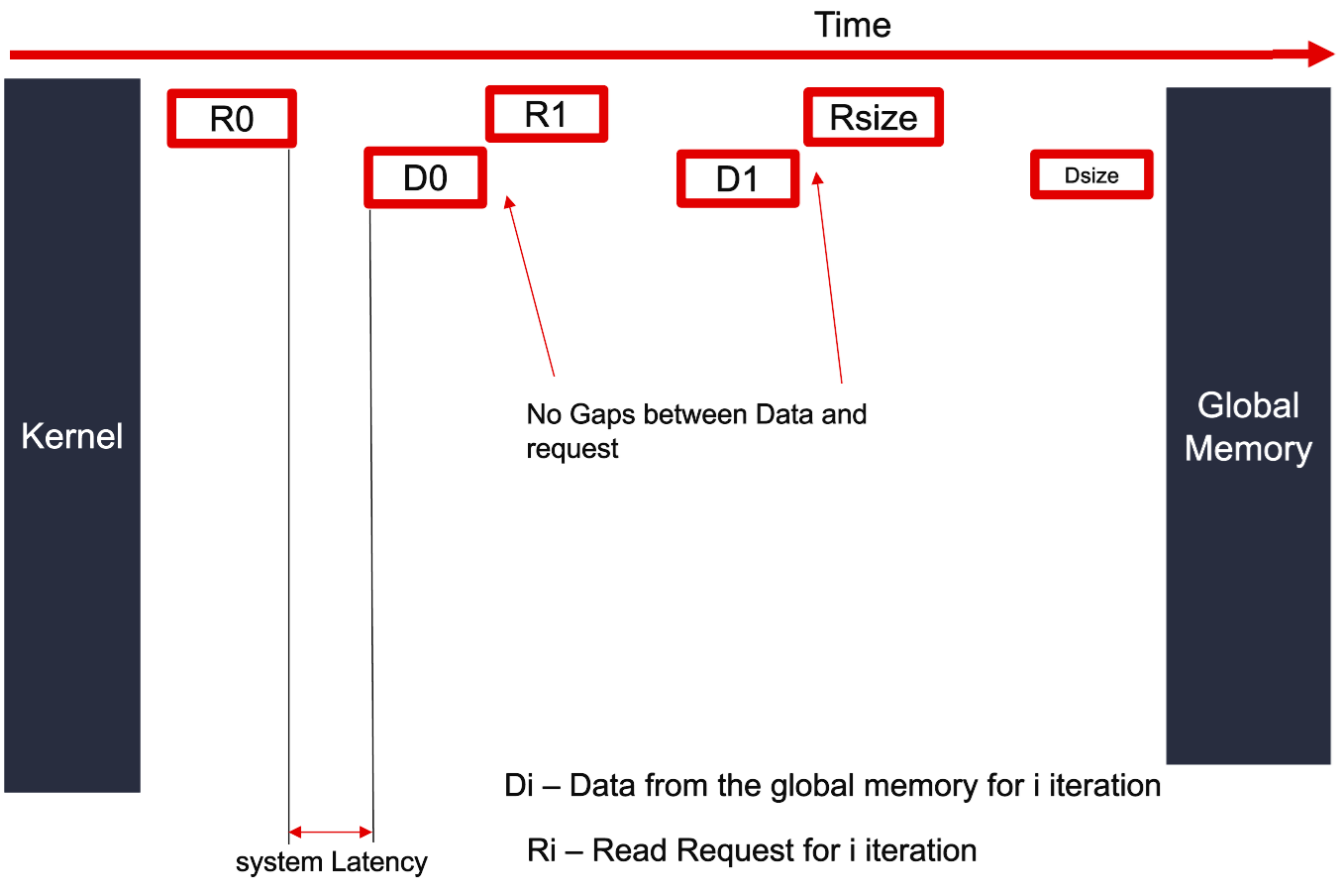
- If the system read/write latency is larger than the default tool latency, Vitis HLS has to wait for the data. Changing the latency will not improve the performance of the system.
- If the read/write latency is less than the tool default, then Vitis HLS sits in an idle state and wastes the remaining kernel cycles. This can impact the performance of the design because during this idle state it does not perform tasks. As you can see from the following figure the difference between the system latency and the default latency parameter will cause the sequential requests to be delayed further in time. This causes a significant loss of throughput.

Figure 67: **Default Tool Latency**



However, when you reduce the tool latency using the LATENCY pragma or directive, the tool will tightly pack the requests for a sequential burst, as shown in the following figure.

Figure 68: **Adjusted Tool Latency**



RECOMMENDED: *Latency has a significant impact on sequential burst. Decreasing the default tool latency can improve the system performance.*

Port Width

The throughput of load-store functions can be further improved by maximizing the number of bytes transferred. The Vitis unified IDE and `v++` supports kernel ports up to 1024 bits wide, which means that a kernel can read or write up to 128 bytes per clock cycle per port.

RECOMMENDED: *You should maximize the port width of the interface, i.e., the bit-width of each AXI port, by setting it to 512 bits (64 bytes).*

Vitis HLS also supports automatic port width optimization by analyzing the memory access pattern of the source code. If the code satisfies the preconditions and limitations for burst access, it will automatically resize the port to 512 bit width in the Vitis kernel flow.

IMPORTANT! *If the size and number of iterations are variable at compile time, then the tool will not automatically widen port widths.*

If the tool cannot automatically widen the port, you can manually change the port width by using Vector Data Types or Arbitrary Precision (AP) Data Types as the data type of the port.

Multiple Ports and Channels

The throughput of load-store functions can be further improved by maximizing concurrent read/writes. In Vitis HLS, the function arguments by default are bundled/mapped/grouped to a single port. Bundling ports into a single port helps save resources. However, a single port can limit the performance of the kernel because all the memory transfers have to go through a single port. The `m_axi` interface has independent READ and WRITE channels, so a single port can read and write simultaneously. The `m_axi` Interface also provides channels in a single port, as described in M_AXI Channels to increase read and write channels in the port.

Using multiple ports lets you increase the bandwidth and throughput of the kernel by creating multiple interfaces to connect to different memory banks, as shown in the Multi-DDR tutorial, or the accesses will be sequential. When multiple arguments are accessing the same memory port or memory bank, an arbiter will sequence the concurrent accesses to the same memory port or bank. Having multiple ports connected to different memory banks increases the throughput of the load and store functions, and as a result, the compute block should also be equally scaled to meet the throughput demand from the load and store functions otherwise it will put back-pressure or stalls on the load-store functions.

RECOMMENDED: *Analyze the concurrent memory reads/writes and have a dedicated/independent port for concurrent accesses.*

Number of Outstanding Reads/Writes

The throughput of load-store functions can be further improved by allowing the system to hide some of the memory latency. The `m_axi_num_read_outstanding` and `m_axi_num_write_outstanding` options of the `config_interface` command, or of the INTERFACE pragma or directive, lets the Kernel control the number of pipelined memory requests sent to the global memory without waiting for the previous request to complete.

Increasing the number of pipelined requests increases the pipeline depth of the read/write requests, which will cost additional BRAM/URAM resources.

***Note*:** In most cases where burst length >=16, the default number of outstanding reads/writes should be sufficient. For a burst of size less than 16, AMD recommends doubling the size of the number of outstanding from the default of 16.

Defining Burst Attributes with the INTERFACE Pragma

To create the optimal AXI4 interface, the following command options are provided in the INTERFACE directive to specify the behavior of the bursts and optimize the efficiency of the AXI4 interface.

Note that some of these options can use internal storage to buffer data and this may have an impact on area and resources:

- **`latency`:** Specifies the expected latency of the AXI4 interface, allowing the design to initiate a bus request several cycles (latency) before the read or write is expected. If this figure it too low, the design will be ready too soon and may stall waiting for the bus. If this figure is too high, bus access may be granted but the bus may stall waiting on the design to start the access. Default latency in Vitis HLS is 64.
- **`max_read_burst_length`:** Specifies the maximum number of data values read during a burst transfer. Default value is 16.
- **`num_read_outstanding`:** Specifies how many read requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design: a FIFO of size `num_read_outstanding*max_read_burst_length*word_size`. Default value is 16.
- **`max_write_burst_length`:** Specifies the maximum number of data values written during a burst transfer. Default value is 16.
- **`num_write_outstanding`:** Specifies how many write requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design: a FIFO of size `num_write_outstanding*max_write_burst_length*word_size`. Default value is 16.

The following INTERFACE pragma example can be used to help explain these options:

```
#pragma HLS interface mode=m_axi port=input offset=slave
bundle=gmem0
depth=1024*1024*16/(512/8) latency=100 num_read_outstanding=32
num_write_outstanding=32
max_read_burst_length=16 max_write_burst_length=16
```

- The interface is specified as having a latency of 100. The HLS compiler seeks to schedule the request for burst access 100 clock cycles before the design is ready to access the AXI4 bus.
- To further improve bus efficiency, the options `num_write_outstanding` and `num_read_outstanding` ensure the design contains enough buffering to store up to 32 read and/or write bursts. Each request will require its own buffer. This allows the design to continue processing until the bus requests are serviced.

- Finally, the options `max_read_burst_length` and `max_write_burst_length` ensure the maximum burst size is 16 and that the AXI4 interface does not hold the bus for longer than this. The HLS tool will partition longer bursts according to the specified burst length, and report this condition with a message like the following:

```
Multiple burst reads of length 192 and bit width 128 in
loop 'VITIS_LOOP_2'(./src/filter.cpp:247:21)has been inferred on port
'mm_read'.
These burst requests might be further partitioned into multiple requests
during RTL generation based on the max_read_burst_length settings.
```

Commands to Configure the Burst

These commands configure global settings for the tool to optimize the AXI4 interface for the system in which it will operate. The efficiency of the operation depends on these values being set accurately. The provided default values are conservative, and might require changing depending on the memory access profile of your design.

Table 21: **Vitis HLS Controls**

Vitis HLS Command	Value	Description
`syn.interface.m_axi_conservative_mode`	bool default=true	Delay M-AXI each write request until the associated write data are entirely available (typically, buffered into the adapter or already emitted). This can slightly increase write latency but can resolve deadlock due to concurrent requests (read or write) on the memory subsystem.
`syn.interface.m_axi_latency`	uint 0 is auto default=0 (for Vivado IP flow) default=64 (for Vitis Kernel flow)	Provide the scheduler with an expected latency for M-AXI accesses. Latency is the delay between a read request and the first read data, or between the last write data and the write response. Note that this number need not be exact, underestimation makes for a lower-latency schedule, but with longer dynamic stalls. The scheduler will account for the additional adapter latency and add a few cycles.
`syn.interface.m_axi_min_bitwidth`	uint default=8	Minimum bitwidth for M-AXI interfaces data channels. Must be a power-of-two between 8 and 1024. Note that this does not necessarily increase throughput if the actual accesses are smaller than the required interface.
`syn.interface.m_axi_max_bitwidth`	uint default=1024	Minimum bitwidth for M-AXI interfaces data channels. Must be a power-of-two between 8 and 1024. Note that this does decrease throughput if the actual accesses are bigger than the required interface as they will be split into a multi-cycle burst of accesses.

Table 21: **Vitis HLS Controls** *(cont'd)*

Vitis HLS Command	Value	Description
`syn.interface.m_axi_max_widen_bitwidth`	uint default=0 (for Vivado IP flow) default=512 (for Vitis Kernel flow)	Allow the tool to automatically widen bursts on M-AXI interfaces up to the chosen bitwidth. Must be a power-of-two between 8 and 1024. Note that burst widening requires strong alignment properties (in addition to burst).
`syn.interface.m_axi_auto_max_ports`	bool default=false	If the option is false, all the M-AXI interfaces that are not explicitly bundled will be bundled into a single common interface, thus minimizing resource usage (single adapter). If the option is true, all the M-AXI interfaces that are not explicitly bundled will be mapped into individual interfaces, thus increasing the resource usage (multiple adapters).
`syn.interface.m_axi_alignment_byte_size`	uint default=1 (for Vivado IP flow) default=64 (for Vitis Kernel flow)	Assume top function pointers that are mapped to M-AXI interfaces are at least aligned to the provided width in byte (power of two). This can help automatic burst widening. Warning: behavior will be incorrect if the pointers are not actually aligned at runtime.
`syn.interface.m_axi_num_read_outstanding`	uint default=16	Default value for M-AXI num_read_outstanding interface parameter.
`syn.interface.m_axi_num_write_outstanding`	uint default=16	Default value for M-AXI num_write_outstanding interface parameter.
`syn.interface.m_axi_max_read_burst_length`	uint default=16	Default value for M-AXI max_read_burst_length interface parameter.
`syn.interface.m_axi_max_write_burst_length`	uint default=16	Default value for M-AXI max_write_burst_length interface parameter.

Stencil Optimizations

The stencil pattern algorithm is a computational pattern used extensively in the field of scientific computing, image processing, and numerical simulations.

- This algorithm updates the value of each element in an array by applying a predetermined pattern or "stencil," which dictates how to amalgamate the values of adjacent elements.
- For instance, within image processing contexts, a stencil computation could calculate a weighted average of a pixel's value with its immediate neighbors to achieve a blurring effect. Stencil computations are demanding in terms of memory bandwidth on hardware like FPGAs, as the non-sequential nature of stencil pixel locations necessitates numerous DDR memory reads, markedly prolonging computation times.

- To mitigate this, stencil pattern algorithms will be implemented using window and line buffer techniques. These techniques optimize data access patterns, reducing the need for multiple off-chip memory reads by caching relevant data in an on-chip memory.

These techniques can improve memory bandwidth utilization and increase the effectiveness of parallelization and pipelining.

Adapting algorithms to leverage line and window buffers can be time-intensive, requiring considerable code refactoring. Vitis HLS introduces stencil optimization/pragma, which can automatically implement the line buffer and window buffer and achieve the same performance.

```
for(int y=0; y<30; ++y)
    {
        for(int x=0; x<1000; ++x)
        {
#pragma HLS pipeline II=1
#pragma HLS array_stencil variable=src

            // Apply 2D filter to the pixel window
            int sum = 0;
            for(int row=0; row<FILTER_V_SIZE; row++)
            {

                for(int col=0; col<FILTER_H_SIZE; col++)
                {
                    unsigned char pixel;
                    int xoffset = (x+col-(FILTER_H_SIZE/2));
                    int yoffset = (y+row-(FILTER_V_SIZE/2));
                    // Deal with boundary conditions : clamp pixels to 0
when outside of image
                    if ( (xoffset<0) || (xoffset>=1000) || (yoffset<0) ||
(yoffset>=30) ) {
                        pixel = 0;
                    } else {
                        pixel = src[yoffset][xoffset];
                    }
                    sum += pixel*coeffs[row][col];
                }
            }
```

Examples of Recommended Coding Styles

As described in Synthesis Summary, Vitis HLS issues a report summarizing burst activities and also identifying burst failures. If bursts of variable lengths are done, then the report will mention that bursts of variable lengths were inferred. The compiler also provides burst messages that can be found in the compiler log, `vitis_hls.log`. These messages are issued before the scheduling step.

Simple Read/Write Burst Inference

The following example is the standard way of reading and writing to the DDR and inferring a read and write burst. The Vitis HLS compiler will report the following burst inferences for the example below:

```
INFO: [HLS 214-115] Burst read of variable length and bit width 32 has been
inferred on port 'gmem'
INFO: [HLS 214-115] Burst write of variable length and bit width 32 has
been inferred on port 'gmem' (./src/vadd.cpp:75:9).
```

The code for this example follows:

```
/****** BEGIN EXAMPLE *******/

#define DATA_SIZE 2048
// Define internal buffer max size
#define BURSTBUFFERSIZE 256

//TRIPCOUNT identifiers
const unsigned int c_min = 1;
const unsigned int c__max = BURSTBUFFERSIZE;
const unsigned int c_chunk_sz = DATA_SIZE;

extern "C" {
void vadd(int *a, int size, int inc_value) {
    // Map pointer a to AXI4-master interface for global memory access
#pragma HLS INTERFACE mode=m_axi port=a  offset=slave bundle=gmem
max_read_burst_length=256 max_write_burst_length=256
    // We also need to map a and return to a bundled axilite slave interface
#pragma HLS INTERFACE mode=s_axilite port=a bundle=control
#pragma HLS INTERFACE mode=s_axilite port=size bundle=control
#pragma HLS INTERFACE mode=s_axilite port=inc_value bundle=control
#pragma HLS INTERFACE mode=s_axilite port=return bundle=control

    int burstbuffer[BURSTBUFFERSIZE];

    // Per iteration of this loop perform BURSTBUFFERSIZE vector addition
    for (int i = 0; i < size; i += BURSTBUFFERSIZE) {
#pragma HLS LOOP_TRIPCOUNT min=c_min*c_min max=c_chunk_sz*c_chunk_sz/
(c_max*c_max)
        int chunk_size = BURSTBUFFERSIZE;
        //boundary checks
        if ((i + BURSTBUFFERSIZE) > size)
            chunk_size = size - i;

        // Use a for loop to create a burst access to memory
        // memcpy is not recommended
        // memcpy(burstbuffer, &a[i], chunk_size * sizeof(int));
        for (i=0; i < chunk_size; i++) {
            burstbuffer[i] = a[i];
        }

    // Calculate and write results to global memory, the sequential write
in a for loop can be
    // inferred as a memory burst access
    calc_write:
        for (int j = 0; j < chunk_size; j++) {
           #pragma HLS LOOP_TRIPCOUNT min=c_size_max max=c_chunk_sz
           #pragma HLS PIPELINE II=1
```

```
            burstbuffer[j] = burstbuffer[j] + inc_value;
            a[i + j] = burstbuffer[j];
        }
    }
}
```

Pipelining Between Bursts

The following example will infer bursts of length N:

```
for(int x=0; x < k; ++x) {
   int off = f(x);
   for(int i = 0; i < N; ++i) {
      #pragma HLS PIPELINE II=1
      ... = gmem[off + i];
   }
}
```

But notice that the outer loop is not pipelined. This means that while there is pipelining inside bursts, there won't be any pipelining between bursts.

To remedy this you can unroll the inner loop and pipeline the outer loop to get pipelining between bursts as well. The following example will still infer bursts of length N, but now there will also be pipelining between bursts leading to higher throughput:

```
for(int x=0; x < k; ++x) {
   #pragma HLS PIPELINE II=N
   int off = f(x);
   for(int i = 0; i < N; ++i) {
      #pragma HLS UNROLL
      ... = gmem[off + i];
   }
}
```

Accessing Row Data from a Two-Dimensional Array

The following is an example of reading/writing to/from a two dimensional array. Vitis HLS infers read and write bursts and issues the following messages:

```
INFO: [HLS 214-115] Burst read of length 256 and bit width 512 has been
inferred on port 'gmem' (./src/row_array_2d.cpp:43:5)
INFO: [HLS 214-115] Burst write of length 256 and bit width 512 has been
inferred on port 'gmem' (./src/row_array_2d.cpp:56:5)
```

Notice that a bit width of 512 is achieved in this example. This is more efficient than the 32 bit width achieved in the simple example above. Bursting wider bit widths is another way bursts can be optimized as discussed in Automatic Port Width Resizing.

The code for this example follows:

```
/****** BEGIN EXAMPLE *******/
// Parameters Description:
//         NUM_ROWS:            matrix height
//         WORD_PER_ROW:        number of words in a row
//         BLOCK_SIZE:          number of words in an array
#define NUM_ROWS   64
#define WORD_PER_ROW 64
#define BLOCK_SIZE (WORD_PER_ROW*NUM_ROWS)

// Default datatype is integer
typedef int DTYPE;
typedef hls::stream<DTYPE> my_data_fifo;

// Read data function: reads data from global memory
void read_data(DTYPE *inx, my_data_fifo &inFifo) {
read_loop_i:
    for (int i = 0; i < NUM_ROWS; ++i) {
    read_loop_jj:
        for (int jj = 0; jj < WORD_PER_ROW; ++jj) {
           #pragma HLS PIPELINE II=1
            inFifo << inx[WORD_PER_ROW * i + jj];
            ;
        }
    }
}

// Write data function - writes results to global memory
void write_data(DTYPE *outx, my_data_fifo &outFifo) {
write_loop_i:
    for (int i = 0; i < NUM_ROWS; ++i) {
    write_loop_jj:
        for (int jj = 0; jj < WORD_PER_ROW; ++jj) {
           #pragma HLS PIPELINE II=1
            outFifo >> outx[WORD_PER_ROW * i + jj];
        }
    }
}

// Compute function is pretty simple because this example is focused on
efficient
// memory access pattern.
void compute(my_data_fifo &inFifo, my_data_fifo &outFifo, int alpha) {
compute_loop_i:
    for (int i = 0; i < NUM_ROWS; ++i) {
    compute_loop_jj:
        for (int jj = 0; jj < WORD_PER_ROW; ++jj) {
           #pragma HLS PIPELINE II=1
            DTYPE inTmp;
            inFifo >> inTmp;
            DTYPE outTmp = inTmp * alpha;
            outFifo << outTmp;
        }
    }
}

extern "C" {
    void row_array_2d(DTYPE *inx, DTYPE *outx, int alpha) {
        // AXI master interface
    #pragma HLS INTERFACE mode=m_axi port = inx offset = slave bundle = gmem
    #pragma HLS INTERFACE mode=m_axi port = outx offset = slave bundle =
gmem
```

```
        // AXI slave interface
    #pragma HLS INTERFACE mode=s_axilite port = inx bundle = control
    #pragma HLS INTERFACE mode=s_axilite port = outx bundle = control
    #pragma HLS INTERFACE mode=s_axilite port = alpha bundle = control
    #pragma HLS INTERFACE mode=s_axilite port = return bundle = control

        my_data_fifo inFifo;
        // By default the FIFO depth is 2, user can change the depth by
using
        // #pragma HLS stream variable=inFifo depth=256
        my_data_fifo outFifo;

        // Dataflow enables task level pipelining, allowing functions and
loops to execute
        // concurrently. For more details please refer to UG902.
    #pragma HLS DATAFLOW
        // Read data from each row of 2D array
        read_data(inx, inFifo);
        // Do computation with the acquired data
        compute(inFifo, outFifo, alpha);
        // Write data to each row of 2D array
        write_data(outx, outFifo);
        return;
    }
}
```

Summary

Write code in such a way that bursting can be inferred. Ensure that none of the preconditions are violated. Bursting does not mean that you will get all your data in one shot – it is about merging the requests together into one request, but the data will arrive sequentially, one after another.

Burst length of 16 is ideal, but even burst lengths of 8 are enough. Bigger bursts have more latency while shorter bursts can be pipelined. Do not confuse bursting with pipelining, but note that bursts can be pipelined with other bursts.

If your bursts are of fixed length, you can unroll the inner loop where bursts are inferred and pipeline the outer loop. This will achieve the same burst length, but also pipelining between the bursts to enable higher throughput.

For greater throughput, focus on widening the interface up to 1024 bits rather than simply achieving longer bursts.

Bigger bursts have higher priority with the AXI interconnect. No dynamic arbitration is done inside the kernel.

You can have two `m_axi` ports connected to same DDR memory to model mutually exclusive access inside kernel, but the AXI interconnect outside the kernel will arbitrate competing requests.

One way to get around the out-of-order access restriction is to create your own buffer in block RAM, store the bursts in this buffer and then use this buffer to do out of order accesses. This is typically called a line buffer and is a common optimization used in video processing.

As another alternative, you can consider adding the CACHE pragma or directive as described in pragma HLS cache to improve performance of the `m_axi` interface.

Review the Burst Optimization section of the Synthesis Summary report to learn more about burst optimizations in the design, and missed burst opportunities. If automatic burst is not occurring in your design, you might want to use the `hls::burst_maxi` data type for manual burst, as described in Using Manual Burst.

Using Manual Burst

Burst transfers improve the throughput of the I/O of the kernel by reading or writing large chunks of data to the global memory. The larger the size of the burst, the higher the throughput, this metric is calculated as follows ((# of bytes transferred)* (kernel frequency)/(Time)). The maximum kernel interface bitwidth is 512 bits and if the kernel is compiled at 300 MHz, then it can theoretically achieve = (80-95% efficiency of the DDR)*(512* 300 MHz)/1 sec = ~17-19 GB/s for a DDR. As explained, Vitis HLS performs automatic burst optimizations which intelligently aggregates the memory accesses of the loops/functions from the user code and performs read/write of a particular size in a single burst request. However, burst transfer also has requirements that can sometimes be overburdening or difficult to meet, as discussed in Preconditions and Limitations of Burst Transfer.

In some cases, where autmatic burst access has failed, an efficient solution is to re-write the code or use manual burst. In such cases, if you are familiar with the AXI4 `m_axi` protocol, and understand hardware transaction modeling, you can implement manual burst transfers using the `hls::burst_maxi` class as described below. Refer to Vitis-HLS-Introductory-Examples/Interface/Memory/manual_burst on GitHub for examples of these concepts. Another solution might be to use cache memory in the AXI4 interface using the CACHE pragma or directive.

hls::burst_maxi Class

The `hls::burst_maxi` class provides a mechanism to perform read/write access to the DDR memory. These methods will translate the class methods usage behavior into respective AXI4 protocol and send and receive requests on the AXI4 bus signals - AW, AR, WDATA, BVALID, RDATA. These methods control the burst behavior of the HLS scheduler. The adapter, which receives the commands from the scheduler, is responsible for sending the data to the DDR memory. These requests will adhere to the user specified INTERFACE pragma options, such as `max_read_burst_length` and `max_write_burst_length`. The class methods should only be used in the kernel code, and not in the test bench (except for the class constructor as described below).

- Constructors:
  - ```
    burst_maxi(const burst_maxi &B) : Ptr(B.Ptr) {}
    ```
  - ```
    burst_maxi(T *p) : Ptr(p) {}
    ```

> **IMPORTANT!** *The HLS design and test bench must be in different files, because the constructor `burst_maxi(T *p)` is only available in C-simulation model.*

- Read Methods:
  - `void read_request(size_t offset, size_t len);`

    This method is used to perform a read request to the `m_axi` adapter. The function returns immediately if the read request queue inside `m_axi` adapter is not full, otherwise it waits until space becomes available.

    - `offset`: Specify the memory offset from which to read the data
    - `len`: Specify the scheduler burst length. This burst length is sent to the adapter, which can then convert it to the standard AXI AMBA protocol
  - `T read();`

    This method is used to transfer the data from the `m_axi` adapter to the scheduler FIFO. If the data is not available, `read()` will be blocking. The `read()` method should be called `len` number of times, as specified in the `read_request()`.
- Write Methods:
  - `void write_request(size_t offset, size_t len);`

    This method is used to perform a write request to the `m_axi` adapter. The function returns immediately if the write request queue inside `m_axi` adapter is not full.

    - `offset`: Specify the memory offset into which the data should be written
    - `len`: Specify the scheduler burst length. This burst length is sent to the adapter, which can then convert it to the standard AXI AMBA protocol
  - `void write(const T &val, ap_int<sizeof(T)> byteenable_mask = -1);`

    This method is used to transfer data from the internal buffer of the scheduler to the `m_axi` adapter. It blocks if the internal write buffer is full. The byteenable_mask is used to enable the bytes in the WDATA. By default it will enable all the bytes of the transfer. The `write()` method should be called `len` number of times, as specified in the `write_request()`.
  - `void write_response();`

    This method blocks until all write responses are back from the global memory. This method should be called the same number of times as `write_request()`.

Using Manual Burst in HLS Design

In the HLS design, when you find that automatic burst transfers are not working as desired, and you cannot optimize the design as needed, you can implement the read and write transactions using the `hls::burst_maxi` object. In this case, you will need to modify your code to replace the original pointer argument with `burst_maxi` as a function argument. These arguments must be accessed by the explicit `read` and `write` methods of the `burst_maxi` class, as shown in the following examples.

The following shows an original code sample, which uses a pointer argument to read data from global memory.

```
void dut(int *A) {
  for (int i = 0; i < 64; i++) {
  #pragma pipeline II=1
      ... = A[i]
  }
}
```

In the modified code below, the pointer is replaced with the `hls::burst_maxi<>` class objects and methods. In the example, the HLS scheduler puts 4 requests of `len` 16 from port `A` to the `m_axi` adapter. The Adapter stores them inside a FIFO and whenever the AW/AR bus is available it will send the request to the global memory. In the 64 loop iterations, the `read()` command issues a blocking call that will wait for the data to come back from the global memory. After the data becomes available the HLS scheduler will read it from the `m_axi` adapter FIFO.

```
#include "hls_burst_maxi.h"
void dut(hls::burst_maxi<int> A) {
  // Issue 4 burst requests
  A.read_request(0, 16); // request 16 elements, starting from A[0]
  A.read_request(128, 16); // request 16 elements, starting from A[128]
  A.read_request(256, 16); // request 16 elements, starting from A[256]
  A.read_request(384, 16); // request 16 elements, starting from A[384]
  for (int i = 0; i < 64; i++) {
  #pragma pipeline II=1
      ... = A.read(); // Read the requested data
  }
}
```

In example 2 below, the HLS scheduler/kernel puts 2 requests from port A to the adapter, the first request of `len` 2, and the second request of `len` 1, for a total of 2 write requests. It then issues corresponding, because the total burst length is 3 write commands. The Adapter stores these requests inside a FIFO and whenever the AW, W bus is available it will send the request and data to the global memory. Finally, two `write_response` commands are used, to await response for the two `write_requests`.

```
void trf(hls::burst_maxi<int> A) {
  A.write_request(0, 2);
  A.write(x); // write A[0]
  A.write_request(10, 1);
  A.write(x, 2); // write A[1] with byte enable 0010
  A.write(x); // write A[10]
  A.write_response(); // response of write_request(0, 2)
  A.write_response(); // response of write_request(10, 1)
}
```

Using Manual Burst in C-Simulation

Manual burst supports read_request() and read() in different dataflow processes to enable the read_request() and read() in parallel (see example below). Similarly, support for write_request(), write() and write_response() is also available.

```
void foo(hls::burst_maxi<int> a) {
  for (int i = 0; i < 10; i++) {
  #pragma HLS pipeline
    a.read_request(i * 16, 10);
  }
}

void bar(hls::burst_maxi<int> a, int *b) {
  for (int i = 0; i < 100; i++) {
  #pragma HLS pipeline
    b[i] = a.read();
  }
}
void test(hls::burst_maxi<int> a, int b[1000]) {
#pragma HLS interface m_axi port=a depth=1000
#pragma HLS dataflow
  foo(a);
  bar(a, b);
}
```

You can pass a regular array to the top function, and the array will be transformed to `hls::burst_maxi` automatically by the constructor.

IMPORTANT! *The HLS design and test bench must be in different files, because the `burst_maxi(T *p)` constructor is only valid for use in C simulation model.*

```
#include "hls_burst_maxi.h"
void dut(hls::burst_maxi<int> A);

int main() {
  int Array[1000];
  dut(Array);
  ......
}
```

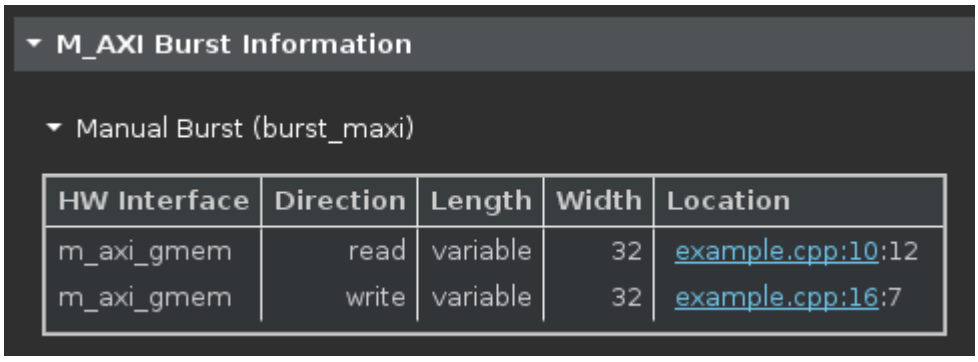
Using Manual Burst to Optimize Performance

Vitis HLS characterizes two types of burst behaviors: pipeline burst, and sequential burst.

- **Pipeline Burst:** Pipeline Burst improves throughput by reading or writing the maximum amount of data in a single request. The compiler infers pipeline burst if the `read_request`, `write_request` and `write_response` calls are outside the loop, as shown in the following code example. In the below example the size is a variable that is sent from the test bench.

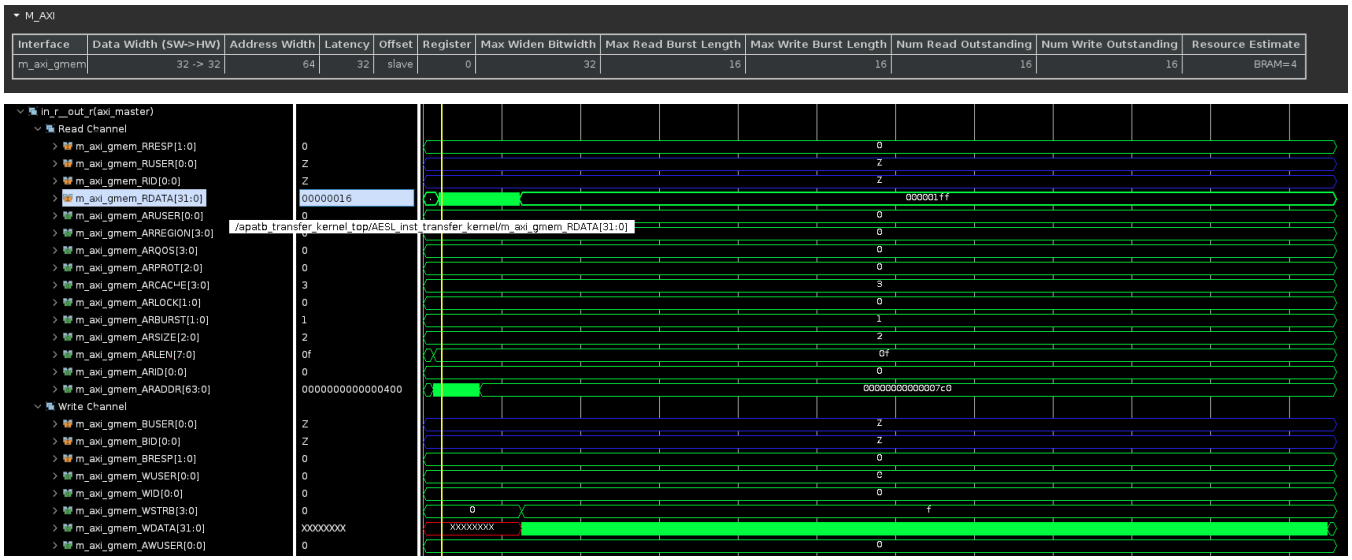
```
9  int buf[8192];
10  in.read_request(0, size);
11  for (int i = 0; i < size; i++) {
12  #pragma HLS PIPELINE II=1
13     buf[i] = in.read();
14     out.write_request(0, size*NT);
17     for (int i = 0; i < NT; i++) {
19        for (int j = 0; j < size; j++) {
20        #pragma HLS PIPELINE II=1
21           int a = buf[j];
22           out.write(a);
23  }
24 }
25 out.write_response();
```

Figure 69: **Synthesis Results**



As you can see from the preceding figure, the tool has inferred the burst from the user code and length is mentioned as variable at compile time.

Figure 70: **Performance Benefits**



During the runtime the HLS compiler sends a burst request of `length = size` and the adapter will partition them into the user-specified `burst_length` pragma option. In this case the default burst length is set to 16, which is used in the `ARlen` and `AWlen` channels. The read/write channel achieved maximum throughput because there are no bubbles during the transfer.

Figure 71: **Co-sim Results**

Modules & Loops	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
transfer_kernel				5735	5735	5735
transfer_kernel_Pipeline_VITIS_LOOP_11_1				517	517	517
transfer_kernel_Pipeline_VITIS_LOOP_18_2_VITIS_LOOP_19_3				5122	5122	5122

- **Sequential Burst:**

  This burst is a sequential burst of smaller data sizes, where the read requests, write requests and write responses are inside the loop body as shown in the below snippet. The drawback of the sequential burst is that the future request (i+1) depends on the previous request (i) to finish because it is waiting for the read request, write request and write response to complete, this will cause gaps between requests. Sequential burst is not as effective as pipeline burst because it is reading or writing a small data size multiple times to compensate for the loop bounds. Although this will limit the improvement to throughput, sequential burst is still better than no burst.

```
  void transfer_kernel(hls::burst_maxi<int> in,hls::burst_maxi<int> out,
const int size )
{
  #pragma HLS INTERFACE m_axi port=in depth=512 latency=32 offset=slave
  #pragma HLS INTERFACE m_axi port=out depth=5120 offset=slave latency=32
```

```
        int buf[8192];


        for (int i = 0; i < size; i++) {
             in.read_request(i, 1);
        #pragma HLS PIPELINE II=1
            buf[i] = in.read();
        }



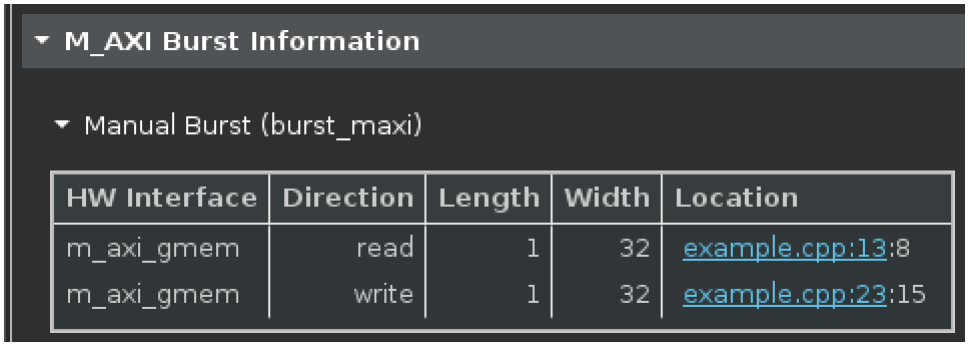
        for (int i = 0; i < NT; i++) {
            for (int j = 0; j < size; j++) {
                out.write_request(j, 1);
#pragma HLS PIPELINE II=1
                int a = buf[j];
                out.write(a);
                out.write_response();

            }

        }

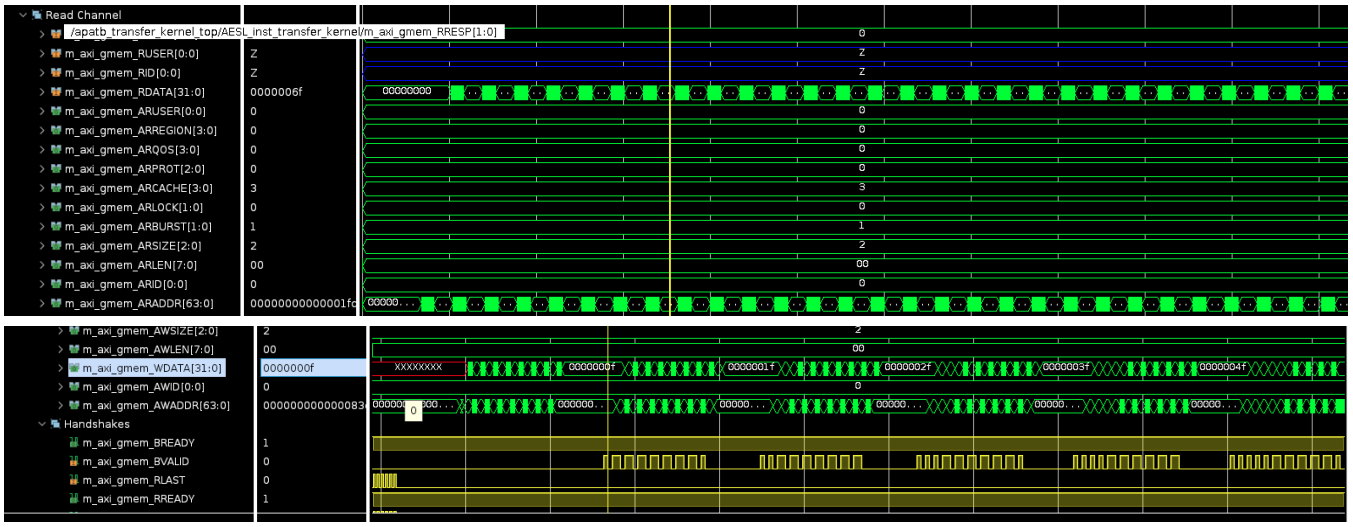
    }
```

Figure 72: **Synthesis Results**



As you can see from the report sample above, the tool achieved a burst of length =1.

Figure 73: **Performance Impacts**



The read/write loop R/WDATA channel has gaps equal to read/write latency, as discussed in AXI4 Master Interface. For the read channel, the loop waits for all the read data to come back from the global memory. For the write channel, the innermost loop waits for the response (BVALID) to come back from the global memory. This results in performance degradation. The co-sim results also show that a 2x degradation in performance for this burst semantics.

Figure 74: **Performance Estimates**



Features and Limitations

1. If the `m_axi` element is a struct:
   - The struct will be packed into a wide int. Disaggregation of the struct is not allowed.
   - The size of struct must be a power-of-2, and should not exceed 1024 bits or the max width specified by the `config_interface -m_axi_max_bitwidth` command.
2. ARRAY_PARTITION and ARRAY_RESHAPE of burst_maxi ports is not allowed.
3. You can apply the INTERFACE pragma or directive to `hls::burst_maxi`, defining an `m_axi` interface. If the `burst_maxi` port is bundled with other ports, all ports in this bundle must be `hls::burst_maxi` and must have the same element type.

```
void dut(hls::burst_maxi<int> A, hls::burst_maxi<int> B, int *C,
hls::burst_maxi<short> D) {
  #pragma HLS interface m_axi port=A offset=slave bundle=gmem // OK
  #pragma HLS interface m_axi port=B offset=slave bundle=gmem // OK
  #pragma HLS interface m_axi port=C offset=slave bundle=gmem // Bad.
```

```
C must also be hls::burst_maxi type, because it shares the same bundle
'gmem' with A and B
  #pragma HLS interface m_axi port=D offset=slave bundle=gmem  // Bad.
D should have 'int' element type,  because it shares the same bundle
'gmem' with A and B
}
```

4. You can use the INTERFACE pragma or directive to specify the `num_read_outstanding` and `num_write_outstanding`, and the `max_read_burst_length` and `max_write_burst_length` to define the size of the internal buffer of the `m_axi` adapter.

```
void dut(hls::burst_maxi<int> A) {
  #pragma HLS interface m_axi
port=A num_read_outstanding=32 num_write_outstanding=32
max_read_burst_length=16 max_write_burst_length=16
}
```

5. The INTERFACE pragma or directive `max_widen_bitwidth` is not supported, because HLS will not change the bit width of `hls::burst_maxi` ports.
6. You must make a `read_request` before `read`, or `write_request` before `write`:

```
void dut(hls::burst_maxi<int> A) {
  ... = A.read();  // Bad because read() before read_request(). You can
catch this error in C-sim.
  A.read_request(0, 1);
}
```

7. If the address and life time of the read group (`read_request()` > `read()`) and write group (`write_request()` > `write()` > `write_response()`) overlap, the tool cannot guarantee the access order. C-simulation will report an error.

```
void dut(hls::burst_maxi<int> A) {
  A.write_request(0, 1);
  A.write(x);
  A.read_request(0, 1);
  ... = A.read();  // What value is read? It is undefined. It could be
original A[0] or updated A[0].
  A.write_response();
}

void dut(hls::burst_maxi<int> A) {
  A.write_request(0, 1);
  A.write(x);
  A.write_response();
  A.read_request(0, 1);
  ... = A.read();  // this will read the updated A[0].
}
```

8. If multiple `hls::burst_maxi` ports are bundled to same `m_axi` adapter and their transaction lifetimes overlap, the behavior is unexpected.

```
void dut(hls::burst_maxi<int> A, hls::burst_maxi<int> B) {
    #pragma HLS INTERFACE m_axi port=A bundle=gmem depth = 10
    #pragma HLS INTERFACE m_axi port=B bundle=gmem depth = 10
    A.read_request(0, 10);
    B.read_request(0, 10);

    for (int i = 0; i < 10; i++) {
```

```
        #pragma HLS pipeline II=1
        …… = A.read(); // get value of A[0], A[2], A[4] …
        …… = B.read();  // get value of A[1], A[3], A[5] …
    }
}
```

9. Read or write requests and read or writes in different dataflow process are not supported. Dataflow checker will report an error: `multiple writes in different dataflow processes are not allowed`.

   For example:

```
void transfer(hls::burst_maxi<int> A)  {
#pragma HLS dataflow
   IssueRequests(A); // issue multiple wirte_request() of A
   Write(A); // multiple writes to A
   GetResponse(A); // write_response() of  A
}
```

Potential Pitfalls

The following are some concerns you must be aware of when implementing manual burst techniques:

- Deadlock: Improper use of manual burst can lead to deadlocks.

  Too many `read_requests` before `read()` commands will cause deadlock because the `read_request` loop will push the request into the read requests FIFO, and this FIFO will only be emptied after the read from the global memory is completed. The job of the `read()` command is to read the data from the adapter FIFO and mark the request done, after which the `read_request` will be popped from the FIFO and a new request can be pushed onto it.

```
//reads/writes. will deadlock if N is larger
for (i = 0; i < N; i++)
 {   A.read_request(i * 128, 16);}
for (i = 0; i < 16 *N; i++) {  … = A.read();}


for (int i = 0; i < N; i++) {
    p.write_request(i * 128, 16);
  }

  for (int i = 0; i < N * 16; i++) {
    p.write(i);
  }

  for (int i = 0; i < N; i++) {
    p.write_response();
  }
```

  In the example above, if N is large then the `read_request` and read FIFO will be full as it tends to N/2. The read request loop would not finish, and the read command loop would not start, which results in deadlock.

  Note: This is case also true for `write_request()` and `write()` commands.

- AXI protocol violation: There should be an equal number of write requests and write responses. An unequal number of requests and responses would lead to AXI protocol violation

AXI Performance Case Study

Introduction

The objective of the case study is to show a step-by-step optimization to improve the throughput of the read/write loops/functions using HLS metrics. These optimizations will improve the kernel time and throughput of the system by performing efficient data transfers from global memory to the kernel. The `transfer_kernel` example below performs a DDR simple read/write (of variable size and NUM_ITERATIONS).



TIP: *The host code, which is not shown, only transfers the data and enqueues the kernel in an in-order queue.*

```
1 #include "config.h"
 2 #include "assert.h"
 3 extern "C" {
 4    void transfer_kernel(wd* in,wd* out, const int size, const int iter )
{
 5 ...
 6         wd buf[256];
 7         int off = (size/16);
 8
 9         read_loop: for (int i = 0; i <off; i++)
10         {
11            buf[i] = in[i];
12         }
13
14     write_loop: L1: for (int i = 0; i < iter; i++) {
15         L2: for (int j = 0; j <off; j++) {
16         #pragma HLS PIPELINE II=1
17            out[j+off*i] = buf[j];
18            }
19         }
20 ...
21    }
22 }
```

This case study is divided into 4 steps:

1. Baseline kernel run time with port width set to 512-bit width
2. Improve performance by changing latency parameter
3. Improve the auto burst inference of the write loop.
4. No further improvements using multiple ports and number write outstanding

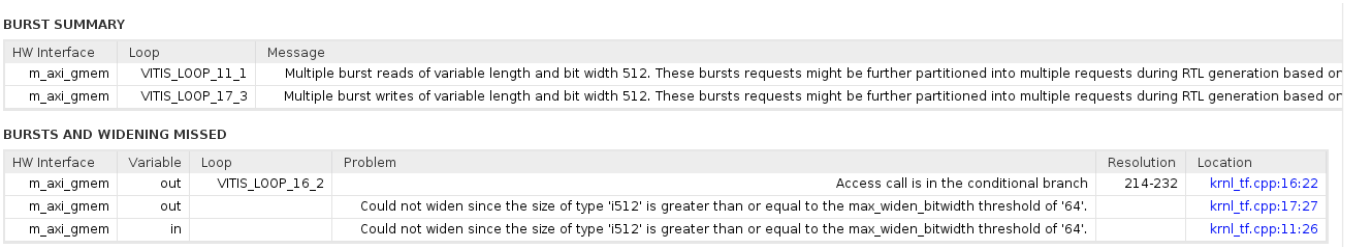
Step 1: Baseline the Kernel with 512-bit Port Width

Baseline the kernel time using the default settings. During this run, the auto burst inferences the following for the read and write loops:

- The Read loop achieves the pipeline burst since the tool can predict the consecutive memory access pattern. So the pipelined requests to read from the DDR of variable size.
- The Write outer loop, L1, gets sequential burst because the compiler iterates over all the combinations and identifies that since the size is unknown at compile-time, it inserts an if condition in the L1 loop before the start of the L2 loop. At the same time, the inner-most loop - L2 achieves pipeline burst. The L2 loop requests a write request of variable size, while L1 waits for all the data of L2 Loop to come back from the DDR to start the next iteration of L1.

After building and running the application, the performance can be evaluated using the Vitis Analyzer tool to view the reports generated by the build process or the run summary. Review the Burst Summary available in the Synthesis Report from Vitis HLS. It confirms the success and failures of the burst for the Read loop and Write loops.

Figure 75: **Synthesis Report - Burst Summary**



BURST SUMMARY

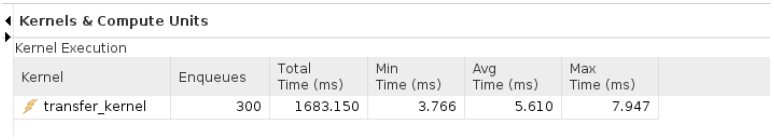
HW Interface	Loop	Message
m_axi_gmem	VITIS_LOOP_11_1	Multiple burst reads of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based or
m_axi_gmem	VITIS_LOOP_17_3	Multiple burst writes of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based or

BURSTS AND WIDENING MISSED

HW Interface	Variable	Loop	Problem	Resolution	Location
m_axi_gmem	out	VITIS_LOOP_16_2	Access call is in the conditional branch	214-232	krnl_tf.cpp:16:22
m_axi_gmem	out		Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:17:27
m_axi_gmem	in		Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:11:26

In Vitis Analyzer, the Profile Summary and Timeline Trace reports are also useful tools to analyze the performance of the FPGA-accelerated application. In the Profile Summary the **Kernels & Compute Unit: Kernel Execution** reports the total time required by the `transfer_kernel` in the baseline build.

Figure 76: **Profile Summary - Kernel Execution**



Kernels & Compute Units

Kernel Execution

Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)
transfer_kernel	300	1683.150	3.766	5.610	7.947

Step 2: Improve Performance Latency

Vitis HLS uses the default latency of 64 kernel cycles, which in some cases may be too high. The latency depends on the system characteristics. For this example, the latency is reduced from the default to 21 kernel cycles. The code can be changed to specify the latency using the INTERFACE pragma or directive as shown in the following example:

```
1 #include "config.h"
 2 #include "assert.h"
 3 extern "C" {
 4     void transfer_kernel(wd* in,wd* out, const int size, const int iter )
{
 5     #pragma HLS INTERFACE m_axi port=in0_index offset=slave latency=21
 6     #pragma HLS INTERFACE m_axi port=out offset=slave latency=21

 7 ...
```

Build and run the application and use Vitis Analyzer to review the reports generated by the build process or the run summary. Review the Synthesis Report from Vitis HLS, and examine the **HW Interface** table to see the specified latency has been applied.

Figure 77: **Synthesis Report - HW Interface**

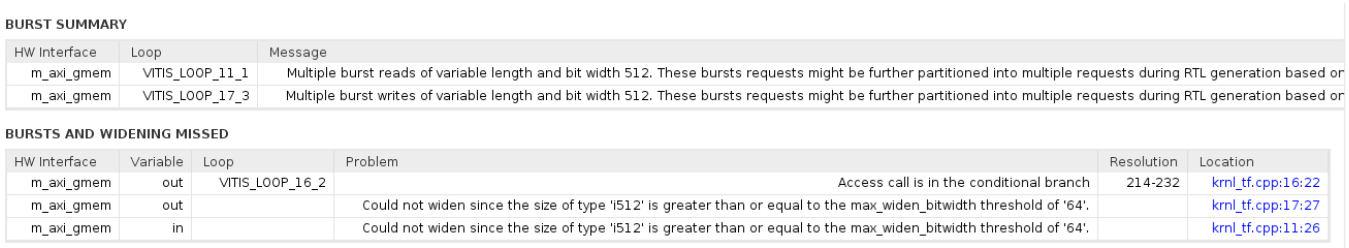


M_AXI

Interface	Data Width (SW->HW)	Address Width	Latency	Offset	Register	Max Widen Bitwidth	Max Read Burst Length	Max Write Burst Length	Num Read Outstanding
m_axi_gmem	512 -> 512	64	21	slave	0	512	16	16	

Review the **Burst Summary** to examine the success or failures of that process.

Figure 78: **Synthesis Report - Burst Summary 2**



BURST SUMMARY

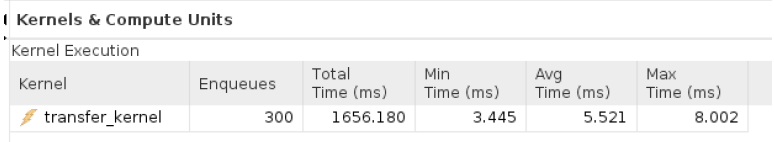
HW Interface	Loop	Message
m_axi_gmem	VITIS_LOOP_11_1	Multiple burst reads of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based or
m_axi_gmem	VITIS_LOOP_17_3	Multiple burst writes of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based or

BURSTS AND WIDENING MISSED

HW Interface	Variable	Loop	Problem	Resolution	Location
m_axi_gmem	out	VITIS_LOOP_16_2	Access call is in the conditional branch	214-232	krnl_tf.cpp:16:22
m_axi_gmem	out		Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:17:27
m_axi_gmem	in		Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:11:26

Examine the Kernel Execution in the Profile Summary report, and notice the performance improvement due to setting the latency for the interface.

Figure 79: **Profile Summary - Kernel Execution 2**



Kernels & Compute Units

Kernel Execution

Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)
transfer_kernel	300	1656.180	3.445	5.521	8.002

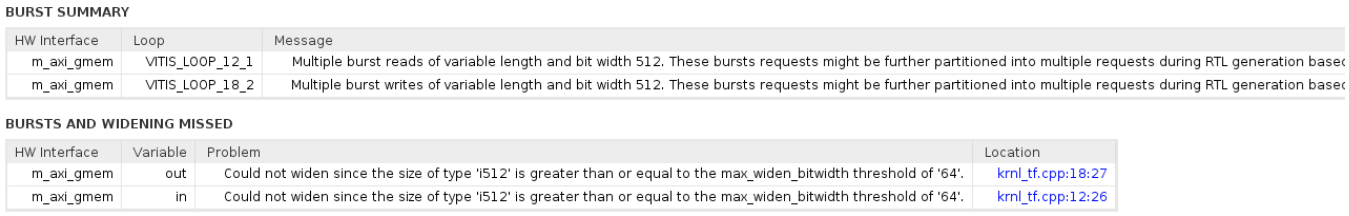
Step 3: Improve the Automatic Burst Inference of the Write Loop

The compiler is pessimistic in auto burst inference because size and loop trip counts are unknown at compile time. You can modify the code to help the compiler infer pipelined burst, as shown below.

```
1 #include "config.h"
  2 #include "assert.h"
  3 extern "C" {
  4     void transfer_kernel(wd* in,wd* out, const int size, const int
iter ) {
  5     #pragma HLS INTERFACE m_axi port=in offset=slave latency=21
  6     #pragma HLS INTERFACE m_axi port=out offset=slave latency=21
  7
  8        int k=0;
  9        wd buf[256];
 10        int off = (size/16);
 11
 12        read_loop: for (int i = 0; i <off; i++)
 13        {
 14           buf[i] = in[i];
 15        }
 16
 17        write_loop: for (int j = 0; j <off*iter; j++) {
 18        #pragma HLS PIPELINE II=1
 19           out[k++] = buf[j%off];
 20        }
 21     }
 22 }
```

Build and run the application and use Vitis Analyzer to review the reports generated by the build process or the run summary. The Synthesis Report confirms that the burst hints to the compiler fixed the sequential burst of the write loop. The **Burst and Widening Missed** messages are related to widening ports to 512 bits. Since this example already has a 512 port width, it can be ignored. If the width isn't 512-bi in your code, you might need to focus on resolving these messages.

Figure 80: **Synthesis Report - Burst Summary 3**



BURST SUMMARY

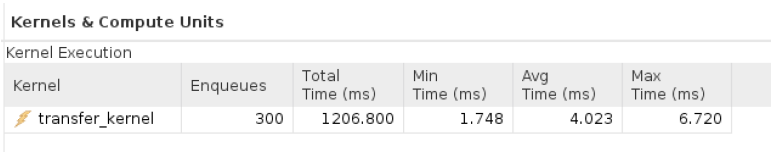
HW Interface	Loop	Message
m_axi_gmem	VITIS_LOOP_12_1	Multiple burst reads of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based
m_axi_gmem	VITIS_LOOP_18_2	Multiple burst writes of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based

BURSTS AND WIDENING MISSED

HW Interface	Variable	Problem	Location
m_axi_gmem	out	Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.	krnl_tf.cpp:18:27
m_axi_gmem	in	Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.	krnl_tf.cpp:12:26

Examine the Kernel Execution in the Profile Summary report, and notice the performance improvement due to the latency change from Step 2, and the pipeline burst for the write loop in the current step.

Figure 81: **Profile Summary - Kernel Execution 3**



Kernels & Compute Units

Kernel Execution

Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)
transfer_kernel	300	1206.800	1.748	4.023	6.720

Summary

There are no further improvements that can be made from the Vitis HLS interface metrics. The case study example does not have concurrent read or write, so targeting multiple ports will not help in this case. In this example the tool has achieved pipeline burst for the maximum throughput, so the number of outstanding reads and writes can also be ignored. No further improvements can be confirmed from the kernel time.

As seen in the case study, implementing efficient load-store functions is dependent on the HLS interface metrics of port width, burst access, latency, multiple ports, and the number of outstanding reads and writes. AMD recommends the following guidelines for improving your system performance:

- Port width: Maximize the port width of the interface, i.e., the bit-width of each AXI port, by using `hls::vector` or `ap_(u)int<512>` as the data type of the port.
- Multiple ports: Analyze the concurrent memory reads/writes and have a dedicated/independent port for concurrent accesses.
- Pipeline burst: The AXI latency parameter does not have an impact on pipelined burst, the user is advised to write code to achieve the pipelined burst which can significantly improve the performance.
- Sequential burst: The AXI latency parameter has a significant impact on sequential burst, decreasing the latency number from the default latency of the tool will improve the performance.
- Num outstanding: In most of the cases of burst length >=16, the default num outstanding should be sufficient. For a burst of size less than 16, AMD recommends doubling the size of the num outstanding from the default(=16).
- Data Re-ordering: Achieving pipelined burst is always recommended, but at times because of the memory access pattern compiler can achieve only a sequential burst. In order to improve the performance, the developer can also consider different ways of storing the data in memory. For instance, accessing data in DRAM in a column-major fashion can be very inefficient. Rather than implementing a dedicated data-mover in the kernel, it may be better to transpose the data in SW and store in row-major order instead which will greatly simply HW access patterns.

Managing Area and Hardware Resources

During synthesis, Vitis HLS performs the following basic tasks:

- Elaborates the C, C++ source code into an internal database containing the operators in the C code, such as additions, multiplications, array reads, and writes.
- Maps the operators onto implementations in the hardware.

  Implementations are the specific hardware components used to create the design (such as adders, multipliers, pipelined multipliers, and block RAM).

Commands, pragmas and directives provide control over each of these steps, allowing you to control the hardware implementation at a fine level of granularity.

Limiting the Number of Operators

Explicitly limiting the number of operators to reduce area may be required in some cases: the default operation of Vitis HLS is to first maximize performance. Limiting the number of operators in a design is a useful technique to reduce the area of the design: it helps reduce area by forcing the sharing of operations. However, this might cause a decline in performance.

The ALLOCATION directive allows you to limit how many operators are used in a design. For example, if a design called `foo` has 317 multiplications but the FPGA only has 256 multiplier resources (DSP macrocells). The ALLOCATION pragma shown below directs Vitis HLS to create a design with a maximum of 256 multiplication (`mul`) operators:

```
dout_t array_arith (dio_t d[317]) {
 static int acc;
 int i;
#pragma HLS ALLOCATION instances=fmul limit=256 operation

 for (i=0;i<317;i++) {
#pragma HLS UNROLL
 acc += acc * d[i];
 }
 rerun acc;
}
```

***Note*:** If you specify an ALLOCATION limit that is greater than needed, Vitis HLS attempts to use the number of resources specified by the limit, or the maximum necessary, which reduces the amount of sharing.

You can use the `type` option to specify if the ALLOCATION directives limits operations, implementations, or functions. The following table lists all the operations that can be controlled using the ALLOCATION directive.

***Note*:** The operations listed below are supported by the ALLOCATION pragma or directive. The BIND_OP pragma or directive supports a subset of operators as described in the command syntax.

Table 22: **Vitis HLS Operators**

Operator	Description
add	Integer Addition
ashr	Arithmetic Shift-Right
dadd	Double-precision floating-point addition
dcmp	Double-precision floating-point comparison
ddiv	Double-precision floating-point division
dmul	Double-precision floating-point multiplication
drecip	Double-precision floating-point reciprocal
drem	Double-precision floating-point remainder
drsqrt	Double-precision floating-point reciprocal square root
dsub	Double-precision floating-point subtraction
dsqrt	Double-precision floating-point square root
fadd	Single-precision floating-point addition
fcmp	Single-precision floating-point comparison
fdiv	Single-precision floating-point division
fmul	Single-precision floating-point multiplication
frecip	Single-precision floating-point reciprocal
frem	Single-precision floating point remainder
frsqrt	Single-precision floating-point reciprocal square root
fsub	Single-precision floating-point subtraction
fsqrt	Single-precision floating-point square root
icmp	Integer Compare
lshr	Logical Shift-Right
mul	Multiplication
sdiv	Signed Divider
shl	Shift-Left
srem	Signed Remainder
sub	Subtraction
udiv	Unsigned Division
urem	Unsigned Remainder

Controlling Hardware Implementation

When synthesis is performed, Vitis HLS uses the timing constraints specified by the clock, the delays specified by the target device together with any directives specified by you, to determine which hardware implementations to use for various operators in the code. For example, to implement a multiplier operation, Vitis HLS could use the combinational multiplier or use a pipeline multiplier.

The implementations which are mapped to operators during synthesis can be limited by specifying the ALLOCATION pragma or directive, in the same manner as the operators. Instead of limiting the total number of multiplication operations, you can choose to limit the number of combinational multipliers, forcing any remaining multiplications to be performed using pipelined multipliers (or vice versa).

The BIND_OP or BIND_STORAGE pragmas or directives are used to explicitly specify which implementations to use for specific operations or storage types. The following command informs Vitis HLS to use a two-stage pipelined multiplier using fabric logic for variable `c`. It is left to Vitis HLS which implementation to use for variable `d`.

```
int foo (int a, int b) {
 int c, d;
#pragma HLS BIND_OP variable=c op=mul impl=fabric latency=2
 c = a*b;
 d = a*c;

 return d;
}
```

In the following example, the BIND_OP pragma specifies that the add operation for variable `temp` is implemented using the `dsp` implementation. This ensures that the operation is implemented using a DSP module primitive in the final design. By default, add operations are implemented using LUTs.

```
void apint_arith(dinA_t  inA, dinB_t  inB,
          dout1_t *out1
  ) {

 dout2_t temp;
#pragma HLS BIND_OP variable=temp op=add impl=dsp

 temp = inB + inA;
 *out1 = temp;

}
```

Refer to the BIND_OP or BIND_STORAGE pragmas or directives to obtain details on the implementations available for assignment to operations or storage types.

In the following example, the BIND_OP pragma specifies the multiplication for `out1` is implemented with a 3-stage pipelined multiplier.

```
void foo(...) {
#pragma HLS BIND_OP variable=out1 op=mul latency=3

 // Basic arithmetic operations
 *out1 = inA * inB;
 *out2 = inB + inA;
 *out3 = inC / inA;
 *out4 = inD % inA;

}
```

If the assignment specifies multiple identical operators, the code must be modified to ensure there is a single variable for each operator to be controlled. For example, in the following code, if only the first multiplication (`inA * inB`) is to be implemented with a pipelined multiplier:

```
*out1 = inA * inB * inC;
```

The code should be changed to the following with the pragma specified on the `Result_tmp` variable:

```
#pragma HLS BIND_OP variable=Result_tmp op=mul latency=3
 Result_tmp = inA * inB;
 *out1 = Result_tmp * inC;
```

DSP Multi-Operation Matching

The HLS compiler attempts to identify Add-Multiply, Multiply-Add, and AMA operations (as well as Subtract-Multiply, Multiply-Subtract operations) to match their assignment to DSPs on the physical device. The HLS compiler uses the following process to match operations to DSPs:

- Inline functions as needed
- Extract common sub-expressions
- Match fanout-free `add->mul`, `mul->add`, and `add->mul->add` (where `add` can also be `sub`) code fragments meeting bitwidth limitations of DSPs
- Schedule
- Select implementation in DSP or LUT guided by the BIND_OP pragma

The ability of the HLS compiler to match operations or expressions to DSP implementation depends on the following:

1. Datatypes and bitwidths
2. Coding style and expressions
3. The use of BIND_OP and other pragmas

Datatypes and Bitwidths

DSP matching is limited by design restrictions of DSP48 and DSP58 on the target device. Refer to *UltraScale Architecture DSP Slice* (UG579) for specific limitations on DSP48, and *Versal Adaptive SoC DSP Engine Architecture Manual* (AM004) for limits on DSP58.

As an example, in Versal devices the following are the port widths for DSP58:

- B 24 bits
- A 34 bits
- D 27 bits

- C 58 bits
- P 58 bits

Coding Style

The style of writing the code matters:

- Bitwidths must be small enough to fit the DSP, including intermediate results
- No fanout is allowed, even as a result of common sub-expression elimination. For example the following code will not be matched for DSP assignment due to the extraction of the sub-expression `a*b`:

```
f1 = a*b+c;
f2= a*b+d;
```



TIP: *To avoid this problem use non-inlined functions containing exactly the expressions to be matched.*

- Non-inlined functions with specified latency, for example MULADD with latency 2, cannot be matched to an accumulator with II=1. The reason is that both multiplier input and the adder input must be ready 2 cycles before the accumulation output is produced. In that case, the MULADD or MAC function (if any) must be inlined.

BIND_OP and Other Pragmas

For DSP matching, some pragmas or directives can prevent the matching of code for implementation in DSPs, while other pragmas or directives can help. In the following code example, the BIND_OP pragma actually prevents the assignment of the MULADD expression to DSP. By assigning the `mul` operation it prevents the HLS compiler from recognizing and assigning the MULADD expression. Removing the pragma in this case will let the compiler make the assignment.

```
static const int NFRAC = 14;
typedef ap_fixed<3+NFRAC,  3, AP_TRN, AP_WRAP> DATA_T;
…
ACC_T MAC( DATA_T din0, COEF_T coef, ACC_T  acc ) {
    acc = din0 * coef + acc;
#pragma HLS BIND_OP variable=acc op=mul impl=dsp latency=2
    return acc;
}
void process ( DATA_T din, DATA_T* dout0) {
#pragma HLS pipeline II=1
…
 LOOP_MAC: for (int i=0; i<l_COEFF; i++) {
    acc = MAC (sr[tdl_index], coeff[i], acc);
    tdl_index = tdl_index-1;
}
```

The PIPELINE pragma causes the loop to be unrolled with expression balancing that also prevents DSP matching. In this case, adding EXPRESSION_BALANCE OFF to disable expression balancing will help the HLS compiler recognize the multi-operation expressions for DSP matching. The reworked code example below will have improved DSP matching.

```
static const int NFRAC = 14;
typedef ap_fixed<3+NFRAC,  3, AP_TRN, AP_WRAP> DATA_T;
…
ACC_T MAC( DATA_T din0, COEF_T coef, ACC_T  acc ) {
    acc = din0 * coef + acc;
    return acc;
}
void process ( DATA_T din, DATA_T* dout0) {
#pragma HLS pipeline II=1
#pragma HLS expression_balance off
…
 LOOP_MAC: for (int i=0; i<l_COEFF; i++) {
    acc = MAC (sr[tdl_index], coeff[i], acc);
    tdl_index = tdl_index-1;
}
```

Controlling Operator Pipelining

The HLS compiler automatically determines the level of pipelining to use for internal operations. You can use the BIND_OP or BIND_STORAGE pragmas with the `-latency` option to explicitly specify the number of pipeline stages and override the number determined by the tool.

RTL synthesis might use the additional pipeline registers to help improve timing issues that might result after place and route. Registers added to the output of the operation typically help improve timing in the output datapath. Registers added to the input of the operation typically help improve timing in both the input datapath and the control logic from the FSM.

You can use the Operator Configuration commands to pipeline all instances of a specific operation used in the design that have the same pipeline depth.

Unrolling Loops in C++ Classes

IMPORTANT! *When loops are used in C++ classes, care should be taken to ensure that the loop induction variable is not a data member of the class as this prevents the loop from being unrolled.*

In this example, loop induction variable `k` is a member of class `loop_class`.

```
template <typename T0, typename T1, typename T2, typename T3, int N>
class loop_class {
private:
 pe_mac<T0, T1, T2> mac;
public:
 T0 areg;
 T0 breg;
 T2 mreg;
```

```
 T1 preg;
    T0 shift[N];
 int k;              // Class Member
   T0 shift_output;
 void exec(T1 *pcout, T0 *dataOut, T1 pcin, T3 coeff, T0 data, int col)
 {
Function_label0:;
#pragma HLS inline off
 SRL:for (k = N-1; k >= 0; --k) {
#pragma HLS unroll // Loop will fail UNROLL
 if (k > 0)
 shift[k] = shift[k-1];
 else
 shift[k] = data;
 }

    *dataOut = shift_output;
   shift_output = shift[N-1];
 }

 *pcout = mac.exec1(shift[4*col], coeff, pcin);
};
```

For Vitis HLS to be able to unroll the loop as specified by the UNROLL pragma directive, the code should be rewritten to remove `k` as a class member and make it local to the `exec` function.

Limitations of Control-Driven Task-Level Parallelism

TIP: *Control-driven TLP requires the DATAFLOW pragma or directive to be specified in the appropriate location of the code.*

The control-driven TLP model optimizes the flow of data between tasks (functions and loops), and ideally pipelined functions and loops for maximum performance. It does not require these tasks to be chained, one after the other, however there are some limitations in how the data is transferred. The following behaviors can prevent or limit the overlapping that the tool can perform in the dataflow model:

- Reading from function inputs or writing to function outputs in the middle of the dataflow region
- Single-producer-consumer violations
- Conditional execution of tasks
- Loops with multiple exit conditions

IMPORTANT! *If any of these coding styles are present, the HLS compiler issues a message describing the situation.*

Reading from Inputs/Writing to Outputs

Reading of inputs of the function should be done at the start of the dataflow region, and writing to outputs should be done at the end of the dataflow region. Reading/writing to the ports of the function can cause the processes to be executed in sequence rather than in an overlapped fashion, adversely impacting performance.

Single-producer-consumer Violations

For the tool to use the dataflow model, all elements passed between tasks must follow a single-producer-consumer model. Each variable must be driven from a single task and only be consumed by a single task. In the following code example, `temp1` fans out and is consumed by both `Loop2` and `Loop3`. This violates the single-producer-consumer model.

```
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {
   int temp1[N];

   Loop1: for(int i = 0; i < N; i++) {
     temp1[i] = data_in[i] * scale;
   }
   Loop2: for(int j = 0; j < N; j++) {
     data_out1[j] = temp1[j] * 123;
   }
   Loop3: for(int k = 0; k < N; k++) {
    data_out2[k] = temp1[k] * 456;
   }
}
```

A modified version of this code uses function `Split` to create a single-producer-consumer design. The following code block example shows how the data flows with the function `Split`. The data now flows between all four tasks, and the tool can use the dataflow model.

```
void Split (in[N], out1[N], out2[N]) {
// Duplicated data
 L1:for(int i=1;i<N;i++) {
 out1[i] = in[i];
 out2[i] = in[i];
 }
}
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {

 int temp1[N], temp2[N]. temp3[N];
 Loop1: for(int i = 0; i < N; i++) {
 temp1[i] = data_in[i] * scale;
 }
 Split(temp1, temp2, temp3);
 Loop2: for(int j = 0; j < N; j++) {
 data_out1[j] = temp2[j] * 123;
 }
 Loop3: for(int k = 0; k < N; k++) {
 data_out2[k] = temp3[k] * 456;
 }
}
```

Bypassing Tasks and Channel Sizing

In addition, data should generally flow from one task to another. If you bypass tasks, this can reduce the performance of the dataflow model. In the following example, `Loop1` generates the values for `temp1` and `temp2`. However, the next task, `Loop2`, only uses the value of `temp1`. The value of `temp2` is not consumed until *after* `Loop2`. Therefore, `temp2` bypasses the next task in the sequence, which can limit the performance of the dataflow model.

```
void foo(int data_in[N], int scale, int data_out1[N], int data_out2[N]) {
  int temp1[N], temp2[N]. temp3[N];
  Loop1: for(int i = 0; i < N; i++) {
  temp1[i] = data_in[i] * scale;
  temp2[i] = data_in[i] >> scale;
  }
  Loop2: for(int j = 0; j < N; j++) {
  temp3[j] = temp1[j] + 123;
  }
  Loop3: for(int k = 0; k < N; k++) {
  data_out[k] = temp2[k] + temp3[k];
  }
}
```

In this case, you should increase the depth of the PIPO buffer used to store `temp2` to be 3, instead of the default depth of 2. This lets the buffer store the value intended for `Loop3`, while `Loop2` is being executed. Similarly, a PIPO that bypasses two processes should have a depth of 4. Set the depth of the buffer with the STREAM pragma or directive:

```
#pragma HLS STREAM type=pipo variable=temp2 depth=3
```

IMPORTANT! *Channel sizing can also similarly affect performance. Having mismatched FIFO/PIPO depths can inadvertently cause synchronization points inside the dataflow region because of back pressure from the FIFO/PIPO.*

Feedback between Tasks

Feedback occurs when the output from a task is consumed by a previous task in the dataflow region. Non-stream feedback channels (For example, scalars or arrays) are not supported in a dataflow region. When the HLS compiler detects feedback, it issues a warning, depending on the situation, and might not use the dataflow model.

However, dataflow can support feedback when used with `hls::streams`. The following example demonstrates this exception.

```
#include "ap_axi_sdata.h"
#include "hls_stream.h"

void firstProc(hls::stream<int> &forwardOUT, hls::stream<int> &backwardIN) {
  static bool first = true;
  int fromSecond;

  //Initialize stream
  if (first)
    fromSecond = 10; // Initial stream value
```

```
  else
    //Read from stream
    fromSecond = backwardIN.read(); //Feedback value
  first = false;

  //Write to stream
  forwardOUT.write(fromSecond*2);
}

void secondProc(hls::stream<int> &forwardIN, hls::stream<int> &backwardOUT)
{
  backwardOUT.write(forwardIN.read() + 1);
}

void top(...) {
#pragma HLS dataflow
  hls::stream<int> forward, backward;
  firstProc(forward, backward);
  secondProc(forward, backward);
}
```

In this simple design, when `firstProc` is executed, it uses 10 as an initial value for input. Because `hls::streams` do not support an initial value, this technique can be used to provide one without violating the single-producer-consumer rule. In subsequent iterations `firstProc` reads from the `hls::stream` through the `backwardIN` interface.

`firstProc` processes the value and sends it to `secondProc`, via a stream that goes *forward* in terms of the original C++ function execution order. `secondProc` reads the value on `forwardIN`, adds 1 to it, and sends it back to `firstProc` via the feedback stream that goes *backwards* in the execution order.

From the second execution, `firstProc` uses the value read from the stream to do its computation, and the two processes can keep going forever, with both forward and feedback communication, using an initial value for the first execution.

Conditional Execution of Tasks

The dataflow model does not optimize tasks that are conditionally executed. The following example highlights this limitation. In this example, the conditional execution of `Loop1` and `Loop2` prevents the tool from optimizing the data flow between these loops, because the data does not flow from one loop into the next.

```
void foo(int data_in1[N], int data_out[N], int sel) {

 int temp1[N], temp2[N];

 if (sel) {
 Loop1: for(int i = 0; i < N; i++) {
 temp1[i] = data_in[i] * 123;
 temp2[i] = data_in[i];
 }
 } else {
 Loop2: for(int j = 0; j < N; j++) {
 temp1[j] = data_in[j] * 321;
 temp2[j] = data_in[j];
```

```
 }
 }
 Loop3: for(int k = 0; k < N; k++) {
 data_out[k] = temp1[k] * temp2[k];
 }
}
```

To ensure each loop is executed in all cases, you must transform the code as shown in the following example. In this example, the conditional statement is moved into the first loop. Both loops are always executed, and data always flows from one loop to the next.

```
void foo(int data_in[N], int data_out[N], int sel) {

 int temp1[N], temp2[N];

 Loop1: for(int i = 0; i < N; i++) {
 if (sel) {
 temp1[i] = data_in[i] * 123;
 } else {
 temp1[i] = data_in[i] * 321;
 }
 }
 Loop2: for(int j = 0; j < N; j++) {
 temp2[j] = data_in[j];
 }
 Loop3: for(int k = 0; k < N; k++) {
 data_out[k] = temp1[k] * temp2[k];
 }
}
```

Loops with Multiple Exit Conditions

Loops with multiple exit points cannot be used in a dataflow region. In the following example, `Loop2` has three exit conditions:

- An exit defined by the value of `N`; the loop will exit when `k>=N`.
- An exit defined by the `break` statement.
- An exit defined by the `continue` statement.

```
#include "ap_int.h"
#define N 16

typedef ap_int<8> din_t;
typedef ap_int<15> dout_t;
typedef ap_uint<8> dsc_t;
typedef ap_uint<1> dsel_t;

void multi_exit(din_t data_in[N], dsc_t scale, dsel_t select, dout_t
data_out[N]) {
 dout_t temp1[N], temp2[N];
 int i,k;

 Loop1: for(i = 0; i < N; i++) {
 temp1[i] = data_in[i] * scale;
 temp2[i] = data_in[i] >> scale;
 }
```

```
 Loop2: for(k = 0; k < N; k++) {
 switch(select) {
        case  0: data_out[k] = temp1[k] + temp2[k];
        case  1: continue;
        default: break;
 }
 }
}
```

Because a loop's exit condition is always defined by the loop bounds, the use of `break` or `continue` statements will prohibit the loop being used in a DATAFLOW region.

Finally, the dataflow model has no hierarchical implementation. If a sub-function or loop contains additional tasks that might benefit from dataflow, you must apply the dataflow model to the loop, the sub-function, or inline the sub-function.

You can also use `std::complex` inside the dataflow region. However, they should be used with an `__attribute__((no_ctor))` as shown in the following example:

```
void proc_1(std::complex<float> (&buffer)[50], const std::complex<float>
*in);
void proc_2(hls::Stream<std::complex<float>> &fifo, const
std::complex<float> (&buffer)[50], std::complex<float> &acc);
void proc_3(std::complex<float> *out, hls::Stream<std::complex<float>>
&fifo, const std::complex<float> acc);

void top(std::complex<float> *out, const std::complex<float> *in) {
#pragma HLS DATAFLOW

  std::complex<float> acc __attribute((no_ctor)); // Here
  std::complex<float> buffer[50] __attribute__((no_ctor)); // Here
  hls::Stream<std::complex<float>, 5> fifo; // Not here

  proc_1(buffer, in);
  proc_2(fifo, buffer, acc);
  proc_3(out, fifo, acc);
}
```

Limitations of Pipelining with Static Variables

Static variables are used to keep data between loop iterations, often resulting in registers in the final implementation. If this is encountered in pipelined functions, Vitis HLS might not be able to optimize the design sufficiently, which would result in initiation intervals longer than required.

The following is a typical example of this situation:

```
function_foo()
{
    static bool change = 0
    if (condition_xyz){
        change = x; // store
    }
    y = change; // load
}
```

If Vitis HLS cannot optimize this code, the stored operation requires a cycle and the load operation requires an additional cycle. If this function is part of a pipeline, the pipeline has to be implemented with a minimum initiation interval of 2 as the static change variable creates a loop-carried dependency.

One way the user can avoid this is to rewrite the code, as shown in the following example. It ensures that only a read or a write operation is present in each iteration of the loop, which enables the design to be scheduled with II=1.

```
function_readstream()
{
    static bool change = 0
    bool change_temp = 0;
    if (condition_xyz)
    {
        change = x; // store
        change_temp = x;
    }
    else
    {
    change_temp = change; // load
    }
    y = change_temp;
}
```

Creating Vitis HLS Components

This section contains the following chapters:

- Launching the Vitis Unified IDE
- Creating an HLS Component
- Running C Simulation
- Running C Synthesis
- Running C/RTL Co-Simulation
- Packaging the RTL Design
- Running Implementation
- Optimizing the HLS Project
- Creating HLS Components from the Command Line

Launching the Vitis Unified IDE

TIP: *The AMD Vitis™ Unified IDE application is installed with the Vitis software platform. It does not need additional installation. For more on Installing and Launching the Vitis Unified IDE refer to the Embedded Design Development Using Vitis (UG1701).*

1. Use the following command to setup the Vitis software platform environment.

   ```
   source <Vitis_Installation_Directory>/settings64.sh
   ```

2. Use the following command to launch the next generation Vitis Unified IDE.

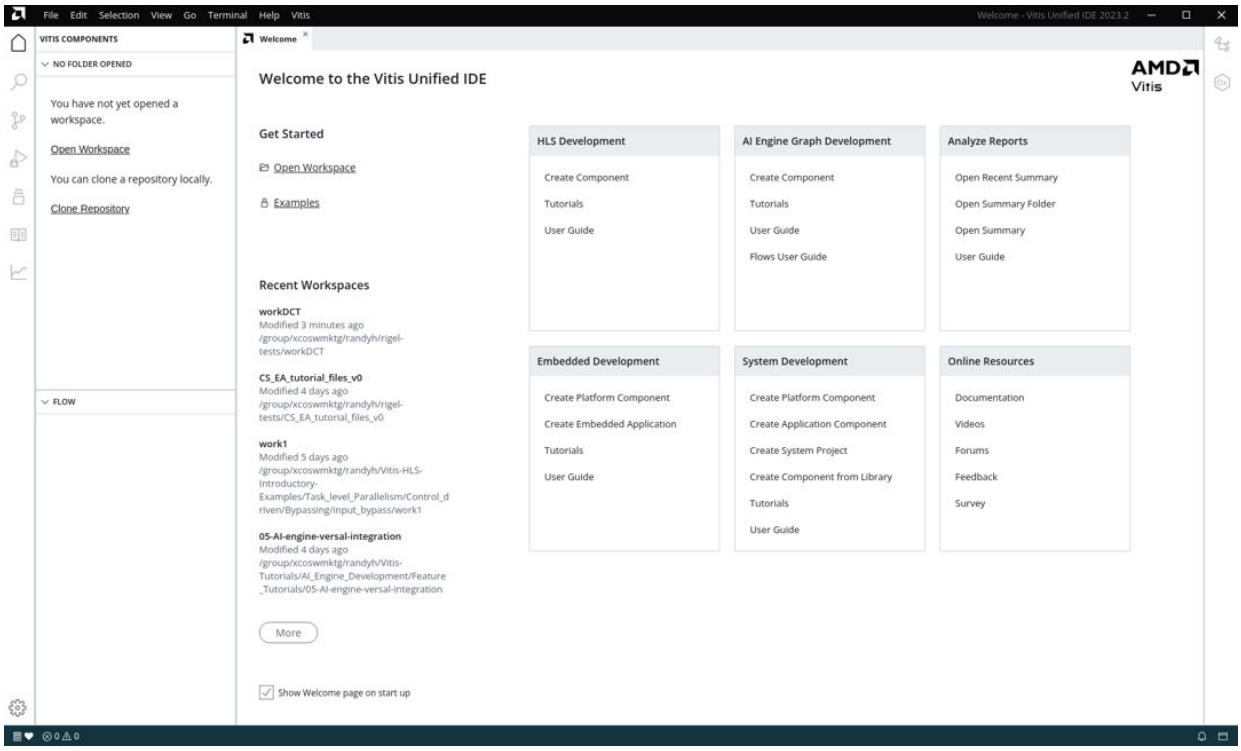
   ```
   vitis -w <workspace>
   ```

   Where `<workspace>` indicates a folder to hold the elements of your HLS component, or other design projects. The workspace is used to group together the source and data files that make up a design, or multiple designs, and stores the configuration of the tool for that workspace.

TIP: *You can also launch the Vitis Unified IDE for HLS components by double-clicking the application icon from the Windows desktop.*

The Vitis Unified IDE opens with the Welcome screen as shown in the following figure.

Figure 82: **Welcome Screen**



Under HLS Development, you have the following options.

- **Create Component:** Launch the Create HLS Component wizard to create a new component. Refer to Building and Running an HLS Component for more information.
- **Create Component from Library:** Create a new HLS component from library functions.
- **Tutorials:** Opens a web browser pointing to the `Vitis-HLS-Introductory-Examples` on GitHub. See Tutorials and Examples.
- **User Guide:** Opens this document in a web browser.

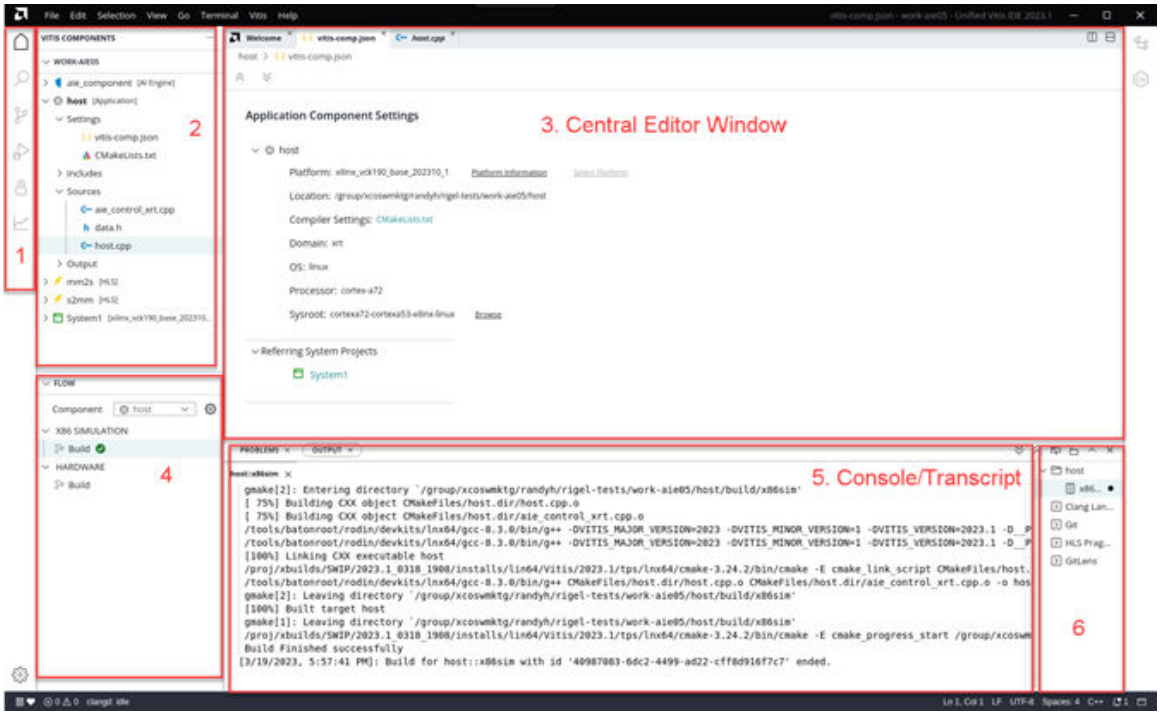
You can see there are other blocks of commands and features available within the tool. Under Online Resources, you will find documentation and tutorials to help you work with the tool.

If you have previously launched the Vitis Unified IDE you can load one of the prior workspaces listed under Recent Workspaces. You can also look under the File menu to see what other options are available.

Features of the Vitis Unified IDE

The new AMD Vitis™ Unified IDE provides many new features, which are described in brief below:

Figure 83: **Vitis Unified IDE**



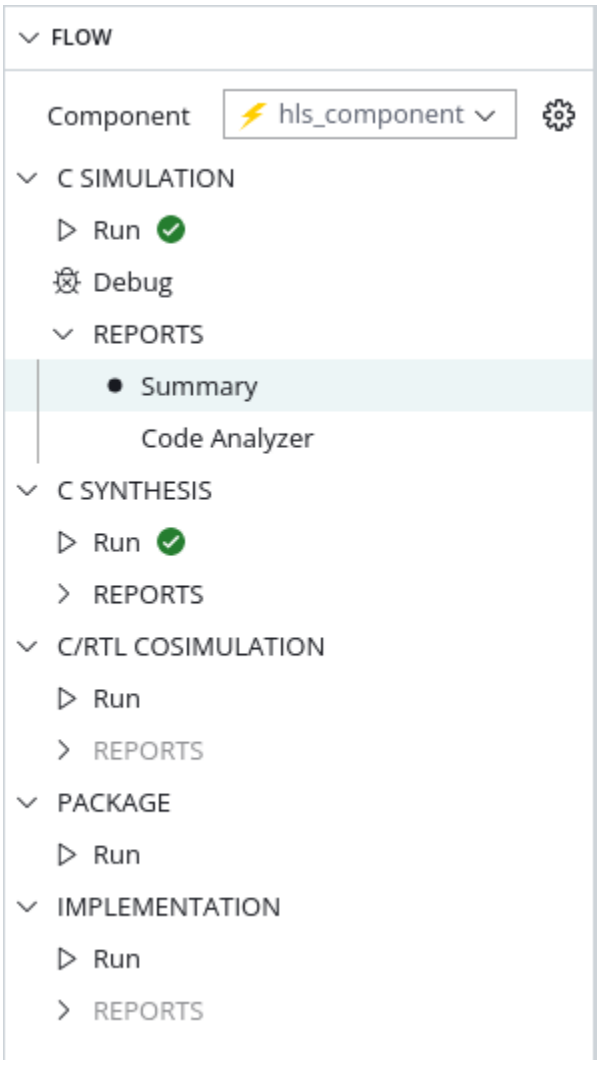
1. The Tool Bar on the left of the screen provides easy access to major features of the tool: the Vitis Component Explorer, the Search function, Source Control, the Debug view, Examples, and the Analysis view.
2. The Vitis Component Explorer can be used to view a virtual hierarchy of the workspace. The view displays a workspace that is structured to help you understand the different elements of the component or project, such a Sources and Outputs folder that don't exist on disk.
3. The Central Editor window is used for editing components, configurations, and source files.
4. The Flow Navigator displays the design flow for the active component. Different components has different work flows, and the work flow of the active component is the one displayed in the Flow Navigator. You can specify the active component by selecting it in the Flow Navigator, or selecting it in the Component Explorer.
5. The Console/Terminal area displays the output transcripts of the tool, and other windows such as the Terminal window and the Pipeline view are also located here. The terminal window displays the folders of the workspace and can be used to run scripts on the content.

6. The Index displays a list of transcripts of the various build steps ran during the session and allows you to reopen a process transcript that was closed earlier.

Using the Flow Navigator

For the HLS component in the Vitis Unified IDE, the Flow Navigator is a process flow representation with each step in the process represented by actions that you can launch to work through the flow. All reports and report viewers are also available through the Flow Navigator as each step is completed.

Figure 84: **Flow Navigator**



The process steps for the HLS component include:

- **C SIMULATION:** Runs C Simulation and the Code Analyzer using the specified source code and test bench and any input data provided as described in Running C Simulation.
- **C SYNTHESIS:** Opens the C Synthesis dialog box, and lists the available reports after synthesis has been run, as discussed in Running C Synthesis.
- **C/RTL CO-SIMULATION:** Opens the C/RTL Cosimulation dialog box, and lists the available reports after simulation has been run, as described in Running C/RTL Co-Simulation.
- **PACKAGE:** Generates the Vivado IP (`.zip`) or Vitis Kernel (`.xo`) and exports it from the synthesized RTL design as described in Packaging the RTL Design.
- **IMPLEMENTATION:** Run Vivado synthesis and implementation to generate more detailed utilization and timing reports as described in Running Implementation.

The Flow Navigator also has a few features to help you view the associated files and design state. As seen in the preceding figure, the Open Settings command () is available next to the Component menu and each process step in the design flow. Select **Open Settings** next to Component to open the `vitis-comp.json` file for the component. Select it next to the process step to open the Config File Editor.

You can cancel a running process from within the Flow Navigator by using the **Cancel Run** command. This command is hidden by default and is revealed by hovering over the running step to reveal the command.

The Flow Navigator displays a warning sign () when certain steps are not complete, or need to be rerun due to changes in the design source or associated files. Hovering over the warning displays a message indicating the possible problem.

Chapter 12

Building and Running an HLS Component

The AMD Vitis™ Unified IDE and `v++` common command line provide the ability to create an HLS component using bottom-up design techniques to ensure the component meets design requirements before integrating it with other elements into a system design. In an HLS component, the tool synthesizes a C or C++ function into RTL code for implementation in the programmable logic (PL) region of an AMD Versal™ adaptive SoC, AMD Zynq™ MPSoC, or AMD FPGA device.

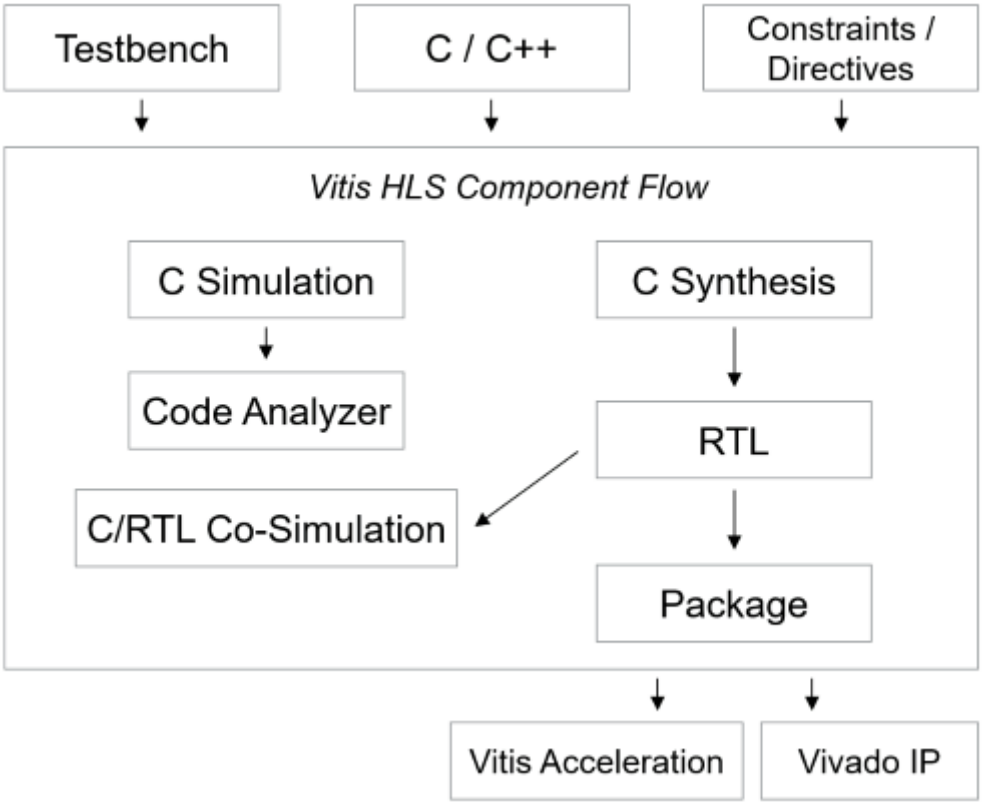
An HLS component can be used to develop and export:

- Vivado IP to be integrated into hardware designs using the Vivado Design Suite and the Embedded Software Development flow as described in *Vitis Unified Software Platform Documentation: Embedded Software Development* (UG1400).
- Vitis kernels for use in hetergeneous compute systems and Data Center application acceleration as described in *Embedded Design Development Using Vitis* (UG1701).

The following are the steps for the development of the C++ function.

1. Architect the C/C++ algorithm based on Design Principles
2. Verify the logic of the source code with a C test bench (C-Simulation)
3. Analyze the parallelism and performance of the design (Code Analyzer)
4. Generate the RTL from the source code (C-Synthesis)
5. Verify the generated RTL module with the C test bench and RTL logic simulation (C/RTL Co-Simulation)
6. Review and analyze the HLS synthesis reports and co-simulation reports (Analyze)
7. Repeat previous steps until performance goals are met

Figure 85: **Vitis HLS Development Flow**



The HLS component implements the design based on the target flow, default tool configuration, configuration commands, and any optimization pragmas or directives you specify for the design. The following sections describe creating and building the HLS component.

Creating an HLS Component

There are a number of ways to create an HLS component in the Vitis Unified IDE. The following steps describe a recommended approach.

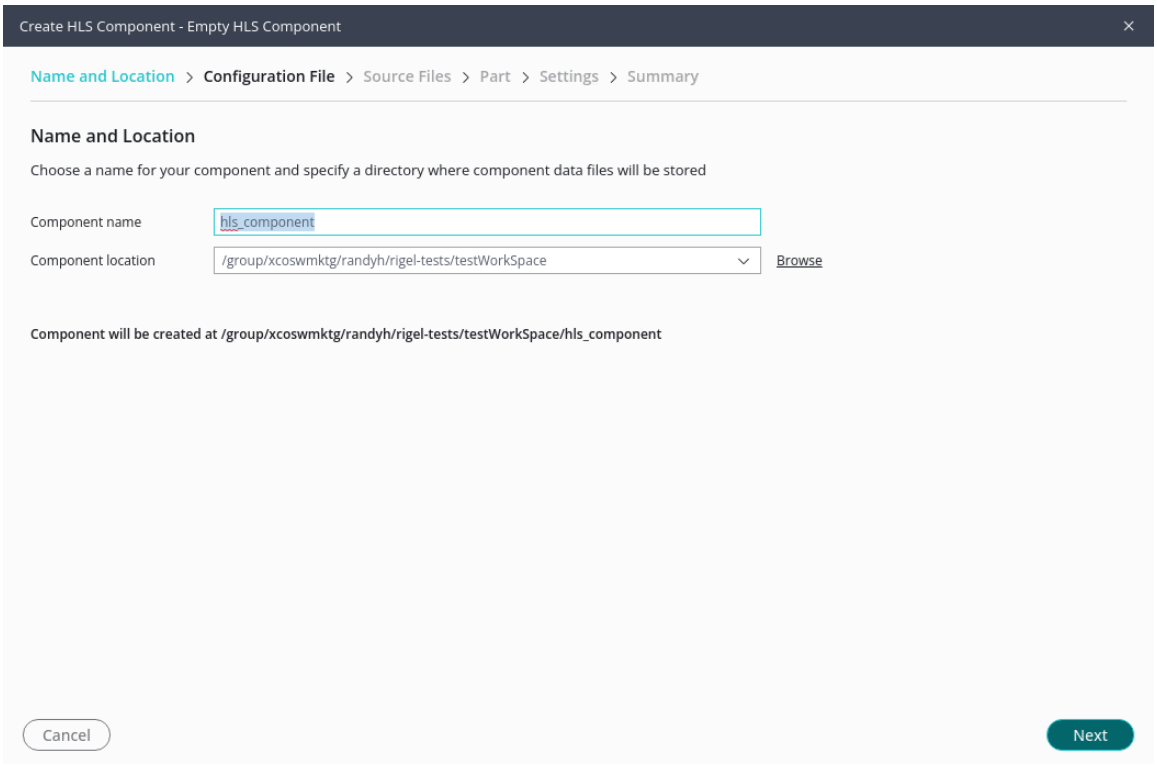
TIP: *This section presents the HLS component in the Vitis IDE. For a command-line approach refer to Creating HLS Components from the Command Line.*

1. With the Vitis IDE opened, from the main menu select **File → New Component → HLS**.

   TIP: *You can also select **New Component** on the Welcome page, or from the right-click menu in the Explorer view.*

2. This opens the Choose Name and Location page of the Create HLS Component wizard as shown below.

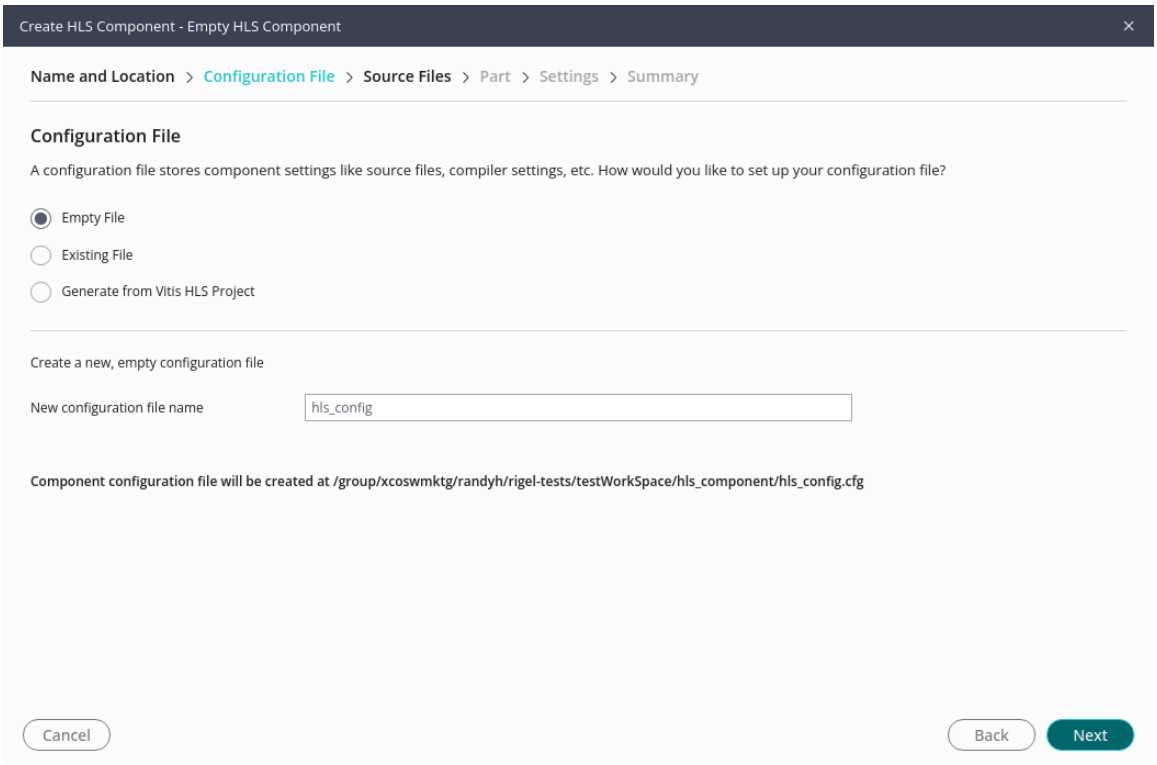
Figure 86: **Create HLS Component - Name and Location**



Enter a Component name and Component location and select **Next**.

3. This opens the Configuration File page of the wizard as shown below. The HLS component configuration file contains commands and directives used for the synthesis, simulation, and export of the component.

Figure 87: **Create HLS Component - Configuration File**



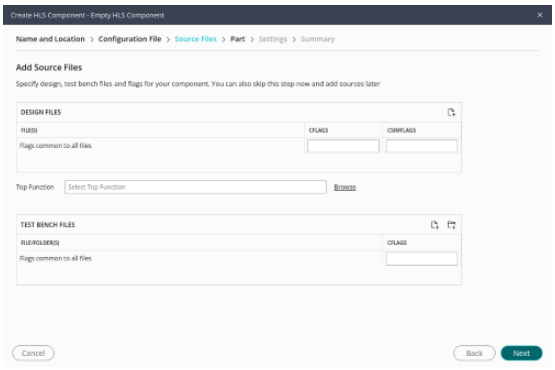
The Configuration File page of the Create HLS Component wizard provides three ways to add a configuration file to your component:

- Empty File: Specifies the creation of a new empty configuration file to be populated after component creation as described in Defining the HLS Config File. Selecting this option lets you specify a name in the New configuration file name field.
- Existing File: Specifies the use of an existing configuration file. Selecting this option lets you enter the path and name in the File field, or select **Browse** to locate the file.
- Generate from Vitis HLS Project: This option generates a new config file from an existing Vitis HLS project `hls.app` file. The tool lets you identify the `hls.app` file inside the project, and enter a New configuration file name for the config file generated from the `hls.app` file.

Choose the method of adding the config file to your HLS component, and specify a name or files as needed. Select **Next** to proceed to the Source Files.

4. The Source Files page of the wizard opens as shown below.

Figure 88: **Create HLS Component - Source Files**



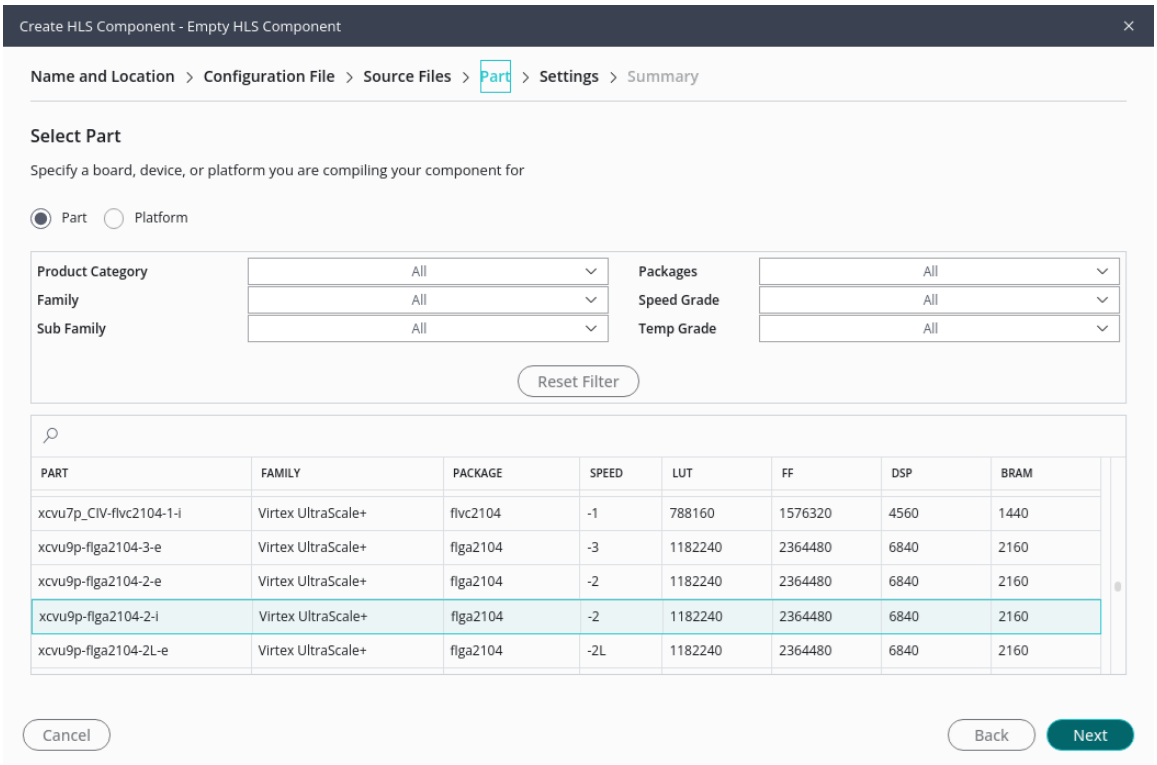
Under Design Files you can you can select **Add files** to select the primary source file for the design, or add multiple files as needed. After adding one or more source files, you can identify the Top Function which is the function to be synthesized as the top-level function for the HLS component. Any functions called by the top-function will also be synthesized into RTL.

Under Test Bench Files you can select **Add files** or select **Add folders** to add files containing test bench and associated files for the HLS component.

For both design files and test bench files you can add CFLAGS and CSIMFLAGS to specify directives to be used during compilation and simulation. You can specify directives to be applied to all files or to specific files from each category. After adding files and folders and directives, select **Next** to proceed.

5. The Select Part page of the wizard is opened as shown below.

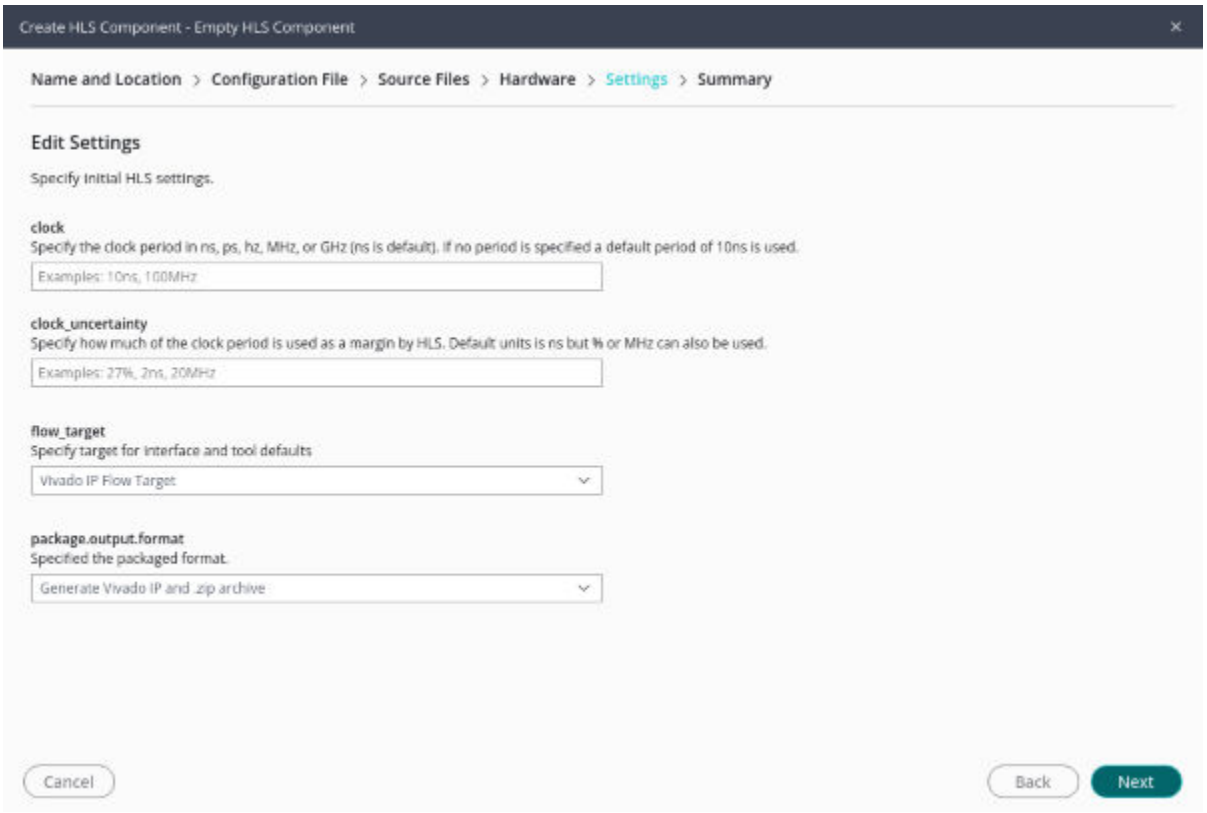
Figure 89: **Create HLS Component - Part**



Select the **Part** or select the **Platform** for the HLS component and select **Next** to proceed.

> **TIP:** *If you select **Platform** when creating the HLS component, the Vitis Unified IDE extracts the part from the specified platform and adds* `part=` *to your configuration file.*

6. This displays the Settings page of the wizard as shown below.



The Settings page lets you specify a clock period or frequency for the design, and a clock uncertainty. The default clock for the tool is a clock period of 10 ns (or 100 MHz), and a clock uncertainty of 27% as described in Specifying the Clock Frequency.

The Settings page also lets you specify the flow_target for the HLS component build process as being either to generate a Vivado IP or a Vitis kernel as described in Target Flow Overview. The interface requirements of the Vivado IP or a Vitis kernel are different as explained in Interfaces of the HLS Design. The default flow_target is the Vivado IP flow.

Below the flow_target you can also specify the package.output.format for the tool to generate when packaging the design. The default output format is the Vivado IP, which generates a `.zip` file to be added to your IP catalog, letting you use the HLS generated RTL design in other designs.

***Note*:** Selecting the Vitis flow_target does not automatically change the output format to Vitis kernel. The default is still Vivado IP, but you can manually change the output format if needed.

Also keep in mind that the HLS component uses a bottom-up design flow, generating only RTL output until you package the output as described in Packaging the RTL Design. The bottom-up flow lets you develop multiple design iterations as you sort through the design challenges, and only package the output when the design is ready. This approach saves significant time each time you run Synthesis or Simulation.

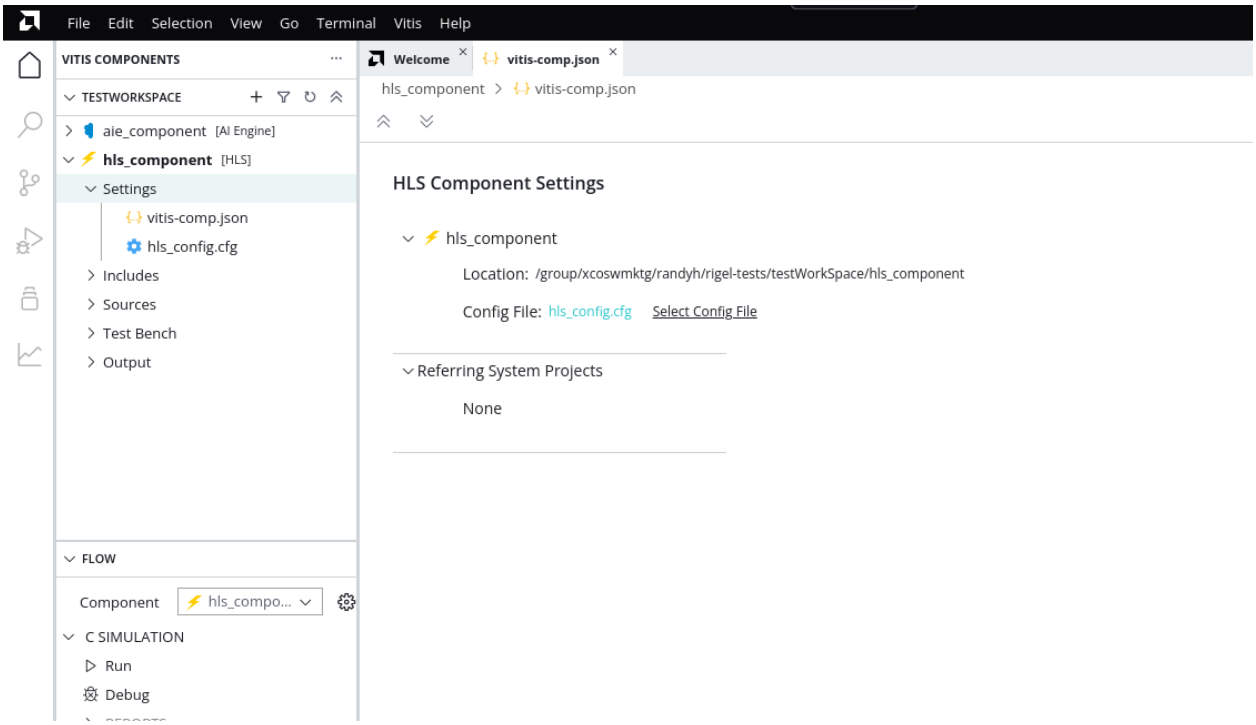
Select **Next** to proceed to the Summary page.

7. The Summary page reflects the choices you have made on the prior pages. Review the summary and select **Finish** to create the HLS component, or select **Back** to return to earlier pages and change your selections.

When the HLS component is created the `vitis-comp.json` file for the component is opened in the central editor window, and the HLS component becomes active in the Flow Navigator as shown below.

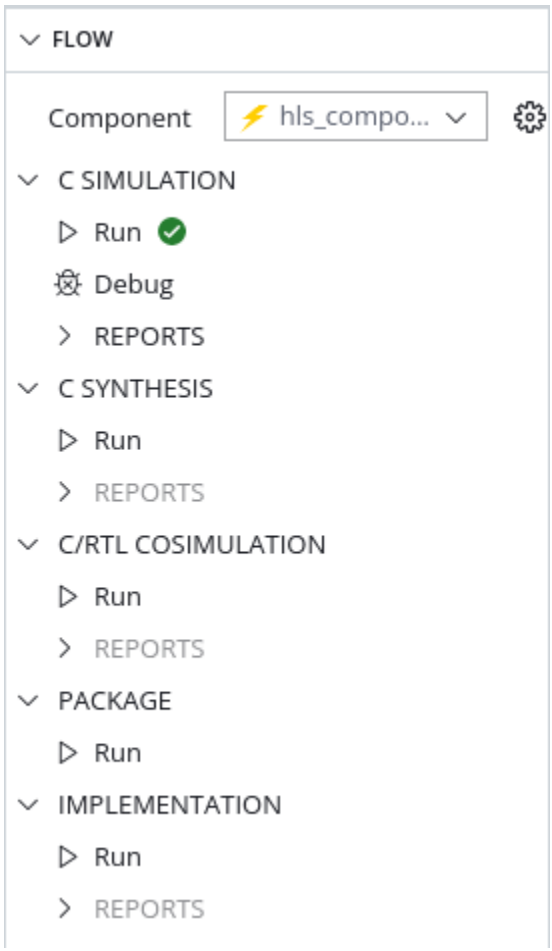
TIP: *When multiple HLS components are opened, the component name appears in the tab alongside the `vitis-comp.json` file name to help you identify the different components.*

Figure 90: **HLS Component - vitis-comp.json**



After creation, the HLS component becomes the active component in the Flow Navigator, or you can select the **Component** to make it the active component in the tool. When the HLS component is the active component, the Flow Navigator enables running C Simulation, C Synthesis, C/RTL Co-simulation, Package, and Implementation as shown in the following image.

Figure 91: **HLS Component - Flow Navigator**



To manage the actions of these steps however, you will need to edit the HLS config file to achieve your design objectives as described below.

Target Flow Overview

The HLS component is destined to either the Vivado IP flow, or the Vitis Kernel flow. The HLS component is crafted based on the target flow, default tool configuration, design constraints, and any optimization pragmas or directives you specify. You can use optimization directives to modify and control the implementation of the internal logic and I/O ports, overriding the default behavior.

Vivado IP Flow

When you select the **Vivado IP Flow** on the Settings page of the Create HLS Component wizard, or specify `flow_target=vivado` in the HLS configuration file, you are configuring the tool to generate RTL files for use in the Vivado Design Suite, for use with other RTL files and Vivado IP, and for use with software applications using software to interact with the hardware.

The HLS IP can be implemented into the programmable logic (PL) region of an AMD Versal™ adaptive SoC, AMD Zynq™ MPSoC, or AMD FPGA device. The IP is generated with device drivers so the PL can be integrated with embedded software applications to manage and run the hardware functions as described in the *Vitis Embedded Software Development Flow Documentation* (UG1400).

The Vivado IP flow provides greater flexibility in your design choices over the Vitis kernel flow, however it leaves the integration and management of the IP to you as well. The Vivado IP flow can support a wide variety of interface specifications and data transfer protocols, but has default interfaces assigned to function arguments as described in Interfaces for Vivado IP Flow. You can also override the default settings by manually assigning the interface specification for your function argument, using the `INTERFACE` pragma or `set_directive_interface` command, to meet the needs of your Vivado design.

Vitis Kernel Flow

When you select the **Vitis Kernel Flow** on the Settings page of the Create HLS Component wizard, or specify `flow_target=vitis` in the HLS configuration file, you are configuring the tool to generate an RTL design suitable for the Vitis development environment and the Xilinx Runtime (XRT). The Vitis development flow can be used in embedded system design for heterogeneous compute systems for AMD Versal™ adaptive SoC, AMD Zynq™ MPSoC devices, or for Data Center application acceleration running on Alveo accelerator cards as described in *Data Center Acceleration using Vitis* (UG1700). The Vitis development flow is more restrictive than the Vivado IP flow, as the design requirements for RTL produced by the HLS tool must meet the specific requirements of the platforms and Xilinx Runtime (XRT).

When specifying the **Vitis Kernel Flow** in the IDE, the tool implements interface ports using the AXI standard as described in Interfaces for Vitis Kernel Flow. If there are no existing `INTERFACE` pragmas or directives in the code, then the following interface protocols will be applied by default.

- AXI4-Lite interfaces (`s_axilite`) are assigned to scalar arguments, control signals for arrays, and the return value of the software function.
- AXI4 Master interfaces (`m_axi`) are assigned to pointer and array arguments of the C/C++ function.
- Vitis HLS automatically tries to infer BURST transactions whenever possible to aggregate memory accesses to maximize the throughput bandwidth and/or minimize the latency.
- Defining a software function argument using an `hls::stream` data type implies an AXI4-Stream (`axis`) port.

Default Settings of Vivado/Vitis Flows

The default settings of the tool vary between the Vivado IP flow and the Vitis Kernel flow. The following table shows the default settings of both flows.

TIP: *Beyond the default configuration, there are additional features of the tool that supports one flow, but not the other, or are configured differently between the two flows. Those differences are highlighted throughout this document.*

Table 23: **Default Configuration**

Configuration	Vivado	Vitis
clock_uncertainty	27%	27%
syn.compile.pipeline_loops	64	64
syn.compile.name_max_length	255	255
vivado.optimization_level	0	0
vivado.phys_opt	none	none
syn.rtl.module_auto_prefix	true	true
syn.rtl.register_reset_num	0	3
syn.schedule.enable_dsp_full_reg	true	true
syn.directive.interface	IP mode	Kernel mode
syn.interface.m_axi_addr64	true	true
syn.interface.m_axi_latency	0	64
syn.interface.m_axi_alignment_byte_size	1	64
syn.interface.m_axi_max_widen_bitwidth	0	512
syn.interface.default_slave_interface	s_axilite	s_axilite
syn.interface.m_axi_offset	slave	slave

Flow Package Settings

Beyond these default settings, you must also package the generated RTL files for use in downstream processes, such as Vivado designs integrating multiple RTL files or Vivado IP into a single design. Or a Vitis kernel integrated with other PL kernels and an AI Engine graph application in a System project. The generated output is determined by the `package.output.format` and `package.output.syn` settings, as described in Packaging the RTL Design. The output format lets you create a packaged format from the RTL files of the design. The default output format is the Vivado IP `.zip` file format, which can be added to your IP catalog, and used in Vivado IP integrator features. You can also specify the output format as XO format to package the RTL files as a kernel for use in the Vitis development flow. The selection of `flow_target` does not automatically specify the output format. You must do this yourself.

Working with Sources

Vitis HLS inputs include:

- C functions written in C and C++11/C++14. This is the primary input to Vitis HLS. The function can contain a hierarchy of sub-functions.

- Configuration file that specifies the clock period, clock uncertainty, and the device target.
- Directives are optional and direct the synthesis process to implement a specific behavior or optimization.
- C test bench and any associated files needed to simulate the C function prior to synthesis, and to verify the RTL output using C/RTL Co-simulation.

You can add the C/C++ source files, directives, and constraints to an HLS component in the Vitis unified IDE as described in Creating an HLS Component. You can also create an HLS config file and use the `v++ -c --mode hls` command as described in Creating HLS Components from the Command Line.

Coding C/C++ Functions

Coding Style

In any C program, the top-level function is called `main()`, generally this function is found in the test bench for C simulation. In the HLS component you can specify any sub-function called by `main()` as the top-level function for high-level synthesis. You *cannot* synthesize the top-level function `main()`. Some additional rules include:

- Only one function is allowed as the top-level function for synthesis.
- Any sub-functions in the hierarchy under the top-level function for synthesis are also synthesized.
- If you want to synthesize functions that are not in the hierarchy under the top-level function for synthesis, you must merge the functions into a single top-level function for synthesis.

C/C++ Language Support

The HLS component supports many C and C++11/C++14 language constructs, and all native data types for each language, including float and double types as discussed in Data Types. However, synthesis is *not* supported for some constructs, including:

- Dynamic memory allocation: An FPGA has a fixed set of resources, and the dynamic creation and freeing of memory resources is not supported.
- Operating system (OS) operations: All data to and from the FPGA must be read from the input ports or written to output ports. OS operations, such as file read/write or OS queries like time and date, are not supported. Instead, the host application or test bench can perform these operations and pass the data into the function as function arguments.

Using Libraries in HLS Components

The Vitis Unified IDE and `v++` common command line support foundational C/C++ libraries allowing common hardware design constructs and functions to be easily modeled in C/C++ and synthesized to RTL. The following libraries are provided with with the tools as described in Section VI: Vitis HLS Libraries Reference:

- Arbitrary Precision Data Types Library: Arbitrary precision data types let your C code use variables with smaller bit-widths than standard C or C++ data types, to enable improved performance and reduced area in hardware.
- HLS Math Library: Used to specify standard math operations for synthesis into RTL and implementation on AMD devices.
- HLS Stream Library: For modeling and compiling streaming data structures.

You can use each of the C libraries in your design by including the library header file in your code. These header files are located in the `include` directory in the Vitis HLS installation area.



IMPORTANT! *The header files for the libraries do not have to be in the include path if the design is built in the AMD Vitis™ Unified IDE as the paths to the library header files are automatically added.*

Vitis Libraries

In addition, the Vitis Libraries are available for use with Vitis Unified IDE and `v++` common command line. The libraries include common functions of math, statistics, linear algebra and DSP; and also supporting domain specific applications, like vision and image processing, quantitative finance, database, data analytics, and data compression. The libraries can be downloaded from https://github.com/Xilinx/Vitis_Libraries and setup for use in the Libraries view in the Vitis Unified IDE, as described in Working with the Vitis Unified IDE in the *Vitis Reference Guide* (UG1702).

The libraries contain functions and constructs that are optimized for implementation on AMD devices. Using these libraries helps to ensure high quality of results (QoR); that the results of synthesis are a high-performance design that optimizes resource usage. Because the libraries are provided in C and C++, you can incorporate the libraries into your top-level function and simulate them to verify the functional correctness before synthesis.



TIP: *The Vitis application acceleration libraries are not available for use on the Windows operating system.*

Working with OpenCV

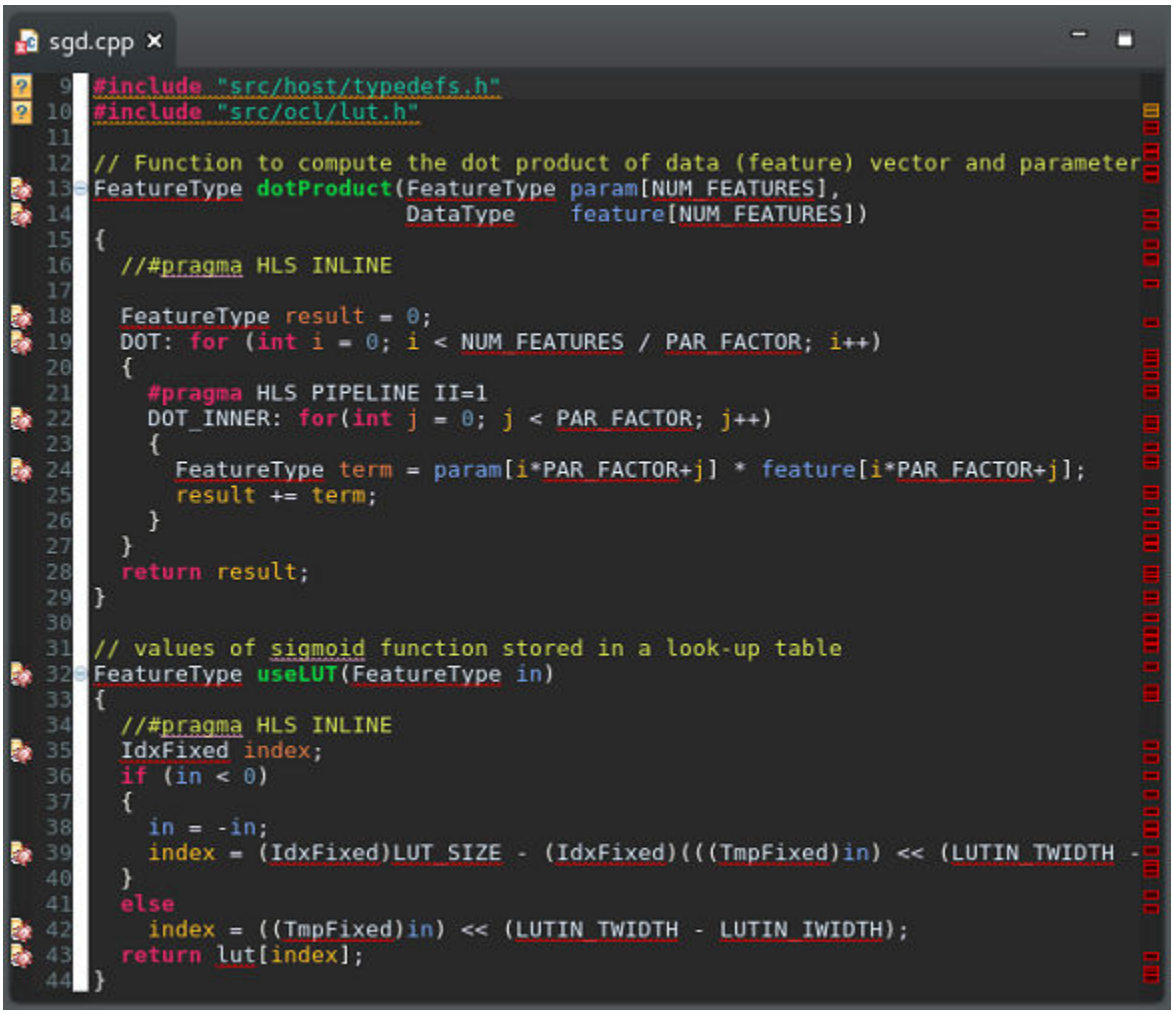
As described at https://github.com/Xilinx/Vitis_Libraries/tree/main/vision, to work with the Vitis Vision Library you must also have installed the OpenCV computer vision library. Vitis HLS does not come with OpenCV pre-installed, but the following link describes how to find and install it: Vitis Libraries - Compiling and Installing OpenCV libraries for use with Vision library.

Resolving References and Viewing #include Files

The Vitis Unified IDE continually parses all header files to resolve coding references. Valid references allow the code to compile correctly of course, but also let you right-click on an `#include` statement to use the **Go to Declaration** command to open the included file. You can also select a function name, variable, or data type and use the **Go to Definition** command to view its definition.

The GUI highlights unresolved references, as shown in the following figure:

Figure 92: **Unresolved References**



- Left sidebar: Highlights unresolved references at the line number of the source code.
- Right sidebar: Displays unresolved references relative to the whole file.

If all your project settings (like the include path) are properly defined, by default the tool will index the C source files automatically. Unresolved references occur when code defined in a header file (`.h` or `.hpp` extension) cannot be resolved. The primary causes of unresolved references are:

- The code was recently added to the file.

  If the code is new, ensure the header file is saved. After saving the header file the tool automatically indexes the header files and updates the code references.
- The header file is not in the search path.

Ensure the header file is included in the C code using an `#include` statement, and the header file is found in the same directory as the source C file, or the location to the header file is in the search path.

> **TIP:** *To explicitly add the search path for a source file edit the `syn.cflags` or `syn.file_cflags` to specify directives to be used during compilation and simulation commands for the source file as discussed in HLS General Options.*

Resolving Comments in the Source Code

In some localizations, non-English comments in the source file appears as strange characters. This can be corrected using the following steps:

1. Select the **File → Preferences → Open Settings (UI)** menu command.
2. From the Preferences view select the **Text Editor** heading and select **Files**.
3. Select the **Encoding** dropdown menu and choose appropriate encoding.
4. Save changes.

Adding RTL Blackbox Functions

The RTL blackbox enables the use of existing Verilog RTL IP in an HLS project. This lets you add RTL code to your C/C++ code for synthesis of the project by Vitis HLS. The RTL IP can be used in a sequential, pipeline, or dataflow region. Refer to Vitis-HLS-Introductory-Examples/Misc/rtl_as_blackbox on GitHub for examples of this technique.

> **TIP:** *Adding an RTL blackbox to your design will restrict the tool from outputting VHDL code, Because the RTL blackbox must be Verilog, the output will be Verilog only.*

Integrating RTL IP into a Vitis HLS project requires the following files:

- C function signature for the RTL code. This can be placed into a header (`.h`) file.
- Blackbox JSON description file as discussed in JSON File for RTL Blackbox.
- RTL IP files.

To use the RTL blackbox in an HLS project, use the following steps.

1. Call the C function signature from within your top-level function, or a sub-function in the Vitis HLS project.
2. Add the blackbox JSON description file to your HLS project using the **Add Files** command from the Vitis HLS IDE as discussed in Creating an HLS Component, or using the `add_files` command:

```
add_files -blackbox my_file.json
```

> **TIP:** *As explained in the next section, the new RTL Blackbox wizard can help you generate the JSON file and add the RTL IP to your project.*

3. Run the Vitis HLS design flow for simulation, synthesis, and co-simulation as usual.

Requirements and Limitations

RTL IP used in the RTL blackbox feature have the following requirements:

- Should be Verilog (`.v`) code.
- Must have a unique clock signal, and a unique active-High reset signal.
- Must have a CE signal that is used to enable or stall the RTL IP.
- Can use either the `ap_ctrl_chain` or `ap_ctrl_none` block level control protocols as described in Block-Level Control Protocols.

Within Vitis HLS, the RTL blackbox feature:

- Supports only C++.
- Cannot connect to top-level interface I/O signals.
- Cannot directly serve as the design-under-test (DUT).
- Does not support `struct` or `class` type interfaces.
- Supports the following interface protocols for ap_ctrl_none as described in JSON File for RTL Blackbox:
  - **Wire inputs and outputs:** For pipeline with II=1 (see rtl_ports attribute in the JSON File Format table in JSON File for RTL Blackbox)
  - **FIFO Inputs and Outputs:** For dataflow with streams
- Supports the following interface protocols for ap_ctrl_chain as described in JSON File for RTL Blackbox:
  - **hls::stream:** The RTL blackbox IP supports the `hls::stream` interface. When this data type is used in the C function, use a `FIFO` RTL port protocol for this argument in the RTL blackbox IP.
  - **Arrays:** The RTL blackbox IP supports RAM interface for arrays. For array arguments in the C function, use one of the following RTL port protocols for the corresponding argument in the RTL blackbox IP:
    . Single port RAM – RAM_1P
    . Dual port RAM – RAM_T2P
  - **Scalars and Input Pointers:** The RTL Blackbox IP supports C scalars and input pointers only in sequential and pipeline regions. They are not supported in a dataflow region. When these constructs are used in the C function, use `wire` port protocol in the RTL IP.

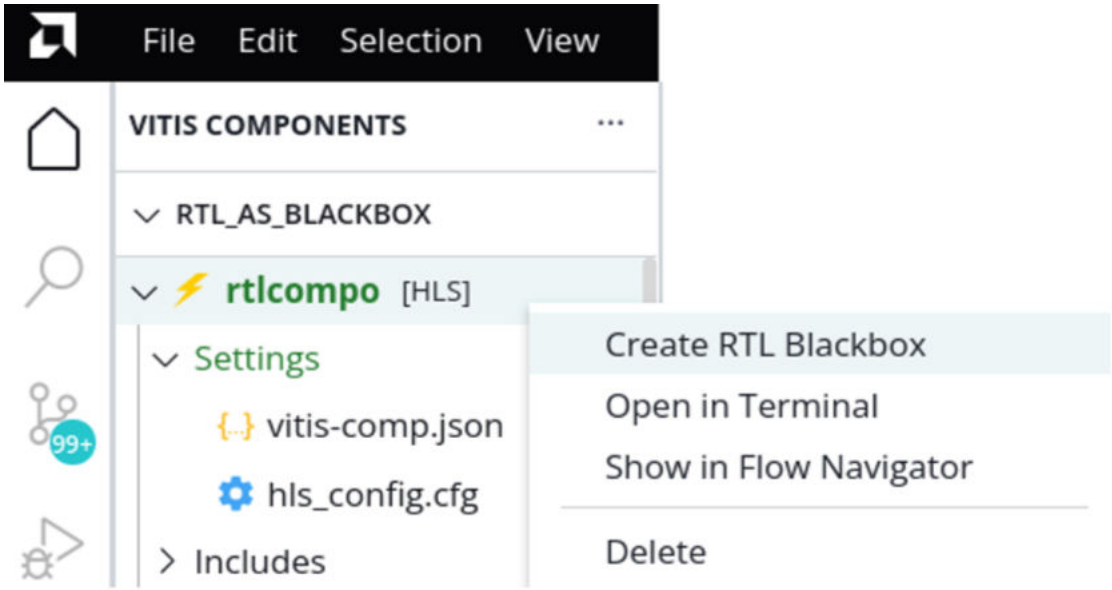
- **Inout and Output Pointers:** The RTL blackbox IP supports inout and output pointers only in sequential and pipeline regions. They are not supported in a dataflow region. When these constructs are used in the C function, the RTL IP should use `ap_vld` for output pointers, and `ap_ovld` for inout pointers.

TIP: *All other Vitis HLS design restrictions also apply when using RTL blackbox in your project.*

Using the RTL Blackbox Wizard

Navigate to the project, right-click to open the **RTL Blackbox Wizard** as shown in the following figure:

Figure 93: **Opening RTL Blackbox Wizard**



The Wizard is organized into pages that break down the process for creating a JSON file. To navigate between pages, click **Next** and select **Back**. Once the options are finalized, you can generate a JSON by clicking **OK**. Each of the following section describes each page and its input options.

C++ Model and Header Files

In the Blackbox C/C++ files page, you provide the C++ files which form the functional model of the RTL IP. This C++ model is only used during C++ simulation and C++/RTL co-simulation. The RTL IP is combined with Vitis HLS results to form the output of synthesis.

Figure 94: **Blackbox C/C++ Files Page**



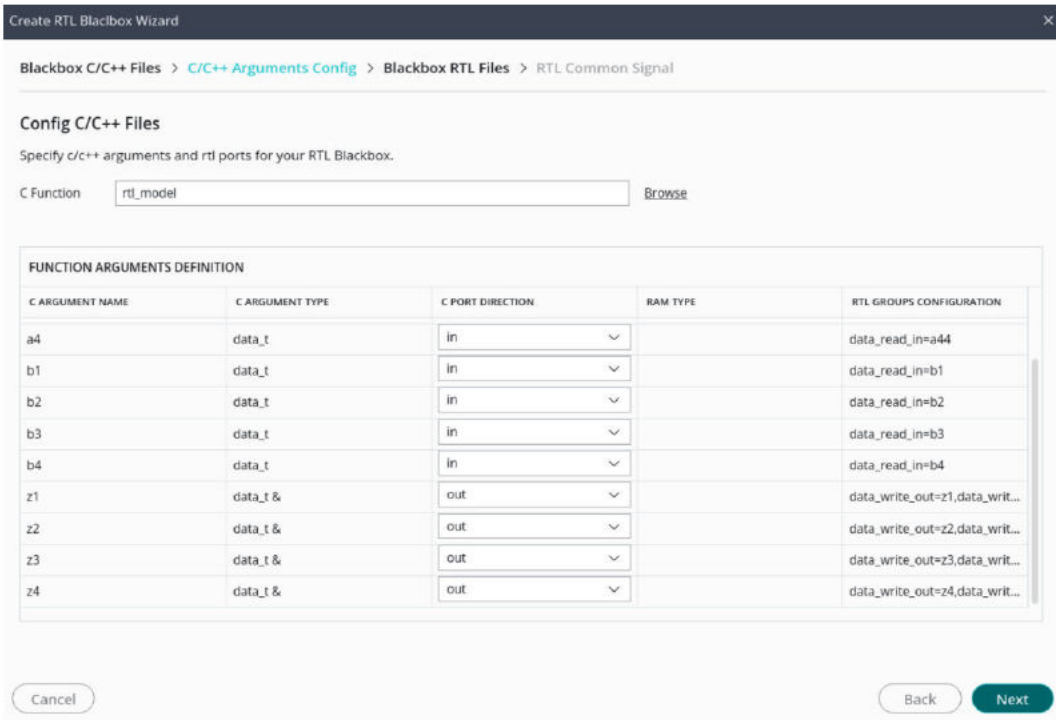
In this page, you can perform the following:

- Click **Add Files** to add files.
- Click **Edit CFLAGS** to provide a linker flag to the functional C model.
- Click **Next** to proceed.

The C File Wizard page lets you specify the values used for the C functional model of the RTL IP. The fields include:

- **C Function:** Specify the C function name of the RTL IP.
- **C Argument Name:** Specify the name(s) of the function arguments. These should relate to the ports on the IP.
- **C Argument Type:** Specify the data type used for each argument.
- **C Port Direction:** Specify the port direction of the argument, corresponding to the port in the IP.
- **RAM Type:** Specify the RAM type used at the interface.
- **RTL Group Configuration:** Specifies the corresponding RTL signal name.

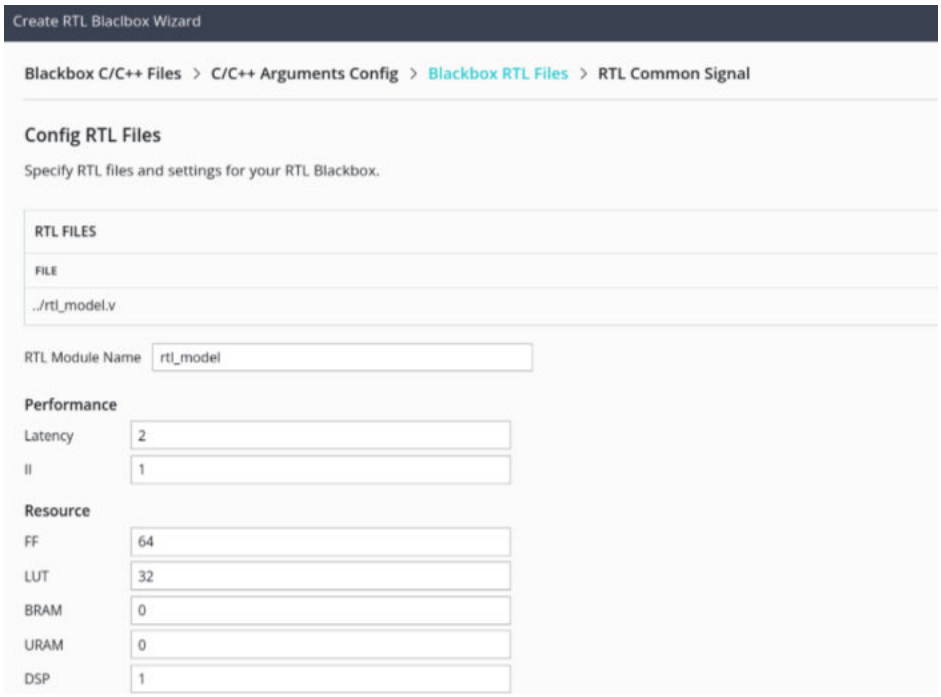
Figure 95: **C File Wizard Page**



Click **Next** to proceed.

RTL IP Definition

Figure 96: **RTL Blackbox Wizard**



The RTL Wizard page lets you define the RTL source for the IP. The fields to define include:

- **RTL Files:** This option is used to add or remove the pre existing RTL IP files.
- **RTL Module Name:** Specify the top level RTL IP module name in this field.
- **Performance:** Specify performance targets for the IP.
  - **Latency:** Latency is the time required for the design to complete. Specify the Latency information in this field.
  - **II:** Define the target II (Initiation Interval). This is the number of clocks cycles before new input can be applied.
- **Resource:** Specify the device resource utilization for the RTL IP. The resource information provided here will be combined with utilization from synthesis to report the overall design resource utilization. You should be able to extract this information from the Vivado Design Suite

Click **Next** to proceed to the RTL Common Signal page, as shown below.

Figure 97: **RTL Common Signals**



- **module_clock:** Specify the name the of the clock used in the RTL IP.
- **module_reset:** Specify the name of the reset signal used in the IP.
- **module_clock_enable:** Specify the name of the clock enable signal in the IP.
- **ap_ctrl_chain_protocol_start:** Specify the name of the block control start signal used in the IP.
- **ap_ctrl_chain_protocol_ready:** Specify the name of the block control ready signal used in the IP.
- **ap_ctrl_chain_protocol_done:** Specify the name of the block control done signal used in the IP.
- **ap_ctrl_chain_protocol_continue:** Specify the name of the block control continue signal used in the RTL IP.

Click **Finish** to automatically generate a JSON file for the specified IP. This can be confirmed through the log message as shown below.

Log Message:

```
"[2019-08-29 16:51:10] RTL Blackbox Wizard Information: the "foo.json" file
has been created in the rtl_blackbox/Source folder."
```

The JSON file can be accessed through the Source file folder, and will be generated as described in the next section.

JSON File for RTL Blackbox

JSON File Format

The following table describes the JSON file format:

Table 24: **JSON File Format**

Item	JSON Attribute	Description
c_function_name		The C++ function name for the blackbox. The `c_function_name` must be consistent with the C function simulation model.
rtl_top_module_name		The RTL function name for the blackbox. The `rtl_top_module_name` must be consistent with the `c_function_name`.
c_files	c_file	Specifies the C file used for the blackbox module.
	cflag	Provides any compile option necessary for the corresponding C file.
rtl_files		Specifies the RTL files for the blackbox module.

Table 24: **JSON File Format** *(cont'd)*

Item	JSON Attribute	Description
c_parameters	c_name	Specifies the name of the argument used for the black box C++ function. Unused `c_parameters` should be deleted from the template.
	c_port_direction	The access direction for the corresponding C argument. • **`in`:** Read only by blackbox C++ function. • **`out`:** Write only by blackbox C++ function. • **`inout`:** Will both read and write by blackbox C++ function.
	RAM_type	Specifies the RAM type to use if the corresponding C argument uses the RTL RAM protocol. Two type of RAM are used: • **RAM_1P:** For 1 port RAM module • **RAM_T2P:** For 2 port RAM module Omit this attribute when the corresponding C argument is not using RTL 'RAM' protocol.
	rtl_ports	Specifies the RTL port protocol signals for the corresponding C argument (`c_name`). Every `c_parameter` should be associated with an `rtl_port`. Five type of RTL port protocols are used, and only wire and FIFO are supported for the ap_ctrl_none control protocol. Refer to the *RTL Port Protocols* table for additional details. • **`wire`:** An argument can be mapped to `wire` if it is a either a scalar with in or out direction, or a pointer with 'in' direction. • **`ap_vld`:** An argument can be mapped to `ap_vld` if it uses pointer with 'out' direction. • **`ap_ovld`:** An argument can be mapped to `ap_ovld` if it use a pointer with an inout direction. • **`FIFO`:** An argument can be mapped to `FIFO` if it uses the `hls::stream` data type. • **`RAM`:** An argument can be mapped to `RAM` if it uses an array type. The array type supports inout directions. The specified RTL port protocols have associated control signals, which also need to be specified in the JSON file.
c_return	c_port_direction	It must be `out`.
	rtl_ports	Specifies the corresponding RTL port name used in the RTL blackbox IP.

Table 24: **JSON File Format** *(cont'd)*

Item	JSON Attribute	Description
rtl_common_signal	module_clock	The unique clock signal for RTL blackbox module.
	module_reset	Specifies the reset signal for RTL blackbox module. The reset signal must be active-High or positive valid.
	module_clock_enable	Specifies the clock enable signal for the RTL blackbox module. The enable signal must be active-High or positive valid.
	ap_ctrl_chain_protocol_idle	The `ap_idle` signal in the `ap_ctrl_chain` protocol for the RTL blackbox module. This signal must be an empty string for the ap_ctrl_none control protocol.
	ap_ctrl_chain_protocol_start	The `ap_start` signal in the `ap_ctrl_chain` protocol for the RTL blackbox module. This signal must be an empty string for the ap_ctrl_none control protocol.
	ap_ctrl_chain_protocol_ready	The `ap_ready` signal in the `ap_ctrl_chain` protocol for the RTL blackbox IP. This signal must be an empty string for the ap_ctrl_none control protocol.
	ap_ctrl_chain_protocol_done	The `ap_done` signal in the `ap_ctrl_chain` protocol for blackbox RTL module. This signal must be an empty string for the ap_ctrl_none control protocol.
	ap_ctrl_chain_protocol_continue	The `ap_continue` signal in the `ap_ctrl_chain` protocol for RTL blackbox module. This signal must be an empty string for the ap_ctrl_none control protocol.
rtl_performance	latency	Specifies the Latency of the RTL blackbox module. It must be a non-negative integer value. For Combinatorial RTL IP specify `0`, otherwise specify the exact latency of the RTL module.
	II	Number of clock cycles before the function can accept new input data. It must be non-negative integer value. `0` means the blackbox can not be pipelined. Otherwise, it means the blackbox module is pipelined.
rtl_resource_usage	FF	Specifies the register utilization for the RTL blackbox module.
	LUT	Specifies the LUT utilization for the RTL blackbox module.
	block RAM	Specifies the block RAM utilization for the RTL blackbox module.
	URAM	Specifies the URAM utilization for the RTL blackbox module.
	DSP	Specifies the DSP utilization for the RTL blackbox module.

Table 25: **RTL Port Protocols**

RTL Port Protocol	C Port Direction	JSON Attribute	RTL Port Direction	User-Defined Name	Notes
wire	in	data_read_in	in	Specifies a user defined name used in the RTL blackbox IP. As an example for wire, if the RTL port name is "flag" then the JSON FILE format is `"data_read-in" : "flag"`.	
ap_vld	out	data_write_out	out		
		data_write_valid	out		
ap_ovld	inout	data_read_in	in		
		data_write_out	out		
		data_write_valid	out		
FIFO	in	FIFO_empty_flag	in		Must be negative valid.
		FIFO_read_enable	out		
		FIFO_data_read_in	in		
	out	FIFO_full_flag	in		Must be negative valid.
		FIFO_write_enable	out		
		FIFO_data_write_out	out		
RAM (RAM_1P)	in	RAM_address	out		
		RAM_clock_enable	out		
		RAM_data_read_in	in		
	out	RAM_address	out		
		RAM_clock_enable	out		
		RAM_write_enable	out		
		RAM_data_write_out	out		
	inout	RAM_address	in		
		RAM_clock_enable	in		
		RAM_write_enable	out		
		RAM_data_write_out	out		
		RAM_data_read_in	in		

Table 25: **RTL Port Protocols** *(cont'd)*

RTL Port Protocol	C Port Direction	JSON Attribute	RTL Port Direction	User-Defined Name	Notes
RAM (RAM_T2P)	in	RAM_address	out	Specifies a user defined name used in the RTL blackbox IP. As an example for wire, if the RTL port name is "flag" then the JSON FILE format is `"data_read-in" : "flag"`.	Signals with `_snd` belong to the second port of the RAM. Signals without `_snd` belong to the first port.
		RAM_clock_enable	out		
		RAM_data_read_in	in		
		RAM_address_snd	out		
		RAM_clock_enable_snd	out		
		RAM_data_read_in_snd	in		
	out	RAM_address	out		
		RAM_clock_enable	out		
		RAM_write_enable	out		
		RAM_data_write_out	out		
		RAM_address_snd	out		
		RAM_clock_enable_snd	out		
		RAM_write_enable_snd	out		
		RAM_data_write_out_snd	out		
	inout	RAM_address	out		
		RAM_clock_enable	out		
		RAM_write_enable	out		
		RAM_data_write_out	out		
		RAM_data_read_in	in		
		RAM_address_snd	out		
		RAM_clock_enable_snd	out		
		RAM_write_enable_snd	out		
		RAM_data_write_out_snd	out		
		RAM_data_read_in_snd	in		

***Note*:** The behavioral C-function model for the RTL blackbox must also adhere to the recommended HLS coding styles.

JSON File Example

This section provides details on manually writing the JSON file required for the RTL blackbox. The following is an example of a JSON file:

```
{
"c_function_name" : "foo",
"rtl_top_module_name" : "foo",
"c_files" :
      [
            {
              "c_file" : "../../a/top.cpp",
              "cflag" : ""
```

```
            },
            {
              "c_file" : "xx.cpp",
              "cflag" : "-D KF"
            }
          ],
"rtl_files" : [
                "../../foo.v",
                "xx.v"
              ],
"c_parameters" : [{
                   "c_name" : "a",
                   "c_port_direction" : "in",
                   "rtl_ports" : {
                                   "data_read_in" : "a"
                                 }
                  },
                  {
                   "c_name" : "b",
                   "c_port_direction" : "in",
                   "rtl_ports" : {
                                   "data_read_in" : "b"
                                 }
                  },
                  {
                   "c_name" : "c",
                   "c_port_direction" : "out",
                   "rtl_ports" : {
                                   "data_write_out" : "c",
                                   "data_write_valid" : "c_ap_vld"
                                 }
                  },
                  {
                   "c_name" : "d",
                   "c_port_direction" : "inout",
                   "rtl_ports" : {
                                   "data_read_in" : "d_i",
                                   "data_write_out" : "d_o",
                                   "data_write_valid" : "d_o_ap_vld"
                                 }
                  },
                  {
                   "c_name" : "e",
                   "c_port_direction" : "in",
                   "rtl_ports" : {
                                   "FIFO_empty_flag" : "e_empty_n",
                                   "FIFO_read_enable" : "e_read",
                                   "FIFO_data_read_in" : "e"
                                 }
                  },
                  {
                   "c_name" : "f",
                   "c_port_direction" : "out",
                   "rtl_ports" : {
                                   "FIFO_full_flag" : "f_full_n",
                                   "FIFO_write_enable" : "f_write",
                                   "FIFO_data_write_out" : "f"
                                 }
                  },
                  {
                   "c_name" : "g",
                   "c_port_direction" : "in",
                   "RAM_type" : "RAM_1P",
```

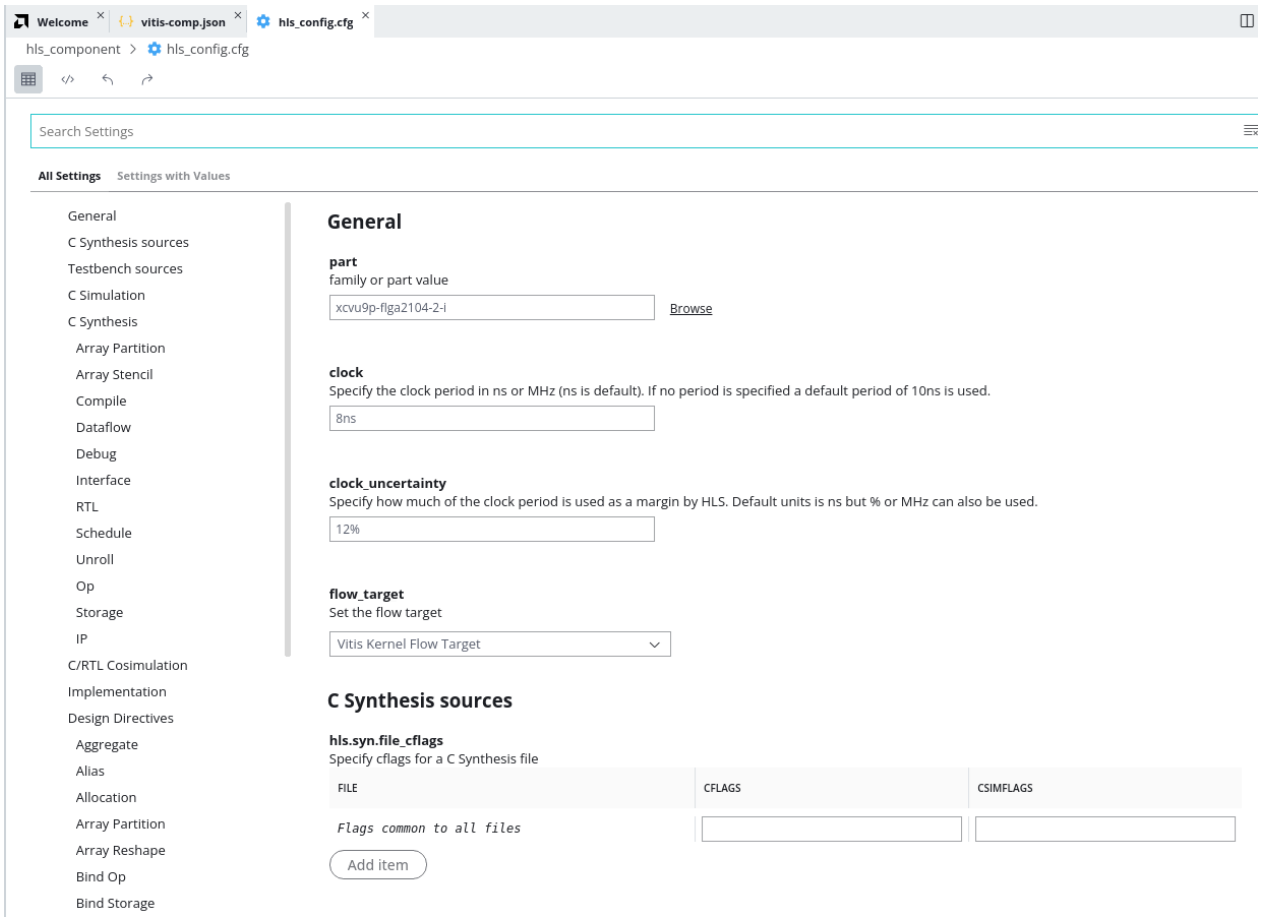
```
                "rtl_ports" : {
                                "RAM_address" : "g_address0",
                                "RAM_clock_enable" : "g_ce0",
                                "RAM_data_read_in" : "g_q0"
                              }
              },
              {
                "c_name" : "h",
                "c_port_direction" : "out",
                "RAM_type" : "RAM_1P",
                "rtl_ports" : {
                                "RAM_address" : "h_address0",
                                "RAM_clock_enable" : "h_ce0",
                                "RAM_write_enable" : "h_we0",
                                "RAM_data_write_out" : "h_d0"
                              }
              },
              {
                "c_name" : "i",
                "c_port_direction" : "inout",
                "RAM_type" : "RAM_1P",
                "rtl_ports" : {
                                "RAM_address" : "i_address0",
                                "RAM_clock_enable" : "i_ce0",
                                "RAM_write_enable" : "i_we0",
                                "RAM_data_write_out" : "i_d0",
                                "RAM_data_read_in" : "i_q0"
                              }
              },
              {
                "c_name" : "j",
                "c_port_direction" : "in",
                "RAM_type" : "RAM_T2P",
                "rtl_ports" : {
                                "RAM_address" : "j_address0",
                                "RAM_clock_enable" : "j_ce0",
                                "RAM_data_read_in" : "j_q0",
                                "RAM_address_snd" : "j_address1",
                                "RAM_clock_enable_snd" : "j_ce1",
                                "RAM_data_read_in_snd" : "j_q1"
                              }
              },
              {
                "c_name" : "k",
                "c_port_direction" : "out",
                "RAM_type" : "RAM_T2P",
                "rtl_ports" : {
                                "RAM_address" : "k_address0",
                                "RAM_clock_enable" : "k_ce0",
                                "RAM_write_enable" : "k_we0",
                                "RAM_data_write_out" : "k_d0",
                                "RAM_address_snd" : "k_address1",
                                "RAM_clock_enable_snd" : "k_ce1",
                                "RAM_write_enable_snd" : "k_we1",
                                "RAM_data_write_out_snd" : "k_d1"
                              }
              },
              {
                "c_name" : "l",
                "c_port_direction" : "inout",
                "RAM_type" : "RAM_T2P",
                "rtl_ports" : {
                                "RAM_address" : "l_address0",
```

```
                                    "RAM_clock_enable" : "l_ce0",
                                    "RAM_write_enable" : "l_we0",
                                    "RAM_data_write_out" : "l_d0",
                                    "RAM_data_read_in" : "l_q0",
                                    "RAM_address_snd" : "l_address1",
                                    "RAM_clock_enable_snd" : "l_ce1",
                                    "RAM_write_enable_snd" : "l_we1",
                                    "RAM_data_write_out_snd" : "l_d1",
                                    "RAM_data_read_in_snd" : "l_q1"
                                  }
                    }],
"c_return" : {
                "c_port_direction" : "out",
                "rtl_ports" : {
                                    "data_write_out" : "ap_return"
                                  }
              },
"rtl_common_signal" : {
                         "module_clock" : "ap_clk",
                         "module_reset" : "ap_rst",
                         "module_clock_enable" : "ap_ce",
                         "ap_ctrl_chain_protocol_idle" : "ap_idle",
                         "ap_ctrl_chain_protocol_start" : "ap_start",
                         "ap_ctrl_chain_protocol_ready" : "ap_ready",
                         "ap_ctrl_chain_protocol_done" : "ap_done",
                         "ap_ctrl_chain_protocol_continue" : "ap_continue"
                       },
"rtl_performance" : {
                      "latency" : "6",
                      "II" : "2"
                    },
"rtl_resource_usage" : {
                         "FF" : "0",
                         "LUT" : "0",
                         "BRAM" : "0",
                         "URAM" : "0",
                         "DSP" : "0"
                       }
}
```

Defining the HLS Config File

The HLS component configuration file contains commands and settings used for the synthesis, simulation, and export of the component. To access the config file select the **Config File** hyperlink on the `vitis-comp.json` tab for the HLS component, or select `hls_config.cfg` in the Vitis Component Explorer view, under Settings of the HLS component. This will open the Config File Editor in the central editor window as displayed below.

Figure 98: **HLS Component Configuration File**



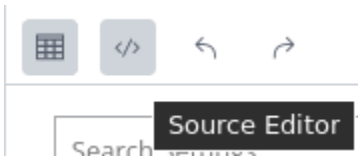
For the HLS component the config file includes the following sections:

- General: Provides general options for the design, such as target part, clock frequency, and flow target. These options are described at HLS General Options.
- C Synthesis sources: Specify C/C++ source files for the HLS design. These are also described in Running C Synthesis.
- Testbench sources: Specify C/C++ test bench files and associated files used during C simulation and C/RTL Co-simulation.
- C Simulation: Specify compilation options for running C simulation as described in Running C Simulation .
- C Synthesis Settings: Specify global synthesis controls for the following categories: Array Partition, Array Stencil, Compile, Dataflow, Debug, Interface, RTL, Schedule, Unroll, Op, Storage, IP. These configuration settings are related to the config file commands described in HLS Config File Commands.
- C/RTL Cosimulation: Provide compilation options for running C/RTL Co-simulation as described in Running C/RTL Co-Simulation.

- Package: Specify options to control the export format of the RTL generated during C synthesis as described in Packaging the RTL Design.
- Implementation: Specify options to control Vivado synthesis and implementation as described in Running Implementation.
- Design Directives: Directives can be added as HLS Pragmas directly to the source code, or as HLS Optimization Directives to the configuration file for use in an HLS component. In the Config File editor you will see the various directives listed, and you can select **Add Item** to open the Directive editor as described in Adding Pragmas and Directives.

The Config File Editor offers both a form-based view of the config file options, and a text-based editor that can be enabled to edit the config file directly. You can switch between the two by using the icons at the top of the editor window.

Figure 99: **Form-Based and Text-Based Editing**



There is a Search bar at the top of the Config File Editor which lets you quickly search through the options using a keyword or phrase. You can also choose to display only the **Settings with Values** to limit the displayed options.

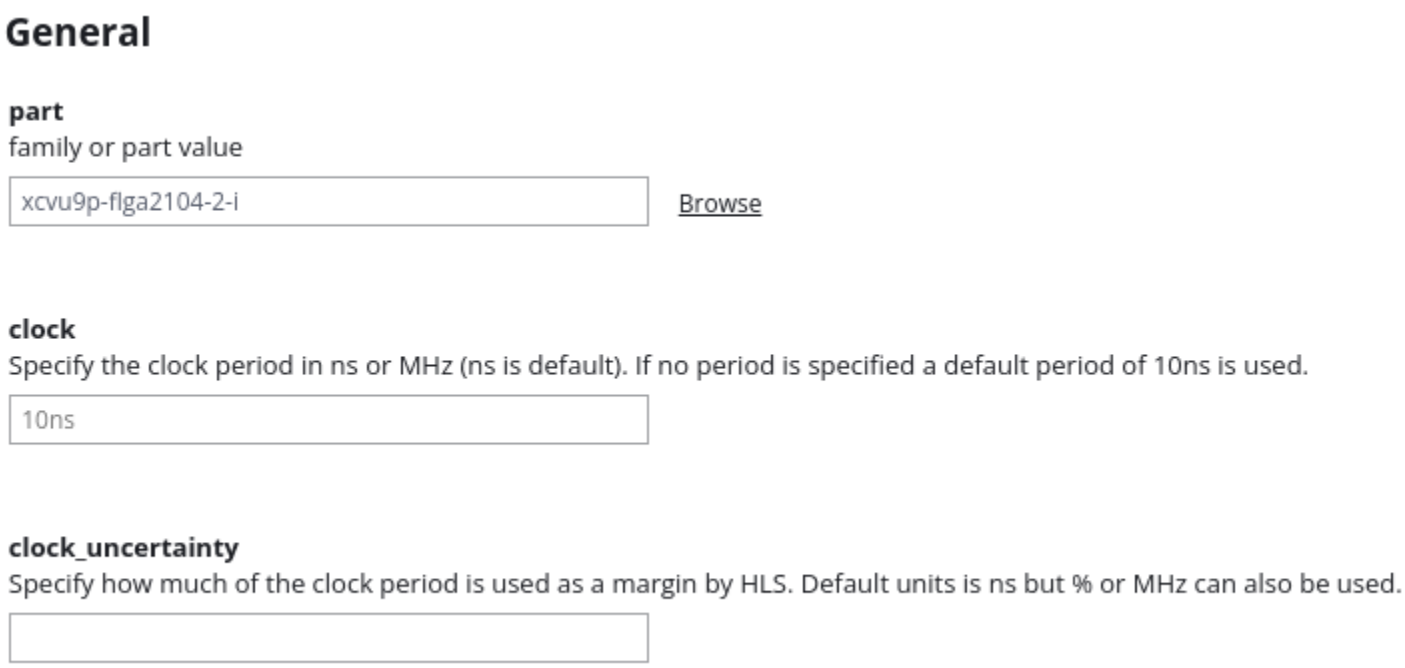
TIP: *Remember to enable the **File→AutoSave** feature or select the **File→Save** command, or `Ctrl-S` to save your changes after editing the config file.*

Specifying the Clock Frequency

For C and C++ designs only a single clock is supported. The same clock is applied to all functions in the design.

The clock period, specified in nanoseconds (ns) or MHz, is defined when the HLS components is created, or can be changed in the HLS config file under the General category as shown below.

Figure 100: **Part and Clock Settings**



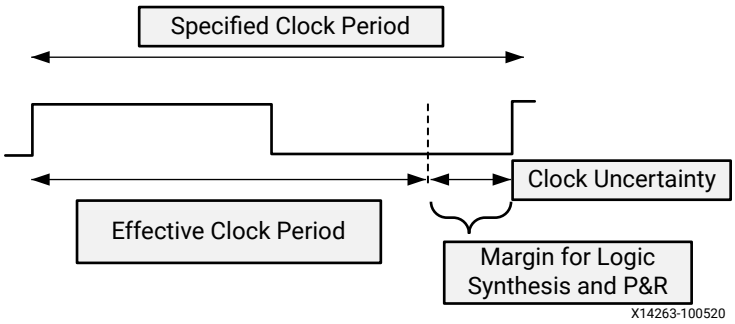
The default clock period is 10 ns. The HLS component also uses the concept of a clock uncertainty to provide a user defined timing margin for scheduling processes within the design. You can define the clock uncertainty for your design in the same way and at the same time as you specify the clock. The default clock uncertainty, when it is not specified, is 27% of the clock period.

IMPORTANT! *If your HLS configuration file specifies* `platform=` *instead of* `part=` *you must use also* `freqhz=` *instead of* `clock=`*. This is because the platform specifies a clock in* `freqhz` *form, and you can only override it using the same form.*

Using the clock frequency and part information for the HLS component the tool estimates the timing of operations in the design. However, it cannot know the final component placement and net routing as these operations are performed by logic synthesis of the output RTL. As such, the tool can only estimated delays.

To calculate the clock period used for synthesis the tool subtracts the clock uncertainty from the clock period, as shown in the following figure.

Figure 101: **Clock Period and Margin**



This provides a user specified margin to ensure downstream processes, such as logic synthesis and place & route, have enough timing margin to complete their operations. If the resources of the device are mostly consumed by the HLS componet the placement of cells and routing of nets to connect the cells might tight and result in a design with larger than expected timing delays. For situations such as this increasing the clock uncertainty can ensure that the HLS component does not create a design with too much logic packed into each clock cycle and allows RTL synthesis to satisfy timing in cases with less than ideal placement and routing options.

The Vitis unified IDE and `v++` command tries to satisfy all constraints: timing, throughput, latency. However, if a constraint cannot be satisfied the tool still produces an RTL design. If the timing constraints inferred by the clock period cannot be met the tool issues message `SCHED-644` as shown below, and creates a design with the best achievable performance.

```
@W [SCHED-644] Max operation delay (<operation_name> 2.39ns) exceeds the
effective
cycle time
```

IMPORTANT! *It is important to review the constraint report after synthesis to determine if all constraints are met: the fact that synthesis produces an output design does not guarantee the design meets all performance constraints. Review the Performance Estimates section of the design report.*

A design report is generated for each function in the hierarchy when synthesis completes and can be viewed in the solution reports folder. The worse case timing for the entire design is reported as the worst case in each function report. There is no need to review every report in the hierarchy.

If the timing violations are too severe to be further optimized and corrected by downstream processes, review the techniques for specifying an exact latency and specifying exact implementation cores before considering a faster target technology.

Clock and Reset Ports

If the design takes more than 1 cycle to complete operation, as most do, a clock-enable port (`ap_ce`) can optionally be added to the config file for the design using the `syn.interface.clock_enable` command as described in Interface Configuration.

The operation of the reset is described in Controlling Initialization and Reset Behavior, and can be modified using the RTL Configuration settings.

Adding Pragmas and Directives

HLS pragmas and directives let you configure the synthesis results for your code.

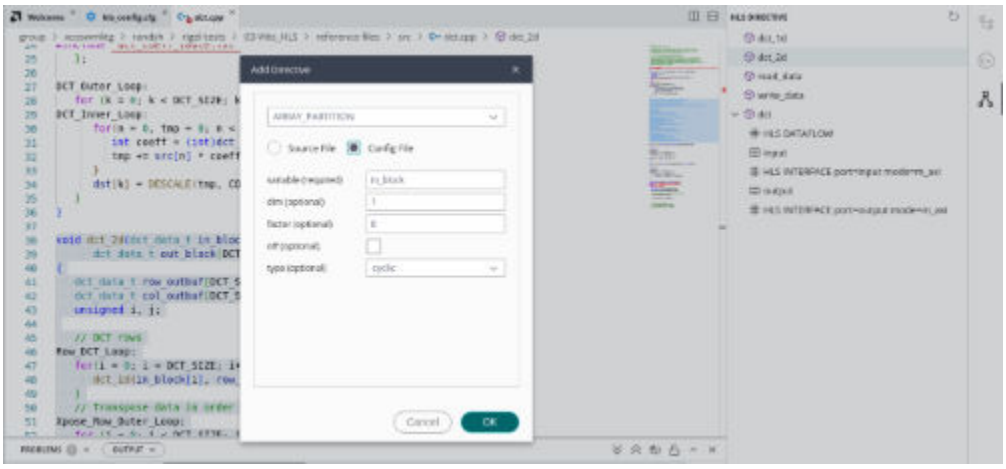
- HLS Pragmas are added to the source code to enable the optimization or change in the original source code. Every time the code is synthesized, it is implemented according to the specified pragmas.
- HLS Optimization Directives can be specified as HLS configuration file commands that are associated with the HLS component. The config files can be reused across multiple HLS components, allowing you to customize the synthesis results for the same code base across different components.

IMPORTANT! *In some cases where pragmas or directives conflict with other pragmas or directives, the synthesis process returns an error until you resolve the conflict. However, in some cases the first pragma or directive encountered takes precedence over the second pragma or directive, and the second is ignored. This information should be reported in the log file or console window.*

To add pragmas or directives to your project:

1. In the Vitis Component Explorer open a Source file of the HLS component by selecting it under the Sources folder.
2. With the Source file opened, select the HLS Directive command ( ) from the far right toolbar menu of the Vitis unified IDE to open the HLS Directive editor, as shown in the following figure.

Figure 102: **HLS Directive Editor**

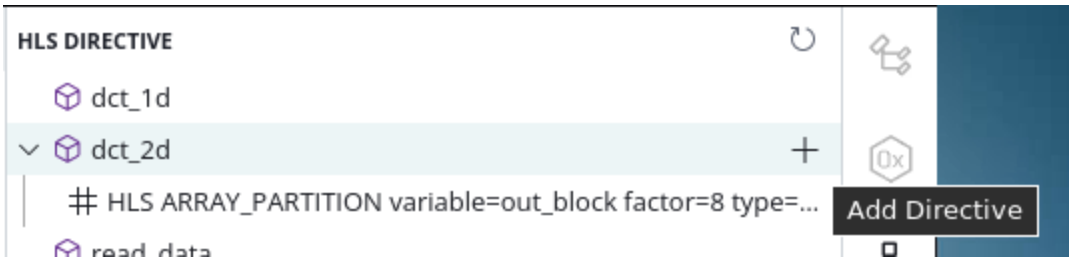


3. The HLS Directive editor displays a hierarchy of the current source code and lets you add a directive or pragma to the design; directives go into the config file and pragmas go into the source file. In the preceding figure, the entry that is added to the config file is:

   ```
   syn.directive.array_partition=dct_2d in_block factor=8 type=cyclic
   ```

   After selecting **OK**, you will see it added to the Config File editor.

4. Select the **Add Directive** command from the HLS Directive editor by selecting the function or loop or variable you want to add a directive to and select the '+' on the right side as shown in the following figure.



   Modify an existing directive by selecting the **Edit Directive** command next to the directive.

Using Directives in Scripts vs. Pragmas in Code

In the HLS Directive editor you can specify either of the following Destination settings:

- **Source File:** The Vitis Unified IDE inserts the directive directly into the C source file as a pragma.
- **Config File:** The IDE inserts the directive into the HLS config file.

The following table describes the advantages and disadvantages of both approaches.

Table 26: **Config File Directive versus HLS Pragma**

Directive Format	Advantages	Disadvantages
HLS Config File	The Source code remains unchanged, while design exploration is handled through the config file entries. This approach is ideal for design exploration.	If the C source files are transferred to a third-party or archived, the `directives.tcl` file must be included. The `directives.tcl` file is required if the results are to be re-created.
Source Code Pragma	The optimization directives are embedded into the C source code. Ideal when the C sources files are shipped to a third-party as C IP. No other files are required to recreate the same results. Useful approach for directives that are unlikely to change, such as TRIPCOUNT and INTERFACE.	If the optimization directives are embedded in the code, they are automatically applied to every HLS component that references that source code.

> **TIP:** *You can use directives to develop different design possibilities, but you might want to convert your final directives to HLS pragmas when finalized. This allows the optimization pragmas to be carried as part of the source code for the design.*

When specifying values for pragma arguments, you can use literal values (for example, 1, 55, 3.14), or pass a macro using #define. The following example shows a pragma with literal values:

```
#pragma HLS ARRAY_PARTITION variable=k_matrix_val  type=cyclic factor=5
```

This example uses defined macros:

```
#define E 5
#pragma HLS ARRAY_PARTITION variable=k_matrix_val  type=cyclic factor=E
```

Applying Directives to the Proper Scope

Although the Vitis Unified IDE lets you apply directives to specific code objects, the directives are added to the scope that contains the object. For example, you can apply the `INTERFACE` pragma to an interface object in the HLS component, but the directive is applied to the top-level function (scope). The interface port (object) is identified in the directive.

You can apply optimization directives to the following objects and scopes:

- **Functions:** When you apply directives to functions, the HLS compiler applies the directive to all objects within the scope of that function. The effect of any directive stops at the next level of the function hierarchy, and does not apply to sub-functions.

  > **TIP:** *Directives that include a recursive option, such as the `PIPELINE` directive, can be applied recursively through the hierarchy.*

- **Interfaces:** The HLS compiler applies the directive to the top-level function, which is the scope that contains the interface.
- **Loops:** Directives apply to all objects within the scope of the loop.

  For example, if you apply the `LOOP_MERGE` directive to a loop, the HLS compiler applies the directive to any sub-loops within the loop, but not to the loop itself. The loop to which the directive is applied is not merged with siblings at the same level of hierarchy.

- **Arrays:** Directives are applied to the scope that contains the array.

Applying Optimization Directives to Global Variables

Directives can only be applied to scopes, or to objects within a scope. As such, they cannot be directly applied to global variables which are declared outside the scope of any function. Therefore, to apply a directive to a global variable you must manually assign it using the following process:

1. With the source code, open the HLS Directive editor

2. Select the scope (function, loop or region) where the global variable is used in the Directive editor.
3. Select **Add Directive** and edit the `variable` name to assign it to the global variable.
4. Click **OK** to add the directive to the source code or config file.

Applying Optimization Directives to Class Objects

Optimization directives can be also applied to objects or scopes defined in a class. The difference is typically that classes are defined in a header file. Use one of the following actions to open the header file:

1. From the Vitis Component Explorer view open the `Includes` folder for an HLS component and double-click the header file to open it. Or, from within an open source code file, place the cursor over the `#include` statement for the header file, hold down the **Ctrl** key, and click the header file to open it in the Code Editor.
2. With the source code, open the HLS Directive editor
3. Select the scope (function, loop or region) where the global variable is used in the Directive editor.
4. Select **Add Directive** and edit the `variable` name to assign it to the class or variable.
5. Click **OK** to add the directive to the source code or config file.

> **CAUTION!** *Care should be taken when applying directives as pragmas to a header file. The file might be used by other people or used in other projects. Any directives added as a pragma are applied each time the header file is included in a design.*

Applying Optimization Directives to Templates

To apply optimization directives manually on templates when using config file commands, specify the template arguments and class when referring to class methods. For example, given the following C++ code:

```
template <uint32 SIZE, uint32 RATE>
void DES10<SIZE,RATE>::calcRUN() {}
```

The following config file command is used to specify the INLINE directive on the function:

```
syn.directive.inline DES10<SIZE,RATE>::calcRUN
```

Using Constants with Pragmas

You can use a constant such as `const int`, or `constexpr` with pragmas or directives. For example:

```
const int MY_DEPTH=1024;
#pragma HLS stream variable=my_var depth=MY_DEPTH
```

You can also use macros in the C code to implement this functionality. The key to using macros is to use a level of hierarchy in the macro. This allows the expansion to be correctly performed. The code can be made to compile as follows:

```
#include <hls_stream.h>
using namespace hls;

#define PRAGMA_SUB(x) _Pragma (#x)
#define PRAGMA_HLS(x) PRAGMA_SUB(x)
#define STREAM_IN_DEPTH 8

void foo (stream<int> &InStream, stream<int> &OutStream) {

// Legal pragmas
PRAGMA_HLS(HLS stream depth=STREAM_IN_DEPTH variable=InStream)
#pragma HLS stream depth=8 variable=OutStream

}
```

Failure to Satisfy Optimization Directives

When optimization directives are applied, the HLS compiler outputs information to the console (and log file) detailing the progress. In the following example the PIPELINE directives was applied to the C function with an II=1 (initiation interval of 1) but synthesis failed to satisfy this objective.

```
INFO: [SCHED 11] Starting scheduling ...
INFO: [SCHED 61] Pipelining function 'array_RAM'.
WARNING: [SCHED 63] Unable to schedule the whole 2 cycles 'load' operation
('d_i_load', array_RAM.c:98) on array 'd_i' within the first cycle (II = 1).
WARNING: [SCHED 63] Please consider increasing the target initiation
interval of the
pipeline.
WARNING: [SCHED 69] Unable to schedule 'load' operation ('idx_load_2',
array_RAM.c:98) on array 'idx' due to limited memory ports.
INFO: [SCHED 61] Pipelining result: Target II: 1, Final II: 4, Depth: 6.
INFO: [SCHED 11] Finished scheduling.
```

IMPORTANT! *If the HLS tool fails to satisfy an optimization directive, it automatically relaxes the optimization target and seeks to create a design with a lower performance target. If it cannot relax the target, it will halt with an error.*

By seeking to create a design which satisfies a lower optimization target, the tool is able to provide three important types of information:

- What target performance can be achieved with the current C code and optimization directives.
- A list of the reasons why it was unable to satisfy the higher performance target.
- A design which can be analyzed to provide more insight and help understand the reason for the failure.

In message `SCHED-69` above the reason given for failing to reach the target II is due to limited ports. The design must access a block RAM, and a block RAM only has a maximum of two ports.

The next step after a failure such as this is to analyze what the issue is. In this example, analyze line 52 of the code and/or use the **Analysis** perspective to determine the bottleneck and if the requirement for more than two ports can be reduced or determine how the number of ports can be increased.

After the design is optimized and the desired performance achieved, the RTL can be verified and the results of synthesis packaged as IP.

Running C Simulation

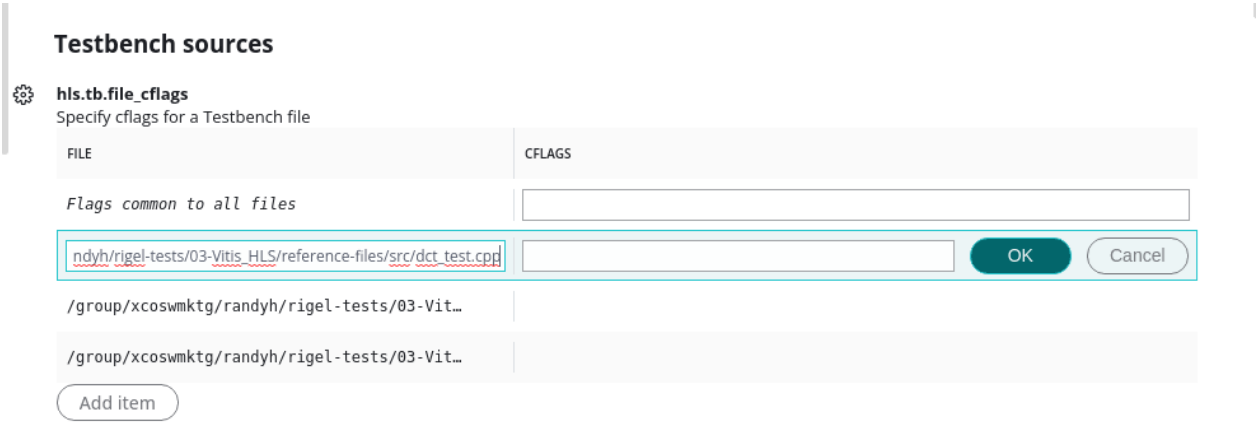
Verification of the HLS component can be separated into distinct processes that occur before and after high-level synthesis. These processes include C Simulation and Code Analysis to validate the C/C++ source code prior to high-level synthesis; and C/RTL Co-simulation to validate the generated RTL design.

Running C simulation on the HLS component requires a test bench as described in Writing a Test Bench. The test bench is self-checking, and validates that the results from the function to be synthesized are correct. If the results are correct the test bench returns a value of 0 to `main()`. Otherwise, the test bench should return any non-zero value. Prior to running simulation though, you must configure the HLS component to support simulation. C Simulation can be run by itself or with the Code Analyzer enabled. By itself, C Simulation validates the logic of the source code using a test bench and provided data. With Code Analyzer the simulation to validate the algorithm is still completed, but a deeper analysis of the source code and prospective performance of the design is also performed. Code Analyzer lets you visualize the potential for task level parallelism, estimate the potential performance of the design, and understand the architectural changes needed to improve the performance of the design.

Loading Test Bench Files

Like C synthesis, the source files required for simulation can be loaded at the time the HLS component is created. However, you can skip that step at creation and add the test bench files prior to running simulation. It is also possible to change or add to the files used for simulation. This is done in the Testbench sources section of the Config Editor.

Figure 103: **HLS Component Testbench Sources**



The Testbench sources displays currently added test bench files, and lets you edit, or delete current files, add new files, and add or modify CFLAGS for the test bench.

- Add item: Select this command to add new test bench files to the HLS component. This opens a File Browser, and lets you navigate to and select the files to add.
- Add CFLAGS: You can add compilation flags (CFLAGS) to be applied to all source files, or to be added to specific source files. As shown in the figure above, simply add the flags to the appropriate text entry box for Flags common to all files. To add flags for specific source files, select the source file and select **Edit item** to add the flags, or modify the file name and path.

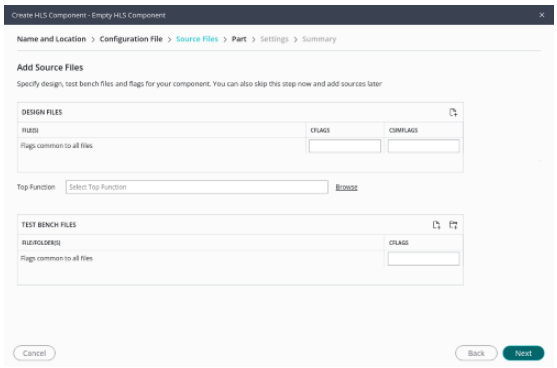
These selections result in the following config file entries:

```
[HLS]
tb.file=../../src/in.dat
tb.file=../../src/out.golden.dat
tb.file=../../src/dct_test.cpp
tb.cflags=<cflag>
tb.file_cflags=../../src/dct_test.cpp,<cflag>
```

Configure the Simulator

You can configure the C simulator prior to running simulation using the C Simulation section of the Config Editor, as shown below.

Figure 104: **HLS Component Simulation Settings**



These configuration commands let you specify how the simulation should run. The configuration options include:

- **O:** By default the design is compiled with debug information enabled, allowing the compilation to be analyzed and debugged. The Optimizing Compile option uses a higher level of optimization effort when compiling the design, but does not add information required by the debugger. This increases the compile time but should reduce the simulation runtime.
- **argv:** Specify any inputs required by your test bench `main()` function.
- **clean:** Remove any existing executable and object files from the project before compiling the code.
- **code_analyzer:** Enable the Code Analyzer tool as described in Using Code Analyzer.
- **ldflags:** Specifies the options passed to the linker for simulation. This option is typically used to pass include path information or library information for the C/C++ test bench.
- **mflags:** Provides for options to be passed to the compiler for C simulation. This is typically used to speed up compilation.
- **setup:** When this option is specified, the simulation binary will be created in the `csim` directory of the current HLS component, but simulation will not be executed. Simulation can be launched later from the compiled executable. The default is false, and simulation is run after setup is complete.

These selections result in the following config file entries as an example:

```
[HLS]
csim.clean=true
csim.O=true
csim.code_analyzer=0
csim.argv=arg1 arg2
```

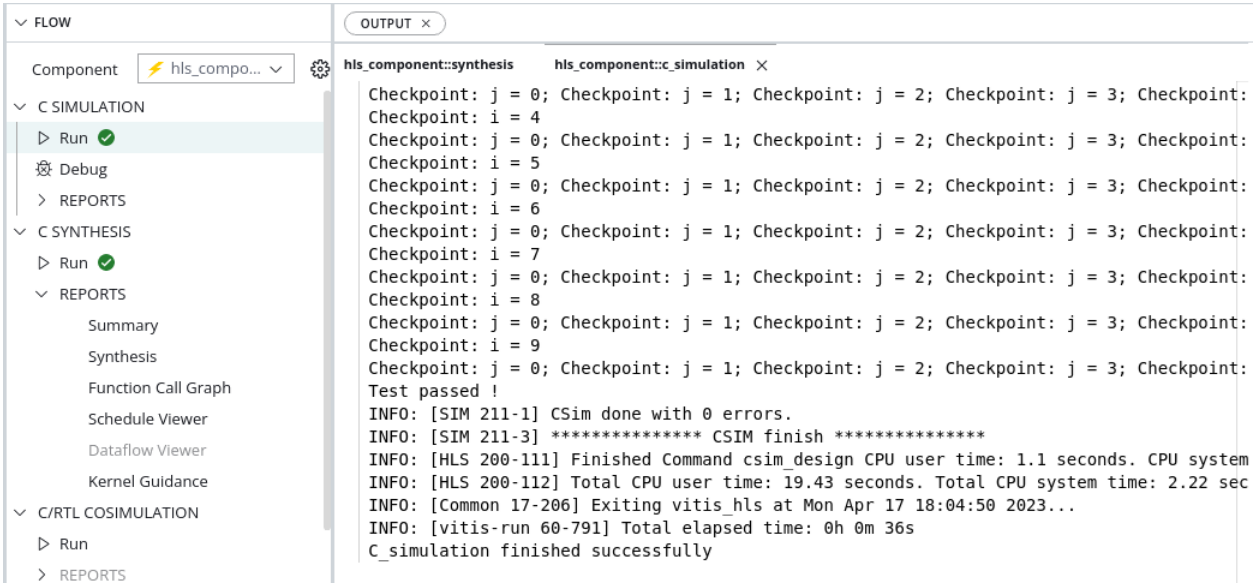
Run Simulation

To run simulation make sure the HLS component is active in the Flow Navigator, or select it from the Component menu to make it the active component in the tool. When the HLS component is the active component, the Flow Navigator enables running C Synthesis, C Simulation, C/RTL Co-simulation, Packaging, and Implementation to build and analyze the HLS component. To simulate the HLS component select **Run** beneath the C SIMULATION heading in the Flow Navigator.

TIP: *You can also select the **Debug** command from the Flow Navigator to launch the debug process as described in Debugging the HLS Component.*

You can track the progress of simulation in the Output window. The transcript for the synthesis run will have the top function name as `<component-name>::c_simulation` as shown below.

Figure 105: **HLS Component Running Simulation**



TIP: *The simulation run in the Vitis IDE uses the `vitis-run --mode hls --csim` command as described in vitis, v++, and vitis-run Commands.*

Results of C Simulation

After clicking **OK** in the dialog box, the C/C++ code is compiled and the C simulation is run. As the simulation runs, the console displays any `printf` statements from the test bench, or `hls::print` statements from the kernel or IP. When the simulation completes successfully, the following message is also returned to the console:

```
INFO: [SIM 211-1] CSim done with 0 errors.
INFO: [SIM 211-3] *************** CSIM finish ***************
Finished C simulation.
```

When the simulation fails, an error is returned:

```
@E Simulation failed: Function 'main' returns nonzero value '1'.
ERROR: [SIM 211-100] 'csim_design' failed: nonzero return value.
INFO: [SIM 211-3] *************** CSIM finish ***************
```

When the simulation is complete a Summary report is generated which shows the simulation command and the time stamp of the run. You can view the Summary report and simulation log file by selecting them freom the Report heading, or by viewing them in the Analysis view. If Code Analyzer was also run there will be a report for that as well.

Writing a Test Bench

When using the HLS component flow, it can be time consuming to synthesize an improperly coded C/C++ function and then analyze the implementation details to determine why the function does not perform as expected. Therefore, the first step in high-level synthesis should be to validate that the C function is correct, before generating RTL code, by performing simulation using a well written test bench. Writing a good test bench can greatly increase your productivity because C functions execute in orders of magnitude faster than RTL simulations. Using C/C++ to develop and validate the algorithm before synthesis is much faster than developing and debugging RTL code.

The test bench includes the `main()` function, as well as any needed sub-functions that are not in the hierarchy of the top-level function designated for synthesis. The main function verifies that the top-level function for synthesis is correct by providing stimuli and calling the function from the test bench, and by consuming and validating its output.



IMPORTANT! *The test bench can accept input arguments that can be provided when C simulation is launched, as described in Running C Simulation. However, the test bench must not require interactive user inputs during execution. The HLS tool can not interact with the simulator and therefore cannot accept user inputs while the test bench executes.*

The following code shows the important features of a self-checking test bench, as an example:

```
int main () {
  //Establish an initial return value. 0 = success
  int ret=0;

  // Call any preliminary functions required to prepare input for the test.

  // Call the top-level function multiple times, passing input stimuli as
needed.
  for(i=0; i<NUM_TRANS; i++){
     top_func(input, output);
  }

  // Capture the output results of the function, write to a file

  // Compare the results of the function against expected results
  ret = system("diff --brief  -w output.dat output.golden.dat");

  if (ret != 0) {
```

```
        printf("Test failed  !!!\n");
        ret=1;
  } else {
        printf("Test passed !\n");
  }

  return ret;
}
```

The test bench should execute the top-level function for multiple transactions, allowing many different data values to be applied and verified. The test bench is only as good as the variety of tests it performs. In addition, your test bench must provide repeated calls to the synthesized function if you want to calculate II during RTL simulation as described in Running C/RTL Co-Simulation.

The self-checking test bench above compares the results of the function, `output.dat`, against known good results in `output.golden.dat`. This is one example of a self-checking test bench. There are many ways to validate your top-level function, and you must code your test bench as appropriate to your code.

In the HLS component flow, the return value of function `main()` indicates the following:

- Zero: Results are correct.
- Non-zero value: Results are incorrect.

The test bench can return any non-zero value. A complex test bench can return different values depending on the type of failure. If the test bench returns a non-zero value after C simulation or C/RTL co-simulation, the tool reports an error and simulation fails.



TIP: *Because the system environment (for example, Linux, Windows, or Tcl) interprets the return value of the `main()` function, it is recommended that you constrain the return value to an 8-bit range for portability and safety.*

Of course, the results of simulation are only as good as the test bench you provide. You are responsible for ensuring that the test bench returns the correct result. If the test bench returns zero, the tool indicates that the simulation has passed, regardless of what occurred during simulation.

Example Test Bench

AMD recommends that you separate the top-level function for synthesis from the test bench, and that you use header files. The following code example shows a design in which the top-level function for the HLS project, `hier_func`, calls two sub-functions:

- `sumsub_func` performs addition and subtraction.
- `shift_func` performs shift.

The data types are defined in the header file (`hier_func.h`). The code for the function follows:

```
#include "hier_func.h"

int sumsub_func(din_t *in1, din_t *in2, dint_t *outSum, dint_t *outSub)
{
 *outSum = *in1 + *in2;
 *outSub = *in1 - *in2;
}

int shift_func(dint_t *in1, dint_t *in2, dout_t *outA, dout_t *outB)
{
 *outA = *in1 >> 1;
 *outB = *in2 >> 2;
}

void hier_func(din_t A, din_t B, dout_t *C, dout_t *D)
{
 dint_t apb, amb;

 sumsub_func(&A,&B,&apb,&amb);
 shift_func(&apb,&amb,C,D);
}
```

As shown, the top-level function can contain multiple sub-functions. There can only be one top-level function for synthesis. To synthesize multiple functions, group them as sub-functions of a single top-level function.

The header file (`hier_func.h`), shown below, demonstrates how to use macros and how `typedef` statements can make the code more portable and readable.



TIP: *Arbitrary Precision (AP) Data Types discusses arbitrary precision data types, and how the `typedef` statement allows the types and therefore the bit-widths of the variables to be refined for both area and performance improvements in the final FPGA implementation.*

```
#ifndef _HIER_FUNC_H_
#define _HIER_FUNC_H_

#include <stdio.h>

#define NUM_TRANS 40

typedef int din_t;
typedef int dint_t;
typedef int dout_t;

void hier_func(din_t A, din_t B, dout_t *C, dout_t *D);

#endif
```

The header file above includes some `#define` statements, such as `NUM_TRANS`, that are not required by the `hier_func` function, but are provided for the test bench, which also includes the same header file.

The following code defines a test bench for the `hier_func` design:

```
#include "hier_func.h"

int main() {
 // Data storage
 int a[NUM_TRANS], b[NUM_TRANS];
 int c_expected[NUM_TRANS], d_expected[NUM_TRANS];
 int c[NUM_TRANS], d[NUM_TRANS];

  //Function data (to/from function)
 int a_actual, b_actual;
 int c_actual, d_actual;

  // Misc
 int retval=0, i, i_trans, tmp;
 FILE *fp;

 // Load input data from files
 fp=fopen(tb_data/inA.dat,r);
 for (i=0; i<NUM_TRANS; i++){
 fscanf(fp, %d, &tmp);
 a[i] = tmp;
 }
 fclose(fp);

 fp=fopen(tb_data/inB.dat,r);
 for (i=0; i<NUM_TRANS; i++){
 fscanf(fp, %d, &tmp);
 b[i] = tmp;
 }
 fclose(fp);

 // Execute the function multiple times (multiple transactions)
 for(i_trans=0; i_trans<NUM_TRANS-1; i_trans++){

 //Apply next data values
 a_actual = a[i_trans];
 b_actual = b[i_trans];

  hier_func(a_actual, b_actual, &c_actual, &d_actual);

 //Store outputs
 c[i_trans] = c_actual;
 d[i_trans] = d_actual;
 }

 // Load expected output data from files
 fp=fopen(tb_data/outC.golden.dat,r);
 for (i=0; i<NUM_TRANS; i++){
 fscanf(fp, %d, &tmp);
 c_expected[i] = tmp;
 }
 fclose(fp);

 fp=fopen(tb_data/outD.golden.dat,r);
 for (i=0; i<NUM_TRANS; i++){
 fscanf(fp, %d, &tmp);
 d_expected[i] = tmp;
 }
 fclose(fp);

 // Check outputs against expected
```

```
 for (i = 0; i < NUM_TRANS-1; ++i) {
 if(c[i] != c_expected[i]){
 retval = 1;
 }
 if(d[i] != d_expected[i]){
 retval = 1;
 }
 }

 // Print Results
 if(retval == 0){
 printf(    *** *** *** *** \n);
 printf(    Results are good \n);
 printf(    *** *** *** *** \n);
 } else {
 printf(    *** *** *** *** \n);
 printf(    Mismatch: retval=%d \n, retval);
 printf(    *** *** *** *** \n);
 }

 // Return 0 if outputs are corre
 return retval;
}
```

Design Files and Test Bench Files

Because the HLS tool reuses the C test bench for RTL verification, it requires that the test bench and any associated files be denoted as test bench files when they are added to the HLS component. Files associated with the test bench are any files that are:

- Accessed by the test bench.
- Required for the test bench to operate correctly.

Examples of such files include the data files `inA.dat` and `inB.dat` in the example test bench. You must add these to the HLS configuration file as test bench files.

The requirement for identifying test bench files in the HLS component does not require that the design and test bench be in separate files (although separate files are recommended). To demonstrate this, a new example is defined from the same code used in Example Test Bench, except a new top-level function is defined. In this example the function `sumsub_func` is defined as the top-level function in the HLS component.



TIP: *You can change the top-level function in the HLS configuration file by selecting the* ***C Synthesis*** *settings, or using the* `syn.top` *config file entry.*

With the `sumsub_func` function defined as the top-level function, the higher-level function, `hier_func` becomes part of the test bench, as it is the calling function for `sumsub_func`. The peer-level `shift_func` function is also now part of the test bench, as it is a required part of the test. Even though these functions are in the same code file as the top-level `sumsub_func` function, they are part of the test bench.

Single File Test Bench and Design

You can also include the design and test bench into a single design file. The following example has the same `hier_func` function as discussed Example Test Bench, except that everything is coded in a single file: top-level function, sub functions, and main function for the test bench.

IMPORTANT! *Having both the test bench and design in a single file requires you to add the file as both a synthesis source file, and a test bench file to the HLS configuration.*

```
#include <stdio.h>

#define NUM_TRANS 40

typedef int din_t;
typedef int dint_t;
typedef int dout_t;

int sumsub_func(din_t *in1, din_t *in2, dint_t *outSum, dint_t *outSub)
{
 *outSum = *in1 + *in2;
 *outSub = *in1 - *in2;
}

int shift_func(dint_t *in1, dint_t *in2, dout_t *outA, dout_t *outB)
{
 *outA = *in1 >> 1;
 *outB = *in2 >> 2;
}

void hier_func(din_t A, din_t B, dout_t *C, dout_t *D)
{
 dint_t apb, amb;

 sumsub_func(&A,&B,&apb,&amb);
 shift_func(&apb,&amb,C,D);
}

int main() {
 // Data storage
 int a[NUM_TRANS], b[NUM_TRANS];
 int c_expected[NUM_TRANS], d_expected[NUM_TRANS];
 int c[NUM_TRANS], d[NUM_TRANS];

 //Function data (to/from function)
 int a_actual, b_actual;
 int c_actual, d_actual;

 // Misc
 int retval=0, i, i_trans, tmp;
 FILE *fp;
 // Load input data from files
 fp=fopen(tb_data/inA.dat,r);
 for (i=0; i<NUM_TRANS; i++){
 fscanf(fp, %d, &tmp);
 a[i] = tmp;
 }
 fclose(fp);

 fp=fopen(tb_data/inB.dat,r);
 for (i=0; i<NUM_TRANS; i++){
```

```
 fscanf(fp, %d, &tmp);
 b[i] = tmp;
 }
 fclose(fp);

// Execute the function multiple times (multiple transactions)
for(i_trans=0; i_trans<NUM_TRANS-1; i_trans++){

 //Apply next data values
 a_actual = a[i_trans];
 b_actual = b[i_trans];

 hier_func(a_actual, b_actual, &c_actual, &d_actual);

 //Store outputs
 c[i_trans] = c_actual;
 d[i_trans] = d_actual;
 }

 // Load expected output data from files
 fp=fopen(tb_data/outC.golden.dat,r);
 for (i=0; i<NUM_TRANS; i++){
 fscanf(fp, %d, &tmp);
 c_expected[i] = tmp;
 }
 fclose(fp);

 fp=fopen(tb_data/outD.golden.dat,r);
 for (i=0; i<NUM_TRANS; i++){
 fscanf(fp, %d, &tmp);
 d_expected[i] = tmp;
 }
 fclose(fp);

 // Check outputs against expected
 for (i = 0; i < NUM_TRANS-1; ++i) {
 if(c[i] != c_expected[i]){
 retval = 1;
 }
 if(d[i] != d_expected[i]){
 retval = 1;
 }
 }

 // Print Results
 if(retval == 0){
 printf(    *** *** *** *** \n);
 printf(    Results are good \n);
 printf(    *** *** *** *** \n);
 } else {
 printf(    *** *** *** *** \n);
 printf(    Mismatch: retval=%d \n, retval);
 printf(    *** *** *** *** \n);
 }

 // Return 0 if outputs are correct
 return retval;
}
```

Sanitizers During C Simulation

A new feature is now available in C simulation: sanitizers. Sanitizers help identify undefined behaviors in the code (test bench and design). Undefined behaviors occur when the code is correct but is specified in the C/C++ language specifications to have unpredictable effects. If the design code can trigger undefined behaviors, the computation performed by the synthesized hardware is likely to be incorrect. Examples of undefined behaviors in C/C++ include integer overflows or array access out of bounds.

Vitis HLS ships two sanitizers to identify undefined behaviors in HLS components.

- The "address sanitizers" identifies undefined memory accesses when C simulation runs.
- The "undefined-behavior sanitizers" identifies other non address-related undefined behaviors during C simulation.

Both sanitizers can be enabled prior to running C simulation in the HLS project options using the GUI (need screenshot) or using the Tcl commands `config_csim -sanitize_address=true` and `config_csim -sanitize_undefined=true`.

In some cases, sanitizers will produce in the console a stack trace pointing at where the problem occur. It can resemble the following example:

```
AddressSanitizer:DEADLYSIGNAL
 =================================================================
 ==44041==ERROR: AddressSanitizer: SEGV on unknown address 0x0000000000c0
(pc 0x7ff6d5c4c284 bp 0x7ffeccaa1d80 sp 0x7ffeccaa1aa0 T0)
 ==44041==The signal is caused by a READ memory access.
 ==44041==Hint: address points to the zero page.
     #0 0x7ff6d5c4c284 in _IO_vfscanf (/lib64/libc.so.6+0x59284) (BuildId:
398944d32cf16a67af51067a326e6c0cc14f90ed)
     #1 0x274110 in __interceptor_vfscanf /path/compiler-rt/lib/asan/../
sanitizer_common/sanitizer_common_interceptors.inc:1523:1
     #2 0x274110 in fscanf /path/compiler-rt/lib/asan/../sanitizer_common/
sanitizer_common_interceptors.inc:1541:1
     #3 0x313c74 in main /project/hls/csim/build/../../../testbench.cpp:36:3
     #4 0x7ff6d5c15504 in __libc_start_main (/lib64/libc.so.6+0x22504)
(BuildId: 398944d32cf16a67af51067a326e6c0cc14f90ed)
     #5 0x253198 in _start (/project/hls/csim/build/csim.exe+0x253198)

 AddressSanitizer can not provide additional info.
 SUMMARY: AddressSanitizer: SEGV (/lib64/libc.so.6+0x59284) (BuildId:
398944d32cf16a67af51067a326e6c0cc14f90ed) in _IO_vfscanf
 ==44041==ABORTING
```

In the example the lines preceded by # and a number indicate a stack entry from 0 (deepest level of function calls) to the last entry in the list (initial function call in your application). Usually, the problem is related to the deepest function call that occurred in the HLS project when the error happened. To find the problematic source code line, you can go down the trace starting from #0 and identifying the first line of code which belongs to your design or test bench. In this example, a segmentation fault happened when executing the line 36 of the test bench file (entry #3 in the stack trace) and should be checked.

Output of C Simulation

In the Vitis Unified IDE the hierarchy of the HLS component is structured as follows: `<workspace>/<component>/<component>/hls/`. You can select the HLS component in the Vitis Component Explorer and select **Open In Terminal** from the right-click menu to explore the contents of the component folders.

When C simulation completes, a `csim` folder is created inside the `<component>/hls` folder. This folder contains the following elements:

- `csim/build`: The primary location for all files related to the C simulation
  - Any files read by the test bench are copied to this folder.
  - The C executable file `csim.exe` is created and run in this folder.
  - Any files written by the test bench are created in this folder.
  - `csim/obj`: Contains object files (`.o`) for the compiled source code, and make dependency files (`.d`) for the source code build.
- `csim/report`: Contains a log file of the C simulation build and run.

Using Code Analyzer

To achieve the best results in high-level synthesis (HLS) code changes are often required to improve the macro architecture of the design. To assist you with this effort, Vitis Unified IDE Code Analyzer provides features which let you visualize the potential for task level parallelism and understand the architectural changes needed to optimize performance.

You can run Code Analyzer as part of the C Simulation step by enabling the `csim.code_analyzer` command in the HLS configuration file as described in C-Simulation Configuration. After running it, the Code Analyzer report becomes available under the Reports of the C Simulation step in the Flow Navigator, or in the Analysis view of the Vitis unified IDE. The features of Code Analyzer include:

- **Dataflow Graph Extraction:** The Code Analyzer report extracts a dataflow graph (DFG) in which top-level statements become dataflow processes (DFG nodes), and the data dependencies of these processes become dataflow channels (DFG edges). The graph can be generated from a function or loop body even when they are not dataflow, helping you better determine how the code might be rewritten in a dataflow form, as described in Abstract Parallel Programming Model for HLS.

- **Performance Metrics:** Performance metrics such as the volume of data, transaction intervals (TI), and throughput are determined by Code Analyzer. The volume and access mode of data can be determined from the C test bench based on profiling information. For example, the tool can determine that variable A infers a port 32 bits wide, and 8 Kb of data is written by process 1 before being read by process 2. Analyzing the design prior to synthesis, Code Analyzer estimates the transaction interval, or time it takes for data transfer to complete, and the throughput of the channel. However, because the estimate occurs prior to synthesis it is less accurate than when calculated from the performance of the synthesized RTL.
- **Performance Guidance:** Identify any major performance blockers the HLS component source code might have. These blockers include cyclic dependencies and memory port contention. Performance Guidance helps you understand code structures that might limit design performance, or identify which metrics can help you understand the level of performance your design can achieve.
- **Graph Transformations:** Based on the dataflow graph decomposition and the measured and estimated metrics, you might determine that the graph is not ideal as shown. You can modify the graph by merging processes to perform what-if type design exploration. When the graph is modified, new performance metrics will be determined from the new architecture. Iterations of this design process could result in a blueprint for an ideally architected solution which you can use as the basis for refactoring your source code.

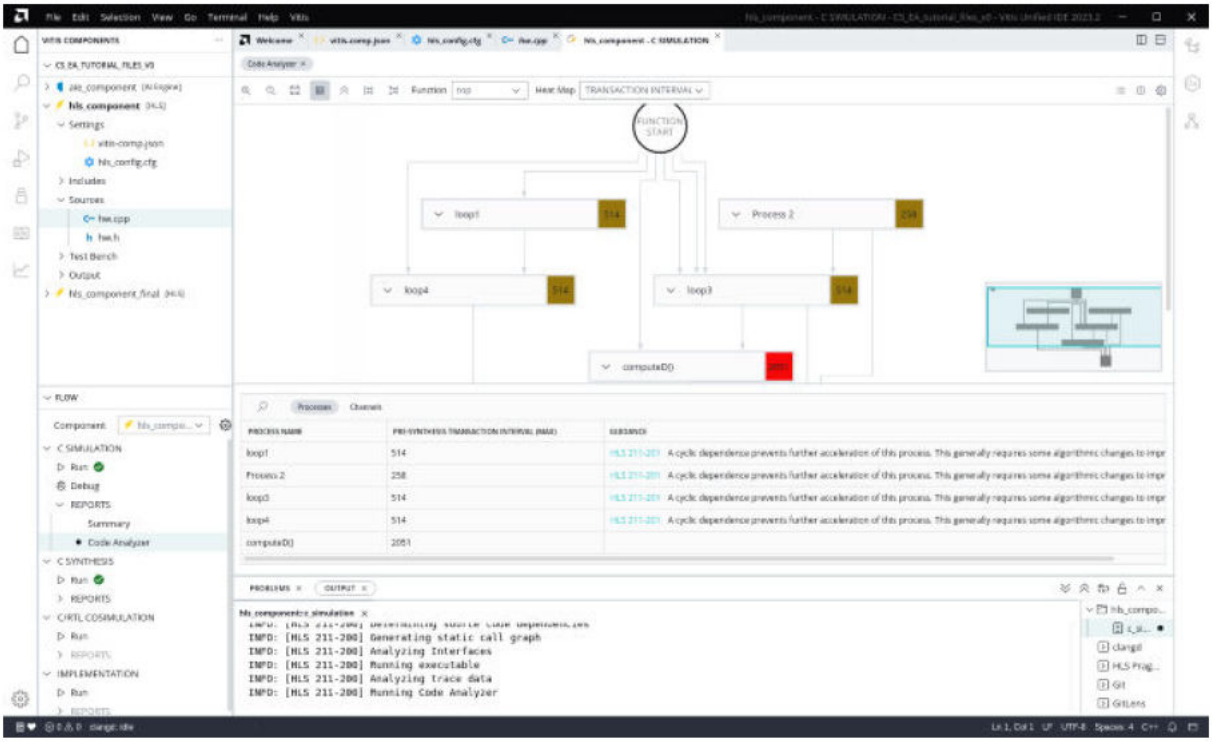


IMPORTANT! *The code can be merged and split in the Code Analyzer report, but any changes you want to carry forward will need to be manually reimplemented in the original design source code.*

Using the Code Analyzer Report

After running the C Simulation command with Code Analyzer enabled, the Code Analyzer report is generated and available to view in the Vitis unified IDE either in the Analysis view, or under the Reports header in the Flow Navigator. The Code Analyzer report initially displays the graph of the processes and channels defined by the top-level function of the component, as shown in the example below. You can change the scope of the report by using the Function selector in the toolbar menu, or by clicking the right arrow in an expanded process in the graph.

Figure 106: **Code Analyzer Report**



Features of the Code Analyzer report include the following:

- **Graph:** The Graph view in the report shows the processes and channels of the design in a dataflow graph. This infers the presence of the DATAFLOW pragma or directive, even if it does not yet exist in the source code. Each process shows the Transaction Interval and Performance Guidance for the element, and includes the source code for that element which can be viewed by expanding the Code view. In the example below you can see the performance as estimated during the pre-synthesis analysis. The dataflow processes (the graph nodes) have their TI displayed in the yellow/red boxes on the top right. The function calls and loops in the code that form the dataflow processes also have some performance metrics shown right after the call or loop header and they could have a Details link with additional guidance which identifies blocking factors for the performance of this loop or function call.

Figure 107: **Graph View**



In addition, you can right-click in the process header and select the **Goto Source** command to open and highlight the source code in its source file.

- **Table:** Beneath the graph the Code Analyzer report displays a table with two tabs: Processes and Channels. It provides a quick summary of the different elements so you can review the analysis in one table.
  - Processes displays the processes of the graph, and also includes the estimated pre-synthesis Transaction Interval (TI), and provides any design Guidance that the analysis might generate.
  - Channels displays the dataflow channels going into and out of each Process in a separate row. Channels are named after the variable defining them, with details of the variable declaration such as bitwidth, the data volume expected to be delivered over the channel, expected throughput, the access mode, and the Producer and Consumer tasks or processes.
- **Toolbar:** The toolbar menu of the Code Analyzer report provides a number of commands to help configure and view the reports.



The preceding figure displays the following commands starting from the left:

- Zoom In/Zoom Out/Zoom Fit: Zoom into the graph diagram as needed.
- Toggle Table: Displays or hides the table of Processes and Channels. This can free up space for the graph if needed.
- Collapse All: Closes any expanded processes in the graph.
- Group All/Ungroup All: Groups or ungroups channels with the same source and destination.
- Function: Provides the context for the current graph. The context can be changed by selecting a new function from the list, or by clicking on the arrows next to loops and function calls in the code of processes.

  - Heat Map: Specifies the reported data for the graph as either the Transaction Interval (TI) for loops or processes, or Performance Guidance messages. In the case of TI the largest result is highlighted in red to indicate that these are the slowest processes, those limiting the performance of the overall dataflow region.
  - Properties: Shows or hides the panel displaying the performance bottlenecks. The panel content is set by clicking on a **Details** link in the code of a process.
  - Info: Provides information related to the use of the tool, returned metrics, or common caveats of design. Worth looking through from time to time.
  - Settings: Specify the throughput units and edge labels in the graph. The available units are (Bits or Bytes) per (Cycle or Second). You can also specify a lower-limit of data volume by setting the **Channel Volume Filter** to grey out lower data signals (control signals for instance) and let you focus on the high-data channels.
- **Overview:** The Overview is a miniature representation of the whole graph that provides a reference to the portion of the graph that is displayed when zoomed into the graph. You can use the Overview to manage the view of the graph by manipulating the boundary. You can also close the Overview to free up space on the graph if desired.

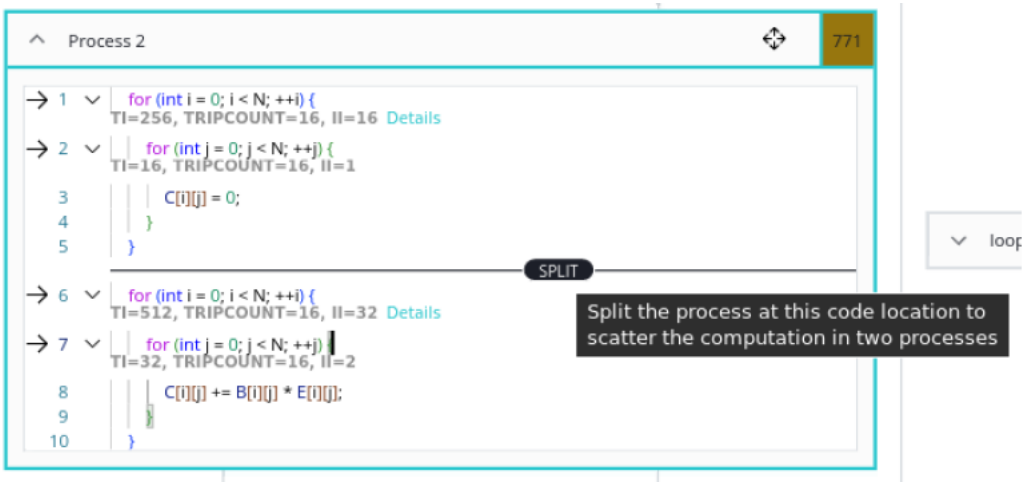
Working with the Graph

When starting the design you need to understand the HLS component source code in depth, identifying the main processes and the dependencies between these processes. Code Analyzer supports this by showing the your code as a dataflow graph as an output of C simulation.

The Code Analyzer report displays the Transaction Interval (TI) for each process, and displays the highest TI using a red background in the heat map. The red indicates the problem areas of your design. However, the pre-synthesis estimates used in the graph do not offer the same fidelity as the post-synthesis or post-implementation metrics. Code Analyzer lets you quickly determine performance potential, identify issues, and resolve them. Synthesis and implementation should be used when more precise information is needed.

You can merge consecutive processes in the source code to explore different dataflow structures of your design, and then split the code back into separate processes as needed. Simply drag and drop one process onto the second process to merge them. Remember, the processes must be sequential in the original source code. The following figure shows two processes that were merged, and now can be split apart again by clicking on the **SPLIT** line in the code.

Figure 108: **Working with the Graph**



If you identify a large bottleneck in the current design, and would like to turn it in a dataflow region, you can refocus the graph to the function or loop body and continue your analysis of the design. Work to resolve the issues in a methodical manner to make the best use of the Code Analyzer.

Ultimately the code must be rewritten in a dataflow form to reflect the results of Code Analyzer. Typically you need to outline the processes in their own functions and add a dataflow pragma in the function, loop, or region. This process can be accelerated by clicking on **Goto Source** on every process in the graph, right-click on the selected source code, and select **Refactor**.

Use Cases

- **Legality:**

  Code Analyzer allows for legality issues, as described in Canonical Body and Canonical Forms, to be identified on dataflow designs before synthesis is attempted. The key issues that can be identified are:

  - Read and written interfaces can be found through "R+W" accesses on the channels originated in the Start node or destined to the End node.
  - Multiple producers/consumer violations can be identified from the table, sorting by channel name and identifying multiple channels with the same variable. Accesses to the Start and End nodes can generally be dismissed.
  - Feedback loops can be found in the table with channel accesses of the mode "R → W" or, potentially, "R+W → W." This analysis can be complemented by the type of the channel to distinguish legal from illegal feedback channels.
  - Non-outlined processes can be identified from the process codes: users should aim at having a single function call per process in their top-level dataflow region and, as much as possible, have this call use variables or constants as arguments.

These issues can be fixed directly in the code and a new run of C Simulation will then refresh the graph with updated metrics and structure.

- **Improve Performance:**

  One of they key component to the performance of a dataflow region is the TI of the processes that constitute the region. Code analyzer can be used to efficiently improve the performance of dataflow processes without HLS synthesis.

  Select **Performance Guidance** in the Heat Map selector of the toolbar menu, and you can identify processes with a performance issue using the issues badge present on graph nodes. Expanding the process code presents the details of the particular problems identified in processes. You can investigate these issues and decide to address them or not depending on the feasibility, location of these problems, and ultimate performance objective. For instance, if you want II=1 in the inner loop of some specific processes, you will need to rewrite your code to fix all the problems presented on that particular loop nest.

  In a related use case, you might want to understand how the TI was computed for particular process, be it for educational or verification purposes. The TI and II annotation next to function calls and loops can be explored inlined with the process source code for this purpose.

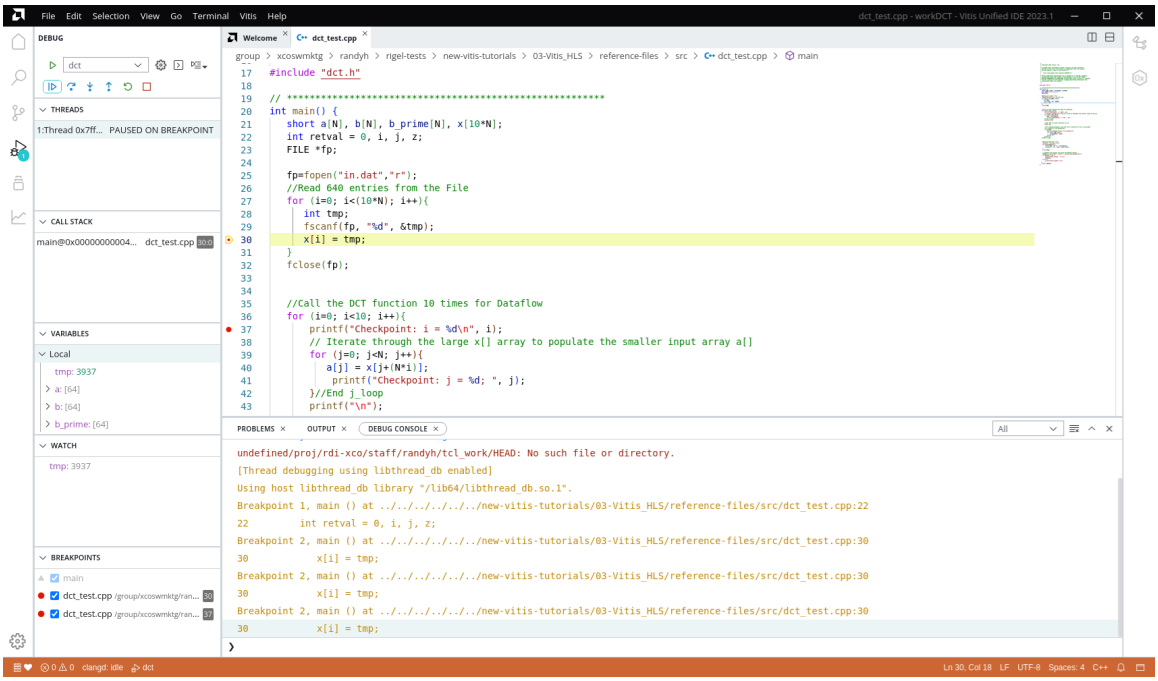
- **Throughput Analysis:**

  Code Analyzer presents estimates of throughput on channels. To complement the analysis and better understand the design performance, you can also access the channel width and its volume (total number of accesses per execution of the region). However you should validate the throughput estimates by synthesizing the design when possible. Code analyzer relies on pre-synthesis estimates that have a lower fidelity than other post-synthesis and post-implementation metrics.

Debugging the HLS Component

Debugging an HLS component is only possible after, or as part of C simulation. The following figure shows the Debug view for an HLS component. You can view the values of variables and expressions directly in the Debug view. In the Variables view, you can edit the values of variables to force the variable to a specific state for instance. In the Watch view, you can monitor the value of expressions. Use the '+' command in the Watch view to add an expression to watch. Refer to Debug View as described in the *Embedded Design Development Using Vitis* (UG1701) for additional details on the available features and views.

Figure 109: **Debug view - HLS Component**

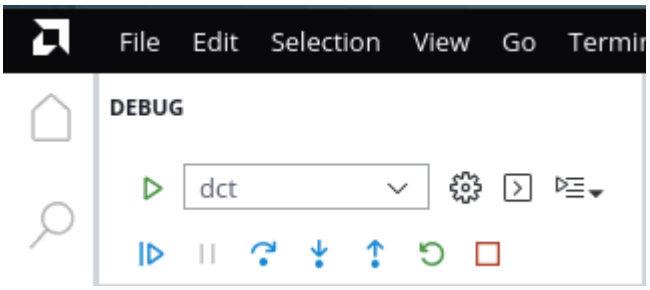


The Debug view acts like most classic C/C++ software debug tools. It includes the ability to insert breakpoints in the code, or to step over or into specific sections of the code. There are many different windows or widgets to manage the debug process. The following is a list of most with a brief descriptrion of how you might use them while debugging the HLS component.

- **Control Panel:**

  The Debug view Control Panel is displayed in the upper-left corner of the screen as shown in the image below. During debugging, you can use the control buttons such as Continue, Step Over, Step Into, Step Out, Restart, and Stop to control the debugging process.

Figure 110: **Debug Control Panel**



- **Threads:** Threads shows the related debugging threads. Threads are created and destroyed during the debugging process. You can switch between multiple threads.
- **Call Stack:** Call Stack shows the function call stack being updated as the application is run.

- **Variables:** You can view the values of variables and expressions directly in the Debug view using the Variables window which shows the current value of global and local variables. When switching threads, the variable information is updated. You can force variables to specific values by selecting a variable and selecting the **Set Value** command from the right-click menu.
- **Watch:** Shows variables and expressions you have specified to watch. Add watch points by selecting **Add Expression** (+).
- **Breakpoints:**

  Vitis Unified IDE sets break points at the main function of the test bench. To add breakpoints, you can open the source file and click the left side of the line number when a red dot appears. You can remove breakpoints by clicking on a previously added breakpoint.

  You can add conditional breakpoints by right-clicking when the red dot appears and select **Add Conditional Breakpoint**. You can also right-click and select **Add Logpoint** to insert a message to be logged when the breakpoint is reached.
- **Source Code Editor:** The Source Code view is opened when Debug is launched from the Flow Navigator or Launch Configuration view.
- **Debug Console:** Displays the transcript of the debug process, and any messages received from the tested application.

Running C Synthesis

Make sure the HLS component is active in the Flow Navigator, or select it from the Component menu to make it the active component in the tool. When the HLS component is the active component, the Flow Navigator enables running C Simulation, C Synthesis, C/RTL Co-simulation, Packaging, and Implementation to build and analyze the HLS component. To synthesize the HLS component select **Run** beneath the C SYNTHESIS heading in the Flow Navigator. Running C synthesis on the HLS component generates the RTL from the C/C++ source code.

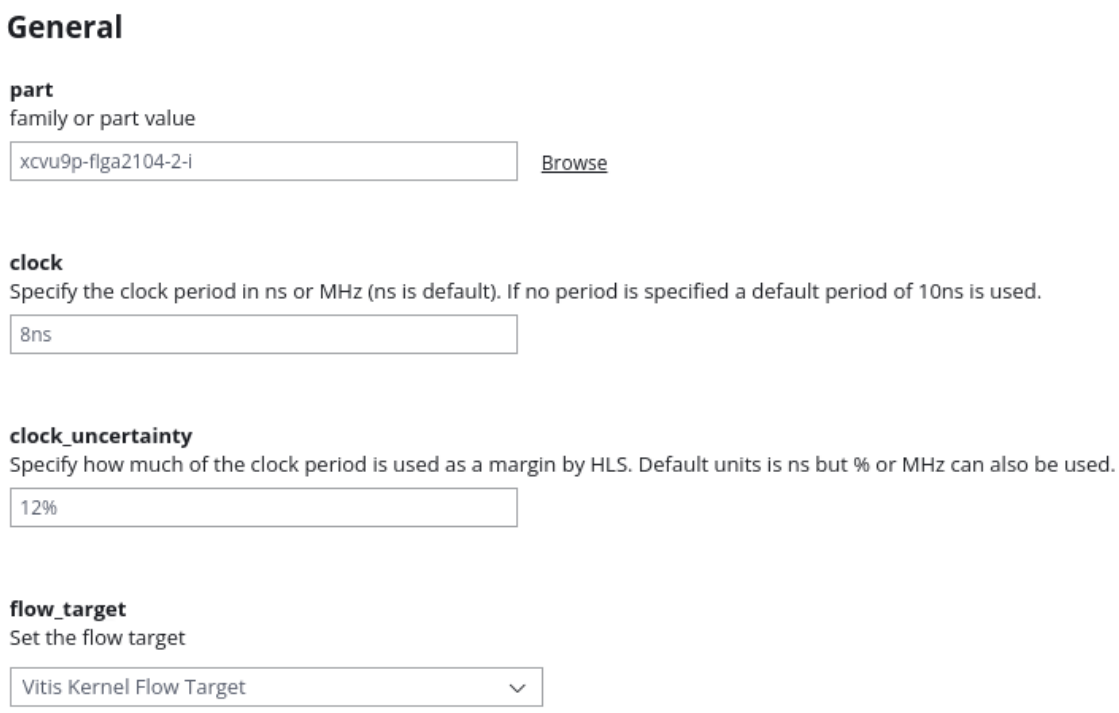


TIP: *Running synthesis in the Vitis Unified IDE uses the* `v++ -c --mode hls` *command.*

Specifying the Flow Target and General Settings

The HLS component can be used to create synthesis results for the Vivado IP flow, or the Vitis kernel flow. The flow target determines the default interface ports applied to the IP or kernel and other details of the synthesized design.

Figure 111: **HLS Component General Settings**



In the preceding figure the flow_target is currently set for the Vitis Kernel flow. The default flow target is the Vivado IP flow. This target is generally set when the HLS component is created, but can be modified as needed.

The part or platform can also be specified under the General settings. These options determine the physical device that the C/C++ code will be synthesized for. The selected device can feature hardware resources that expand or restrict the implementation choices of the RTL synthesis.

Finally, the clock period and clock_uncertainty can be specified to define the timing details of the RTL design. These can affect the timing and scheduling of operations in the design.

These selections result in the following config file entries:

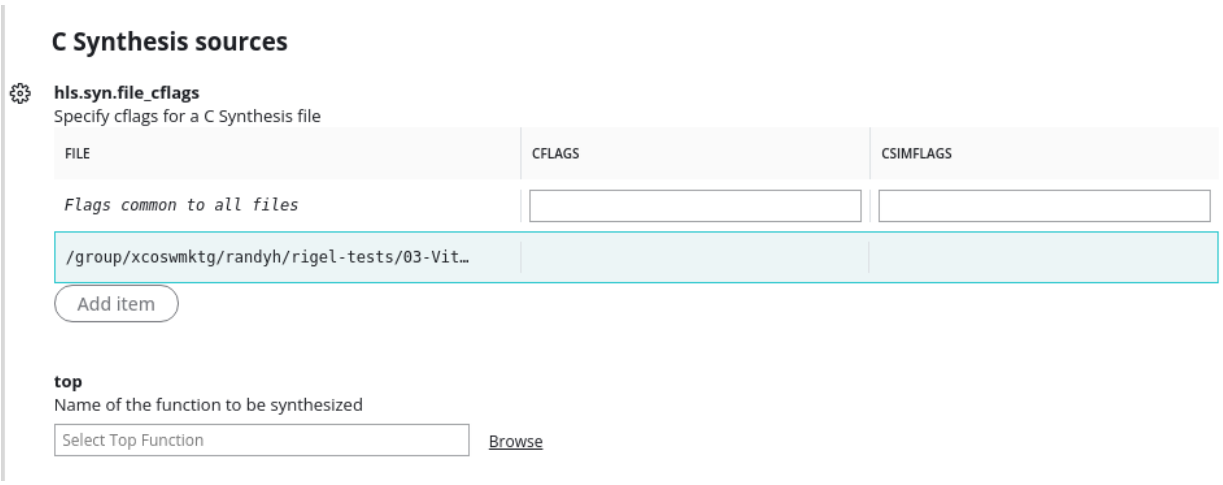
```
part=xcvc1902-vsva2197-2MP-e-S

[hls]
flow_target=vitis
clock=8ns
clock_uncertainty=15%
```

Loading Source Files and Identifying the Top Function

Generally the source code for the HLS component is defined or loaded at the time of creation. However, you can bypass that step when creating the component, and then you will need to add the source files prior to synthesizing the design. To add source files to an existing HLS component open the component config file and navigate to the C Synthesis sources section as shown below. This section of the Config Editor lets you add one or more source files, and specify the top function.

Figure 112: **HLS Component Synthesis Sources**



The C Synthesis sources displays currently added source files, and lets you edit, or delete current source files, add new source files, and add or modify CFLAGS and CSIMFLAGS.

- Add item: Select this command to add new source files to the HLS component. This opens a File Browser, and lets you navigate to and select source files to add.
- Add CFLAGS or CSIMFLAGS: You can add compilation flags for synthesis (CFLAGS) and for simulation (CSIMFLAGS) to be applied to all source files, or to be added to specific source files. As shown in the preceding figure, simply add the flags to the appropriate text entry box for Flags common to all files. To add flags for specific source files, select the source file and select **Edit item** to add the flags, or modify the file name and path.
- top: Lets you specify the function to use as the top-level RTL module for synthesis. The top-level module determines the RTL ports that will be added, and which sub-functions to include in the HLS component. This opens the Select Top Function dialog box that displays a list of functions defined in the source files for you to choose the top function.

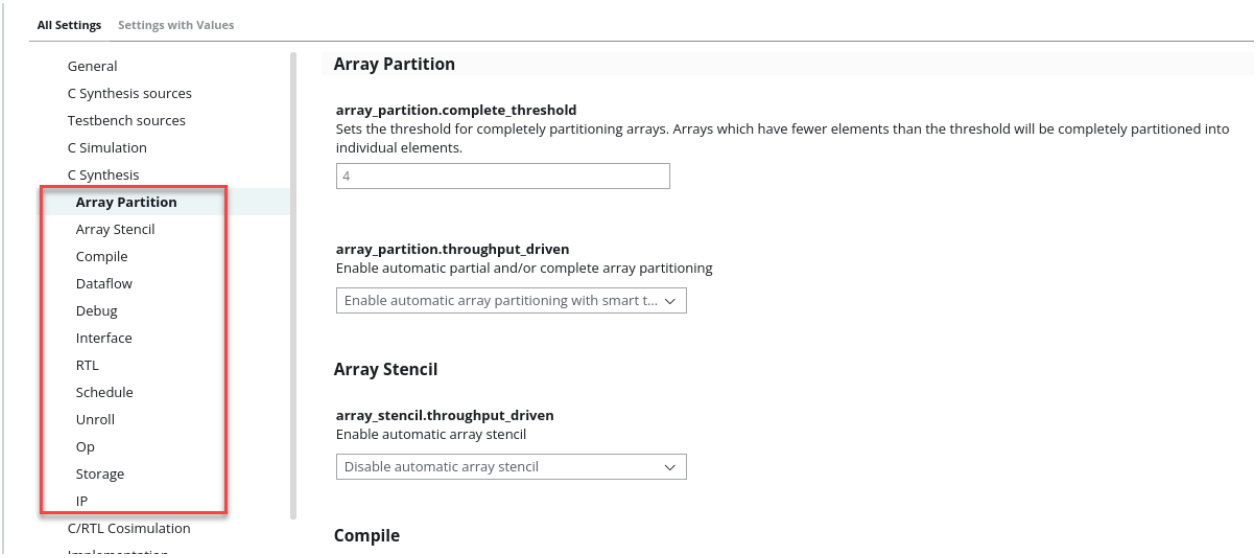
These selections result in the following config file entries:

```
[HLS]
syn.file=<path/to/file.cpp>
syn.file_cflags=<path/to/file.cpp>,<cflag>
syn.file_csimflags=<path/to/file.cpp>,<csimflag>
syn.cflags=<cflag for all files>
syn.csimflags=<csimflag for all files>
syn.top=<top function name>
```

Configuring Default Settings

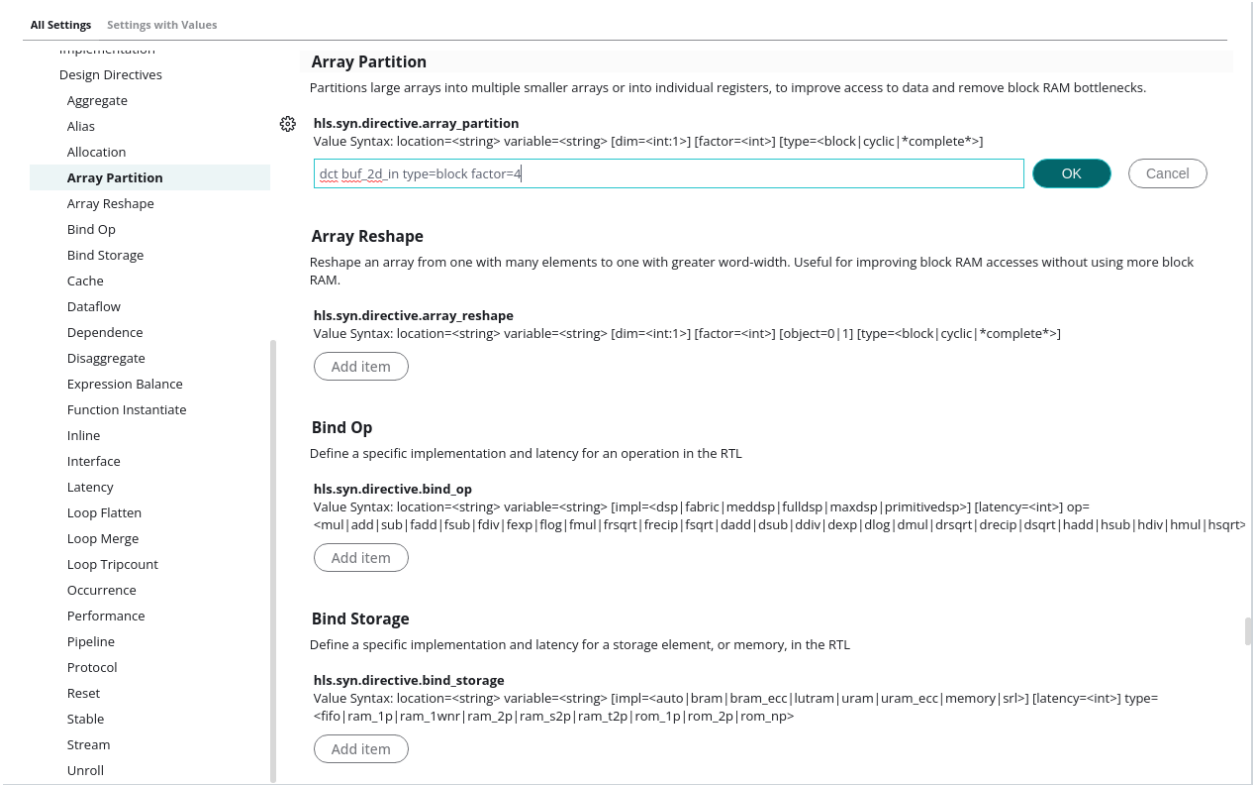
For the HLS component, the Vitis Unified IDE offers a variety of commands that can be used to configure the default settings of the tool for synthesis and simulation. These configuration commands are presented in Config Editor under the heading of C Synthesis as shown in the figure below. Generally these options are documented as presented in the figure, such as Compile Options, Interface Configuration, and RTL Configuration.

Figure 113: **HLS Component Default Settings**



Assigning Design Directives

Figure 114: **HLS Component Directives**



Design directives let you to customize the synthesis results for the source code. Change the directives across multiple synthesis runs to change the results or optimize your design. They can be added as HLS pragmas directly to the source code, or as HLS Optimization Directives to the configuration file for use in an HLS component. In the Config File editor you will see the various directives listed, and you can select **Add Item** to open the Directive editor.

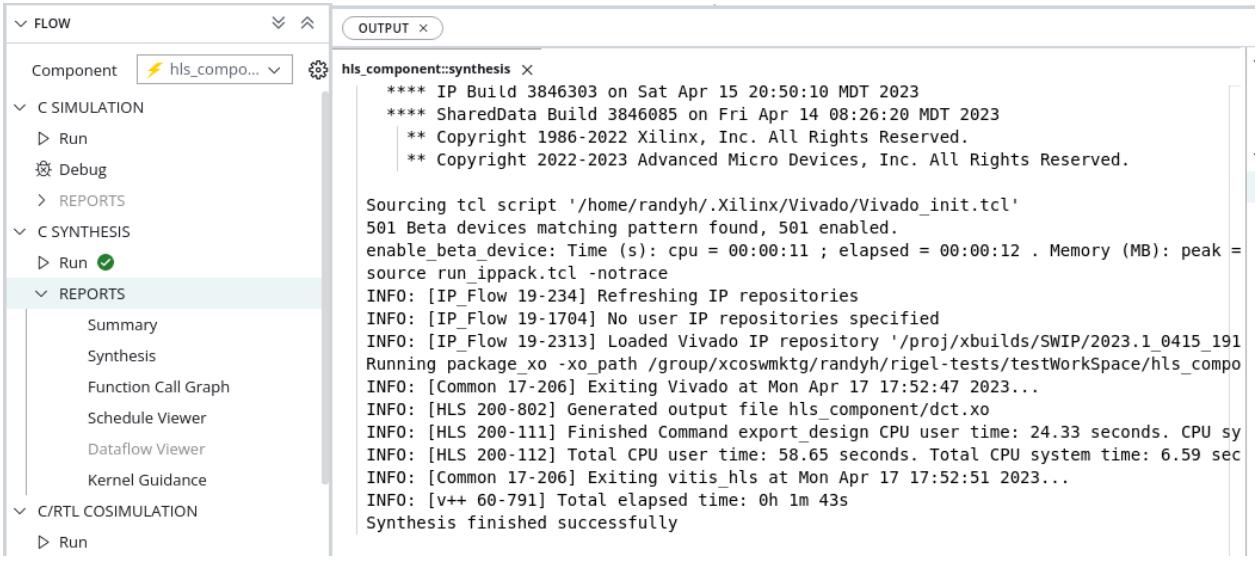
Some example design directive entries in the config file:

```
[HLS]
syn.directive.dataflow=dct
syn.directive.array_partition=dct buf_2d_in type=block factor=4
syn.directive.pipeline=dct2d II=4
```

Run Synthesis

With the key elements of the HLS component defined in the config file you are ready to run Synthesis. Select **Run** from the Flow Navigator to begin Synthesis. You can track the progress of the synthesis run in the Output window. The transcript for the synthesis run will have the top function name as `<component-name>::synthesis`. During the synthesis process messages are transcripted to the console window, and to the `vitis_hls.log` file.

Figure 115: **HLS Component Running Synthesis**



After synthesis is complete, you should see the Synthesis finished successfully message at the end of the transcript. You will also see the Reports folder under the Run command populated as shown in the preceding figure. The reports available after synthesis are as follows:

- Summary: reports the command line used and the time stamp on the results.
- Synthesis: reports the synthesis results with information on quality of results, HW interfaces, burst transactions and more. Refer to Synthesis Summary for more information.
- Function Call Graph: Displays the HLS component after C Synthesis or C/RTL Co-simulation to show the throughput of the design in terms of latency and II as described in Function Call Graph.
- Schedule Viewer: Shows each operation and control step of the function, and the clock cycle that it executes in, as described in Schedule Viewer .
- Dataflow Viewer: Only available when the DATAFLOW pragma or directive is used in the design. This report shows the dataflow structure inferred by the tool as described inDataflow Viewer.
- Kernel Guidance: reports Guidance messages to provide design advice for the source code and synthesis results.

Improving Synthesis Runtime and Capacity

The HLS compiler schedules objects. Whether the object is a floating-point multiply operation or a single register, it is still an object to be scheduled. The floating-point multiply can take multiple cycles to complete, and use many resources to implement, but at the level of scheduling it is still one object.

The HLS compiler schedules operations hierarchically. The operations within a loop are scheduled, then the loop, the sub-functions and operations within a function are scheduled. Runtime for the compiler increases when:

- There are more objects to schedule.
- There is more freedom and more possibilities to explore.

Unrolling loops and partitioning arrays creates more objects to schedule and potentially increases the runtime. Inlining functions creates more objects to schedule at this level of hierarchy and also increases runtime. Be very careful about simply partitioning all arrays, unrolling all loops and inlining all functions. These optimizations can be required to meet performance targets, but you can expect synthesis runtime increase. Use the optimization strategies discussed in Section II: HLS Programmers Guide and judiciously apply these optimizations.

If the loops must be unrolled, or if the use of the PIPELINE directive in the hierarchy above has automatically unrolled the loops, consider capturing the loop body as a separate function. This will capture all the logic into one function instead of creating multiple copies of the logic when the loop is unrolled: one set of objects in a defined hierarchy will be scheduled faster. Remember to pipeline this function if the unrolled loop is used in pipelined region.

The degrees of freedom in the code can also impact runtime. Consider the HLS compiler to be an expert designer who by default is given the task of finding the design with the highest throughput, lowest latency and minimum area. The more constrained the tool is, the fewer options it has to explore and the faster it will run. Consider using latency constraints over scopes within the code: loops, functions or regions. Setting a LATENCY directive with the same minimum and maximum values reduces the possible optimization searches within that scope.

Output of C Synthesis

In the Vitis Unified IDE the hierarchy of the HLS component is structured as follows: `<workspace>/<component>/<component>/hls/`. You can select the HLS component in the Vitis Component Explorer and select **Open In Terminal** from the right-click menu to explore the contents of the component folders.

When synthesis completes, the `syn` folder is created inside the `<component>/hls` folder. This folder contains the following elements:

- The `verilog` and `vhdl` folders contain the output RTL files.
  - The top-level file has the same name as the top-level function for synthesis.
  - There is one RTL file created for each sub-function that has not been inlined into a higher level function.
  - There could be additional RTL files to implement sub-blocks of the RTL hierarchy, such as block RAM, and pipelined multipliers.

- The `report` folder contains a report file for the top-level function and one for every sub-function that has not been in-lined into a higher level function by the HLS compiler. The report for the top-level function provides details on the entire design.

> **IMPORTANT!** *Do not directly use the RTL files generated in the `syn/verilog` or `syn/vhdl` folder for synthesis in the Vivado tool. Instead you must use the exported output files generated by the package process as described in Packaging the RTL Design. The HLS compiler sometimes uses IP from the Vivado IP catalog in the synthesized RTL code, such as with floating point designs, and the `verilog` and `vhdl` folders will not contain the complete design.*

Synthesis Summary

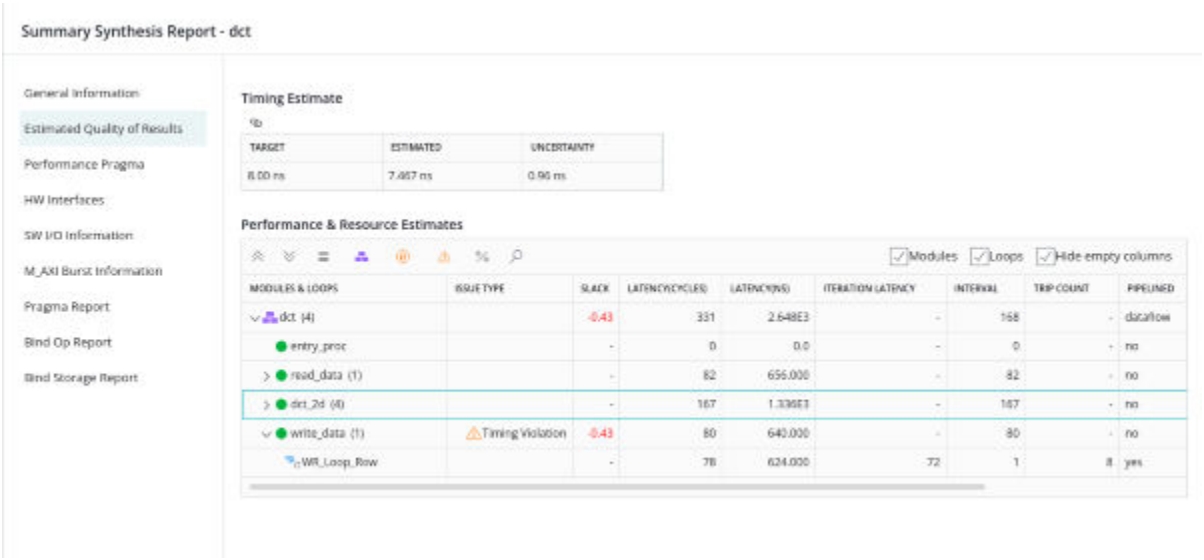
When synthesis completes, Vitis HLS generates a Synthesis Summary report for the top-level function that opens automatically in the information pane.

The specific sections of the Synthesis Summary are detailed below.

General Information

Provides information on when the report was generated, the version of the software used, the project name, the solution name and target flow, and the technology details.

Figure 116: **Synthesis Summary Report**



Timing Estimate

Displays a quick estimate of the timing specified by the solution, as explained in Specifying the Clock Frequency. This includes the Target clock period specified, and the period of Uncertainty. The clock period minus the uncertainty results in the Estimated clock period.

> **TIP:** *These values are only estimates of the HLS tool. More accurate timing estimates can be provided by Vivado synthesis and implementation as explained in Running Implementation.*

Performance & Resource Estimates

The Performance Estimate columns report the timing slack, latency, and initiation interval for the top-level function and any sub-blocks instantiated in the top-level. Each sub-function called at this level in the C/C++ source is an instance in the generated RTL block, unless the sub-function was in-lined into the top-level function using the INLINE pragma or directive, or automatically in-lined.

The Slack column displays any timing issues in the implementation.

The two Latency columns display the number of cycles it takes to produce the output, and display latency in time (ns).

The Iteration Latency is the latency of a single iteration for a loop.

The Initiation Interval is the number of clock cycles before new inputs can be applied. In the absence of any PIPELINE directives, the latency is one cycle less than the initiation interval (the next input is read after the final output is written).

> **TIP:** *When latency is displayed as a "?" it means that the tool cannot determine the number of loop iterations. If the latency or throughput of the design is dependent on a loop with a variable index, the HLS compiler reports the latency of the loop as being unknown. In this case, use the LOOP_TRIPCOUNT pragma or directive to manually specify the number of loop iterations. The LOOP_TRIPCOUNT value is only used to ensure the generated reports show meaningful ranges for latency and interval and does not impact the results of synthesis.*

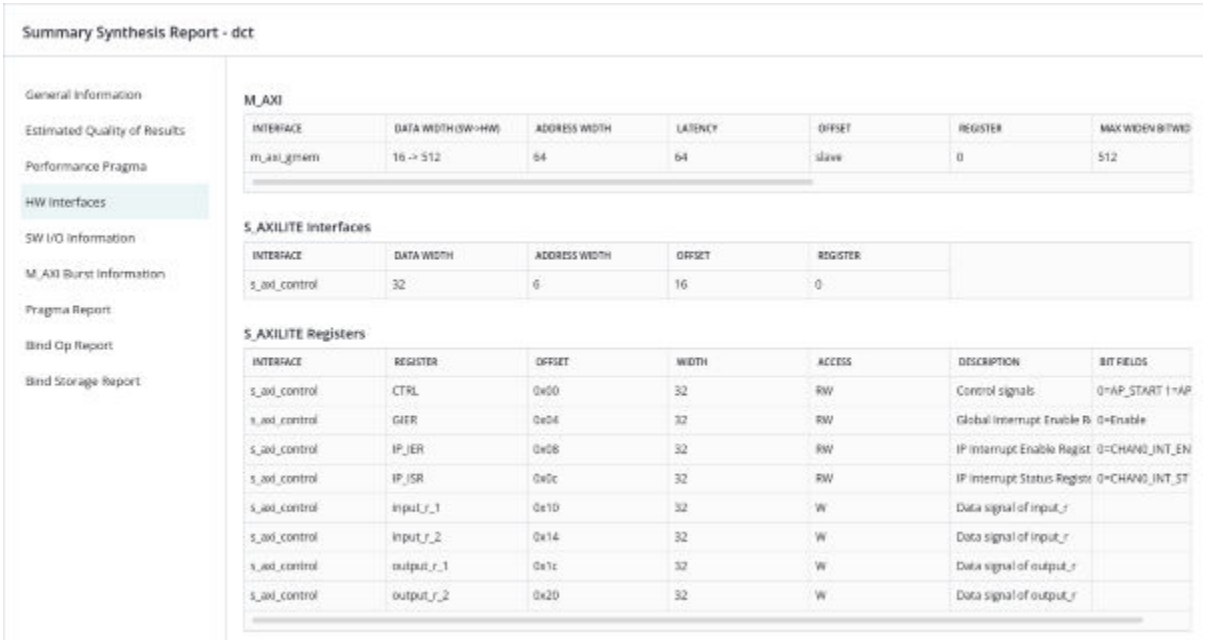
The Trip Count column displays the number of iterations a specific loop makes in the implemented hardware. This reflects any unrolling of the loop in hardware.

The resource estimate columns of the report indicates the estimated resources needed to implement the software function in the RTL code. Estimates of the block RAM, DSP, FFs, and LUTs are provided.

HW Interfaces

The HW Interfaces section of the synthesis report provides tables for the different hardware interfaces generated during synthesis. The type of hardware interfaces generated by the tool depends on the flow target specified by the solution, as well as any INTERFACE pragmas or directives applied to the code. In the following image, the solution targets the Vitis Kernel flow, and therefore generates AXI interfaces as required.

Figure 117: **HW Interfaces**



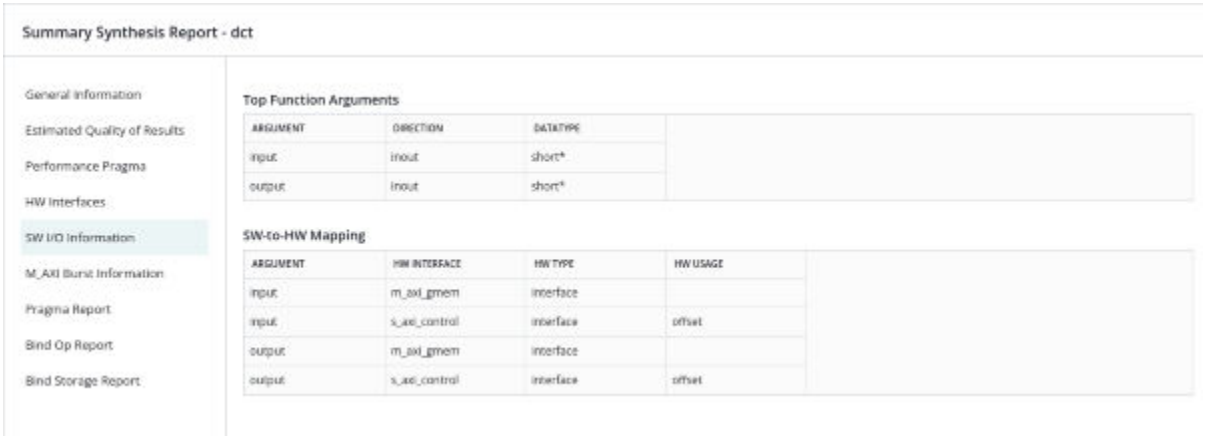
The following should be observed when reviewing these tables:

- Separate tables are provided for the different interfaces.
- Columns are provided to display different properties of the interface. For the M_AXI interface, these include the Data Width and Max Widen Bitwidth columns which indicate whether Automatic Port Width Resizing has occurred, and to what extent. In the example above, you can see that the port was widened to 512 bits from the 16 bits specified in the software. For more information, see Automatic Port Width Resizing.
- The Latency column displays the latency of the interface:
  - In an `ap_memory` interface, the column displays the read latency of the RAM resource driving the interface.
  - For an `m_axi` interface, the column displays the expected latency of the AXI4 interface, allowing the design to initiate a bus request a number of cycles (latency) before the read or write is expected.
- The Bundle column displays any specified bundle names from the INTERFACE pragma or directive.
- Additional columns display burst and read and write properties of the M_AXI interface as described in the INTERFACE pragma or directive.
- The Bit Fields column displays the bits used by an the registers in an `s_axilite` interface.

SW I/O Information

Highlights how the function arguments from the C/C++ source is associated with the port names in the generated RTL code. Additional details of the software and hardware ports are provided as shown below. The SW argument is expanded into multiple HW interfaces. For example, the `input` argument is related to three HW interfaces, the `m_axi` for data, and the `s_axi_lite` for required control signals.

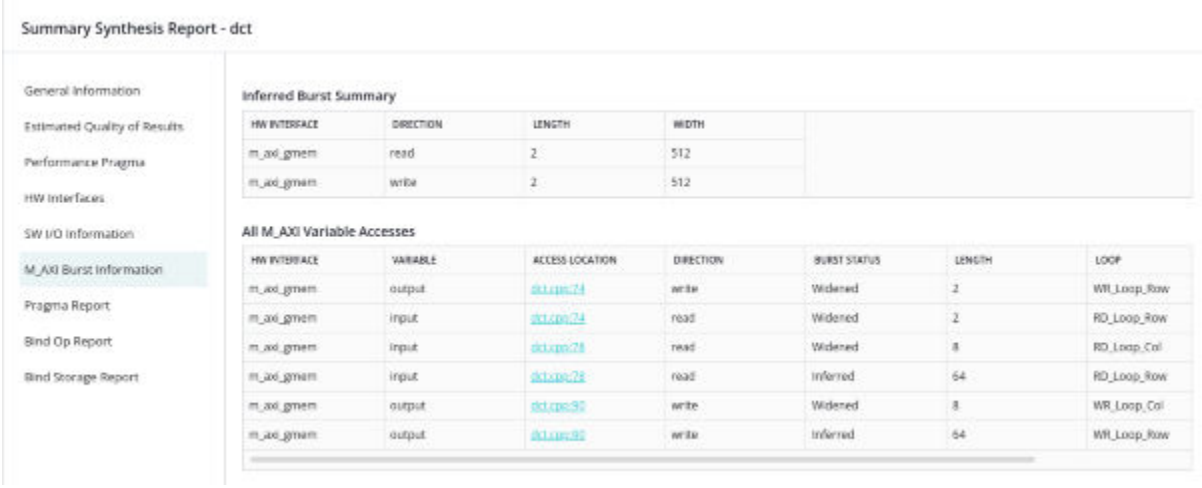
Figure 118: **SW I/O Information**



M_AXI Burst Information

In the M_AXI Burst Information section the Burst Summary table reports the successful burst transfers, with a link to the associated source code. The reported burst length refers to either `max_read_burst_length` or `max_write_burst_length` and represents the number of data values read/written during a burst transfer. For example, in a case where the input type is integer (32 bits), and HLS auto-widens the interface to 512 bits, each burst transfers 1024 integers. Because the widened interface can carry 16 integers at a time, the result is 64 beat bursts. The Burst Missed table reports why a particular burst transfer was missed with a link to Guidance messages related to the burst failures to help with resolution.

Figure 119: **M_AXI Burst Information**

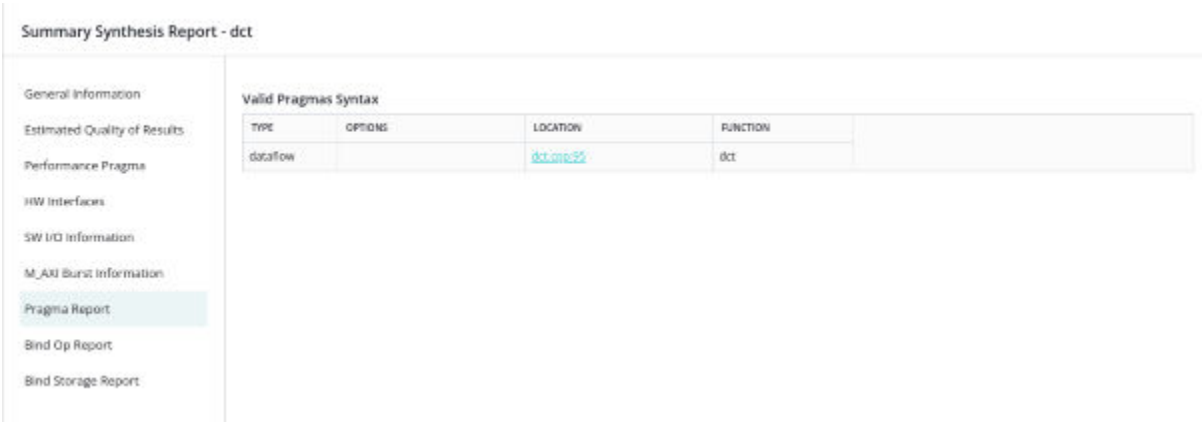


Pragma Report

Displays the pragmas in the design: valid, ignored, inferred. This report is intended to summarize issues that can otherwise be found in the log files. It lets you quickly identify issues with the pragmas used in your design, to see which ones might not have been used as expected. Valid pragmas are separately reported so you can see all pragmas in use in the design.

TIP: *A link is provided to the location of the pragma in the source code. Click the link to view the source code.*

Figure 120: **Pragma Report**



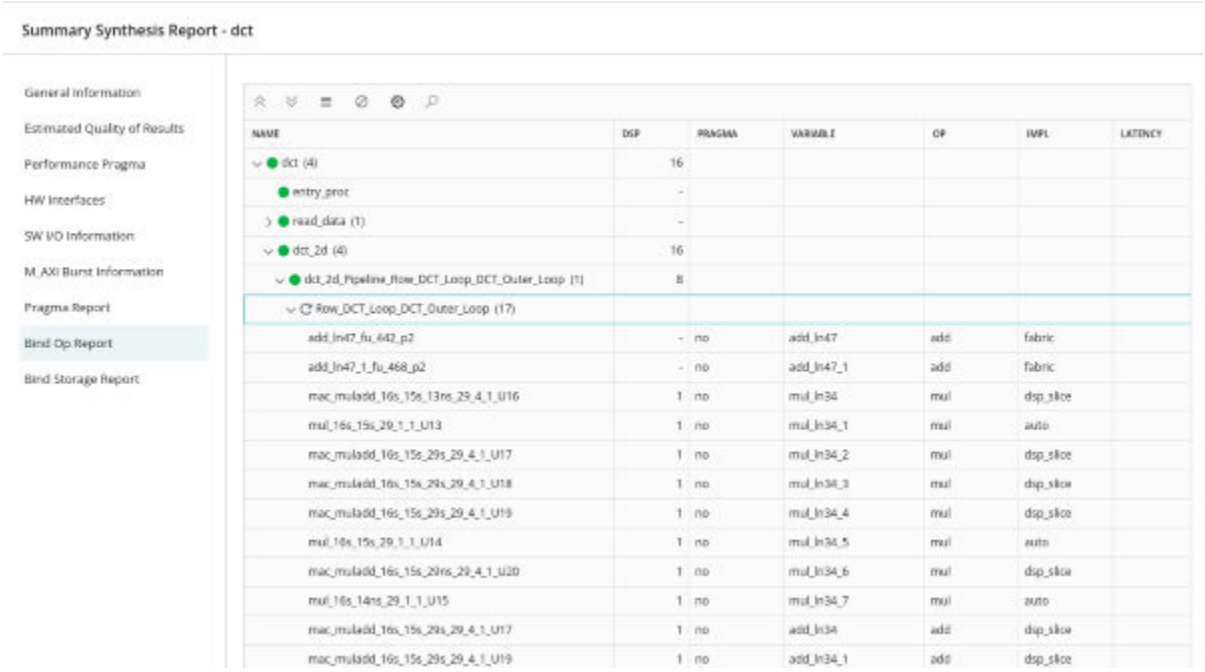
Bind Op and Bind Storage Reports

The Bind Op and Bind Storage reports are added to the Synthesis Summary report. Both reports can help you understand choices made by the HLS compiler when it maps operations to resources. The tool will map operations to the right resources with the right latency. You can influence this process by using the BIND_OP pragma or directive, and requesting a particular resource mapping and latency. The Bind Op report will show which of the mappings were automatically done versus those enforced by the use of a pragma. Similarly, the Bind Storage report shows the mappings of arrays to memory resources on the platform like block RAM/ LUTRAM/URAM.

The Bind Op report displays the implementation details of the kernel or IP. The hierarchy of the top-level function is displayed and variables are listed with any HLS pragmas or directives applied, the operation defined, the implementation used by the HLS tool, and any applied latency.

This report is useful for examining the programmable logic implementation details specified by the RTL design.

Figure 121: **Synthesis Summary**



As shown above, the Bind Op report highlights certain important characteristics in your design. Currently, it calls out the number of DSPs used in the design and shows in a hierarchy where these DSPs are used in the design. The table also highlights whether the particular resource allocation was done because of a user-specified pragma and if so, a "yes" entry will be present in the Pragma column. If no entry exists in the Pragma column, it means that the resource was auto inferred by the tool. The table also shows the RTL names of the resources allocated for each module in the user's design and you can hierarchy descend down the hierarchy to see the various resources.

It does not show all the inferred resources but instead shows resources of interest such as arithmetic, floating-point, and DSPs. The particular implementation choice of fabric (implemented using LUTs) or DSP is also shown. Finally, the latency of the resource is also shown. This is helpful in understand and increasing the latency of resources if needed to add pipeline stages to the design. This is extremely useful when attempting to break a long combinational path when trying to solve timing issues during implementation.

Each resource allocation is correlated to the source code line where the corresponding op was inferred from and the user can right-click on the resource and select the "Goto Source" option to see this correlation. Finally, the second table below the Bind Op report illustrates any global config settings that can also alter the resource allocation algorithm used by the tool. In the above example, the implementation choice for a `dadd` (double precision floating point addition) operation has been fixed to a `fulldsp` implementation. Similarly, the latency of a `ddiv` operation has been fixed to 2.

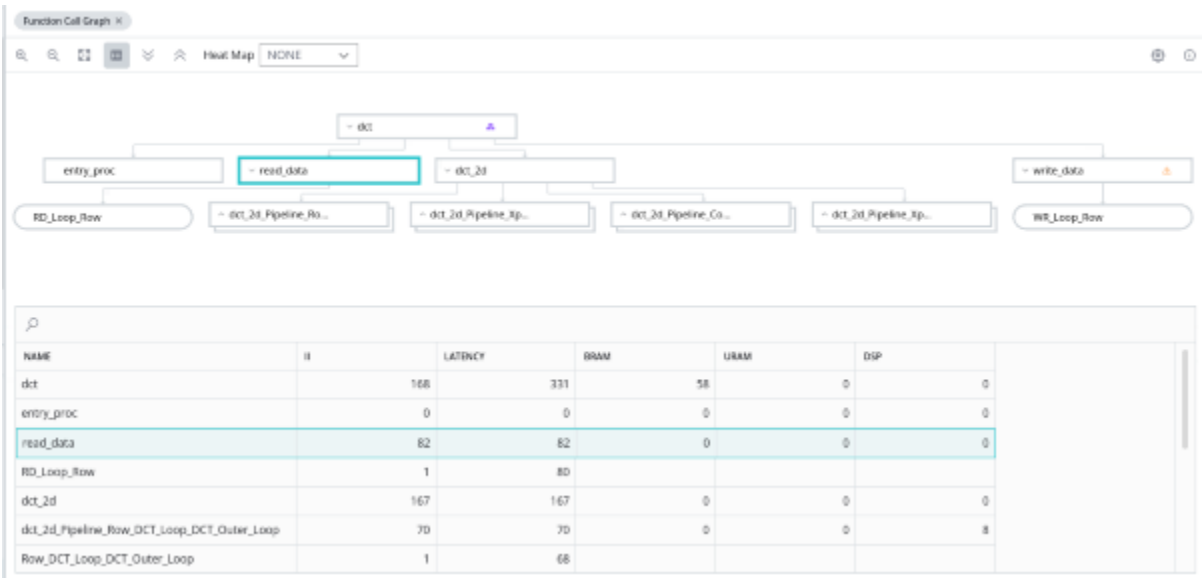
Similar to the BIND_OP pragma, the BIND_STORAGE pragma can be used to select a particular memory type (such as single port or dual port) and/or a particular memory implementation (such as BRAM/LUTRAM/URAM/SRL, etc.) and a latency value. The Bind Storage report highlights the storage mappings used in the design. Currently, it calls out the number of Block RAMs and URAMs used in the design. The table also highlights whether the particular storage resource allocation was done because of a user-specified pragma and if so, a "yes" entry will be present in the Pragma column. If no entry exists in the Pragma column, then this means that the storage resource was auto inferred by the tool. The particular storage type, as well as the implementation choice, are also shown along with the variable name and latency.

Using this information, you can review the storage resource allocation in the design and make design choices by altering the eventual storage implementation depending upon availability. Finally, a second table in the Bind Storage report will be shown if there are any global settings as described in Storage Configuration that can also alter the storage resource allocation algorithm used by the tool.

Function Call Graph

The Function Call Graph illustrates your full design after C Synthesis or C/RTL Co-simulation. The report shows the throughput of the design in terms of latency and II. It helps identify the critical path in your design and helps you identify bottlenecks in the design to improve throughput. It can also show the paths through the design where throughput might be imbalanced leading to FIFO stalls and/or deadlock.

Figure 122: **Performance Metrics Synthesis**



In some cases, the displayed hierarchy of the design might not be the same as your source code as a result of HLS optimizations that convert loops into function pipelines, etc. Functions that are in-lined will no longer be visible in the call graph, as they are no longer separate functions in the synthesized code. If multiple instances of a function are created, each unique instance of the function is shown in the call graph. This lets you see what functions contribute to a calling function's latency and II.

The graph as shown above displays functions as rectangular boxes, and loops as oval boxes. The graph displays hierarchy and nested functions or loops. A table under the graph displays each function or loop with II, latency, and resource or timing data depending on the specific view.

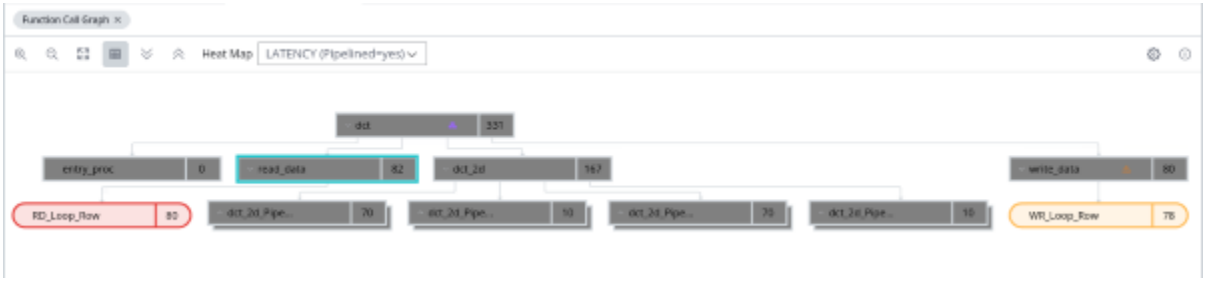
The Function Call Graph report is available under both C SYNTHESIS and C/RTL COSIMULATION headings in the Flow Navigator. The performance and resource metrics that are shown in the graph from the C SYNTHESIS phase are estimates from the HLS tool. The Function Call Graph report available after C/RTL COSIMULATION is more detailed, providing actual II and latency numbers reported along with stalling percentages.

TIP: *For more accurate resource and timing estimates, logic synthesis or implementation can be performed as part of Running Implementation.*

You can also use the **Heat Map** feature to highlight several metrics of interest:

- II (min, max, avg)
- Latency (min, max, avg)
- Resource Utilization
- Stalling Time Percentage

Figure 123: **Performance Metrics**



The heat map uses color coding to highlight problematic modules. Using a color scale of red to green where red indicates the high value of the metric (highest II or highest latency) while green indicates a low value of the metric in question. The colors that are neither red nor green represent the range of values that are in between the highest and lowest values. As shown above, this helps in quickly identifying the modules that need attention. In the example shown above, a heat map for LATENCY is shown and the red module indicates where high latency is observed.

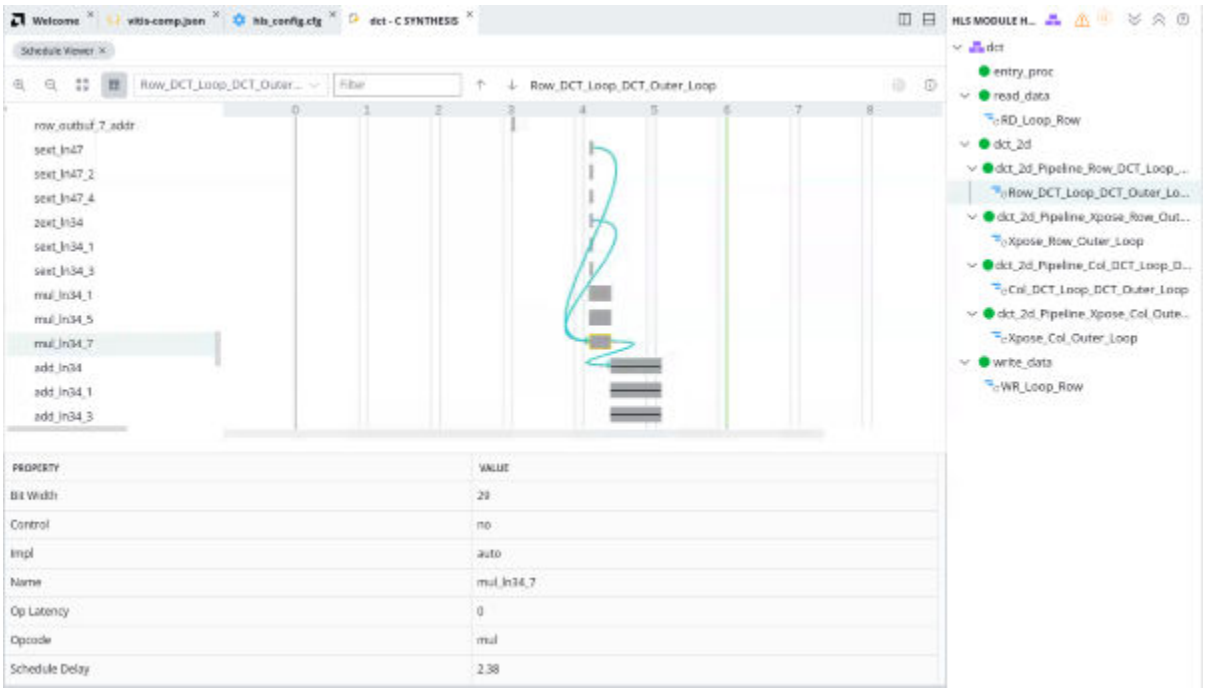
As mentioned before, the Function Call Graph illustrates at a high level, the throughput numbers of your design. The user can view the Function Call Graph as a cockpit from which further investigations can be carried out. Right-click on any of the displayed modules to display a menu of options that you can use to display additional information. This lets you see the overall design and then jump into specific parts of the design which need extra attention. Additional reports include the Schedule Viewer, Synthesis Summary report, Dataflow Viewer, and source files. The Function Call Graph lets you see the full picture of your design and have the latency and II information of each module available for analysis.

TIP: *On the right hand side of the report there is a Settings command and a Legend command to let you display additional details of the report.*

Schedule Viewer

The Schedule Viewer provides a detailed view of the synthesized RTL, showing each operation and control step of the function, and the clock cycle that it executes in. It helps you to identify any loop dependencies that are preventing parallelism, timing violations, and data dependencies.

Figure 124: **Schedule Viewer**



- The left vertical axis shows the names of operations and loops that will get implemented as logic in the RTL hierarchy. Operations are in topological order indicating that an operation on one line can only be driven by operations from a previous line, and will only drive an operation in a later line. Depending upon the type of violations found the Schedule Viewer shows additional information for each operation

> **TIP:** *Each operation is associated with lines of source code. Right-click the operation to use the* ***Goto Source*** *command to open the input source code associated with the operation.*

- The top horizontal axis shows the clock cycles in consecutive order.
- The vertical dashed line in each clock cycle shows the reserved portion of the clock period due to clock uncertainty. This time is left by the tool for the Vivado back-end processes, like place and route.
- Each operation is shown as a gray box in the table. The box is horizontally sized according to the delay of the operation as percentage of the total clock cycle. In case of function calls, the provided cycle information is equivalent to the operation latency. Multi-cycle operations are shown as gray boxes with a horizontal line through the center of the box.
- The Schedule Viewer also displays general operator data dependencies as solid blue lines. As shown in the preceding figure, when selecting an operation you can see solid blue arrows highlighting the specific operator dependencies. This gives you the ability to perform detailed analysis of data dependencies. A green dotted line indicates an inter-iteration data dependency. Memory dependencies are displayed using golden lines.

> **TIP:** *By default all dependencies (blue lines) are shown between each operation in the critical timing path.*

- On the far right the HLS Module Hierachy is displayed alongside the Schedule Viewer to let you quickly navigate through the hierarchy of the design.

In the following figure the loop called `COL_DCT_LOOP_DCT_OUTER_LOOP` is selected. This is a pipelined loop and the initiation interval (II) is explicitly stated in the loop bar. Any pipelined loop is visualized unfolded, meaning one full iteration is shown in the schedule viewer. Overlap, as defined by II, is marked by a thick clock boundary on the loop marker. The total latency of a single iteration is equivalent to the number of cycles covered by the loop marker. In this case, it is six clock cycles.

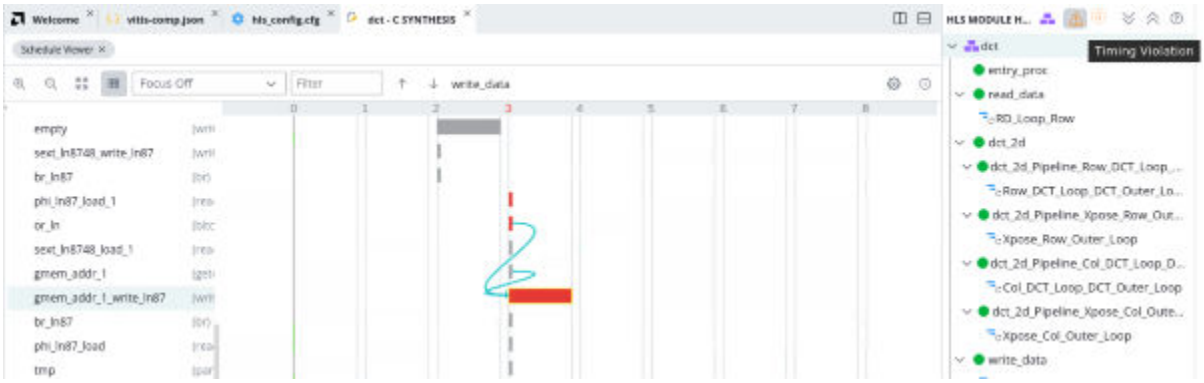
Figure 125: **Pipelined Loop**



The Schedule Viewer displays a menu bar at the top of the report that includes the following features:

- Zoom In/Zoom Out/Zoom Fit commands
- A text field to enter search terms for specific operations or steps, and arrow commands (Previous Match or Next Match) to scroll up or down through the list of objects that match your search text
- Legend command for the display.

Figure 126: **Timing Violations**



You can quickly locate timing violations and II violations using the toolbar menu in the in the Module Hierarchy view as shown in the preceding figure. To locate the operations causing the violation in the source code, right-click the operation and use the **Goto Source** command. A timing violation is a path of operations requiring more time than the available clock cycle. To visualize this, the problematic operation is represented in the Schedule Viewer in a red box.

Properties View

At the bottom of the Schedule Viewer, as shown in the top figure, is the Properties view that displays the properties of a currently selected object in the Schedule Viewer. This lets you see details of the specific function, loop, or operation that is selected in the Schedule Viewer. The types of elements that can be selected, and the properties displayed include:

- Functions or Loops
  - **Initiation Interval (II):** The number of clock cycles before the function or loop can accept new input data.
  - **Loop Iteration Latency:** The number of clock cycles it takes to complete one iteration of the loop.
  - **Latency:** The number of clock cycles (and time) required for the function to compute all output values, or for the loop to complete all iterations.
  - **Name:** Name of function or loop.
  - **Pipelined:** Indicates that the function or loop are pipelined in the RTL design.
  - **Slack:** The timing slack for the function or loop.
  - **Tripcount:** The number of iterations a loop completes.
  - **Resource Utilization:** Displays the number of BRAM, DSP, LUT, or FF used to implement the function or loop.
- Operation and Storage Mapping
  - **Bitwidth:** Bitwidth of the Operation.
  - **Impl:** Defines the implementation used for the specified operation or storage.
  - **Name:** Name of operation.
  - **Op Latency:** Displays the default or specified latency for the binding of the operation or storage.
  - **Opcode:** Operation which has been scheduled, for example, `add`, `sub`, and `mult`. For more information, refer to the BIND_OP or BIND_STORAGE pragmas or directives.
  - **Schedule Delay:** Specifies the delay associated with the operation.

Dataflow Viewer

The `DATAFLOW` optimization is a dynamic optimization which can only be fully understood after RTL Co-simulation is complete. Due to this fact, after C synthesis the Dataflow viewer lets you see the dataflow structure inferred by the tool, inspect the channels (FIFO/PIPO), and examine the effect of channel depth on performance. Performance data is back-annotated to the Dataflow viewer after co-simulation.

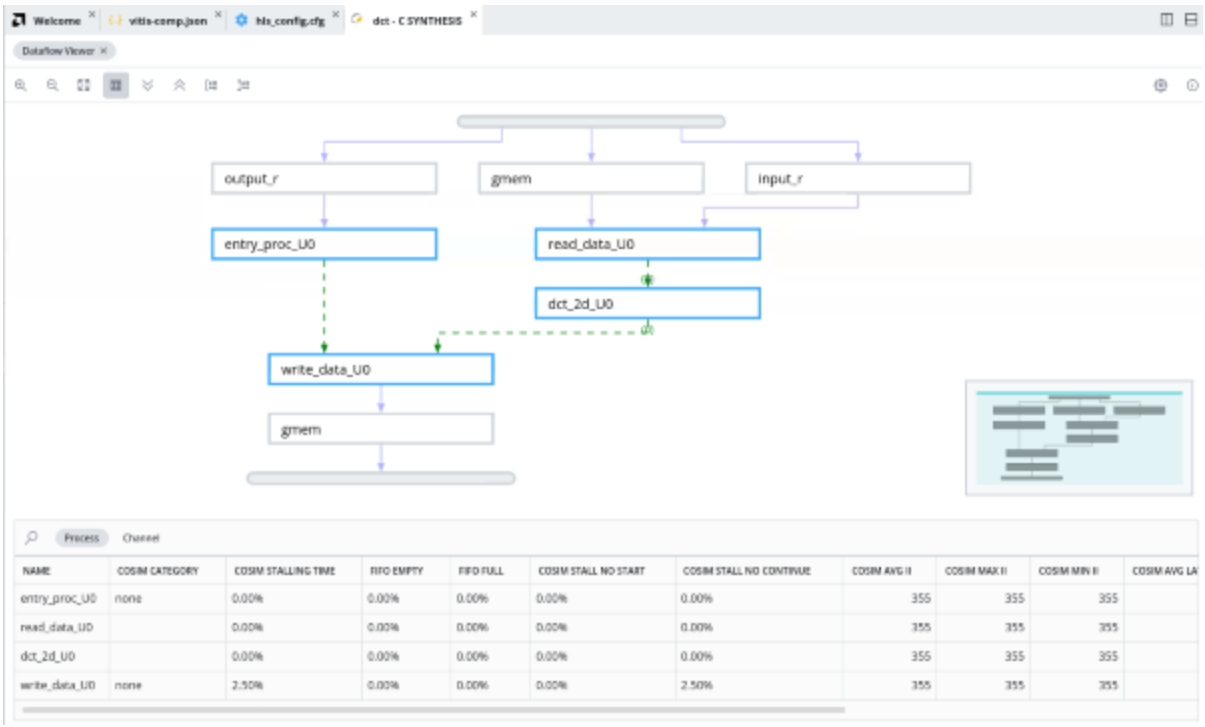
IMPORTANT! *You can open the Dataflow viewer without running RTL co-simulation, but your view will not contain important performance information such as read/write block times, co-sim depth, and stall times.*

You must apply the `DATAFLOW` pragma or directive to your design for the Dataflow Viewer report to be generated. You can apply dataflow to the top-level function, or specify regions of a function, or loops. The Dataflow viewer displays a representation of the dataflow graph structure, showing the different processes and the underlying producer-consumer connections.

The Synthesis Summary, the Module Hierarchy, and the Function Call Graph will display the icon beside the top-level function to indicate the presence of the DATAFLOW pragma.

TIP: *The diagram below is generated from the default `m_axi` interface of the Vitis Kernel flow. The use of the Vivado IP flow can result in a different dataflow diagram.*

Figure 127: **Dataflow Viewer**



Features of the Dataflow viewer include the following:

- Source Code browser
- Automatic cross-probing from process/channel to source code.
- Filtering of ports and channel types.
- Process and Channel table details the characteristics of the design:
  - Channel Profiling (FIFO sizes etc), enabled from the C/RTL Cosimulation settings in the Config File Editor.

    IMPORTANT! *You must use* `cosim.enable_dataflow_profiling=true` *in the HLS config file to capture data for the Dataflow viewer, and your test bench must run at least two iterations of the top-level function.*
  - Process Read Blocking/Write Blocking/Stalling Time reported after RTL co-simulation.
  - Process Latency and II displayed.
  - Channel type and widths are displayed in the Channel table.
  - Automatic cross-probing from Process and Channel table to the Graph and Source browser.
  - Hover over channel or process to display tooltips with design information.

The Dataflow viewer can help with performance debugging your designs. When your design deadlocks during RTL co-simulation, the GUI will open the Dataflow viewer and highlight the channels and processes involved in the deadlock so you can determine if the cause is insufficient FIFO depth, for instance.

When your design does not perform as expected, the Process and Channels table can help you understand why. A process can stall waiting to read input, or can stall because it cannot write output. The channel table provides you with stalling percentages, as well as identifying if the process is "read blocked" or "write blocked."

The Dataflow Viewer displays a menu bar at the top of the report that includes the following features:

- Zoom In/Zoom Out/Zoom Fit commands
- Toggle Table command to show or hide the table beneath the graph
- Group All/Ungroup All groups associated channels together to simpligy the diagram
- Legend command for the display.

Running C/RTL Co-Simulation

Make sure the HLS component is active in the Flow Navigator, or select it from the Component menu to make it the active component in the tool. When the HLS component is the active component, the Flow Navigator enables running C Simulation, C Synthesis, C/RTL Co-simulation, Packaging, and Implementation to build and analyze the HLS component. For C/RTL Co-simulation of the HLS component select **Run** beneath the C/RTL COSIMULATION heading in the Flow Navigator.

Running Co-simulation on the HLS component requires a test bench, or test bench files, which must be loaded. Prior to running co-simulation, you must also configure the HLS component to support C/RTL co-simulation. Configure the design using the following steps.

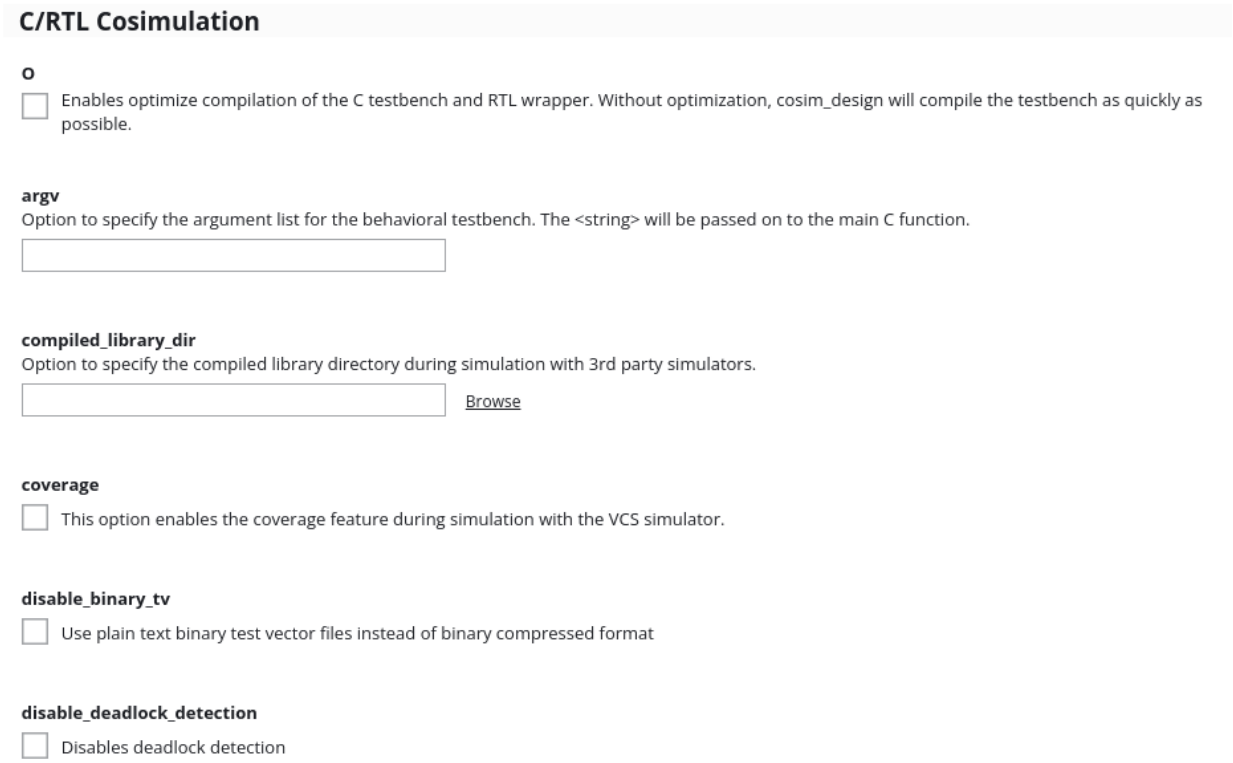


IMPORTANT! *Random input test vectors in the C/C++ test bench (for example: `std::random_device`) are not supported for co-simulation. It is recommended that random values be generated once and saved to a file which the C/C++ test bench can then use during co-simulation.*

Configure the Simulator

You can configure the simulator prior to running simulation using the C/RTL Co-Simulation section of the Config Editor, as shown below.

Figure 128: **HLS Component RTL Co-Simulation Settings**



These configuration commands let you specify how the simulation should run. These configuration commands are all documented in Co-Simulation Configuration. Refer to that content for more detailed information. Some of the settings for C/RTL Co-Simulation include the following:

- **trace_level:** Specifies the level of trace file output written to the `sim/Verilog` or `sim/VHDL` directory of the current solution when the simulation executes. Options include:
  - **all:** Output all port and signal waveform data being saved to the trace file.
  - **port:** Output waveform trace data for the top-level ports only.
  - **port_hier:** Output waveform trace data for the complete port hierarchy.
  - **none:** Do not output trace data.
- **random_stall:** Applies a randomized stall for each data transmission.
- **wave_debug:** Enables waveform visualization of all processes in the RTL simulation. This option is only supported when using Vivado logic simulator. Enabling this will launch the Simulator GUI to let you examine dataflow activity in the waveforms generated by simulation. Refer to the *Vivado Design Suite User Guide: Logic Simulation* (UG900) for more information on that tool.

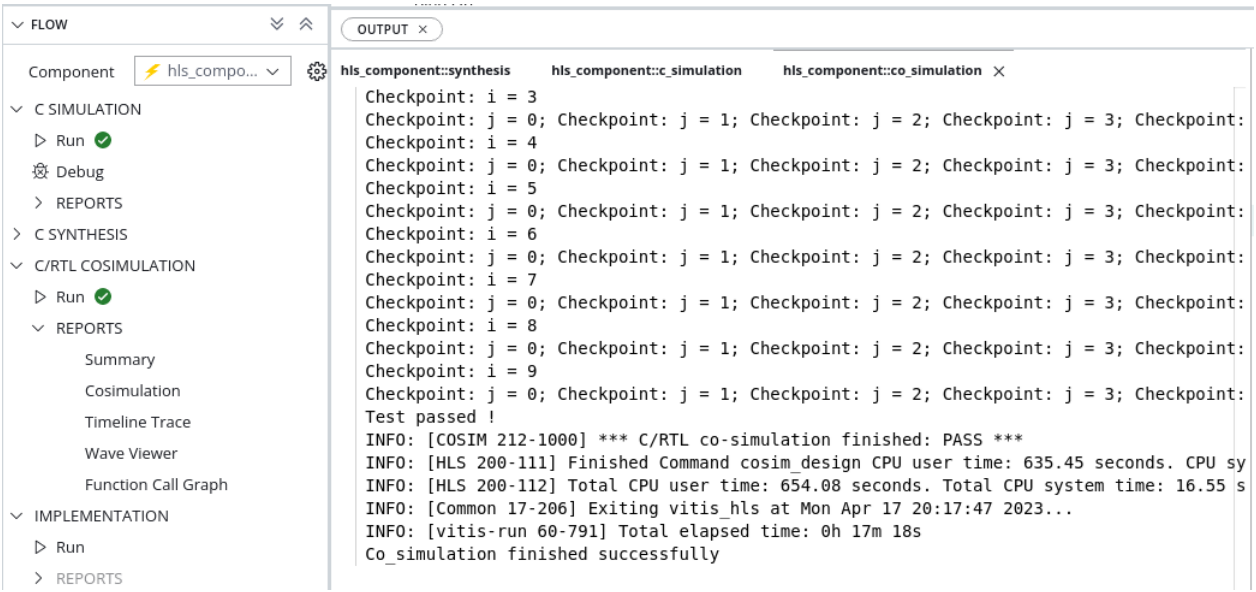
- **disable_deadlock_detection:** Disables deadlock detection, and opening the Cosim Deadlock Detection in co-simulation.
- **enable_dataflow_profiling:** Enables capturing profile data for display in the Dataflow Viewer.
- **Dynamic Deadlock Prevention:** Prevent deadlocks by enabling automatic FIFO channel size tuning for dataflow profiling during co-simulation.

Run C/RTL Co-Simulation

With the C/RTL Co-simulation setup defined in the config file you are ready to select **Run** from the Flow Navigator to begin simulation. You can track the progress of simulation in the Output window. The transcript for the synthesis run will have the top function name as `<component-name>::co_simulation` as shown below.

TIP: *The simulation run in the Vitis unified IDE uses the `vitis-run --mode hls --cosim` command.*

Figure 129: **HLS Component Running C/RTL Co-Simulation**



After the simulation is complete you should see the Co_simulation finished successfully message at the end of the transcript. You will also see the Reports folder under the Run command populated as shown in the preceding figure. The reports available after co-simulation include:

- Summary: reports the command line used and the time stamp on the results.
- Cosimulation: displays general information about the design, displays specific options used during Co-simulation, and displays performance and resource estimates for the design hierarchy. If `enable_dataflow_profiling` is enabled, performance data will be back annotated to the Dataflow Viewer report as described in Dataflow Viewer.

> **IMPORTANT!** *If you want to calculate II, you must ensure there are at least two transactions in the RTL simulation.*

- Timeline Trace: As described in Timeline Trace Viewer.
- Wave Viewer: As described in Viewing Simulation Waveforms.
- Function Call Graph: Displays the post Co-simulation call graph as described in Function Call Graph.

Output of C/RTL Co-Simulation

In the Vitis Unified IDE the hierarchy of the HLS component is structured as follows: `<workspace>/<component>/<component>/hls/`. You can select the HLS component in the Vitis Component Explorer and select **Open In Terminal** from the right-click menu to explore the contents of the component folders.

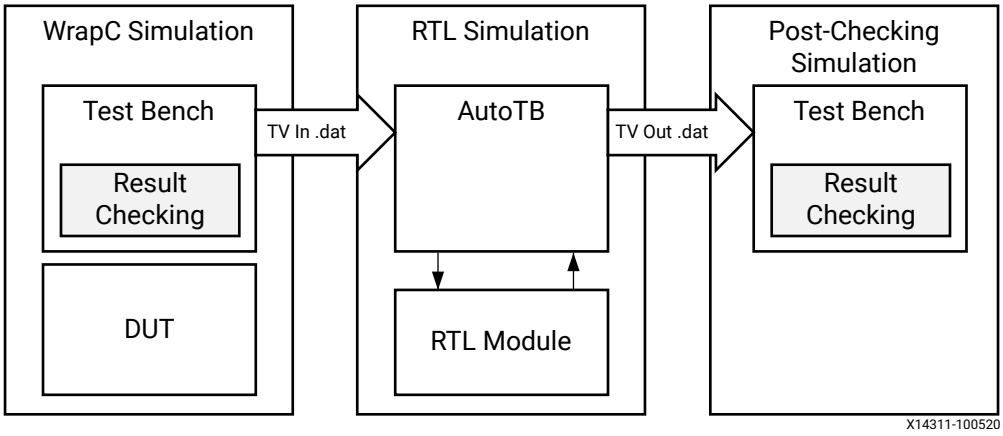
When C/RTL Cosimulation completes, the `sim` folder is created inside the solution folder. This folder contains the following elements:

- A verification folder named `sim/verilog` or `vhdl` is created for each RTL language that is verified.
  - The RTL files used for simulation are stored in the `verilog` or `vhdl` folder.
  - The RTL simulation is executed in the verification folder.
  - Any outputs, such as trace files and waveform files, are written to the `verilog` or `vhdl` folder.
- The `sim/report` folder contains the report and log file for each type of RTL simulated.
- Additional folders `sim/autowrap`, and `tv`, `wrap` and `wrap_pc` are work folders used by the HLS compiler. There are no user files in these folders.

> **TIP:** *If the `cosim.setup` option was selected in the config file, an executable is created in the `sim/<hdl>` folder but the simulation is not run. The simulation can be manually run by executing the simulation `.sh` or `.exe` in a Terminal.*

Automatically Verifying the RTL

Figure 130: **C/RTL Verification Flow**



C/RTL co-simulation uses a C test bench, running the `main()` function, to automatically verify the RTL design running in behavioral simulation. The C/RTL verification process consists of three phases:

1. The C simulation is executed and the inputs to the top-level function, or the Design-Under-Test (DUT), are saved as "input vectors."
2. The "input vectors" are used in an RTL simulation using the RTL created by C synthesis in Vivado simulator, or a supported third-party HDL simulator. The outputs from the RTL, or results of simulation, are saved as "output vectors."
3. The "output vectors" from the RTL simulation are returned to the `main()` function of the C test bench to verify the results are correct. The C test bench performs verification of the results, in some cases by comparing to known good results.

The following messages are output by the HLS compiler as verification progresses:

While running C simulation:

```
INFO: [COSIM 212-14] Instrumenting C test bench ...
   Build using "../bin/g++"
   Compiling dct_test.cpp_pre.cpp.tb.cpp
   Compiling dct_inline.cpp_pre.cpp.tb.cpp
   Compiling apatb_dct.cpp
   Generating cosim.tv.exe
INFO: [COSIM 212-302] Starting C TB testing ...
Test passed !
```

At this stage, because the C simulation was executed, any messages written by the C test bench will be output to the Console window and log file.

While running RTL simulation:

```
INFO: [COSIM 212-333] Generating C post check test bench ...
INFO: [COSIM 212-12] Generating RTL test bench ...
INFO: [COSIM 212-1] *** C/RTL co-simulation file generation completed. ***
INFO: [COSIM 212-323] Starting verilog/vhdl simulation.
INFO: [COSIM 212-15] Starting XSIM ...
```

At this stage, any messages from the RTL simulation are output in console window or log file.

While checking results back in the C test bench:

```
INFO: [COSIM 212-316] Starting C post checking ...
Test passed !
INFO: [COSIM 212-1000] *** C/RTL co-simulation finished: PASS ***
```

The following are requirements of C/RTL co-simulation:

- The test bench must be self-checking as described in Writing a Test Bench, and return a value of 0 if the test passes or returns a non-zero value if the test fails.
- Co-simulation Requirements for Interface Synthesis must be met.
- Any `arrays` or `structs` on the design interface cannot use the optimization directives listed in Unsupported Optimizations for Co-Simulation.
- Any third-party simulators must be available in the search path to be launched by the Vitis Unified IDE or `vitis-run` command.
- If a third-party simulator is used the IP simulation libraries must be compiled for use with third-party simulators as described in Simulating IP Cores.

Co-simulation Requirements for Interface Synthesis

To use the C/RTL co-simulation feature to verify the RTL design, at least one of the following conditions must be true:

- Top-level function must be synthesized using an `ap_ctrl_chain` or `ap_ctrl_hs` block-level protocol
- Design must be purely combinational
- Top-level function must have an initiation interval of 1
- Interfaces must be all arrays that are streaming and implemented with `axis` or `ap_hs` interface modes

  Note: The `hls::stream` variables are automatically implemented as `ap_fifo` interfaces.

If at least one of these conditions is not met, C/RTL co-simulation halts with the following message:

```
@E [SIM-345] Cosim only supports the following 'ap_ctrl_none' designs: (1)
combinational designs; (2) pipelined design with task interval of 1; (3)
designs with
array streaming or hls_stream ports.
@E [SIM-4] *** C/RTL co-simulation finished: FAIL ***
```

IMPORTANT! *If the design is specified to use the block-level IO protocol* `ap_ctrl_none` *and the design contains any* `hls::stream` *variables which employ non-blocking behavior, C/RTL co-simulation is not guaranteed to complete.*

If any top-level function argument is specified as an AXI4-Lite interface, the function return must also be specified as an AXI4-Lite interface.

Verification of DATAFLOW and DEPENDENCE

C/RTL co-simulation automatically verifies aspects of the DATAFLOW and DEPENDENCE directives.

If the DATAFLOW directive is used to pipeline tasks, it inserts channels between the tasks to facilitate the flow of data between them. It is typical for the channels to be implemented with FIFOs and the FIFO depth specified using the STREAM directive, or the `syn.dataflow.fifo_depth` configuration command as described in Dataflow Configuration.

If a FIFO depth is too small, the RTL simulation can stall. For example, if a FIFO is specified with a depth of 2 but the producer task writes three values before any data values are read by the consumer task, the FIFO blocks the producer. In some conditions this can cause the entire design to stall as described in Cosim Deadlock Detection.

In this case, C/RTL co-simulation issues a message as shown below, indicating the channel in the DATAFLOW region is causing the RTL simulation to stall.

```
////////////////////////////////////////////////////////////////////////////
//
// ERROR!!! DEADLOCK DETECTED at 1292000 ns! SIMULATION WILL BE STOPPED! //
////////////////////////////////////////////////////////////////////////////
//
//////////////////////////
// Dependence cycle 1:
// (1): Process: hls_fft_1kxburst.fft_rank_rad2_nr_man_9_U0
//      Channel: hls_fft_1kxburst.stage_chan_in1_0_V_s_U, FULL
//      Channel: hls_fft_1kxburst.stage_chan_in1_1_V_s_U, FULL
//      Channel: hls_fft_1kxburst.stage_chan_in1_0_V_1_U, FULL
//      Channel: hls_fft_1kxburst.stage_chan_in1_1_V_1_U, FULL
// (2): Process: hls_fft_1kxburst.fft_rank_rad2_nr_man_6_U0
//      Channel: hls_fft_1kxburst.stage_chan_in1_2_V_s_U, EMPTY
//      Channel: hls_fft_1kxburst.stage_chan_in1_2_V_1_U, EMPTY
////////////////////////////////
// Total 1 cycles detected!
////////////////////////////////////////////////////////////////
```

The Dataflow Viewer will show any processes involved in deadlock displayed in red. It will also show which channels are full (in red) versus empty (in white). In this case, review the implementation of the channels between the tasks and ensure any FIFOs are large enough to hold the data being generated.

In a similar manner, the RTL test bench is also configured to automatically check the validity of false dependencies specified using the DEPENDENCE directive. A warning message during co-simulation indicates the dependency is not false, and the corresponding directive must be removed to achieve a functionally valid design.

TIP: *The `cosim.disable_deadlock_detection=true` configuration command disables these checks.*

Unsupported Optimizations for Co-Simulation

For Vivado IP mode, automatic RTL verification does not support cases where multiple transformations are performed on arrays on the interface, or arrays within structs.

IMPORTANT! *This feature is not supported for the Vitis kernel flow.*

In order for automatic verification to be performed, arrays on the function interface, or array inside structs on the function interface, can use any of the following optimizations, but not two or more:

- Vertical mapping on arrays of the same size
- Reshape
- Partition, for dimension 1 of the array

Automatic RTL verification does not support any of the following optimizations used on a top-level function interface:

- Horizontal mapping.
- Vertical mapping of arrays of different sizes.
- Conditional access on the AXI4-Stream with register slice enabled.
- Mapping arrays to streams.

Simulating IP Cores

When the design is implemented with floating-point cores, bit-accurate models of the floating-point cores must be made available to the RTL simulator. This is automatically accomplished if the RTL simulation is performed using the Vivado logic simulator. However, for supported third-party HDL simulators, the AMD floating-point library must be pre-compiled and added to the simulator libraries.

For example, to compile the AMD floating-point library in Verilog for use with the VCS simulator, open the Vivado IDE and enter the following command in the Tcl Console window:

```
compile_simlib -simulator vcs_mx -family all -language verilog
```

This creates the floating-point library in the current directory for VCS. See the Vivado Tcl Console window for the directory name. In this example, it is `./rev3_1`.

You must refer to this library from within the Vitis unified IDE by specifying the `cosim.compiled_library_dir` config file command as described in Co-Simulation Configuration, or by running C/RTL co-simulation using the following command:

```
cosim_design -tool vcs -compiled_library_dir <path_to_library>/rev3_1
```

Analyzing RTL Simulations

When the C/RTL co-simulation completes, the simulation report opens and shows the measured latency and II. These results will differ from values reported after HLS synthesis, which are based on the absolute shortest and longest paths through the design. The results provided after C/RTL co-simulation show the actual values of latency and II for the given simulation data set (and can change if different input stimuli is used).

In non-pipelined designs, C/RTL co-simulation measures latency between `ap_start` and `ap_done` signals. The II is 1 more than the latency, because the design reads new inputs 1 cycle after all operations are complete. The design only starts the next transaction after the current transaction is complete.

In pipelined designs, the design might read new inputs before the first transaction completes, and there could be multiple `ap_start` and `ap_ready` signals before a transaction completes. In this case, C/RTL co-simulation measures the latency as the number of cycles between data input values and data output values. The II is the number of cycles between `ap_ready` signals, which the design uses to requests new inputs.

***Note*:** For pipelined designs, the II value for C/RTL co-simulation is only determined if the design is simulated for multiple transactions.

Viewing Simulation Waveforms

To view waveform data during RTL co-simulation, you must enable the following in the Config File Editor, or the config file:

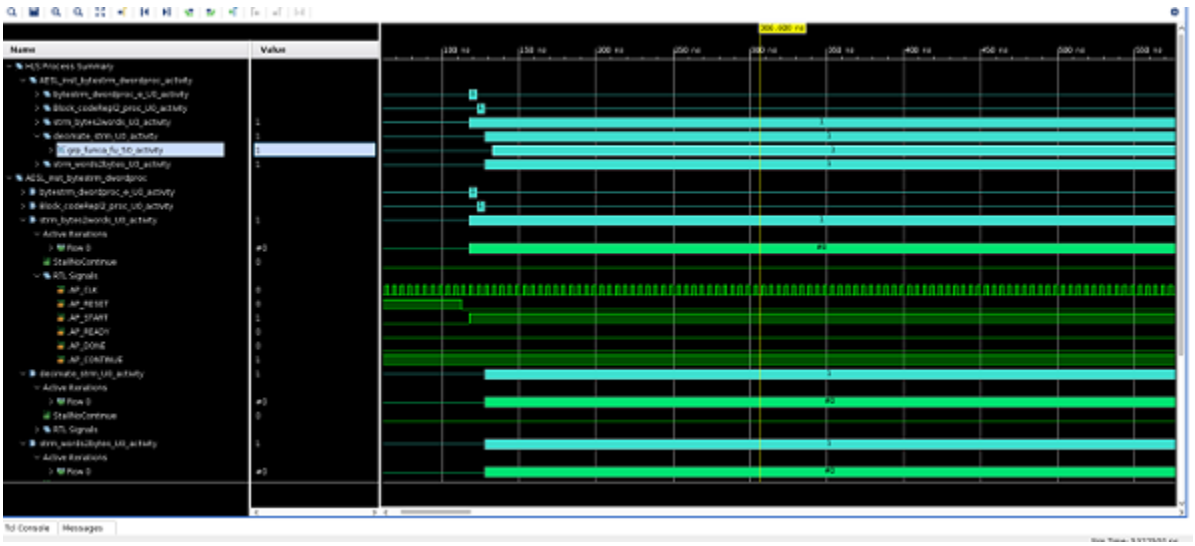
- Under tool select **xsim** as the RTL simulator, or set `cosim.tool=xsim` in the config file
- Under trace_level select **port** or **all**, or set `cosim.trace_level=port` in the config file

Vivado simulator GUI opens and displays all the processes in the RTL design. Visualizing the active processes within the HLS design allows detailed profiling of process activity and duration within each activation of the top module. The visualization helps you to analyze individual process performance, as well as the overall concurrent execution of independent processes. Processes dominating the overall execution have the highest potential to improve performance, provided process execution time can be reduced.

This visualization is divided into two sections:

- HLS process summary contains a hierarchical representation of the activity report for all processes.
  - **DUT name:** <name>
  - **Function:** <function name>
- Dataflow analysis provides detailed activity information about the tasks inside the dataflow region.
  - **DUT name:** <name>
  - **Function:** <function name>
  - **Dataflow/Pipeline Activity:** Shows the number of parallel executions of the function when implemented as a dataflow process.
  - **Active Iterations:** Shows the currently active iterations of the dataflow. The number of rows is dynamically incremented to accommodate for the visualization of any concurrent execution.
  - **StallNoContinue:** A stall signal that tells if there were any output stalls experienced by the dataflow processes (the function is done, but it has not received a continue from the adjacent dataflow process).
  - **RTL Signals:** The underlying RTL control signals that interpret the transaction view of the dataflow process.

Figure 131: **Waveform Viewer**



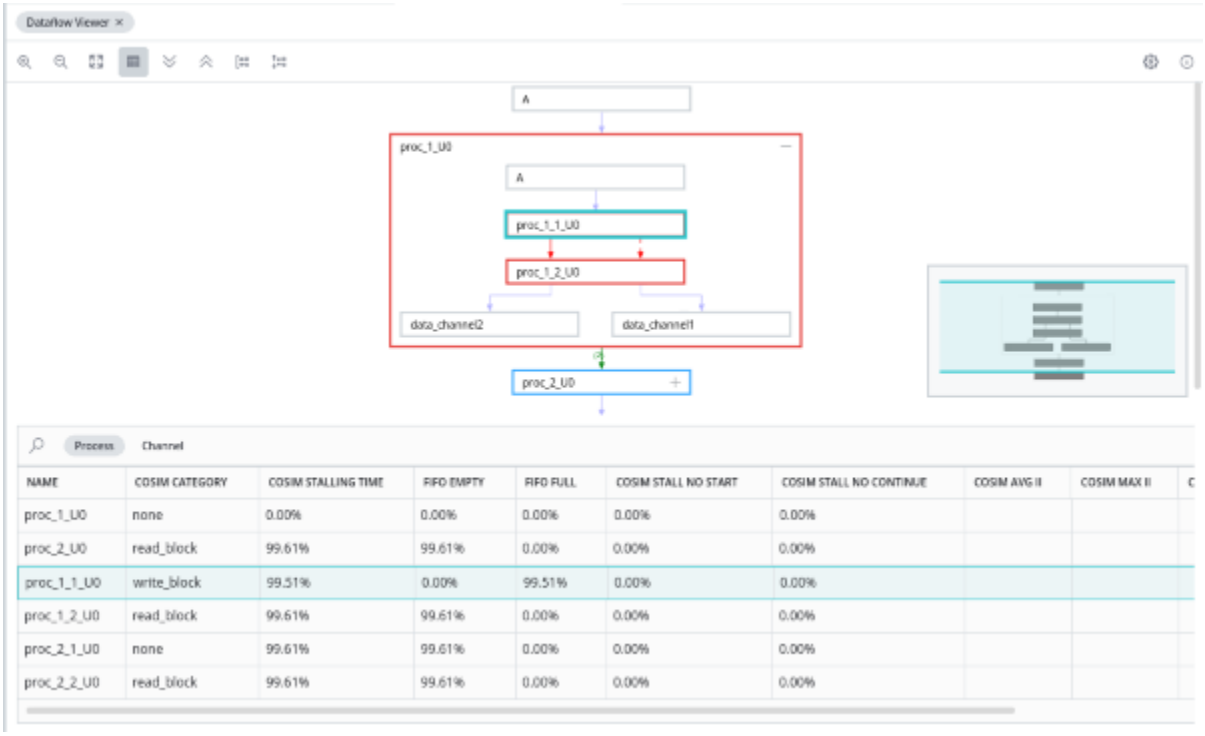
After C/RTL co-simulation completes, you can reopen the RTL waveforms in the Vivado IDE by selecting **Wave Viewer** under the REPORTS header for C/RTL COSIMULATION.

IMPORTANT! *When you open the Vivado IDE using this method, because the waveform was previously generated you can only use the waveform analysis features, such as zoom, pan, and waveform radix.*

Cosim Deadlock Detection

A deadlock is a situation in which processes inside a DATAFLOW region share the same channels, effectively preventing each other from writing to it or reading from it, resulting in both processes getting stuck. This scenario is common when there are either FIFO's or a mix of PIPOs and FIFOs as channels inside the DATAFLOW. Deadlock detection visualizes this scenario on the Dataflow Viewer report, as shown in the following figure.

Figure 132: **Deadlock Viewer**



Deadlock detection, which is enabled by default, highlights any problematic processes and channels in the Dataflow Viewer when they are encountered during C/RTL Co-simulation. The Dataflow report opens automatically opens after co-simulation detects the deadlock situation. The viewer also provides a cross-probing capability to link between the problematic dataflow channels and the associated source code. You can use the information in solving the issue with less time and effort. You can disable the feature using the `cosim.disable_deadlock_detection` config file command.

In the example shown above the dataflow region consists of two processes which are communicating through PIPO and FIFO. The first loop in `proc_1` writes 10 data items in `data_channel1`, before writing anything in `data_array`. Because of the insufficient FIFO depth the `data_channel` loop does not complete which blocks the rest of the process. Then `proc_2` stalls because it cannot read the data from `data_channel2` (because it is empty), and cannot remove data from `data_channel1`. This creates a deadlock that requires increasing the size of `data_channel1` to at least 10. This example is demonstrated in the deadlock detection portion of the 03-dataflow_debug_and_optimization tutorial on GitHub.

The output of the C/RTL Co-Simulation process is shown below. The simulation ends in an error because of the deadlock detection, and the Dataflow Viewer report is opened automatically.

```
////////////////////////////////////////////////////////////////////////////
///////
// Inter-Transaction Progress: Completed Transaction / Total Transaction
// Intra-Transaction Progress: Measured Latency / Latency Estimation * 100%
//
// RTL Simulation : "Inter-Transaction Progress" ["Intra-Transaction
Progress"] @ "Simulation Time"
////////////////////////////////////////////////////////////////////////////
////////
// RTL Simulation : 0 / 1 [0.00%] @ "105000"
////////////////////////////////////////////////////////////////////////////
//
// ERROR!!! DEADLOCK DETECTED at 132000 ns! SIMULATION WILL BE STOPPED! //
////////////////////////////////////////////////////////////////////////////
//
//////////////////////////
// Dependence cycle 1:
// (1): Process: example_example.proc_1_U0
//      Channel: example_example.data_channel1_U, FULL
// (2): Process: example_example.proc_2_U0
//      Channel: example_example.data_array_U, EMPTY
////////////////////////////////////////////////////////////////////////////
// Totally 1 cycles detected!
////////////////////////////////////////////////////////////////////////////
```

Timeline Trace Viewer

The Timeline Trace report is available after C/RTL Co-Simulation. The Timeline Trace viewer displays the runtime profile of the functions of your design. It is especially useful to see the behavior of dataflow regions after Co-simulation, as there is no need to launch the Vivado logic simulator to view the timeline.

Figure 133: **Timeline Trace Viewer**



Timeline Trace viewer displays multiple iterations through the various sub-functions of a dataflow region as shown in the preceding figure. It shows where the functions are starting and ending, and displays the Co-simulation data in tables below the timeline. To generate the Timeline Trace during C/RTL Co-simulatio you should enable `cosim.trace_level=all` and `cosim.enable_dataflow_profiling=true` options in the config file, or from the Config File Editor.

The Timeline Trace view also shows FIFO and PIPO channel stall/starve states with Full and Empty markers as shown above. In the preceding figure, you can see the `read_data` PIPO is empty, resulting in stalls 68% of the time as reported in the table below the graph.

The Dataflow Viewer displays a menu bar at the top of the report that includes the following features:

- Zoom In/Zoom Out/Zoom Fit/Zoom Full commands
- Toggle Table command to show or hide the table beneath the graph
- Expand All/Collapse All to expand or collapse the design hierarchy
- Previous Marker/Next Marker lets you move from one marker to the next in the timeline
- Delete All Markers remove markers from the timeline
- Legend command for the display

Troubleshooting C/RTL Co-Simulation

When C/RTL co-simulation completes, Vitis HLS typically indicates that the simulations passed and the functionality of the RTL design matches the initial C code. When the C/RTL co-simulation fails, Vitis HLS issues the following message:

```
@E [SIM-4] *** C/RTL co-simulation finished: FAIL ***
```

Following are the primary reasons for a C/RTL co-simulation failure:

- Incorrect environment setup
- Unsupported or incorrectly applied optimization directives
- Issues with the C test bench or the C source code

To debug a C/RTL co-simulation failure, run the checks described in the following sections. If you are unable to resolve the C/RTL co-simulation failure, see Adaptive Computing Support for support resources, such as answers, documentation, downloads, and forums.

Setting Up the Environment

Check the environment setup as shown in the following table.

Table 27: **Debugging Environment Setup**

Questions	Actions to Take
Are you using a third-party simulator?	Ensure the path to the simulator executable is specified in the system search path. When using the Vivado simulator, you do not need to specify a search path. Ensure that you have compiled the simulation libraries as discussed in Simulating IP Cores.
Are you running Linux?	Ensure that your setup files (for example `.cshrc` or `.bashrc`) do not have a change directory command. When C/RTL co-simulation starts, it spawns a new shell process. If there is a `cd` command in your setup files, it causes the shell to run in a different location and eventually C/RTL co-simulation fails.

Optimization Directives

Check the optimization directives as shown in the following table.

Table 28: **Debugging Optimization Directives**

Questions	Actions to Take
Are you using the DEPENDENCE directive?	Remove the DEPENDENCE directives from the design to see if C/RTL co-simulation passes. If co-simulation passes, it likely indicates that the TRUE or FALSE setting for the DEPENDENCE directive is incorrect as discussed in Verification of DATAFLOW and DEPENDENCE.
Does the design use `volatile` pointers on the top-level interface?	Ensure the DEPTH option is specified on the INTERFACE directive. When `volatile` pointers are used on the interface, you must specify the number of reads/writes performed on the port in each transaction or each execution of the C function.
Are you using FIFOs with the DATAFLOW optimization?	Check to see if C/RTL co-simulation passes with the standard ping-pong buffers. Check to see if C/RTL co-simulation passes without specifying the size for the FIFO channels. This ensures that the channel defaults to the size of the array in the C code. Reduce the size of the FIFO channels until C/RTL co-simulation stalls. Stalling indicates a channel size that is too small. Review your design to determine the optimal size for the FIFOs. You can use the STREAM directive to specify the size of individual FIFOs.
Are you using supported interfaces?	Ensure you are using supported interface modes. For details, see Co-simulation Requirements for Interface Synthesis.
Are you applying multiple optimization directives to arrays on the interface?	Ensure you are using optimizations that are designed to work together. For details, see Unsupported Optimizations for Co-Simulation.
Are you using arrays on the interface that are mapped to streams?	To use interface-level streaming (the top-level function of the DUT), use `hls::stream`.

C Test Bench and C Source Code

Check the C test bench and C source code as shown in the following table.

Table 29: **Debugging the C Test Bench and C Source Code**

Questions	Actions to Take
Does the C test bench check the results and return the value 0 (zero) if the results are correct?	Ensure the C test bench returns the value 0 for C/RTL co-simulation. Even if the results are correct, the C/RTL co-simulation feature reports a failure if the C test bench fails to return the value 0.
Is the C test bench creating input data based on a random number?	Change the test bench to use a fixed seed for any random number generation. If the seed for random number generation is based on a variable, such as a time-based seed, the data used for simulation is different each time the test bench is executed, and the results can vary.
Are you using pointers on the top-level interface that are accessed multiple times?	Use a `volatile` pointer for any pointer that is accessed multiple times within a single transaction (one execution of the C function). If you do not use a `volatile` pointer, everything except the first read and last write is optimized out to adhere to the C standard.

Table 29: **Debugging the C Test Bench and C Source Code** *(cont'd)*

Questions	Actions to Take
Does the C code contain undefined values or perform out-of-bounds array accesses?	Confirm all arrays are correctly sized to match all accesses. Loop bounds that exceed the size of the array are a common source of issues (for example, N accesses for an array sized at N-1). Confirm that the results of the C simulation are as expected and that output values were not assigned random data values. Consider using the industry-standard `Valgrind` application outside of the HLS design environment to confirm that the C code does not have undefined or out-of-bounds issues. It is possible for a C function to execute and complete even if some variables are undefined or are out-of-bounds. In the C simulation, undefined values are assigned a random number. In the RTL simulation, undefined values are assigned an unknown or X value.
Are you using floating-point math operations in the design?	Check that the C test bench results are within an acceptable error range instead of performing an exact comparison. For some of the floating point math operations, the RTL implementation is not identical to the C. For details, see Verification and Math Functions. Ensure that the RTL simulation models for the floating-point cores are provided to the third-party simulator. For details, see Simulating IP Cores.
Are you using AMD IP blocks and a third-party simulator?	Ensure that the path to the AMD IP simulation models is provided to the third-party simulator.
Are you using the `hls::stream` construct in the design that changes the data rate (for example, decimation or interpolation)?	Analyze the design and use the STREAM directive to increase the size of the FIFOs used to implement the `hls::stream`. By default, an `hls::stream` is implemented as a FIFO with a depth of 2. If the design results in an increase in the data rate (for example, an interpolation operation), a default FIFO size of 2 might be too small and cause the C/RTL co-simulation to stall.
Are you using very large data sets in the simulation?	Use the `reduce_diskspace` option when executing C/RTL co-simulation. In this mode, HLS only executes 1 transaction at a time. The simulation might run marginally slower, but this limits storage and system capacity issues. The C/RTL co-simulation feature verifies all transaction at one time. If the top-level function is called multiple times (for example, to simulate multiple frames of video), the data for the entire simulation input and output is stored on disk. Depending on the machine setup and OS, this might cause performance or execution issues.

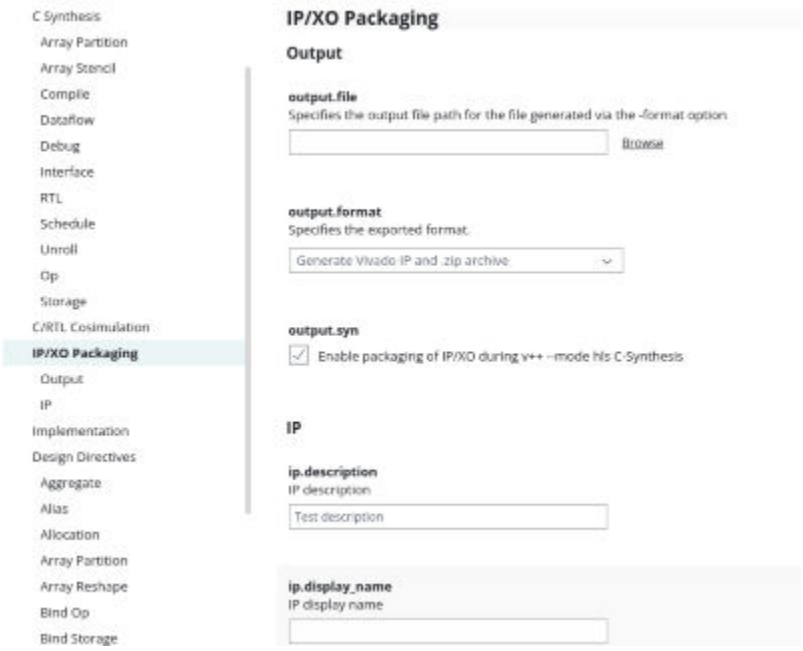
Packaging the RTL Design

The final step in the HLS component flow is to package the RTL design into a form that can be used by other tools in the design flow, such as in the Vivado Design Suite as part of a larger system design. Click the **Package** command in the Flow Navigator to export the RTL as a Vivado IP, Vitis kernel, or a synthesized checkpoint.

Configure Packaging

You can configure the packaging process to export the IP or kernel according to your settings prior to running package. The Config File Editor separates Packaging into Output and IP sections as shown below.

Figure 134: **Config File Package Settings**



Specifying the Output Format and Output File

The output.format for the RTL synthesis results must be specified. The default format is RTL, which lets the tool synthesize the Verilog and VHDL code from the C/C++ source files, but does not generate the Vivado IP or Vitis kernel as an output product. This approach lets you run synthesis quickly without having to generate hardware files at every iteration. However, to use the RTL design in downstream processes like embedded software design, or Application projects, you must generate the hardware files. The flow you are supporting in your design determines the best choice for output format.

The Package options, and their default values are listed below:

- **package.output.file:**

  The output.file name and location are determined by the tool, and default to the name of the top function specified for the HLS component. However, you can specify the name and location to override the default values.

- **package.output.format:**

The `output.format` command supports several different formats as shown in the following table.

Table 30: **Package Output Formats**

Package Format	Default Location	Comments
`package.out.format=ip_catalog`	`<comp-name>.zip`	The IP is exported as a ZIP file that can be added to the Vivado IP catalog. The `impl/ip` folder also contains the contents of the unzipped IP.
`package.out.format=xo`	`<comp-name>.xo`	The XO file output can be used for linking by the Vitis compiler in the application acceleration development flow. You can link the Vitis kernel with other kernels, and the target accelerator card, to build the `xclbin` file for your accelerated application.
`package.out.format=sysgen`	`<comp-name>.zip`	The IP is exported as a ZIP file that for use with the Vivado edition of System Generator for DSP. The `impl/ip` folder also contains the contents of the unzipped IP.
`package.out.format=rtl`	Creates Verilog and VHDL folders in the HLS component working directory.	This options lets you skip the generation of the packaged IP or XO modules to save time while iterating on the initial design.

- **package.out.syn:** Enable or disable the creation of the IP or XO during synthesis. Specify `false` to disable generation of the packaged IP or XO, or specify `true` to enable it. This options lets you skip the generation of the packaged IP or XO modules to save time while iterating on the initial design.

These selections result in the following config file entries:

```
[HLS]
package.output.format=xo
package.output.file=../../<filename>
package.output.syn=false
```

IP Configuration

When you specify `package.output.format=ip_catalog` in the HLS configuration file, you can also specify additional fields that will be applied to the generated IP, such as the Vendor, Library, Name, and Version (VLNV) of the IP.

The Configuration information is used to differentiate between multiple instances of the same IP when it is loaded into the Vivado IP catalog. For example, if an implementation is packaged for the IP catalog, and then a new solution is created and packaged as IP, the new solution by default has the same name and configuration information. If the new solution is also added to the IP catalog, the IP catalog will identify it as an updated version of the same IP and the last version added to the IP catalog will be used.

The IP options, and their default values are listed below:

- **package.ip.vendor:** xilinx.com
- **package.ip.library:** hls
- **package.ip.name:** Default to HLS component name
- **package.ip.version:** 1.0
- **package.ip.description:** An IP generated by HLS component
- **package.ip.display_name:** This field left blank
- **package.ip.taxonomy:** This field left blank
- **package.ip.xdc_file:** This field left blank
- **package.ip.xdc_ooc_file:** This field left blank



TIP: *The packaging process in the Vitis Unified IDE uses the* `vitis-run --mode hls --package` *command as described in vitis, v++, and vitis-run Commands.*

After the packaging process is complete, the ZIP file archive or XO kernel is written to the specified `package.output.file` location, or written in the HLS component folder. The exported IP file can be imported into the Vivado IP catalog and used in any design. The exported Vitis kernel can be used with the `v++ --link` command as paret of a larger system design.

Software Driver Files

For designs that include AXI4-Lite slave interfaces, a set of software driver files is created during the export process. These C driver files can be included in a Vitis embedded software development project, and used to access the AXI4-Lite slave port.

The software driver files are written to directory `solution/impl/ip/drivers` and are included in the packaged IP `export.zip`. Refer to AXI4-Lite Interface for details on the C driver files.

Output of RTL Export

The HLS compiler writes to the `impl` folder of the active solution folder when you run the **Package** command.

The output files and folders include the following:

- `component.xml`: The IP component file that defines the interfaces and architecture.
- `<component_name>.zip`: The zip archive of the IP and its contents. The zip file can be directly added to the Vivado IP catalog.
- `<component_name>.xo`: The compiled kernel object for use in the Vitis application acceleration development flow.

- `impl/ip`: The IP contents unzipped.
- `impl/ip/example`: A folder with a Tcl script used to generate the packaged IP, and a shell script to export the IP.
- `impl/report`: The report for the synthesized, or placed and routed IP is written to this folder.
- `impl/verilog`: Contains the Verilog format RTL output files.
- `impl/vhdl`: Contains the VHDL format RTL output files.



TIP: *If the Implementation step in Flow Navigator is performed, the output folders also include the `project.xpr` file that can be opened in the Vivado Design Suite.*



IMPORTANT! *You should not use the files in the `verilog` or `vhdl` output folders directly for your own use. Instead, use the packaged IP output files (`impl/ip`) for the resons stated below.*

In cases where the HLS compiler uses Vivado IP in the synthesized design, such as with floating point designs, the `impl/verilog` or `impl/vhdl` directory includes a script to create the IP during RTL synthesis. If the files in the `verilog` or `vhdl` folders are copied out and used for RTL synthesis, it is your responsibility to correctly use any script files present in those folders. If the package IP output files (`impl/ip`) are used, this process is performed automatically. If C/RTL co-simulation has been executed in the HLS component, the Vivado project also contains an RTL test bench, and the design can be simulated.

Running Implementation

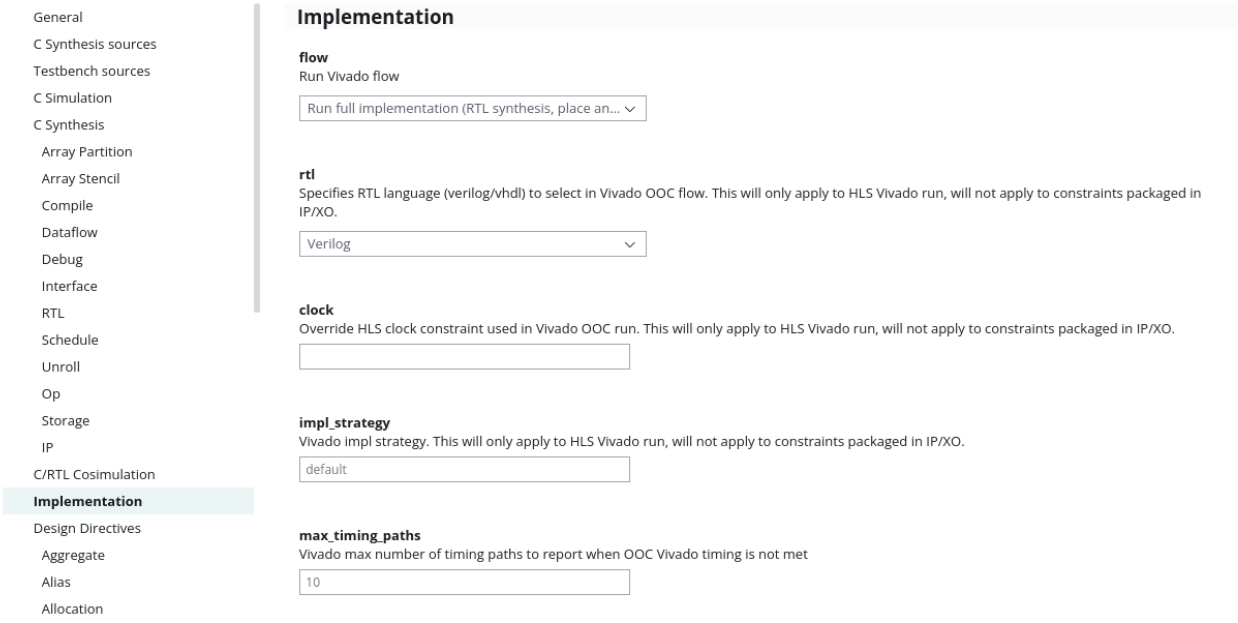
Make sure the HLS component is active in the Flow Navigator, or select it from the Component menu to make it the active component in the tool. When the HLS component is the active component, the Flow Navigator enables running C Simulation, C Synthesis, C/RTL Co-simulation, Packaging, and Implementation to build and analyze the HLS component. Select **Run** beneath the IMPLEMENTATION heading in the Flow Navigator.

When the HLS compiler reports the results of high-level synthesis, it provides an estimate of the results with projected clock frequencies and resource utilization (LUTs, DSPs, BRAMs, etc). These results are only estimates because the tool cannot know what optimizations or routing delays will be in the final synthesized or implemented design. To get a better analysis of the RTL design you can actually run Vivado synthesis and place and route on the generated RTL design, and review the results of timing and resource utilization. However, even these results are only improved estimates, because the IP or kernel being used in a larger design with other IP and kernels will yield different results.

Configure Implementation

You can configure Vivado synthesis and implementation prior to running it by using the Implementation section of the Config File Editor, as shown below, or by editing the HLS config file directly.

Figure 135: **HLS Component Implementation Settings**



The configuration commands for Implementation include:

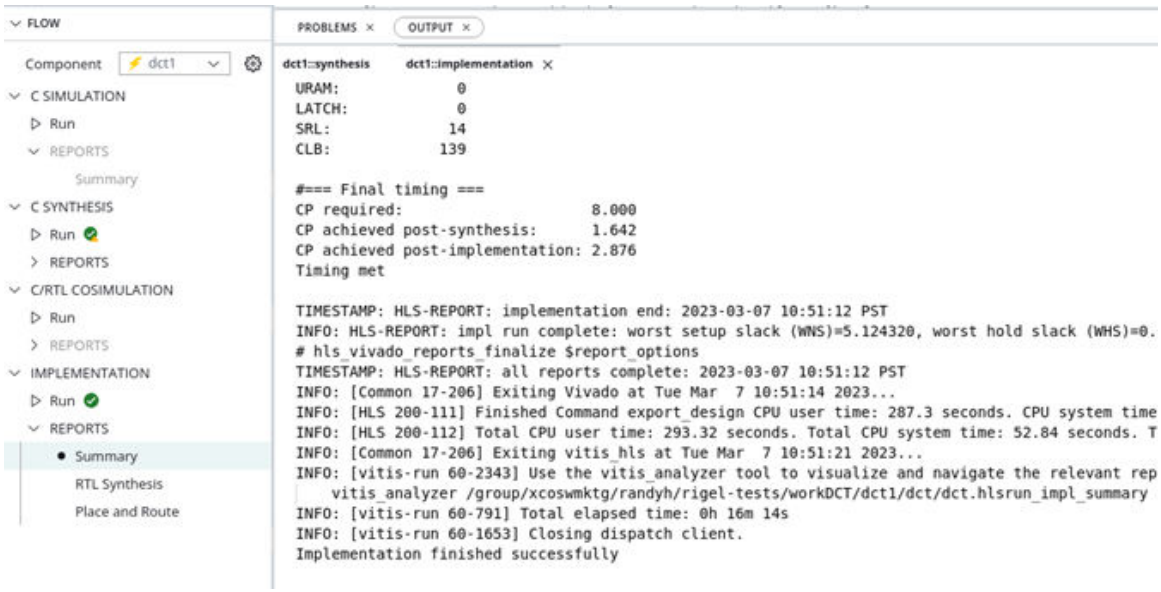
- **vivado.flow:** Specify to run only synthesis or both synthesis and implementation. Synthesis alone will run faster than both synthesis and implementation, but will lack some of details of the implementation run. The default is `syn`.
- **vivado.rtl:** Specifies the language to use when running Vivado out-of-context flow. The default is Verilog.
- **vivado.clock:** Specify the clock period to use during synthesis or implementation. When not specified, the default clock specified when the HLS component is created is used.
- **vivado.impl_strategy:** Specify the strategy to employ in the implementation run. This is only for use during the implementation run for resource utilization and timing estimates, and does not affect the generated Vivado IP or Vitis kernels.
- **vivado.max_timing_paths:** Specify the number of timing paths to extract from the Timing Summary report. The specified number of worst case paths are returned.
- **vivado.optimization_level:** This is a general feature to manage the optimizations performed by the Vivado tool. The higher the setting, the more optimizations are employed, and the longer the runtime as a result.

- **vivado.pblock:** Specifies a Pblock range or value to use during placement and routing to limit the area available for the design.
- **vivado.phys_opt:** Specify the physical optimization to run. Choices include: `none`, `place`, `route`, and `all`.
- **vivado.report_level:** Defines the report-level generated during synthesis or implementation. The report can include the utilization and timing summary, timing path details, or a failfast report, which is the default.
- **vivado.synth_design_args:** Specify options for the `synth_design` command.
- **vivado.synth_strategy:** Specify the strategy to employ in the Vivado synthesis run.

Run Implementation

With the Implementation setup defined in the config file you are ready to select **Run** from the Flow Navigator. You can track the progress of the implementation run in the Output window. The transcript for the run will have the top function name as `<component-name>::implementation` as shown below.

Figure 136: **HLS Component Running Implementation**



TIP: *The implementation run in the Vitis unified IDE uses the `vitis-run --mode hls --impl` command.*

After the implementation is complete you should see the Implementation finished successfully message at the end of the transcript. You will also see the Reports folder under the IMPLEMENTATION heading populated with the following reports:

- Summary: Reports the command line used and the time stamp on the results

- RTL Synthesis: Reports the results of synthesis including resource use and timing
- Place and Route: As described in Implementation Report



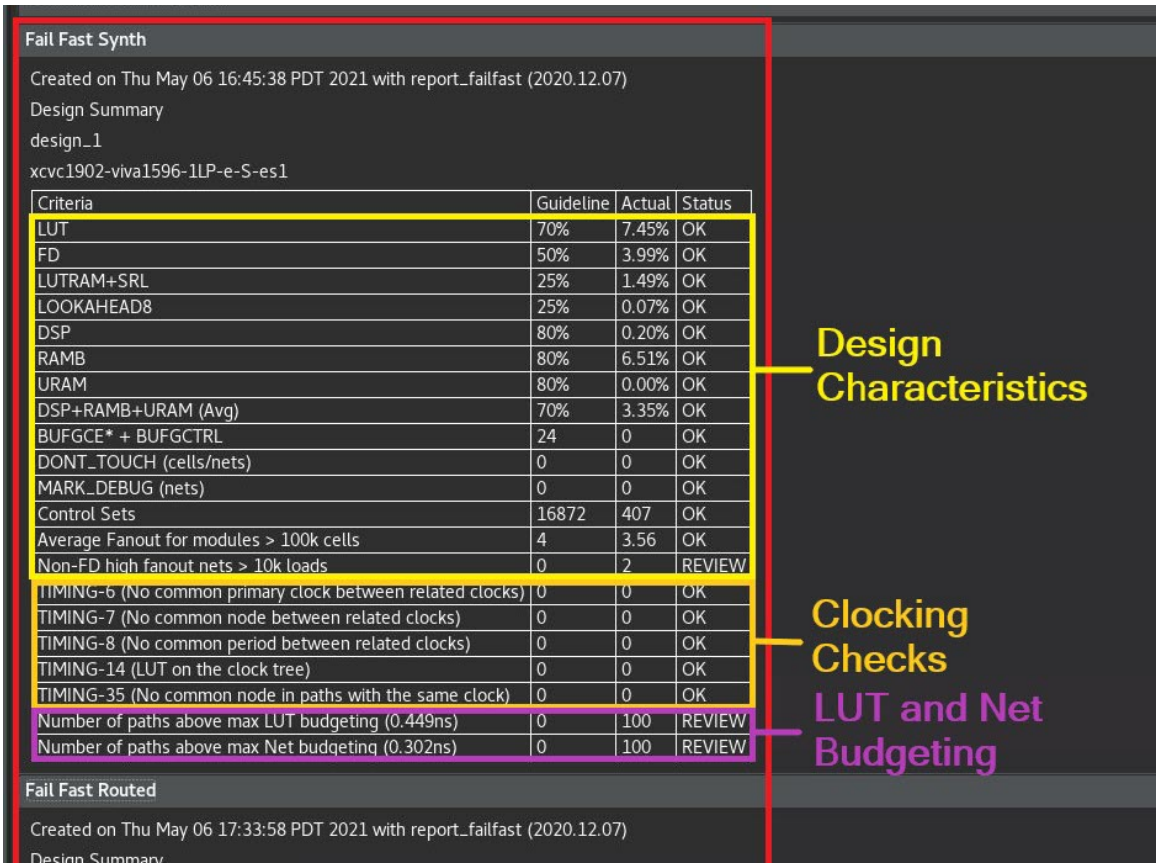
TIP: *You can cancel the Implementation run using the **Cancel Run** command from the Flow Navigator.*

Implementation Report

The Implementation Report contains the results of Synthesis and Place and Route if it was run. The sections of the report include the following:

- **General Information:** Provides general information related to the design and implementation.
- **Run Constraints and Options:** Reports the constraints and options that were set for the RTL Synthesis run and/or the Place & Route run. This shows you what constraints were set and/or modified for the run.
- **Resource Usage/Final Timing:** The Resource Usage and the Final Timing sections show a quick summary of the resources and timing achieved by either the RTL Synthesis run or the Place & Route run. These sections give a very high-level overview of the resource utilization and status on whether timing goals were met or not. The information in the succeeding sections provide details useful in debugging timing issues.
- **Resources:** A detailed per-module split up of resources is shown in this table. In addition, the tables can also show the original variable and source location information from the source code. If a particular resource was the result of a user-specified pragma, then this can also be shown in the table. This allows you to relate your C code with the synthesized RTL implementation. Inspecting this report is very beneficial because this is after Vivado has synthesized the design and therefore, functional blocks like DSPs and other logic units have all now been instantiated in the circuit.
- **Fail Fast:** The fail fast reports that Vivado provides can guide your investigation into specific issues encountered by the tool. In the fail fast report, you should look into anything with the Status of REVIEW to improve the implementation and timing closure. Different sections of the fail fast report include:
  - Design Characteristics: The default utilization guidelines are based on SSI technology devices and can be relaxed for non-SSI technology devices. Designs with one or more REVIEW checks are feasible but are difficult to implement.
  - Clocking Checks: These checks are critical and must be addressed.
  - LUT and Net Budgeting: Use a conservative method to better predict which logic paths are unlikely to meet timing after placement with high device utilization.

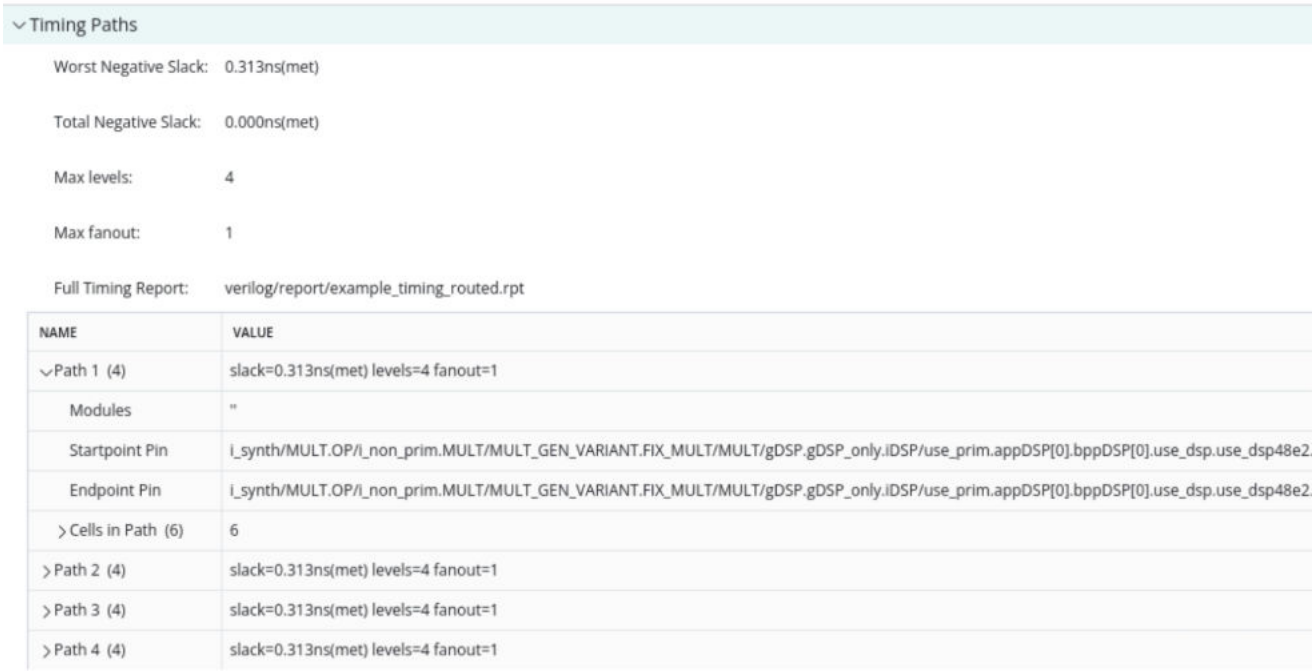
Figure 137: **Design Characteristics**



- **Timing Paths:** The Timing Paths reports show the timing critical paths that result in the worst slack for the design. By default, the tool will show the top 10 worst negative slack paths. Each path in the table has detailed information that shows the combination path between one flip-flop to another. Breaking these long combinational paths will be required to address the timing issues. So you need to analyze these paths and reason where they are coming from and map these paths back to your C code. Using both these paths and the resources table presented earlier can help in determining and correlating the path back to your source code.

  In the following figure, you can see that the top 10 negative slack paths in the Place & Route report actually have higher logic levels (9) as compared to after RTL Synthesis (5), and the max fanout also got worse (64 → 9366). This clearly shows how congestion in the design is causing high logic levels and higher fanouts which in turn causes issues for meeting timing. Using such clues, you can modify your design to remove some of this congestion either by rewriting the C code or making some different design decisions with respect to BRAM/LUTRAM/URAM resource choices.

Figure 138: **Timing Paths Report**



Optimizing the HLS Project

After analysis, you will most likely need or want to optimize the performance of your function. Even if it is performing well there might be opportunities for improvement. This section discusses the mechanisms for applying optimizations to your project. Refer to Optimizing Techniques and Troubleshooting Tips for a discussion of the various types of optimizations you can perform.

You can add optimization directives directly into the source code as compiler pragmas, using various HLS Pragmas, or you can use configuration commands as described in HLS Optimization Directives to configure and apply optimization directives to an HLS component.

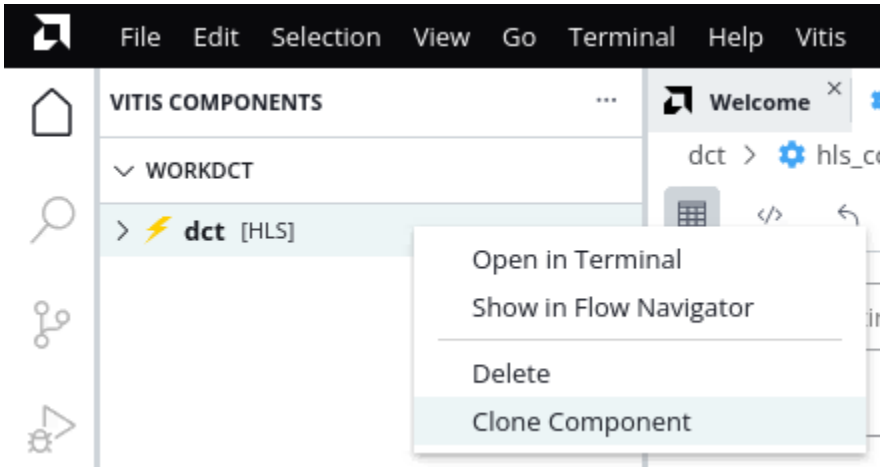
In addition to optimization pragmas and directives, the HLS compiler provides a number of configuration commands to let you manage the default results of simulation and synthesis as described in HLS Config File Commands.

Cloning HLS Components

The most typical use of the HLS compiler is to create an initial design, analyze the results, and then perform optimizations to meet the desired area and performance goals. This is often an iterative process, requiring multiple steps and multiple optimizations to achieve the desired results. When you begin to explore possible solutions to design challenges, you can clone an HLS component to let you preserve one while using the second for exploration.

To clone an HLS component right-click the component in the Vitis Component Explorer and select the **Clone Component** command, as shown in the following figure. This opens the Clone Component dialog box, prompting you for a new name for the component.

Figure 139: **Clone Component**



After creating the cloned component, the design will include the same source files, test bench files, top-level function, and HLS configuration file settings. You can edit the configuration file settings or other settings of the new HLS component and begin working through the build process. Keep the original HLS component in its current state, or develop two different branching approaches to the design.

Component Comparison Feature

The Component Comparison tool allows users to compare a set of HLS components in terms of performance and utilization. Reports can be generated in tabular or graphical format. In addition, the report data can be exported in CSV format.

Compare Reports can be generated for HLS Components only and the components must exist in an opened Vitis unified software platform workspace. To use this feature, the HLS component(s) selected for comparison should have at least one of the following stages completed successfully:

- C Synthesis
- C/RTL Co-simulation
- Implementation

IMPORTANT! *Only completed stage data will be available in the generated HLS Compare Report.*

To generate a report, go to the VITIS COMPONENTS panel in the Vitis Unified IDE and with the mouse hover over any of the HLS components, select its "HLS Compare Reports" icon. The icon will become visible when you hover over it. Alternatively, go to View and then select HLS Compare Reports.

The HLS Compare Report panel will appear. Check the boxes of the desired components from the list of available HLS components. Finally, select 'Compare'.

An HLS Compare Report file in tabular format is generated and visible in the editor panel. Component information and metrics values are populated in tabular form for the completed stages of the selected HLS components.

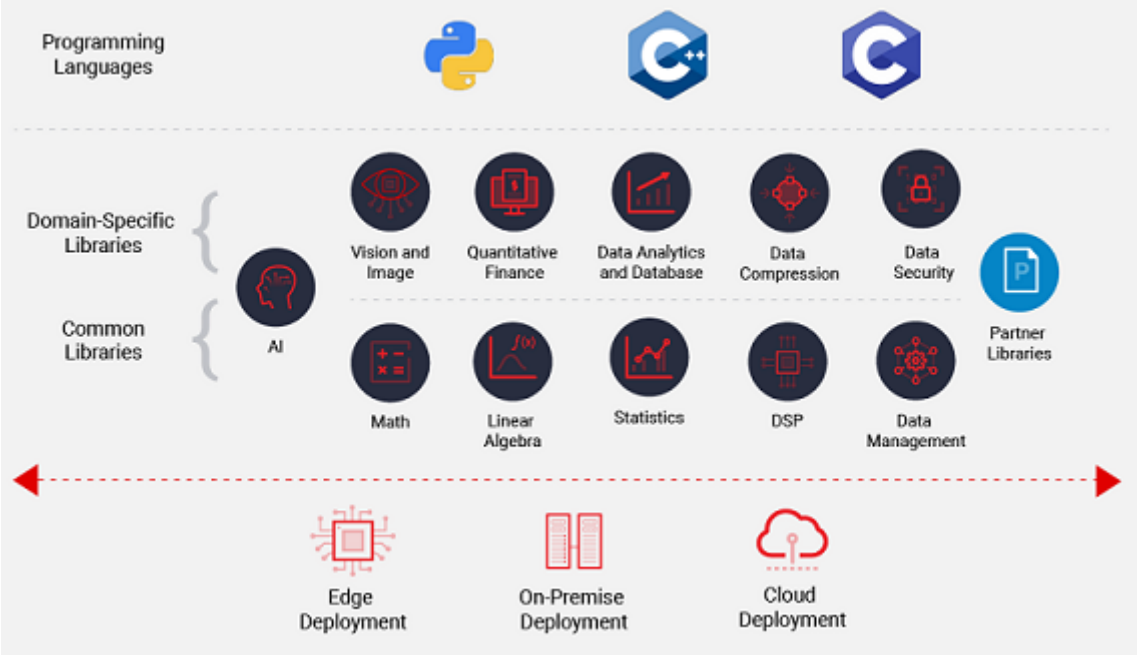
- To export the tabular data as a CSV formatted file, select the export (a cloud with a down arrow) icon in the upper right corner of the report. A Compare Reports Export CSV File panel will appear. Enter the file name and select Save.
- To generate an HLS Compare Report in graphical format, from the tabular HLS Compare Report select the Graph Compare (a bar graph) icon in the upper right corner of the report.
- To switch back to the tabular report, select the Table Compare (a spreadsheet) icon in the upper right corner of the report.

L1 Library Wizard Flow

The L1 Library Wizard Flow feature provides a tool-assisted approach to access a subset of the Vitis Accelerated Libraries HLS functions. You can select library functions and generate examples as well as parametrized template functions that can be incorporated into your design. In this way, you can quickly add IP-based functionality to their design from pre-built HLS-based IP.

The Vitis Libraries contain an extensive set of open-source, performance-optimized libraries that offer out-of-the-box acceleration with minimal to zero-code changes to your existing applications. The common Vitis accelerated libraries for Math, Statistics, Linear Algebra, and digital signal processing (DSP) offer a set of core functionality for a wide range of diverse applications. The domain-specific Vitis accelerated libraries offer out-of-the-box acceleration for workloads like Vision and Image Processing, Quantitative Finance, Database and Data Analytics, and Data Compression.

Figure 140: **Vitis Libraries**



Three types of implementations are provided in the Vitis libraries, namely L1 primitives, L2 kernels and L3 software APIs. These implementations are organized in their corresponding directories L1, L2 and L3. The L1 primitive implementations can be leveraged by FPGA hardware developers. The L2 kernel implementations provide usage examples for Vitis host code developers. The L3 software API implementations provide C, C++, and Python function interfaces to allow pure software developers to offload operations to pre-built FPGA images, also called overlays. The Vitis vision library provides a software interface for computer vision functions accelerated on FPGA and AI Engine devices.

The Vitis L1 Library Flow feature offers a tool-assisted approach to quickly incorporate L1 Library HLS functions into your design for the Vitis Solver and Vision Library subfolders only. The Solver Library provides a collection of matrix decomposition operations, linear solvers, and eigenvalue solvers. The Vitis vision library has been designed to work in the Vitis development environment and provides a software interface for computer vision functions accelerated on an FPGA using Vitis HLS. Vitis vision library functions are mostly similar in functionality to their OpenCV equivalent.

***Note*:** To use the Vitis Vision library, the OpenCV library v4.4.0 must be installed and accessible on your system. For instructions on how to install the OpenCV library on Windows, see Install/Setup OpenCV on Windows 10 and for Linux, see Compiling and Installing OpenCV libraries for use with Vision library.

There are three tool-assisted flows to access the L1 HLS functions:

1. Examples view
2. Create component from Library from the Welcome Page

3. Auto-completion from xf:: namespace in Vitis Unified IDE editor

> **TIP:** *To ensure access to the L1 Library HLS functions, ensure the Solver and Vision subfolders of the Vitis Libraries have been downloaded.*

Flow 1 - Examples - Create L1 HLS function examples

The Examples VIEW flow allows you to select and generate an L1 Library example. The result is a specified L1 Library HLS component project with both test bench and source code files.

To generate an L1 Library Example:

1. Select the Examples icon in the Vitis Unified IDE.
2. In the Examples panel, go to the **HLS Examples → VItis Accelerated Libraries Repository → Vitis Vision Library → L1 → Examples**.
3. Select the desired example, for example, AMD 3D LUT L1 Test.
4. Click on the **Create HLS Component Template** button.
5. Set the component name and location. Select **Next**.
6. Select **Finish** to generate the example HLS component.

Flow 2 - Create component from Library

The "Create component from Library" flow allows you to select an HLS template function and parametrize it. An HLS component project is created with the instantiated template function in a C++ wrapper for the synthesizable file. No test bench file is provided. To perform this flow:

1. Select **Create Component from Library** from the Welcome page.
2. Enter the component name and location in the "Create Component from Library" pop-up panel and select **Next**.
3. Select the desired L1 HLS function and select **Next**.
4. In the parametrization panel, configure the parameters.
    a. Grey text fields are required. In some cases, legal integer ranges are specified. For example, "int(16~2160)" means the field value can be an integer in the range 16 to 2160 inclusive.
5. After defining the parameters, select "Next."
6. Select the Part then select "Next." You can use the default part by selecting "Next" immediately.
7. Edit the Settings then select "Next." You can use default settings by selecting "Next" immediately.
8. Review the new HLS component settings and select "Finish."

Flow 3 - Auto-completion from the xf:: namespace in IDE Editor

The "auto-completion from the xf:: namespace" flow provides a way to insert a parametrized HLS template function into source code that is being actively edited.

1. In the Vitis Unified IDE editor, open a C++ source code file.
2. Type "xf::" in an empty line in the file editor panel.
3. A list of available HLS functions will be displayed for auto-completion. Select a function, for example, xf::cv::absdiff.
4. Once the function is selected, the HLS function's parametrization window opens, fill in the Parameters and Ports sections of the panel.
    a. In the Parameters section, grey text fields are required. In some cases, legal integer ranges are specified. For example, "int(16~2160)" means the field value can be an integer in the range 16 to 2160 inclusive.
    b. In the Ports section, you can rename the arguments to match the source code context.

Creating HLS Components from the Command Line

While the prior content described building and running an HLS component using the Vitis Unified IDE, this section focues on doing that using the command line tools that are available.

The `v++ -c --mode hls` compiler command is used to build an HLS component. The command uses a configuration file command language. Running C simulation, Code Analyzer, C/RTL Co-simulation, implementation, and export all rely on the `vitis-run` command. The different steps for the command-line flows are described below.



TIP: *The following examples are based on the DCT design in the Getting Started With Vitis HLS. The example includes the HLS component, with source files and test bench.*

Running C-Synthesis

To build the `dct` HLS component the `v++` command-line will look as follows:

```
v++ -c --mode hls --config ./dct/hls_config.cfg --work_dir dct
```

Where:

- `--config` specifies a config file with the compiler directives for the build, and to configure the simulator for the run
- `--work_dir` provides a work directory to build the component



TIP: *When creating an HLS component from the command line, the `--work_dir` specifies the HLS component folder, and the parent folder of the `--work_dir` becomes the workspace for launching the Vitis IDE.*

The contents of a configuration file can vary, but for synthesis the `dct` HLS component requires the following commands in the `hls_config.cfg` file:

```
part=xczu9eg-ffvb1156-2-e

[hls]
syn.file=./src/dct.cpp
syn.top=dct
```

```
flow_target=vitis
clock=8ns
clock_uncertainty=12%
syn.output.format=rtl
syn.directive.pipeline=dct_2d II=4
```

IMPORTANT! *If your HLS configuration file uses `platform=` instead of `part=` then you must also specify `freqhz=` instead of `clock=` as shown here to change the default clock frequency of the platform.*

The success of the synthesis command largely depends on the contents of the configuration file. There are a few key required elements, and then there are a number of options that you can specify. From the config file provided above, the required elements for synthesis are the `part`, `syn.file`, and `syn.top`. The `flow_target`, `clock`, and `clock_uncertainty` are not required except to override the default values. The `syn.directive.xxx` commands are used to provide specific optimization to the synthesis of the function.

Running C Simulation or Code Analyzer

To run the HLS component in C simulation or in Code Analyzer, use the `vitis-run` command. C simulation does not require C Synthesis to have been run as it does not require the RTL code generated by synthesis.

```
vitis-run --mode hls --csim --config ./dct/hls_config.cfg --work_dir dct
```

Where:

- `--csim` specifies the target for the run.
- `--config` specifies a config file as indicated for synthesis, but includes C simulation specific requirements as shown below.
- `--work_dir` provides a work directory to build the component as indicated for synthesis.

The contents of a configuration file can vary, but for C simulation the config file requires the source files and top specified for synthesis, but also require test bench and input files, as well as csim configuration settings as explained in C-Simulation Configuration:

```
part=xczu9eg-ffvb1156-2-e

[hls]
clock=8ns
clock_uncertainty=12%
flow_target=vitis
syn.output.format=rtl
syn.file=./src/dct.cpp
syn.file=./src/dct.h
syn.top=dct
tb.file=./src/dct_coeff_table.txt
tb.file=./src/dct_test.cpp
```

```
tb.file=./src/in.dat
tb.file=./src/out.golden.dat
csim.clean=true
csim.code_analyzer=false
syn.directive.pipeline=dct_2d II=4
```



TIP: *Change* `csim.code_analyzer` *to* `true` *to enable the Code Analyzer as well as simulation.*

Running C/RTL Co-Simulation

To run C/RTL Co-simulation on the HLS component use the `vitis-run` command as shown below:

```
vitis-run --mode hls --cosim --config ./dct/hls_config.cfg --work_dir dct
```

The contents of the configuration file required for C/RTL Co-Simulation include the following:

```
part=xczu9eg-ffvb1156-2-e

[hls]
clock=8ns
clock_uncertainty=12%
flow_target=vitis
syn.output.format=rtl
syn.file=./src/dct.cpp
syn.file=./src/dct.h
syn.top=dct
tb.file=./src/dct_coeff_table.txt
tb.file=./src/dct_test.cpp
tb.file=./src/in.dat
tb.file=./src/out.golden.dat
syn.directive.pipeline=dct_2d II=4
cosim.enable_dataflow_profiling=true
cosim.enable_fifo_sizing=true
cosim.trace_level=port
cosim.wave_debug=true
```

Note: The `hls_config.cfg` file above shows some of the settings for the C/RTL co-simulation tool. Refer to Co-Simulation Configuration for more information.

Running Implementation

To run Vivado synthesis or implementation on the HLS component use the `vitis-run` command as shown below:

```
vitis-run --mode hls --impl --config ./dct/hls_config.cfg --work_dir dct
```

Exporting the IP/XO

To export a Vivado IP or Vitis kernel from the synthesized HLS component, you can use the Vitis-run --package command as shown below:

```
vitis-run --mode hls --package --config ./dct/hls_config.cfg --work_dir dct
```

The contents of the configuration file required to export the package IP or XO include the following:

```
part=xcvu9p-flga2104-2-i

[hls]
syn.file=/group/xcoswmktg/randyh/rigel-tests/03-Vitis_HLS/reference-
files/src/dct.cpp
syn.top=dct
syn.output.format=xo
```

***Note*:** The `--package` command exports an IP or an XO from the previously synthesized HLS component. You can export a Vitis kernel as an IP. However, you cannot export a Vivado flow IP as an XO unless it meets the specific requirements of the Vitis kernel.

AMD

Section IV

Vitis HLS Command Reference

This section contains the following chapters:

- vitis, v++, and vitis-run Commands
- HLS Config File Commands
  - HLS Configuration Options
  - HLS Optimization Directives
- HLS Pragmas
- HLS Tcl Commands
  - Configuration Commands
  - Project Commands
  - Optimization Directives

vitis, v++, and vitis-run Commands

vitis command

Launch the Vitis Unified IDE using the `vitis` command with the following syntax:

```
vitis -w <workspace>
```

You can also use a number of other modes when working with the Vitis tool. You can use `-h` or `--help` to show the supported modes of the `vitis` command.

```
vitis -h

Syntax: vitis [-g (default) | -a | -w | -i | -s | -h | -v]

Options:
  -a/--analyze [<summary file | folder>]
         Open the summary file in the Analysis view.
         Opening a folder opens the summary files found in the folder.
         If no file or folder is specified, opens the Analysis view.
  -g/--gui
         Launches Vitis IDE (default option).
  -w/--workspace <workspace_location>
         Launches Vitis IDE with the given workspace location.
  -i/--interactive
         Launches Vitis python interactive shell.
  -s/--source <python_script>
         Runs the given python script.
  -j/--jupyter
         Launches Vitis Jupyter Web UI.
  -h/--help
         Display help message.
  -v/--version
         Display Vitis version.
```

The Vitis Unified IDE launches either the `v++ -c --mode hls` command to synthesize the HLS component, or launches the `vitis-run` command to run C Simulation, Code Analyzer, C/RTL Co-Simulation, Package, and Implementation as described in Creating HLS Components from the Command Line. These commands are explained below.

v++ Command

The `v++ -c --mode hls` command is used to synthesize the HLS component for either the Vivado IP or Vitis kernel flow targets as described in Target Flow Overview, or compile the HLS component for use in software emulation of the System Project in the Vitis Unified IDE as described in Building and Running an HLS Component.

When using the v++ command in HLS component mode, the configuration file specifies a number of options for the tool and optimization directives for the source code as described in HLS Config File Commands. You can see some of these options by running the following command:

```
v++ -c --mode hls -h
usage: v++ [options] <input file...>

Generic options:
  -D [ --define ] arg     <name=definition> Predefine name as a macro with
                          definition. This option is passed to the openCL
                          preprocessor.
  -I [ --include ] arg    Add the directory to the list of directories to
be
                          searched for header files. This option is passed
to
                          the openCL preprocessor
  -c [ --compile ]        Run a compile mode
  -h [ --help ]           Print usage message
  -o [ --output ] arg     Set output file name. Default: a.xclbin (link,
                          build), a.xo (compile)
  -v [ --version ]        Print version information
  --config arg            Config file
  --input_files arg       Specify input file(s). Input file(s) can also be
                          specified positionally without using the
                          --input_files option.
  --log_dir arg           Specify a directory to copy internally generated
log
                          files to
  --report_dir arg        Specify a directory to copy report files to
  --work_dir arg          Specify a working directory for output files and
                          directories

Options allowed in a config file:

  --part arg              Specify a part

[hls] section:
  --hls.* arg             Specify hls options
```

vitis-run Command

The `vitis-run` command is used to run process steps on the HLS component using the `--mode hls` option. To see what command options are supported for the `vitis-run` command use the following command:

```
vitis-run --mode hls -h
usage: vitis-run [options] <input file...>

Generic options:
  -h [ --help ]           Print usage message
  -v [ --version ]        Print version information
  --config arg            Config file
  --cosim                 Specify hls run option cosim
  --csim                  Specify hls run option csim
  --impl                  Specify hls run option impl for vivado ooc
  --input_file arg        Specify an input file as positional argument with
                          --tcl.
```

```
  --mode arg                [hls] Specify a mode.
  --package                 Specify hls run option package
  --tcl                     Option to support Tcl script flow
  --work_dir arg            Specify a working directory for output files and
                            directories

Options allowed in a config file:

  --part arg                Specify a part
  --platform              Specify a platform to define the part

[hls] section:
  --hls.* arg               Specify hls options
```

hls_init.tcl

When you start AMD Vitis™ HLS the tool looks for a Tcl initialization script in the following location:

1. User Specific: In a local user directory, for all versions of the tool:
   - • On Windows: `%APPDATA%/Xilinx/HLS_init.tcl`
   - • On Linux: `$HOME/.Xilinx/HLS_init.tcl`

The `HLS_init.tcl` lets you use AMD Vitis™ HLS commands to initialize the tool prior to opening a project.



TIP: *There is no `HLS_init.tcl` script in the software installation. You must create one if needed.*

HLS Config File Commands

The HLS compilation mode provides access to numerous features for the development, optimization, analysis, and export of Vitis kernels (`.xo`) or Vivado IP (`.zip`) files. The HLS mode can be reached by using the following command, or by launching the Vitis Unified IDE:

```
v++ -c --mode hls -h [options] <input_files...>
```

The HLS compilation options should be entered into a configuration file for use with the `v++` command using the `--config` option. The HLS options should be placed under a section head of `[hls]` in the config file. For example the following config file specifies the part, the source file, the test bench files, and the flow target. Notice that `part` is not specified under the `[HLS]` header because this is a general option for the `v++` compiler.

```
part=xcvu11p-flga2577-1-e

[hls]
clock=8
flow_target=vitis
syn.file=../../src/dct.cpp
syn.top=dct
tb.file=../../src/out.golden.dat
tb.file=../../src/in.dat
tb.file=../../src/dct_test.cpp
tb.file=../../src/dct_coeff_table.txt
syn.output.format=xo
clock_uncertainty=15%
```

Options Allowed in a Config File or on the Command Line

When working with the Vitis Unified IDE the following `v++` command-line options are available for use from within a configuration file. These options do not belong under any header in the configuration file.

- **--platform <arg>:** Specify path to a platform specification (`.xpfm`) to target the current component or project to.
- **--freqhz <arg>:** Specify a clock frequency in Hz.

  IMPORTANT! *If your HLS configuration file specifies `platform=` instead of `part=` you must use also use `freqhz=` instead of `clock=`. This is because the platform specifies a clock in `freqhz` form, and you can only override it using the same form.*

- **--part <arg>:** Specify a target device for the HLS component.

HLS General Options

IMPORTANT! *The following options must appear in the HLS configuration file under the* `[hls]` *header.*

- **clock:**

  Specify the clock period in `ns` or MHz (`ns` is default). If no period is specified a default period of 10 ns is used.

  ```
  clock=8ns
  ```

  IMPORTANT! *If your HLS configuration file uses* `platform=` *instead of* `part=` *then you must also specify* `freqhz=` *instead of* `clock=` *as shown here to change the default clock frequency of the platform.*

- **clock_uncertainty:**

  Specify how much of the clock period is used as a margin by HLS. The margin of uncertainty is subtracted from the clock period to create an effective clock period. The clock uncertainty is defined in `ns`, or as a percentage of the clock period. The clock uncertainty defaults to 27% of the clock period. When specifying a value, the default units is `ns` but % or MHz can also be used.

  ```
  clock_uncertainty=15%
  ```

- **flow_target:**

  Set the flow target to synthesize either a Vitis kernel (`vitis`) or a Vivado IP (`vivado`). The Vitis kernel is used in the Application Acceleration flow, while the Vivado IP can be used in the embedded software design flow.

  IMPORTANT! *There are differences in the interface definition supported by Vivado IP or Vitis kernels.*

C-Synthesis Sources

- **syn.cflags:**

  Defines compilation flags to be applied to all `syn.file` defined source files for use during synthesis.

  ```
  syn.cflags=-I../../src/
  ```

- **syn.csimflags:**

Defines compilation flags to be applied to all `syn.file` source files for use during C-simulation or RTL/Co-simulation.

- **syn.file:**

  Specify the file path and name of a source file to be used during synthesis of the HLS component. Multiple files require multiple `syn.file` statements.

  The file paths can be specified as either absolute or relative, where relative paths are relative to the location of the config file, whether inside the HLS component or outside the component.

  ```
  syn.file=../../src/dct.cpp
  ```

- **syn.file_cflags:**

  Apply a compilation flag for synthesis to the specified source file. Specify the file path and name first, followed by a comma, followed by the cflags:

  ```
  syn.file_cflags=../../src/dct.cpp,-I../../src/
  ```

- **syn.file_csimflags:**

  Apply a compilation flag for simulation to the specified source file. Specify the file path and name first, followed by a comma, followed by the csimflags.

  ```
  syn.file_csimflags=../../src/dct.cpp,-Wno-unknown-pragmas
  ```

- **syn.blackbox.file:**

  Specify the JSON file to be used for an RTL blackbox. The information in this file is used by the HLS compiler during synthesis and when running RTL/Co-simulation.

  ```
  syn.blackbox.file=../../RTL/fft.json
  ```

- **syn.top:**

  Specifies the name of the function to be synthesized as the top-level function for the HLS component. This can be used to identify the top function in source code where multiple functions are defined.

  ```
  syn.top=dct
  ```

  IMPORTANT! *Any functions called by the top-level function will also become part of the HLS component.*

Test bench Sources

- **tb.cflags arg:**

Defines compilation flags to be applied to all `tb.file` defined source files for use during simulation or co-simulation.

```
tb.cflags=-Wno-unknown-pragmas
```

- **tb.file arg:**

  Specify the file path and name of a test bench source file to be used during simulation or co-simulation of the HLS component. Multiple files require multiple `tb.file` statements.

  The file paths can be specified as either absolute or relative, where relative paths are relative to the location of the config file, whether inside the HLS component or outside the component.

```
tb.file=../../src/dct_test.cpp
```

- **tb.file_cflags arg:**

  Apply a compilation flag for simulation or co-simulation to the specified test bench source file. Specify the file path and name first, followed by a comma, followed by the cflags:

```
syn.file_cflags=../../src/dct.cpp,-Wno-unknown-pragmas
```

Array Partition Configuration

The `syn.array_partition` commands specify the default behavior for array partitioning for the whole design. These settings can be overridden for specific arrays using the `syn.directive.array_partition`.

- **syn.array_partition.complete_threshold <value>:**

  Sets the threshold for completely partitioning arrays. Arrays which have fewer elements than the specified value will be completely partitioned into individual elements.

```
syn.array_partition.complete_threshold=12
```

- **syn.array_partition.throughput_driven [auto | off]:**

  Enable automatic partial and/or complete array partitioning.

  - `auto`: Enable automatic array partitioning with smart trade-offs between area and throughput. This is the default value.
  - `off`: Disable automatic array partitioning.

```
syn.array_partition.throughput_driven=off
```

C-Simulation Configuration

The `csim` options apply to the C-simulation process used to validate the C/C++ language for the design. Refer to Running C Simulation in the *Vitis High-Level Synthesis User Guide* (UG1399) for more information.

- **csim.O:**

  Enables optimizing compilation which eliminates debug constructs. The default is false and compilation is done in debug mode to enable debugging.

  ```
  csim.O=true
  ```

- **csim.argv:**

  Specifies an argument list for the behavioral test bench. The specified `<arg>` will be passed to the `main()` function in the C test bench.

  ```
  csim.argv=arg1 arg2
  ```

- **csim.clean:**

  Enables clean build. The default is false. Without this option the design will compile incrementally.

  ```
  csim.clean=true
  ```

- **csim.code_analyzer:**

  Enable code analysis and interactive Code Analyzer report.

  ```
  csim.code_analyzer=0
  ```

- **csim.ldflags:**

  Specifies the options passed to the linker for simulation. This option is typically used to pass include path information or library information for the C/C++ test bench.

  ```
  csim.ldflags=ldExample
  ```

- **csim.mflags:**

  Provides for options to be passed to the compiler for C simulation. This is typically used to speed up compilation.

  ```
  csim.mflags=mExample
  ```

- **csim.setup:**

When this option is specified, the simulation binary will be created in the `csim` directory of the current HLS component, but simulation will not be executed. Simulation can be launched later from the compiled executable. The default is false, and simulation is run after setup is complete.

```
csim.setup=true
```

Co-Simulation Configuration

The `cosim` options apply to the C/RTL Co-Simulation process used to validate the RTL produced by HLS synthesis. This includes using the C/C++ test bench used earlier in C-simulation and using the RTL design in behavioral simulation as described in C/RTL Co-Simulation in Vitis HLS in the *Vitis High-Level Synthesis User Guide* (UG1399).

- **cosim.O:**

  Enables optimizing compilation which eliminates debug constructs. The default is false and compilation is done in debug mode to enable debugging. Enabling optimized compilation of the C/C++ test bench and RTL wrapper increases compilation time, but results in better runtime performance.

  ```
  cosim.O=true
  ```

- **cosim.argv:**

  Specifies an argument list for the behavioral test bench. The specified `<arg>` will be passed to the `main()` function in the C test bench.

  ```
  cosim.argv=arg1 arg2
  ```

- **cosim.compiled_library_dir:**

  Specifies the compiled library directory used during simulation with third-party simulators. The `<arg>` is the path name to the compiled library directory. The library must be compiled ahead of time using the `compile_simlib` command as explained in the *Vivado Design Suite Tcl Command Reference Guide* (UG835).

  ```
  cosim.compiled_library_dir=../../simLib
  ```

- **cosim.coverage:**

  Enables the coverage feature during simulation with the VCS simulator.

  ```
  cosim.coverage=true
  ```

- **cosim.disable_binary_tv:**

Disables the binary test vector format in co-simulation.

```
cosim.disable_binary_tv=true
```

- **cosim.disable_deadlock_detection:**

  Disables deadlock detection, and opening the Cosim Deadlock Viewer in co-simulation.

  ```
  cosim.disable_deadlock_detection=true
  ```

- **cosim.disable_dependency_check:**

  Disables dependency checks when running co-simulation.

  ```
  cosim.disable_dependency_check=true
  ```

- **cosim.enable_dataflow_profiling:**

  This option enables the dataflow channel profiling to track channel sizes during co-simulation. You must enable this feature to capture dataflow data as described in the Dataflow viewer section of the *Vitis High-Level Synthesis User Guide* (UG1399).

  ```
  cosim.enable_dataflow_profiling=true
  ```

- **cosim.enable_fifo_sizing:**

  Enables automatic FIFO channel size tuning for dataflow profiling during co-simulation.

  ```
  cosim.enable_fifo_sizing=true
  ```

- **cosim.enable_tasks_with_m_axi:**

  Enables stable `m_axi` interfaces for use with `hls::task`.

  ```
  cosim.enable_tasks_with_m_axi=true
  ```

- **cosim.hwemu_trace_dir:**

  Specifies the location of test vectors generated during hardware emulation to be used as a test bench during co-simulation. The test vectors are generated by the `syn.rtl.cosim_trace_generation` command as described in RTL Configuration. This argument lets you specify the kernel and instance name of the Vitis kernel in the hardware emulation simulation results to locate the test vectors for the HLS component.

  ```
  cosim.hwemu_trace_dir=../../dct/dct_2
  ```

- **cosim.ldflags <arg>:**

Specifies the options passed to the linker for simulation. This option is typically used to pass include path information or library information for the C/C++ test bench.

```
cosim.ldflags=ldExample
```

- **cosim.mflags <arg>:**

  Provides for options to be passed to the compiler for C simulation. This is typically used to speed up compilation.

  ```
  cosim.mflags=mExample
  ```

- **cosim.random_stall:**

  Enable random stalling of top level interfaces during co-simulation.

  ```
  cosim.random_stall=true
  ```

- **cosim.rtl:**

  Specifies either Verilog or VHDL as the language to use for C/RTL co-simulation. The default is Verilog.

  ```
  cosim.rtl=vhdl
  ```

- **cosim.setup:**

  When this option is specified, the simulation binary will be created in the `cosim` directory of the current HLS component, but simulation will not be executed. Simulation can be launched later from the compiled executable. The default is false, and co-simulation is run after setup is complete.

  ```
  cosim.setup=true
  ```

- **cosim.stable_axilite_update:**

  Enable `s_axilite` to configure registers which are stable compared with the prior transaction.

  ```
  cosim.stable_axilite_update=true
  ```

- **cosim.tool:**

  Specify the HDL simulator to be used to co-simulate the RTL with the C testbench. The Vivado simulator (xsim) is the default, unless otherwise specified.

  - auto
  - vcs
  - modelsim

- riviera
- isim
- xsim
- ncsim
- xceilum

```
cosim.tool=modelsim
```

- **cosim.trace_level:**

  Determines the level of waveform trace data to save during C/RTL co-simulation.

  - `none` does not save trace data. This is the default.
  - `all` results in all port and signal waveforms being saved to the trace file.
  - `port` only saves waveform traces for the top-level ports.
  - `port_hier` save the trace information for all ports in the design hierarchy.

  ```
  cosim.trace_level=port
  ```

  The trace file is saved in the `sim/Verilog` or `sim/VHDL` folder of the component when the simulation executes, depending on the selection used with the `cosim.rtl` option.

- **cosim.user_stall:**

  Specifies the JSON stall file to be used during co-simulation. The stall file can be generated using the `cosim_stall` command.

  ```
  cosim.user_stall=../../stall.json
  ```

- **cosim.wave_debug:**

  Opens the Vivado simulator GUI to view waveforms and simulation results. Enables waveform viewing of all processes in the generated RTL, as in the dataflow and sequential processes. This option is only supported when using Vitis simulator for co-simulation by setting `cosim.tool=xsim`. See Viewing Simulation Waveforms for more information.

  ```
  cosim.wave_debug=true
  ```

Compile Options

The `syn.compile` commands specify the default behavior for compilation of the HLS component.

- **syn.compile.design_size_max_warning:** Specifies the design size that triggers a warning related to slow compilation or poor QoR.

```
syn.compile.design_size_maximum_warning=300000
```

- **syn.compile.enable_auto_rewind:**

  When true enables an alternative HLS implementation for pipelined loops which uses automatic loop rewind as described in the Rewinding Pipelined Loops for Performance section of the *Vitis High-Level Synthesis User Guide* (UG1399) for additional information.

```
syn.compile.enable_auto_rewind=1
```

- **syn.compile.ignore_long_run_time:**

  Disable long runtime warning.

```
syn.compile.ignore_long_run_time=1
```

- **syn.compile.name_max_length <arg>:**

  The `name_max_length` option will specify the maximum length of function names. If the length is over the threshold, the last part of the name will be truncated.

```
syn.compile.name_max_length=13
```

- **syn.compile.no_signed_zeros:**

  The `no_signed_zeros` option will ignore the signedness of floating point zero so that compiler can do aggressive optimizations on floating point operations.

```
syn.compile.no_signed_zeros=1
```

- **syn.compile.pipeline_flush_in_task arg:**

  Specifies that pipelines in `hls::tasks` will be flushing (`flp`) by default to reduce the probability of deadlocks in C/RTL Co-simulation. This option applies to pipelines that achieve an II=1 with the default option of `ii1`. However, you can also specify it as applying `always` to enable flushing pipelines in either `hls::tasks` or dataflow, or can be completely disabled using `never`.

  IMPORTANT! *Flushing pipelines (`flp`) are compatible with the rewind option specified in the PIPELINE pragma or directive when the `syn.compile.enable_auto_rewind` option is also specified.*

  - **`always`:** Always make pipelines flushable in `hls::tasks` or dataflow regardless of II.
  - **`never`:** Never make pipeline flushable unless specifically overridden by other directives or pragmas.

- `ii1`: Make pipelines that achieve II=1 flushable in `hls::tasks`. This is the default setting.

```
syn.compile.pipeline_flush_in_task=always
```

- **syn.compile.pipeline_loops arg:**

  Loops with a tripcount equal to or greater than the specified value will be pipelined automatically.

```
syn.compile.pipeline_loops=20
```

- **syn.compile.pipeline_style arg:**

  Set default pipeline style, this is a preference not a hard constraint. Refer to Flushing Pipelines and Pipeline Types in *Vitis High-Level Synthesis User Guide* (UG1399) for more information. The three styles are: stallable (`stp`), flushable (`flp`), and free-running (`frp`). The default is free-running.

```
syn.compile.pipeline_style=flp
```

- **syn.compile.pragma_strict_mode:**

  Enable errors instead of warnings for unrecognized and improper pragma syntax.

```
syn.compile.pragma_strict_mode=1
```

- **syn.compile.unsafe_math_optimizations:**

  The `unsafe_math_optimizations` option will ignore the signedness of floating point zero and enable associative floating point operations so that compiler can do aggressive optimizations on floating point operations.

```
syn.compile.unsafe_math_optimizations=1
```

> **IMPORTANT!** *Using this option might change the result of any floating point calculations and result in a mismatch in C/RTL co-simulation. Ensure your test bench is tolerant of differences and checks for a margin of difference, not exact values.*

Dataflow Configuration

Synthesis: Dataflow Options

The `syn.dataflow` commands configure the dataflow analysis for the whole design. These settings specify the default memory channel and FIFO depth used by `syn.directive.dataflow`.

- **syn.dataflow.default_channel:**

  By default a RAM memory configured in ping-pong fashion is used to buffer the data between functions or loops when dataflow pipelining is used. When streaming data is used (where the data is always read and written in consecutive order), a FIFO memory will be more efficient and can be selected as the default memory type.

  The available channels are `fifo` and `pingpong`. The default is `pingpong`.

  ```
  syn.dataflow.default_channel=fifo
  ```

  TIP: *Arrays must be set to streaming using the `set_directive_stream` command to perform FIFO accesses.*

- **syn.dataflow.disable_fifo_sizing_opt:**

  Disable FIFO sizing optimizations that increase resource usage; this can improve performance and reduce deadlocks.

  ```
  syn.dataflow.disable_fifo_sizing_opt=1
  ```

- **syn.dataflow.fifo_depth:**

  An integer value specifying the default depth of FIFOs. This option has no effect when `pingpong` memories are used. By default the FIFOs depth used in the channel will be set to the size of the largest producer or consumer (whichever is largest). In some cases this approach can be too conservative and result in FIFOs which are larger than needed. This option can be used to specify the depth when you know the FIFOs are larger than required.

  ```
  syn.dataflow.fifo_depth=6
  ```

  IMPORTANT! *Be careful when using this option as incorrect use can result in a design which fails to operate correctly*

- **syn.dataflow.override_user_fifo_depth:**

  Specify a depth for every `hls::stream`, overriding any user settings.

  ```
  syn.dataflow.override_user_fifo_depth=12
  ```

  This is useful for checking if a deadlock is due to insufficient FIFO depths in the design. By setting the override to a very large value (for example, the maximum depth printed by co-simulation at the end of simulation), if there is no deadlock, then you can use the FIFO depth profiling options of co-simulation and the GUI to find the minimum depth that ensures performance and avoids deadlocks.

- **syn.dataflow.scalar_fifo_depth:**

An integer value specifying the minimum depth of the scalar value propagation FIFOs as described in the Specifying Compiler-Created FIFO Depth section of the *Vitis High-Level Synthesis User Guide* (UG1399). These FIFOs are used to forward the value of scalar arguments of the dataflow regions to processes which have predecessors in the region itself. They do not affect functional correctness, but an insufficient automatically computed size can result in loss of performance and even deadlock.

```
syn.dataflow.scalar_fifo_depth=4
```

When this option is not specified, the minimum depth is the value of the `syn.dataflow.fifo_depth` option, or it is 2. As a rule of thumb, a good value is the average number of times the process forwarding the scalar value can start before the last process that reads it starts.

- **syn.dataflow.start_fifo_depth:**

  An integer value specifying the minimum depth of the start propagation FIFOs as described in Specifying Compiler-Created FIFO Depth of the *Vitis High-Level Synthesis User Guide* (UG1399). These FIFOs are used to forward the `ap_start` handshake signal to processes which have predecessors in the region. They do not affect functional correctness, but an insufficient automatically computed size can result in loss of performance.

  ```
  syn.dataflow.start_fifo_depth=5
  ```

  When this option is not specified, the minimum depth is the value of the `syn.dataflow.fifo_depth` option, or it is 2. As a rule of thumb, a good value is the expected average number of times a process should be allowed to start in advance compared to its successors.

- **syn.dataflow.strict_mode:**

  Set severity for dataflow canonical form messages. The available modes are: `error`, `warning`, `off`. The default is `warning`.

  ```
  syn.dataflow.strict_mode=error
  ```

- **syn.dataflow.strict_stable_sync:**

  Force synchronization of stable ports with `ap_done`. Refer to the Stable Arrays section of the *Vitis High-Level Synthesis User Guide* (UG1399) for more information.

  ```
  syn.dataflow.strict_stable_sync=1
  ```

- **syn.dataflow.task_level_fifo_depth:**

Default task-level FIFO depth (used for FIFOs automatically created to transfer scalars between processes). A FIFO is synchronized by `ap_ctrl_chain`. The write is the `ap_done` of the producer, the read is the `ap_ready` of the consumer. Like a PIPO in terms of synchronization, and like a FIFO in terms of access.

```
syn.dataflow.task_level_fifo_depth=7
```

Debug Options

The `syn.debug` commands enable debugging in the HLS component and specify where to write debugging output.

- **syn.debug.enable:**

  Enable debug file generation. When not enabled the HLS component can be optimized during compilation, however it will not support debug.

  ```
  syn.debug.enable=1
  ```

  TIP: *This option relates to the `v++ -c -g` option, and must be manually enabled in the HLS component when debugging is needed at the Application level.*

- **syn.debug.directory:**

  Specifies the location of to write the output of HLS debugging. When not specified the location is set to `hls/.debug`.

  ```
  syn.debug.directory=../../debug
  ```

Interface Configuration

The `syn.interface` commands configure the default interface settings applied to the HLS component. These settings can be overridden for specific top-level interface ports using `syn.directive.interface`.

- **syn.interface.clock_enable:**

  Add a clock-enable port (`ap_ce`) to the design. The clock enable prevents all clock operations when it is active-Low, and disables all sequential operations. The default is false.

  ```
  syn.interface.clock_enable=1
  ```

- **syn.interface.default_slave_interface:**

Specify the default slave interface. The two modes are `s_axilite` and `none`. The default is `s_axilite`.

```
syn.interface.default_slave_interface=none
```

- **syn.interface.m_axi_addr64:**

  Enable 64-bit addressing for all `m_axi` interfaces. This is enabled by default. When disabled, the `m_axi` interfaces uses 32-bit addressing.

  ```
  syn.interface.m_axi_addr64=1
  ```

- **syn.interface.m_axi_alignment_byte_size:**

  Specify default alignment byte size for all `m_axi` interfaces. The default when this option is not specified is 64 bytes for Vitis kernel flow, or 1 byte for Vivado IP flow as described in the Default of Vivado/Vitis Flows section of the *Vitis High-Level Synthesis User Guide* (UG1399).

  ```
  syn.interface.m_axi_alignment_byte_size=16
  ```

  TIP: *A value of 0 is not valid.*

- **syn.interface.m_axi_auto_id_channel:**

  Enable automatic assignment of channel IDs for `m_axi` interfaces. This is disabled by default. Refer to the AXI4 Master Interface section of the *Vitis High-Level Synthesis User Guide* (UG1399) for additional information.

  ```
  syn.interface.m_axi_auto_id_channel=1
  ```

- **syn.interface.m_axi_auto_max_ports:**

  Enable automatic creation of separate `m_axi` interface adapters for each argument or port on the interface. This is disabled by default, reducing the `m_axi` interfaces to minimum needed. Refer to the section M_AXI Bundles of the *Vitis High-Level Synthesis User Guide* (UG1399) or more information.

  ```
  syn.interface.m_axi_auto_max_ports=1
  ```

- **syn.interface.m_axi_buffer_impl:**

  Specify the implementation resource for all buffers internal to the `m_axi` adapters. The choices are `auto`, `lutram`, `bram`, `uram`. The default is `bram`.

  ```
  syn.interface.m_axi_buffer_impl=lutram
  ```

- **syn.interface.m_axi_cache_impl:**

Specify the implementation resource for cache added to the `m_axi` adapters. The choices are `auto`, `lutram`, `bram`, `uram`. The default is `auto`.

```
syn.interface.m_axi_cache_impl=lutram
```

- **syn.interface.m_axi_conservative_mode:**

  Configure all `m_axi` adapters to work in conservative mode, waiting to issue a write request until the associated write data is entirely available (typically, buffered into the adapter or already emitted). It uses a buffer inside the M_AXI adapter to store all the data for a burst (both in case of reading and writing). This feature is enabled by default, and might slightly increase write latency but can resolve deadlock due to concurrent requests (read or write) on the memory subsystem. Disable conservative mode by setting it to `false`.

  ```
  syn.interface.m_axi_conservative_mode=0
  ```

- **syn.interface.m_axi_flush_mode:**

  Configure all `m_axi` adapters to be flushable, writing or reading garbage data if a burst is interrupted due to pipeline blocking (missing data inputs when not in conservative mode or missing output space). This is disabled by default.

  ```
  syn.interface.m_axi_flush_mode=1
  ```

- **syn.interface.m_axi_latency:**

  Globally specify the expected latency of the `m_axi` interface, allowing the design to initiate a bus request a number of cycles (latency) before the read or write is expected. The default value is 64 for the Vitis Kernel flow, and 0 for the Vivado IP flow, as described in the section Defaults of Vivado and Vitis Flows in the *Vitis High-Level Synthesis User Guide* (UG1399).

  ```
  syn.interface.m_axi_latency=5
  ```

- **syn.interface.m_axi_max_bitwidth:**

  Specifies the maximum bitwidth for the `m_axi` interfaces data channel. The default is 1024 bits. The specified value must be a power-of-two, between 8 and 1024. This decreases throughput if the actual accesses are bigger than the required interface, as they will be split into a multi-cycle burst of accesses.

  ```
  syn.interface.m_axi_max_bitwidth=128
  ```

- **syn.interface.m_axi_max_read_burst_length:**

  Specifies a global maximum number of data values read during a burst transfer for all `m_axi` interfaces. The default is 16.

  ```
  syn.interface.m_axi_max_read_burst_length=12
  ```

- **syn.interface.m_axi_max_widen_bitwidth:**

  Enable automatic port width resizing to widen bursts for the `m_axi` interface, up to the chosen bitwidth. The specified value must be a power of 2 between 8 and 1024, and must align with the `-m_axi_alignment_size`. The default value is 512 for the Vitis Kernel flow, and 0 for the Vivado IP flow.

  ```
  syn.interface.m_axi_max_widen_bitwidth=64
  ```

- **syn.interface.m_axi_max_write_burst_length:**

  Specifies a global maximum number of data values written during a burst transfer for all `m_axi` interfaces. The default is 16.

  ```
  syn.interface.m_axi_max_write_burst_length=12
  ```

- **syn.interface.m_axi_min_bitwidth:**

  Specifies the minimum bitwidth for `m_axi` interfaces data channel. The default is 8 bits. The value must be a power of 2, between 8 and 1024. This does not necessarily increase throughput if the actual accesses are smaller than the required interface.

  ```
  syn.interface.m_axi_min_bitwidth=64
  ```

- **syn.interface.m_axi_num_read_outstanding:**

  Specifies how many read requests can be made to the `m_axi` interface without a response, before the design stalls. This implies internal storage in the design, and a FIFO of size:

  ```
  num_read_outstanding*max_read_burst_length*word_size
  ```

  The default value is 16.

  ```
  syn.interface.m_axi_num_read_outstanding=8
  ```

- **syn.interface.m_axi_num_write_outstanding:**

  Specifies how many write requests can be made to the `m_axi` interface without a response, before the design stalls. This implies internal storage in the design, and a FIFO of size:

  ```
  num_write_outstanding*max_write_burst_length*word_size
  ```

  The default value is 16.

  ```
  syn.interface.m_axi_num_write_outstanding=8
  ```

- **syn.interface.m_axi_offset:**

  Specify the default offset mechanism for all `m_axi` interfaces. The options are:

  - `off`: No offset port is generated.
  - `slave`: Generates an offset port and automatically maps it to an `s_axilite` interface. This is the default value.
  - `direct`: Generates a scalar input offset port for directly passing the address offset into the IP through the offset port.

```
syn.interface.m_axi_offset=slave
```

- **syn.interface.register_io:**

  Globally enables registers for all scalar inputs, outputs, or all scalar ports on the top function (arrays are always registered). The options are `off`, `scalar_in`, `scalar_out`, `scalar_all`. The default is `off`.

```
syn.interface.register_io=scalar_out
```

- **syn.interface.s_axilite_auto_restart_counter:**

  Enables the auto-restart behavior for kernels. Use 1 to enable the auto-restart feature, or 0 to disable it which is the default. When enabled, the tool establishes the auto-restart bit in the `ap_ctrl_chain` control protocol for the `s_axilite` interface. For more information refer to Auto-Restarting Mode in the *Vitis High-Level Synthesis User Guide* (UG1399).

```
syn.interface.s_axilite_auto_restart_counter=1
```

- **syn.interface.s_axilite_data64:**

  Enable 64 bit data width for `s_axilite` interface. Use 1 to enable 64 bit data width and 0 to disable it. The default is 32 bit data width.

```
syn.interface.s_axilite_data64=1
```

- **syn.interface.s_axilite_interrupt_mode:**

  Specify the interrupt mode for `s_axilite` interface to be Clear on Read (`cor`) or Toggle on Write (`tow`). Clear on Read interrupt can be completed in a single transaction, while `tow` requires two. `Tow` is the default interrupt mode.

```
syn.interface.s_axilite_interrupt_mode=cor
```

- **syn.interface.s_axilite_mailbox:** Enables the creation of a mailboxes for non-stream non-stable `s_axilite` arguments. The mailbox feature is used in the setting and management of auto-restart kernels as described in the Auto-Restarting Mode section in the *Vitis High-Level Synthesis User Guide* (UG1399).The available options are:
  - `in`: Enable mailbox for only input arguments
  - `out`: Enable mailbox for only output arguments

- `both`: Enable mailbox for input and output arguments
- `none`: No mailbox created. This is the default value.

```
syn.interface.s_axilite_mailbox=in
```

- **syn.interface.s_axilite_status_regs:**

  Enables exposure of ECC error bits in the `s_axilite` register file via two clear-on-read (COR) counters per block RAM or URAM with ECC enabled. Options are `ecc` to enable or `off` to disable the feature. The default is disabled.

```
syn.interface.s_axilite_status_regs=ecc
```

- **syn.interface.s_axilite_sw_reset:**

  Enable SW reset in `s_axilite` adapter.

```
syn.interface.s_axilite_sw_reset=1
```

Package Options

Output Options

The package.output options are used to specify the nature of the exported files generated from the synthesized RTL design. These options include the following:

- **package.output.file:** Specifies the output file path and name for the exported file. When not specified the Vitis Unified IDE will name the output file after the HLS component. An example command would be:

```
package.output.file=../kernel.xo
```

- **package.output.format:** Specifies the exported format of the output generated after synthesis. The supported values are:
  - `package.output.format=ip_catalog`: A format suitable for adding to the Vivado IP catalog.
  - `package.output.format=xo`: A format accepted by the `v++` compiler for linking in the Vitis application acceleration flow.
  - `package.output.format=syn_dcp`: Synthesized checkpoint file for Vivado Design Suite. If this option is used, RTL synthesis is automatically executed. Vivado implementation can be optionally added using `vivado.flow=impl`
  - `package.output.format=sysgen`: Generate an IP and .zip archive for use in System Generator.

  - `package.output.format=rtl`: Outputs the RTL files that are generated by C synthesis, and does not package the IP or XO used for downstream processes.

    TIP: *The RTL format is useful for development, analysis, and debug of the HLS component. However, to export files you must specify one of the other output formats.*

- **package.output.syn:** Enables the running of the Package step as part of the C synthesis step to generate the required export files during synthesis.

IP Options

The `package.ip` commands specify details of the IP being generated by the HLS component. These are settings to define the IP Vendor, Library, Name, and Version (VLNV) for example to identify it in the IP catalog.

- **package.ip.description:**

  Provides a description for the catalog entry for the generated IP, used when packaging the IP.

  ```
  package.ip.description=details of IP
  ```

- **package.ip.display_name:**

  Provides a display name for the catalog entry for the generated IP, used when packaging the IP.

  ```
  package.ip.display_name=randy1
  ```

- **package.ip.library:**

  Provides the library component of the `<Vendor>:<Library>:<Name>:<Version>` (VLNV) identifier for generated IP.

  ```
  package.ip.library=testLib
  ```

- **package.ip.name:**

  Provides the name component of the `<Vendor>:<Library>:<Name>:<Version>` (VLNV) identifier for the generated IP.

  ```
  package.ip.name=randy1
  ```

- **package.ip.taxonomy:**

  Specifies the taxonomy for the catalog entry for the generated IP, used when packaging the IP. Taxonomy is a category to help identify the purpose or application of the IP.

  ```
  package.ip.taxonomy=video
  ```

- **package.ip.vendor:**

Provides the vendor component of the `<Vendor>:<Library>:<Name>:<Version>` (VLNV) identifier for generated IP.

```
package.ip.vendor=randyCom
```

- **package.ip.version:**

  Provides the version component of the `<Vendor>:<Library>:<Name>:<Version>` (VLNV) identifier for generated IP.

  ```
  package.ip.version=2.3
  ```

- **package.ip.xdc_file:**

  Specify an XDC file whose contents will be included in the packaged IP for use during implementation in the Vivado tool.

  ```
  package.ip.xdc_file=../../ip.xdc
  ```

- **package.ip.xdc_ooc_file:**

  Specify an out-of-context (OOC) XDC file whose contents will be included in packaged IP and used during out-of-context Vivado synthesis for the exported IP.

  ```
  package.ip.xdc_ooc_file=../../ooc,xdc
  ```

Operator Configuration

The `syn.op` command lets you configure the default implementation style, latency, and precision for different operators used for the HLS component. You can add multiple `syn.op` commands to a config file to specify the details of different operators. If an operator is not specified then the tool determines the default values for the component.

You can override the default settings specified by the `syn.op` command by using the syn.directive.bind_op command for specific variables.

- **syn.op:**

  The syntax for the `syn.op` command is as follows:

  ```
  syn.op=op:mul impl:dsp
  syn.op=op:add impl:fabric latency:6
  syn.op=op:fmacc precision:high
  syn.op=op:hdiv latency:5
  ```

  - `syn.op=`: Starts the command

- `op:<operator>`: Specifies the `op` keyword followed by the operator being defined. The supported operators include:
  - `mul`: integer multiplication operation
  - `add`: integer add operation
  - `sub`: integer subtraction operation
  - `fadd`: single precision floating-point add operation
  - `fsub`: single precision floating-point subtraction operation
  - `fdiv`: single precision floating-point divide operation
  - `fexp`: single precision floating-point exponential operation
  - `flog`: single precision floating-point logarithmic operation
  - `fmul`: single precision floating-point multiplication operation
  - `frsqrt`: single precision floating-point reciprocal square root operation
  - `frecip`: single precision floating-point reciprocal operation
  - `fsqrt`: single precision floating-point square root operation
  - `dadd`: double precision floating-point add operation
  - `dsub`: double precision floating-point subtraction operation
  - `ddiv`: double precision floating-point divide operation
  - `dexp`: double precision floating-point exponential operation
  - `dlog`: double precision floating-point logarithmic operation
  - `dmul`: double precision floating-point multiplication operation
  - `drsqrt`: double precision floating-point reciprocal square root operation
  - `drecip`: double precision floating-point reciprocal operation
  - `dsqrt`: double precision floating-point square root operation
  - `hadd`: half precision floating-point add operation
  - `hsub`: half precision floating-point subtraction operation
  - `hdiv`: half precision floating-point divide operation
  - `hmul`: half precision floating-point multiplication operation
  - `hsqrt`: half precision floating-point square root operation

    - `facc`: single precision floating-point accumulate operation
    - `fmacc`: single precision floating-point multiply-accumulate operation
    - `fmadd`: single precision floating-point multiply-add operation

    > **TIP:** *Comparison operators, such as `dcmp`, are implemented in LUTs and cannot be implemented outside of the fabric, or mapped to DSPs, and so are not configurable with the `syn.op` or `syn.directive.bind_op` commands.*

- `impl:<value>`: Specifies the implementation (`impl`) keyword followed by the value for the specified operator. When `impl` is not specified, the default is for the tool to determine the best implementation for a given operator. Supported values include:
    - `all`: All implementations. This is the default setting.
    - `dsp`: Use DSP resources
    - `fabric`: Use non-DSP resources
    - `meddsp`: Floating Point IP Medium Usage of DSP resources
    - `fulldsp`: Floating Point IP Full Usage of DSP resources
    - `maxdsp`: Floating Point IP Max Usage of DSP resources
    - `primitivedsp`: Floating Point IP Primitive Usage of DSP resources
    - `auto`: enable inference of combined `facc` | `fmacc` | `fmadd` operators
    - `none`: disable inference of combined `facc` | `fmacc` | `fmadd` operators
- `latency:<value>`: Specifies the `latency` keyword followed by the value. Defines the default latency for the binding of the operator to the implementation resource. The valid value range varies for each implementation (`impl`) of the operation. The default is -1, which lets the tool apply the standard latency for the implementation resource.

    > **TIP:** *You can specify the latency for a specific operation without specifying the implementation. This leaves the tool free to choose the implementation while managing the latency.*

- `precision:<value>`: Used for floating point operators (`facc`, `fmacc`, and `fmadd`), this specifies the `precision` keyword followed by one of the following:
    - `low`: Use a low precision (60 bit and 100 bit integer) accumulation implementation when available. This option is only available on certain non-AMD Versal™ devices, and can cause RTL/Co-Sim mismatches due to insufficient precision with respect to C++ simulation. However, it can always be pipelined with an II=1 without source code changes, though it uses approximately 3X the resources of `standard` precision floating point accumulation.

  - `high`: Use high precision (one extra bit) fused multiply-add implementation when available. This option is useful for high-precision applications and is very efficient on Versal devices, although it can cause RTL/Co-Sim mismatches due to the extra precision with respect to C++ simulation. It uses more resources than `standard` precision floating point accumulation.
  - `standard`: standard precision floating point accumulation and multiply-add is suitable for most uses of floating-point, and is the default setting. It always uses a true floating-point accumulator that can be pipelined with II=1 on Versal devices, and II that is typically between 3 and 5 (depending on clock frequency and target device) on non-Versal devices.

RTL Configuration

The `syn.rtl` commands configure various attributes of the compiled RTL, the type of reset used, and the encoding of the state machines. It also allows you to use specific identification in the RTL. By default, these options are applied to the top-level design and all RTL blocks within the design.

- **syn.rtl.cosim_trace_generation:**

  Generate test vectors during hardware emulation in the Vitis tool flow when the kernel is synthesized as a Vitis kernel, to be used as a test bench for C/RTL Co-simulation in future design iterations.

  ```
  syn.rtl.cosim_trace_generation=1
  ```

- **syn.rtl.deadlock_detection:**

  Enables simulation or synthesis deadlock detection in the top-level RTL of an exported IP/XO file. The options are as follows:

  - none: Deadlock detection disabled
  - sim: Enables deadlock detection only for simulation/emulation. This is the default setting.
  - hw: Deadlock detection enabled in synthesized RTL IP. Adds `ap_local_deadlock` and `ap_local_block` signals to the IP to enable local and global deadlock detection.

  ```
  syn.rtl.deadlock_detection=hw
  ```

- **syn.rtl.deadlock_diagnosis:**

  Enable deadlock detection diagnosis for Vitis kernels (.xo) during hardware emulation of the Application.

  ```
  syn.rtl.deadlock_diagnosis=1
  ```

- **syn.rtl.fsm_encoding:**

  Specify the 'fsm_encoding' RTL attribute value to guide RTL synthesis. The options are as follows:

  - auto: Allow the RTL Synthesis tool to determine the best state machine encoding
  - gray: Use gray state machine encoding
  - johnson: Use johnson state machine encoding
  - one_hot: Use one_hot state machine encoding
  - sequential: Use sequential state machine encoding
  - none: Disable state machine encoding, the state machine is synthesized exactly using the state code specified in the RTL (which is one_hot)

  ```
  syn.rtl.fsm_encoding=gray
  ```

- **syn.rtl.fsm_safe_state:**

  Specify the 'fsm_safe_state' RTL attribute value to guide RTL synthesis. This attribute can affect the quality of synthesis results, typically resulting in less performance with greater area. The options are as follows:

  - auto_safe_state: Implies Hamming-3 encoding.
  - default_state: Return the state machine to the default state.
  - power_on_state: Return the state machine to the POWER_ON state.
  - reset_state: Return the state machine to the RESET state.
  - none: Attribute not added to RTL, the state machine will not include safe state logic.

  ```
  syn.rtl.fsm_safe_state=auto_safe_state
  ```

- **syn.rtl.header:**

  Specify a file whose contents will be inserted at the beginning of all generated RTL files. This allows you to ensure that the generated RTL files contain user specified content.

  ```
  syn.rtl.header=../../myHeader.txt
  ```

- **syn.rtl.kernel_profile:**

  Add top level event and stall ports required by kernel profiling.

  ```
  syn.rtl.kernel_profile=1
  ```

> **IMPORTANT!** *This option relates to the `v++ -c --profile.stall` command, and must be manually added to the HLS component to ensure the stall profiling is available for use in the linked Application.*

- **syn.rtl.module_auto_prefix:**

  Specifies the top level function name as the prefix value for generated RTL modules. This option is ignored if `syn.rtl.module_prefix` is also specified. This is enabled by default.

  ```
  syn.rtl.module_auto_prefix=1
  ```

- **syn.rtl.module_prefix:**

  Specify a prefix to be used for all generated RTL module names. Use this to override the defaul module prefix of the top-level function.

  ```
  syn.rtl.module_prefix=newTop
  ```

- **syn.rtl.mult_keep_attribute:**

  Enable keep attribute.

  ```
  syn.rtl.mult_keep_attribute=1
  ```

- **syn.rtl.register_all_io:**

  Use a register by default for all I/O signals.

  ```
  syn.rtl.register_all_io=1
  ```

- **syn.rtl.register_reset_num:**

  Number of registers to add to reset signal.

  ```
  syn.rtl.register_reset_num=2
  ```

- **syn.rtl.reset:** Variables initialized in the C/C++ code are always initialized to the same value in the RTL and therefore in the bitstream. This initialization is performed only at power-on. It is not repeated when a reset is applied to the design.

  The setting applied with the `-reset` option determines how registers and memories are reset.

  - `none`: No reset is added to the design.
  - `control`: Resets control registers, such as those used in state machines and those used to generate I/O protocol signals. This is the default setting.
  - `state`: Resets control registers and registers or memories derived from static or global variables in the C/C++ code. Any static or global variable initialized in the C/C++ code is reset to its initialized value.

  - `all`: Resets all registers and memories in the design. Any static or global variable initialized in the C/C++ code is reset to its initialized value.

```
syn.rtl.reset=state
```

- **syn.rtl.reset_async:**

  Causes all registers to use an asynchronous reset. If this option is not specified a synchronous reset is used.

```
syn.rtl.reset_async=1
```

- **syn.rtl.reset_level:**

  Defines the polarity of the reset signal to be either active-Low or active-High. The default setting is active-High.

```
syn.rtl.reset_level=low
```



TIP: *The AXI protocol requires an active-Low reset. If your design uses AXI interfaces the tool will define this reset level with a warning if the `syn.rtl.reset_level` is active-High.*

Schedule Setting

The `syn.schedule` command configures the default type of scheduling performed.

- **syn.schedule.enable_dsp_full_reg:**

  Specifies that the DSP signals should be fully registered. This is enabled by default.

```
syn.schedule.enable_dsp_full_reg=0
```

Storage Configuration

Sets the global default options for the HLS micro-architecture binding of FIFO storage elements to memory resources.

The default configuration defined by `syn.storage` for FIFO storage can be overridden by syn.directive.bind_storage for a specific design element, or specifying the `storage_type` option for syn.directive.interface for objects on the interface.

- **syn.storage:**

The syntax for the `syn.storage` command is as follows:

```
syn.storage=fifo impl=auto auto_srl_max_bits=512 auto_srl_max_depth=3
```

- `syn.storage=fifo`: Starts the command to configure FIFOs.

  Note: FIFO is the only type supported at this time.

- `impl=<value>`: Specifies the implementation (`impl`) keyword followed by the value. When `impl` is not specified, the default is `auto`, allowing the tool to determine the best implementation for a given operator. Supported values include: `auto`, `bram`, `lutram`, `uram`, `memory`, `srl`
- `auto_srl_max_bits=<value>`: Only valid when for `impl:auto` (the default). Specifies the maximum allowed SRL total bits (depth * width) for auto implementations. The default is 1024.
- `auto_srl_max_depth=<value>`: Specifies the maximum allowed SRL depth for auto-srl implementation. The default is 2.

Implementation Configuration

The `syn.vivado` commands configure the Vivado synthesis and implementation runs used to derive resource utilization and timing estimates. These settings generally do not affect the RTL created for the HLS component, but can affect the example hardware design used for estimates.

- **vivado.clock:**

  Override the HLS clock constraint used in the Vivado out-of-context run for determination of timing analysis. This does not affect the output IP or XO files.

  ```
  vivado.clock=37
  ```

- **vivado.flow:**

  Run the Vivado flow for synthesis only (`syn`), or for implementation (`impl`). Running synthesis will be faster than running implementation, but will lack some of details of the implementation run. The default setting is `impl`.

  ```
  vivado.flow=syn
  ```

  TIP: *This is different from the `flow_target` option that defines the target as being either the Vitis kernel flow or the Vivado IP flow.*

- **vivado.impl_strategy:**

Specify a Vivado synthesis or implementation strategy. Strategy names can be found in Defining Implementation Strategies in the *Vivado Design Suite User Guide: Implementation* (UG904). This is used for resource usage and timing analysis and will not affect the output files.

```
vivado.impl_strategy=Performance_Explore
```

- **vivado.max_timing_paths:**

  Specify the max number of timing paths to report when the timing is not met in the Vivado synthesis or implementation.

  ```
  vivado.max_timing_paths=12
  ```

- **vivado.optimization_level:**

  Vivado optimization level, sets other `vivado_*` options. This will not apply to IP/XO output. The choices are 0, 1, 2, 3. This only applies for report generation and will not apply to the exported IP. The default setting is 0.

  ```
  vivado.optimization_level=2
  ```

- **vivado.pblock:**

  Specify a Pblock range to use during implementation for reporting purposes.

  ```
  vivado.pblock={SLICE_X8Y105:SLICE_X23Y149}
  ```

- **vivado.phys_opt:**

  Run Vivado physical optimization at the specified implementation stage. The choices are no optimization (`none`), placement optimization (`place`), routing optimization (`route`), or both placement and routing (`all`). This option only applies to the implementation run used to generate resource and timing estimates.

  ```
  vivado.phys_opt=route
  ```

- **vivado.report_level:**

  Specify the report level for Vivado synthesis and implementation run. The valid values and the associated reports are:

  - 0: Post-synthesis utilization. Post-implementation utilization and timing.
  - 1: Post-synthesis utilization, timing, and analysis. Post-implementation utilization, timing, and analysis.

  - 2: Post-synthesis utilization, timing, analysis, and failfast. Post-implementation utilization, timing, and failfast. This is the default setting.

```
vivado.report_level=1
```

- **vivado.rtl:**

  Specifies RTL language (`verilog`/`vhdl`) to select in Vivado flow.

```
vivado.rtl=vhdl
```

- **vivado.synth_design_args:**

  Specifies arguments for the Vivado `synth_design` command. Available `synth_design` arguments can be found in the *Vivado Design Suite Tcl Command Reference Guide* (UG835).

```
vivado.synth_design_args=arg1 arg2
```

- **vivado.synth_strategy:**

  Specifies a Vivado synthesis strategy to use when generating the example design to use for resource and timing estimates. Specific synthesis strategies can be found in *Vivado Design Suite User Guide: Synthesis* (UG901).

```
vivado.synth_strategy=Synthesis_Defaults
```

Unroll Setting

The `syn.unroll` setting provides a global tripcount threshold below which loops are automatically unrolled. Loops with a greater tripcount can be unrolled using `syn.directive.unroll`.

- **syn.unroll.tripcount_threshold:**

  All loops which have fewer iterations than the specified value are automatically unrolled. The default value is 0, which means that loops are not automatically unrolled.

```
syn.unroll.tripcount_threshold=6
```

HLS Optimization Directives

IMPORTANT! *The following options must appear in the HLS configuration file under the `[hls]` header.*

Directives, or the `syn.directive.xxx` commands allow you to customize the synthesis results for the same source code across multiple implementations. Change the directives to change the results. The `syn.directive.xxx` commands are intended for use in the config files associated with the new Vitis IDE as described in Building and Running an HLS Component in the *Vitis High-Level Synthesis User Guide* (UG1399).

Note: You can also use pragmas in your source code, rather than directives in your config file. This will have the same result, but also have the added advantage of being stored directly in your source code. Refer to the HLS Pragmas section of the *Vitis High-Level Synthesis User Guide* (UG1399) for more information.

Directives applied through a configuration file must include a `<location>` as an argument to the directive. The `<location>` defines what element of the source code the directive applies to, such as function, loop, region, or variable.

The example syntax for `syn.directive.xxx` commands include the location and the arguments for the directive. For example:

```
syn.directive.pipeline=dct2d II=4
```

Where `dct2d` is the location (function name) to apply the PIPELINE directive to, and II=4 is one of the possible arguments to the directive.

syn.directive.aggregate

Description

This directive collects the data fields of a struct into a single wide scalar. Any arrays declared within the struct and Vitis HLS performs a similar operation as `syn.directive.array_reshape`, and completely partitions and reshapes the array into a wide scalar and packs it with other elements of the struct.



TIP: *Arrays of structs are restructured as arrays of aggregated elements. The optimization does not support structs that contain other structs.*

The bit alignment of the resulting new wide-word can be inferred from the declaration order of the struct elements. The first element takes the least significant sector of the word and so forth until all fields are mapped.

Syntax

```
syn.directive.aggregate=[OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the variable which will be packed.
- `<variable>` is the struct variable to be packed.

Options

- `compact=[bit | byte | none | auto]`: Specifies the alignment of the aggregated struct. Alignment can be on the bit-level (packed), the byte-level (padded), none, or automatically determined by the tool which is the default behavior.

Examples

The following example aggregates the variable `AB` which is a struct pointer containing three 8-bit fields (`typedef struct {unsigned char R, G, B;}pixel`) in function `func`, into a new 24-bit pointer, aligning data at the bit-level.

```
syn.directive.aggregate=func AB compact=bit
```

See Also

- syn.directive.array_reshape
- syn.directive.disaggregate

syn.directive.alias

Description

Specify that two or more `M_AXI` pointer arguments point to the same underlying buffer in memory (DDR or HBM) and indicate any aliasing between the pointers by setting the distance or offset between them.

IMPORTANT! *`syn.directive.alias` applies to top-level function arguments mapped to `M_AXI` interfaces.*

Vitis HLS considers different pointers to be independent channels and generally does not provide any dependency analysis. However, in cases where the host allocates a single buffer for multiple pointers, this relationship can be communicated through `syn.directive.alias` and dependency analysis can be maintained. This enables data dependence analysis in Vitis HLS by defining the distance between pointers in the buffer.

Requirements for `syn.directive.alias`:

- All ports must be assigned to `M_AXI` interfaces and also assigned to different bundles, as shown in the example below
- Each port can only be used in one `syn.directive.alias` command
- All ports assigned to `syn.directive.alias` must have the same depth
- When offset is specified, each port can have one offset
- The offset for the `M_AXI` port must be specified as `slave` or `direct`, `offset=off` is not supported

Syntax

```
syn.directive.alias=[OPTIONS] <location> <ports>
```

- `<location>` is the location string in the format `function[/label]` that the ALIAS pragma applies to.
- `<ports>` specifies the ports to alias.

Options

- **`distance=<integer>`**: Specifies the difference between the pointer values passed to the ports in the list.
- **`offset=<string>`**: Specifies the offset of the pointer passed to each port in the `ports` list with respect to the origin of the array.

Example

For the following function `top`:

```
void top(int *arr0, int *arr1, int *arr2, int *arr3, ...) {
  #pragma HLS interface M_AXI port=arr0 bundle=hbm0 depth=0x40000000
  #pragma HLS interface M_AXI port=arr1 bundle=hbm1 depth=0x40000000
  #pragma HLS interface M_AXI port=arr2 bundle=hbm2 depth=0x40000000
  #pragma HLS interface M_AXI port=arr3 bundle=hbm3 depth=0x40000000
```

The following command defines aliasing for the specified array pointers, and defines the `distance` between them:

```
syn.directive.alias=top arr0,arr1,arr2,arr3 distance=10000000
```

Alternatively, the following command specifies the `offset` between pointers, to accomplish the same effect:

```
syn.directive.alias=top arr0,arr1,arr2,arr3
offset=00000000,10000000,20000000,30000000
```

See Also

- syn.directive.interface

syn.directive.allocation

Description

Specifies instance restrictions for resource allocation.

`syn.directive.allocation` can limit the number of RTL instances and hardware resources used to implement specific functions, loops, or operations. For example, if the C/C++ source has four instances of a function `foo_sub`, the `syn.directive.allocation` command can ensure that there is only one instance of `foo_sub` in the final RTL. All four instances are implemented using the same RTL block. This reduces resources used by the function, but negatively impacts performance by sharing those resources.

The operations in the C/C++ code, such as additions, multiplications, array reads, and writes, can also be limited by the `syn.directive.allocation` command.

Syntax

```
syn.directive.allocation=[OPTIONS] <location> <instances>
```

- `<location>` is the location string in the format `function[/label]`.
- `<instances>` is a function or operator. The function can be any function in the original C/C++ code that has not been either inlined by the `syn.directive.allocation` command or inlined automatically by Vitis HLS.

Options

- **`limit=<integer>`:**

  Sets a maximum limit on the number of instances (of the type defined by the `type` option) to be used in the RTL design.

- **`type=[function|operation]`:** The instance type can be `function` (default) or `operation`. For a complete list of supported operations refer to Operator Configuration.

Examples

Given a design `foo_top` with multiple instances of function `foo`, limits the number of instances of `foo` in the RTL to 2.

```
syn.directive.allocation=limit=2 type=function foo_top foo
```

Limits the number of multipliers used in the implementation of `My_func` to 1. This limit does not apply to any multipliers that might reside in sub-functions of `My_func`. To limit the multipliers used in the implementation of any sub-functions, specify an allocation directive on the sub-functions or inline the sub-function into function `My_func`.

```
syn.directive.allocation=limit=1 type=operation My_func mul
```

See Also

- syn.directive.inline

syn.directive.array_partition

Description

> **IMPORTANT!** *`syn.directive.array_partition` and `syn.directive.array_reshape` are not supported for `M_AXI` Interfaces on the top-level function. Instead you can use the `hls::vector` data types as described in the Vector Data Types section of the Vitis High-Level Synthesis User Guide (UG1399).*

`syn.directive.array_partition` partitions an array into smaller arrays or individual elements.

This partitioning:

- Results in RTL with multiple small memories or multiple registers instead of one large memory
- Effectively increases the number of read and write ports for the storage
- Potentially improves the throughput of the design
- Requires more memory instances or registers

Syntax

```
syn.directive.array_partition=[OPTIONS] <location> <array>
```

- `<location>` is the location (in the format `function[/label]`) which contains the array variable.
- `<array>` is the array variable to be partitioned.

Options

- **`dim=<integer>`:** Specifies which dimension of the array is to be partitioned.
  - The dimension is relevant for multi-dimensional arrays only.
  - If a value of 0 is used, all dimensions are partitioned with the specified options.
  - Any other value partitions only that dimension. For example, if a value 1 is used, only the first dimension is partitioned.
- **`type=(block|cyclic|complete)`:**
  - `block` partitioning creates smaller arrays from consecutive blocks of the original array. This effectively splits the array into *N* equal blocks where *N* is the integer defined by the `-factor` option.
  - `cyclic` partitioning creates smaller arrays by interleaving elements from the original array. For example, if `-factor 3` is used:
    - Element 0 is assigned to the first new array.

    - Element 1 is assigned to the second new array.
    - Element 2 is assigned to the third new array.
    - Element 3 is assigned to the first new array again.
  - `complete` partitioning decomposes the array into individual elements. For a one-dimensional array, this corresponds to resolving a memory into individual registers. For multi-dimensional arrays, specify the partitioning of each dimension, or use `-dim 0` to partition all dimensions.

  The default is `complete`.

- **`factor=<integer>`:** Used for `block` or `cyclic` partitioning only, this option specifies the number of smaller arrays that are to be created.
- **`off=true`:** Disables the ARRAY_PARTITION feature for the specified variable. Does not work with `dim`, `factor`, or `type`.

Example 1

Partitions array AB[13] in function `func` into four arrays. Because four is not an integer factor of 13:

- Three arrays have three elements.
- One array has four elements (AB[9:12]).

```
syn.directive.array_partition=func AB type=block factor=4
```

Partitions array AB[6][4] in function `func` into two arrays, each of dimension [6][2].

```
syn.directive.array_partition=func AB type=block factor=2 dim=2
```

Partitions all dimensions of AB[4][10][6] in function `func` into individual elements.

```
syn.directive.array_partition=func AB type=complete dim=0
```

Example 2

Partitioned arrays can be addressed in your code by the new structure of the partitioned array, as shown in the following code example. When using the following directive:

```
syn.directive.array_partition=top b type=complete dim=1
```

The code can be structured as follows:

```
struct SS
{
  int x[N];
  int y[N];
};
```

```
int top(SS *a, int b[4][6], SS &c) {...}

syn.directive.interface mode=ap_memory top b[0]
syn.directive.interface mode=ap_memory top b[1]
syn.directive.interface mode=ap_memory top b[2]
syn.directive.interface mode=ap_memory top b[3]
```

See Also

- syn.directive.array_reshape

syn.directive.array_reshape

Description

> **IMPORTANT!** *`syn.directive.array_partition` and `syn.directive.array_reshape` are not supported for `M_AXI` Interfaces on the top-level function. Instead you can use the `hls::vector` data types as described in Vector Data Types in Vitis High-Level Synthesis User Guide (UG1399).*

`syn.directive.array_reshape` combines array partitioning with vertical array mapping to create a single new array with fewer elements but wider words.

The `syn.directive.array_reshape` command has the following features:

- Splits the array into multiple arrays (like `syn.directive.array_partition`).
- Automatically recombine the arrays vertically to create a new array with wider words.

Syntax

```
syn.directive.array_reshape=[OPTIONS] <location> <array>
```

- `<location>` is the location (in the format `function[/label]`) that contains the array variable.
- `<array>` is the array variable to be reshaped.

Options

- **`dim=<integer>`:** Specifies which dimension of the array is to be partitioned.
  - The dimension is relevant for multidimensional arrays only.
  - If a value of 0 is used, all dimensions are partitioned with the specified options.
  - Any other value partitions only that dimension. For example, if a value 1 is used, only the first dimension is partitioned.
- **`type=(block|cyclic|complete)`:**

  - `block` reshaping creates smaller arrays from consecutive blocks of the original array. This effectively splits the array into *N* equal blocks where *N* is the integer defined by the `-factor` option and then combines the *N* blocks into a single array with `word-width*N`. The default is `complete`.
  - `cyclic` reshaping creates smaller arrays by interleaving elements from the original array. For example, if `-factor 3` is used, element 0 is assigned to the first new array, element 1 to the second new array, element 2 is assigned to the third new array, and then element 3 is assigned to the first new array again. The final array is a vertical concatenation (word concatenation, to create longer words) of the new arrays into a single array.
  - `complete` reshaping decomposes the array into temporary individual elements and then recombines them into an array with a wider word. For a one-dimension array this is equivalent to creating a very-wide register (if the original array was *N* elements of *M* bits, the result is a register with `N*M` bits). This is the default.
- **`factor=<integer>`:** Used for `block` or `cyclic` partitioning only, this option specifies the number of smaller arrays that are to be created.
- **`object`:**

  ***Note*:** Relevant for container arrays only.

  Applies reshape on the objects within the container. If the option is specified, all dimensions of the objects will be reshaped, but all dimensions of the container will be kept.
- **`off=true`:** Disables the ARRAY_RESHAPE feature for the specified variable.

Example 1

Reshapes 8-bit array AB[17] in function `func` into a new 32-bit array with five elements.

Because four is not an integer factor of 17:

- Index 17 of the array, AB[17], is in the lower eight bits of the reshaped fifth element.
- The upper eight bits of the fifth element are unused.

```
syn.directive.array_reshape=type=block factor=4 func AB
```

Partitions array AB[6][4] in function `func`, into a new array of dimension [6][2], in which dimension 2 is twice the width.

```
syn.directive.array_reshape=type=block factor=2 dim=2 func AB
```

Reshapes 8-bit array AB[4][2][2] in function `func` into a new single element array (a register), 4*2*2*8 (= 128)-bits wide.

```
syn.directive.array_reshape=type=complete dim=0 func AB
```

Example 2

Reshaped arrays can be addressed in your code by the new structure of the array, as shown in the following code example. When using the following directive:

```
syn.directive.array_reshape=top b type=complete dim=0
```

The code can be structured as follows:

```
struct SS
{
  int x[N];
  int y[N];
};

int top(SS *a, int b[4][6], SS &c) {...}
```

And the array interface defined as:

```
syn.directive.interface=mode=ap_memory top b[0]
```

See Also

- syn.directive.array_partition
- syn.directive.interface

syn.directive.bind_op

Description

TIP: *This pragma or directive does not play a part in DSP multi-operation matching (MULADD, AMA, ADDMUL and corresponding Subtraction operations). In fact the use of BIND_OP for assigning an operator to DSP can prevent the HLS compiler from matching multi-operation expressions.*

Vitis HLS implements the different operations in the code using specific hardware resources (or implementations). The tool automatically determines the resource to use, or you can define `syn.op` can be defined to generally specify the implementation as explained in Operator Configuration.

The `syn.directive.bind_op` command indicates that for the specified variable an operation (for example `mul`, `add`, or `sub`) should be mapped to a specific implementation (`impl`) in the RTL. For example, to indicate that a specific multiplier operation (`mul`) is implemented in the device fabric rather than a DSP, you can use the `syn.directive.bind_op` command.

You can also specify the latency with `syn.directive.bind_op` using the `latency` option as described below.

IMPORTANT! *To use the `latency` option, the operation must have an available multi-stage implementation. The HLS tool provides a multi-stage implementation for all basic arithmetic operations (add, subtract, multiply, and divide), and all floating-point operations.*

Syntax

```
syn.directive.bind_op=[OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the variable.
- `<variable>` is the variable to be assigned. The variable in this case is one that is assigned the result of the operation that is the target of this directive.

Options

- **`op=<value>`:** Defines the operation to bind to a specific implementation resource. Supported functional operations include: `mul`, `add`, `sub`

  Supported floating point operations include: `fadd`, `fsub`, `fdiv`, `fexp`, `flog`, `fmul`, `frsqrt`, `frecip`, `fsqrt`, `dadd`, `dsub`, `ddiv`, `dexp`, `dlog`, `dmul`, `drsqrt`, `drecip`, `dsqrt`, `hadd`, `hsub`, `hdiv`, `hmul`, and `hsqrt`

  TIP: *Floating-point operations include single precision (f), double-precision (d), and half-precision (h).*

- **`impl=<value>`:** Defines the implementation to use for the specified operation.Supported implementations for functional operations include `fabric` and `dsp`.Supported implementations for floating point operations include: `fabric`, `meddsp`, `fulldsp`, `maxdsp`, and `primitivedsp`.

  ***Note*:** `primitivedsp` is only available on Versal devices.

- **`latency=<int>`:** Defines the default latency for the implementation of the operation. The valid latency varies according to the specified `op` and `impl`. The default is -1, which lets Vitis HLS choose the latency.The tables below reflect the supported combinations of operation, implementation, and latency.

Table 31: **Supported Combinations of Functional Operations, Implementation, and Latency**

Operation	Implementation	Min Latency	Max Latency
add	fabric	0	4
add	dsp	0	4
mul	fabric	0	4
mul	dsp	0	4
sub	fabric	0	4
sub	dsp	0	0

TIP: *Comparison operators, such as* `dcmp`*, are implemented in LUTs and cannot be implemented outside of the fabric, or mapped to DSPs, and so are not configurable with the* `config_op` *or* `bind_op` *commands.*

Table 32: **Supported Combinations of Floating Point Operations, Implementation, and Latency**

Operation	Implementation	Min Latency	Max Latency
fadd	fabric	0	13
fadd	fulldsp	0	12
fadd	primitivedsp	0	3
fsub	fabric	0	13
fsub	fulldsp	0	12
fsub	primitivedsp	0	3
fdiv	fabric	0	29
fexp	fabric	0	24
fexp	meddsp	0	21
fexp	fulldsp	0	30
flog	fabric	0	24
flog	meddsp	0	23
flog	fulldsp	0	29
fmul	fabric	0	9
fmul	meddsp	0	9
fmul	fulldsp	0	9
fmul	maxdsp	0	7
fmul	primitivedsp	0	4
fsqrt	fabric	0	29
frsqrt	fabric	0	38
frsqrt	fulldsp	0	33
frecip	fabric	0	37
frecip	fulldsp	0	30
dadd	fabric	0	13
dadd	fulldsp	0	15
dsub	fabric	0	13
dsub	fulldsp	0	15
ddiv	fabric	0	58
dexp	fabric	0	40
dexp	meddsp	0	45
dexp	fulldsp	0	57
dlog	fabric	0	38
dlog	meddsp	0	49
dlog	fulldsp	0	65

Table 32: **Supported Combinations of Floating Point Operations, Implementation, and Latency** *(cont'd)*

Operation	Implementation	Min Latency	Max Latency
dmul	fabric	0	10
dmul	meddsp	0	13
dmul	fulldsp	0	13
dmul	maxdsp	0	14
dsqrt	fabric	0	58
drsqrt	fulldsp	0	111
drecip	fulldsp	0	36
hadd	fabric	0	9
hadd	meddsp	0	12
hadd	fulldsp	0	12
hsub	fabric	0	9
hsub	meddsp	0	12
hsub	fulldsp	0	12
hdiv	fabric	0	16
hmul	fabric	0	7
hmul	fulldsp	0	7
hmul	maxdsp	0	9
hsqrt	fabric	0	16

Examples

In the following example, a two-stage pipelined multiplier using fabric logic is specified to implement the multiplication for variable `c` of the function `foo`.

```
int foo (int a, int b) {
int c, d;
c = a*b;
d = a*c;
return d;
}
```

And the `syn.directive.bind_op` command is as follows:

```
syn.directive.bind_op=op=mul impl=fabric latency=2 foo c
```

TIP: *The HLS tool selects the implementation to use for variable `d` because no `syn.directive.bind_op` is specified for that variable.*

See Also

- syn.directive.bind_storage

syn.directive.bind_storage

Description

Vitis HLS assigns arrays in the code using specific memory resources (or types). The tool automatically determines the resource to use. Alternatively, you can define `syn.storage` to generally specify the memory type, as explained in Storage Configuration.

The `syn.directive.bind_storage` command assigns a specific variable in the code (an array, or function argument) to a specific memory type (`type`) in the RTL. For example, you can use the `syn.directive.bind_storage` command to specify which type of memory, and which implementation to use for an array variable. Also, this allows you to control whether the array is implemented as a single or a dual-port RAM.

IMPORTANT! *This feature is important for arrays on the top-level function interface, because the memory type associated with the array determines the number and type of ports needed in the RTL, as discussed in the Arrays on the Interface section of the Vitis High-Level Synthesis User Guide (UG1399). However, for variables assigned to top-level function arguments you must assign the memory type and implementation using the `storage_type` and `storage_impl` options of `syn.directive.interface`.*

You can also use the `latency` option of `syn.directive.bind_storage` to specify the latency of the implementation. For block RAMs on the interface, the `latency` option allows you to model off-chip, non-standard SRAMs at the interface, for example supporting an SRAM with a latency of 2 or 3. For internal operations, the `latency` option allows the operation to be implemented using more pipelined stages. These additional pipeline stages can help resolve timing issues during RTL synthesis.

IMPORTANT! *To use the `latency` option, the memory must have an available multi-stage implementation. The HLS tool provides a multi-stage implementation for all block RAMs.*

Syntax

```
syn.directive.bind_storage=[OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the variable.
- `<variable>` is the variable to be assigned.

TIP: *If the variable is an argument of a top-level function, then use the `storage_type` and `storage_impl` options of `syn.directive.interface`.*

Options

- **`type=<value>`:** Defines the type of memory to bind to the specified variable. Supported types include: `fifo`, `ram_1p`, `ram_1wnr`, `ram_2p`, `ram_s2p`, `ram_t2p`, `rom_1p`, `rom_2p`, and `rom_np`.

Table 33: **Storage Types**

Type	Description
FIFO	A FIFO. Vitis HLS determines how to implement this in the RTL, unless the `-impl` option is specified.
RAM_1P	A single-port RAM. Vitis HLS determines how to implement this in the RTL, unless the `-impl` option is specified.
RAM_1WNR	A RAM with 1 write port and N read ports, using N banks internally.
RAM_2P	A dual-port RAM that allows read operations on one port and both read and write operations on the other port.
RAM_S2P	A dual-port RAM that allows read operations on one port and write operations on the other port.
RAM_T2P	A true dual-port RAM with support for both read and write on both ports.
ROM_1P	A single-port ROM. Vitis HLS determines how to implement this in the RTL, unless the `-impl` option is specified.
ROM_2P	A dual-port ROM.
ROM_NP	A multi-port ROM.



TIP: *If you specify a single port RAM for a PIPO, then the binder will typically allocate a merged-PIPO, with only one bank, and reader and writer accessing different halves of the bank. The info message reported by the HLS compiler says "Implementing PIPO using a single memory for all blocks." This is often cheaper for small PIPOs, where all banks can share a single RAM block. However, if you specify a dual-port RAM for a PIPO to get higher bandwidth and the scheduler uses both ports either in the producer or in the consumer, then the binder will typically allocate a split-PIPO, where the producer will use 1 port and the consumer 2 ports of different banks. The info message reported by the HLS compiler in this case says "Implementing PIPO using a separate memory for each block."*

- **`impl=<value>`:** Defines the implementation for the specified memory type. Supported implementations include: `bram`, `bram_ecc`, `lutram`, `uram`, `uram_ecc`, `srl`, `memory`, and `auto` as described below.

Table 34: **Supported Implementation**

Name	Description
MEMORY	Generic memory for FIFO, lets the Vivado tool choose the implementation.
URAM	UltraRAM resource
URAM_ECC	UltraRAM with ECC
SRL	Shift Register Logic resource
LUTRAM	Distributed RAM resource
BRAM	Block RAM resource
BRAM_ECC	Block RAM with ECC
AUTO	Vitis HLS automatically determine the implementation of the variable.

Table 35: **Supported Implementations by FIFO/RAM/ROM**

Type	Command/Pragma	Scope	Supported Implementations
FIFO	`bind_storage`[syn.directive.bind_storage]	local	**AUTO**, BRAM, LUTRAM, URAM, MEMORY, SRL
FIFO	`config_storage`	global	**AUTO**, BRAM, LUTRAM, URAM, MEMORY, SRL
RAM* | ROM*	`bind_storage`	local	**AUTO** BRAM, BRAM_ECC, LUTRAM, URAM, URAM_ECC
RAM* | ROM*	`config_storage`[2]	global	N/A
RAM_1P	`set_directive_interface s_axilite -storage_impl`	local	**AUTO**, BRAM, URAM
	`config_interface -m_axi_buffer_impl`	global	AUTO, **BRAM**, LUTRAM, URAM

Notes:

1. When no implementation is specified the directive uses AUTOSRL behavior as a default. However, this value cannot be specified.
2. `config_storage` only supports FIFO types.

- **`latency=<int>`:** Defines the default latency for the binding of the storage type to the implementation. The valid latency varies according to the specified `type` and `impl`. The default is -1, which lets Vitis HLS choose the latency.

Table 36: **Supported Combinations of Memory Type, Implementation, and Latency**

Type	Implementation	Min Latency	Max Latency
FIFO	BRAM	1	4
FIFO	LUTRAM	1	4
FIFO	MEMORY	1	4
FIFO	SRL	1	4
FIFO	URAM	1	4
RAM_1P	AUTO	1	3
RAM_1P	BRAM	1	3
RAM_1P	LUTRAM	1	3
RAM_1P	URAM	1	3
RAM_1WNR	AUTO	1	3
RAM_1WNR	BRAM	1	3
RAM_1WNR	LUTRAM	1	3
RAM_1WNR	URAM	1	3
RAM_2P	AUTO	1	3
RAM_2P	BRAM	1	3
RAM_2P	LUTRAM	1	3
RAM_2P	URAM	1	3
RAM_S2P	BRAM	1	3

Table 36: **Supported Combinations of Memory Type, Implementation, and Latency** *(cont'd)*

Type	Implementation	Min Latency	Max Latency
RAM_S2P	BRAM_ECC	1	3
RAM_S2P	LUTRAM	1	3
RAM_S2P	URAM	1	3
RAM_S2P	URAM_ECC	1	3
RAM_T2P	BRAM	1	3
RAM_T2P	URAM	1	3
ROM_1P	AUTO	1	3
ROM_1P	BRAM	1	3
ROM_1P	LUTRAM	1	3
ROM_2P	AUTO	1	3
ROM_2P	BRAM	1	3
ROM_2P	LUTRAM	1	3
ROM_NP	BRAM	1	3
ROM_NP	LUTRAM	1	3

IMPORTANT! *Any combinations of memory type and implementation that are not listed in the prior table are not supported by* `syn.directive.bind_storage`.

Examples

In the following example, the `coeffs[128]` variable is an argument of the function `func1`. The directive specifies that `coeffs` uses a single port RAM implemented on a BRAM core from the library.

```
syn.directive.bind_storage=func1 coeffs type=RAM_1P impl=bram
```

TIP: *The ports created in the RTL to access the values of* `coeffs` *are defined in the RAM_1P core.*

See Also

- syn.directive.bind_op

syn.directive.cache

Description

In FPGA, non-burst accesses to the DDR memory are very expensive and can impact the overall performance of the design. Hence, it is important to devise a scheme that reduces the time needed to access the necessary information. An efficient solution is to re-write the code or use manual burst, but if that does not work then another solution might be to use a cache memory.

Cache provides a temporary storage area in the M_AXI adapter so the design can more quickly retrieve data. The effectiveness of the cache mechanism is measured in a hit/miss ratio and it is based on a property of computer programs called locality of reference, or the tendency of a program to access the same set of memory locations repetitively over a short period of time. It suggests that when a particular memory block is accessed, it is likely to be accessed again in the near future, along with memory located in close proximity. For instance, when executing consecutive instructions in memory, the next set of instructions to be accessed will most likely be within a contiguous block of data nearby.



TIP: *You control the cache implementation resources using the* `syn.interface.m_axi_cache_impl` *command.*

Syntax

```
syn.directive.cache=<location> port=<name> lines=<value> depth=<value>
```

Where:

- **`<location>`:** Specifies the function where the specified ports can be found. This is the top function.
- **`port=<name>`:** Specifies the port to add cache to. This is a required argument.
- **`lines=<value>`:** Indicates the number of cache lines. The number of lines can be specified as 1, which indicates a single cache line, or a value greater than 1 expressed as a power of 2, indicating multiple cache lines. This is an optional value, and defaults to 1 if not specified.
- **`depth=<value>`:** Specifies the size of each line in words. The depth must be specified as a power of 2, and indicates the size in words of the pointer datatype for each line. This is an optional value, and defaults to the max burst length if not specified. The max burst length defaults to 16, but can be globally specified by the `syn.interface.m_axi_max_read_burst_length` command as described in Interface Configuration, or by the `syn.directive.interface` command for specific interfaces.

Limitations

The CACHE directive or pragma has the following limitations:

- Cache is only supported for read-only port

- The cache is implemented as a Single port, Single way cache.
- A cache is associated with each read channel of a m_axi port. If there are no channel specifications for a bundle, then all read ports mapped to the bundle must:
  - Either not have a cache or all have a cache with the same line size in bytes and the same number of lines.
  - If the word sizes are different, then also the depths must be different, so that the line size in bytes is the same.
  - In the code below the depth (For example, number of words) of a line must be 4 times larger for the char* than for the int*, because one int contains 4 char:

```
void top(int *int_arr, char *char_arr, …) {
#pragma HLS interface m_axi port=int_arr bundle=gmem …
#pragma HLS interface m_axi port=char_arr bundle=gmem …
#pragma HLS cache port=int_arr lines=8 depth=8
#pragma HLS cache port=char_arr lines=8 depth=32
```

  - This code, on the other hand, would be illegal because the lines have different line sizes:

```
void top(int *int_arr, char *char_arr, …) {
#pragma HLS interface m_axi port=int_arr bundle=gmem …
#pragma HLS interface m_axi port=char_arr bundle=gmem …
#pragma HLS cache port=int_arr lines=8 depth=8
#pragma HLS cache port=char_arr lines=8 depth=16
```

  - This code would be illegal because the number of lines is different:

```
void top(int *int_arr, char *char_arr, …) {
#pragma HLS interface m_axi port=int_arr bundle=gmem …
#pragma HLS interface m_axi port=char_arr bundle=gmem …
#pragma HLS cache port=int_arr lines=8 depth=8
#pragma HLS cache port=char_arr lines=4 depth=32
```

  - Finally, this code would be illegal because only one array has the cache:

```
void top(int *int_arr, char *char_arr, …) {
#pragma HLS interface m_axi port=int_arr bundle=gmem …
#pragma HLS interface m_axi port=char_arr bundle=gmem …
#pragma HLS cache port=int_arr lines=8 depth=8
y
```

  - If channels are specified for the bundle, then cache sizes for each channel can be different, and some channels may have a cache and others may not. Within a channel the same rules as above apply. For example, this code would now be perfectly legal, but consume more resources than the previous example due to the existence of two caches:

```
void top(int *int_arr, char *char_arr, …) {
#pragma HLS interface m_axi port=int_arr bundle=gmem channel=0 …
#pragma HLS interface m_axi port=char_arr bundle=gmem channel=1 …
#pragma HLS cache port=int_arr lines=8 depth=32
#pragma HLS cache port=char_arr lines=8 depth=32
```

  - The cache will generate line-sized read requests. This means that: In cosimulation, the array size (in words of the given datatype) must be an integer multiple of the line size (also specified in words of the given datatype).

- In deployment, the array allocation in DRAM performed by the host code must be aligned to the line size, and the total size of the array must be a multiple of the line size.
- Otherwise, this error message is generated during cosim:

```
ERROR: Index ... out of bound 0 to ...
```

Example

The following example shows a design where overlapping access will cause the burst to fail. Using the CACHE pragma or directive will improve the performance of the design.

```
extern "C" {
  void dut(
    const double *in,  // Read-Only Vector 1
    double *out,       // Output Result
    int size                 // Size in integer
  )
#pragma HLS INTERFACE m_axi port=in  bundle=aximm depth = 1026
#pragma HLS INTERFACE m_axi port=out bundle=aximm depth = 1024
#pragma HLS cache port=in lines=8 depth=128
for(int i = 0; i < size; i++)
    {
      out[i] = in[i] + in[i + 1];
    }
}
```

See Also

- pragma HLS interface
- Interface Configuration
- pragma HLS cache

syn.directive.dataflow

Description

All operations are performed sequentially in a C/C++ description. In the absence of any directives that limit resources (such as `set_directive_allocation`), Vitis HLS seeks to minimize latency and improve concurrency. Data dependencies can limit this. For example, functions or loops that access arrays must finish all read/write accesses to the arrays before they complete. This prevents the next function or loop that consumes the data from starting operation.

However, it is possible for the operations in a function or loop to start operation before the previous function or loop completes all its operations. `syn.directive.dataflow` specifies that dataflow optimization be performed on the functions or loops, improving the concurrency of the RTL implementation. When `syn.directive.dataflow` is specified, the HLS tool analyzes the dataflow between sequential functions or loops and creates channels (based on ping-pong RAMs or FIFOs) that allow consumer functions or loops to start operation before the producer functions or loops have completed. This allows functions or loops to operate in parallel, which decreases latency and improves the throughput of the RTL.

TIP: *The `syn.dataflow.xxx` command specifies the default memory channel and FIFO depth used by `syn.directive.dataflow` as explained in Dataflow Configuration.*

If no initiation interval (number of cycles between the start of one function or loop and the next) is specified, Vitis HLS attempts to minimize the initiation interval and start operation as soon as data is available. For the DATAFLOW optimization to work, the data must flow through the design from one task to the next. The following coding styles prevent the HLS tool from performing the `DATAFLOW` optimization.

- Single-producer-consumer violations
- Feedback between tasks
- Conditional execution of tasks
- Loops with multiple exit conditions

IMPORTANT! *If any of these coding styles are present, the HLS tool issues a message and does not perform DATAFLOW optimization.*

Finally, the DATAFLOW optimization has no hierarchical implementation. If a sub-function or loop contains additional tasks that might benefit from the DATAFLOW optimization, you must apply the optimization to the loop and the sub-function, or inline the sub-function.

Syntax

```
syn.directive.dataflow=<location> disable_start_propagation
```

- `<location>` is the location (in the format `function[/label]`) at which dataflow optimization is to be performed.
- `disable_start_propagation` disables the creation of a start FIFO used to propagate a start token to an internal process. Such FIFOs can sometimes be a bottleneck for performance.

Examples

Specifies DATAFLOW optimization within function `foo`.

```
syn.directive.dataflow=foo
```

See Also

- syn.directive.allocation

syn.directive.dependence

Description

Vitis HLS detects dependencies within loops: dependencies within the same iteration of a loop are loop-independent dependencies, and dependencies between different iterations of a loop are loop-carried dependencies.

These dependencies are impacted when operations can be scheduled, especially during function and loop pipelining.

- **Loop-independent dependence:** The same element is accessed in a single loop iteration.

```
for (i=1;i<N;i++) {
 A[i]=x;
 y=A[i];
}
```

- **Loop-carried dependence:** The same element is accessed from a different loop iteration.

```
for (i=1;i<N;i++) {
 A[i]=A[i-1]*2;
}
```

Under certain circumstances, such as variable dependent array indexing or when an external requirement needs to be enforced (for example, two inputs are never the same index), the dependence analysis might be too conservative and fail to filter out false dependencies. The `syn.directive.dependence` command allows you to explicitly define the dependencies and eliminate a false dependence.

IMPORTANT! *Specifying a false dependency when the dependency is not false can result in incorrect hardware. Ensure dependencies are correct (true or false) before specifying them.*

Syntax

```
syn.directive.dependence=[OPTIONS] <location>
```

- **`<location>`:** The location in the code, specified as `function[/label]`, where the dependence is defined.

Options

- **`class=(array | pointer)`:** Specifies a class of variables in which the dependence needs clarification. This is mutually exclusive with the `variable` option.

- `variable=<variable>`: Defines a specific variable to apply the dependence directive. Mutually exclusive with the `class` option.

  > **IMPORTANT!** *You cannot specify a `dependence` for function arguments that are bundled with other arguments in an `m_axi` interface. This is the default configuration for `m_axi` interfaces on the function. You also cannot specify a dependence for an element of a struct, unless the struct has been disaggregated.*

- `dependent=(true | false)`: This argument should be specified to indicate whether a dependence is `true` and needs to be enforced, or is `false` and should be removed. However, when not specified, the tool will return a warning that the value was not specified and will assume a value of `false`.

- `direction=(RAW | WAR | WAW)`:

  Note: Relevant only for loop-carried dependencies.

  Specifies the direction for a dependence:

  - `RAW` **(Read-After-Write - true dependence):** The write instruction uses a value used by the read instruction.
  - `WAR` **(Write-After-Read - anti dependence):** The read instruction gets a value that is overwritten by the write instruction.
  - `WAW` **(Write-After-Write - output dependence):** Two write instructions write to the same location, in a certain order.

- `distance=<integer>`:

  Note: Relevant only for loop-carried dependencies where `-dependent` is set to `true`.

  Specifies the inter-iteration distance for array access.

- `type=(intra | inter)`: Specifies whether the dependence is:
  - Within the same loop iteration (`intra`), or
  - Between different loop iterations (`inter`) (default).

Examples

Removes the dependence between `Var1` in the same iterations of `loop_1` in function `func`.

```
syn.directive.dependence=variable=Var1 type=intra \
dependent=false func/loop_1
```

The dependence on all arrays in `loop_2` of function `func` informs Vitis HLS that all reads must happen *after* writes in the same loop iteration.

```
syn.directive.dependence=class=array type=intra \
dependent=true direction=RAW func/loop_2
```

See Also

- syn.directive.disaggregate
- syn.directive.pipeline

syn.directive.disaggregate

Description

The `syn.directive.disaggregate` command lets you deconstruct a `struct` variable into its individual elements. The number and type of elements created are determined by the contents of the struct itself.

IMPORTANT! *Structs used as arguments to the top-level function are aggregated by default, but can be disaggregated with this directive or pragma.*

Syntax

```
syn.directive.disaggregate=<location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) where the variable to disaggregate is found.
- `<variable>` specifies the struct variable name.

Options

This command has no options.

Example 1

The following example shows disaggregates the struct variable `a` in function `top`:

```
syn.directive.disaggregate=top a
```

Example 2

Disaggregated structs can be addressed in your code by the using standard C/C++ coding style as shown below. When using the directives:

```
syn.directive.disaggregate=top a
syn.directive.disaggregate=top c
```

The code can be structured as follows.

```
struct SS
{
  int x[N];
  int y[N];
};

int top(SS *a, int b[4][6], SS &c) {

syn.directive.interface=mode=s_axilite top a->x
syn.directive.interface=mode=s_axilite top a->y

syn.directive.interface=mode=ap_memory top c.x
syn.directive.interface=mode=ap_memory top c.y
```

IMPORTANT! *Notice the different methods shown above for accessing an element of a pointer (a) versus an element of a reference (c)*

Example 3

The following example shows the `Dot` struct containing the `RGB` struct as an element. If you apply `syn.directive.disaggregate` to variable `Arr`, then only the top-level `Dot` struct is disaggregated.

```
struct Pixel {
char R;
char G;
char B;
};

struct Dot {
Pixel RGB;
unsigned Size;
};

#define N 1086
void DUT(Dot Arr[N]) {
...
}

syn.directive.disaggregate=DUT Arr
```

If you want to disaggregate the whole struct, `Dot` and `RGB`, then you can assign the `set_directive_disaggregate` as shown below.

```
void DUT(Dot Arr[N]) {
#pragma HLS disaggregate variable=Arr->RGB
...
}

syn.directive.disaggregate=DUT Arr->RGB
```

The results in this case will be:

```
void DUT(char Arr_RGB_R[N], char Arr_RGB_G[N], char Arr_RGB_B[N], unsigned
Arr_Size[N]) {
...
}
```

See Also

- syn.directive.aggregate

syn.directive.expression_balance

Description

Sometimes C/C++ code is written with a sequence of operations, resulting in a long chain of operations in RTL. With a small clock period, this can increase the latency in the design. By default, the Vitis HLS tool rearranges the operations using associative and commutative properties. The rearrangement creates a balanced tree that can shorten the chain, potentially reducing latency in the design at the cost of extra hardware.

Expression balancing rearranges operators to construct a balanced tree and reduce latency.

- For integer operations expression balancing is on by default but can be disabled.
- For floating-point operations, expression balancing is off by default but can be enabled.

The `syn.directive.expression_balance` command allows this expression balancing to be turned off, or on, within a specified scope.

Syntax

```
syn.directive.expression_balance=[OPTIONS] <location>
```

- `<location>` is the location (in the format `function[/label]`) where expression balancing should be disabled, or enabled.

Options

- `off`: Turns off expression balancing at the specified location. Specifying the `syn.directive.expression_balance` command enables expression balancing in the specified scope. Adding the `off` option disables it.

Examples

Disables expression balancing within function `My_Func`.

```
syn.directive.expression_balance=off My_Func
```

Explicitly enables expression balancing in function `My_Func2`.

```
set_directive_expression_balance=My_Func2
```

syn.directive.function_instantiate

Description

By default:

- Functions remain as separate hierarchy blocks in the RTL, or are decomposed (inlined) into higher-level functions.
- All instances of a function, at the same level of hierarchy, uses the same RTL implementation (block).

The `syn.directive.function_instantiate` command is used to create a unique RTL implementation for each instance of a function, allowing each instance to be optimized around a specific argument or variable.

By default, the following code results in a single RTL implementation of function `func_sub` for all three instances, or if `func_sub` is a small function it is inlined into function `func`.



TIP: *By default, the Vitis HLS tool automatically inlines small functions. This is true even for function instantiations. Using the `set_directive_inline off` option can be used to prevent this automatic inlining.*

```
char func_sub(char inval, char incr)
{
  return inval + incr;
}
void func(char inval1, char inval2, char inval3,
  char *outval1, char *outval2, char * outval3)
{
  *outval1 = func_sub(inval1, 1);
  *outval2 = func_sub(inval2, 2);
  *outval3 = func_sub(inval3, 3);
}
```

Using the directive as shown in the example section below results in three versions of function `func_sub`, each independently optimized for variable `incr`.

Syntax

```
syn.directive.function_instantiate=<location> <variable>
```

- `<location>` is the location (in the format `function[/region label]`) where the instances of a function are to be made unique.
- `<variable>` specifies the function argument to be specified as a constant in the various function instantiations.

Options

This command has no options.

Examples

For the example code shown above, the following Tcl (or pragma placed in function `func_sub`) allows each instance of function `func_sub` to be independently optimized with respect to input `incr`.

```
syn.directive.inline off=func_sub
syn.directive.function_instantiate=func_sub incr
```

See Also

- syn.directive.allocation
- syn.directive.inline

syn.directive.inline

Description

`syn.directive.inline` removes a function as a separate entity in the RTL hierarchy. After inlining, the function is dissolved into the calling function, and no longer appears as a separate level of hierarchy.

IMPORTANT! *Inlining a child function also dissolves any pragmas or directives applied to that function. In Vitis HLS, any directives applied in the child context are ignored.*

In some cases, inlining a function allows operations within the function to be shared and optimized more effectively with the calling function. However, an inlined function cannot be shared or reused, so if the parent function calls the inlined function multiple times, this can increase the area and resource utilization.

By default, inlining is only performed on the next level of function hierarchy.

Syntax

```
syn.directive.inline=[OPTIONS] <location>
```

- `<location>` is the location (in the format `function[/label]`) where inlining is to be performed.

Options

- **`off`**: By default, Vitis HLS performs inlining of smaller functions in the code. Using the `off` option disables inlining for the specified function.
- **`recursive`**: By default, only one level of function inlining is performed. The functions within the specified function are not inlined. The `recursive` option inlines all functions recursively within the specified function hierarchy.

Examples

The following example inlines function `func_sub1`, but no sub-functions called by `func_sub1`.

```
syn.directive.inline=func_sub1
```

This example inlines function `func_sub1`, recursively down the hierarchy, excluding function `func_sub2`:

```
syn.directive.inline=recursive func_sub1
syn.directive.inline=off func_sub2
```

See Also

- syn.directive.allocation

syn.directive.interface

Description

`syn.directive.interface` is only supported for use on the top-level function, and cannot be used for sub-functions of the HLS component. The INTERFACE pragma or directive specifies how RTL ports are created from the function arguments during interface synthesis, as described in the Defining Interfaces section of the *Vitis High-Level Synthesis User Guide* (UG1399). The Vitis HLS tool automatically determines the I/O protocol used by any sub-functions.

Ports in the RTL implementation are derived from the data type and direction of the arguments of the top-level function and function return, the `flow_target` for the HLS component, the default interface configuration settings as specified by `syn.interface.xxx` commands described in Interface Configuration, and by `syn.directive.interface`. Each function argument can be specified to have its own I/O protocol (such as valid handshake or acknowledge handshake).

> **TIP:** *Global variables required on the interface must be explicitly defined as an argument of the top-level function as described in the Global Variables section of the Vitis High-Level Synthesis User Guide (UG1399). However, if a global variable is accessed, but all read and write operations are local to the design, the resource is created in the design. There is no need for an I/O port in the RTL.*

The interface also defines the execution control protocol of the HLS component as described in the Block-Level Control Protocols section of the *Vitis High-Level Synthesis User Guide* (UG1399). The control protocol controls when the HLS component (or block) starts execution, and when the block completes operation, is idle and ready for new inputs.

Syntax

```
syn.directive.interface=[OPTIONS] <location> <port>
```

- `<location>` is the location (in the format `function[/label]`) where the function interface or registered output is to be specified.
- `<port>` is the parameter (function argument) for which the interface has to be synthesized. The port name is not required when block control modes are specified: `ap_ctrl_chain`, `ap_ctrl_hs`, or `ap_ctrl_none`.

Options

> **TIP:** *Many of the options specified below have default values that are defined with `syn.interface.xxx` commands. You can define local values for the interface defined here to override the default values.*

- **`mode=<mode>`:**

  The supported modes, and how the tool implements them in RTL, can be broken down into three categories as follows:

  1. Port-Level Protocols:
     - **`ap_none`:** No port protocol. The interface is a simple data port.
     - **`ap_stable`:** No protocol. The interface is a simple data port, but the Vitis HLS tool assumes the data port is always stable after reset which allows optimizations to remove unnecessary registers.
     - **`ap_vld`:** Implements the data port with an associated `valid` signal to indicate when the data is valid for reading or writing.

- **ap_ack:** Implements the data port with an associated `acknowledge` signal to acknowledge that the data was read or written.
- **ap_hs:** Implements the data port with both `valid` and `acknowledge` signals to provide a two-way handshake to indicate when the data is valid for reading and writing and to acknowledge that the data was read or written
- **ap_ovld:** Implements the output data port with an associated `valid` signal to indicate when the data is valid for reading or writing.

  > **TIP:** *For `ap_ovld` Vitis HLS implements the input argument or the input half of any read/write arguments with mode `ap_none`.*

- **ap_memory:** Implements array arguments as a standard RAM interface. If you use the RTL design in Vivado IP integrator the interface is composed of separate ports.
- **ap_fifo:** Implements the port with a standard FIFO interface using data input and output ports with associated active-Low FIFO `empty` and `full` ports.

  Note: You can only use the `ap_fifo` interface on read arguments or write arguments. `ap_fifo` mode does not support bidirectional read/write arguments.

2. AXI Interface Protocols:
   - **s_axilite:** Implements the port as an AXI4-Lite interface. The tool produces an associated set of C driver files when exporting the generated RT for the HLS component.
   - **m_axi:** Implements the port as an AXI4 interface. You can use the `syn.interface.m_axi_addr64` command to specify either 32-bit (default) or 64-bit address ports and to control any address offset.
   - **axis:** Implements the port as an AXI4-Stream interface.
3. Block-Level Control Protocols:
   - **ap_ctrl_chain:** Implements a set of block-level control ports to `start` the design operation, `continue` operation, and indicate when the design is `idle`, `done`, and `ready` for new input data.
   - **ap_ctrl_hs:** Implements a set of block-level control ports to `start` the design operation and to indicate when the design is `idle`, `done`, and `ready` for new input data.
   - **ap_ctrl_none:** No block-level I/O protocol.

     Note: Using the `ap_ctrl_none` mode might prevent the design from being verified using C/RTL co-simulation.

- **bundle=<string>:**

  By default, the HLS tool groups (or bundles) function arguments that are compatible into a single interface port in the RTL code. All interface ports with compatible options, such as `mode`, `offset`, and `bundle`, are grouped into a single interface port.

> **TIP:** *This default can be changed using the* `syn.interface.m_axi_auto_max_ports` *command.*

This `bundle=<string>` option lets you define bundles to group ports together, overriding the default behavior. The `<string>` specifies the bundle name. The port name used in the generated RTL code is derived automatically from a combination of the `mode` and `bundle`, or is named as specified by `name`.

> **IMPORTANT!** `bundle` *names must be specified using lower-case characters.*

- **`clock=<string>`:** By default, the AXI4-Lite interface clock is the same clock as the system clock. This option is used to set specify a separate clock for an AXI4-Lite interface. If the `bundle` option is used to group multiple top-level function arguments into a single AXI4-Lite interface, the clock option need only be specified on one of bundle members.
- **`channel=<string>`:** To enable multiple channels on an `m_axi` interface specify the channel ID. Multiple `m_axi` interfaces can be combined into a single `m_axi` adapter using separate channel IDs.
- **`depth=<int>`:** Specifies the maximum number of samples for the test bench to process. This setting indicates the maximum size of the FIFO needed in the verification adapter that the HLS tool creates for RTL co-simulation.

  > **TIP:** *While* `depth` *is usually an option, it is needed to specify the size of pointer arguments for RTL co-simulation.*

- **`interrupt=<int>`:** Only used by `ap_vld/ap_hs`. This option enables the I/O to be managed in interrupt, by creating the corresponding bits in the `ISR` and `IER` in the `s_axilite` register file. The integer value N=16..31 specifies the bit position in both registers (by default assigned contiguously from 16).
- **`latency=<value>`:** This option can be used on `ap_memory` and `m_axi` interfaces.
  - In an `ap_memory` interface the option specifies the read latency of the RAM resource driving the interface. By default, a read operation of 1 clock cycle is used. This option allows an external RAM with more than 1 clock cycle of read latency to be modeled.
  - In an `m_axi` interface the option specifies the expected latency of the AXI4 interface, allowing the design to initiate a bus request the specified number of cycles (latency) before the read or write is expected. If this figure it too low, the design will be ready too soon and might stall waiting for the bus. If this figure is too high, bus access might be idle waiting on the design to start the access.
- **`max_read_burst_length=<int>`:** For use with the `m_axi` interface, this option specifies the maximum number of data values read during a burst transfer. Refer to the AXI Burst Transfers section of the *Vitis High-Level Synthesis User Guide* (UG1399) for more information.
- **`max_write_burst_length=<int>`:** For use with the `m_axi` interface, this option specifies the maximum number of data values written during a burst transfer.

- **max_widen_bitwidth=<int>:** Specifies the maximum bit width available for the interface when automatically widening the interface. This overrides the default value specified by the `syn.interface.m_axi_max_bitwidth` command. Refer to Automatic Port Width Resizing in the *Vitis High-Level Synthesis User Guide* (UG1399) for more information.
- **name=<string>:** Specifies a name for the port which will be used in the generated RTL. By default the port name is derived automatically from a combination of the `mode` and `bundle` unless specified by `name`.
- **num_read_outstanding=<int>:** For use with the `m_axi` interface, this option specifies how many read requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, and a FIFO of size:

  ```
  num_read_outstanding*max_read_burst_length*word_size
  ```

- **num_write_outstanding=<int>:** For use with the `m_axi` interface, this option specifies how many write requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, and a FIFO of size:

  ```
  num_write_outstanding*max_write_burst_length*word_size
  ```

- **offset=<string>:** Controls the address offset in AXI4-Lite (`s_axilite`) and AXI4 memory mapped (`m_axi`) interfaces for the specified port.
  - In an `s_axilite` interface, `<string>` specifies the address in the register map.
  - In an `m_axi` interface this option overrides the global option specified by the `config_interface -m_axi_offset` option, and `<string>` is specified as:
    - `off`: Do not generate an offset port.
    - `direct`: Generate a scalar input offset port.
    - `slave`: Generate an offset port and automatically map it to an AXI4-Lite slave interface. This is the default offset.
- **register:** Registers the signal and any associated protocol signals and instructs the signals to persist until at least the last cycle of the function execution. The `syn.interface.register_io` command controls default registering of all interfaces on the top function, while this option lets you override the default for the current interface. This option applies to the following interface modes:
  - `s_axilite`
  - `ap_fifo`
  - `ap_none`
  - `ap_stable`
  - `ap_hs`
  - `ap_ack`

  - `ap_vld`
  - `ap_ovld`

> **TIP:** *The `register` option cannot be used on the return port of the function (`port=return`). Use the `syn.directive.latency` instead.*

- **`register_mode=(both|forward|reverse|off)`:** This option applies to AXI4-Stream interfaces, and specifies if registers are placed on the forward path (`TDATA` and `TVALID`), the reverse path (`TREADY`), on both paths, or if none of the ports signals are to be registered (`off`). The default is `register_mode=both`.

> **TIP:** *AXI4-Stream side-channel signals are considered to be data signals and are registered whenever the `TDATA` is registered.*

- **`storage_impl=<impl>`:** For use with `mode=s_axilite` only, this options defines a storage implementation to assign to the interface. Supported `<impl>` values include `auto`, `bram`, and `uram`. The default is `auto`.

> **TIP:** *`uram` is a synchronous memory with a single clock for two ports available only on certain devices. Therefore `uram` cannot be specified for an `s_axilite` adapter with two clocks, or when the specified part does not support `uram`.*

- **`storage_type=<type>`:**

  For use with `mode=ap_memory` or `mode=bram` only, this options defines a storage type (for example, RAM_T2P) to assign to the variable.

  Supported types include: `ram_1p`, `ram_1wnr`, `ram_2p`, `ram_s2p`, `ram_t2p`, `rom_1p`, `rom_2p`, and `rom_np`.

> **TIP:** *For objects that are not defined on the interface of the top-level function this can be defined using `syn.directive.bind_storage`.*

Example 1

This example disables function-level handshakes for function `func`.

```
syn.directive.interface=mode=ap_ctrl_none func return
```

Example 2

Argument `InData` in function `func` is specified to have a `ap_vld` interface and the input should be registered.

```
set_directive_interface=func InData mode=ap_vld register
```

See Also

- syn.directive.bind_storage
- syn.directive.latency

syn.directive.latency

Description

Specifies a maximum or minimum latency value, or both, on a function, loop, or region.

Vitis HLS always aims for minimum latency. The behavior of the tool when minimum and maximum latency values are specified is as follows:

- Latency is less than the minimum: If Vitis HLS can achieve less than the minimum specified latency, it extends the latency to the specified value, potentially enabling increased sharing.
- Latency is greater than the minimum: The constraint is satisfied. No further optimizations are performed.
- Latency is less than the maximum: The constraint is satisfied. No further optimizations are performed.
- Latency is greater than the maximum: If Vitis HLS cannot schedule within the maximum limit, it increases effort to achieve the specified constraint. If it still fails to meet the maximum latency, it issues a warning. Vitis HLS then produces a design with the smallest achievable latency.



TIP: *You can also use* `syn.directive.latency` *to limit the efforts of the tool to find an optimum solution. Specifying latency constraints for scopes within the code: loops, functions, or regions, reduces the possible solutions within that scope and can improve tool compilation time. Refer to the Improving Synthesis Runtime and Capacity section of the Vitis High-Level Synthesis User Guide (UG1399) for more information.*

To limit the total latency of all loop iterations, the latency directive should be applied to a region that encompasses the entire loop, as in this example: `syn.directive.latency Region_All_Loop_A max=10`

```
Region_All_Loop_A: {
Loop_A: for (i=0; i<N; i++)
  {
  ..Loop Body...
  }
}
```

In this case, even if the loop is unrolled, the latency directive sets a maximum limit on all loop operations.

Syntax

```
syn.directive.latency=[OPTIONS] <location>
```

- `<location>` is the location (function, loop or region) (in the format `function[/label]`) to be constrained.

Options

- **`max=<integer>`:** Limits the maximum latency.
- **`min=<integer>`:** Limits the minimum latency.

Examples

Function `foo` is specified to have a minimum latency of 4 and a maximum latency of 8.

```
syn.directive.latency=min=4 max=8 foo
```

In function `foo`, loop `loop_row` is specified to have a maximum latency of 12.

```
syn.directive.latency=max=12 foo/loop_row
```

syn.directive.loop_flatten

Description

Flattens nested loops into a single loop hierarchy.

In the RTL implementation, it costs a clock cycle to move between loops in the loop hierarchy. Flattening nested loops allows them to be optimized as a single loop. This saves clock cycles, potentially allowing for greater optimization of the loop body logic.

RECOMMENDED: *Apply this directive to the inner-most loop in the loop hierarchy. Only perfect and semi-perfect loops can be flattened in this manner.*

- **Perfect loop nests:**
  - Only the innermost loop has loop body content.
  - There is no logic specified between the loop statements.
  - All loop bounds are constant.
- **Semi-perfect loop nests:**
  - Only the innermost loop has loop body content.
  - There is no logic specified between the loop statements.
  - The outermost loop bound can be a variable.

- **Imperfect loop nests:**

  When the inner loop has variables bounds (or the loop body is not exclusively inside the inner loop), try to restructure the code, or unroll the loops in the loop body to create a perfect loop nest.

Syntax

```
syn.directive.loop_flatten=[OPTIONS] <location>
```

- `<location>` is the location (inner-most loop), in the format `function[/label]`.

Options

- **`off`:** Option to prevent loop flattening from taking place, and can prevent some loops from being flattened while all others in the specified location are flattened.

  IMPORTANT! *The presence of the LOOP_FLATTEN pragma or directive enables the optimization. The addition of `off` disables it.*

Examples

Flattens `loop_1` in function `foo` and all (perfect or semi-perfect) loops above it in the loop hierarchy, into a single loop.

```
set_directive_loop_flatten=foo/loop_1
```

Prevents loop flattening in `loop_2` of function `foo`.

```
set_directive_loop_flatten=off foo/loop_2
```

See Also

- syn.directive.loop_flatten

syn.directive.loop_merge

Description

Merges all loops into a single loop. Merging loops:

- Reduces the number of clock cycles required in the RTL to transition between the loop-body implementations.
- Allows the loops be implemented in parallel (if possible).

The rules for loop merging are:

- If the loop bounds are variables, they must have the same value (number of iterations).
- If loops bounds are constants, the maximum constant value is used as the bound of the merged loop.
- Loops with both variable bound and constant bound cannot be merged.
- The code between loops to be merged cannot have side effects. Multiple execution of this code should generate the same results.
  - `a=b` is allowed
  - `a=a+1` is not allowed.
- Loops cannot be merged when they contain FIFO reads. Merging changes the order of the reads. Reads from a FIFO or FIFO interface must always be in sequence.

Syntax

```
syn.directive.loop_merge=[options] <location>
```

- `<location>` is the location (in the format `function[/label]`) at which the loops reside.

Options

- **`force`**: Forces loops to be merged even when Vitis HLS issues a warning. You must assure that the merged loop will function correctly.

Examples

Merges all consecutive loops in function `foo` into a single loop.

```
syn.directive.loop_merge=foo
```

All loops inside `loop_2` of function `foo` (but not `loop_2` itself) are merged by using the `force` option.

```
syn.directive.loop_merge=force foo/loop_2
```

See Also

- syn.directive.loop_flatten
- syn.directive.unroll

syn.directive.loop_tripcount

Description

The *loop tripcount* is the total number of iterations performed by a loop. Vitis HLS reports the total latency of each loop (the number of cycles to execute all iterations of the loop). This loop latency is therefore a function of the tripcount (number of loop iterations).

IMPORTANT! *`syn.directive.loop_tripcount` is for analysis only, and does not impact the results of synthesis.*

The tripcount can be a constant value. It might depend on the value of variables used in the loop expression (for example, `x<y`) or control statements used inside the loop.

Vitis HLS cannot determine the tripcount in some cases. These cases include, for example, those in which the variables used to determine the tripcount are:

- Input arguments, or
- Variables calculated by dynamic operation

In the following example, the maximum iteration of the for-loop is determined by the value of input `num_samples`. The value of `num_samples` is not defined in the C function, but comes into the function from the outside.

```
void foo (num_samples, ...) {
   int i;
   ...
   loop_1: for(i=0;i< num_samples;i++) {
     ...
     result = a + b;
   }
}
```

In cases where the loop latency is unknown or cannot be calculated, `syn.directive.loop_tripcount` allows you to specify minimum, maximum, and average iterations for a loop. This lets the tool analyze how the loop latency contributes to the total design latency in the reports and helps you determine appropriate optimizations for the design.

TIP: *If a C assert macro is used to limit the size of a loop variable, Vitis HLS can use it to both define loop limits for reporting and create hardware that is exactly sized to these limits.*

Syntax

```
syn.directive.loop_tripcount=[OPTIONS] <location>
```

- `<location>` is the location of the loop (in the format `function[/label]`) at which the tripcount is specified.

Options

- `avg=<integer>`: Specifies the average number of iterations.
- `max=<integer>`: Limits the maximum number of iterations.
- `min=<integer>`: Limits the minimum number of iterations.

Examples

`loop_1` in function `foo` is specified to have a minimum tripcount of 12, and a maximum tripcount of 16:

```
syn.directive.loop_tripcount=min=12 max=16 avg=14 foo/loop_1
```

syn.directive.occurrence

Description

When pipelining functions or loops, the OCCURRENCE pragma or directive specifies that the code in a pipelined function call within the pipelined function or loop is executed at a lower rate than the surrounding function or loop. This allows the pipelined call that is executed at the lower rate to be pipelined at a slower rate, and potentially shared within the top-level pipeline. For example:

- A loop iterates *N* times.
- Part of the loop is protected by a conditional statement and only executes *M* times, where *N* is an integer multiple of *M*.
- The code protected by the conditional is said to have an occurrence of *N*/*M*.

Identifying a region with an OCCURRENCE rate allows the functions and loops in this region to be pipelined with an initiation interval that is slower than the enclosing function or loop.

Syntax

```
syn.directive.occurrence=[OPTIONS] <location>
```

- `<location>` specifies the block of code that contains the pipelined function call(s) with a slower rate of execution.

Options

- `cycle=<int>`: Specifies the occurrence *N*/*M* where:
  - *N* is the number of times the enclosing function or loop is executed.
  - *M* is the number of times the conditional region is executed.

> **IMPORTANT!** *N must be an integer multiple of M.*

- `off=true`: Disable occurrence for the specified function.

Examples

Region `Cond_Region` in function `foo` has an occurrence of 4. It executes at a rate four times slower than the code that encompasses it.

```
syn.directive.occurrence=cycle=4 foo/Cond_Region
```

See Also

- syn.directive.pipeline

syn.directive.performance

Description

> **TIP:** *`syn.directive.performance` applies to loops and loop nests, and requires a known loop tripcount to determine the performance. If your loop has a variable tripcount then you must also specify `syn.directive.tripcount`.*

The `syn.directive.performance` lets you specify a high-level constraint, `target_ti` or `target_tl`, defining the number of clock cycles between successive starts of a loop, and lets the tool infer lower-level UNROLL, PIPELINE, ARRAY_PARTITION, and INLINE directives needed to achieve the desired result. The `syn.directive.performance` does not guarantee the specified value will be achieved, and so it is only a target.

The `target_ti` is the interval between successive starts of the loop, or between the start of the first iteration of the loop, and the next start of the first iteration of the loop. In the following code example, a `target_ti=T` would mean the target interval for the start of loop `L2` between two consecutive iterations of `L1` should be 100 cycles.

```
const int T = 100;
L1: for (int i=0; i<N; i++)
  L2: for (int j=0; j<M; j++){
   #pragma HLS PERFORMANCE target_ti=T
   ...
   }
```

The `target_tl` is the interval between start of the loop and end of the loop, or between the start of the first iteration of the loop and the completion of the last iteration of the loop. In the preceding code example a `target_tl=T` means the target completion of loop `L2` for a single iteration of `L1` should be 100 cycles.

***Note*:** `syn.directive.inline` is applied automatically to functions inside any pipelined loop that has II=1 to improve throughput. If you apply the PERFORMANCE pragma or directive that infers a pipeline with II=1, it will also trigger the auto-inline optimization. You can disable this for specific functions by using `syn.directive.inline off`.

The transaction interval is the initiation interval (II) of the loop times the number of iterations, or tripcount: target_ti = II * loop tripcount. Conversely, `target_ti` = FreqHz / Operations per second.

For example, assuming an image processing function that processes a single frame per invocation with a throughput goal of 60 fps, then the target throughput for the function is 60 invocations per second. If the clock frequency is 180 MHz, then `target_ti` is 180M/60, or 3 million clock cycles per function invocation.

Syntax

Specify the directive for a labeled loop.

```
syn.directive.performance=<location> [Options]
```

Where:

`<location>` specifies the loop in the format `function/loop_label`.

Options

- **`target_ti=<value>`:** Specifies a target transaction interval defined as the number of clock cycles for the loop to complete an iteration. The transaction interval refers to the number of clock cycles from the first transaction of a loop, or nested loop, to the start of the next transaction of the loop. The <value> can be specified as an integer, floating point, or constant expression that is resolved by the tool as an integer.

  ***Note*:** A warning will be returned if truncation occurs.

- **`target_tl=<value>`:**

  Specifies a target latency defined as the number of clock cycles for the loop to complete all iterations. The transaction latency is defined as the interval between the start of the first iteration of the loop, and the completion of the last iteration of the loop. The <value> can be specified as an integer, floating point, or constant expression that is resolved by the tool as an integer.

- **unit=[sec | cycle]:** Specifies the unit associated with the `target_ti` or `target_tl` values. The unit can either be specified as seconds, or clock cycles. When the unit is specified as seconds, a unit can be specified with the value to indicate nanoseconds (ns), picoseconds (ps), microseconds (us).

Example 1

The loop labeled `loop_1` is specified to have target transaction interval of 4 clock cycles:

```
syn.directive.performance=loop_1 target_ti=4
```

See Also

- syn.directive.inline

syn.directive.pipeline

Description

Reduces the initiation interval (II) for a function or loop by allowing the concurrent execution of operations as described in the Pipelining Paradigm section of the *Vitis High-Level Synthesis User Guide* (UG1399). A pipelined function or loop can process new inputs every N clock cycles, where N is the initiation interval (`II`). An II of 1 processes a new input every clock cycle.

As a default behavior, with `syn.directive.pipeline` the tool will generate the minimum II for the design according to the specified clock period constraint. The emphasis will be on meeting timing, rather than on achieving II unless the `-II` option is specified.

The default type of pipeline is defined by the `syn.compile.pipeline_style` command as described in Compile Options, but can be overridden in the PIPELINE pragma or directive.

If Vitis HLS cannot create a design with the specified II, it issues a warning and creates a design with the lowest achievable II. You can then analyze the design with the warning messages to determine what steps must be taken to create a design that satisfies the required initiation interval.

Syntax

```
syn.directive.pipeline=[OPTIONS] <location>
```

Where:

- `<location>` is the location (in the format `function[/label]`) to be pipelined.

Options

- **`II=<integer>`**: Specifies the desired initiation interval for the pipeline. Vitis HLS tries to meet this request. Based on data dependencies, the actual result might have a larger II.
- **`off`**: Turns off pipeline for a specific loop or function. This can be used when `config_compile -pipeline_loops` is used to globally pipeline loops.
- **`rewind`**:

Note: Applicable only to a loop.

Optional keyword. Enables rewinding as described in Rewiding Pipelined Loops for Performance of*Vitis High-Level Synthesis User Guide* (UG1399). This enables continuous loop pipelining with no pause between one execution of the loop ending and the next execution starting. Rewinding is effective only if there is one single loop (or a perfect loop nest) inside the top-level function. The code segment before the loop:

- Is considered as initialization.
- Is executed only once in the pipeline.
- Cannot contain any conditional operations (`if-else`).

- **`style=<stp | frp | flp>`:** Specifies the type of pipeline to use for the specified function or loop. For more information on pipeline styles refer to the Flushing Pipeline and Pipeline Types section of the *Vitis High-Level Synthesis User Guide* (UG1399). The types of pipelines include:
  - **`stp`:** Stall pipeline. Runs only when input data is available otherwise it stalls. This is the default setting, and is the type of pipeline used by Vitis HLS for both loop and function pipelining. Use this when a flushable pipeline is not required. For example, when there are no performance or deadlock issue due to stalls.
  - **`flp`:** This option defines the pipeline as a flushable pipeline. This type of pipeline typically consumes more resources and/or can have a larger II because resources cannot be shared among pipeline iterations.
  - **`frp`:** Free-running, flushable pipeline. Runs even when input data is not available. Use this when you need better timing due to reduced pipeline control signal fanout, or when you need improved performance to avoid deadlocks. However, this pipeline style can consume more power as the pipeline registers are clocked even if there is no data.

  IMPORTANT! *This is a hint not a hard constraint. The tool checks design conditions for enabling pipelining. Some loops might not conform to a particular style and the tool reverts to the default style (`stp`) if necessary.*

Examples

Function `func` is pipelined with the specified initiation interval.

```
syn.directive.pipeline=func II=1
```

See Also

- syn.directive.dependence

syn.directive.protocol

Description

This commands specifies a region of code, a protocol region, in which no clock operations will be inserted by Vitis HLS unless explicitly specified in the code. The tool will not insert any clocks between operations in the region, including those which read from or write to function arguments. The order of read and writes will therefore be strictly followed in the synthesized RTL.

A region of code can be created in the C/C++ code by enclosing the region in braces "{ }" and naming it. The following defines a region named `io_section`:

```
io_section:{
...
lines of code
...
}
```

A clock operation can be explicitly specified in C code using an `ap_wait()` statement, and can be specified in C++ code by using the `wait()` statement.



TIP: *The `ap_wait` and `wait` statements have no effect on the simulation of the design.*

Syntax

```
syn.directive.protocol=[OPTIONS] <location>
```

The `<location>` specifies the location (in the format `function[/label]`) at which the protocol region is defined.

Options

- **`mode=[floating | fixed]`:**
  - `floating`: Lets code statements outside the protocol region overlap and execute in parallel with statements in the protocol region in the final RTL. The protocol region remains cycle accurate, but outside operations can occur at the same time. This is the default mode.
  - `fixed`: The fixed mode ensures that statements outside the protocol region do not execute in parallel with the protocol region.

Examples

The example code defines a protocol region, `io_section` in function `foo`. The following directive defines that region as a fixed mode protocol region:

```
syn.directive.protocol=mode=fixed foo/io_section
```

syn.directive.reset

Description

The RESET pragma or directive adds or disables reset ports for specific state variables (global or static).

The reset port is used to restore the registers and block RAM, connected to the port, to an initial value any time the reset signal is applied. Globally, the presence and behavior of the RTL reset port is controlled using the `syn.rtl.reset` configuration settings. The reset configuration settings include the ability to define the polarity of the reset, and specify whether the reset is synchronous or asynchronous, but more importantly it controls, through the reset option, which registers are reset when the reset signal is applied. For more information, see Controlling Initialization and Reset Behaviour in *Vitis High-Level Synthesis User Guide* (UG1399).

More specific control over reset is provided through the RESET pragma. For global or static variables the RESET pragma is used to explicitly enable a reset when none is present, or the variable can be removed from the reset by turning `off` the pragma. This can be particularly useful when static or global arrays are present in the design.

***Note*:** For public variables of a class, the RESET pragma must be used as the reset configuration settings only apply to variables declared at the function or file level. In addition, the RESET pragma must be applied to an object of the class in the top-function or sub-function, and cannot be applied to private variables of the class.

Syntax

Place the pragma in the C source within the boundaries of the variable life cycle.

```
syn.directive.reset=<location> [ variable=<a> | off=true ]
```

Where:

- **`location>`:** Specifies the location (in the format `function[/label]`) at which the variable is defined.
- **`variable=<a>`:** Specifies the variable to which the RESET pragma is applied.
- **`off` or `off=true`:** Indicates that reset is not generated for the specified variable.

Examples

Adds reset to variable `a` in function `foo` even when the global reset (`syn.rtl.reset`) setting is `none` or `control`.

```
syn.directive.reset=foo a
```

Removes reset from variable `static int a` in function `foo` even when the global reset setting is `state` or `all`.

```
syn.directive.reset=off foo a
```

The following example shows the use of the RESET pragma or directive with class variables and methods. A variable declared in a class must be a public static variable in the class and can have the RESET pragma specified for the variable in a method of the class, or after an object of the class has been created in the top function or sub-function of the HLS design.

```
class bar {
public:
  static int a; // class level static; must be public
  int a1;       // class-level non-static; must be public
  bar(...) {
  // #pragma reset does not work here for a or a1
  }
  // this is the function that is called in the top
  void run(...) {
  // #pragma reset does not work here for non-static variable a1
  // but does work for static variable a
  #pragma reset variable=a
  }
};
static int c; // file level
int d; // file level
void top(...) {
#pragma HLS top
  static int b; // function level
  bar t;
  static bar s;

  // #pragma reset works here for b, c and d, as well as t and s members
  // except for t.a1 because it is not static
  #pragma reset variable=b
  #pragma reset variable=c
  #pragma reset variable=d
  #pragma reset variable=t.a
  #pragma reset variable=s.a
  #pragma reset variable=s.a1
  t.run(...);
  s.run(...);
}
```

The `syn.directive.reset` command specified for the function `top` shown above would be as follows:

```
syn.directive.reset=top variable=b
syn.directive.reset=top variable=c
syn.directive.reset=top variable=d
syn.directive.reset=top variable=t.a
syn.directive.reset=top variable=s.a
syn.directive.reset=top variable=s.a1
```

See Also

- RTL Configuration

syn.directive.stable

Description

`syn.directive.stable` is applied to arguments of a DATAFLOW or PIPELINE region and is used to indicate that an input or output of this region can be ignored when generating the synchronizations at entry and exit of the DATAFLOW region. This means that the reading accesses of that argument do not need to be part of the "first stage" of the task-level (resp. fine-grain) pipeline for inputs, and the write accesses do not need to be part of the last stage of the task-level (resp. fine-grain) pipeline for outputs.

The pragma can be specified at any point in the hierarchy, on a scalar or an array, and automatically applies to all the DATAFLOW or PIPELINE regions below that point. The effect of STABLE for an input is that a DATAFLOW or PIPELINE region can start another iteration even though the value of the previous iteration has not been read yet. For an output, this implies that a write of the next iteration can occur although the previous iteration is not done.

Syntax

```
syn.directive.stable=<location> <variable>
```

- `<location>` is the function name or loop name where the directive is to be constrained.
- `<variable>` is the name of the array to be constrained.

Examples

In the following example, without the STABLE directive, `proc1` and `proc2` would be synchronized to acknowledge the reading of their inputs (including `A`). With the directive, `A` is no longer considered as an input that needs synchronization.

```
void dataflow_region(int A[...], int B[…] ...
    proc1(...);
    proc2(A, ...);
```

The directives for this example would be scripted as:

```
syn.directive.stable=dataflow_region variable=A
syn.directive.dataflow dataflow_region
```

See Also

- syn.directive.dataflow
- syn.directive.pipeline

syn.directive.stream

Description

By default, array variables are implemented as RAM:

- Top-level function array parameters are implemented as a RAM interface port.
- General arrays are implemented as RAMs for read-write access.
- Arrays involved in sub-functions, or loop-based DATAFLOW optimizations are implemented as a RAM ping-pong buffer channel.

If the data stored in the array is consumed or produced in a sequential manner, a more efficient communication mechanism is to use streaming data, where FIFOs are used instead of RAMs. When an argument of the top-level function is specified as INTERFACE `mode=ap_fifo`, the array is automatically implemented as streaming. See the Interfaces for Vivado IP Flow section of the *Vitis High-Level Synthesis User Guide* (UG1399) or more information.

IMPORTANT! *To preserve the accesses, it might be necessary to prevent compiler optimizations (in particular dead code elimination) by using the `volatile` qualifier as described in the Type Qualifiers section of the Vitis High-Level Synthesis User Guide (UG1399).*

Syntax

```
syn.directive.stream=[OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the array variable.
- `<variable>` is the array variable to be implemented as a FIFO.

Options

- **`depth=<integer>`:**

  ***Note*:** Relevant only for array streaming in dataflow channels.

  By default, the depth of the FIFO implemented in the RTL is the same size as the array specified in the C code. This options allows you to modify the size of the FIFO.

  When the array is implemented in a DATAFLOW region, it is common to the use the `-depth` option to reduce the size of the FIFO. For example, in a `DATAFLOW` region where all loops and functions are processing data at a rate of II = 1, there is no need for a large FIFO because data is produced and consumed in each clock cycle. In this case, you can use the `-depth` to reduce the FIFO size to 2 to substantially reduce the area of the RTL design.

This same functionality is provided for all arrays in a DATAFLOW region using the `config_dataflow` command with the `-depth` option. The `-depth` option used with `set_directive_stream` overrides the default specified using `config_dataflow`.

- **type=<arg>:** Specify a mechanism to select between FIFO, PIPO, synchronized shared (`shared`), and un-synchronized shared (`unsync`). The supported types include:
  - `fifo`: A FIFO buffer with the specified `depth`.
  - `pipo`: A regular Ping-Pong buffer, with as many "banks" as the specified depth (default is 2).
  - `shared`: A shared channel, synchronized like a regular Ping-Pong buffer, with depth, but without duplicating the array data. Consistency can be ensured by setting the depth small enough, which acts as the distance of synchronization between the producer and consumer.

    TIP: *The default depth for shared is 1.*

  - `unsync`: Does not have any synchronization except for individual memory reads and writes. Consistency (read-write and write-write order) must be ensured by the design itself.

Examples

Specifies array `A[10]` in function `func` to be streaming and implemented as a FIFO.

```
syn.directive.stream=func A type=fifo
```

Array B in named loop `loop_1` of function `func` is set to streaming with a FIFO depth of 12. In this case, place the pragma inside `loop_1`.

```
syn.directive.stream=depth=12 type=fifo func/loop_1 B
```

Array C has streaming implemented as a PIPO.

```
syn.directive.stream=type=pipo func C
```

See Also

- syn.directive.dataflow
- syn.directive.interface
- Dataflow Configuration

syn.directive.top

Description

Attaches a name to a function, which can then be used by the `set_top` command to set the named function as the top. This is typically used to synthesize member functions of a class in C++.

Syntax

```
syn.directive.top=[OPTIONS] <location>
```

- `<location>` is the function to be renamed.

Options

- **`name=<string>`**: Specifies the name of the function to be used by the `syn.top` command as described in HLS General Options.

Examples

Function `foo_long_name` is renamed to `DESIGN_TOP`, which is then specified as the top-level. If the pragma is placed in the code, the `set_top` command must still be issued in the top-level specified in the GUI project settings.

```
syn.directive.top=name=DESIGN_TOP foo_long_name
```

Followed by the `syn.top=DESIGN_TOP` command.

See Also

- `syn.top` in HLS General Options

syn.directive.unroll

Description

Transforms loops by creating multiples copies of the loop body.

A loop is executed for the number of iterations specified by the loop induction variable. The number of iterations can also be impacted by logic inside the loop body (for example, break or modifications to any loop exit variable). The loop is implemented in the RTL by a block of logic representing the loop body, which is executed for the same number of iterations.

The `syn.directive.unroll` command allows the loop to be fully or partially unrolled. Fully unrolling the loop creates as many copies of the loop body in the RTL as there are loop iterations. Partially unrolling a loop by a factor *N*, creates *N* copies of the loop body and adjusting the loop iteration accordingly.

If the factor *N* used for partial unrolling is not an integer multiple of the original loop iteration count, the original exit condition must be checked after each unrolled fragment of the loop body.

To unroll a loop completely, the loop bounds must be known at compile time. This is not required for partial unrolling.

Syntax

```
syn.directive.unroll=[OPTIONS] <location>
```

- `<location>` is the location of the loop (in the `format function[/label]`) to be unrolled.

Options

- **`factor=<integer>`:** Specifies a non-zero integer indicating that partial unrolling is requested.

  The loop body is repeated this number of times. The iteration information is adjusted accordingly.

- **`skip_exit_check`:** Effective only if a factor is specified (partial unrolling).
  - Fixed bounds

    No exit condition check is performed if the iteration count is a multiple of the factor.

    If the iteration count is *not* an integer multiple of the factor, the tool:

    . Prevents unrolling.

    . Issues a warning that the exit check must be performed to proceed.
  - Variable bounds

    The exit condition check is removed. You must ensure that:

    ◦ The variable bounds is an integer multiple of the factor.

    ◦ No exit check is in fact required.

- **`off=true`:** Disable unroll for the specified loop.

Examples

Unrolls loop `L1` in function `foo`. Place the pragma in the body of loop `L1`.

```
syn.directive.unroll=foo/L1
```

Specifies an unroll factor of 4 on loop `L2` of function `foo`. Removes the exit check. Place the pragma in the body of loop `L2`.

```
syn.directive.unroll=skip_exit_check factor=4 foo/L2
```

See Also

- syn.directive.loop_flatten
- syn.directive.loop_merge

Chapter 16

HLS Pragmas

Optimizations in Vitis HLS

In the AMD Vitis™ software platform, a kernel defined in the C/C++ language, or OpenCL™ C, must be compiled into the register transfer level (RTL) that can be implemented into the programmable logic of an AMD device. The `v++` compiler calls the Vitis High-Level Synthesis (HLS) tool to synthesize the RTL code from the kernel source code.

The HLS tool is intended to work with the Vitis IDE project without interaction. However, the HLS tool also provides pragmas that can be used to optimize the design, reduce latency, improve throughput performance, and reduce area and device resource usage of the resulting RTL code. These pragmas can be added directly to the source code for the kernel.

The HLS pragmas include the optimization types specified in the following table.

Table 37: **Vitis HLS Pragmas by Type**

Type	Attributes
Kernel Optimization	• pragma HLS aggregate • pragma HLS alias • pragma HLS disaggregate • pragma HLS expression_balance • pragma HLS latency • pragma HLS performance • pragma HLS protocol • pragma HLS reset • pragma HLS top • pragma HLS stable
Function Inlining	• pragma HLS inline
Interface Synthesis	• pragma HLS interface • pragma HLS stream
Task-level Pipeline	• pragma HLS dataflow • pragma HLS stream
Pipeline	• pragma HLS pipeline • pragma HLS occurrence
Loop Unrolling	• pragma HLS unroll • pragma HLS dependence

Table 37: **Vitis HLS Pragmas by Type** *(cont'd)*

Type	Attributes
Loop Optimization	• pragma HLS loop_flatten • pragma HLS loop_merge • pragma HLS loop_tripcount
Array Optimization	• pragma HLS array_partition • pragma HLS array_reshape
Structure Packing	• pragma HLS aggregate • pragma HLS dataflow
Resource Utilization	• pragma HLS allocation • pragma HLS bind_op • pragma HLS bind_storage • pragma HLS function_instantiate

pragma HLS aggregate

Description

Collects and groups the data fields of a struct into a single scalar with a wider word width.

The AGGREGATE pragma is used for grouping all the elements of a struct into a single wide vector to allow all members of the struct to be read and written to simultaneously. The bit alignment of the resulting new wide-word can be inferred from the declaration order of the struct elements. The first element takes the LSB of the vector, and the final element of the struct is aligned with the MSB of the vector.

If the struct contains arrays, the AGGREGATE pragma performs a similar operation as ARRAY_RESHAPE, and combines the reshaped array with the other elements in the struct. Any arrays declared inside the struct are completely partitioned and reshaped into a wide scalar and packed with other scalar elements.

IMPORTANT! *You should exercise some caution when using the AGGREGATE optimization on struct objects with large arrays. If an array has 4096 elements of type int, this will result in a vector (and port) of width 4096×32=131072 bits. The Vitis HLS tool can create this RTL design, however it is very unlikely logic synthesis will be able to route this during the FPGA implementation.*

Syntax

Place the pragma near the definition of the struct variable to aggregate:

```
#pragma HLS aggregate variable=<variable> compact=<arg>
```

Where:

- `variable=<variable>`: Specifies the variable to be grouped.
- `compact=[bit | byte | none | auto]`: Specifies the alignment of the aggregated struct. Alignment can be on the bit-level, the byte-level, none, or automatically determined by the tool which is the default behavior.

Example 1

Aggregates struct pointer `AB` with three 8-bit fields (`typedef struct {unsigned char R, G, B;} pixel`) in function `func`, into a new 24-bit pointer aligned on the bit-level.

```
typedef struct{
unsigned char R, G, B;
} pixel;

pixel AB;
#pragma HLS aggregate variable=AB compact=bit
```

Example 2

Aggregates struct array `AB[17]` with three 8-bit field fields (R, G, B) into a new 17 element array of 24-bits.

```
typedef struct{
unsigned char R, G, B;
} pixel;

pixel AB[17];
#pragma HLS aggregate variable=AB
```

See Also

- set_directive_aggregate
- pragma HLS array_reshape
- pragma HLS disaggregate

pragma HLS alias

Description

Specify that two or more `M_AXI` pointer arguments point to the same underlying buffer in memory (DDR or HBM) and indicate any aliasing between the pointers by setting the distance or offset between them.

IMPORTANT! *The ALIAS pragma applies to top-level function arguments mapped to `M_AXI` interfaces.*

Vitis HLS considers different pointers to be independent channels and generally does not provide any dependency analysis. However, in cases where the host allocates a single buffer for multiple pointers, this relationship can be communicated through the ALIAS pragma or directive and dependency analysis can be maintained. The ALIAS pragma enables data dependence analysis in Vitis HLS by defining the distance between pointers in the buffer.

Requirements for ALIAS:

- All ports assigned to an ALIAS pragma must be in assigned to `M_AXI` interfaces and assigned to different bundles or channels. See example below with bundles.
- Each port can only be used in one ALIAS pragma or directive
- The `depth` of all ports assigned to an ALIAS pragma must be the same
- When offset is specified, the number of ports and number of offsets specified must be the same: one offset per port
- The offset for the INTERFACE must be specified as slave or direct, `offset=off` is not supported

Syntax

```
pragma HLS alias ports=<list> [distance=<int> | offset=<list...>]
```

Where:

- **`ports=<list>`:** specifies the ports to alias.
- **`distance=<integer>`:** Specifies the difference between the pointer values passed to the ports in the list.
- **`offset=<list>`:** Specifies the offset of the pointer passed to each port in the `ports` list with respect to the origin of the array.

***Note*:** `offset` and `distance` are mutually exclusive.

Example

For the following function `top`:

```
void top(int *arr0, int *arr1, int *arr2, int *arr3, ...) {
  #pragma HLS interface mode=m_axi port=arr0 bundle=hbm0 depth=0x40000000
  #pragma HLS interface mode=m_axi port=arr1 bundle=hbm1 depth=0x40000000
  #pragma HLS interface mode=m_axi port=arr2 bundle=hbm2 depth=0x40000000
  #pragma HLS interface mode=m_axi port=arr3 bundle=hbm3 depth=0x40000000
```

The following pragma defines aliasing for the specified array pointers, and defines the `distance` between them:

```
#pragma HLS ALIAS ports=arr0,arr1,arr2,arr3 distance=10000000
```

Alternatively, the following pragma specifies the `offset` between pointers, to accomplish the same effect:

```
#pragma HLS ALIAS ports=arr0,arr1,arr2,arr3
offset=00000000,10000000,20000000,30000000
```

See Also

- set_directive_alias
- pragma HLS interface

pragma HLS allocation

Description

Specifies restrictions to limit resource allocation in the implemented kernel. The ALLOCATION pragma or directive can limit the number of RTL instances and hardware resources used to implement specific functions, loops, or operations. The ALLOCATION pragma is specified inside the body of a function, a loop, or a region of code.

For example, if the C source has four instances of a function `foo_sub`, the ALLOCATION pragma can ensure that there is only one instance of `foo_sub` in the final RTL. All four instances of the C function are implemented using the same RTL block. This reduces resources used by the function, but negatively impacts performance by sharing those resources.

Template functions can also be specified for ALLOCATION by specifying the function pointer instead of the function name, as shown in the examples below.

The operations in the C code, such as additions, multiplications, array reads, and writes, can also be limited by the ALLOCATION pragma.

Syntax

Place the pragma inside the body of the function, loop, or region where it will apply.

IMPORTANT! *The order of the arguments below is important. The `<type>` as operation or function must follow the `allocation` keyword.*

```
#pragma HLS allocation <type> instances=<list>
limit=<value>
```

Where:

- **`<type>`:** The type is specified as one of the following:
  - **`function`:** Specifies that the allocation applies to the functions listed in the `instances=` list. The function can be any function in the original C or C++ code that has *not* been:
    - Inlined by the `pragma HLS inline`, or the `set_directive_inline` command, or
    - Inlined automatically by the Vitis HLS tool.
  - **`operation`:** Specifies that the allocation applies to the operations listed in the `instances=` list.
- **`instances=<list>`:** Specifies the names of functions from the C code, or operators.

  For a complete list of operations that can be limited using the `ALLOCATION` pragma, refer to the config_op command.
- **`limit=<value>`:** Optionally specifies the limit of instances to be used in the kernel.

Example 1

Given a design with multiple instances of function `foo`, this example limits the number of instances of `foo` in the RTL for the hardware kernel to two.

```
#pragma HLS allocation function instances=foo limit=2
```

Example 2

Limits the number of multiplier operations used in the implementation of the function `my_func` to one. This limit does not apply to any multipliers outside of `my_func`, or multipliers that might reside in sub-functions of `my_func`.

TIP: *To limit the multipliers used in the implementation of any sub-functions, specify an allocation directive on the sub-functions or inline the sub-function into function `my_func`.*

```
void my_func(data_t angle) {
#pragma HLS allocation operation instances=mul limit=1
...
}
```

Example 3

The ALLOCATION pragma can also be used on template functions as shown below. The identification is generally based on the function name, but in the case of template functions it is based on the function pointer:

```
template <typename DT>
void foo(DT a, DT b){
}
// The following is valid
#pragma HLS ALLOCATION function instances=foo<DT>
...
// The following is not valid
#pragma HLS ALLOCATION function instances=foo
```

See Also

- set_directive_allocation
- pragma HLS inline

pragma HLS array_partition

Description

IMPORTANT! *`Array_Partition` and `Array_Reshape` pragmas and directives are not supported for `M_AXI` Interfaces on the top-level function. Instead you can use the `hls::vector` data types as described in Vector Data Types.*

Partitions an array into smaller arrays or individual elements and provides the following:

- Results in RTL with multiple small memories or multiple registers instead of one large memory.
- Effectively increases the amount of read and write ports for the storage.
- Potentially improves the throughput of the design.
- Requires more memory instances or registers.

Syntax

Place the pragma in the C source within the boundaries of the function where the array variable is defined.

```
#pragma HLS array_partition variable=<name> \
type=<type>  factor=<int>  dim=<int> off=true
```

Where:

- **`variable=<name>`:** A required argument that specifies the array variable to be partitioned.

- `type=<type>`: Optionally specifies the partition type. The default type is `complete`. The following types are supported:
  - `cyclic`: Cyclic partitioning creates smaller arrays by interleaving elements from the original array. The array is partitioned cyclically by putting one element into each new array before coming back to the first array to repeat the cycle until the array is fully partitioned. For example, if `factor=3` is used:
    - Element 0 is assigned to the first new array
    - Element 1 is assigned to the second new array.
    - Element 2 is assigned to the third new array.
    - Element 3 is assigned to the first new array again.
  - `block`: Block partitioning creates smaller arrays from consecutive blocks of the original array. This effectively splits the array into N equal blocks, where N is the integer defined by the `factor=` argument.
  - `complete`: Complete partitioning decomposes the array into individual elements. For a one-dimensional array, this corresponds to resolving a memory into individual registers. This is the default `<type>`.
- `factor=<int>`: Specifies the number of smaller arrays that are to be created.

  IMPORTANT! *For complete type partitioning, the factor is not specified. For block and cyclic partitioning, the `factor=` is required.*
- `dim=<int>`: Specifies which dimension of a multi-dimensional array to partition. Specified as an integer from 0 to <N>, for an array with <N> dimensions:
  - If a value of 0 is used, all dimensions of a multi-dimensional array are partitioned with the specified type and factor options.
  - Any non-zero value partitions only the specified dimension. For example, if a value 1 is used, only the first dimension is partitioned.
- `off=true`: Disables the ARRAY_PARTITION feature for the specified variable. Does not work with `dim`, `factor`, or `type`.

Example 1

This example partitions the 13 element array, AB[13], into four arrays using block partitioning:

```
#pragma HLS array_partition variable=AB type=block factor=4
```

TIP: *Because four is not an integer factor of 13:*

- Three of the new arrays have three elements each
- One array has four elements (AB[9:12])

Example 2

This example partitions dimension two of the two-dimensional array, AB[6][4] into two new arrays of dimension [6][2]:

```
#pragma HLS array_partition variable=AB type=block factor=2 dim=2
```

Example 3

This example partitions the second dimension of the two-dimensional `in_local` array into individual elements.

```
int in_local[MAX_SIZE][MAX_DIM];
#pragma HLS ARRAY_PARTITION variable=in_local type=complete dim=2
```

Example 4

Partitioned arrays can be addressed in your code by the new structure of the array, as shown in the following code example;

```
struct SS
{
  int x[N];
  int y[N];
};

int top(SS *a, int b[4][6], SS &c) {
#pragma HLS array_partition type=complete dim=1 variable=b
#pragma HLS interface mode=ap_memory port = b[0]
#pragma HLS interface mode=ap_memory port = b[1]
#pragma HLS interface mode=ap_memory port = b[2]
#pragma HLS interface mode=ap_memory port = b[3]
```

See Also

- set_directive_array_partition
- pragma_HLS_array_reshape

pragma HLS array_reshape

Description

IMPORTANT! *`Array_Partition` and `Array_Reshape` pragmas and directives are not supported for `M_AXI` Interfaces on the top-level function. Instead you can use the `hls::vector` data types as described in Vector Data Types.*

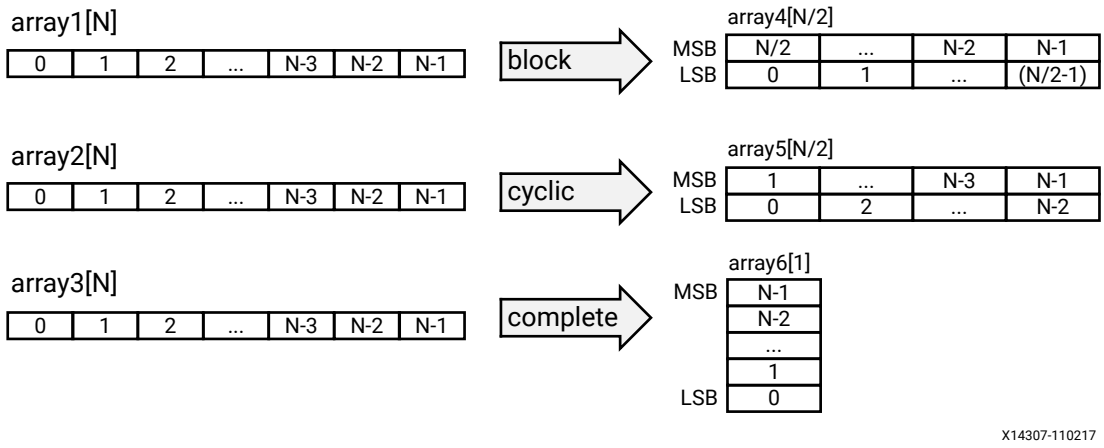
The ARRAY_RESHAPE pragma reforms the array with vertical remapping and concatenating elements of arrays by increasing bit-widths. This reduces the number of block RAM consumed while providing parallel access to the data. This pragma creates a new array with fewer elements but with greater bit-width, allowing more data to be accessed in a single clock cycle.

Given the following code:

```
void foo (...) {
int array1[N];
int array2[N];
int array3[N];
#pragma HLS ARRAY_RESHAPE variable=array1 type=block factor=2 dim=1
#pragma HLS ARRAY_RESHAPE variable=array2 type=cyclic factor=2 dim=1
#pragma HLS ARRAY_RESHAPE variable=array3 type=complete dim=1
...
}
```

The ARRAY_RESHAPE pragma transforms the arrays into the form shown in the following figure.

Figure 141: **ARRAY_RESHAPE Pragma**



Syntax

Place the pragma in the C source within the region of a function where the array variable is defines.

```
#pragma HLS array_reshape variable=<name> type=<type>  factor=<int>
dim=<int> off=true
```

Where:

- `variable=<name>`: Required argument that specifies the array variable to be reshaped.
- `type=<type>`: Optionally specifies the partition type. The default type is `complete`. The following types are supported:

  - `cyclic`: Cyclic reshaping creates smaller arrays by interleaving elements from the original array. For example, if `factor=3` is used, element 0 is assigned to the first new array, element 1 to the second new array, element 2 is assigned to the third new array, and then element 3 is assigned to the first new array again. The final array is a vertical concatenation (word concatenation, to create longer words) of the new arrays into a single array.
  - `block`: Block reshaping creates smaller arrays from consecutive blocks of the original array. This effectively splits the array into `<N>` equal blocks where `<N>` is the integer defined by `factor=`, and then combines the `<N>` blocks into a single array with `word-width*N`.
  - `complete`: Complete reshaping decomposes the array into temporary individual elements and then recombines them into an array with a wider word. For a one-dimension array this is equivalent to creating a very-wide register (if the original array was N elements of M bits, the result is a register with `N*M` bits). This is the default type of array reshaping.
- `factor=<int>`: Specifies the amount to divide the current array by (or the number of temporary arrays to create). A factor of 2 splits the array in half, while doubling the bit-width. A factor of 3 divides the array into three, with triple the bit-width.

  IMPORTANT! *For complete type partitioning, the factor is not specified. For block and cyclic reshaping, the `factor=` is required.*

- `dim=<int>`: Specifies which dimension of a multi-dimensional array to partition. Specified as an integer from 0 to `<N>`, for an array with `<N>` dimensions:
  - If a value of 0 is used, all dimensions of a multi-dimensional array are partitioned with the specified type and factor options.
  - Any non-zero value partitions only the specified dimension. For example, if a value 1 is used, only the first dimension is partitioned.
- `object`: A keyword relevant for container arrays only. When the keyword is specified the ARRAY_RESHAPE pragma applies to the objects in the container, reshaping all dimensions of the objects within the container, but all dimensions of the container itself are preserved. When the keyword is not specified the pragma applies to the container array and not the objects.
- `off=true`: Disables the ARRAY_RESHAPE feature for the specified variable.

Example 1

Reshapes an 8-bit array with 17 elements, `AB[17]`, into a new 32-bit array with five elements using block mapping.

```
#pragma HLS array_reshape variable=AB type=block factor=4
```

TIP: *`factor=4` indicates that the array should be divided into four; this means that 17 elements are reshaped into an array of five elements, with four times the bit-width. In this case, the last element, `AB[17]`, is mapped to the lower eight bits of the fifth element, and the rest of the fifth element is empty.*

Example 2

Reshapes the two-dimensional array `AB[6][4]` into a new array of dimension [6][2], in which dimension 2 has twice the bit-width.

```
#pragma HLS array_reshape variable=AB type=block factor=2 dim=2
```

Example 3

Reshapes the three-dimensional 8-bit array, `AB[4][2][2]` in function `func`, into a new single element array (a register), 128-bits wide (4×2×2×8).

```
#pragma HLS array_reshape variable=AB type=complete dim=0
```

TIP: *`dim=0` means to reshape all dimensions of the array.*

Example 4

Partitioned arrays can be addressed in your code by the new structure of the array, as shown in the following code example;

```
struct SS
{
  int x[N];
  int y[N];
};

int top(SS *a, int b[4][6], SS &c) {
#pragma HLS array_reshape type=complete dim=0 variable=b
#pragma HLS interface mode=ap_memory port=b[0]
```

See Also

- syn.directive.array_reshape
- pragma HLS array_partition

pragma HLS array_stencil

Description

#pragma HLS array_stencil variable=<name > <off>

Use "#pragma HLS array_stencil variable=xxx off" to disable auto stencil on a variable in a loop.

Global Auto Control

config_array_stencil -throughput_driven <off|on>

Code Canonical Forms and Limitations:

- Stencil optimization can only be applied to the top level.
- Loop bounds need to be constant for multi-dimensional stencil patterns.
- The pragma must be located within the loop.
- The loop where stencil optimization is applied must require a pipeline pragma with II=1.
- Stencil optimization requires nested loops to be perfect nested loops.
- The access pattern should have a fixed pattern.
- The access pattern order must follow the loop order, for example, it must have a loop increment +/- 1.

pragma HLS bind_op

Description

TIP: *This pragma or directive does not play a part in DSP multi-operation matching (MULADD, AMA, ADDMUL and corresponding Subtraction operations). In fact the use of BIND_OP for assigning an operator to DSP can prevent the HLS compiler from matching multi-operation expressions. Refer to DSP Multi-Operation Matching for more information.*

Vitis HLS implements the operations in the code using specific implementations. The BIND_OP pragma specifies that for a specific variable, an operation (`mul`, `add`, `div`) should be mapped to a specific device resource for implementation (`impl`) in the RTL. If the BIND_OP pragma is not specified, Vitis HLS automatically determines the resources to use for operations.

For example, to indicate that a specific multiplier operation (`mul`) is implemented in the device fabric rather than a DSP, you can use the BIND_OP pragma.

You can also specify the latency of the operation using the `latency` option.

IMPORTANT! *To use the `latency` option, the operation must have an available multi-stage implementation. The HLS tool provides a multi-stage implementation for all basic arithmetic operations (add, subtract, multiply, and divide), and all floating-point operations.*

Syntax

Place the pragma in the C source within the body of the function where the variable is defined.

```
#pragma HLS bind_op variable=<variable> op=<type>\
impl=<value> latency=<int>
```

Where:

- **`variable=<variable>`:** Defines the variable to assign the BIND_OP pragma to. The variable in this case is one that is assigned the result of the operation that is the target of this pragma.
- **`op=<type>`:** Defines the operation to bind to a specific implementation resource. Supported functional operations include: `mul`, `add`, and `sub` Supported floating point operations include: `fadd`, `fsub`, `fdiv`, `fexp`, `flog`, `fmul`, `frsqrt`, `frecip`, `fsqrt`, `dadd`, `dsub`, `ddiv`, `dexp`, `dlog`, `dmul`, `drsqrt`, `drecip`, `dsqrt`, `hadd`, `hsub`, `hdiv`, `hmul`, and `hsqrt`

  TIP: *Floating point operations include single precision (f), double-precision (d), and half-precision (h).*

- **`impl=<value>`:** Defines the implementation to use for the specified operation. Supported implementations for functional operations include `fabric`, and `dsp`.Supported implementations for floating point operations include: `fabric`, `meddsp`, `fulldsp`, `maxdsp`, and `primitivedsp`.

  ***Note*:** `primitivedsp` is only available on AMD Versal™ devices.

- **`latency=<int>`:** Defines the default latency for the implementation of the operation. The valid latency varies according to the specified `op` and `impl`. The default is -1, which lets Vitis HLS choose the latency.The tables below reflect the supported combinations of operation, implementation, and latency.

Table 38: **Supported Combinations of Functional Operations, Implementation, and Latency**

Operation	Implementation	Min Latency	Max Latency
add	fabric	0	4
add	dsp	0	4
mul	fabric	0	4
mul	dsp	0	4
sub	fabric	0	4
sub	dsp	0	0



TIP: *Comparison operators, such as `dcmp`, are implemented in LUTs and cannot be implemented outside of the fabric, or mapped to DSPs, and so are not configurable with the `config_op` or `bind_op` commands.*

Table 39: **Supported Combinations of Floating Point Operations, Implementation, and Latency**

Operation	Implementation	Min Latency	Max Latency
fadd	fabric	0	13
fadd	fulldsp	0	12
fadd	primitivedsp	0	3
fsub	fabric	0	13
fsub	fulldsp	0	12
fsub	primitivedsp	0	3
fdiv	fabric	0	29
fexp	fabric	0	24
fexp	meddsp	0	21
fexp	fulldsp	0	30
flog	fabric	0	24
flog	meddsp	0	23
flog	fulldsp	0	29
fmul	fabric	0	9
fmul	meddsp	0	9
fmul	fulldsp	0	9
fmul	maxdsp	0	7
fmul	primitivedsp	0	4
fsqrt	fabric	0	29
frsqrt	fabric	0	38
frsqrt	fulldsp	0	33
frecip	fabric	0	37
frecip	fulldsp	0	30
dadd	fabric	0	13
dadd	fulldsp	0	15
dsub	fabric	0	13
dsub	fulldsp	0	15
ddiv	fabric	0	58
dexp	fabric	0	40
dexp	meddsp	0	45
dexp	fulldsp	0	57
dlog	fabric	0	38
dlog	meddsp	0	49
dlog	fulldsp	0	65
dmul	fabric	0	10
dmul	meddsp	0	13
dmul	fulldsp	0	13
dmul	maxdsp	0	14

Table 39: **Supported Combinations of Floating Point Operations, Implementation, and Latency** *(cont'd)*

Operation	Implementation	Min Latency	Max Latency
dsqrt	fabric	0	58
drsqrt	fulldsp	0	111
drecip	fulldsp	0	36
hadd	fabric	0	9
hadd	meddsp	0	12
hadd	fulldsp	0	12
hsub	fabric	0	9
hsub	meddsp	0	12
hsub	fulldsp	0	12
hdiv	fabric	0	16
hmul	fabric	0	7
hmul	fulldsp	0	7
hmul	maxdsp	0	9
hsqrt	fabric	0	16

Example

In the following example, a two-stage pipelined multiplier using fabric logic is specified to implement the multiplication for variable `c` of the function `foo`.

```
int foo (int a, int b) {
int c, d;
#pragma HLS BIND_OP variable=c op=mul impl=fabric latency=2
c = a*b;
d = a*c;
return d;
}
```



TIP: *The HLS tool selects the implementation to use for variable* `d`.

See Also

- set_directive_bind_op
- pragma_HLS_bind_storage

pragma HLS bind_storage

Description

The BIND_STORAGE pragma assigns a variable (array, or function argument) in the code to a specific memory type (`type`) in the RTL. If the pragma is not specified, the Vitis HLS tool determines the memory type to assign. The HLS tool implements the memory using specified implementations (`impl`) in the hardware.

For example, you can use the pragma to specify which memory type to use to implement an array. This lets you control whether the array is implemented as a single or a dual-port RAM for example.

IMPORTANT! *This feature is important for arrays on the top-level function interface, because the memory type associated with the array determines the number and type of ports needed in the RTL, as discussed in Arrays on the Interface. However, for variables assigned to top-level function arguments you must assign the memory type and implementation using the `-storage_type` and `-storage_impl` options of the INTERFACE pragma or directive.*

You can also specify the latency of the implementation. For block RAMs on the interface, the `latency` option lets you model off-chip, non-standard SRAMs at the interface, for example supporting an SRAM with a latency of 2 or 3. For internal operations, the `latency` option allows the memory to be implemented using more pipelined stages. These additional pipeline stages can help resolve timing issues during RTL synthesis.

IMPORTANT! *To use the `latency` option, the operation must have an available multi-stage implementation. The HLS tool provides a multi-stage implementation for all block RAMs.*

Syntax

Place the pragma in the C/C++ source within the body of the function where the variable is defined.

```
#pragma HLS bind_storage variable=<variable> type=<type>\
[ impl=<value> latency=<int> ]
```

Where:

- **`variable=<variable>`:** Defines the variable to assign the BIND_STORAGE pragma to. This is required when specifying the pragma.

  TIP: *If the variable is an argument of a top-level function, then use the `-storage_type` and `-storage_impl` options of the INTERFACE pragma or directive.*

- `type=<type>`: Defines the type of memory to bind to the specified variable. Supported types include: `fifo`, `ram_1p`, `ram_1wnr`, `ram_2p`, `ram_s2p`, `ram_t2p`, `rom_1p`, `rom_2p`, `rom_np`.

Table 40: **Storage Types**

Type	Description
FIFO	A FIFO. Vitis HLS determines how to implement this in the RTL, unless the `-impl` option is specified.
RAM_1P	A single-port RAM. Vitis HLS determines how to implement this in the RTL, unless the `-impl` option is specified.
RAM_1WNR	A RAM with 1 write port and N read ports, using N banks internally.
RAM_2P	A dual-port RAM that allows read operations on one port and both read and write operations on the other port.
RAM_S2P	A dual-port RAM that allows read operations on one port and write operations on the other port.
RAM_T2P	A true dual-port RAM with support for both read and write on both ports.
ROM_1P	A single-port ROM. Vitis HLS determines how to implement this in the RTL, unless the `-impl` option is specified.
ROM_2P	A dual-port ROM.
ROM_NP	A multi-port ROM.



TIP: *If you specify a single port RAM for a PIPO, then the binder will typically allocate a merged-PIPO, with only one bank, and reader and writer accessing different halves of the bank. The info message reported by the HLS compiler says "Implementing PIPO using a single memory for all blocks." This is often cheaper for small PIPOs, where all banks can share a single RAM block. However, if you specify a dual-port RAM for a PIPO to get higher bandwidth and the scheduler uses both ports either in the producer or in the consumer, then the binder will typically allocate a split-PIPO, where the producer will use 1 port and the consumer 2 ports of different banks. The info message reported by the HLS compiler in this case says "Implementing PIPO using a separate memory for each block."*

- `impl=<value>`: Defines the implementation for the specified memory type. Supported implementations include: `bram`, `bram_ecc`, `lutram`, `uram`, `uram_ecc`, `srl`, `memory`, and `auto` as described below.

Table 41: **Supported Implementation**

Name	Description
MEMORY	Generic memory lets the Vivado tool choose the implementation.
URAM	UltraRAM resource
URAM_ECC	UltraRAM with ECC
SRL	Shift Register Logic resource
LUTRAM	Distributed RAM resource
BRAM	Block RAM resource
BRAM_ECC	Block RAM with ECC
AUTO	Vitis HLS automatically determine the implementation of the variable.

Table 42: **Supported Implementations by FIFO/RAM/ROM**

Type	Command/Pragma	Scope	Supported Implementations
FIFO	`bind_storage`[1]	local	**AUTO**, BRAM, LUTRAM, URAM, MEMORY, SRL
FIFO	`config_storage`	global	**AUTO**, BRAM, LUTRAM, URAM, MEMORY, SRL
RAM* | ROM*	`bind_storage`	local	**AUTO** BRAM, BRAM_ECC, LUTRAM, URAM, URAM_ECC
RAM* | ROM*	`config_storage`[2]	global	N/A
RAM_1P	`set_directive_interface s_axilite -storage_impl`	local	**AUTO**, BRAM, URAM
	`config_interface -m_axi_buffer_impl`	global	AUTO, **BRAM**, LUTRAM, URAM

Notes:

1. When no implementation is specified the directive uses AUTOSRL behavior as a default. However, this value cannot be specified.
2. `config_storage` only supports FIFO types.

- **`latency=<int>`:** Defines the default latency for the binding of the type. As shown in the following table, the valid latency varies according to the specified `type` and `impl`. The default is -1, that lets Vitis HLS choose the latency.

Table 43: **Supported Combinations of Memory Type, Implementation, and Latency**

Type	Implementation	Min Latency	Max Latency
FIFO	BRAM	1	4
FIFO	LUTRAM	1	4
FIFO	MEMORY	1	4
FIFO	SRL	1	4
FIFO	URAM	1	4
RAM_1P	AUTO	1	3
RAM_1P	BRAM	1	3
RAM_1P	LUTRAM	1	3
RAM_1P	URAM	1	3
RAM_1WNR	AUTO	1	3
RAM_1WNR	BRAM	1	3
RAM_1WNR	LUTRAM	1	3
RAM_1WNR	URAM	1	3
RAM_2P	AUTO	1	3
RAM_2P	BRAM	1	3
RAM_2P	LUTRAM	1	3
RAM_2P	URAM	1	3
RAM_S2P	BRAM	1	3

Table 43: **Supported Combinations of Memory Type, Implementation, and Latency** *(cont'd)*

Type	Implementation	Min Latency	Max Latency
RAM_S2P	BRAM_ECC	1	3
RAM_S2P	LUTRAM	1	3
RAM_S2P	URAM	1	3
RAM_S2P	URAM_ECC	1	3
RAM_T2P	BRAM	1	3
RAM_T2P	URAM	1	3
ROM_1P	AUTO	1	3
ROM_1P	BRAM	1	3
ROM_1P	LUTRAM	1	3
ROM_2P	AUTO	1	3
ROM_2P	BRAM	1	3
ROM_2P	LUTRAM	1	3
ROM_NP	BRAM	1	3
ROM_NP	LUTRAM	1	3

IMPORTANT! *Any combinations of memory type and implementation that are not listed in the prior table are not supported by* `set_directive_bind_storage`.

Example

The pragma specifies that the variable `coeffs` uses a single port RAM implemented on a BRAM core from the library.

```
#pragma HLS bind_storage variable=coeffs type=RAM_1P impl=bram
```

TIP: *The ports created in the RTL to access the values of* `coeffs` *are defined in the RAM_1P.*

See Also

- set_directive_bind_storage
- pragma HLS bind_op

pragma HLS cache

Description

In FPGA, non-burst accesses to the DDR memory are very expensive and can impact the overall performance of the design. Hence, it is important to devise a scheme that reduces the time needed to access the necessary information. An efficient solution is to re-write the code or use manual burst, but if that does not work then another solution might be to use a cache memory.

Cache provides a temporary storage area in the M_AXI adapter so the design can more quickly retrieve data. The effectiveness of the cache mechanism is measured in a hit/miss ratio and it is based on a property of computer programs called locality of reference, or the tendency of a program to access the same set of memory locations repetitively over a short period of time. It suggests that when a particular memory block is accessed, it is likely to be accessed again in the near future, along with memory located in close proximity. For instance, when executing consecutive instructions in memory, the next set of instructions to be accessed will most likely be within a contiguous block of data nearby.



TIP: *You control the cache implementation resources using the* `syn.interface.m_axi_cache_impl` *command.*

Syntax

```
pragma HLS cache port=<name> lines=<value> depth=<value>
```

Where:

- `port=<name>`: Specifies the port to add cache to.
- `lines=<value>`: Indicates the number of cache lines. The number of lines can be specified as 1, which indicates a single cache line, or a value greater than 1 expressed as a power of 2, indicating multiple cache lines. This is an optional value, and defaults to 1 if not specified.
- `depth=<value>`: Specifies the size of each line in words. The depth must be specified as a power of 2, and indicates the size in words of the pointer datatype for each line. This is an optional value, and defaults to the max burst length if not specified. The max burst length defaults to 16, but can be globally specified by the `syn.interface.m_axi_max_read_burst_length` command as described in Interface Configuration, or by the `syn.directive.interface` command for specific interfaces.

Limitations

The CACHE directive or pragma has the following limitations:

- Cache is only supported for read-only port
- The cache is implemented as a Single port, Single way cache.

- Cache is not supported for multiple ports connected to the same bundle

Example

The following example shows a design where overlapping access will cause the burst to fail. Using the CACHE pragma or directive will improve the performance of the design.

```
extern "C" {
  void dut(
    const double *in,  // Read-Only Vector 1
    double *out,       // Output Result
    int size                 // Size in integer
  )
#pragma HLS INTERFACE m_axi port=in  bundle=aximm depth = 1026
#pragma HLS INTERFACE m_axi port=out bundle=aximm depth = 1024
#pragma HLS cache port=in lines=8 depth=128
for(int i = 0; i < size; i++)
    {
      out[i] = in[i] + in[i + 1];
    }
}
```

See Also

- pragma HLS interface
- Interface Configuration
- syn.directive.cache
- set_directive_cache

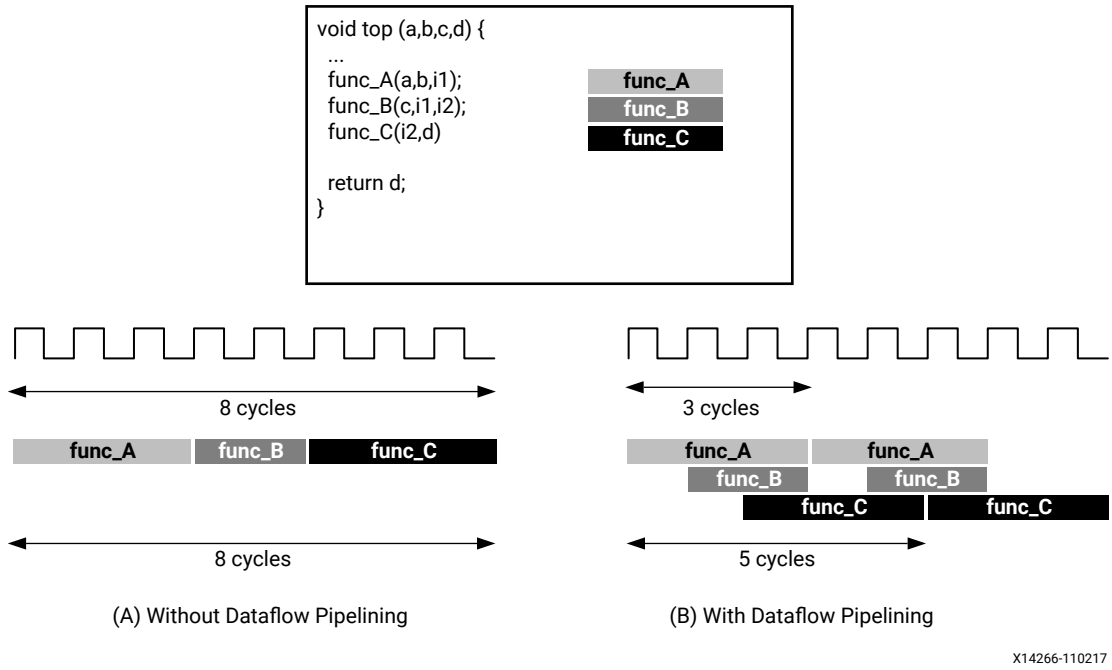
pragma HLS dataflow

Description

The DATAFLOW pragma enables task-level pipelining, allowing functions and loops to overlap in their operation, increasing the concurrency of the RTL implementation and increasing the overall throughput of the design.

All operations are performed sequentially in a C description. In the absence of any directives that limit resources (such as `pragma HLS allocation`), the Vitis HLS tool seeks to minimize latency and improve concurrency. However, data dependencies can limit this. For example, functions or loops that access arrays must finish all read/write accesses to the arrays before they complete. This prevents the next function or loop that consumes the data from starting operation. The DATAFLOW optimization enables the operations in a function or loop to start operation before the previous function or loop completes all its operations.

Figure 142: **DATAFLOW Pragma**



When the DATAFLOW pragma is specified, the HLS tool analyzes the dataflow between sequential functions or loops and creates channels (based on ping pong RAMs or FIFOs) that allow consumer functions or loops to start operation before the producer functions or loops have completed. This allows functions or loops to operate in parallel, which decreases latency and improves the throughput of the RTL.

If no initiation interval (number of cycles between the start of one function or loop and the next) is specified, the HLS tool attempts to minimize the initiation interval and start operation as soon as data is available.

TIP: *The `config_dataflow` command specifies the default memory channel and FIFO depth used in dataflow optimization.*

For the DATAFLOW optimization to work, the data must flow through the design from one task to the next. The following coding styles prevent the HLS tool from performing the DATAFLOW optimization. Refer to Limitations of Control-Driven Task-Level Parallelism for additional details.

- Single-producer-consumer violations
- Feedback between tasks
- Conditional execution of tasks
- Loops with multiple exit conditions

IMPORTANT! *If any of these coding styles are present, the HLS tool issues a message and does not perform DATAFLOW optimization.*

Finally, the DATAFLOW optimization has no hierarchical implementation. If a sub-function or loop contains additional tasks that might benefit from the DATAFLOW optimization, you must apply the optimization to the loop, the sub-function, or inline the sub-function.

Syntax

Place the pragma in the C source within the boundaries of the region, function, or loop.

```
#pragma HLS dataflow [disable_start_propagation]
```

- `disable_start_propagation`: Optionally disables the creation of a start FIFO used to propagate a start token to an internal process. Such FIFOs can sometimes be a bottleneck for performance.

Example

Specifies DATAFLOW optimization within the loop `wr_loop_j`.

```
        wr_loop_j: for (int j = 0; j < TILE_PER_ROW; ++j) {
#pragma HLS DATAFLOW
            wr_buf_loop_m: for (int m = 0; m < TILE_HEIGHT; ++m) {
                wr_buf_loop_n: for (int n = 0; n < TILE_WIDTH; ++n) {
#pragma HLS PIPELINE
                    // should burst TILE_WIDTH in WORD beat
                    outFifo >> tile[m][n];
                }
            }
            wr_loop_m: for (int m = 0; m < TILE_HEIGHT; ++m) {
                wr_loop_n: for (int n = 0; n < TILE_WIDTH; ++n) {
#pragma HLS PIPELINE

outx[TILE_HEIGHT*TILE_PER_ROW*TILE_WIDTH*i+TILE_PER_ROW*TILE_WIDTH*m+TILE_WI
DTH*j+n] = tile[m][n];
                }
            }
```

See Also

- set_directive_dataflow
- config_dataflow
- pragma_HLS_allocation
- pragma_HLS_pipeline

pragma HLS dependence

Description

Vitis HLS detects dependencies within loops: dependencies within the same iteration of a loop are loop-independent dependencies, and dependencies between different iterations of a loop are loop-carried dependencies. The DEPENDENCE pragma allows you to provide additional information to define, negate loop dependencies, and allow loops to be pipelined with lower intervals.

- **Loop-independent dependence:** The same element is accessed in a single loop iteration.

```
for (i=1;i<N;i++) {
 A[i]=x;
 y=A[i];
}
```

- **Loop-carried dependence:** The same element is accessed from a different loop iteration.

```
for (i=1;i<N;i++) {
 A[i]=A[i-1]*2;
}
```

These dependencies impact when operations can be scheduled, especially during function and loop pipelining.

Under some circumstances, such as variable dependent array indexing or when an external requirement needs to be enforced (for example, two inputs are never the same index), the dependence analysis might be too conservative and fail to filter out false dependencies. The DEPENDENCE pragma allows you to explicitly define the dependencies and eliminate a false dependence.

The dependence pragma also applies to function call arguments:

- Inter: dependence between successive calls
- Intra: dependence inside individual call

```
 void foo(int a[3], int x) {
  #pragma HLS pipeline
  #pragma HLS dependence variable=a inter true distance=3
  a[x] += ...;
}

void dut(int a[3], int x, ...) {
  #pragma HLS dataflow
  foo(a, x);
  bar(...);
}
```

> **IMPORTANT!** *Specifying a false dependency, when in fact the dependency is not false, can result in incorrect hardware. Ensure dependencies are correct (true or false) before specifying them.*

Syntax

Place the pragma within the boundaries of the function where the dependence is defined.

```
#pragma HLS dependence variable=<variable> <class> \
<type> <direction> distance=<int> <dependent>
```

Where:

- **`variable=<variable>`:** Optionally specifies the variable to consider for the dependence.

  > **IMPORTANT!** *You cannot specify a `dependence` for function arguments that are bundled with other arguments in an `m_axi` interface. This is the default configuration for `m_axi` interfaces on the function. You also cannot specify a dependence for an element of a struct, unless the struct has been disaggregated.*

- **`class=[array | pointer]`:** Optionally specifies a class of variables in which the dependence needs clarification. Valid values include `array` or `pointer`.

  > **TIP:** *`<class>` and `variable=` should not be specified together as you can specify dependence for a variable, or a class of variables within a function.*

- **`type=[inter | intra]`:** Valid values include `intra` or `inter`. Specifies whether the dependence is:

  - **`intra`:** Dependence within the same loop iteration. When dependence `<type>` is specified as `intra`, and `<dependent>` is false, the HLS tool might move operations freely within a loop, increasing their mobility and potentially improving performance or area. When `<dependent>` is specified as true, the operations must be performed in the order specified.

  - **`inter`:** dependence between different loop iterations. This is the default `<type>`. If dependence <type> is specified as `inter`, and `<dependent>` is false, it allows the HLS tool to perform operations in parallel if the function or loop is pipelined, or the loop is unrolled, or partially unrolled, and prevents such concurrent operation when `<dependent>` is specified as true.

- **`direction=[RAW | WAR | WAW]`:** This is relevant for loop-carry dependencies only, and specifies the direction for a dependence:

  - **`RAW` (Read-After-Write - true dependence):** The write instruction uses a value used by the read instruction.

  - **`WAR` (Write-After-Read - anti dependence):** The read instruction gets a value that is overwritten by the write instruction.

  - `WAW` **(Write-After-Write - output dependence):** Two write instructions write to the same location, in a certain order.

- **`distance=<int>`:** Specifies the inter-iteration distance for array access. Relevant only for loop-carry dependencies where dependence is set to `true`.
- **`<dependent>`:** The `<dependent>` argument should be specified to indicate whether a dependence is `true` and needs to be enforced, or is `false` and should be removed. However, when not specified, the tool will return a warning that the value was not specified and will assume a value of `false`. The accepted values are `true` or `false`.

Example 1

In the following example, the HLS tool does not have any knowledge about the value of `cols` and conservatively assumes that there is always a dependence between the write to `buff_A[1][col]` and the read from `buff_A[1][col]`. In an algorithm such as this, it is unlikely `cols` will ever be zero, but the HLS tool cannot make assumptions about data dependencies. To overcome this deficiency, you can use the `DEPENDENCE` pragma to state that there is no dependence between loop iterations (in this case, for both `buff_A` and `buff_B`).

```
void foo(int rows, int cols, ...){
  for (row = 0; row < rows + 1; row++) {
    for (col = 0; col < cols + 1; col++) {
    #pragma HLS PIPELINE II=1
    #pragma HLS dependence variable=buff_A type=inter false
    #pragma HLS dependence variable=buff_B type=inter false
      if (col < cols) {
         buff_A[2][col] = buff_A[1][col]; // read from buff_A[1][col]
         buff_A[1][col] = buff_A[0][col]; // write to buff_A[1][col]
         buff_B[1][col] = buff_B[0][col];
      }
    }
  }
}
```

Example 2

Removes the dependence between `Var1` in the same iterations of `loop_1` in function `func`.

```
#pragma HLS dependence variable=Var1 type=intra false
```

Example 3

Defines the dependence on all arrays in `loop_2` of function `func` to inform the HLS tool that all reads must happen after writes (RAW) in the same loop iteration.

```
#pragma HLS dependence class=array type=intra direction=RAW true
```

See Also

- set_directive_dependence

- pragma_HLS_disaggregate
- pragma_HLS_pipeline

pragma HLS disaggregate

Description

The DISAGGREGATE pragma lets you deconstruct a `struct` variable into its individual elements. The number and type of elements created are determined by the contents of the struct itself.

IMPORTANT! *Structs used as arguments to the top-level function are aggregated by default, but can be disaggregated with this directive or pragma. Refer to AXI4-Stream Interfaces for important information about disaggregating structs associated with streams.*

Syntax

Place the pragma in the C source within the boundaries of the region, function, or loop.

```
#pragma HLS disaggregate variable=<variable>
```

Options

Where:

- `variable=<variable>`: Specifies the struct variable to disaggregate.

Example 1

The following example shows the struct variable `a` in function `top` will be disaggregated:

```
#pragma HLS disaggregate variable=a
```

Example 2

Disaggregated structs can be addressed in your code by the using standard C/C++ coding style as shown below. Notice the different methods for accessing the pointer element (a) versus the reference element (c);

```
struct SS
{
  int x[N];
  int y[N];
};

int top(SS *a, int b[4][6], SS &c) {
#pragma HLS disaggregate variable = a
```

```
#pragma HLS interface s_axilite port = a->x
#pragma HLS interface s_axilite port = a->y

// Following is now supported
#pragma HLS disaggregate variable = c
#pragma HLS interface ap_memory port = c.x
#pragma HLS interface ap_memory port = c.y
```

Example 3

The following example shows the `Dot` struct containing the `RGB` struct as an element. If you apply the DISAGGREGATE pragma to variable `Arr`, then only the top-level `Dot` struct is disaggregated.

```
struct Pixel {
char R;
char G;
char B;
};

struct Dot {
Pixel RGB;
unsigned Size;
};

#define N 1086
void DUT(Dot Arr[N]) {
#pragma HLS disaggregate variable=Arr
...
}
```

If you want to disaggregate the whole struct, `Dot` and `RGB`, then you can assign the DISAGGREGATE pragma as shown below.

```
void DUT(Dot Arr[N]) {
#pragma HLS disaggregate variable=Arr->RGB
...
}
```

The results in this case will be:

```
void DUT(char Arr_RGB_R[N], char Arr_RGB_G[N], char Arr_RGB_B[N], unsigned
Arr_Size[N]) {
#pragma HLS disaggregate variable=Arr->RGB
...
}
```

See Also

- set_directive_disaggregate
- pragma_HLS_aggregate

pragma HLS expression_balance

Description

Sometimes C/C++ code is written with a sequence of operations, resulting in a long chain of operations in RTL. With a small clock period, this can increase the latency in the design. By default, the Vitis HLS tool rearranges the operations using associative and commutative properties. This rearrangement creates a balanced tree that can shorten the chain, potentially reducing latency in the design at the cost of extra hardware.

Expression balancing rearranges operators to construct a balanced tree and reduce latency.

- For integer operations expression balancing is on by default but may be disabled.
- For floating-point operations, expression balancing is off by default but may be enabled.

The EXPRESSION_BALANCE pragma allows this expression balancing to be disabled, or to be expressly enabled, within a specified scope.

Syntax

Place the pragma in the C source within the boundaries of the required location.

```
#pragma HLS expression_balance off
```

Where:

- `off`: Turns off expression balancing at this location.Specifying `#pragma HLS expression_balance` enables expression balancing in the specified scope. Adding `off` disables it.

Example 1

Disables expression balancing within function `my_Func`.

```
void my_func(char inval, char incr) {
  #pragma HLS expression_balance off
```

Example 2

This example explicitly enables expression balancing in function `my_Func`.

```
void my_func(char inval, char incr) {
  #pragma HLS expression_balance
```

See Also

- set_directive_expression_balance

pragma HLS function_instantiate

Description

The FUNCTION_INSTANTIATE pragma is an optimization technique that has the area benefits of maintaining the function hierarchy but provides an additional powerful option: performing targeted local optimizations on specific instances of a function. This can simplify the control logic around the function call and potentially improve latency and throughput.

By default:

- Functions remain as separate hierarchy blocks in the RTL, or is decomposed (or inlined) into a higher level function.
- All instances of a function, at the same level of hierarchy, make use of a single RTL implementation (block).

The FUNCTION_INSTANTIATE pragma is used to create a unique RTL implementation for each instance of a function, allowing each instance to be locally optimized according to the function call. This pragma exploits the fact that some inputs to a function can be a constant value when the function is called, and uses this to both simplify the surrounding control structures and produce smaller more optimized function blocks.

Without the FUNCTION_INSTANTIATE pragma, the following code results in a single RTL implementation of function `func_sub` for all three instances of the function in `func`. Each instance of function `func_sub` is implemented in an identical manner. This is fine for function reuse and reducing the area required for each instance call of a function, but means that the control logic inside the function must be more complex to account for the variation in each call of `func_sub`.

```
char func_sub(char inval, char incr) {
#pragma HLS INLINE OFF
#pragma HLS FUNCTION_INSTANTIATE variable=incr
 return inval + incr;
}
void func(char inval1, char inval2, char inval3,
 char *outval1, char *outval2, char * outval3)
{
 *outval1 = func_sub(inval1, 1);
 *outval2 = func_sub(inval2, 2);
 *outval3 = func_sub(inval3, 3);
}
```



TIP: *The Vitis HLS tool automatically decomposes (or inlines) small functions into higher-level calling functions. This is true even for function instantiations. Using the INLINE pragma with the `OFF` option can be used to prevent this automatic inlining.*

In the code sample above, the FUNCTION_INSTANTIATE pragma results in three different implementations of function `func_sub`, each optimized for the specified values of `incr`, reducing the area and improving the performance of the function implementation.

Syntax

Place the pragma in the C source within the boundaries of the required location.

```
#pragma HLS FUNCTION_INSTANTIATE variable=<variable>
```

Where:

- **`variable=<variable>`:** A required argument that defines the function argument to use as a constant.

Examples

In the following example, the `FUNCTION_INSTANTIATE` pragma, placed in function `swInt`, allows each instance of function `swInt` to be independently optimized with respect to the `maxv` function argument.

```
void swInt(unsigned int *readRefPacked, short *maxr, short *maxc, short
*maxv){
#pragma HLS FUNCTION_INSTANTIATE variable=maxv
    uint2_t d2bit[MAXCOL];
    uint2_t q2bit[MAXROW];
#pragma HLS array partition variable=d2bit,q2bit cyclic factor=FACTOR

    intTo2bit<MAXCOL/16>((readRefPacked + MAXROW/16), d2bit);
    intTo2bit<MAXROW/16>(readRefPacked, q2bit);
    sw(d2bit, q2bit, maxr, maxc, maxv);
}
```

See Also

- set_directive_function_instantiate
- pragma HLS allocation
- pragma HLS inline

pragma HLS inline

Description

Removes a function as a separate entity in the hierarchy. After inlining, the function is dissolved into the calling function and no longer appears as a separate level of hierarchy in the RTL.

IMPORTANT! *Inlining a function also dissolves any pragmas or directives applied to that function. In Vitis HLS, any pragmas or directives applied to the inlined function are ignored.*

In some cases, inlining a function allows operations within the function to be shared and optimized more effectively with the calling function. However, an inlined function cannot be shared or reused, so if the parent function calls the inlined function multiple times, this can increase the area required for implementing the RTL.

The INLINE pragma applies differently to the scope it is defined in depending on how it is specified:

- **`INLINE`:** Without arguments, the pragma means that the function it is specified in should be inlined upward into any calling functions.
- **`INLINE off`:** Specifies that the function it is specified in should *not* be inlined upward into any calling functions. This disables the inline of a specific function that can be automatically inlined or inlined as part of recursion.
- **`INLINE recursive`:** Applies the pragma to the body of the function it is assigned in. It applies downward, recursively inlining the contents of the function.

By default, inlining is only performed on the next level of function hierarchy, not sub-functions. However, the `recursive` option lets you specify inlining through levels of the hierarchy.

Syntax

Place the pragma in the C source within the body of the function or region of code.

```
#pragma HLS inline <recursive | off>
```

Where:

- **`recursive`:** By default, only one level of function inlining is performed, and functions within the specified function are not inlined. The `recursive` option inlines all functions recursively within the specified function or region.

  IMPORTANT! *Recursive inlining of functions in a dataflow region can result in a single function in the region which does not meet the conditions of dataflow. In this case the `INLINE recursive` pragma or directive is ignored.*

- **`off`:** Disables function inlining to prevent specified functions from being inlined. For example, if `recursive` is specified in a function, this option can prevent a particular called function from being inlined when all others are.

  TIP: *The Vitis HLS tool automatically inlines small functions, and using the INLINE pragma with the `off` option can be used to prevent this automatic inlining.*

Example 1

The following example inlines all functions within the body of `func_top` inlining recursively down through the function hierarchy, except function `func_sub` is not inlined. The recursive pragma is placed in function `func_top`. The pragma to disable inlining is placed in the function `func_sub`:

```
func_sub (p, q) {
#pragma HLS inline off
int q1 = q + 10;
func(p1,q);// foo_3
...
}
void func_top { a, b, c, d} {
  #pragma HLS inline recursive
  ...
  func(a,b);//func_1
  func(a,c);//func_2
  func_sub(a,d);
  ...
}
```

TIP: *Notice in this example that INLINE RECURSIVE applies downward to the contents of function `func_top`, but INLINE OFF applies to `func_sub` directly.*

Example 2

This example inlines the `copy_output` function into any functions or regions calling `copy_output`.

```
void copy_output(int *out, int out_lcl[OSize * OSize], int output) {
#pragma HLS INLINE
    // Calculate each work_item's result update location
    int stride = output * OSize * OSize;

    // Work_item updates output filter/image in DDR
    writeOut: for(int itr = 0; itr < OSize * OSize; itr++) {
    #pragma HLS PIPELINE
        out[stride + itr] = out_lcl[itr];
    }
```

See Also

- set_directive_inline
- pragma_HLS_allocation

pragma HLS interface

Description

The INTERFACE pragma or directive is only supported for use on the top-level function, and cannot be used for sub-functions of the HLS component. It specifies how RTL ports are created from the function arguments during interface synthesis, as described in Defining Interfaces. The Vitis HLS tool automatically determines the I/O protocol used by any sub-functions.

Ports in the RTL implementation are derived from the data type and direction of the arguments of the top-level function and function return, the `flow_target` for the HLS component, the default interface configuration settings as specified by config_interface, and by the INTERFACE pragma or directive. Each function argument can be specified to have its own I/O protocol (such as valid handshake or acknowledge handshake).



TIP: *Global variables required on the interface must be explicitly defined as an argument of the top-level function as described in Global Variables. If a global variable is accessed, but all read and write operations are local to the design, the resource is created in the design. There is no need for an I/O port in the RTL.*

The interface also defines the execution control protocol of the HLS component as described in Block-Level Control Protocols. The control protocol controls when the HLS component (or block) starts execution, and when the block completes operation, is idle and ready for new inputs.

Syntax

Place the pragma within the boundaries of the function.

```
#pragma HLS interface mode=<mode> port=<name> direct_io=<value>[OPTIONS]
```

Where:

- `mode=<mode>`:

  The supported modes, and how the tool implements them in RTL, can be broken down into three categories as follows:

  1. Port-Level Protocols:
     - `ap_none`: No port protocol. The interface is a simple data port.
     - `ap_vld`: Implements the data port with an associated `valid` signal to indicate when the data is valid for reading or writing.
     - `ap_ack`: Implements the data port with an associated `acknowledge` signal to acknowledge that the data was read or written.

- `ap_hs`: Implements the data port with both `valid` and `acknowledge` signals to provide a two-way handshake to indicate when the data is valid for reading and writing and to acknowledge that the data was read or written.
- `ap_ovld`: Implements the output data port with an associated `valid` signal to indicate when the data is valid for reading or writing.

  TIP: *For `ap_ovld` Vitis HLS implements the input argument or the input half of any read/write arguments with mode `ap_none`.*

- `direct_io=<true/false>`: Specify true for function argument which the INTERFACE applies to. See the Mapping Direct I/O Streams to SAXI Lite Interface section for the use case of this option.
- `ap_memory`: Implements array arguments as a standard RAM interface. If you use the RTL design in Vivado IP integrator the interface is composed of separate ports.
- `bram`: Implements array arguments as a standard RAM interface. If you use the RTL design in Vivado IP integrator the memory interface is composed of a single port.
- `ap_fifo`: Implements the port with a standard FIFO interface using data input and output ports with associated active-Low FIFO `empty` and `full` ports.

  ***Note*:** You can only use the `ap_fifo` interface on read arguments or write arguments. `ap_fifo` mode does not support bidirectional read/write arguments.

2. AXI Interface Protocols:
   - `s_axilite`: Implements the port as an AXI4-Lite interface. The tool produces an associated set of C driver files when exporting the generated RT for the HLS component.
   - `m_axi`: Implements the port as an AXI4 interface. You can use the `config_interface -m_axi_addr64` command to specify either 32-bit (default) or 64-bit address ports and to control any address offset.
   - `axis`: Implements the port as an AXI4-Stream interface.
3. Block-Level Control Protocols:
   - `ap_ctrl_chain`: Implements a set of block-level control ports to `start` the design operation, `continue` operation, and indicate when the design is `idle`, `done`, and `ready` for new input data.
   - `ap_ctrl_hs`: Implements a set of block-level control ports to `start` the design operation and to indicate when the design is `idle`, `done`, and `ready` for new input data.
   - `ap_ctrl_none`: No block-level I/O protocol.

     ***Note*:** Using the `ap_ctrl_none` mode might prevent the design from being verified using C/RTL co-simulation.

- `port=<name>`: Specifies the name of the function argument or function return which the INTERFACE pragma applies to.

> **TIP:** *Block-level I/O protocols (`ap_ctrl_none`, `ap_ctrl_hs`, or `ap_ctrl_chain`) can be assigned to a port for the function `return` value.*

Options

> **TIP:** *Many of the options specified below have global values that are defined in the config_interface command. Set local values for the interface defined here to override the global values.*

- **`bundle=<string>`:** By default, the HLS tool groups or bundles function arguments with compatible options into interface ports in the RTL code. All AXI4-Lite (`s_axilite`) interfaces are bundled into a single AXI4-Lite port whenever possible. Similarly, all function arguments specified as an AXI4 (`m_axi`) interface are bundled into a single AXI4 port by default. All interface ports with compatible options, such as `mode`, `offset`, and `bundle`, are grouped into a single interface port. The port name is derived automatically from a combination of the mode and bundle, or is named as specified by `-name`.

  > **IMPORTANT!** *When specifying the `bundle` name you should use all lower-case characters.*

- **`channel=<string>`:** To enable multiple channels on an `m_axi` interface specify the channel ID. Multiple `m_axi` interfaces can be combined into a single `m_axi` adapter using separate channel IDs.

- **`clock=<name>`:** By default, the AXI4-Lite interface clock is the same clock as the system clock. This option is used to set specify a separate clock for an AXI4-Lite interface. If the `-bundle` option is used to group multiple top-level function arguments into a single AXI4-Lite interface, the clock option need only be specified on one of bundle members.

- **`depth=<int>`:** Specifies the maximum number of samples for the test bench to process. This setting indicates the maximum size of the FIFO needed in the verification adapter that the HLS tool creates for RTL co-simulation.

  > **TIP:** *While `depth` is usually an option, it is needed to specify the size of pointer arguments for RTL co-simulation.*

- **`interrupt=<int>`:** Only used by `ap_vld/ap_hs`. This option enables the I/O to be managed in interrupt, by creating the corresponding bits in the `ISR` and `IER` in the `s_axilite` register file. The integer value N=16..31 specifies the bit position in both registers (by default assigned contiguously from 16).

- **`latency=<value>`:** This option can be used on `ap_memory` and M_AXI interfaces.

  - In an `ap_memory` interface, the interface option specifies the read latency of the RAM resource driving the interface. By default, a read operation of 1 clock cycle is used. This option allows an external RAM with more than 1 clock cycle of read latency to be modeled.

  - In an M_AXI interface, this option specifies the expected latency of the AXI4 interface, allowing the design to initiate a bus request <value> number of cycles (latency) before the read or write is expected. If this figure it too low, the design will be ready too soon and might stall waiting for the bus. If this figure is too high, bus access might be idle waiting on the design to start the access.

- **`max_read_burst_length=<int>`:** For use with the M_AXI interface, this option specifies the maximum number of data values read during a burst transfer. Refer to AXI Burst Transfers for more information.

- **`max_write_burst_length=<int>`:**

  - For AXI4 (`m_axi`) interfaces, this option specifies the maximum number of data values written during a burst transfer.

    TIP: *If the port is a read-only port, then set the `num_write_outstanding=1` and `max_write_burst_length=2` to conserve memory resources. For write-only ports, set the `num_read_outstanding=1` and `max_read_burst_length=2`.*

- **`max_widen_bitwidth=<int>`:** Specifies the maximum bit width available for the interface when automatically widening the interface. This overrides the global value specified by the `config_interface -m_axi_max_bitwidth` command.

- **`name=<string>`:** Specifies a name for the port which will be used in the generated RTL.

- **`num_read_outstanding=<int>`:** For AXI4 (`m_axi`) interfaces, this option specifies how many read requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, a FIFO of size:

  ```
  num_read_outstanding*max_read_burst_length*word_size
  ```

- **`num_write_outstanding=<int>`:** For AXI4 (`m_axi`) interfaces, this option specifies how many write requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, a FIFO of size:

  ```
  num_write_outstanding*max_write_burst_length*word_size
  ```

- **`offset=<string>`:** Controls the address offset in AXI4-Lite (`s_axilite`) and AXI4 memory mapped (`m_axi`) interfaces for the specified port.

  - In an `s_axilite` interface, `<string>` specifies the address in the register map.
  - In an `m_axi` interface this option overrides the global option specified by the `config_interface -m_axi_offset` option, and `<string>` is specified as:
    - `off`: Do not generate an offset port.
    - `direct`: Generate a scalar input offset port.
    - `slave`: Generate an offset port and automatically map it to an AXI4-Lite slave interface. This is the default offset.

- **register:** An optional keyword to register the signal and any relevant protocol signals, and causes the signals to persist until at least the last cycle of the function execution. The `-register_io` option of the `config_interface` command globally controls registering all inputs/outputs on the top function. This option applies to the following interface modes:
  - `s_axilite`
  - `ap_fifo`
  - `ap_none`
  - `ap_hs`
  - `ap_ack`
  - `ap_vld`
  - `ap_ovld`

  TIP: *The use of the register option on the return port of the function (`port=return`) is not supported. Instead use the LATENCY pragma or directive:*

  ```
  #pragma HLS LATENCY min=1 max=1
  ```

- **register_mode=<forward|reverse|both|off>:** This option applies to AXI4-Stream interfaces, and specifies if registers are placed on the forward path (`TDATA` and `TVALID`), the reverse path (`TREADY`), on both paths (`TDATA`, `TVALID`, and `TREADY`), or if none of the ports signals are to be registered (`off`). The default is `both`. AXI4-Stream side-channel signals are considered to be data signals and are registered whenever the `TDATA` is registered.
- **storage_impl=<impl>:** For use with `s_axilite` only. This options defines a storage implementation to assign to the interface. Supported implementation values include `auto`, `bram`, and `uram`. The default is **auto**.

  TIP: *`uram` is a synchronous memory with only a single clock for two ports. Therefore `uram` cannot be specified for an `s_axilite` adapter with a second clock.*

- **storage_type=<value>:** For use with the `ap_memory` interface only. This option specifies a storage type (that is, RAM_T2P) to assign to the variable.Supported types include: `ram_1p`, `ram_1wnr`, `ram_2p`, `ram_s2p`, `ram_t2p`, `rom_1p`, `rom_2p`, and `rom_np`.

  TIP: *This can also be specified using the BIND_STORAGE pragma or directive for an object not on the interface.*

Example 1

In this example, both function arguments are implemented using an AXI4-Stream interface:

```
void example(int A[50], int B[50]) {
  //Set the HLS native interface types
  #pragma HLS INTERFACE mode=axis port=A
  #pragma HLS INTERFACE mode=axis port=B
  int i;
  for(i = 0; i < 50; i++){
    B[i] = A[i] + 5;
  }
}
```

Example 2

The following turns off block-level I/O protocols, and is assigned to the function return value:

```
#pragma HLS interface mode=ap_ctrl_none port=return
```

The function argument `InData` is specified to use the `ap_vld` interface and also indicates the input should be registered:

```
#pragma HLS interface mode=ap_vld register port=InData
```

Example 3

This example defines the INTERFACE standards for the ports of the top-level `transpose` function. Notice the use of the `bundle=` option to group signals.

```
// TOP LEVEL - TRANSPOSE
void transpose(int* input, int* output) {
    #pragma HLS INTERFACE mode=m_axi port=input offset=slave bundle=gmem0
    #pragma HLS INTERFACE mode=m_axi port=output offset=slave bundle=gmem1

    #pragma HLS INTERFACE mode=s_axilite port=input bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=output bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=return bundle=control

    #pragma HLS dataflow
```

See Also

- set_directive_interface
- pragma_HLS_bind_storage

pragma HLS latency

Description

Specifies a minimum or maximum latency value, or both, for the completion of functions, loops, and regions.

- **Latency:** Number of clock cycles required to produce an output.
- **Function latency:** Number of clock cycles required for the function to compute all output values, and return.
- **Loop latency:** Number of cycles to execute all iterations of the loop.

Vitis HLS always tries to minimize latency in the design. When the LATENCY pragma is specified, the tool behavior is as follows:

- Latency is greater than the minimum, or less than the maximum: The constraint is satisfied. No further optimizations are performed.
- Latency is less than the minimum: If the HLS tool can achieve less than the minimum specified latency, it extends the latency to the specified value, potentially enabling increased sharing.
- Latency is greater than the maximum: If the HLS tool cannot schedule within the maximum limit, it increases effort to achieve the specified constraint. If it still fails to meet the maximum latency, it issues a warning, and produces a design with the smallest achievable latency in excess of the maximum.



TIP: *You can also use the LATENCY pragma to limit the efforts of the tool to find an optimum solution. Specifying latency constraints for scopes within the code: loops, functions, or regions, reduces the possible solutions within that scope and can improve tool runtime. Refer to Improving Synthesis Runtime and Capacity for more information.*

If the intention is to limit the total latency of all loop iterations, the latency directive should be applied to a region that encompasses the entire loop, as in this example:

```
Region_All_Loop_A: {
#pragma HLS latency max=10
Loop_A: for (i=0; i<N; i++)
  {
  ..Loop Body...
  }
}
```

In this case, even if the loop is unrolled, the latency directive sets a maximum limit on all loop operations.

If Vitis HLS cannot meet a maximum latency constraint it relaxes the latency constraint and tries to achieve the best possible result.

If a minimum latency constraint is set and Vitis HLS can produce a design with a lower latency than the minimum required it inserts dummy clock cycles to meet the minimum latency.

Syntax

Place the pragma within the boundary of a function, loop, or region of code where the latency must be managed.

```
#pragma HLS latency min=<int> max=<int>
```

Where:

- **`min=<int>`:** Optionally specifies the minimum latency for the function, loop, or region of code.
- **`max=<int>`:** Optionally specifies the maximum latency for the function, loop, or region of code.

***Note*:** Although both min and max are described as optional, at least one must be specified.

Example 1

Function `foo` is specified to have a minimum latency of 4 and a maximum latency of 8.

```
int foo(char x, char a, char b, char c) {
  #pragma HLS latency min=4 max=8
  char y;
  y = x*a+b+c;
  return y
}
```

Example 2

In the following example, `loop_1` is specified to have a maximum latency of 12. Place the pragma in the loop body as shown.

```
void foo (num_samples, ...) {
  int i;
  ...
  loop_1: for(i=0;i< num_samples;i++) {
  #pragma HLS latency max=12
    ...
    result = a + b;
  }
}
```

Example 3

The following example creates a code region and groups signals that need to change in the same clock cycle by specifying zero latency.

```
// create a region { } with a latency = 0
{
  #pragma HLS LATENCY max=0 min=0
  *data = 0xFF;
  *data_vld = 1;
}
```

See Also

- set_directive_latency

pragma HLS loop_flatten

Description

Allows nested loops to be flattened into a single loop hierarchy with improved latency.

In the RTL implementation, it requires one clock cycle to move from an outer loop to an inner loop, and from an inner loop to an outer loop. Flattening nested loops allows them to be optimized as a single loop. This saves clock cycles, potentially allowing for greater optimization of the loop body logic.

Apply the LOOP_FLATTEN pragma to the loop body of the inner-most loop in the loop hierarchy. Only perfect and semi-perfect loops can be flattened in this manner:

- **Perfect loop nests:**
  - Only the innermost loop has loop body content.
  - There is no logic specified between the loop statements.
  - All loop bounds are constant.
- **Semi-perfect loop nests:**
  - Only the innermost loop has loop body content.
  - There is no logic specified between the loop statements.
  - The outermost loop bound can be a variable.
- **Imperfect loop nests:** When the inner loop has variable bounds (or the loop body is not exclusively inside the inner loop), try to restructure the code, or unroll the loops in the loop body to create a perfect loop nest.

Syntax

Place the pragma in the C source within the boundaries of the nested loop.

```
#pragma HLS loop_flatten off
```

Where:

- `off`: Optional keyword. Prevents flattening from taking place, and can prevent some loops from being flattened while all others in the specified location are flattened.



IMPORTANT! *The presence of the LOOP_FLATTEN pragma or directive enables the optimization. The addition of `off` disables it.*

Example 1

Flattens `loop_1` in function `foo` and all (perfect or semi-perfect) loops above it in the loop hierarchy, into a single loop. Place the pragma in the body of `loop_1`.

```
void foo (num_samples, ...) {
  int i;
  ...
  loop_1: for(i=0;i< num_samples;i++) {
   #pragma HLS loop_flatten
   ...
    result = a + b;
  }
}
```

Example 2

Prevents loop flattening in `loop_1`.

```
loop_1: for(i=0;i< num_samples;i++) {
   #pragma HLS loop_flatten off
   ...
```

See Also

- set_directive_loop_flatten
- pragma_HLS_loop_merge
- pragma_HLS_unroll

pragma HLS loop_merge

Description

Merges consecutive loops into a single loop to reduce overall latency, increase sharing, and improve logic optimization. Merging loops:

- Reduces the number of clock cycles required in the RTL to transition between the loop-body implementations.
- Allows the loops be implemented in parallel (if possible).

The LOOP_MERGE pragma will seek to merge all loops within the scope it is placed. For example, if you apply a LOOP_MERGE pragma in the body of a loop, the Vitis HLS tool applies the pragma to any sub-loops within the loop but not to the loop itself.

The rules for merging loops are:

- If the loop bounds are variables, they must have the same value (number of iterations).
- If the loop bounds are constants, the maximum constant value is used as the bound of the merged loop.
- Loops with both variable bounds and constant bounds cannot be merged.
- The code between loops to be merged cannot have side effects. Multiple execution of this code should generate the same results (a=b is allowed, a=a+1 is not).
- Loops cannot be merged when they contain FIFO reads. Merging changes the order of the reads. Reads from a FIFO or FIFO interface must always be in sequence.

Syntax

Place the pragma in the C source within the required scope or region of code.

```
#pragma HLS loop_merge force
```

Where:

- `force`: Optional keyword to force loops to be merged even when the HLS tool issues a warning.



IMPORTANT! *In this case, you must manually ensure that the merged loop will function correctly.*

Examples

Merges all consecutive loops in function `foo` into a single loop.

```
void foo (num_samples, ...) {
  #pragma HLS loop_merge
  int i;
  ...
  loop_1: for(i=0;i< num_samples;i++) {
   ...
```

All loops inside `loop_2` (but not `loop_2` itself) are merged by using the `force` option. Place the pragma in the body of `loop_2`.

```
loop_2: for(i=0;i< num_samples;i++) {
#pragma HLS loop_merge force
...
```

See Also

- set_directive_loop_merge
- pragma_HLS_loop_flatten
- pragma_HLS_unroll

pragma HLS loop_tripcount

Description

When manually applied to a loop, specifies the total number of iterations performed by a loop.

IMPORTANT! *The LOOP_TRIPCOUNT pragma or directive is for analysis only, and does not impact the results of synthesis.*

The Vitis HLS tool reports the total latency of each loop, which is the number of clock cycles to execute all iterations of the loop. Therefore, the loop latency is a function of the number of loop iterations, or tripcount.

The tripcount can be a constant value. It can depend on the value of variables used in the loop expression (for example, x < y), or depend on control statements used inside the loop. In some cases, the HLS tool cannot determine the tripcount, and the latency is unknown. This includes cases in which the variables used to determine the tripcount are:

- Input arguments or
- Variables calculated by dynamic operation.

In the following example, the maximum iteration of the for-loop is determined by the value of input `num_samples`. The value of `num_samples` is not defined in the C function, but comes into the function from the outside.

```
void foo (num_samples, ...) {
   int i;
   ...
   loop_1: for(i=0;i< num_samples;i++) {
     ...
     result = a + b;
   }
}
```

In cases where the loop latency is unknown or cannot be calculated, the LOOP_TRIPCOUNT pragma lets you specify minimum, maximum, and average iterations for a loop. This lets the tool analyze how the loop latency contributes to the total design latency in the reports, and helps you determine appropriate optimizations for the design.

TIP: *If a C assert macro is used in to limit the size of a loop variable Vitis HLS can use it to both define loop limits for reporting, and create hardware that is exactly sized to these limits.*

Syntax

Place the pragma in the C source within the body of the loop.

```
#pragma HLS loop_tripcount min=<int> max=<int> avg=<int>
```

Where:

- **`max= <int>`:** Specifies the maximum number of loop iterations.
- **`min=<int>`:** Specifies the minimum number of loop iterations.
- **`avg=<int>`:** Specifies the average number of loop iterations.

Examples

In the following example, `loop_1` in function `foo` is specified to have a minimum tripcount of 12, and a maximum tripcount of 16:

```
void foo (num_samples, ...) {
  int i;
  ...
  loop_1: for(i=0;i< num_samples;i++) {
   #pragma HLS loop_tripcount min=12 max=16
   ...
    result = a + b;
  }
}
```

See Also

- set_directive_loop_tripcount

pragma HLS occurrence

Description

The OCCURRENCE pragma or directive can be applied to a function call within a region. The feature works with pipelined function calls within a conditional block such as an `if` statement or `switch` statement. The pragma specifies how often the pipelined function is called inside the current pipeline. Identifying a function within a condition block with the OCCURRENCE pragma allows the functions and loops in that region to be pipelined with a higher initiation interval that is slower than the enclosing function or loop.

When pipelining functions or loops, the OCCURRENCE pragma specifies that a pipelined function call within the pipelined function or loop is executed less frequently than the enclosing function or loop. This allows the pipelined function call that is executed less often to be pipelined at a slower rate, thus potentially improving the resource sharing potential within the top-level pipeline.

Use the following process to determine the OCCURRENCE cycle value:

- A loop iterates `<N>` times.
- However, part of the loop body is enabled by a conditional statement, and as a result only executes `<M>` times, where `<N>` is an integer multiple of `<M>`.
- The conditional code has an occurrence that is `N/M` times slower than the rest of the loop body.

For example, in a loop that executes 10 times, a conditional statement within the loop that only executes two times has an occurrence of 5 (or 10/2).

Syntax

Place the pragma in the C source within a conditional block of code that contains the pipelined sub-function call(s).

```
#pragma HLS occurrence cycle=<int> off=true
```

Where:

- **`cycle=<int>`:** Specifies the occurrence value `N/M`, where `<N>` is the number of times the enclosing function or loop executes, and `<M>` is the number of times the conditional region executes.

IMPORTANT! *<N> must be an integer multiple of <M>.*

- `off=true`: Disable occurrence for the specified function.

Examples

In this example, the call of sub-function within the if statement (but not the memory read and write, because they are not inside a pipelined function) has an occurrence of 4 (it executes at a rate four times less often than the surrounding code that contains it). Hence, while without the OCCURRENCE pragma it would be pipelined with the same II as the caller, with the OCCURRENCE pragma it will be pipelined with an II=4. This will expose more resource sharing opportunities within it and with other functions.

```
void subfunction(...) {
#pragma HLS pipeline II=...
// Without the occurrence pragma,
// this will be automatically pipelined with an II=1,
// regardless of its own pipeline pragma,
// since it is called in a pipeline with II=1
// With the pragma, it has an II=4.
  ...
}
for (int i = 0; i < 16; i++) {
#pragma HLS pipeline II=1
  if (i % 4 == 0) {
#pragma HLS occurrence cycle=4
    subfunction(...);
    a[i+1] = b[i];
  }
}
```

See Also

- set_directive_occurrence
- pragma_HLS_pipeline

pragma HLS performance

Description

TIP: *The PERFORMANCE pragma or directive applies to loops and loop nests, and requires a known loop tripcount to determine the performance. If your loop has a variable tripcount then you must also specify LOOP_TRIPCOUNT pragma or directive.*

The PERFORMANCE pragma or directive lets you specify a high-level constraint, `target_ti` or `target_tl`, defining the number of clock cycles between successive starts of a loop, and lets the tool infer lower-level UNROLL, PIPELINE, ARRAY_PARTITION, and INLINE directives needed to achieve the desired result. The PERFORMANCE pragma or directive does not guarantee the specified value will be achieved, and so it is only a target.

The `target_ti` is the interval between successive starts of the loop, or between the start of the first iteration of the loop, and the next start of the first iteration of the loop. In the following code example, a `target_ti=T` would mean the target interval for the start of loop `L2` between two consecutive iterations of `L1` should be 100 cycles.

```
const int T = 100;
L1: for (int i=0; i<N; i++)
  L2: for (int j=0; j<M; j++){
pragma HLS performance target_ti=T
   ...
   }
```

The `target_tl` is the interval between start of the loop and end of the loop, or between the start of the first iteration of the loop and the completion of the last iteration of the loop. For example, in the preceding code example a `target_tl=T` means the target completion of loop `L2` for a single iteration of `L1` should be 100 cycles.

***Note*:** The INLINE pragma is applied automatically to functions inside any pipelined loop that has II=1 to improve throughput. If you apply the PERFORMANCE pragma or directive that infers a pipeline with II=1, it will also trigger the auto-inline optimization. You can disable this for specific functions by using `#pragma HLS INLINE off`.

The transaction interval is the initiation interval (II) of the loop times the number of iterations, or tripcount: target_ti = II * loop tripcount. Conversely, `target_ti` = FreqHz / Operations per second.

For example, assuming an image processing function that processes a single frame per invocation with a throughput goal of 60 fps, then the target throughput for the function is 60 invocations per second. If the clock frequency is 180 MHz, then `target_ti` is 180M/60, or 3 million clock cycles per function invocation.

Syntax

Place the pragma within the boundary a loop, or the outer loop of a loop nest.

```
#pragma HLS performance target_ti=<value> target_tl=<value> unit=[sec|cycle]
```

Where:

- **`target_ti=<value>`:** Specifies a target transaction interval defined as the number of clock cycles for the function, loop, or region of code to complete an iteration. The <value> can be specified as an integer, floating point, or constant expression that is resolved by the tool as an integer.

Note: A warning will be returned if truncation occurs.

- `target_tl=<value>`:

  Specifies a target latency defined as the number of clock cycles for the loop to complete all iterations. The transaction latency is defined as the interval between the start of the first iteration of the loop , and the completion of the last iteration of the loop. The <value> can be specified as an integer, floating point, or constant expression that is resolved by the tool as an integer.

- **unit=[sec | cycle]:** Specifies the unit associated with the `target_ti` or `target_tl` values. The unit can either be specified as seconds, or clock cycles. When the unit is specified as seconds, a unit can be specified with the value to indicate nanoseconds (ns), picoseconds (ps), microseconds (us).

Example 1

The outer loop is specified to have target transaction interval of 1000 clock cycles.

```
  for (int i =0; i < 1000; ++i) {
#pragma HLS performance target_ti=1000
    for (int j = 0; j < 8; ++j) {
      int tmp = b_buf[j].read();
      b[i * 8 + j] = tmp + 2;
    }
  }
```

See Also

- set_directive_performance
- pragma HLS inline

pragma HLS pipeline

Description

Reduces the initiation interval (II) for a function or loop by allowing the concurrent execution of operations. The default type of pipeline is defined by the `config_compile -pipeline_style` command, but can be overridden in the PIPELINE pragma or directive.

A pipelined function or loop can process new inputs every `<N>` clock cycles, where `<N>` is the II of the loop or function. An II of 1 processes a new input every clock cycle. You can specify the initiation interval through the use of the II option for the pragma.

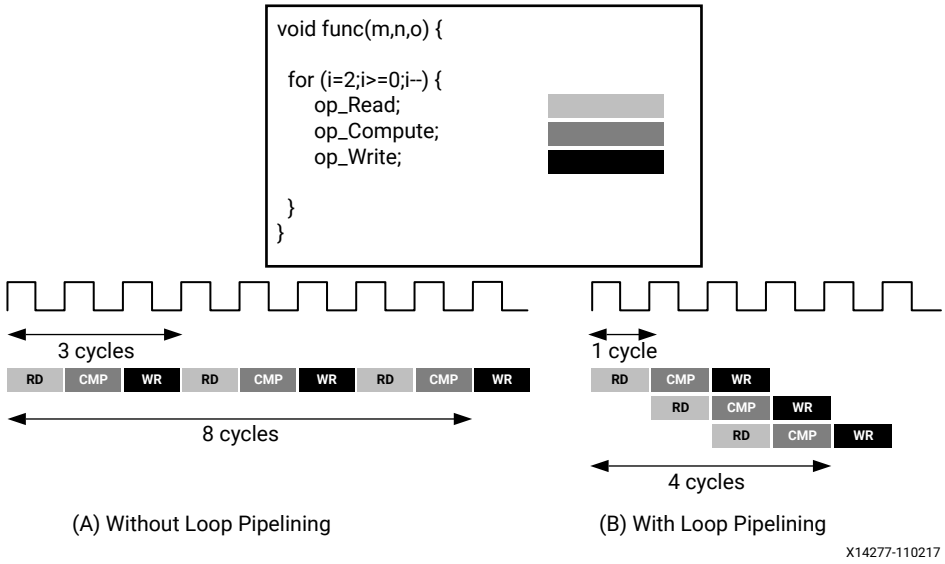
As a default behavior, with the PIPELINE pragma or directive Vitis HLS will generate the minimum II for the design according to the specified clock period constraint. The emphasis will be on meeting timing, rather than on achieving II unless the `II` option is specified.

If the Vitis HLS tool cannot create a design with the specified II, it issues a warning and creates a design with the lowest possible II.

You can then analyze this design with the warning message to determine what steps must be taken to create a design that satisfies the required initiation interval.

Pipelining a loop allows the operations of the loop to be implemented in a concurrent manner as shown in the following figure. In the figure, (A) shows the default sequential operation where there are three clock cycles between each input read (II=3), and it requires eight clock cycles before the last output write is performed. (B) shows the pipelined operations that show one cycle between reads (II=1), and 4 cycles to the last write.

Figure 143: **Loop Pipeline**



IMPORTANT! *Loop carry dependencies can prevent pipelining. Use the DEPENDENCE pragma or directive to provide additional information to overcome loop-carry dependencies, and allow loops to be pipelined (or pipelined with lower intervals).*

Syntax

Place the pragma in the C source within the body of the function or loop.

```
#pragma HLS pipeline II=<int> off rewind style=<value>
```

Where:

- `II=<int>`: Specifies the desired initiation interval for the pipeline. The HLS tool tries to meet this request. Based on data dependencies, the actual result might have a larger initiation interval.
- `off`: Optional keyword. Turns off pipeline for a specific loop or function. This can be used to disable pipelining for a specific loop when `config_compile -pipeline_loops` is used to globally pipeline loops.
- `rewind`: Optional keyword. Enables rewinding as described in Rewinding Pipelined Loops for Performance. This enables continuous loop pipelining with no pause between one execution of the loop ending and the next execution starting. Rewinding is effective only if there is one single loop (or a perfect loop nest) inside the top-level function. The code segment before the loop:
  - Is considered as initialization.
  - Is executed only once in the pipeline.
  - Cannot contain any conditional operations (if-else).

  TIP: *This feature is only supported for pipelined loops; it is not supported for pipelined functions.*
- `style=<stp | frp | flp>`: Specifies the type of pipeline to use for the specified function or loop. For more information on pipeline styles refer to Flushing Pipelines and Pipeline Types. The types of pipelines include:
  - `stp`: Stall pipeline. Runs only when input data is available otherwise it stalls. This is the default setting, and is the type of pipeline used by Vitis HLS for both loop and function pipelining. Use this when a flushable pipeline is not required. For example, when there are no performance or deadlock issue due to stalls.
  - `flp`: This option defines the pipeline as a flushable pipeline. This type of pipeline typically consumes more resources and/or can have a larger II because resources cannot be shared among pipeline iterations.
  - `frp`: Free-running, flushable pipeline. Runs even when input data is not available. Use this when you need better timing due to reduced pipeline control signal fanout, or when you need improved performance to avoid deadlocks. However, this pipeline style can consume more power as the pipeline registers are clocked even if there is no data.

  IMPORTANT! *This is a hint not a hard constraint. The tool checks design conditions for enabling pipelining. Some loops might not conform to a particular style and the tool reverts to the default style (`stp`) if necessary.*

Examples

In this example, function `func` is pipelined with an initiation interval of 1.

```
void func { a, b, c, d} {
  #pragma HLS pipeline II=1
  ...
}
```

See Also

- set_directive_pipeline
- pragma_HLS_dependence
- config_compile

pragma HLS protocol

Description

This commands specifies a region of code, a protocol region, in which no clock operations will be inserted by Vitis HLS unless explicitly specified in the code. Vitis HLS will not insert any clocks between operations in the region, including those which read from or write to function arguments. The order of read and writes will therefore be strictly followed in the synthesized RTL.

A region of code can be created in the C/C++ code by enclosing the region in braces "{ }" and naming it. The following defines a region named `io_section`:

```
io_section:{
...
lines of code
...
}
```

A clock operation can be explicitly specified in C/C++ code using an `ap_wait()` statement, and may be specified in C++ code by using the `wait()` statement. The `ap_wait` and `wait` statements have no effect on the simulation of the design.

Syntax

Place the pragma in the C source within the body of the function or protocol region.

```
#pragma HLS protocol [floating | fixed]
```

Options

- `floating`: Lets code statements outside the protocol region overlap and execute in parallel with statements in the protocol region in the final RTL. The protocol region remains cycle accurate, but outside operations can occur at the same time. This is the default mode.
- `fixed`: The fixed mode ensures that statements outside the protocol region do not execute in parallel with the protocol region.

Examples

This example defines a protocol region, `io_section` in function `foo` where the pragma defines that region as a floating protocol region as the default mode:

```
io_section: {
#pragma HLS protocol
...
}
```

See Also

- set_directive_protocol

pragma HLS reset

Description

The RESET pragma or directive adds or disables reset ports for specific state variables (global or static).

The reset port is used to restore the registers and block RAM, connected to the port, to an initial value any time the reset signal is applied. Globally, the presence and behavior of the RTL reset port is controlled using the `syn.rtl.reset` configuration settings (or `config_rtl -reset`). The reset configuration settings include the ability to define the polarity of the reset, and specify whether the reset is synchronous or asynchronous, but more importantly it controls, through the reset option, which registers are reset when the reset signal is applied. For more information, see Controlling Initialization and Reset Behavior.

More specific control over reset is provided through the RESET pragma. For global or static variables the RESET pragma is used to explicitly enable a reset when none is present, or the variable can be removed from the reset by turning `off` the pragma. This can be particularly useful when static or global arrays are present in the design.

***Note*:** For public variables of a class, the RESET pragma must be used as the reset configuration settings only apply to variables declared at the function or file level. In addition, the RESET pragma must be applied to an object of the class in the top-function or sub-function, and cannot be applied to private variables of the class.

Syntax

Place the pragma in the C source within the boundaries of the variable life cycle.

```
#pragma HLS reset variable=<a> off
```

Where:

- **`variable=<a>`:** Specifies the variable to which the RESET pragma is applied.
- **`off`:** Indicates that reset is not generated for the specified variable.

Example 1

This example adds reset to the variable `a` in function `foo` even when the global reset setting is `none` or `control`.

```
void foo(int in[3], char a, char b, char c, int out[3]) {
#pragma HLS reset variable=a
```

Example 2

Removes reset from variable `a` in function `foo` even when the global reset setting is `state` or `all`.

```
void foo(int in[3], char a, char b, char c, int out[3]) {
#pragma HLS reset variable=a off
```

Example 3

The following example shows the use of the RESET pragma or directive with class variables and methods. A variable declared in a class must be a public static variable in the class and can have the RESET pragma specified for the variable in a method of the class, or after an object of the class has been created in the top function or sub-function of the HLS design.

```
class bar {
public:
  static int a; // class level static; must be public
  int a1;       // class-level non-static; must be public
  bar(...) {
  // #pragma reset does not work here for a or a1
  }
  // this is the function that is called in the top
  void run(...) {
  // #pragma reset does not work here for non-static variable a1
  // but does work for static variable a
  #pragma reset variable=a
```

```
  }
};
static int c; // file level
int d; // file level
void top(...) {
#pragma HLS top
  static int b; // function level
  bar t;
  static bar s;

  // #pragma reset works here for b, c and d, as well as t and s members
  // except for t.a1 because it is not static
  #pragma reset variable=b
  #pragma reset variable=c
  #pragma reset variable=d
  #pragma reset variable=t.a
  #pragma reset variable=s.a
  #pragma reset variable=s.a1
  t.run(...);
  s.run(...);
}
```

See Also

- set_directive_reset
- config_rtl

pragma HLS stable

Description

Arguments marked as stable cannot change during kernel execution. Inputs that only change when the kernel is not running, such as ports that provide configuration data, should be marked with a STABLE pragma or directive. The STABLE Pragma can only be used for Control-Driven task modeling because in Data-driven Task-level Parallelism there is no way to detect when the kernel is idle.

The STABLE pragma informs Vitis HLS of the following:

- The data applied to the port remains stable during normal operation but is not a constant value that can be optimized.
- The fanout from this port is not required to be registered.

Pragma STABLE pragma in DATAFLOW networks tells the compiler that input does not need to be read by the first dataflow process and that output does not need to be written by the last dataflow process in a network. This can dramatically improve II.

Syntax

```
#pragma HLS stable variable=<a>
```

Where:

- **`variable=<a>`:** Specifies the name of the scalar or array variable to which the STABLE pragma is applied.

Examples

In the following example, without the STABLE pragma, `proc1` and `proc2` would be synchronized to acknowledge the reading of their inputs (including `A`). With the pragma, `A` is no longer considered as an input that needs synchronization.

```
void dataflow_region(int A[...], int B[…] ...
#pragma HLS stable variable=A
#pragma HLS dataflow
    proc1(...);
    proc2(A, ...);
```

See Also

- set_directive_stable
- pragma HLS dataflow

pragma HLS stream

Description

By default, array variables are implemented as RAM:

- Top-level function array parameters are implemented as a RAM interface port.
- General arrays are implemented as RAMs for read-write access.
- Arrays involved in sub-functions, or loop-based DATAFLOW optimizations are implemented as a RAM ping pong buffer channel.

If the data stored in the array is consumed or produced in a sequential manner, a more efficient communication mechanism is to use streaming data as specified by the STREAM pragma, where FIFOs are used instead of RAMs.

IMPORTANT! *When an argument of the top-level function is specified as INTERFACE type `ap_fifo`, the array is automatically implemented as streaming. See Defining Interfaces for more information.*

Syntax

Place the pragma in the C source within the boundaries of the required location.

```
#pragma HLS stream variable=<variable> type=<type> depth=<int>
```

Where:

- **`variable=<variable>`:** Specifies the name of the array to implement as a streaming interface.
- **`depth=<int>`:** Relevant only for array streaming in DATAFLOW channels. By default, the depth of the FIFO implemented in the RTL is the same size as the array specified in the C code. This option lets you modify the size of the FIFO to specify a different depth.

  When the array is implemented in a DATAFLOW region, it is common to use the `depth` option to reduce the size of the FIFO. For example, in a DATAFLOW region when all loops and functions are processing data at a rate of II=1, there is no need for a large FIFO because data is produced and consumed in each clock cycle. In this case, the `depth` option can be used to reduce the FIFO size to 1 to substantially reduce the area of the RTL design.

  TIP: *The `config_dataflow -depth` command provides the ability to stream all arrays in a DATAFLOW region. The `depth` option specified in the STREAM pragma overrides the `config_dataflow -depth` setting for the specified `<variable>`.*

- **type=<arg>:** Specify a mechanism to select between FIFO, PIPO, synchronized shared (`shared`), and un-synchronized shared (`unsync`). The supported types include:
  - `fifo`: A FIFO buffer with the specified `depth`.
  - `pipo`: A Ping-Pong buffer with as many “banks” as the specified depth (default is 2)
  - `shared`: A shared channel, synchronized like a regular Ping-Pong buffer, with depth, but without duplicating the array data. Consistency can be ensured by setting the depth small enough, which acts as the distance of synchronization between the producer and consumer.

    TIP: *The default depth for shared is 1.*

  - `unsync`: Does not have any synchronization except for individual memory reads and writes. Consistency (read-write and write-write order) must be ensured by the design itself.

Example 1

The following example specifies array `A[10]` to be streaming, and implemented as a FIFO.

```
#pragma HLS STREAM variable=A
```

Example 2

In this example, array `B` is set to streaming with a FIFO depth of 12.

```
#pragma HLS STREAM variable=B depth=12 type=fifo
```

Example 3

Array `C` has streaming implemented as a PIPO.

```
#pragma HLS STREAM variable=C type=pipo
```

See Also

- set_directive_stream
- pragma HLS dataflow
- pragma HLS interface
- config_dataflow

pragma HLS top

Description

Attaches a name to a function, which can then be used with the `set_top` command to synthesize the function and any functions called from the specified top-level. This is typically used to synthesize member functions of a class in C/C++.

Specify the TOP pragma in an active solution, and then use the `set_top` command with the new name.

Syntax

Place the pragma in the C source within the boundaries of the required location.

```
#pragma HLS top name=<string>
```

Where:

- **`name=<string>`:** Specifies the name to be used by the `set_top` command.

Examples

Function `foo_long_name` is designated the top-level function, and renamed to `DESIGN_TOP`. After the pragma is placed in the code, the `set_top` command must still be issued from the Tcl command line, or from the top-level specified in the IDE project settings.

```
void foo_long_name () {
  #pragma HLS top name=DESIGN_TOP
  ...
}
```

Followed by the `set_top DESIGN_TOP` command.

See Also

- set_directive_top
- set_top

pragma HLS unroll

Description

You can unroll loops to create multiple independent operations rather than a single collection of operations. The UNROLL pragma transforms loops by creating multiples copies of the loop body in the RTL design, which allows some or all loop iterations to occur in parallel.

Loops in the C/C++ functions are kept rolled by default. When loops are rolled, synthesis creates the logic for one iteration of the loop, and the RTL design executes this logic for each iteration of the loop in sequence. A loop is executed for the number of iterations specified by the loop induction variable. The number of iterations can also be impacted by logic inside the loop body (for example, `break` conditions or modifications to a loop exit variable). Using the UNROLL pragma you can unroll loops to increase data access and throughput.

The UNROLL pragma allows the loop to be fully or partially unrolled. Fully unrolling the loop creates a copy of the loop body in the RTL for each loop iteration, so the entire loop can be run concurrently. Partially unrolling a loop lets you specify a factor *N*, to create *N* copies of the loop body and reduce the loop iterations accordingly.



TIP: *To unroll a loop completely, the loop bounds must be known at compile time. This is not required for partial unrolling.*

Partial loop unrolling does not require *N* to be an integer factor of the maximum loop iteration count. The Vitis HLS tool adds an exit check to ensure that partially unrolled loops are functionally identical to the original loop. For example, given the following code:

```
for(int i = 0; i < X; i++) {
  pragma HLS unroll factor=2
  a[i] = b[i] + c[i];
}
```

Loop unrolling by a factor of 2 effectively transforms the code to look like the following code where the `break` construct is used to ensure the functionality remains the same, and the loop exits at the appropriate point.

```
for(int i = 0; i < X; i += 2) {
  a[i] = b[i] + c[i];
  if (i+1 >= X) break;
  a[i+1] = b[i+1] + c[i+1];
}
```

In the example above, because the maximum iteration count, `X`, is a variable, the HLS tool might not be able to determine its value, so it adds an exit check and control logic to partially unrolled loops. However, if you know that the specified unrolling factor, 2 in this example, is an integer factor of the maximum iteration count `X`, the `skip_exit_check` option lets you remove the exit check and associated logic. This helps minimize the area and simplify the control logic.



TIP: *When the use of pragmas like ARRAY_PARTITION or ARRAY_RESHAPE let more data be accessed in a single clock cycle, the HLS tool automatically unrolls any loops consuming this data, if doing so improves the throughput. The loop can be fully or partially unrolled to create enough hardware to consume the additional data in a single clock cycle. This automatic unrolling is controlled using the `config_unroll` command.*

Syntax

Place the pragma in the C source within the body of the loop to unroll.

```
#pragma HLS unroll factor=<N> skip_exit_check off=true
```

Where:

- **`factor=<N>`:** Specifies a non-zero integer indicating that partial unrolling is requested. The loop body is repeated the specified number of times, and the iteration information is adjusted accordingly. If `factor=` is not specified, the loop is fully unrolled.
- **`skip_exit_check`:** Optional keyword that applies only if partial unrolling is specified with `factor=`. The elimination of the exit check is dependent on whether the loop iteration count is known or unknown:
  - Fixed bounds

    No exit condition check is performed if the iteration count is a multiple of the factor.

If the iteration count is *not* an integer multiple of the factor, the tool:

- Prevents unrolling.
- Issues a warning that the exit check must be performed to proceed.

- Variable bounds

  The exit condition check is removed. You must ensure that:

  - The variable bounds is an integer multiple of the factor.
  - No exit check is in fact required.

- `off=true`: Disable unroll for the specified loop.

Example 1

The following example fully unrolls `loop_1` in function `foo`. Place the pragma in the body of `loop_1` as shown.

```
loop_1: for(int i = 0; i < N; i++) {
  #pragma HLS unroll
  a[i] = b[i] + c[i];
}
```

Example 2

This example specifies an unroll factor of 4 to partially unroll `loop_2` of function `foo`, and removes the exit check.

```
void foo (...) {
  int8 array1[M];
  int12 array2[N];
  ...
  loop_2: for(i=0;i<M;i++) {
    #pragma HLS unroll skip_exit_check factor=4
    array1[i] = ...;
    array2[i] = ...;
    ...
  }
  ...
}
```

See Also

- set_directive_unroll
- pragma HLS loop_flatten
- pragma HLS loop_merge

HLS Tcl Commands

Project Commands

Project commands are AMD Vitis™ HLS commands that let you create and manage projects and solutions. The commands must be run in the interactive mode, `vitis_hls -i`, or can be run as a script using the `-f` option as described in vitis, v++, and vitis-run Commands.

The features of these commands are also available through the Vitis HLS GUI when performing specific functions as described in sections like Creating an HLS Component and Running C/RTL Co-Simulation.



TIP: *When running the commands through the GUI, the Tcl commands are added to a script of your project written to `solution/constraints/script.tcl`.*

add_files

Description

Adds design source files to the current project.

The tool searches the current directory for any header files included in the design source. To use header files stored in other directories, use the `-cflags` option to include those directories to the search path.

Syntax

```
add_files [OPTIONS] <src_files>
```

- `<src_files>` lists one or more supported source files.

Options

- **`-blackbox <file_name.json>`:** Specify the JSON file to be used for RTL blackbox. The information in this file is used by the HLS compiler during synthesizing and running C/C++ and co-simulation.
- **`-cflags <string>`:** A string with any GCC compilation options.

- **`-csimflags <string>`:** A string with any desired simulation compilation options. Flags specified with this option are only applied to simulation compilation, which includes C/C++ simulation and RTL co-simulation, not synthesis compilation. This option does not impact the `-cflags` option.
- **`-tb`:** Specifies any files used as part of the design test bench. These files are not synthesized. They are used when simulation is run by the `csim_design` or `cosim_design` commands.

  Do not use the `-tb` option when adding source files for the design. Use separate `add_files` commands to add design files and simulation files.

Examples

Add three design files to the project.

```
add_files a.cpp
add_files b.cpp
add_files c.cpp
```

Add multiple files with a single command line.

```
add_files "a.cpp b.cpp c.cpp"
```

Use the `-tb` option to add test bench files to the project. This example adds multiple files with a single command, including:

- The test bench `a_test.cpp`
- All data files read by the test bench:
  - `input_stimuli.dat`
  - `out.gold.dat`

  ```
  add_files -tb "a_test.cpp input_stimuli.dat out.gold.dat"
  ```

  If the test bench data files in the previous example are stored in a separate directory (for example `test_data`), the directory can be added to the project in place of the individual data files.

  ```
  add_files -tb a_test.cpp
  add_files -tb test_data
  ```

apply_ini

Description

Apply the specified INI file to the current project. The INI file can be used instead of a Tcl script to create and configure a project in the Vitis HLS tool. The INI file is written by the Vitis HLS tool using the `write_ini` command.

***Note*:** The INI file represents a solution in the current project. You must use the `open_project` and `open_solution` commands before using the `apply_ini` command in the project.

Syntax

```
apply_ini <ini_file> [OPTIONS]
```

- `<ini_file>` specifies the INI file to read into the AMD Vitis™ HLS tool.

Options

- **`-exclude <string>`:** Specify INI section glob patterns to exclude from the current project when reading the file.
- **`-include <string>`:** Specify INI section glob patterns to include in the current project when reading the file.
- **`-quiet <true | false>`:** Suppress all warning and information messages when reading the INI file. Note that errors will still be returned.
- **`-show <true | false>`:** Show verbose messages when reading the INI file.

Examples

Read the following INI file and apply it to the current project and solution:

```
apply_ini project1.ini
```

cat_ini

Description

Concatenate one or more INI files into a single file and output the results to stdout or to a specified file. The resulting INI can be read into the Vitis HLS tool using the `apply_ini` command.

Syntax

```
cat_ini { <ini_file1> <ini_file2> ...} [OPTIONS]
```

- `{ <ini_file1> <ini_file2> ...}` specifies one or more INI files to read into the AMD Vitis™ HLS tool and concatenate into a single file. The braces "{}" ar required to group multiple inputs.

Options

- **`-exclude <string>`:** Specify INI section glob patterns to exclude when writing the file.

- `-include <string>`:

  Specify glob patterns found in the INI files to include in the output file. Anything not included will be excluded.

  > **TIP:** *`-exclude` and `-include` are mutually exclusive, and cannot be used together.*

- `-out <file name>`: Specifies the INI file name to write as output. When not specified, the output is written to stdout.
- `-quiet <true | false>`: Suppress all warning and information messages when writing the INI file. Note that errors will still be returned. The default value is false.
- `-show <true | false>`: Show verbose messages when writing the INI file. The default value is false.

Examples

This example joins the specified INI files and returns them to stdout:

```
cat_ini {./run.ini ./test.ini}
```

close_project

Description

Closes the current project. The project is no longer active in the Vitis HLS session.

The `close_project` command:

- Prevents you from entering any project-specific or solution-specific commands.
- Is not required. Opening or creating a new project closes the current project.

Syntax

```
close_project
```

Options

This command has no options.

Examples

```
close_project
```

- Closes the current project.

- Saves all results.

close_solution

Description

Closes the current solution. The current solution is no longer active in the Vitis HLS session.

The `close_solution` command does the following:

- Prevents you from entering any solution-specific commands.
- Is not required. Opening or creating a new solution closes the current solution.

Syntax

```
close_solution
```

Options

This command has no options.

Examples

```
close_solution
```

- Closes the current solution.
- Saves all results.

cosim_design

Description

Executes post-synthesis co-simulation of the synthesized RTL with the original C/C++-based test bench.



TIP: *To specify the files for the test bench run the following command:*

```
add_files -tb
```

The simulation results are written to the `sim/Verilog` or `sim/VHDL` folder of the active solution, depending on the setting of the `-rtl` option.

Syntax

```
cosim_design [OPTIONS]
```

Options

- `-O`: Enables optimized compilation of the C/C++ test bench and RTL wrapper. This increases compilation time, but results in better runtime performance.
- **`-argv <string>`:** The `<string>` is passed onto the main C/C++ function.

  Specifies an argument list for the behavioral test bench.
- **`-compiled_library_dir <string>`:** Specifies the compiled library directory during simulation with third-party simulators. The `<string>` is the path name to the compiled library directory. The library must be compiled ahead of time using the `compile_simlib` command as explained in the *Vivado Design Suite Tcl Command Reference Guide* (UG835).
- **`-coverage`:** Enables the coverage feature during simulation with the VCS simulator.
- **`-disable_binary_tv`:** Use plain text test vector files instead of compressed binary format.
- **`-disable_deadlock_detection`:** Disables the deadlock detection feature in co-simulation.
- **`-disable_dependency_check`:** Disables dependency checks when running co-simulation.
- **`-enable_changing_stable_inputs`:** If set the cosim test bench can change the stable inputs of an `ap_ctrl_none` top design between top calls in cosimulation. This typically results in cosim errors and must be used only for designs whose test benches are tolerant of a change of a stable input occurs when the design is not idle.
- **`-enable_dataflow_profiling`:** This option turns on the dataflow channel profiling to track channel sizes during co-simulation.
- **`-enable_fifo_sizing`:** This option turns on automatic FIFO channel size tuning for dataflow profiling during co-simulation.
- **`-enable_tasks_with_m_axi`:** This option enables simulation of `hls::task` and `ap_ctrl_none` with stable top pointers.
- **`-hwemu_trace_dir <kernel_name>/<instance_name>`:** Specifies the location of test vectors generated during hardware emulation to be used during co-simulation. The test vectors are generated by the `config_export -cosim_trace_generation` command. The argument lets you specify the kernel and instance name of the Vitis kernel in the hardware emulation simulation results to locate the test vectors.
- **`-ldflags <string>`:** Specifies the options passed to the linker for co-simulation.

  This option is typically used to pass include path information or library information for the C/C++ test bench.
- **`-random_stall`:** Enable random stalling of top-level interfaces during co-simulation.

- `-rtl [verilog | vhdl]`: Specifies which RTL language to use for C/RTL co-simulation. The default is Verilog.
- `-setup`: Creates all simulation files created in the `sim/<HDL>` directory of the active solution. The simulation is not executed, but can be run later from a command shell.
- `-stable_axilite_update`: Enable `s_axilite` to configure registers which are stable compared with the prior transaction.
- `-trace_level [*none* | all | port | port_hier]`: Determines the level of waveform trace data to save during C/RTL co-simulation.
  - `none` does not save trace data. This is the default.
  - `all` results in all port and signal waveforms being saved to the trace file.
  - `port` only saves waveform traces for the top-level ports.
  - `port_hier` save the trace information for all ports in the design hierarchy.

  The trace file is saved in the `sim/Verilog` or `sim/VHDL` folder of the current solution when the simulation executes, depending on the selection used with the `-rtl` option.
- `-user_stall <string>`: Specifies the JSON stall file to be used during co-simulation. The stall file can be generated using the `cosim_stall` command.
- `-wave_debug`: Opens the Vivado simulator GUI to view waveforms and simulation results. Enables waveform viewing of all processes in the generated RTL, as in the dataflow and sequential processes. This option is only supported when using Vivado simulator for co-simulation by setting `-tool xsim`. See Viewing Simulation Waveforms for more information.

Examples

Performs verification using the Vivado simulator:

```
cosim_design
```

Uses the VCS simulator to verify the Verilog RTL and enable saving of the waveform trace file:

```
cosim_design -tool VCS -rtl verilog -coverage -trace_level all
```

Verifies the VHDL RTL using ModelSim. Values 5 and 1 are passed to the test bench function and used in the RTL verification:

```
cosim_design -tool modelsim -rtl vhdl -argv "5 1"
```

cosim_stall

Description

Command for development of the co-simulation stall file in JSON format.

Syntax

```
cosim_stall [OPTIONS]
```

Options

- `-check <string>`: Specify the JSON format stall file to use when running co-simulation.
- `-generate <string>`: Generate a JSON stall file to be used during co-simulation.
- `-list`: List all ports which can apply stall during co-simulation. This option returns a list of ports based on the current design.

Examples

The following example generates the specified stall file, and then specifies the file for use during co-simulation:

```
cosim_stall -generate my_cosim_stall.json
cosim_stall -check my_cosim_stall.json
```

create_clock

Description

Creates a virtual clock for the current solution.

The command can be executed only in the context of an active solution. The clock period is a constraint that drives optimization (chaining as many operations as feasible in the given clock period).

C and C++ designs support only a single clock.

Syntax

```
create_clock -period <number> [OPTIONS]
```

Options

- `-name <string>`: Specifies the clock name. If no name is given, a default name is used.
- `-period <number>`: Specifies the clock period in ns or MHz.

- If no units are specified, ns is assumed.
- If no period is specified, a default period of 10 ns is used.

Examples

Species a clock period of 50 ns:

```
create_clock -period 50
```

Uses the default name and period of 10 ns to specify the clock:

```
create_clock
```

Specifies clock frequency in MHz:

```
create_clock -period 100MHz
```

create_derived_clock

Description

Create a constraint for a secondary clock used in a `#pragma` HLS interface s_axilite clock option, relative to the primary clock frequency.

Syntax

```
create_derived_clock [OPTIONS]
```

Options

- **`-name`:** Name of the secondary clock pin.
- **`-multiply_by`:** Frequency multiplier (default: 1).
- **`-divide_by`:** Frequency divisor (default: 1).

csim_design

Description

Compiles and runs pre-synthesis C/C++ simulation using the provided C/C++ test bench.



TIP: *To specify the files for the test bench run the following command:*

```
add_files -tb
```

The simulation results are written to the `csim` folder inside the active solution.

Syntax

```
csim_design [OPTIONS]
```

Options

- `-O`: Enables optimized compilation of the C/C++ test bench. This increases compilation time, but results in better runtime performance.
- `-argv <string>`: Specifies the argument list for the behavioral test bench. The `<string>` is passed onto the `main()` C/C++ function of the test bench.
- `-clean`: Enables a clean build. Without this option, `csim_design` compiles incrementally.
- `-ldflags <string>`: Specifies the options passed to the linker for simulation. This option is typically used to pass include path information or library information for the C/C++ test bench.
- `-sanitize_address`: The option controls whether the address sanitizer is enabled during C simulation. The address sanitizer errors on addressing issues such as out of bounds array accesses.
- `-sanitize_undefined`: The option controls whether the undefined behavior sanitizer is enabled during C simulation. This sanitizer errors when a C/C++ construct with undefined behavior is executed during C simulation.
- `-setup`: When this option is specified, the simulation binary is created in the `csim` directory of the active solution, but simulation will not be executed. Simulation can be launched later from the compiled executable.

Examples

Compiles and runs C/C++ simulation:

```
csim_design
```

Compiles source design and test bench to generate the simulation binary. Does not execute the simulation binary:

```
csim_design -O -setup
```



TIP: *To run the simulation, execute `run.sh` in a command terminal, from the `csim/build` directory of the active solution.*

csynth_design

Description

Synthesizes the Vitis HLS project for the active solution.

The command can be executed only in the context of an active solution. The elaborated design in the database is scheduled and mapped onto RTL, based on any constraints that are set.

Syntax

```
csynth_design [OPTIONS]
```

Options

- **`-dump_cfg`**: Write a pre-synthesis control flow graph (CFG).
- **`-dump_post_cfg`**: Write a post-synthesis control flow graph (CFG).
- **`-synthesis_check`**: Runs a pre-synthesis design rule check, but does not generate RTL.

Examples

Runs Vitis HLS synthesis on the top-level design.

```
csynth_design
```

delete_project

Description

Deletes the directory associated with the project.

The `delete_project` command checks the corresponding project directory `<project>` to ensure that it is a valid Vitis HLS project before deleting it. If the specified project directory does not exist in the current work directory, the command has no effect.

Syntax

```
delete_project <project>
```

- `<project>` is the project name.

Options

This command has no options.

Examples

Deletes the `Project_1` by removing the directory and all its contents.

```
delete_project Project_1
```

delete_solution

Description

Removes a solution from an active project, and deletes the `<solution>` sub-directory from the project directory.

If the solution does not exist in the project directory, the command has no effect.

Syntax

```
delete_solution <solution>
```

- `<solution>` is the solution to be deleted.

Options

This command has no options.

Examples

Deletes solution `solution1` from the active project by deleting the sub-directory.

```
delete_solution solution1
```

enable_beta_device

Description

Enables specified beta access devices in the Vitis HLS tool set.

Syntax

```
enable_beta_device <pattern>
```

- `<pattern>` Specifies a pattern matching the beta devices to enable.

Options

This command has no options.

Examples

The following example enables all beta devices in the release:

```
enable_beta_device *
```

export_design

Description

Exports and packages the generated RTL code as a packaged IP for use in the Vivado Design Suite, or as a compiled Vitis kernel object (`.xo`) for the Vitis application acceleration development flow.

Supported formats include:

- Vivado IP for inclusion in the IP catalog.
- Vitis application acceleration kernel (`.xo`).
- Synthesized or implemented design checkpoint (DCP) format.
- Vivado IP and ZIP archive for use in the System Generator for DSP tool.

The packaged project is written to the `solution/impl` folder of the active solution.

Syntax

```
export_design [OPTIONS]
```

Options

- **`-description <string>`:** Provides a description for the catalog entry for the generated IP, used when packaging the IP.
- **`-display_name <string>`:** Provides a display name for the catalog entry for the generated IP, used when packaging the IP.
- **`-flow (syn | impl)`:** Obtains more accurate timing and resource usage data for the generated RTL using Vivado synthesis and implementation. The option `syn` performs RTL synthesis. The option `impl` performs both RTL synthesis and implementation, including a detailed place and route of the RTL netlist. In the Vitis HLS IDE, these options appear as check boxes labeled **Vivado Synthesis** and **Vivado Synthesis, place and route stage**.
- **`-format (ip_catalog | xo | syn_dcp | sysgen)`:** Specifies the format to package the IP. The supported formats are:
  - **`ip_catalog`:** A format suitable for adding to the AMD IP catalog.
  - **`xo`:** A format accepted by the v++ compiler for linking in the Vitis application acceleration flow.
  - **`syn_dcp`:** Synthesized checkpoint file for Vivado Design Suite. If this option is used, RTL synthesis is automatically executed. Vivado implementation can be optionally added.
  - **`sysgen`:** Generate a Vivado IP and .zip archive for use in System Generator.

- **`-ipname <string>`:** Provides the name component of the `Vendor:Library:Name:Version` (VLNV) identifier for generated IP.
- **`-library <string>`:** Provides the library component of the `Vendor:Library:Name:Version` (VLNV) identifier for generated IP.
- **`-output <string>`:** Specifies the path and file name of the output IP, XO, or DCP files. By default the file is written to the `solution/impl` folder of the current project if `-output` is not specified.
- **`-rtl (verilog | VHDL)`:** Specifies which HDL is used when the `-flow` option is executed. If not specified, Verilog is the default language for the Vivado synthesized netlist.
- **`-taxonomy <string>`:** Specifies the taxonomy for the catalog entry for the generated IP, used when packaging the IP.
- **`-vendor <string>`:** Provides the vendor component of the `Vendor:Library:Name:Version` (VLNV) identifier for generated IP.
- **`-version <string>`:** Provides the version component of the `Vendor:Library:Name:Version` (VLNV) identifier for generated IP.

Examples

Exports RTL for the Vitis application acceleration flow:

```
export_design -format xo
```

Exports the RTL as VHDL code in the Vivado IP catalog format. The VHDL is synthesized in Vivado synthesis tool to obtain better timing and usage data:

```
export_design -rtl vhdl -format ip_catalog -flow syn
```

get_clock_period

Description

This command returns the clock period for specified clock objects, or returns the default clock period for the active solution.

Syntax

```
get_clock_period [OPTIONS]
```

Options

- **`-default`:** Return the default period if the clock period is not specified.
- **`-name <string>`:** Get the clock period for the specified clock.

- `-units [ns | ps | hz | MHz | GHz]`: Return the clock period in the specified units. By default Vitis HLS returns the clock period in the same units as it was specified.

Examples

The following example creates a clock, `ap_clk`, and specifies the clock period in MHz. Then it gets the clock period for the clock as ns:

```
create_clock -name ap_clk -period 200MHz
get_clock_period -name ap_clk -ns
```

get_clock_uncertainty

Description

This command returns the clock uncertainty for specified clock, or returns the default clock uncertainty for the active solution.

Syntax

```
get_clock_uncertainty [clock_name]
```

- `<clock_name>` indicates the clock to get the uncertainty for.

Options

- `-default (true | false):`

  Return the default uncertainty value if it has not been set by user. If `true` then the default uncertainty is returned, unless set by the user. If `false`, then the default is not returned.

  .

Examples

The following example gets the clock uncertainty for the specified clock:

```
get_clock_uncertainty clk1
```

get_component

Description

This command gets the active component.

Syntax

```
get_component [OPTIONS]
```

Options

- `-data_json [=true|false]`: Return the component meta-data json file absolute path.
- `-directory [=true|false]`: Return the component directory absolute path.
- `-ide_json [=true|false]`: Return the component Vitis Unified IDE json file absolute path.
- `-ini [=true|false]`: Return the component config ini file absolute path.
- `-name [=true|false]`: Return the component name.
- `-work_dir [=true|false]`: Return the component work_dir absolute path.

get_derived_clock

Description

Description This command returns the derived clock for AXI4-Lite secondary clock for specified clock.

Syntax

```
get_derived_clock [clock_name]  <clock_name> indicates the derived clock.
```

get_files

Description

This command gets the files that have been added to the active solution.

Syntax

```
get_files [OPTIONS]
```

Options

- `-cflags`: Return any compiler flags specified with the files.
- `-csimflags`: Return any C simulation flags specified with the files.
- `-fullpath`: Return the full path of the files.

- `-tb`: Return only the files that were added as part of the test bench (added with the `-tb` option).

Examples

The following example gets the added test bench files from the current solution, and returns the full path for the files:

```
get_files -tb -fullpath
```

get_part

Description

This command returns the AMD device used in the active solution.

Syntax

```
get_part
```

Options

There are no options for this command.

Examples

The following example returns the part used in the active solution:

```
get_part
```

get_project

Description

This command gets information for the currently opened project.

Syntax

```
get_project [OPTIONS]
```

Options

- `-directory`: Return the full path to the project directory.
- `-ide (classic|vitis)`: (default is classic)Returns project ide setting.
- `-name`: Return the project name.

- `-solutions`: Return a list of all the solution names in the project.

Examples

The following example gets the full path for the current project:

```
get_project -directory
```

get_solution

Description

This command returns information related to the active solution.

Syntax

```
get_solution [OPTIONS]
```

Options

- `-directory`: Returns the full path to the active solution.
- `-flow_target`: Returns the flow target for the active solution.
- `-json`: Return the absolute path to the solution meta-data JSON file
- `-name`: Returns the solution name.

Examples

The following example returns the flow target for the active solution:

```
get_solution -flow_target
```

get_top

Description

This command returns the name of the top-level function for the open Vitis HLS project.

Syntax

```
get_top
```

Options

There are no options for this command.

Examples

The following example returns the top-level function for the open project:

```
get_top
```

help

Description

- When specified without a command name, the `help` command lists all Vitis HLS Tcl commands.
- When used with a Vitis HLS Tcl command as an argument, the `help` command returns details of the specified command.



TIP: *For recognized Vitis HLS commands, auto-completion using the tab key is available when entering the command name.*

Syntax

```
help <cmd>
```

- `<cmd>` specifies a command name to return the help for. If no command is specified, a list of all Vitis HLS commands will be returned.

Options

- **`cmd <string>`**: Command name
- **`-ini [=true|false]`**: Show help for command ini
- **`-list [=true|false]`**: List matching help entries instead of showing full help
- **`-width <int:120>`**: Set width of output

Examples

Displays help for all commands and directives:

```
help
```

Displays help for the `add_files` command:

```
help add_files
```

list_part

Description

This command returns names of supported device families, parts, or boards.

If no argument is provided, the command will return all supported part families. To return specific parts of a family, provide the family name as an argument.

Syntax

```
list_part [name] [OPTIONS]
```

- `<family>` specifies a device family to return the specific devices of. If no `<family>` is specified, the `list_part` command returns a list of available device families.

Options

- **`-name <string>`:** Family, part, board name, or glob pattern.
- **`-board[=false|true]`:** Returns list of board names instead of part names.
- **`-clock_regions`:** Return a list of clock regions for the specified part. Must be specified with the `-name` command.
- **`-slr_pblocks`:** Return the SLR Pblock dictionary for the specified part. Must be specified with the `-name` command.

Examples

Returns all supported device families.

```
list_part
```

Returns the clock regions of the current part in the active solution.

```
list_part -name [get_part] -clock_regions
```

open_component

Description

The open_component command opens an existing component or creates a new one.

Syntax

```
open_component [name] [OPTIONS]
```

Options

- `-flow_target <vivado|vitis>`: Specify target for interface, flow, and tool defaults.
  - Vivado: Vivado IP Flow Target
  - Vitis: Vitis Kernel Flow Target
- `-reset [=true|false]`: Resets the component by removing all data which already exists.

open_project

Description

Opens an existing project, or creates a new one if the specified project does not exist.

IMPORTANT! *In Vitis HLS, the `open_project` command returns an error when opening a Vivado HLS project, unless the `-upgrade` or `-reset` option is used.*

There can only be one active project in a Vitis HLS session. To close a project:

- Use the `close_project` command, or
- Open or create another project with the `open_project` or `open_tcl_project` commands.

Use the `delete_project` command to completely delete the project directory (removing it from the disk) and any solutions associated it.

Syntax

```
open_project [OPTIONS] <name>
```

- `<name>` specifies the project name.

Options

- `-ide (classic|vitis)`: Returns project ide setting.
- `-reset`:
  - Resets the project by removing any data that already exists in the project.
  - Removes any previous project information on design source files, header file search paths, and the top-level function. The associated solution directories and files are kept, but might now have invalid results.

RECOMMENDED: *Use the `-reset` option when executing Vitis HLS with Tcl scripts. Otherwise, each new `add_files` command adds additional files to the existing data.*

- `-upgrade`: Upgrade a Vivado HLS project to Vitis HLS.

Examples

Opens an existing project named `Project_1`, or creates a new one if it does not exist:

```
open_project Project_1
```

Opens a project and removes any existing data:

```
open_project -reset Project_2
```

open_solution

Description

Opens an existing solution or creates a new one in the currently active project. There can only be one active solution at any given time in a Vitis HLS session. As described in Target Flow Overview, the solution targets either the Vivado IP flow, or the Vitis Kernel flow. The default flow is the Vivado IP flow if no flow target is specified.

IMPORTANT! *Attempting to open or create a solution when there is no open project results in an error.*

Each solution is managed in a sub-directory of the current project. A new solution is created if the specified solution does not exist in the open project. To close a solution:

- Run the `close_solution` command, or
- Open another solution with the `open_solution` command.

Use the `delete_solution` command to remove a solution from the project and delete the corresponding sub-directory.

Syntax

```
open_solution [OPTIONS] <name>
```

- `<name>` specifies the solution name.

TIP: *You can specify both the project name and the solution name in order to use open_solution to open the project and solution in a single command:* `open_solution dctProj/solution1`

Options

- `-flow_target [vitis | vivado]:`

  - **vivado**: Configures the solution to run in support of the Vivado IP generation flow, requiring strict use of pragmas and directives, and exporting the results as Vivado IP. This is the default flow when `-flow_target` is not specified.
  - **vitis**: Configures the solution for use in the Vitis application acceleration development flow. This configures the Vitis HLS tool to properly infer interfaces for the function arguments without the need to specify the INTERFACE pragma or directive, and to output the synthesized RTL code as a Vitis kernel object file (`.xo`).
- **-reset**:
  - Resets the solution data if the solution already exists. Any previous solution information on libraries, constraints, and directives is removed.
  - Also removes synthesis, verification, and implementation results.

Examples

Opens an existing solution named `Solution_1` in the open project, or creates a new solution if one with the specified name does not exist. The solution is configured for creating kernel objects (`.xo`) for use in the Vitis tool flow.

```
open_solution -flow_target vitis Solution_1
```

Opens and resets the specified solution in the open project. Removes any existing data from the solution.

```
open_solution -reset Solution_2
```

open_tcl_project

Description

Create a project by sourcing a Tcl file, but skipping all design commands in the Tcl script: `cosim_design`, `csynth_design`, and `csim_design`. This command only creates and configures the project from a Tcl script. This lets you create a project using Tcl scripts from existing projects without running simulation or synthesis.

There can only be one active project in a Vitis HLS session. To close a project:

- Use the `close_project` command, or
- Open or create another project with the `open_tcl_project` or `open_project` commands.

Use the `delete_project` command to completely delete the project directory (removing it from the disk) and any solutions associated it.

Syntax

```
open_tcl_project <tclfile>
```

- `<tclfile>` specifies the path and name of a Tcl script to use when creating a project.

Options

This command has no options.

Examples

Creates and opens a project from the specified Tcl script:

```
open_tcl_project run_hls.tcl
```

project2ini

Description

Create a Config file (INI format) from an existing project. This lets you specify the project name and generate a configuration file from the project. Convert a HLS project's solution(s) into INI file(s).

Syntax

```
project2ini <project> [OPTIONS]
```

- `<project>` specifies the project name to generate an INI file from.

Options

- **-all [=true|false]:** Write all options regardless if current value is set/unset. . The possible values are true and false, and the default value is false.
- **-filepaths <absolute|none|relative> :** Control handling of relative file paths in output
- **-fileroot <string> :** Path to make file paths relative to (defaults to inifile directory)
- **-outdir <string> :** Output directory name, default to project directory
- **-package_defaults [=true|false] :** Include packaging default values
- **-quiet [=true|false]:** Suppress all messages (will still throw an error)
- **-show [=true|false] :** Show verbose messages
- **-solution <string:*> :** Solution names to convert (glob pattern)

Examples

:

```
project2ini Project_1
```

set_clock_uncertainty

Description

Sets a margin on the clock period defined by `create_clock`.

The margin of uncertainty is subtracted from the clock period to create an effective clock period. The clock uncertainty is defined in `ns`, or as a percentage of the clock period. The clock uncertainty defaults to 27% of the clock period.

Vitis HLS optimizes the design based on the effective clock period, providing a margin for downstream tools to account for logic synthesis and routing. The command can be executed only in the context of an active solution. Vitis HLS still uses the specified clock period in all output files for verification and implementation.

Syntax

```
set_clock_uncertainty <uncertainty> <clock_list>
```

- `<uncertainty>`: A value, specified in ns, representing how much of the clock period is used as a margin. The uncertainty can also be specified as a percentage of the clock period. The default uncertainty is 27% of the clock period.
- `<clock_list>`: A list of clocks to which the uncertainty is applied. If none is provided, it is applied to all clocks.

Options

This command has no options.

Examples

Specifies an uncertainty or margin of 0.5 ns on the clock. This effectively reduces the clock period that Vitis HLS can use by 0.5 ns.

```
set_clock_uncertainty 0.5
```

set_part

Description

Sets a target device, device family, or board for the current solution. The command can be executed only in the context of an active solution.



TIP: *Each solution in a project can target a separate device or device family.*

Syntax

```
set_part <device_specification>
```

- `<device_specification>` is a device specification that sets the target device for Vitis HLS synthesis and implementation.
- The device specification includes `<device>`, `<package>`, and `<speed_grade>` information.
- Specifying the `<device_family>` uses the default device for the device family.

Options

- `-board`: Specify the part as defined on a board.

Examples

The FPGA libraries provided with Vitis HLS can be added to the current solution by providing the device family name as shown below. In this case, the default device, package, and speed grade specified in the Vitis HLS FPGA library for the Virtex 7 device family are used.

```
set_part virtex7
```

The FPGA libraries provided with Vitis HLS can optionally specify the specific device with package and speed grade information.

```
set_part xc6vlx240tff1156-1
```

Specifies the part through the definition of a board.

```
set_part -board u200
```

set_top

Description

Defines the top-level function to be synthesized.

IMPORTANT! *Any functions called from the top-level function will also become part of the HLS design.*

Syntax

```
set_top <name>
```

- `<name>` is the function to be synthesized by HLS.

Options

This command has no options.

Examples

Sets the top-level function as `foo`.

```
set_top foo
```

write_ini

Description

Write the current project settings to the specified INI file. This can be used to export a project to an INI file rather than creating a Tcl script for the project. The INI can be read into the Vitis HLS tool using the `apply_ini` command, or can be used with the `v++ -c --mode hls` command.

Syntax

```
write_ini <ini_file> [OPTIONS]
```

`<ini_file>` specifies the INI file to create.

Options

- **`-all <true | false>`:** Write all options regardless if current value is set/unset. This ensures that even default values will be exported to the INI file as it is written.
- **`-appfile <string>`:** Project app file to use instead of current open project. Can be used to generate INI files from projects that are not currently opened.
- **`-apsfile <string>`:** Solution aps file to use instead of current open solution, or from an external project.
- **`-filepaths <absolute|relative|none>`:** Control handling of relative file paths in output. The default is `none`.

- `-fileroot <string>`: For use with the `-filepaths relative` option, this specifies the path to make file paths relative to. The default is relative to the directory where the tool was launched.
- `-include <string>`: Specify a string for glob patterns to include.
- `-package_defaults [=true|false]`: Write the default values of the various package config settings to the INI file.
- `-quiet <true | false>`: Suppress all warning and information messages when writing the INI file. Errors will still be returned.
- `-show <true | false>`: Show verbose messages when writing the INI file.
- `-solution_only [=true|false]`: Do not include project data in the generated INI file.
- `-workdir <string>`: Specify the work directory in the generated INI file.

Examples

This exports the current project to the specified INI file, exporting all options whether they are user specified or in their default values:

```
write_ini project1.ini -all=true
```

Configuration Commands

The configuration commands let you configure the AMD Vitis™ HLS tool to control the results of synthesis and simulation, specify defaults for pragmas and directives, and specify the outputs generated by default. The commands must be run in the interactive mode, `vitis_hls -i`, or can be run as a script using the `-f` option as described in vitis, v++, and vitis-run Commands.

These configuration commands can also be set in the Vitis HLS IDE using the **Solution Settings** dialog box as described in Defining the HLS Config File.

config_array_partition

Description

Specifies the default behavior for array partitioning.

Syntax

```
config_array_partition [OPTIONS]
```

Options

- `-throughput_driven <off | auto>`:

  Enable automatic partial and/or complete array partitioning.

  - `auto` : Enable automatic array partitioning with smart trade-offs between area and throughput. This is the default value.
  - `off` : Disable automatic array partitioning.

- `-complete_threshold <uint:4>`:

  Sets the threshold for completely partitioning arrays. Arrays with fewer elements than the specified threshold will be completely partitioned into individual elements.

Examples

Partitions all arrays in the design, except global arrays, with less than 12 elements into individual elements.

```
config_array_partition -complete_threshold 12
```

config_compile

Description

Configures the default behavior of front-end compiling.

Syntax

```
config_compile [OPTIONS]
```

Options

- `-double_acc_bits=N`: Accumulator precision for floating-point type double. The default is 269.
- `-enable_auto_rewind[=true|false]`: When TRUE uses alternative HLS implementation of pipelined loops which enables automatic loop rewind. This accepts values of TRUE or FALSE. The default value is TRUE.
- `-float_acc_bits=M`: Accumulator precision for floating-point type float. The default is 45.
- `-ignore_long_run_time[=true|false]`: Do not report the "long runtime" warning. This accepts values of TRUE or FALSE. The default value is FALSE.

- `-name_max_length <value>`: Specifies the maximum length of function names. If the length of the name is longer than the threshold, the last part of the name is truncated, and digits are added to make the name unique when required. The default is 256.
- `-no_signed_zeros[=true|false]`: Ignores the signedness of floating-point zero so that the compiler can perform aggressive optimizations on floating-point operations. This accepts values of TRUE or FALSE. The default value is FALSE.

  IMPORTANT! *Using this option might change the result of any floating point calculations and result in a mismatch in C/RTL co-simulation. Please ensure your test bench is tolerant of differences and checks for a margin of difference, not exact values.*

- `-pipeline_flush_in_task <always | never | ii1>`: Specifies that pipelines in `hls::tasks` will be flushing (`flp`) by default to reduce the probability of deadlocks in C/RTL Co-simulation. This option applies to pipelines that achieve an II=1 with the default option of `ii1`. However, you can also specify it as applying `always` to enable flushing pipelines in either `hls::tasks` or dataflow, or can be completely disabled using `never`. For more information refer to Flushing Pipelines and Pipeline Types.

  IMPORTANT! *Flushing pipelines (`flp`) are compatible with the rewind option specified in the PIPELINE pragma or directive when the `config_compile -enable_auto_rewind` option is also enabled.*

  - `always`: Always make pipelines flushable in `hls::tasks` or dataflow regardless of II.
  - `never`: Never make pipeline flushable unless specifically overridden by other directives or pragmas.
  - `ii1`: Make pipelines that achieve II=1 flushable in `hls::tasks`. This is the default setting.

- `-pipeline_loops <threshold>`:

  Specifies the lower limit for automatically pipelining loops and nested loops. In the case of a single un-nested loop, the loop will be pipelined if the trip count of the loop is greater than the specified threshold. The default value is 64, causing Vitis HLS to automatically pipeline loops with a trip count greater than 64. For example, a loop with a trip count of 65 will be pipelined with the default threshold, but not a loop with a trip count of 64.

  For nested loops, the loops are evaluated in pairs starting with the innermost loop. If the inner loop trip count exceeds the specified threshold it is marked for pipelining, and the analysis continues to the parent loop of the pair. If the trip count of the parent loop exceeds the specified threshold then it is also marked for pipelining, and the inner loop (previously marked for pipelining) is unrolled into the parent loop. The analysis continues upward if there are additional loops.

  However, other optimizations affect the analysis, and can prevent the pipeline behavior you are looking for. For example, you might need to use `loop_flatten off` to prevent flattening and facilitate loop pipeline.

- `-pipeline_style <stp | flp | frp>`: Specifies the default type of pipeline used by Vitis HLS for the PIPELINE pragma or directive, or for loop pipelining due to the `-pipeline_loops` threshold specified above. For more information on pipeline styles, refer to Flushing Pipelines and Pipeline Types.

  IMPORTANT! *This is a hint not a hard constraint. The tool checks design conditions for enabling pipelining. Some loops might not conform to a particular style and the tool reverts to the default style (stp) if necessary.*

  - `flp`: Flushable pipeline architecture: flushes when input data is not available then stalls waiting for new data.
  - `frp`: Free-running, flushable pipeline. Runs even when input data is not available. Use this when you need better timing due to reduced pipeline control signal fanout, or when you need improved performance to avoid deadlocks. However, this pipeline style can consume more power, as the pipeline registers are clocked even if there is no data.
  - `stp`: Stall pipeline. Runs only when input data is available otherwise it stalls. This is the default style when a flushable pipeline is not required, such as when there are no performance or deadlock issue due to stalls.
- `-pragma_strict_mode[=true|false]`: Enable error messages for misplaced or misused pragmas.
- `-pre_tcl <arg>`: Specify a Tcl script to run prior to starting the `csynth_design` command.
- `-unsafe_math_optimizations[=true|false]`: Ignores the signedness of floating-point zero and enables associative floating-point operations so that compiler can perform aggressive optimizations on floating-point operations. This accepts values of TRUE or FALSE. The default value is FALSE.

  Note: Using this option might change the result of any floating point calculations and result in a mismatch in C/RTL co-simulation. Please ensure your test bench is tolerant of differences and checks for a margin of difference, not exact values.

Examples

Pipeline the innermost loop with a tripcount higher than 30, or pipeline the parent loop of the innermost loop when its tripcount is less than or equal 30:

```
config_compile -pipeline_loops 30
```

Ignore the signedness of floating-point zero:

```
config_compile -no_signed_zeros
```

Ignore the signedness of floating-point zero and enable the associative floating-point operations:

```
config_compile -unsafe_math_optimizations
```

config_cosim

Description

Lets you configure the settings of the C/RTL Co-simulation command (`cosim_design`).

Syntax

```
config_cosim [OPTIONS]
```

Options

- **`-argv <string>`:** The `<string>` is passed onto the main C/C++ function.

  Specifies an argument list for the behavioral test bench.

- **`-compiled_library_dir <string>`:** Specifies the compiled library directory during simulation with third-party simulators. The `<string>` is the path name to the compiled library directory. The library must be compiled ahead of time using the `compile_simlib` command as explained in the *Vivado Design Suite Tcl Command Reference Guide* (UG835).

- **`-coverage`:** Enables the coverage feature during simulation with the VCS simulator.

- **`-disable_deadlock_detection`:** Disables the deadlock detection feature in co-simulation.

- **`-disable_dependency_check`:** Disables dependency checks when running co-simulation.

- **`-enable_dataflow_profiling`:** This option turns on the dataflow channel profiling to track channel sizes during co-simulation.

- **`-enable_fifo_sizing`:** This option turns on automatic FIFO channel size tuning for dataflow profiling during co-simulation.

- **`-enable_tasks_with_m_axi`:** Enables stable `m_axi` interfaces for use with `hls::task`.

- **`-hwemu_trace_dir <kernel_name>/<instance_name>`:** Specifies the location of test vectors generated during hardware emulation to be used during co-simulation. The test vectors are generated by the `config_export -cosim_trace_generation` command. The argument lets you specify the kernel and instance name of the Vitis kernel in the hardware emulation simulation results to locate the test vectors.

- **`-random_stall`:** Enable random stalling of top-level interfaces during co-simulation.

- **`-rtl [verilog | vhdl]`:** Specifies which RTL language to use for C/RTL co-simulation. The default is Verilog.

- **`-setup`:** Creates all simulation files created in the `sim/<HDL>` directory of the active solution. The simulation is not executed, but can be run later from a command shell.

- **`-stable_axilite_update`**: Enable `s_axilite` to configure registers which are stable compared with the prior transaction.
- **`-trace_level [*none* | all | port | port_hier]`**: Determines the level of waveform trace data to save during C/RTL co-simulation.
  - `none` does not save trace data. This is the default.
  - `all` results in all port and signal waveforms being saved to the trace file.
  - `port` only saves waveform traces for the top-level ports.
  - `port_hier` save the trace information for all ports in the design hierarchy.

  The trace file is saved in the `sim/Verilog` or `sim/VHDL` folder of the current solution when the simulation executes, depending on the selection used with the `-rtl` option.
- **`-user_stall <string>`**: Specifies the JSON stall file to be used during co-simulation. The stall file can be generated using the `cosim_stall` command.
- **`-wave_debug`**: Opens the AMD Vivado™ simulator GUI to view waveforms and simulation results. Enables waveform viewing of all processes in the generated RTL, as in the dataflow and sequential processes. This option is only supported when using Vivado simulator for co-simulation by setting `-tool xsim`. See Viewing Simulation Waveforms for more information.

config_csim

Description

Lets you configure the settings of the C simulation command (`csim_design`).

Syntax

```
config_csim [OPTIONS]
```

Options

- **`-argv <string>`**: Specifies the argument list for the behavioral test bench. The `<string>` is passed onto the `main()` C/C++ function of the test bench.
- **`-clean`**: Enables a clean build. Without this option, `csim_design` compiles incrementally.
- **`-sanitize_address`**: Enable the Clang sanitizer for address.
- **`-sanitize_undefined`**: Enable the Clang sanitizer for undefined.
- **`-setup`**: When this option is specified, the simulation binary will be created in the `csim` directory of the active solution, but simulation will not be executed. Simulation can be launched later from the compiled executable.

config_dataflow

Description

- Specifies the default behavior of dataflow pipelining (implemented by the `set_directive_dataflow` command).
- Allows you to specify the default channel memory type and depth.

Syntax

```
config_dataflow [OPTIONS]
```

Options

- **`-default_channel [fifo | pingpong]`:** By default, a RAM memory, configured in `pingpong` fashion, is used to buffer the data between functions or loops when dataflow pipelining is used. When streaming data is used (that is, the data is always read and written in consecutive order), a FIFO memory is more efficient and can be selected as the default memory type.

  TIP: *Set arrays to streaming using the set_directive_stream command to perform FIFO accesses.*

- **`-disable_fifo_sizing_opt`:** Disable FIFO sizing optimizations that can increase resource usage and might improve performance and reduce deadlocks.
- **`-fifo_depth <integer>`:** Specifies the default depth of the FIFOs. The default depth is 2.

  This option has no effect when ping-pong memories are used. If not specified, the default depth is 2, or if this is an array converted into a FIFO, the default size is the size of the original array. In some cases, this might be too conservative and introduce FIFOs that are larger than necessary. Use this option when you *know* that the FIFOs are larger than required.

  CAUTION! *Be careful when using this option. Insufficient FIFO depth might lead to deadlock situations.*

- **`-override_user_fifo_depth <value>`:**

  Use the specified depth for every `hls::stream`, overriding any user settings.

  ***Note*:** This is useful for checking if a deadlock is due to insufficient FIFO depths in the design. By setting it to a very large value (for example, the maximum depth printed by co-simulation at the end of simulation), if there is no deadlock, then you can use the FIFO depth profiling options of co-simulation and the GUI to find the minimum depth that ensures performance and avoids deadlocks.

- **`-scalar_fifo_depth <integer>`:** Specifies the minimum for scalar propagation FIFO.

Through scalar propagation, the compiler converts the scalar from C/C++ code into FIFOs. The minimal sizes of these FIFOs can be set with `-scalar_fifo_depth`. If this option is not provided, then the value of `-fifo_depth` is used.

- **`-start_fifo_depth <integer>`:** Specifies the minimum depth of start propagation FIFOs.

  This option is only valid when the channel between the producer and consumer is a FIFO. This option uses the same default value as the `-fifo_depth` option, which is 2. Such FIFOs can sometimes cause deadlocks, in which case you can use this option to increase the depth of the FIFO.

- **`-strict_mode [off | warning | error]`:** Set the severity for messages related to dataflow canonical form.

- **`-strict_stable_sync[=true|false]`:** Force synchronization of stable ports with `ap_done`.

- **`-task_level_fifo_depth <integer>`:** Specifies the depth of the task level FIFO.

  A FIFO is synchronized by `ap_ctrl_chain`. The write is the `ap_done` of the producer, the read is the `ap_ready` of the consumer. Like a PIPO in terms of synchronization, and like a FIFO in terms of access.

Examples

Changes the default channel from ping-pong memories to FIFOs:

```
config_dataflow -default_channel fifo
```

Changes the default channel from ping-pong memories to FIFOs with a depth of 6:

```
config_dataflow -default_channel fifo -fifo_depth 6
```

CAUTION! *If the design implementation requires a FIFO with greater than six elements, this setting results in a design that fails RTL verification. Be careful when using this option, because it is a user override.*

To find the cause of deadlocks, try to increase all the FIFO depths significantly, especially those that are reported by C/RTL co-simulation. If the deadlock disappears with a large "N", then it is due to insufficient FIFO depths. You can test this as follows:

```
config_dataflow -fifo_depth N -start_fifo_depth N -scalar_fifo_depth N
-task_level_fifo_depth N
```

config_debug

Generate HLS debug files used in the Vitis application acceleration development flow.

Description

Configures the default behavior of front-end compiling.

Syntax

```
config_debug [OPTIONS]
```

Options

- **`-directory <path>`:** Specifies the location of HLS debugging output. If not specified, location is set to `solution/.debug`.
- **`-enable[=true|false]`:** Enable generation of HLS debugging files used in Vitis debug flow.

Examples

The following example enables the debug files:

```
config_debug -enable true
```

config_export

Description

Configures options for export_design which can either run downstream tools or package a Vivado IP or Vitis compiled kernel object (`.xo`).

Syntax

```
config_export [OPTIONS]
```

Options

- **`-cosim_trace_generation=<true | false>`:** Generate test vectors during hardware emulation in the Vitis tool flow when the kernel is synthesized as a Vitis kernel, to be used during C/RTL Co-simulation in future iterations.
- **`-description <string>`:** Provides a description for the catalog entry for the generated IP, used when packaging the IP.
- **`-display_name <string>`:** Provides a display name for the catalog entry for the generated IP, used when packaging the IP.
- **`-enable_verbose_report`:** Enable a verbose report.

- `-flow (none | syn | impl)`: Lets you obtain more accurate timing and resource usage data for the generated RTL using Vivado synthesis and implementation. The option `syn` performs RTL synthesis. The option `impl` performs both RTL synthesis and implementation, including a detailed place and route of the RTL netlist. The default option is `none` which does not run either synthesis or implementation.

  TIP: *In the Vitis HLS IDE, these options appear as check boxes labeled* ***Vivado Synthesis*** *and* ***Vivado Synthesis, place and route stage****.*

- `-format (ip_catalog | xo | syn_dcp | sysgen)`: Specifies the format to package the IP. The supported formats are:
  - `ip_catalog`: A format suitable for adding to the AMD IP catalog.
  - `xo`: A format accepted by the v++ compiler for linking in the Vitis application acceleration flow.
  - `syn_dcp`: Synthesized checkpoint file for Vivado Design Suite. If this option is used, RTL synthesis is automatically executed. Vivado implementation can be optionally added.
  - `sysgen`: Generates a Vivado IP and .zip archive for use in System Generator.
- `-ip_xdc_file <arg>`: Specify an XDC file whose contents will be included in the packaged IP for use during implementation in the Vivado tool.
- `-ip_xdc_ooc_file <arg>`: Specify an out-of-context (OOC) XDC file whose contents will be included in packaged IP and used during out-of-context Vivado synthesis for the exported IP.
- `-ipname <string>`: Provides the name component of the `<Vendor>:<Library>:<Name>:<Version>` (VLNV) identifier for generated IP.
- `-library <string>`: Provides the library component of the `<Vendor>:<Library>:<Name>:<Version>` (VLNV) identifier for generated IP.
- `-output <string>`: Specifies the output location of the generated IP, `.xo`, or DCP files. The file is written to the `solution/impl` folder of the current project if no output path is specified.
- `-rtl (verilog | VHDL)`: Specifies which HDL is used when the `-flow` option is executed. If not specified, Verilog is the default language for the Vivado synthesized netlist.
- `-taxonomy <string>`: Specifies the taxonomy for the catalog entry for the generated IP, used when packaging the IP.
- `-vendor <string>`: Provides the vendor component of the `<Vendor>:<Library>:<Name>:<Version>` (VLNV) identifier for generated IP.
- `-version <string>`: Provides the version component of the `<Vendor>:<Library>:<Name>:<Version>` (VLNV) identifier for generated IP.

- `-vivado_clock <arg>`: Override the specified HLS clock constraint used in Vivado OOC run. This is only used for reporting purposes and will not apply to the exported IP.
- `-vivado_impl_strategy <string>`: Specifies Vivado implementation strategy name. The default name is 'default'.
- `-vivado_max_timing_paths <uint:10>`: Specify the max number of timing paths to report when the timing is not met in the Vivado synthesis or implementation.
- `-vivado_optimization_level (0 | 1 | 2 | 3)`: Vivado optimization level. This option sets other `vivado_*` options. This only applies for report generation and will not apply to the exported IP. The default setting is 0.
- `-vivado_pblock <arg>`: Specify a Pblock range to use during implementation for reporting purposes. This will not apply to the exported IP.
- `-vivado_phys_opt (none | place | route | all)`: Specifies whether Vivado physical optimization should be run during Vivado implementation. Valid values are:
  - `none`: Do not run (default).
  - `place`: Run post-place.
  - `route`: Run post-route.
  - `all`: Run post-place and post-route.
- `-vivado_report_level (0 | 1 | 2)`: Specifies how many Vivado reports are generated, and does not apply to the exported IP. The valid values and the associated reports are:
  - `0`: Post-synthesis utilization. Post-implementation utilization and timing.
  - `1`: Post-synthesis utilization, timing, and analysis. Post-implementation utilization, timing, and analysis.
  - `2`: Post-synthesis utilization, timing, analysis, and failfast. Post-implementation utilization, timing, and failfast. This is the default setting.
- `-vivado_synth_design_args <string>`: Specifies extra arguments to pass to the Vivado `synth_design` command. The default is `-directive sdx_optimization_effort_high`.
- `-vivado_synth_strategy <string>`: Specifies Vivado synth strategy name. The default strategy is "default".

Examples

The following example exports the Vitis `.xo` to the specified file:

```
export_design -description "Kernel Export" -display_name kernel_export \
-flow impl -format xo -output "tmp/hls_tests/kernel.xo"
```

config_interface

Description

Specifies the default interface options used to implement the RTL ports of each function during interface synthesis.

Syntax

```
config_interface [OPTIONS]
```

Options

- **`-ap_memory_io: [=bus | port]`:** The default is set to port. Specifies how ap_memory interfaces will be modeled in RTL and IP.
- **`-clock_enable[=true|false]`:** Adds a clock-enable port (`ap_ce`) to the design. The default is false.

  The clock enable prevents all clock operations when it is active-Low. It disables all sequential operations
- **`-default_slave_interface [none | s_axilite]`:** Enables the default for the slave interface as either `none`, which is the default for the Vivado IP flow, or as `s_axilite` which is the default for the Vitis Kernel flow, as described in Target Flow Overview.
- **`-m_axi_addr64[=true|false]`:** Globally enables 64-bit addressing for all `m_axi` ports in the design. By default, this is enabled for the Vitis flow, and otherwise disabled.
- **`-m_axi_alignment_byte_size <size>`:** Specifies the memory alignment boundary for `m_axi` interfaces provided as bitwidth. The `<size>` value must be a valid power of 2. A value of 0 is an invalid value. The default value is 64 when `open_solution -flow_target vitis`, and 1 when the `-flow_target=vivado`, aligning to a single byte.

  IMPORTANT! *Burst behavior will be incorrect if pointers are not aligned at runtime.*
- **`-m_axi_auto_id_channel[=true|false]`:** Enable automatic assignment of channel IDs for `m_axi` interfaces. Refer to M_AXI Channels for more information.

- **-m_axi_auto_max_ports[=true|false]:** If the option is `true`, all the `m_axi` interfaces that are not explicitly bundled, with INTERFACE pragmas or directives, will be mapped into individual interfaces, thus increasing the resource utilization (multiple adapters). The default is `false` and `m_axi` ports are bundled into a single interface.
- **-m_axi_buffer_impl [auto | lutram | uram | bram]:** Select the implementation for all `m_axi` internal buffers. The choices are `auto`, `lutram`, `bram`, `uram`. The default is `bram`.
- **-m_axi_cache_impl [auto | lutram | uram | bram]:**

  Specify the implementation resource for cache added to the `m_axi` adapters. The choices are `auto`, `lutram`, `bram`, `uram`. The default is `auto`.
- **-m_axi_conservative_mode=<true|false>:** This mode tells the `m_axi` not to issue a write request until the associated write data is entirely available (typically, buffered into the adapter or already emitted). It uses a buffer inside the MAXI adapter to store all the data for a burst (both in case of reading and in case of writing). This is enabled (`true`) by default, and could slightly increase write latency but can resolve deadlock due to concurrent requests (read or write) on the memory subsystem. This feature can be disabled by setting it to `false`.

  TIP: *This mode can be safely set to `false` to save this internal buffering if the design implements buffering that is larger than the max write burst length using an alternative approach.*
- **-m_axi_flush_mode:** Configure the `m_axi` adapter to be flushable, writing or reading garbage data if a burst is interrupted due to pipeline blocking, missing data inputs when not in conservative mode, or missing output space. The default is `false`. This is enabled when the option is specified.
- **-m_axi_latency <latency>:** Globally specifies the expected latency of the `m_axi` interface, allowing the design to initiate a bus request a number of cycles (latency) before the read or write is expected. The default value is 64 when `open_solution -flow_target vitis`, and 0 when `-flow_target vivado`.
- **-m_axi_max_bitwidth <size>:** Specifies the maximum bitwidth for the `m_axi` interfaces data channel. The default is 1024 bits. The specified value must be a power-of-two, between 8 and 1024. Note that this decreases throughput if the actual accesses are bigger than the required interface, as they will be split into a multi-cycle burst of accesses.
- **-m_axi_max_read_buffer_size <size>:** Overrides the size of the buffers for read data in the MAXI adapter, currently computed as the read burst size times the number of pending read bursts, in case the MAXI fabric provides some buffering already.
- **-m_axi_max_read_burst_length <size>:** Specifies a global maximum number of data values read during a burst transfer for all `m_axi` interfaces. The default is 16.

- **-m_axi_max_widen_bitwidth <size>:** Automatic port width resizing to widen bursts for the m_axi interface, up to the chosen bitwidth. The specified value must be a power of 2 between 8 and 1024, and must align with the -m_axi_alignment_size. The default value is 512 when open_solution -flow_target vitis, and 0 when the -flow_target vivado.
- **-m_axi_max_write_buffer_size <size>:** The option allows the designer to override the size of the buffers for write data in the MAXI adapter, currently computed as the write burst size times the number of pending write bursts, in case the MAXI fabric provides some buffering already.
- **-m_axi_max_write_burst_length <size>:** Specifies a global maximum number of data values written during a burst transfer for all m_axi interfaces. The default is 16.
- **-m_axi_min_bitwidth <size>:** Specifies the minimum bitwidth for the m_axi interfaces data channel. The default is 8 bits. The value must be a power of 2, between 8 and 1024. This does not necessarily increase throughput if the actual accesses are smaller than the required interface.
- **-m_axi_min_pending_requests <num>:** Set the minimum number of pending requests in the m_axi interface, to avoid m_axi adapter deadlocks due to too many requests scheduled before the first response comes back. Default value is 0.
- **-m_axi_num_read_outstanding <size>:** Specifies how many read requests can be made to the m_axi interface without a response, before the design stalls. The default value is 16. This implies internal storage in the design, and a FIFO of size:

  ```
  num_read_outstanding*max_read_burst_length*word_size
  ```

- **-m_axi_num_write_outstanding <size>:** Specifies how many write requests can be made to the m_axi interface without a response, before the design stalls. The default value is 16. This implies internal storage in the design, and a FIFO of size:

  ```
  num_write_outstanding*max_write_burst_length*word_size
  ```

- **-m_axi_offset [off | direct | slave]:** Globally controls the offset ports for all m_axi interfaces in the design.
  - **off:** No offset port is generated. This is the default value in the Vivado IP flow.
  - **direct:** Generates a scalar input offset port for directly passing the address offset into the IP through the offset port.
  - **slave:** Generates an offset port and automatically maps it to an AXI4-Lite slave. This is the default value.

- `-s_axilite_auto_restart_counter [ 0 | 1 ]`: Enables the auto-restart behavior for kernels. Use 0 to disable the auto-restart feature, which is the default, or use 1 to enable the feature. When enabled, the tool establishes the auto-restart bit in the `ap_ctrl_chain` control protocol for the `s_axilite` interface. For more information refer to *Continuously Running Kernels* in *Vitis Unified Software Platform Documentation: Application Acceleration Development* (UG1393).
- `-s_axilite_data64[=true|false]`: Set the data width for the `s_axilite` interface to 64 bits.
- `-s_axilite_interrupt_mode[=cor|tow]`: Specify the interrupt mode for `s_axilite` interface to be Clear on Read (`cor`) or Toggle on Write (`tow`). Clear on Read interrupt can be completed in a single transaction, while `tow` requires two. `Tow` is the default interrupt mode.
- `-s_axilite_mailbox [both | in | out]`: Enables the creation of a mailboxes for non-stream non-stable `s_axilite` arguments. The mailbox feature is used in the setting and management of never-ending kernels as described in *Continuously Running Kernels* in *Vitis Unified Software Platform Documentation: Application Acceleration Development* (UG1393). The argument values specify:
  - **both** : Enable mailbox for input and output arguments
  - **in** : Enable mailbox for only input arguments
  - **out** : Enable mailbox for only output arguments
  - **none** : No mailbox created (default)
- `-s_axilite_status_regs [ecc | off]`:

  Enables exposure of ECC error bits in the `s_axilite` register map via two clear-on-read (COR) counters per BRAM or URAM with ECC enabled.

  - **off:** No status registers generated. This is the default setting.
  - **ecc:** Enable counters for ECC errors for BRAMs and URAMs
- `-s_axilite_sw_reset[=false|true]`: Enable the software reset of a kernel in the `s_axilite` adapter.

Examples

The following example adds a clock enable port to the IP:

```
config_interface -clock_enable
```

config_op

Description

Sets the default options for micro-architecture binding of an operator (add, mul, sub...) to an FPGA implementation resource, and specify its latency.

Binding is the process in which operators (such as addition, multiplication, and shift) are mapped to specific RTL implementations. For example, a `mult` operation implemented as a combinational or pipelined RTL multiplier.

This command can be used multiple times to configure the default binding of different operation types to different implementation resources, or specify the default latency for that operation. The default configuration defined by `config_op` can be overridden by specifying the BIND_OP pragma or directive for a specific design element.

Syntax

```
config_op [OPTIONS] <op>
```

- **<op>:** Specifies the type of operation for the specified variable. Supported values include:
  - mul: integer multiplication operation
  - add: integer add operation
  - sub: integer subtraction operation
  - fadd: single precision floating-point add operation
  - fsub: single precision floating-point subtraction operation
  - fdiv: single precision floating-point divide operation
  - fexp: single precision floating-point exponential operation
  - flog: single precision floating-point logarithmic operation
  - fmul: single precision floating-point multiplication operation
  - frsqrt: single precision floating-point reciprocal square root operation
  - frecip: single precision floating-point reciprocal operation
  - fsqrt: single precision floating-point square root operation
  - dadd: double precision floating-point add operation
  - dsub: double precision floating-point subtraction operation
  - ddiv: double precision floating-point divide operation
  - dexp: double precision floating-point exponential operation

- dlog: double precision floating-point logarithmic operation
- dmul: double precision floating-point multiplication operation
- drsqrt: double precision floating-point reciprocal square root operation
- drecip: double precision floating-point reciprocal operation
- dsqrt: double precision floating-point square root operation
- hadd: half precision floating-point add operation
- hsub: half precision floating-point subtraction operation
- hdiv: half precision floating-point divide operation
- hmul: half precision floating-point multiplication operation
- hsqrt: half precision floating-point square root operation
- facc: single precision floating-point accumulate operation
- fmacc: single precision floating-point multiply-accumulate operation
- fmadd: single precision floating-point multiply-add operation



TIP: *Comparison operators, such as* `dcmp`*, are implemented in LUTs and cannot be implemented outside of the fabric, or mapped to DSPs, and so are not configurable with the* `config_op` *or* `bind_op` *commands.*

Options

- `-impl [dsp | fabric | meddsp | fulldsp | maxdsp | primitivedsp | auto | none | all]`: Defines the default implementation style for the specified operation. The default is to let the tool choose which implementation to use. The selections include:
  - all: All implementations. This is the default setting.
  - dsp: Use DSP resources
  - fabric: Use non-DSP resources
  - meddsp: Floating Point IP Medium Usage of DSP resources
  - fulldsp: Floating Point IP Full Usage of DSP resources
  - maxdsp: Floating Point IP Max Usage of DSP resources
  - primitivedsp: Floating Point IP Primitive Usage of DSP resources
  - auto: enable inference of combined facc | fmacc | fmadd operators
  - none: disable inference of combined facc | fmacc | fmadd operators

- `-latency <value>`: Defines the default latency for the binding of the type to the implementation resource. The valid value range varies for each implementation (`-impl`) of the operation. The default is -1, which applies the standard latency for the implementation resource.

  TIP: *The latency can be specified for a specific operation without specifying the implementation detail. This leaves Vitis HLS to choose the implementation while managing the latency.*

- `-precision [low | high | standard]`: Applies to `facc`, `fmacc`, and `fmadd` operators. Specify the precision for the given operator.
  - `low`: Use a low precision (60 bit and 100 bit integer) accumulation implementation when available. This option is only available on certain non-Versal devices, and may cause RTL/Co-Sim mismatches due to insufficient precision with respect to C++ simulation. However, it can always be pipelined with an II=1 without source code changes, though it uses approximately 3X the resources of `standard` precision floating point accumulation.
  - `high`: Use high precision (one extra bit) fused multiply-add implementation when available. This option is useful for high-precision applications and is very efficient on Versal devices, although it may cause RTL/Co-Sim mismatches due to the extra precision with respect to C++ simulation. It uses more resources than `standard` precision floating point accumulation.
  - `standard`: standard precision floating point accumulation and multiply-add is suitable for most uses of floating-point, and is the default setting. It always uses a true floating-point accumulator that can be pipelined with II=1 on Versal devices, and II that is typically between 3 and 5 (depending on clock frequency and target device) on non-Versal devices.

Example 1

The following example binds the addition operation to the fabric, with the specified latency:

```
config_op add -impl fabric -latency 2
```

Example 2

The following example enables the floating point accumulator with low-precision to achieve II=1 on a non-Versal device:

```
config_op facc -impl auto -precision low
```

config_rtl

Description

Configures various attributes of the output RTL, and the type of reset used. It also allows you to use specific identification in the RTL. By default, these options are applied to the top-level design and all RTL blocks within the design.

Syntax

```
config_rtl [OPTIONS]
```

Options

- **`-deadlock_detection <none | sim | hw | hw_diagnosis>`:** Enables simulation or synthesis deadlock detection in top level RTL of exported IP/XO file. The options are as follows:
  - `none` : Deadlock detection disabled
  - `sim` : Enables deadlock detection only for simulation/emulation (default)
  - `hw` : Deadlock detection enabled in synthesized and simulatable RTL IP. Adds `ap_local_deadlock` and `ap_local_block` signals to the IP to enable local and global deadlock detection.
  - `hw_diagnosis`: Deadlock detection enabled with additional diagnosis logic in generated RTL code.
- **`-fsm_encoding [one_hot|sequential|johnson|gray|auto|none]`:** Default is none. Specifies the 'fsm_encoding' RTL attribute value to guide RTL synthesis.
  - `auto` : Allows the RTL Synthesis tool to determine the best state machine encoding.
  - `gray` : Uses gray state machine encoding.
  - `johnson` : Uses johnson state machine encoding.
  - `one_hot` : Uses one_hot state machine encoding.
  - `sequential` : Uses sequential state machine encoding.
  - `none` : Disables state machine encoding, the state machine is synthesized as logic.
- **`-fsm_safe_state [auto_safe_state|reset_state|power_on_state|default_state|none]`:** Default is none. Specifies the 'fsm_safe_state' RTL attribute value to guide RTL synthesis. This attribute can affect the quality of synthesis results, typically resulting in less performance with greater area.
  - `auto_safe_state` : Implies Hamming-3 encoding.
  - `default_state` : Return the state machine to the default state.
  - `power_on_state` : Return the state machine to the POWER_ON state.
  - `reset_state` : Return the state machine to the RESET state.
  - `none` : Attribute not added to RTL, the state machine will not include safe state logic.
- **`-header <string>`:** Places the contents of file <string> at the top (as comments) of all output RTL and simulation files.

> **TIP:** *Use this option to ensure that the output RTL files contain user specified identification.*

- `-kernel_profile`: Add top level event and stall ports required by kernel profiling.
- `-module_auto_prefix`: Specifies the top level function name as the prefix value. This option is ignored if `config_rtl -module_prefix` is also specified. This is enabled by default.
- `-module_prefix <string>`: Specifies a user-defined prefix to be added to all RTL entity/module names.
- `-mult_keep_attribute`: Enable keep attribute.
- `-register_all_io`: Register all I/O signals by default. The default is `false`. This is enabled when the option is specified.
- `-register_reset_num <int>`: Specifies the number of registers to add to the reset signal. In the Vivado IP flow the default is 0. For the Vitis kernel flow, the default value is 3.
- `-reset [none | control | state | all]`: Variables initialized in the C/C++ code are always initialized to the same value in the RTL and therefore in the bitstream. This initialization is performed only at power-on. It is not repeated when a reset is applied to the design.

  The setting applied with the `-reset` option determines how registers and memories are reset.

  - `none`: No reset is added to the design.
  - `control`: Resets control registers, such as those used in state machines and those used to generate I/O protocol signals. This is the default setting.
  - `state`: Resets control registers and registers or memories derived from static or global variables in the C/C++ code. Any static or global variable initialized in the C/C++ code is reset to its initialized value.
  - `all`: Resets all registers and memories in the design. Any static or global variable initialized in the C/C++ code is reset to its initialized value.
- `-reset_async`: Causes all registers to use a asynchronous reset. If this option is not specified, a synchronous reset is used.
- `-reset_level (low | high)`: Allows the polarity of the reset signal to be either active-Low or active-High. The default is `High`.

> **TIP:** *The AXI protocol requires an active-Low reset. If your design uses AXI interfaces the tool will define this reset level with a warning if the `config_rtl -reset_level` is active-High.*

Examples

Configures the output RTL to have all registers reset with an asynchronous active-Low reset.

```
config_rtl -reset all -reset_async -reset_level low
```

Adds the contents of `my_message.txt` as a comment to all RTL output files.

```
config_rtl -header my_mesage.txt
```

config_schedule

Description

Configures the default type of scheduling performed by Vitis HLS.

Syntax

```
config_schedule [OPTIONS]
```

Options

- **`-enable_dsp_full_reg[=true|false]`:** Specifies that the DSP signals should be fully registered. The default is `true`.

Examples

The following example disables registering DSP signals:

```
config_schedule -enable_dsp_full_reg=false
```

config_sim

Description

Configures the C simulation.

Syntax

```
config_sim [OPTIONS]
```

Options

- `O:`: Enables optimized compilation for both csim and cosim and disables csim Code Analyzer. When false uses debug mode compilation.

- **`argv <string>`:** Specifies the argument list for the behavioral test bench. The <string> is passed onto the main() C/C++ function of the test bench.
- **`ldflags <string>`:** Specifies flags to be passed to the linker for both csim and cosim. This is typically used to pass library information for the C test bench and design.

config_storage

Description

Sets the global default options for Vitis HLS micro-architecture binding of FIFO storage elements to memory resources.

The default configuration defined by `config_storage` for FIFO strorage can be overridden by specifying the BIND_STORAGE pragma or directive for a specific design element, or specifying the `storage_type` option for the INTERFACE pragma or directive for objects on the interface.

Syntax

```
config_storage [OPTIONS] <type>
```

- **`<type>`:** Configures the `fifo` type.

Options

- **`-auto_srl_max_bits <value>`:** Specifies the maximum allowed SRL total bits (depth * width) for auto-srl implementations (`-impl autosrl`). The default is 1024.
- **`-auto_srl_max_depth <value>`:** Specifies the maximum allowed SRL depth for auto-srl implementation (`-impl autosrl`). The default is 2.
- **`-impl [auto | bram | lutram | uram | memory | srl]`:** Defines the device resource to use in binding the specified type.

Examples

The following example configures the default binding of `fifo`:

```
config_storage fifo -impl uram
```

config_unroll

Description

Automatically unroll loops based on the loop index limit (or tripcount).

Syntax

```
config_unroll [OPTIONS] <value>
```

Options

- **`-tripcount_threshold <value>`:** All loops which have fewer iterations than the specified value are automatically unrolled. The default value is 0.

Example

The following command ensures all loops which have fewer than 18 iterations are automatically unrolled during scheduling.

```
config_unroll -tripcount_threshold 18
```

Optimization Directives

Directives, or the `set_directive_*` commands, can be specified as Tcl commands that are associated with a specific solution, or set of solutions. Allowing you to customize the synthesis results for the same source code across different solutions. This lets you preserve the original code while engaging in what-if analysis of the design.

Directives must be run in the interactive mode, `vitis_hls -i`, or can be run as a script using the `-f` option as described in vitis, v++, and vitis-run Commands.

Pragmas are directives that you can apply in the source code, rather than as a Tcl script, and so change the synthesis results for all implementations of your code. There are HLS pragmas for every `set_directive` command, so you can choose how you want to work with your AMD Vitis™ HLS project. Refer to HLS Pragmas for information on the different pragmas.

Directives and pragmas are also available through the Vitis HLS IDE for assignment to specific elements of your source code, as described in Adding Pragmas and Directives.



TIP: *When running the commands through the IDE, the Tcl commands are added to a script of your project written to `solution/constraints/script.tcl`.*

set_directive_aggregate

Description

This directive collects the data fields of a struct into a single wide scalar. Any arrays declared within the struct and Vitis HLS performs a similar operation as `set_directive_array_reshape`, and completely partitions and reshapes the array into a wide scalar and packs it with other elements of the struct.



TIP: *Arrays of structs are restructured as arrays of aggregated elements.*

The bit alignment of the resulting new wide-word can be inferred from the declaration order of the struct elements. The first element takes the least significant sector of the word and so forth until all fields are mapped.

***Note*:** The AGGREGATE optimization does not pack the structs, and cannot be used on structs that contain other structs.

Syntax

```
set_directive_aggregate [OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the variable which will be packed.
- `<variable>` is the struct variable to be packed.

Options

- **`-compact [bit | byte | none | auto]`:** Specifies the alignment of the aggregated struct. Alignment can be on the bit-level (packed), the byte-level (padded), none, or automatically determined by the tool which is the default behavior.

Examples

Aggregates struct pointer `AB` with three 8-bit fields (`typedef struct {unsigned char R, G, B;}` pixel) in function `func`, into a new 24-bit pointer, aligning data at the bit-level.

```
set_directive_aggregate func AB -compact bit
```

See Also

- pragma HLS aggregate
- set_directive_array_reshape
- set_directive_disaggregate

set_directive_alias

Description

Specify that two or more `M_AXI` pointer arguments point to the same underlying buffer in memory (DDR or HBM) and indicate any aliasing between the pointers by setting the distance or offset between them.

IMPORTANT! *The ALIAS pragma applies to top-level function arguments mapped to `M_AXI` interfaces.*

Vitis HLS considers different pointers to be independent channels and generally does not provide any dependency analysis. However, in cases where the host allocates a single buffer for multiple pointers, this relationship can be communicated through the ALIAS pragma or directive and dependency analysis can be maintained. The ALIAS pragma enables data dependence analysis in Vitis HLS by defining the distance between pointers in the buffer.

Requirements for ALIAS:

- All ports assigned to an ALIAS pragma must be in assigned to `M_AXI` interfaces and assigned to different bundles, as shown in the example below
- Each port can only be used in one ALIAS pragma or directive
- The `depth` of all ports assigned to an ALIAS pragma must be the same
- When offset is specified, the number of ports and number of offsets specified must be the same: one offset per port
- The offset for the INTERFACE must be specified as slave or direct, `offset=off` is not supported

Syntax

```
set_directive_alias [OPTIONS] <location> <ports>
```

- `<location>` is the location string in the format `function[/label]` that the ALIAS pragma applies to.
- `<ports>` specifies the ports to alias.

Options

- **`-distance <integer>`:** Specifies the difference between the pointer values passed to the ports in the list.
- **`-offset <string>`:** Specifies the offset of the pointer passed to each port in the `ports` list with respect to the origin of the array.

Example

For the following function `top`:

```
void top(int *arr0, int *arr1, int *arr2, int *arr3, ...) {
  #pragma HLS interface M_AXI port=arr0 bundle=hbm0 depth=0x40000000
  #pragma HLS interface M_AXI port=arr1 bundle=hbm1 depth=0x40000000
  #pragma HLS interface M_AXI port=arr2 bundle=hbm2 depth=0x40000000
  #pragma HLS interface M_AXI port=arr3 bundle=hbm3 depth=0x40000000
```

The following command defines aliasing for the specified array pointers, and defines the `distance` between them:

```
set_directive_alias "top" arr0,arr1,arr2,arr3 -distance 10000000
```

Alternatively, the following command specifies the `offset` between pointers, to accomplish the same effect:

```
set_directive_alias top arr0,arr1,arr2,arr3 -offset
00000000,10000000,20000000,30000000
```

See Also

- pragma HLS alias
- set_directive_interface

set_directive_allocation

Description

Specifies instance restrictions for resource allocation.

The ALLOCATION pragma or directive can limit the number of RTL instances and hardware resources used to implement specific functions, loops, or operations. For example, if the C/C++ source has four instances of a function `foo_sub`, the `set_directive_allocation` command can ensure that there is only one instance of `foo_sub` in the final RTL. All four instances are implemented using the same RTL block. This reduces resources used by the function, but negatively impacts performance by sharing those resources.

The operations in the C/C++ code, such as additions, multiplications, array reads, and writes, can also be limited by the `set_directive_allocation` command.

Syntax

```
set_directive_allocation [OPTIONS] <location> <instances>
```

- `<location>` is the location string in the format `function[/label]`.
- `<instances>` is a function or operator.

The function can be any function in the original C/C++ code that has not been either inlined by the `set_directive_inline` command or inlined automatically by Vitis HLS.

For a complete list of operations that can be limited using the `ALLOCATION` pragma, refer to the config_op command.

Options

- **`-limit <integer>`:**

  Sets a maximum limit on the number of instances (of the type defined by the `-type` option) to be used in the RTL design.

- **`-type [function|operation]`:** The instance type can be `function` (default) or `operation`.

Examples

Given a design `foo_top` with multiple instances of function `foo`, limits the number of instances of `foo` in the RTL to 2.

```
set_directive_allocation -limit 2 -type function foo_top foo
```

Limits the number of multipliers used in the implementation of `My_func` to 1. This limit does not apply to any multipliers that might reside in sub-functions of `My_func`. To limit the multipliers used in the implementation of any sub-functions, specify an allocation directive on the sub-functions or inline the sub-function into function `My_func`.

```
set_directive_allocation -limit 1 -type operation My_func mul
```

See Also

- pragma HLS allocation
- set_directive_inline

set_directive_array_partition

Description



IMPORTANT! *`Array_Partition` and `Array_Reshape` pragmas and directives are not supported for `M_AXI` Interfaces on the top-level function. Instead you can use the `hls::vector` data types as described in Vector Data Types.*

Partitions an array into smaller arrays or individual elements.

This partitioning:

- Results in RTL with multiple small memories or multiple registers instead of one large memory.
- Effectively increases the amount of read and write ports for the storage.
- Potentially improves the throughput of the design.
- Requires more memory instances or registers.

Syntax

```
set_directive_array_partition [OPTIONS] <location> <array>
```

- `<location>` is the location (in the format `function[/label]`) which contains the array variable.
- `<array>` is the array variable to be partitioned.

Options

- **`-dim <integer>`:**

  Note: Relevant for multi-dimensional arrays only.

  Specifies which dimension of the array is to be partitioned.

  - If a value of 0 is used, all dimensions are partitioned with the specified options.
  - Any other value partitions only that dimension. For example, if a value 1 is used, only the first dimension is partitioned.

- **`-factor <integer>`:**

  Note: Relevant for type `block` or `cyclic` partitioning only.

  Specifies the number of smaller arrays that are to be created.

- **`-type (block|cyclic|complete)`:**

  - `block` partitioning creates smaller arrays from consecutive blocks of the original array. This effectively splits the array into *N* equal blocks where *N* is the integer defined by the `-factor` option.
  - `cyclic` partitioning creates smaller arrays by interleaving elements from the original array. For example, if `-factor 3` is used:
    - Element 0 is assigned to the first new array.
    - Element 1 is assigned to the second new array.
    - Element 2 is assigned to the third new array.
    - Element 3 is assigned to the first new array again.

  - `complete` partitioning decomposes the array into individual elements. For a one-dimensional array, this corresponds to resolving a memory into individual registers. For multi-dimensional arrays, specify the partitioning of each dimension, or use `-dim 0` to partition all dimensions.

  The default is `complete`.

- **`-off` or `off=true`:** Disables the ARRAY_PARTITION feature for the specified variable. Does not work with `dim`, `factor`, or `type`.

Example 1

Partitions array AB[13] in function `func` into four arrays. Because four is not an integer factor of 13:

- Three arrays have three elements.
- One array has four elements (AB[9:12]).

```
set_directive_array_partition -type block -factor 4 func AB
```

Partitions array AB[6][4] in function `func` into two arrays, each of dimension [6][2].

```
set_directive_array_partition -type block -factor 2 -dim 2 func AB
```

Partitions all dimensions of AB[4][10][6] in function `func` into individual elements.

```
set_directive_array_partition -type complete -dim 0 func AB
```

Example 2

Partitioned arrays can be addressed in your code by the new structure of the array, as shown in the following code example;

```
struct SS
{
  int x[N];
  int y[N];
};

int top(SS *a, int b[4][6], SS &c) {...}

set_directive_array_partition top b -type complete -dim 1
set_directive_interface -mode ap_memory top b[0]
set_directive_interface -mode ap_memory top b[1]
set_directive_interface -mode ap_memory top b[2]
set_directive_interface -mode ap_memory top b[3]
```

See Also

- pragma HLS array_partition
- set_directive_array_reshape

set_directive_array_reshape

Description

IMPORTANT! *`Array_Partition` and `Array_Reshape` pragmas and directives are not supported for `M_AXI` Interfaces on the top-level function. Instead you can use the `hls::vector` data types as described in Vector Data Types.*

Combines array partitioning with vertical array mapping to create a single new array with fewer elements but wider words.

The `set_directive_array_reshape` command has the following features:

- Splits the array into multiple arrays (like `set_directive_array_partition`).
- Automatically recombine the arrays vertically to create a new array with wider words.

Syntax

```
set_directive_array_reshape [OPTIONS] <location> <array>
```

- `<location>` is the location (in the format `function[/label]`) that contains the array variable.
- `<array>` is the array variable to be reshaped.

Options

- **`-dim <integer>`:**

  Note: Relevant for multi-dimensional arrays only.

  Specifies which dimension of the array is to be reshaped.

  - If the value is set to 0, all dimensions are partitioned with the specified options.
  - Any other value partitions only that dimension. The default is 1.

- **`-factor <integer>`:**

  Note: Relevant for type `block` or `cyclic` reshaping only.

  Specifies the number of temporary smaller arrays to be created.

- **`-object`:**

  Note: Relevant for container arrays only.

  Applies reshape on the objects within the container. If the option is specified, all dimensions of the objects will be reshaped, but all dimensions of the container will be kept.

- **`-type (block|cyclic|complete)`:**

- `block` reshaping creates smaller arrays from consecutive blocks of the original array. This effectively splits the array into *N* equal blocks where *N* is the integer defined by the `-factor` option and then combines the *N* blocks into a single array with `word-width*N`. The default is `complete`.
- `cyclic` reshaping creates smaller arrays by interleaving elements from the original array. For example, if `-factor 3` is used, element 0 is assigned to the first new array, element 1 to the second new array, element 2 is assigned to the third new array, and then element 3 is assigned to the first new array again. The final array is a vertical concatenation (word concatenation, to create longer words) of the new arrays into a single array.
- `complete` reshaping decomposes the array into temporary individual elements and then recombines them into an array with a wider word. For a one-dimension array this is equivalent to creating a very-wide register (if the original array was *N* elements of *M* bits, the result is a register with `N*M` bits). This is the default.

- **`-off` or `off=true`:** Disables the ARRAY_RESHAPE feature for the specified variable.

Example 1

Reshapes 8-bit array AB[17] in function `func` into a new 32-bit array with five elements.

Because four is not an integer factor of 17:

- Index 17 of the array, AB[17], is in the lower eight bits of the reshaped fifth element.
- The upper eight bits of the fifth element are unused.

```
set_directive_array_reshape -type block -factor 4 func AB
```

Partitions array AB[6][4] in function `func`, into a new array of dimension [6][2], in which dimension 2 is twice the width.

```
set_directive_array_reshape -type block -factor 2 -dim 2 func AB
```

Reshapes 8-bit array AB[4][2][2] in function `func` into a new single element array (a register), 4*2*2*8 (= 128)-bits wide.

```
set_directive_array_reshape -type complete -dim 0 func AB
```

Example 2

Partitioned arrays can be addressed in your code by the new structure of the array, as shown in the following code example;

```
struct SS
{
  int x[N];
  int y[N];
};
```

```
int top(SS *a, int b[4][6], SS &c) {...}

set_directive_array_reshape top b -type complete -dim 0
set_directive_interface -mode ap_memory top b[0]
```

See Also

- pragma HLS array_reshape
- set_directive_array_partition

set_directive_bind_op

Description

TIP: *This pragma or directive does not play a part in DSP multi-operation matching (MULADD, AMA, ADDMUL and corresponding Subtraction operations). In fact the use of BIND_OP for assigning an operator to DSP can prevent the HLS compiler from matching multi-operation expressions. Refer to DSP Multi-Operation Matching for more information.*

Vitis HLS implements the operations in the code using specific implementations. The `set_directive_bind_op` command specifies that for a specified variable, an operation (`mul`, `add`, `sub`) should be mapped to a specific device resource for implementation (`impl`) in the RTL. If this command is not specified, Vitis HLS automatically determines the resource to use.

For example, to indicate that a specific multiplier operation (`mul`) is implemented in the device fabric rather than a DSP, you can use the `set_directive_bind_op` command.

You can also specify the latency of the operation using the `-latency` option.

IMPORTANT! *To use the `-latency` option, the operation must have an available multi-stage implementation. The HLS tool provides a multi-stage implementation for all basic arithmetic operations (add, subtract, multiply, and divide), and all floating-point operations.*

Syntax

```
set_directive_bind_op [OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the variable.
- `<variable>` is the variable to be assigned. The variable in this case is one that is assigned the result of the operation that is the target of this directive.

Options

- **`-op <value>`:** Defines the operation to bind to a specific implementation resource. Supported functional operations include: `mul`, `add`, `sub`

Supported floating point operations include: `fadd`, `fsub`, `fdiv`, `fexp`, `flog`, `fmul`, `frsqrt`, `frecip`, `fsqrt`, `dadd`, `dsub`, `ddiv`, `dexp`, `dlog`, `dmul`, `drsqrt`, `drecip`, `dsqrt`, `hadd`, `hsub`, `hdiv`, `hmul`, and `hsqrt`

TIP: *Floating-point operations include single precision (f), double-precision (d), and half-precision (h).*

- **`-impl <value>`:** Defines the implementation to use for the specified operation.Supported implementations for functional operations include `fabric` and `dsp`.Supported implementations for floating point operations include: `fabric`, `meddsp`, `fulldsp`, `maxdsp`, and `primitivedsp`.

  ***Note*:** `primitivedsp` is only available on Versal devices.

- **`-latency <int>`:** Defines the default latency for the implementation of the operation. The valid latency varies according to the specified `op` and `impl`. The default is -1, which lets Vitis HLS choose the latency.The tables below reflect the supported combinations of operation, implementation, and latency.

Table 44: **Supported Combinations of Functional Operations, Implementation, and Latency**

Operation	Implementation	Min Latency	Max Latency
add	fabric	0	4
add	dsp	0	4
mul	fabric	0	4
mul	dsp	0	4
sub	fabric	0	4
sub	dsp	0	0

TIP: *Comparison operators, such as `dcmp`, are implemented in LUTs and cannot be implemented outside of the fabric, or mapped to DSPs, and so are not configurable with the `config_op` or `bind_op` commands.*

Table 45: **Supported Combinations of Floating Point Operations, Implementation, and Latency**

Operation	Implementation	Min Latency	Max Latency
fadd	fabric	0	13
fadd	fulldsp	0	12
fadd	primitivedsp	0	3
fsub	fabric	0	13
fsub	fulldsp	0	12
fsub	primitivedsp	0	3
fdiv	fabric	0	29
fexp	fabric	0	24

Table 45: **Supported Combinations of Floating Point Operations, Implementation, and Latency** *(cont'd)*

Operation	Implementation	Min Latency	Max Latency
fexp	meddsp	0	21
fexp	fulldsp	0	30
flog	fabric	0	24
flog	meddsp	0	23
flog	fulldsp	0	29
fmul	fabric	0	9
fmul	meddsp	0	9
fmul	fulldsp	0	9
fmul	maxdsp	0	7
fmul	primitivedsp	0	4
fsqrt	fabric	0	29
frsqrt	fabric	0	38
frsqrt	fulldsp	0	33
frecip	fabric	0	37
frecip	fulldsp	0	30
dadd	fabric	0	13
dadd	fulldsp	0	15
dsub	fabric	0	13
dsub	fulldsp	0	15
ddiv	fabric	0	58
dexp	fabric	0	40
dexp	meddsp	0	45
dexp	fulldsp	0	57
dlog	fabric	0	38
dlog	meddsp	0	49
dlog	fulldsp	0	65
dmul	fabric	0	10
dmul	meddsp	0	13
dmul	fulldsp	0	13
dmul	maxdsp	0	14
dsqrt	fabric	0	58
drsqrt	fulldsp	0	111
drecip	fulldsp	0	36
hadd	fabric	0	9
hadd	meddsp	0	12
hadd	fulldsp	0	12
hsub	fabric	0	9
hsub	meddsp	0	12

Table 45: **Supported Combinations of Floating Point Operations, Implementation, and Latency** *(cont'd)*

Operation	Implementation	Min Latency	Max Latency
hsub	fulldsp	0	12
hdiv	fabric	0	16
hmul	fabric	0	7
hmul	fulldsp	0	7
hmul	maxdsp	0	9
hsqrt	fabric	0	16

Examples

In the following example, a two-stage pipelined multiplier using fabric logic is specified to implement the multiplication for variable `<c>` of the function `foo`.

```
int foo (int a, int b) {
int c, d;
c = a*b;
d = a*c;
return d;
}
```

And the `set_directive` command is as follows:

```
set_directive_bind_op -op mul -impl fabric -latency 2 "foo" c
```

TIP: *The HLS tool selects the core to use for variable `<d>`.*

See Also

- pragma HLS bind_op
- set_directive_bind_storage

set_directive_bind_storage

Description

The `set_directive_bind_storage` command assigns a variable (array, or function argument) in the code to a specific memory type (`type`) in the RTL. If the command is not specified, the Vitis HLS tool determines the memory type to assign. The HLS tool implements the memory using specified implementations (`impl`) in the hardware. For example, you can use the `set_directive_bind_storage` command to specify which type of memory, and which implementation to use for an array variable. Also, this allows you to control whether the array is implemented as a single or a dual-port RAM.

IMPORTANT! *This feature is important for arrays on the top-level function interface, because the memory type associated with the array determines the number and type of ports needed in the RTL, as discussed in Arrays on the Interface. However, for variables assigned to top-level function arguments you must assign the memory type and implementation using the* `-storage_type` *and* `-storage_impl` *options of the INTERFACE pragma or directive.*

You can use the `-latency` option to specify the latency of the implementation. For block RAMs on the interface, the `-latency` option allows you to model off-chip, non-standard SRAMs at the interface, for example supporting an SRAM with a latency of 2 or 3. For internal operations, the `-latency` option allows the operation to be implemented using more pipelined stages. These additional pipeline stages can help resolve timing issues during RTL synthesis.

IMPORTANT! *To use the* `-latency` *option, the operation must have an available multi-stage implementation. The HLS tool provides a multi-stage implementation for all block RAMs.*

For best results, AMD recommends that you use `-std=c99` for C and `-fno-builtin` for C and C++. To specify the C compile options, such as `-std=c99`, use the Tcl command `add_files` with the `-cflags` option. Alternatively, select the **Edit CFLAGs** button in the Project Settings dialog box as described in Creating an HLS Component.

Syntax

```
set_directive_bind_storage [OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the variable.
- `<variable>` is the variable to be assigned.

TIP: *If the variable is an argument of a top-level function, then use the* `-storage_type` *and* `-storage_impl` *options of the INTERFACE pragma or directive.*

Options

- `-type`: Defines the type of memory to bind to the specified variable. Supported types include: `fifo`, `ram_1p`, `ram_1wnr`, `ram_2p`, `ram_s2p`, `ram_t2p`, `rom_1p`, `rom_2p`, and `rom_np`.

Table 46: **Storage Types**

Type	Description
FIFO	A FIFO. Vitis HLS determines how to implement this in the RTL, unless the `-impl` option is specified.
RAM_1P	A single-port RAM. Vitis HLS determines how to implement this in the RTL, unless the `-impl` option is specified.
RAM_1WNR	A RAM with 1 write port and N read ports, using N banks internally.
RAM_2P	A dual-port RAM that allows read operations on one port and both read and write operations on the other port.

Table 46: **Storage Types** *(cont'd)*

Type	Description
RAM_S2P	A dual-port RAM that allows read operations on one port and write operations on the other port.
RAM_T2P	A true dual-port RAM with support for both read and write on both ports.
ROM_1P	A single-port ROM. Vitis HLS determines how to implement this in the RTL, unless the `-impl` option is specified.
ROM_2P	A dual-port ROM.
ROM_NP	A multi-port ROM.

TIP: *If you specify a single port RAM for a PIPO, then the binder will typically allocate a merged-PIPO, with only one bank, and reader and writer accessing different halves of the bank. The info message reported by the HLS compiler says "Implementing PIPO using a single memory for all blocks." This is often cheaper for small PIPOs, where all banks can share a single RAM block. However, if you specify a dual-port RAM for a PIPO to get higher bandwidth and the scheduler uses both ports either in the producer or in the consumer, then the binder will typically allocate a split-PIPO, where the producer will use 1 port and the consumer 2 ports of different banks. The info message reported by the HLS compiler in this case says "Implementing PIPO using a separate memory for each block."*

- `-impl <value>`: Defines the implementation for the specified memory type. Supported implementations include: `bram`, `bram_ecc`, `lutram`, `uram`, `uram_ecc`, `srl`, `memory`, and `auto` as described below.

Table 47: **Supported Implementation**

Name	Description
MEMORY	Generic memory for FIFO, lets the Vivado tool choose the implementation.
URAM	UltraRAM resource
URAM_ECC	UltraRAM with ECC
SRL	Shift Register Logic resource
LUTRAM	Distributed RAM resource
BRAM	Block RAM resource
BRAM_ECC	Block RAM with ECC
AUTO	Vitis HLS automatically determine the implementation of the variable.

TIP: *URAMs do not support a read-first output write_mode (unlike BRAMs) when a read and a write to the same address is mapped to the same memory port. BRAM supports the following write-modes:* `write thru`, `read first`, `no change`. *URAM only supports* `no change`. *Vitis HLS will issue the following warning message when it cannot schedule memory operations in the same cycle on a URAM port:*

```
Usage of URAM can potentially cause worse II as Vitis HLS does not
exploit
read-first mode for URAMs. Consider using BRAMs instead.
```

Table 48: **Supported Implementations by FIFO/RAM/ROM**

Type	Command/Pragma	Scope	Supported Implementations
FIFO	`bind_storage`[1]	local	**AUTO**, BRAM, LUTRAM, URAM, MEMORY, SRL
FIFO	`config_storage`	global	**AUTO**, BRAM, LUTRAM, URAM, MEMORY, SRL
RAM* | ROM*	`bind_storage`	local	**AUTO** BRAM, BRAM_ECC, LUTRAM, URAM, URAM_ECC
RAM* | ROM*	`config_storage`[2]	global	N/A
RAM_1P	`set_directive_interface s_axilite -storage_impl`	local	**AUTO**, BRAM, URAM
	`config_interface -m_axi_buffer_impl`	global	AUTO, **BRAM**, LUTRAM, URAM

Notes:

1. When no implementation is specified the directive uses AUTOSRL behavior as a default. However, this value cannot be specified.
2. `config_storage` only supports FIFO types.

- `-latency <int>`: Defines the default latency for the binding of the storage type to the implementation. The valid latency varies according to the specified `type` and `impl`. The default is -1, which lets Vitis HLS choose the latency.

Table 49: **Supported Combinations of Memory Type, Implementation, and Latency**

Type	Implementation	Min Latency	Max Latency
FIFO	BRAM	1	4
FIFO	LUTRAM	1	4
FIFO	MEMORY	1	4
FIFO	SRL	1	4
FIFO	URAM	1	4
RAM_1P	AUTO	1	3
RAM_1P	BRAM	1	3
RAM_1P	LUTRAM	1	3
RAM_1P	URAM	1	3
RAM_1WNR	AUTO	1	3
RAM_1WNR	BRAM	1	3
RAM_1WNR	LUTRAM	1	3
RAM_1WNR	URAM	1	3
RAM_2P	AUTO	1	3
RAM_2P	BRAM	1	3
RAM_2P	LUTRAM	1	3
RAM_2P	URAM	1	3
RAM_S2P	BRAM	1	3

Table 49: **Supported Combinations of Memory Type, Implementation, and Latency** *(cont'd)*

Type	Implementation	Min Latency	Max Latency
RAM_S2P	BRAM_ECC	1	3
RAM_S2P	LUTRAM	1	3
RAM_S2P	URAM	1	3
RAM_S2P	URAM_ECC	1	3
RAM_T2P	BRAM	1	3
RAM_T2P	URAM	1	3
ROM_1P	AUTO	1	3
ROM_1P	BRAM	1	3
ROM_1P	LUTRAM	1	3
ROM_2P	AUTO	1	3
ROM_2P	BRAM	1	3
ROM_2P	LUTRAM	1	3
ROM_NP	BRAM	1	3
ROM_NP	LUTRAM	1	3

IMPORTANT! *Any combinations of memory type and implementation that are not listed in the prior table are not supported by* `set_directive_bind_storage`.

Examples

In the following example, the `coeffs[128]` variable is an argument to the function `func1`. The directive specifies that `coeffs` uses a single port RAM implemented on a BRAM core from the library.

```
set_directive_bind_storage -impl bram "func1" coeffs RAM_1P
```

TIP: *The ports created in the RTL to access the values of* `coeffs` *are defined in the RAM_1P core.*

See Also

- pragma HLS bind_storage

set_directive_cache

Description

In FPGA, non-burst accesses to the DDR memory are very expensive and can impact the overall performance of the design. Hence, it is important to devise a scheme that reduces the time needed to access the necessary information. An efficient solution is to re-write the code or use manual burst, but if that does not work then another solution might be to use a cache memory.

Cache provides a temporary storage area in the M_AXI adapter so the design can more quickly retrieve data. The effectiveness of the cache mechanism is measured in a hit/miss ratio and it is based on a property of computer programs called locality of reference, or the tendency of a program to access the same set of memory locations repetitively over a short period of time. It suggests that when a particular memory block is accessed, it is likely to be accessed again in the near future, along with memory located in close proximity. For instance, when executing consecutive instructions in memory, the next set of instructions to be accessed will most likely be within a contiguous block of data nearby.



TIP: *You control the cache implementation resources using the* `config_interface -m_axi_cache_impl` *command.*

Syntax

```
set_directive_cache <location> port=<name> lines=<value> depth=<value>
```

Where:

- `<location>`: Specifies the function where the specified ports can be found. This is the top function.
- `port=<name>`: Specifies the port to add cache to.
- `lines=<value>`: Indicates the number of cache lines. The number of lines can be specified as 1, which indicates a single cache line, or a value greater than 1 expressed as a power of 2, indicating multiple cache lines
- `depth=<value>`: Specifies the size of each line in words. The depth must be specified as a power of 2, and indicates the size in words of the pointer datatype for each line. This is an optional value, and defaults to the max burst length if not specified. The max burst length defaults to 16, but can be globally specified by the `config_interface -m_axi_max_read_burst_length` command, or by the `set_directive_interface` command for specific interfaces.

Limitations

The CACHE directive or pragma has the following limitations:

- Cache is only supported for read-only port

- The cache is implemented as a Single port, Single way cache.
- Cache is not supported for multiple ports connected to the same bundle

Example

The following example shows a design where overlapping access will cause the burst to fail. Using the CACHE pragma or directive will improve the performance of the design.

```
extern "C" {
  void dut(
    const double *in,  // Read-Only Vector 1
    double *out,       // Output Result
    int size                 // Size in integer
  )
#pragma HLS INTERFACE m_axi port=in  bundle=aximm depth = 1026
#pragma HLS INTERFACE m_axi port=out bundle=aximm depth = 1024
#pragma HLS cache port=in lines=8 depth=128
for(int i = 0; i < size; i++)
    {
      out[i] = in[i] + in[i + 1];
    }
}
```

See Also

- pragma HLS interface
- config_interface
- pragma HLS cache

set_directive_dataflow

Description

Specifies that dataflow optimization be performed on the functions or loops, improving the concurrency of the RTL implementation.

All operations are performed sequentially in a C/C++ description. In the absence of any directives that limit resources (such as `set_directive_allocation`), Vitis HLS seeks to minimize latency and improve concurrency. Data dependencies can limit this. For example, functions or loops that access arrays must finish all read/write accesses to the arrays before they complete. This prevents the next function or loop that consumes the data from starting operation.

It is possible for the operations in a function or loop to start operation before the previous function or loop completes all its operations. When the DATAFLOW optimization is specified, the HLS tool analyzes the dataflow between sequential functions or loops and creates channels (based on ping-pong RAMs or FIFOs) that allow consumer functions or loops to start operation before the producer functions or loops have completed. This allows functions or loops to operate in parallel, which decreases latency and improves the throughput of the RTL.

TIP: *The `config_dataflow` command specifies the default memory channel and FIFO depth used in DATAFLOW optimization.*

If no initiation interval (number of cycles between the start of one function or loop and the next) is specified, Vitis HLS attempts to minimize the initiation interval and start operation as soon as data is available.

For the DATAFLOW optimization to work, the data must flow through the design from one task to the next. The following coding styles prevent the HLS tool from performing the `DATAFLOW` optimization. Refer to Limitations of Control-Driven Task-Level Parallelism for additional details.

- Single-producer-consumer violations
- Feedback between tasks
- Conditional execution of tasks
- Loops with multiple exit conditions

IMPORTANT! *If any of these coding styles are present, the HLS tool issues a message and does not perform DATAFLOW optimization.*

Finally, the DATAFLOW optimization has no hierarchical implementation. If a sub-function or loop contains additional tasks that might benefit from the DATAFLOW optimization, you must apply the optimization to the loop, the sub-function, or inline the sub-function.

Syntax

```
set_directive_dataflow <location> -disable_start_propagation
```

- `<location>` is the location (in the format `function[/label]`) at which dataflow optimization is to be performed.
- `-disable_start_propagation` disables the creation of a start FIFO used to propagate a start token to an internal process. Such FIFOs can sometimes be a bottleneck for performance.

Examples

Specifies dataflow optimization within function `foo`.

```
set_directive_dataflow foo
```

See Also

- pragma HLS dataflow
- set_directive_allocation
- config_dataflow

set_directive_dependence

Description

Vitis HLS detects dependencies within loops: dependencies within the same iteration of a loop are loop-independent dependencies, and dependencies between different iterations of a loop are loop-carried dependencies.

These dependencies are impacted when operations can be scheduled, especially during function and loop pipelining.

- **Loop-independent dependence:** The same element is accessed in a single loop iteration.

```
for (i=1;i<N;i++) {
 A[i]=x;
 y=A[i];
}
```

- **Loop-carried dependence:** The same element is accessed from a different loop iteration.

```
for (i=1;i<N;i++) {
 A[i]=A[i-1]*2;
}
```

Under certain circumstances, such as variable dependent array indexing or when an external requirement needs to be enforced (for example, two inputs are never the same index), the dependence analysis might be too conservative and fail to filter out false dependencies. The `set_directive_dependence` command allows you to explicitly define the dependencies and eliminate a false dependence.

IMPORTANT! *Specifying a false dependency when the dependency is not false can result in incorrect hardware. Ensure dependencies are correct (true or false) before specifying them.*

Syntax

```
set_directive_dependence -dependent <arg> [OPTIONS] <location>
```

- `-dependent (true | false)`: This argument should be specified to indicate whether a dependence is `true` and needs to be enforced, or is `false` and should be removed. However, when not specified, the tool will return a warning that the value was not specified and will assume a value of `false`.

- `<location>`: The location in the code, specified as `function[/label]`, where the dependence is defined.

Options

- `-class (array | pointer)`: Specifies a class of variables in which the dependence needs clarification. This is mutually exclusive with the `-variable` option.
- `-dependent (true | false)`: Specify if a dependence needs to be enforced (true) or removed (false).
- `-direction (RAW | WAR | WAW)`:

  Note: Relevant only for loop-carried dependencies.

  Specifies the direction for a dependence:

  - `RAW` **(Read-After-Write - true dependence):** The write instruction uses a value used by the read instruction.
  - `WAR` **(Write-After-Read - anti dependence):** The read instruction gets a value that is overwritten by the write instruction.
  - `WAW` **(Write-After-Write - output dependence):** Two write instructions write to the same location, in a certain order.
- `-distance <integer>`:

  Note: Relevant only for loop-carried dependencies where `-dependent` is set to `true`.

  Specifies the inter-iteration distance for array access.
- `-type (intra | inter)`: Specifies whether the dependence is:
  - Within the same loop iteration (`intra`), or
  - Between different loop iterations (`inter`) (default).
- `-variable <variable>`: Defines a specific variable to apply the dependence directive. Mutually exclusive with the `-class` option.



IMPORTANT! *You cannot specify a `dependence` for function arguments that are bundled with other arguments in an `m_axi` interface. This is the default configuration for `m_axi` interfaces on the function. You also cannot specify a dependence for an element of a struct, unless the struct has been disaggregated.*

Examples

Removes the dependence between `Var1` in the same iterations of `loop_1` in function `func`.

```
set_directive_dependence -variable Var1 -type intra \
-dependent false func/loop_1
```

The dependence on all arrays in `loop_2` of function `func` informs Vitis HLS that all reads must happen *after* writes in the same loop iteration.

```
set_directive_dependence -class array -type intra \
-dependent true -direction RAW func/loop_2
```

See Also

- pragma HLS dependence
- set_directive_disaggregate
- set_directive_pipeline

set_directive_disaggregate

Description

The `set_directive_disaggregate` command lets you deconstruct a `struct` variable into its individual elements. The number and type of elements created are determined by the contents of the struct itself.

IMPORTANT! *Structs used as arguments to the top-level function are aggregated by default, but can be disaggregated with this directive or pragma. Refer to AXI4-Stream Interfaces for important information about disaggregating structs associated with streams.*

Syntax

```
set_directive_disaggregate <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) where the variable to disaggregate is found.
- `<variable>` specifies the struct variable name.

Options

This command has no options.

Example 1

The following example shows the struct variable `a` in function `top` will be disaggregated:

```
set_directive_disaggregate top a
```

Example 2

Disaggregated structs can be addressed in your code by the using standard C/C++ coding style as shown below. Notice the different methods for accessing the pointer element (a) versus the reference element (c);

```
struct SS
{
  int x[N];
  int y[N];
};

int top(SS *a, int b[4][6], SS &c) {

set_directive_disaggregate top a
set_directive_interface -mode s_axilite top a->x
set_directive_interface -mode s_axilite top a->y

set_directive_disaggregate top c
set_directive_interface -mode ap_memory top c.x
set_directive_interface -mode ap_memory top c.y
```

Example 3

The following example shows the `Dot` struct containing the `RGB` struct as an element. If you apply `set_directive_disaggregate` to variable `Arr`, then only the top-level `Dot` struct is disaggregated.

```
struct Pixel {
char R;
char G;
char B;
};

struct Dot {
Pixel RGB;
unsigned Size;
};

#define N 1086
void DUT(Dot Arr[N]) {
...
}

set_directive_disaggregate DUT Arr
```

If you want to disaggregate the whole struct, `Dot` and `RGB`, then you can assign the `set_directive_disaggregate` as shown below.

```
void DUT(Dot Arr[N]) {
#pragma HLS disaggregate variable=Arr->RGB
...
}

set_directive_disaggregate DUT Arr->RGB
```

The results in this case will be:

```
void DUT(char Arr_RGB_R[N], char Arr_RGB_G[N], char Arr_RGB_B[N], unsigned
Arr_Size[N]) {
...
}
```

See Also

- pragma HLS disaggregate
- set_directive_aggregate

set_directive_expression_balance

Description

Sometimes C/C++ code is written with a sequence of operations, resulting in a long chain of operations in RTL. With a small clock period, this can increase the latency in the design. By default, the Vitis HLS tool rearranges the operations using associative and commutative properties. As described in Optimizing Logic Expressions, this rearrangement creates a balanced tree that can shorten the chain, potentially reducing latency in the design at the cost of extra hardware.

Expression balancing rearranges operators to construct a balanced tree and reduce latency.

- For integer operations expression balancing is on by default but may be disabled.
- For floating-point operations, expression balancing is off by default but may be enabled.

The `set_directive_expression_balance` command allows this expression balancing to be turned off, or on, within a specified scope.

Syntax

```
set_directive_expression_balance [OPTIONS] <location>
```

- `<location>` is the location (in the format `function[/label]`) where expression balancing should be disabled, or enabled.

Options

- `-off`: Turns off expression balancing at the specified location. Specifying the `set_directive_expression_balance` command enables expression balancing in the specified scope. Adding the -off option disables it.

Examples

Disables expression balancing within function `My_Func`.

```
set_directive_expression_balance -off My_Func
```

Explicitly enables expression balancing in function `My_Func2`.

```
set_directive_expression_balance My_Func2
```

See Also

- pragma HLS expression_balance

set_directive_function_instantiate

Description

By default:

- Functions remain as separate hierarchy blocks in the RTL, or are decomposed (inlined) into higher-level functions.
- All instances of a function, at the same level of hierarchy, uses the same RTL implementation (block).

The `set_directive_function_instantiate` command is used to create a unique RTL implementation for each instance of a function, allowing each instance to be optimized around a specific argument or variable.

By default, the following code results in a single RTL implementation of function `func_sub` for all three instances, or if `func_sub` is a small function it is inlined into function `func`.

TIP: *By default, the Vitis HLS tool automatically inlines small functions. This is true even for function instantiations. Using the `set_directive_inline off` option can be used to prevent this automatic inlining.*

```
char func_sub(char inval, char incr)
{
  return inval + incr;
}
void func(char inval1, char inval2, char inval3,
  char *outval1, char *outval2, char * outval3)
```

```
{
  *outval1 = func_sub(inval1, 1);
  *outval2 = func_sub(inval2, 2);
  *outval3 = func_sub(inval3, 3);
}
```

Using the directive as shown in the example section below results in three versions of function `func_sub`, each independently optimized for variable `incr`.

Syntax

```
set_directive_function_instantiate <location> <variable>
```

- `<location>` is the location (in the format `function[/region label]`) where the instances of a function are to be made unique.
- `<variable>` specifies the function argument to be specified as a constant in the various function instantiations.

Options

This command has no options.

Examples

For the example code shown above, the following Tcl (or pragma placed in function `func_sub`) allows each instance of function `func_sub` to be independently optimized with respect to input `incr`.

```
set_directive_inline -off func_sub
set_directive_function_instantiate func_sub incr
```

See Also

- pragma HLS function_instantiate
- set_directive_allocation
- set_directive_inline

set_directive_inline

Description

Removes a function as a separate entity in the RTL hierarchy. After inlining, the function is dissolved into the calling function, and no longer appears as a separate level of hierarchy.

IMPORTANT! *Inlining a function also dissolves any pragmas or directives applied to that function. In Vitis HLS, any pragmas or directives applied to the inlined function are ignored.*

In some cases, inlining a function allows operations within the function to be shared and optimized more effectively with the calling function. However, an inlined function cannot be shared or reused, so if the parent function calls the inlined function multiple times, this can increase the area and resource utilization.

By default, inlining is only performed on the next level of function hierarchy.

Syntax

```
set_directive_inline [OPTIONS] <location>
```

- `<location>` is the location (in the format `function[/label]`) where inlining is to be performed.

Options

- **`-off`:** By default, Vitis HLS performs inlining of smaller functions in the code. Using the `-off` option disables inlining for the specified function.
- **`-recursive`:** By default, only one level of function inlining is performed. The functions within the specified function are not inlined. The `-recursive` option inlines all functions recursively within the specified function hierarchy.

> **IMPORTANT!** *Recursive inlining of functions in a dataflow region can result in a single function in the region which does not meet the conditions of dataflow. In this case the `INLINE -recursive` pragma or directive is ignored.*

Examples

The following example inlines function `func_sub1`, but no sub-functions called by `func_sub1`.

```
set_directive_inline func_sub1
```

This example inlines function `func_sub1`, recursively down the hierarchy, except function `func_sub2`:

```
set_directive_inline -recursive func_sub1
set_directive_inline -off func_sub2
```

See Also

- pragma HLS inline
- set_directive_allocation

set_directive_interface

Description

`set_directive_interface` is only supported for use on the top-level function, and cannot be used for sub-functions of the HLS component. The INTERFACE pragma or directive specifies how RTL ports are created from the function arguments during interface synthesis, as described in Defining Interfaces. The Vitis HLS tool automatically determines the I/O protocol used by any sub-functions.

Ports in the RTL implementation are derived from the data type and direction of the arguments of the top-level function and function return, the `flow_target` for the HLS component, the default interface configuration settings as specified by config_interface, and by this directive. Each function argument can be specified to have its own I/O protocol (such as valid handshake or acknowledge handshake).



TIP: *Global variables required on the interface must be explicitly defined as an argument of the top-level function as described in Global Variables. If a global variable is accessed, but all read and write operations are local to the design, the resource is created in the design. There is no need for an I/O port in the RTL.*

The interface also defines the execution control protocol of the HLS component as described in Block-Level Control Protocols. The control protocol controls when the HLS component (or block) starts execution, and when the block completes operation, is idle and ready for new inputs.

Syntax

```
set_directive_interface [OPTIONS] <location> <port>
```

- `<location>` is the location (in the format `function[/label]`) where the function interface or registered output is to be specified.
- `<port>` is the parameter (function argument) for which the interface has to be synthesized. The port name is not required when block control modes are specified: `ap_ctrl_chain`, `ap_ctrl_hs`, or `ap_ctrl_none`.

Options



TIP: *Many of the options specified below have global values that are defined in the config_interface command. Set local values for the interface defined here to override the global values.*

- **`-address_mode <byte|word>` Specifies if addressing should use byte or word increment.`mode=<mode>`:**

  The supported modes, and how the tool implements them in RTL, can be broken down into three categories as follows:

  1. Port-Level Protocols:

- `ap_none`: No port protocol. The interface is a simple data port.
- `ap_vld`: Implements the data port with an associated `valid` signal to indicate when the data is valid for reading or writing.
- `ap_ack`: Implements the data port with an associated `acknowledge` signal to acknowledge that the data was read or written.
- `ap_hs`: Implements the data port with both `valid` and `acknowledge` signals to provide a two-way handshake to indicate when the data is valid for reading and writing and to acknowledge that the data was read or written.
- `ap_ovld`: Implements the output data port with an associated `valid` signal to indicate when the data is valid for reading or writing.

  TIP: *For `ap_ovld` Vitis HLS implements the input argument or the input half of any read/write arguments with mode `ap_none`.*

- `ap_memory`: Implements array arguments as a standard RAM interface. If you use the RTL design in Vivado IP integrator the interface is composed of separate ports.
- `ap_fifo`: Implements the port with a standard FIFO interface using data input and output ports with associated active-Low FIFO `empty` and `full` ports.

  Note: You can only use the `ap_fifo` interface on read arguments or write arguments. `ap_fifo` mode does not support bidirectional read/write arguments.

2. AXI Interface Protocols:
   - `s_axilite`: Implements the port as an AXI4-Lite interface. The tool produces an associated set of C driver files when exporting the generated RT for the HLS component.
   - `m_axi`: Implements the port as an AXI4 interface. You can use the `config_interface -m_axi_addr64` command to specify either 32-bit (default) or 64-bit address ports and to control any address offset.
   - `axis`: Implements the port as an AXI4-Stream interface.
3. Block-Level Control Protocols:
   - `address_mode (byte|word)`: Specifies if addressing should use byte or word increment.
   - `ap_ctrl_chain`: Implements a set of block-level control ports to `start` the design operation, `continue` operation, and indicate when the design is `idle`, `done`, and `ready` for new input data.
   - `ap_ctrl_hs`: Implements a set of block-level control ports to `start` the design operation and to indicate when the design is `idle`, `done`, and `ready` for new input data.
   - `ap_ctrl_none`: No block-level I/O protocol.

Note: Using the `ap_ctrl_none` mode might prevent the design from being verified using C/RTL co-simulation.

- **`-bundle <string>`:**

  By default, the HLS tool groups (or bundles) function arguments that are compatible into a single interface port in the RTL code. All interface ports with compatible options, such as `mode`, `offset`, and `bundle`, are grouped into a single interface port.

  TIP: *This default can be changed using the `config_interface -m_axi_auto_max_ports` command.*

  This `-bundle <string>` option lets you define bundles to group ports together, overriding the default behavior. The `<string>` specifies the bundle name. The port name used in the generated RTL code is derived automatically from a combination of the `mode` and `bundle`, or is named as specified by `name`.

  IMPORTANT! *`bundle` names must be specified using lower-case characters.*

- **`-channel <string>`:** To enable multiple channels on an `m_axi` interface specify the channel ID. Multiple `m_axi` interfaces can be combined into a single `m_axi` adapter using separate channel IDs.
- **`-clock <string>`:** By default, the AXI4-Lite interface clock is the same clock as the system clock. This option is used to set specify a separate clock for an AXI4-Lite interface. If the `-bundle` option is used to group multiple top-level function arguments into a single AXI4-Lite interface, the clock option need only be specified on one of bundle members.
- **`-depth <int>`:** Specifies the maximum number of samples for the test bench to process. This setting indicates the maximum size of the FIFO needed in the verification adapter that the HLS tool creates for RTL co-simulation.

  TIP: *While `depth` is usually an option, it is needed to specify the size of pointer arguments for RTL co-simulation.*

- **`-interrupt <int>`:** Only used by `ap_vld/ap_hs`. This option enables the I/O to be managed in interrupt, by creating the corresponding bits in the `ISR` and `IER` in the `s_axilite` register file. The integer value N=16..31 specifies the bit position in both registers (by default assigned contiguously from 16).
- **`-latency <value>`:** This option can be used on `ap_memory` and M_AXI interfaces.
  - In an `ap_memory` interface, the interface option specifies the read latency of the RAM resource driving the interface. By default, a read operation of 1 clock cycle is used. This option allows an external RAM with more than 1 clock cycle of read latency to be modeled.

  - In an M_AXI interface, this option specifies the expected latency of the AXI4 interface, allowing the design to initiate a bus request <value> number of cycles (latency) before the read or write is expected. If this figure it too low, the design will be ready too soon and might stall waiting for the bus. If this figure is too high, bus access might be idle waiting on the design to start the access.

- **`-max_read_burst_length <int>`:** For use with the M_AXI interface, this option specifies the maximum number of data values read during a burst transfer. Refer to AXI Burst Transfers for more information.

- **`-max_write_burst_length <int>`:** For use with the M_AXI interface, this option specifies the maximum number of data values written during a burst transfer.

- **`-max_widen_bitwidth <int>`:** Specifies the maximum bit width available for the interface when automatically widening the interface. This overrides the global value specified by the `config_interface -m_axi_max_bitwidth` command.

- **`-name <string>`:** Specifies a name for the port which will be used in the generated RTL. By default the port name is derived automatically from a combination of the `mode` and `bundle` unless specified by `name`.

- **`-num_read_outstanding <int>`:** For use with the M_AXI interface, this option specifies how many read requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, and a FIFO of size:

  ```
  num_read_outstanding*max_read_burst_length*word_size
  ```

- **`-num_write_outstanding <int>`:** For use with the M_AXI interface, this option specifies how many write requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design, and a FIFO of size:

  ```
  num_write_outstanding*max_write_burst_length*word_size
  ```

- **`-offset <string>`:** Controls the address offset in AXI4-Lite (`s_axilite`) and AXI4 memory mapped (`m_axi`) interfaces for the specified port.

  - In an `s_axilite` interface, `<string>` specifies the address in the register map.
  - In an `m_axi` interface this option overrides the global option specified by the `config_interface -m_axi_offset` option, and `<string>` is specified as:
    - `off`: Do not generate an offset port.
    - `direct`: Generate a scalar input offset port.
    - `slave`: Generate an offset port and automatically map it to an AXI4-Lite slave interface. This is the default offset.

- **`-register`:** Registers the signal and any associated protocol signals and instructs the signals to persist until at least the last cycle of the function execution. This option applies to the following scalar interfaces for the top-level function:

- `s_axilite`
- `ap_fifo`
- `ap_none`
- `ap_hs`
- `ap_ack`
- `ap_vld`
- `ap_ovld`

> **TIP:** *The use of the register option on the return port of the function (`port=return`) is not supported. Instead use the LATENCY pragma or directive:*
>
> ```
> #pragma HLS LATENCY min=1 max=1
> ```

- **`-register_mode (both|forward|reverse|off)`:** This option applies to AXI4-Stream interfaces, and specifies if registers are placed on the forward path (`TDATA` and `TVALID`), the reverse path (`TREADY`), on both paths (`TDATA`, `TVALID`, and `TREADY`), or if none of the ports signals are to be registered (`off`). The default is `both`. AXI4-Stream side-channel signals are considered to be data signals and are registered whenever the `TDATA` is registered.
- **`-storage_impl=<impl>`:** For use with `s_axilite` only. This options defines a storage implementation to assign to the interface. Supported implementation values include `auto`, `bram`, and `uram`. The default is **auto**.

  > **TIP:** *`uram` is a synchronous memory with only a single clock for two ports. Therefore `uram` cannot be specified for an `s_axilite` adapter with a second clock.*

- **`-storage_type=<type>`:** For use with the `ap_memory` interface only. This options defines a storage type (for example, RAM_T2P) to assign to the variable. Supported types include: `ram_1p`, `ram_1wnr`, `ram_2p`, `ram_s2p`, `ram_t2p`, `rom_1p`, `rom_2p`, and `rom_np`.

  > **TIP:** *This can also be specified using the BIND_STORAGE pragma or directive for the variable for objects that are not defined on the interface.*

Examples

Turns off function-level handshakes for function `func`.

```
set_directive_interface -mode ap_ctrl_none func
```

Argument `InData` in function `func` is specified to have a `ap_vld` interface and the input should be registered.

```
set_directive_interface -mode ap_vld -register func InData
```

See Also

- pragma HLS interface
- set_directive_bind_storage

set_directive_latency

Description

Specifies a maximum or minimum latency value, or both, on a function, loop, or region.

Vitis HLS always aims for minimum latency. The behavior of the tool when minimum and maximum latency values are specified is as follows:

- Latency is less than the minimum: If Vitis HLS can achieve less than the minimum specified latency, it extends the latency to the specified value, potentially enabling increased sharing.
- Latency is greater than the minimum: The constraint is satisfied. No further optimizations are performed.
- Latency is less than the maximum: The constraint is satisfied. No further optimizations are performed.
- Latency is greater than the maximum: If Vitis HLS cannot schedule within the maximum limit, it increases effort to achieve the specified constraint. If it still fails to meet the maximum latency, it issues a warning. Vitis HLS then produces a design with the smallest achievable latency.



TIP: *You can also use the LATENCY pragma or directive to limit the efforts of the tool to find an optimum solution. Specifying latency constraints for scopes within the code: loops, functions, or regions, reduces the possible solutions within that scope and can improve tool runtime. Refer to Improving Synthesis Runtime and Capacity for more information.*

If the intention is to limit the total latency of all loop iterations, the latency directive should be applied to a region that encompasses the entire loop, as in this example: `set_directive Region_All_Loop_A`

```
Region_All_Loop_A: {
Loop_A: for (i=0; i<N; i++)
  {
  ..Loop Body...
  }
}
```

In this case, even if the loop is unrolled, the latency directive sets a maximum limit on all loop operations.

If Vitis HLS cannot meet a maximum latency constraint it relaxes the latency constraint and tries to achieve the best possible result.

If a minimum latency constraint is set and Vitis HLS can produce a design with a lower latency than the minimum required it inserts dummy clock cycles to meet the minimum latency.

Syntax

```
set_directive_latency [OPTIONS] <location>
```

- `<location>` is the location (function, loop or region) (in the format `function[/label]`) to be constrained.

Options

- **`-max <integer>`:** Specifies the maximum latency.
- **`-min <integer>`:** Specifies the minimum latency.

Examples

Function `foo` is specified to have a minimum latency of 4 and a maximum latency of 8.

```
set_directive_latency -min=4 -max=8 foo
```

In function `foo`, loop `loop_row` is specified to have a maximum latency of 12. Place the pragma in the loop body.

```
set_directive_latency -max=12 foo/loop_row
```

See Also

- pragma HLS latency

set_directive_loop_flatten

Description

Allows nested loops to be flattened into a single loop hierarchy with improved latency.

In the RTL implementation, it requires one clock cycle to move from an outer loop to an inner loop, and from an inner loop to an outer loop. Flattening nested loops allows them to be optimized as a single loop. This saves clock cycles, potentially allowing for greater optimization of the loop body logic.

Apply the LOOP_FLATTEN pragma to the loop body of the inner-most loop in the loop hierarchy. Only perfect and semi-perfect loops can be flattened in this manner:

- **Perfect loop nests:**
  - Only the innermost loop has loop body content.

  - There is no logic specified between the loop statements.
  - All loop bounds are constant.
- **Semi-perfect loop nests:**
  - Only the innermost loop has loop body content.
  - There is no logic specified between the loop statements.
  - The outermost loop bound can be a variable.
- **Imperfect loop nests:** When the inner loop has variable bounds (or the loop body is not exclusively inside the inner loop), try to restructure the code, or unroll the loops in the loop body to create a perfect loop nest.

Syntax

```
set_directive_loop_flatten [OPTIONS] <location>
```

- `<location>` is the location (inner-most loop), in the format `function[/label]`.

Options

- **`-off` or `off=true`:** Option to prevent loop flattening from taking place, and can prevent some loops from being flattened while all others in the specified location are flattened.



IMPORTANT! *The presence of the LOOP_FLATTEN pragma or directive enables the optimization. The addition of `-off` disables it.*

Examples

Flattens `loop_1` in function `foo` and all (perfect or semi-perfect) loops above it in the loop hierarchy, into a single loop. Place the pragma in the body of `loop_1`.

```
set_directive_loop_flatten foo/loop_1
```

Prevents loop flattening in `loop_2` of function `foo`. Place the pragma in the body of `loop_2`.

```
set_directive_loop_flatten -off foo/loop_2
```

See Also

- set_directive_loop_flatten
- pragma HLS loop_merge
- pragma HLS unroll

set_directive_loop_merge

Description

Merges all loops into a single loop. Merging loops:

- Reduces the number of clock cycles required in the RTL to transition between the loop-body implementations.
- Allows the loops be implemented in parallel (if possible).

The rules for loop merging are:

- If the loop bounds are variables, they must have the same value (number of iterations).
- If loops bounds are constants, the maximum constant value is used as the bound of the merged loop.
- Loops with both variable bound and constant bound cannot be merged.
- The code between loops to be merged cannot have side effects. Multiple execution of this code should generate the same results.
  - `a=b` is allowed
  - `a=a+1` is not allowed.
- Loops cannot be merged when they contain FIFO reads. Merging changes the order of the reads. Reads from a FIFO or FIFO interface must always be in sequence.

Syntax

```
set_directive_loop_merge <location>
```

- `<location>` is the location (in the format `function[/label]`) at which the loops reside.

Options

- **`-force`**: Forces loops to be merged even when Vitis HLS issues a warning. You must assure that the merged loop will function correctly.

Examples

Merges all consecutive loops in function `foo` into a single loop.

```
set_directive_loop_merge foo
```

All loops inside `loop_2` of function `foo` (but not `loop_2` itself) are merged by using the `-force` option.

```
set_directive_loop_merge -force foo/loop_2
```

See Also

- pragma HLS loop_merge
- set_directive_loop_flatten
- set_directive_unroll

set_directive_loop_tripcount

Description

The *loop tripcount* is the total number of iterations performed by a loop. Vitis HLS reports the total latency of each loop (the number of cycles to execute all iterations of the loop). This loop latency is therefore a function of the tripcount (number of loop iterations).

IMPORTANT! *The LOOP_TRIPCOUNT pragma or directive is for analysis only, and does not impact the results of synthesis.*

The tripcount can be a constant value. It might depend on the value of variables used in the loop expression (for example, `x<y`) or control statements used inside the loop.

Vitis HLS cannot determine the tripcount in some cases. These cases include, for example, those in which the variables used to determine the tripcount are:

- Input arguments, or
- Variables calculated by dynamic operation

In the following example, the maximum iteration of the for-loop is determined by the value of input `num_samples`. The value of `num_samples` is not defined in the C function, but comes into the function from the outside.

```
void foo (num_samples, ...) {
   int i;
   ...
   loop_1: for(i=0;i< num_samples;i++) {
     ...
     result = a + b;
   }
}
```

In cases where the loop latency is unknown or cannot be calculated, `set_directive_loop_tripcount` allows you to specify minimum, maximum, and average iterations for a loop. This lets the tool analyze how the loop latency contributes to the total design latency in the reports and helps you determine appropriate optimizations for the design.

TIP: *If a C assert macro is used to limit the size of a loop variable, Vitis HLS can use it to both define loop limits for reporting and create hardware that is exactly sized to these limits.*

Syntax

```
set_directive_loop_tripcount [OPTIONS] <location>
```

- `<location>` is the location of the loop (in the format `function[/label]`) at which the tripcount is specified.

Options

- **`-avg <integer>`:** Specifies the average number of iterations.
- **`-max <integer>`:** Specifies the maximum number of iterations.
- **`-min <integer>`:** Specifies the minimum number of iterations.

Examples

`loop_1` in function `foo` is specified to have a minimum tripcount of 12, and a maximum tripcount of 16:

```
set_directive_loop_tripcount -min 12 -max 16 -avg 14 foo/loop_1
```

See Also

- pragma HLS loop_tripcount

set_directive_occurrence

Description

`set_directive_occurrence` can be applied to a function call within a region. The feature works with pipelined function calls within a conditional block such as an `if` statement or `switch` statement. The directive specifies how often the pipelined function is called inside the current pipeline. Identifying a function within a condition block with `set_directive_occurrence` allows the functions and loops in that region to be pipelined with a higher initiation interval that is slower than the enclosing function or loop.

When pipelining functions or loops, `set_directive_occurrence` specifies that a pipelined function call within the pipelined function or loop is executed less frequently than the enclosing function or loop. This allows the pipelined function call that is executed less often to be pipelined at a slower rate, thus potentially improving the resource sharing potential within the top-level pipeline.

Use the following process to determine `set_directive_occurrence -cycle` value:

- A loop iterates `<N>` times.

- However, part of the loop body is enabled by a conditional statement, and as a result only executes <M> times, where <N> is an integer multiple of <M>.
- The conditional code has an occurrence that is N/M times slower than the rest of the loop body.

For example, in a loop that executes 10 times, a conditional statement within the loop that only executes two times has an occurrence of 5 (or 10/2).

Syntax

```
set_directive_occurrence [OPTIONS] <location>
```

- <location> specifies the block of code that contains the pipelined function call(s) with a slower rate of execution.

Options

- **-cycle <*int*>:** Specifies the occurrence *N*/*M*, where <N> is the number of times the enclosing function or loop executes, and <M> is the number of times the conditional region executes.

  IMPORTANT! *N must be an integer multiple of M.*

- **-off or off=true:** Disable occurrence for the specified function.

Examples

In this example, the call of sub-function within the if statement (but not the memory read and write, because they are not inside a pipelined function) has an occurrence of 4 (it executes at a rate four times less often than the surrounding code that contains it). Hence, while without set_directive_occurrence it would be pipelined with the same II as the caller, with the directive it will be pipelined with an II=4. This will expose more resource sharing opportunities within it and with other functions.

```
void subfunction(...) {
#pragma HLS pipeline II=...
// Without the occurrence pragma,
// this will be automatically pipelined with an II=1,
// regardless of its own pipeline pragma,
// since it is called in a pipeline with II=1
// With the pragma, it has an II=4.
  ...
}
for (int i = 0; i < 16; i++) {
#pragma HLS pipeline II=1
Cond_Region:
  if (i % 4 == 0) {
    subfunction(...);
    a[i+1] = b[i];
  }
}
```

Region `Cond_Region` in function `subfunction` has an occurrence of 4. It executes at a rate four times slower than the code that encompasses it.

```
set_directive_occurrence -cycle 4 foo/Cond_Region
```

See Also

- pragma HLS occurrence
- set_directive_pipeline

set_directive_performance

Description



TIP: *`set_directive_performance` applies to loops and loop nests, and requires a known loop tripcount to determine the performance. If your loop has a variable tripcount then you must also specify `set_directive_loop_tripcount`.*

The `set_directive_performance` lets you specify a high-level constraint, `target_ti` or `target_tl`, defining the number of clock cycles between successive starts of a loop, and lets the tool infer lower-level UNROLL, PIPELINE, ARRAY_PARTITION, and INLINE directives needed to achieve the desired result. The `set_directive_performance` does not guarantee the specified value will be achieved, and so it is only a target.

The `target_ti` is the interval between successive starts of the loop, or between the start of the first iteration of the loop, and the next start of the first iteration of the loop. In the following code example, a `target_ti=T` would mean the target interval for the start of loop `L2` between two consecutive iterations of `L1` should be 100 cycles.

```
const int T = 100;
L1: for (int i=0; i<N; i++)
  L2: for (int j=0; j<M; j++){
pragma HLS performance target_ti=T
   ...
   }
```

The `target_tl` is the interval between start of the loop and end of the loop, or between the start of the first iteration of the loop and the completion of the last iteration of the loop. For example, in the preceding code example a `target_tl=T` means the target completion of loop `L2` for a single iteration of `L1` should be 100 cycles.

***Note*:** `set_directive_inline` is applied automatically to functions inside any pipelined loop that has II=1 to improve throughput. If you apply the PERFORMANCE pragma or directive that infers a pipeline with II=1, it will also trigger the auto-inline optimization. You can disable this for specific functions by using `set_directive_inline off`.

The transaction interval is the initiation interval (II) of the loop times the number of iterations, or tripcount: target_ti = II * loop tripcount. Conversely, `target_ti` = FreqHz / Operations per second.

For example, assuming an image processing function that processes a single frame per invocation with a throughput goal of 60 fps, then the target throughput for the function is 60 invocations per second. If the clock frequency is 180 MHz, then `target_ti` is 180M/60, or 3 million clock cycles per function invocation.

Syntax

Specify the directive for a labeled loop.

```
set_directive_performance <location> [OPTIONS]
```

Where:

`<location>` specifies the loop in the format `function/loop_label`.

Options

- **`-target_ti=<value>`:** Specifies a target transaction interval defined as the number of clock cycles for the loop to complete an iteration. The transaction interval refers to the time the loop starts processing a transaction and the time it begins processing the next transaction. The <value> can be specified as an integer, floating point, or constant expression that is resolved by the tool as an integer.

  ***Note*:** A warning will be returned if truncation occurs.

- **`-target_tl=<value>`:**

  Specifies a target latency defined as the number of clock cycles for the loop to complete all iterations. The transaction latency is defined as the interval between the start of the first iteration of the loop , and the completion of the last iteration of the loop. The <value> can be specified as an integer, floating point, or constant expression that is resolved by the tool as an integer.

- **unit=[sec | cycle]:** Specifies the unit associated with the `target_ti` or `target_tl` values. The unit can either be specified as seconds, or clock cycles. When the unit is specified as seconds, a unit can be specified with the value to indicate nanoseconds (ns), picoseconds (ps), microseconds (us).

Example 1

The loop labeled `loop_1` in the function `func` is specified to have target transaction interval of 4 clock cycles:

```
set_directive_performance func/loop_1 -target_ti=4
```

See Also

- pragma HLS performance
- set_directive_inline

set_directive_pipeline

Description

Reduces the initiation interval (II) for a function or loop by allowing the concurrent execution of operations as described in Pipelining Loops. A pipelined function or loop can process new inputs every N clock cycles, where N is the initiation interval (`II`). An II of 1 processes a new input every clock cycle.

As a default behavior, with the PIPELINE pragma or directive Vitis HLS will generate the minimum II for the design according to the specified clock period constraint. The emphasis will be on meeting timing, rather than on achieving II unless the `-II` option is specified.

The default type of pipeline is defined by the `config_compile -pipeline_style` command, but can be overridden in the PIPELINE pragma or directive.

If Vitis HLS cannot create a design with the specified II, it:

- Issues a warning.
- Creates a design with the lowest possible II.

You can then analyze this design with the warning messages to determine what steps must be taken to create a design that satisfies the required initiation interval.

Syntax

```
set_directive_pipeline [OPTIONS] <location>
```

Where:

- `<location>` is the location (in the format `function[/label]`) to be pipelined.

Options

- **`-II <integer>`**: Specifies the desired initiation interval for the pipeline. Vitis HLS tries to meet this request. Based on data dependencies, the actual result might have a larger II.
- **`-off`**: Turns off pipeline for a specific loop or function. This can be used when `config_compile -pipeline_loops` is used to globally pipeline loops.
- **`-rewind`**:

  Note: Applicable only to a loop.

Optional keyword. Enables rewinding as described in Rewinding Pipelined Loops for Performance. This enables continuous loop pipelining with no pause between one execution of the loop ending and the next execution starting. Rewinding is effective only if there is one single loop (or a perfect loop nest) inside the top-level function. The code segment before the loop:

- Is considered as initialization.
- Is executed only once in the pipeline.
- Cannot contain any conditional operations (`if-else`).

- `-style <stp | frp | flp>`: Specifies the type of pipeline to use for the specified function or loop. For more information on pipeline styles refer to Flushing Pipelines and Pipeline Types. The types of pipelines include:
  - `stp`: Stall pipeline. Runs only when input data is available otherwise it stalls. This is the default setting, and is the type of pipeline used by Vitis HLS for both loop and function pipelining. Use this when a flushable pipeline is not required. For example, when there are no performance or deadlock issue due to stalls.
  - `flp`: This option defines the pipeline as a flushable pipeline. This type of pipeline typically consumes more resources and/or can have a larger II because resources cannot be shared among pipeline iterations.
  - `frp`: Free-running, flushable pipeline. Runs even when input data is not available. Use this when you need better timing due to reduced pipeline control signal fanout, or when you need improved performance to avoid deadlocks. However, this pipeline style can consume more power as the pipeline registers are clocked even if there is no data.



IMPORTANT! *This is a hint not a hard constraint. The tool checks design conditions for enabling pipelining. Some loops might not conform to a particular style and the tool reverts to the default style (`stp`) if necessary.*

Examples

Function `func` is pipelined with the specified initiation interval.

```
set_directive_pipeline func II=1
```

See Also

- pragma HLS pipeline
- set_directive_dependence
- config_compile

set_directive_protocol

Description

This commands specifies a region of code, a protocol region, in which no clock operations will be inserted by Vitis HLS unless explicitly specified in the code. Vitis HLS will not insert any clocks between operations in the region, including those which read from or write to function arguments. The order of read and writes will therefore be strictly followed in the synthesized RTL.

A region of code can be created in the C/C++ code by enclosing the region in braces "{ }" and naming it. The following defines a region named `io_section`:

```
io_section:{
...
lines of code
...
}
```

A clock operation can be explicitly specified in C code using an `ap_wait()` statement, and can be specified in C++ code by using the `wait()` statement. The `ap_wait` and `wait` statements have no effect on the simulation of the design.

Syntax

```
set_directive_protocol [OPTIONS] <location>
```

The `<location>` specifies the location (in the format `function[/label]`) at which the protocol region is defined.

Options

- **`-mode [floating | fixed]:`**
  - `floating`: Lets code statements outside the protocol region overlap and execute in parallel with statements in the protocol region in the final RTL. The protocol region remains cycle accurate, but outside operations can occur at the same time. This is the default mode.
  - `fixed`: The fixed mode ensures that statements outside the protocol region do not execute in parallel with the protocol region.

Examples

The example code defines a protocol region, `io_section` in function `foo`. The following directive defines that region as a fixed mode protocol region:

```
set_directive_protocol -mode fixed foo/io_section
```

See Also

- pragma HLS protocol

set_directive_reset

Description

The RESET pragma or directive adds or disables reset ports for specific state variables (global or static).

The reset port is used to restore the registers and block RAM, connected to the port, to an initial value any time the reset signal is applied. Globally, the presence and behavior of the RTL reset port is controlled using the `config_rtl -reset` command. The reset configuration settings include the ability to define the polarity of the reset, and specify whether the reset is synchronous or asynchronous, but more importantly it controls, through the reset option, which registers are reset when the reset signal is applied. For more information, see Controlling Initialization and Reset Behavior.

More specific control over reset is provided through the RESET pragma. For global or static variables the RESET pragma is used to explicitly enable a reset when none is present, or the variable can be removed from the reset by turning `off` the pragma. This can be particularly useful when static or global arrays are present in the design.

***Note*:** For public variables of a class, the RESET pragma must be used as the reset configuration settings only apply to variables declared at the function or file level. In addition, the RESET pragma must be applied to an object of the class in the top-function or sub-function, and cannot be applied to private variables of the class.

Syntax

Place the pragma in the C source within the boundaries of the variable life cycle.

```
set_directive_reset <location> [ variable=<a> | off ]
```

Where:

- **`location>`:** Specifies the location (in the format `function[/label]`) at which the variable is defined.
- **`variable=<a>`:** Specifies the variable to which the RESET pragma is applied.
- **`off` or `off=true`:** Indicates that reset is not generated for the specified variable.

Examples

Adds reset to variable `a` in function `foo` even when the global reset (`config_rtl -reset`) setting is `none` or `control`.

```
set_directive_reset foo a
```

Removes reset from variable `static int a` in function `foo` even when the global reset setting is `state` or `all`.

```
set_directive_reset -off foo a
```

The following example shows the use of the RESET pragma or directive with class variables and methods. A variable declared in a class must be a public static variable in the class and can have the RESET pragma specified for the variable in a method of the class, or after an object of the class has been created in the top function or sub-function of the HLS design.

```
class bar {
public:
  static int a; // class level static; must be public
  int a1;       // class-level non-static; must be public
  bar(...) {
  // #pragma reset does not work here for a or a1
  }
  // this is the function that is called in the top
  void run(...) {
  // #pragma reset does not work here for non-static variable a1
  // but does work for static variable a
  #pragma reset variable=a
  }
};
static int c; // file level
int d; // file level
void top(...) {
#pragma HLS top
  static int b; // function level
  bar t;
  static bar s;

  // #pragma reset works here for b, c and d, as well as t and s members
  // except for t.a1 because it is not static
  #pragma reset variable=b
  #pragma reset variable=c
  #pragma reset variable=d
  #pragma reset variable=t.a
  #pragma reset variable=s.a
  #pragma reset variable=s.a1
  t.run(...);
  s.run(...);
}
```

The `set_directive_reset` command specified for the function `top` shown above would be as follows:

```
set_directive_reset top variable=b
set_directive_reset top variable=c
set_directive_reset top variable=d
set_directive_reset top variable=t.a
set_directive_reset top variable=s.a
set_directive_reset top variable=s.a1
```

Example 3

See Also

- pragma HLS reset
- config_rtl

set_directive_stable

Description

Arguments marked as stable cannot change during kernel execution. Inputs that only change when the kernel is not running, such as ports that provide configuration data, should be marked with a STABLE pragma or directive. The STABLE Pragma can only be used for Control-Driven task modeling because in Data-driven Task-level Parallelism there is no way to detect when the kernel is idle.

The STABLE pragma informs Vitis HLS of the following:

- The data applied to the port remains stable during normal operation but is not a constant value that can be optimized.
- The fanout from this port is not required to be registered.

Pragma STABLE pragma in DATAFLOW networks tells the compiler that input does not need to be read by the first dataflow process and that output does not need to be written by the last dataflow process in a network. This can improve II dramatically.

```
set_directive_stable <location> <variable>
```

- `<location>` is the function name or loop name where the directive is to be constrained.
- `<variable>` is the name of the scalar or array variable to be constrained.

Examples

In the following example, without the STABLE directive, `proc1` and `proc2` would be synchronized to acknowledge the reading of their inputs (including `A`). With the directive, `A` is no longer considered as an input that needs synchronization.

```
void dataflow_region(int A[...], int B[…] ...
    proc1(...);
    proc2(A, ...);
```

The directives for this example would be scripted as:

```
set_directive_stable dataflow_region variable=A
set_directive_dataflow dataflow_region
```

See Also

- pragma HLS stable
- set_directive_dataflow

set_directive_stream

Description

By default, array variables are implemented as RAM:

- Top-level function array parameters are implemented as a RAM interface port.
- General arrays are implemented as RAMs for read-write access.
- Arrays involved in sub-functions, or loop-based DATAFLOW optimizations are implemented as a RAM ping-pong buffer channel.

If the data stored in the array is consumed or produced in a sequential manner, a more efficient communication mechanism is to use streaming data, where FIFOs are used instead of RAMs. When an argument of the top-level function is specified as INTERFACE type `ap_fifo`, the array is automatically implemented as streaming. See Defining Interfaces for more information.

IMPORTANT! *To preserve the accesses, it might be necessary to prevent compiler optimizations (in particular dead code elimination) by using the `volatile` qualifier as described in Type Qualifiers.*

Syntax

```
set_directive_stream [OPTIONS] <location> <variable>
```

- `<location>` is the location (in the format `function[/label]`) which contains the array variable.
- `<variable>` is the array variable to be implemented as a FIFO.

Options

- `-depth <integer>`:

  Note: Relevant only for array streaming in dataflow channels.

  By default, the depth of the FIFO implemented in the RTL is the same size as the array specified in the C code. This options allows you to modify the size of the FIFO.

  When the array is implemented in a DATAFLOW region, it is common to the use the `-depth` option to reduce the size of the FIFO. For example, in a `DATAFLOW` region where all loops and functions are processing data at a rate of II = 1, there is no need for a large FIFO because data is produced and consumed in each clock cycle. In this case, the `-depth` option can be used to reduce the FIFO size to 2 to substantially reduce the area of the RTL design.

  This same functionality is provided for all arrays in a DATAFLOW region using the `config_dataflow` command with the `-depth` option. The `-depth` option used with `set_directive_stream` overrides the default specified using `config_dataflow`.

- **-type <arg>:** Specify a mechanism to select between FIFO, PIPO, synchronized shared (`shared`), and un-synchronized shared (`unsync`). The supported types include:
  - `fifo`: A FIFO buffer with the specified `depth`
  - `pipo`: A Ping-Pong buffer with as many "banks" as the specified depth (default is 2)
  - `shared`: A shared channel, synchronized like a regular Ping-Pong buffer, with depth, but without duplicating the array data. Consistency can be ensured by setting the depth small enough, which acts as the distance of synchronization between the producer and consumer

    TIP: *The default depth for shared is 1.*

  - `unsync`: Does not have any synchronization except for individual memory reads and writes. Consistency (read-write and write-write order) must be ensured by the design itself

Examples

Specifies array `A[10]` in function `func` to be streaming and implemented as a FIFO.

```
set_directive_stream func A -type fifo
```

Array B in named loop `loop_1` of function `func` is set to streaming with a FIFO depth of 12. In this case, place the pragma inside `loop_1`.

```
set_directive_stream -depth 12 -type fifo func/loop_1 B
```

Array C has streaming implemented as a PIPO.

```
set_directive_stream -type pipo func C
```

See Also

- pragma HLS stream
- set_directive_dataflow
- set_directive_interface
- config_dataflow

set_directive_top

Description

Attaches a name to a function, which can then be used by the `set_top` command to set the named function as the top. This is typically used to synthesize member functions of a class in C++.

RECOMMENDED: *Specify the directive in an active solution. Use the `set_top` command with the new name.*

Syntax

```
set_directive_top [OPTIONS]
```

Options

- **`-name <string>`:** Specifies the name of the function to be used by the `set_top` command.

Examples

Function `foo_long_name` is renamed to `DESIGN_TOP`, which is then specified as the top-level. If the pragma is placed in the code, the `set_top` command must still be issued in the top-level specified in the GUI project settings.

```
set_directive_top -name DESIGN_TOP foo_long_name
```

Followed by the `set_top DESIGN_TOP` command.

See Also

- pragma HLS top
- set_top

set_directive_unroll

Description

Transforms loops by creating multiples copies of the loop body.

A loop is executed for the number of iterations specified by the loop induction variable. The number of iterations can also be impacted by logic inside the loop body (for example, break or modifications to any loop exit variable). The loop is implemented in the RTL by a block of logic representing the loop body, which is executed for the same number of iterations.

The `set_directive_unroll` command allows the loop to be fully unrolled. Unrolling the loop creates as many copies of the loop body in the RTL as there are loop iterations, or partially unrolled by a factor *N*, creating *N* copies of the loop body and adjusting the loop iteration accordingly.

If the factor *N* used for partial unrolling is not an integer multiple of the original loop iteration count, the original exit condition must be checked after each unrolled fragment of the loop body.

To unroll a loop completely, the loop bounds must be known at compile time. This is not required for partial unrolling.

Syntax

```
set_directive_unroll [OPTIONS] <location>
```

- `<location>` is the location of the loop (in the `format function[/label]`) to be unrolled.

Options

- **`-factor <integer>`:** Specifies a non-zero integer indicating that partial unrolling is requested.

  The loop body is repeated this number of times. The iteration information is adjusted accordingly.

- **`-skip_exit_check`:** Effective only if a factor is specified (partial unrolling).
  - Fixed bounds

    No exit condition check is performed if the iteration count is a multiple of the factor.

    If the iteration count is *not* an integer multiple of the factor, the tool:

    - Prevents unrolling.
    - Issues a warning that the exit check must be performed to proceed.
  - Variable bounds

The exit condition check is removed. You must ensure that:

- The variable bounds is an integer multiple of the factor.
- No exit check is in fact required.

- **`-off` or `off=true`:** Disable unroll for the specified loop.

Examples

Unrolls loop `L1` in function `foo`. Place the pragma in the body of loop `L1`.

```
set_directive_unroll foo/L1
```

Specifies an unroll factor of 4 on loop `L2` of function `foo`. Removes the exit check. Place the pragma in the body of loop `L2`.

```
set_directive_unroll -skip_exit_check -factor 4 foo/L2
```

See Also

- pragma HLS unroll
- set_directive_loop_flatten
- set_directive_loop_merge

Section V

Vitis HLS C Driver Reference

This section contains the following chapter:

- AXI4-Lite Slave C Driver Reference

AXI4-Lite Slave C Driver Reference

X<DUT>_Initialize

Syntax

```
int X<DUT>_Initialize(X<DUT> *InstancePtr, uintptr BaseAddress);
int X<DUT>_Initialize(X<DUT> *InstancePtr, u16 DeviceId);
int X<DUT>_Initialize(X<DUT> *InstancePtr, const char* InstanceName);
```

Description

`int X<DUT>_Initialize(X<DUT> *InstancePtr, uintptr BaseAddress)`: For use on standalone systems, initialize the specified `BaseAddress` of a device. This API will write a proper value to `InstancePtr` which then can be used in other APIs.

Note: The `DeviceId` is no longer populated into the driver config struct. Instead the `BaseAddress` is used to identify the driver instance. The `DeviceId` will also not be populated in the `xparameters.h` file. Software applications using these drivers developed in the classic Vitis or Vitis HLS tools will need to be updated to reflect this change using `#ifdef SDT` as shown in the following example:

```
#ifndef SDT
#define GPIO_EXAMPLE_DEVICE_ID  XPAR_GPIO_0_DEVICE_ID
#else
#define    XGPIO_AXI_BASEADDRESS    XPAR_XGPIO_0_BASEADDR
#endif
```

`int X<DUT>_Initialize(X<DUT> *InstancePtr, u16 DeviceId)`: For use on standalone systems, initialize a device. This API will write a proper value to `InstancePtr` which then can be used in other APIs. AMD recommends calling this API to initialize a device except when an MMU is used in the system, in which case refer to function `X<DUT>_CfgInitialize`.

`int X<DUT>_Initialize(X<DUT> *InstancePtr, const char* InstanceName)`: For use on Linux systems, initialize a specifically named `uio` device. Create up to five memory mappings and assign the slave base addresses by `mmap`, utilizing the `uio` device information in sysfs.

Where the various arguments are defined as:

- **`InstancePtr`**: A pointer to the device instance.

- **DeviceId**: Device ID as defined in `xparameters.h`.
- **BaseAddress**: The BaseAddress as defined in `xparameters.h`.
- **InstanceName**: The name of the `uio` device.

Return

`XST_SUCCESS` indicates success, otherwise fail.

X<DUT>_CfgInitialize

Syntax

```
X<DUT>_CfgInitializeint X<DUT>_CfgInitialize(X<DUT> *InstancePtr,
X<DUT>_Config *ConfigPtr);
```

Description

Initialize a device when an MMU is used in the system. In such a case the effective address of the AXI4-Lite slave is different from that defined in `xparameters.h` and API is required to initialize the device.

- **InstancePtr**: A pointer to the device instance.
- **ConfigPtr**: A pointer to a `X<DUT>_Config`.

Return

`XST_SUCCESS` indicates success, otherwise fail.

X<DUT>_LookupConfig

Syntax

```
X<DUT>_Config* X<DUT>_LookupConfig(uintptr BaseAddress);
X<DUT>_Config* X<DUT>_LookupConfig(u16 DeviceId);
```

Description

This function is used to obtain the configuration information of the specified `BaseAddress` or `DeviceID`. Where:

- **BaseAddress**: The BaseAddress as defined in `xparameters.h`.
- **DeviceId**: Device ID as defined in `xparameters.h`.

Note: The `DeviceId` is no longer populated into the driver config struct. Instead the `BaseAddress` is used to identify the driver instance. The `DeviceId` will also not be populated in the `xparameters.h` file. Software applications using these drivers developed in the classic Vitis or Vitis HLS tools will need to be updated to reflect this change using `#ifdef SDT` as shown in the following example:

```
#ifndef SDT
#define GPIO_EXAMPLE_DEVICE_ID  XPAR_GPIO_0_DEVICE_ID
#else
#define    XGPIO_AXI_BASEADDRESS    XPAR_XGPIO_0_BASEADDR
#endif
```

Return

A pointer to a `X<DUT>_LookupConfig` variable that holds the configuration information of the specified `BaseAddress` or device whose ID is `DeviceId`. Returns `NULL` if no match is found.

X<DUT>_Release

Syntax

```
int X<DUT>_Release(X<DUT> *InstancePtr);
```

Description

Release the `uio` device. Delete the mappings by `munmap`. The mapping will automatically be deleted if the process is terminated.

- **InstanceName**: The name of the `uio` device.

Return

`XST_SUCCESS` indicates success, otherwise fail.

X<DUT>_Start

Syntax

```
void X<DUT>_Start(X<DUT> *InstancePtr);
```

Description

Start the device. This function will assert the `ap_start` port on the device. Available only if there is `ap_start` port on the device.

- **`InstancePtr`:** A pointer to the device instance.

X<DUT>_IsDone

Syntax

```
void X<DUT>_IsDone(X<DUT> *InstancePtr);
```

Description

Check if the device has finished the previous execution: this function will return the value of the `ap_done` port on the device. Available only if there is an `ap_done` port on the device.

- **`InstancePtr`:** A pointer to the device instance.

X<DUT>_IsIdle

Syntax

```
void X<DUT>_IsIdle(X<DUT> *InstancePtr);
```

Description

Check if the device is in idle state: this function will return the value of the `ap_idle` port. Available only if there is an `ap_idle` port on the device.

- **`InstancePtr`:** A pointer to the device instance.

X<DUT>_IsReady

Syntax

```
void X<DUT>_IsReady(X<DUT> *InstancePtr);
```

Description

Check if the device is ready for the next input: this function will return the value of the `ap_ready` port. Available only if there is an `ap_ready` port on the device.

- **`InstancePtr`:** A pointer to the device instance.

X<DUT>_Continue

Syntax

```
void XExample_Continue(XExample *InstancePtr);
```

Description

Assert port `ap_continue`. Available only if there is an `ap_continue` port on the device.

- **`InstancePtr`:** A pointer to the device instance.

X<DUT>_EnableAutoRestart

Syntax

```
void X<DUT>_EnableAutoRestart(X<DUT> *InstancePtr);
```

Description

Enables “auto restart” on device. When this is enabled,

- Port `ap_start` will be asserted as soon as `ap_done` is asserted by the device and the device will auto-start the next transaction.
- Alternatively, if the block-level I/O protocol `ap_ctrl_chain` is implemented on the device, the next transaction will auto-restart (`ap_start` will be asserted) when `ap_ready` is asserted by the device and if `ap_continue` is asserted when `ap_done` is asserted by the device.

Available only if there is an `ap_start` port.

- **`InstancePtr`:** A pointer to the device instance.

X<DUT>_DisableAutoRestart

Syntax

```
void X<DUT>_DisableAutoRestart(X<DUT> *InstancePtr);
```

Description

Disable the “auto restart” function. Available only if there is an `ap_start` port.

- **`InstancePtr`:** A pointer to the device instance.

X<DUT>_Set_ARG

Syntax

```
void X<DUT>_Set_ARG(X<DUT> *InstancePtr, u32 Data);
```

Description

Write a value to port `ARG` (a scalar argument of the top-level function). Available only if `ARG` is an input port.

- **`InstancePtr`:** A pointer to the device instance.
- **`Data`:** Value to write.

X<DUT>_Set_ARG_vld

Syntax

```
void X<DUT>_Set_ARG_vld(X<DUT> *InstancePtr);
```

Description

Assert port `ARG_vld`. Available only if `ARG` is an input port and implemented with an `ap_hs` or `ap_vld` interface protocol.

- **`InstancePtr`:** A pointer to the device instance.

X<DUT>_Set_ARG_ack

Syntax

```
void X<DUT>_Set_ARG_ack(X<DUT> *InstancePtr);
```

Description

Assert port `ARG_ack`. Available only if `ARG` is an output port and implemented with an `ap_hs` or `ap_ack` interface protocol.

- **`InstancePtr`:** A pointer to the device instance.

X<DUT>_Get_ARG

Syntax

```
u32 X<DUT>_Get_ARG(X<DUT> *InstancePtr);
```

Description

Read a value from `ARG`. Only available if port `ARG` is an output port on the device.

- **`InstancePtr`:** A pointer to the device instance.

Return

Value of `ARG`.

X<DUT>_Get_ARG_vld

Syntax

```
u32 X<DUT>_Get_ARG_vld(X<DUT> *InstancePtr);
```

Description

Read a value from `ARG_vld`. Only available if port `ARG` is an output port on the device and implemented with an `ap_hs` or `ap_vld` interface protocol.

- `InstancePtr`: A pointer to the device instance.

Return

Value of `ARG_vld`.

X<DUT>_Get_ARG_ack

Syntax

```
u32 X<DUT>_Get_ARG_ack(X<DUT> *InstancePtr);
```

Description

Read a value from `ARG_ack`. Only available if port `ARG` is an input port on the device and implemented with an `ap_hs` or `ap_ack` interface protocol.

- `InstancePtr`: A pointer to the device instance.

Return

Value of `ARG_ack`.

X<DUT>_Get_ARG_BaseAddress

Syntax

```
u32 X<DUT>_Get_ARG_BaseAddress(X<DUT> *InstancePtr);
```

Description

Return the base address of the array inside the interface. Only available when `ARG` is an array grouped into the AXI4-Lite interface.

- `InstancePtr`: A pointer to the device instance.

Return

Base address of the array.

X<DUT>_Get_ARG_HighAddress

Syntax

```
u32 X<DUT>_Get_ARG_HighAddress(X<DUT> *InstancePtr);
```

Description

Return the address of the uppermost element of the array. Only available when `ARG` is an array grouped into the AXI4-Lite interface.

- `InstancePtr`: A pointer to the device instance.

Return

Address of the uppermost element of the array.

X<DUT>_Get_ARG_TotalBytes

Syntax

```
u32 X<DUT>_Get_ARG_TotalBytes(X<DUT> *InstancePtr);
```

Description

Return the total number of bytes used to store the array. Only available when `ARG` is an array grouped into the AXI4-Lite interface.

If the elements in the array are less than 16-bit, AMD Vitis™ HLS groups multiple elements into the 32-bit data width of the AXI4-Lite interface. If the bit width of the elements exceeds 32-bit, Vitis HLS stores each element over multiple consecutive addresses.

- `InstancePtr`: A pointer to the device instance.

Return

The total number of bytes used to store the array.

X<DUT>_Get_ARG_BitWidth

Syntax

```
u32 X<DUT>_Get_ARG_BitWidth(X<DUT> *InstancePtr);
```

Description

Return the bit width of each element in the array. Only available when `ARG` is an array grouped into the AXI4-Lite interface.

If the elements in the array are less than 16-bit, Vitis HLS groups multiple elements into the 32-bit data width of the AXI4-Lite interface. If the bit width of the elements exceeds 32-bit, Vitis HLS stores each element over multiple consecutive addresses.

- `InstancePtr`: A pointer to the device instance.

Return

The bit-width of each element in the array.

X<DUT>_Get_ARG_Depth

Syntax

```
u32 X<DUT>_Get_ARG_Depth(X<DUT> *InstancePtr);
```

Description

Return the total number of elements in the array. Only available when `ARG` is an array grouped into the AXI4-Lite interface.

If the elements in the array are less than 16-bit, Vitis HLS groups multiple elements into the 32-bit data width of the AXI4-Lite interface. If the bit width of the elements exceeds 32-bit, Vitis HLS stores each element over multiple consecutive addresses.

- `InstancePtr`: A pointer to the device instance.

Return

The total number of elements in the array.

X<DUT>_Write_ARG_Words

Syntax

```
u32 X<DUT>_Write_ARG_Words(X<DUT> *InstancePtr, int offset, int *data, int
length);
```

Description

Write the length of a 32-bit word into the specified address of the AXI4-Lite interface. This API requires the offset address from `BaseAddress` and the length of the data to be stored. Only available when `ARG` is an array grouped into the AXI4-Lite interface.

- **`InstancePtr`:** A pointer to the device instance.
- **`offset`:** The address in the AXI4-Lite interface.
- **`data`:** A pointer to the data value to be stored.
- **`length`:** The length of the data to be stored.

Return

Write length of data from the specified address.

X<DUT>_Read_ARG_Words

Syntax

```
u32 X<DUT>_Read_ARG_Words(X<DUT> *InstancePtr, int offset, int *data, int
length);
```

Description

Read the length of a 32-bit word from the array. This API requires the data target, the offset address from `BaseAddress`, and the length of the data to be stored. Only available when `ARG` is an array grouped into the AXI4-Lite interface.

- **`InstancePtr`:** A pointer to the device instance.
- **`offset`:** The address in the `ARG`.
- **`data`:** A pointer to the data buffer.
- **`length`:** The length of the data to be stored.

Return

Read length of data from the specified address.

X<DUT>_Write_ARG_Bytes

Syntax

```
u32 X<DUT>_Write_ARG_Bytes(X<DUT> *InstancePtr, int offset, char *data, int
length);
```

Description

Write the length of bytes into the specified address of theAXI4-Lite interface. This API requires the offset address from BaseAddress and the length of the data to be stored. Only available when `ARG` is an array grouped into the AXI4-Lite interface.

- **`InstancePtr`:** A pointer to the device instance.
- **`offset`:** The address in the `ARG`.
- **`data`:** A pointer to the data value to be stored.
- **`length`:** The length of the data to be stored.

Return

Write length of data from the specified address.

X<DUT>_Read_ARG_Bytes

Syntax

```
u32 X<DUT>_Read_ARG_Bytes(X<DUT> *InstancePtr, int offset, char *data, int
length);
```

Description

Read the length of bytes from the array. This API requires the data target, the offset address from Base Address, and the length of data to be loaded. Only available when `ARG` is an array grouped into the AXI4-Lite interface.

- **`InstancePtr`:** A pointer to the device instance.

- `offset`: The address in the `ARG`.
- `data`: A pointer to the data buffer.
- `length`: The length of the data to be loaded.

Return

Read length of data from the specified address.

X<DUT>_InterruptGlobalEnable

Syntax

```
void X<DUT>_InterruptGlobalEnable(X<DUT> *InstancePtr);
```

Description

Enable the interrupt output. Interrupt functions are available only if there is `ap_start`.

- `InstancePtr`: A pointer to the device instance.

X<DUT>_InterruptGlobalDisable

Syntax

```
void X<DUT>_InterruptGlobalDisable(X<DUT> *InstancePtr);
```

Description

Disable the interrupt output.

- `InstancePtr`: A pointer to the device instance.

X<DUT>_InterruptEnable

Syntax

```
void X<DUT>_InterruptEnable(X<DUT> *InstancePtr, u32 Mask);
```

Description

Enable the interrupt source. There can be at most two interrupt sources (`source 0` for `ap_done` and `source 1` for `ap_ready`).

- **`InstancePtr`:** A pointer to the device instance.
- **`Mask`:** Bit mask.
  - Bit n = 1: enable interrupt source n.
  - Bit n = 0: no change.

X<DUT>_InterruptDisable

Syntax

```
void X<DUT>_InterruptDisable(X<DUT> *InstancePtr, u32 Mask);
```

Description

Disable the interrupt source.

- **`InstancePtr`:** A pointer to the device instance.
- **`Mask`:** Bit mask.
  - Bit n = 1: disable interrupt source n.
  - Bit n = 0: no change.

X<DUT>_InterruptClear

Syntax

```
void X<DUT>_InterruptClear(X<DUT> *InstancePtr, u32 Mask);
```

Description

Clear the interrupt status.

- **`InstancePtr`:** A pointer to the device instance.
- **`Mask`:** Bit mask.

- Bit n = 1: toggle interrupt source n.
- Bit n = 0: no change.

X<DUT>_InterruptGetEnabled

Syntax

```
u32 X<DUT>_InterruptGetEnabled(X<DUT> *InstancePtr);
```

Description

Check which interrupt sources are enabled.

- `InstancePtr`: A pointer to the device instance.

Return

Bit mask.

- Bit n = 1: enabled.
- Bit n = 0: disabled.

X<DUT>_InterruptGetStatus

Syntax

```
u32 X<DUT>_InterruptGetStatus(X<DUT> *InstancePtr);
```

Description

Check which interrupt sources are triggered.

- `InstancePtr`: A pointer to the device instance.

Return

Bit mask.

- Bit n = 1: triggered.
- Bit n = 0: not triggered.

AMD

Section VI

Vitis HLS Libraries Reference

This section contains the following chapters:

- C/C++ Builtin Functions
- Arbitrary Precision Data Types Library
- HLS Print Function
- HLS Math Library
- HLS Stream Library
- HLS Vector Library
- HLS Task Library
- HLS Split/Merge Library
- HLS Stream of Blocks Library
- HLS IP Libraries

C/C++ Builtin Functions

Vitis HLS supports the following C/C++ builtin functions:

- `__builtin_clz(unsigned int x)`: Returns the number of leading 0-bits in x, starting at the most significant bit position. If x is 0, the result is undefined.
- `__builtin_ctz(unsigned int x)`: Returns the number of trailing 0-bits in x, starting at the least significant bit position. If x is 0, the result is undefined.

The following example shows how these functions may be used. This example returns the sum of the number of leading zeros in in0 and trailing zeros in in1:

```
int foo (int in0, int in1) {
 int ldz0 = __builtin_clz(in0);
 int ldz1 = __builtin_ctz(in1);
 return (ldz0 + ldz1);
}
```

Chapter 20

Arbitrary Precision Data Types Library

C-based native data types are on 8-bit boundaries (8, 16, 32, 64 bits). RTL buses (corresponding to hardware) support arbitrary lengths. HLS needs a mechanism to allow the specification of arbitrary precision bit-width and not rely on the artificial boundaries of native C data types: if a 17-bit multiplier is required, you should not be forced to implement this with a 32-bit multiplier.

AMD Vitis™ HLS provides both integer and fixed-point arbitrary precision data types for C++. The advantage of arbitrary precision data types is that they allow the C code to be updated to use variables with smaller bit-widths and then for the C simulation to be re-executed to validate that the functionality remains identical or acceptable.

Using Arbitrary Precision Data Types

Vitis HLS provides arbitrary precision integer data types that manage the value of the integer numbers within the boundaries of the specified width, as shown in the following table.

Table 50: **Arbitrary Precision Data Types**

Language	Integer Data Type	Required Header
C++	ap_[u]int<W> (1024 bits) Can be extended to 4K bits wide as explained in C++ Arbitrary Precision Integer Types.	#include “ap_int.h”
C++	ap_[u]fixed<W,I,Q,O,N>	#include “ap_fixed.h”

The header files define the arbitrary precision types are also provided with Vitis HLS as a standalone package with the rights to use them in your own source code. The package, `xilinx_hls_lib_<release_number>.tgz`, is provided in the `include` directory in the Vitis HLS installation area.

Arbitrary Integer Precision Types with C++

The header file `ap_int.h` defines the arbitrary precision integer data type for the C++ `ap_[u]int` data types. To use arbitrary precision integer data types in a C++ function:

- Add header file `ap_int.h` to the source code.
- Change the bit types to `ap_int<N>` for signed types or `ap_uint<N>` for unsigned types, where `N` is a bit-size from 1 to 1024.

The following example shows how the header file is added and two variables implemented to use 9-bit integer and 10-bit unsigned integer types:

```
#include "ap_int.h"

void foo_top () {

 ap_int<9>  var1;           // 9-bit
 ap_uint<10>  var2;         // 10-bit unsigned
```

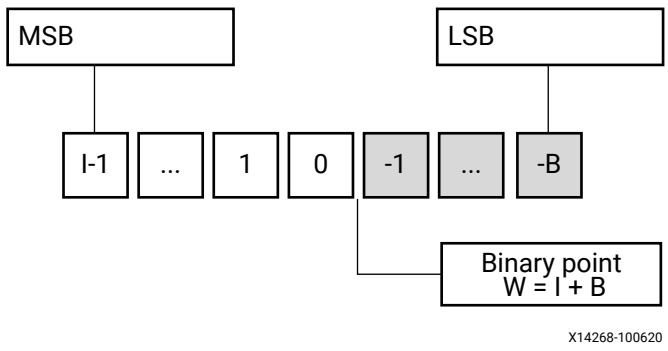
IMPORTANT! *One disadvantage of AP data types is that arrays are not automatically initialized with a value of 0. You must manually initialize the array if desired.*

Arbitrary Precision Fixed-Point Data Types

In Vitis HLS, it is important to use fixed-point data types, because the behavior of the C++ simulations performed using fixed-point data types match that of the resulting hardware created by synthesis. This allows you to analyze the effects of bit-accuracy, quantization, and overflow with fast C-level simulation.

These data types manage the value of real (non-integer) numbers within the boundaries of a specified total width (W) and integer width (I) with the expression (W = I + B), as shown in the following figure.

Figure 144: **Fixed-Point Data Type**



TIP: *In the preceding figure the integer value (I) specifies the number of integer bits to the left of the binary point, including the sign bit.*

Fixed-Point Identifier Summary

The following table provides a brief overview of operations supported by fixed-point types.

Table 51: **Fixed-Point Identifier Summary**

Identifier	Description	
W	Word length in bits	

I	The number of bits used to represent the integer value, that is, the number of integer bits to the *left* of the binary point, including the sign bit.
	When I is negative, as shown in the example below, it represents the number of *implicit* sign bits (for signed representation), or the number of *implicit* zero bits (for unsigned representation) to the *right* of the binary point. For example,
	`ap_fixed<2, 0> a = -0.5;  // a can be -0.5,`
	`ap_ufixed<1, 0> x = 0.5;  // 1-bit representation. x can be 0 or 0.5`
	`ap_ufixed<1, -1> y = 0.25;  // 1-bit representation. y can be 0 or 0.25`
	`const ap_fixed<1, -7> z = 1.0/256;  // 1-bit representation for z = 2^-8`	

Q	Quantization mode: This dictates the behavior when greater precision is generated than can be defined by smallest fractional bit in the variable used to store the result.	
	ap_fixed Types	Description
	AP_RND	Round to plus infinity
	AP_RND_ZERO	Round to zero
	AP_RND_MIN_INF	Round to minus infinity
	AP_RND_INF	Round to infinity
	AP_RND_CONV	Convergent rounding
	AP_TRN	Truncation to minus infinity (default)
	AP_TRN_ZERO	Truncation to zero
O	Overflow mode: This dictates the behavior when the result of an operation exceeds the maximum (or minimum in the case of negative numbers) possible value that can be stored in the variable used to store the result.	
	ap_fixed Types	Description
	AP_SAT[1]	Saturation
	AP_SAT_ZERO[1]	Saturation to zero
	AP_SAT_SYM[1]	Symmetrical saturation
	AP_WRAP	Wrap around (default)
	AP_WRAP_SM	Sign magnitude wrap around

Table 51: **Fixed-Point Identifier Summary** *(cont'd)*

Identifier	Description
N	This defines the number of saturation bits in overflow wrap modes.

Notes:

1. Using the AP_SAT* modes can result in higher resource usage as extra logic will be needed to perform saturation and this extra cost can be as high as 20% additional LUT usage.
2. Fixed-point math functions from the `hls_math` library do not support the `ap_[u]fixed` template parameters Q,O, and N, for quantization mode, overflow mode, and the number of saturation bits, respectively. The quantization and overflow modes are only effective when an `ap_[u]fixed` variable is on the left hand of assignment or being initialized, but not during the calculation.

Example Using ap_fixed

In this example the Vitis HLS `ap_fixed` type is used to define an 18-bit variable with 6 bits representing the numbers above the decimal point (including the sign bit) and 12-bits representing the value below the decimal point. The variable is specified as signed, the quantization mode is set to round to plus infinity and the default wrap-around mode is used for overflow.

```
#include <ap_fixed.h>
...
ap_fixed<18,6,AP_RND > my_type;
...
```

C++ Arbitrary Precision Integer Types

The native data types in C++ are on 8-bit boundaries (8, 16, 32, and 64 bits). RTL signals and operations support arbitrary bit-lengths.

Vitis HLS provides arbitrary precision data types for C++ to allow variables and operations in the C++ code to be specified with any arbitrary bit-widths: 6-bit, 17-bit, 234-bit, up to 1024 bits.



TIP: *The default maximum width allowed is 1024 bits. You can override this default by defining the macro `AP_INT_MAX_W` with a positive integer value less than or equal to 4096 before inclusion of the `ap_int.h` header file.*

Arbitrary precision data types have are two primary advantages over the native C++ types:

- Better quality hardware: If for example, a 17-bit multiplier is required, arbitrary precision types can specify that exactly 17-bit are used in the calculation.

  Without arbitrary precision data types, such a multiplication (17-bit) must be implemented using 32-bit integer data types and result in the multiplication being implemented with multiple DSP modules.

- Accurate C++ simulation/analysis: Arbitrary precision data types in the C++ code allows the C++ simulation to be performed using accurate bit-widths and for the C++ simulation to validate the functionality (and accuracy) of the algorithm before synthesis.

The arbitrary precision types in C++ have none of the disadvantages of those in C:

- C++ arbitrary types can be compiled with standard C++ compilers (there is no C++ equivalent of `apcc`).
- C++ arbitrary precision types do not suffer from Integer Promotion Issues.

It is not uncommon for users to change a file extension from `.c` to `.cpp` so the file can be compiled as C++, where neither of these issues are present.

For the C++ language, the header file `ap_int.h` defines the arbitrary precision integer data types `ap_(u)int<W>`. For example, `ap_int<8>` represents an 8-bit signed integer data type and `ap_uint<234>` represents a 234-bit unsigned integer type.

The `ap_int.h` file is located in the directory `$HLS_ROOT/include`, where `$HLS_ROOT` is the Vitis HLS installation directory.

The code shown in the following example is a repeat of the code shown in the Basic Arithmetic example in Standard Types. In this example, the data types in the top-level function to be synthesized are specified as `dinA_t`, `dinB_t`, and so on.

```
#include "cpp_ap_int_arith.h"

void cpp_ap_int_arith(din_A  inA, din_B  inB, din_C  inC, din_D  inD,
 dout_1 *out1, dout_2 *out2, dout_3 *out3, dout_4 *out4
) {

 // Basic arithmetic operations
 *out1 = inA * inB;
 *out2 = inB + inA;
 *out3 = inC / inA;
 *out4 = inD % inA;

}
```

In this latest update to this example, the C++ arbitrary precision types are used:

- Add header file `ap_int.h` to the source code.
- Change the native C++ types to arbitrary precision types `ap_int<N>` or `ap_uint<N>`, where `N` is a bit-size from 1 to 1024 (as noted above, this can be extended to 4K-bits if required).

The data types are defined in the header `cpp_ap_int_arith.h`.

Compared with the Basic Arithmetic example in Standard Types, the input data types have simply been reduced to represent the maximum size of the real input data (for example, 8-bit input `inA` is reduced to 6-bit input). The output types have been refined to be more accurate, for example, `out2`, the sum of `inA` and `inB`, need only be 13-bit and not 32-bit.

The following example shows basic arithmetic with C++ arbitrary precision types.

```
#ifndef _CPP_AP_INT_ARITH_H_
#define _CPP_AP_INT_ARITH_H_

#include <stdio.h>
#include "ap_int.h"

#define N 9

// Old data types
//typedef char dinA_t;
//typedef short dinB_t;
//typedef int dinC_t;
//typedef long long dinD_t;
//typedef int dout1_t;
//typedef unsigned int dout2_t;
//typedef int32_t dout3_t;
//typedef int64_t dout4_t;

typedef ap_int<6> dinA_t;
typedef ap_int<12> dinB_t;
typedef ap_int<22> dinC_t;
typedef ap_int<33> dinD_t;

typedef ap_int<18> dout1_t;
typedef ap_uint<13> dout2_t;
typedef ap_int<22> dout3_t;
typedef ap_int<6> dout4_t;

void cpp_ap_int_arith(dinA_t inA,dinB_t inB,dinC_t inC,dinD_t inD,dout1_t
*out1,dout2_t *out2,dout3_t *out3,dout4_t *out4);

#endif
```

If arbitrary precision integers are synthesized, it results in a design that is functionally identical to Standard Types. Rather than use the C++ `cout` operator to output the results to a file, the built-in `ap_int` method `.to_int()` is used to convert the `ap_int` results to integer types used with the standard `fprintf` function.

```
fprintf(fp, %d*%d=%d; %d+%d=%d; %d/%d=%d; %d mod %d=%d;\n,
 inA.to_int(), inB.to_int(), out1.to_int(),
 inB.to_int(), inA.to_int(), out2.to_int(),
 inC.to_int(), inA.to_int(), out3.to_int(),
 inD.to_int(), inA.to_int(), out4.to_int());
```

C++ Arbitrary Precision Integer Types: Reference Information

Vitis HLS provides a C++ template class, `ap_[u]int<>`, that implements arbitrary precision (or bit-accurate) integer data types with consistent, bit-accurate behavior between software and hardware modeling.

This class provides all arithmetic, bitwise, logical and relational operators allowed for native C integer types. In addition, this class provides methods to handle some useful hardware operations, such as allowing initialization and conversion of variables of widths greater than 64 bits.

This section includes comprehensive information on the methods, synthesis behavior, and all aspects of using the `ap_(u)int<N>` arbitrary precision integer types.

- Techniques for assigning constant and initialization values to arbitrary precision integers (including values greater than 1024-bit).
- A description of Vitis HLS helper methods, such as printing, concatenating, bit-slicing and range selection functions.
- A description of operator behavior, including a description of shift operations (a negative shift values, results in a shift in the opposite direction).

Compiling ap_[u]int<> Types

To use the `ap_[u]int<>` classes, you must include the `ap_int.h` header file in all source files that reference `ap_[u]int<>` variables.

When compiling software models that use these classes it might be necessary to specify the location of the Vitis HLS header files, for example by adding the `-I/<HLS_HOME>/include` option for `g++` compilation.

Declaring/Defining ap_[u] Variables

There are separate signed and unsigned classes:

- `ap_int<int_W>` (signed)
- `ap_uint<int_W>` (unsigned)

The template parameter `int_W` specifies the total width of the variable being declared.

User-defined types may be created with the C/C++ `typedef` statement as shown in the following examples:

```
include "ap_int.h"// use ap_[u]fixed<> types

typedef ap_uint<128> uint128_t; // 128-bit user defined type
ap_int<96> my_wide_var; // a global variable declaration
```

The default maximum width allowed is 1024 bits. This default may be overridden by defining the macro `AP_INT_MAX_W` with a positive integer value less than or equal to 4096 before inclusion of the `ap_int.h` header file.

CAUTION! *Setting the value of `AP_INT_MAX_W` too High can cause slow software compile and runtimes.*

Following is an example of overriding `AP_INT_MAX_W`:

```
#define AP_INT_MAX_W 4096 // Must be defined before next line
#include "ap_int.h"

ap_int<4096> very_wide_var;
```

Initialization and Assignment from Constants (Literals)

The class constructor and assignment operator overloads, allows initialization of and assignment to `ap_[u]int<>` variables using standard C/C++ integer literals.

This method of assigning values to `ap_[u]int<>` variables is subject to the limitations of C++ and the system upon which the software will run. This typically leads to a 64-bit limit on integer literals (for example, for those `LL` or `ULL` suffixes).

To allow assignment of values wider than 64-bits, the `ap_[u]int<>` classes provide constructors that allow initialization from a string of arbitrary length (less than or equal to the width of the variable).

By default, the string provided is interpreted as a hexadecimal value as long as it contains only valid hexadecimal digits (that is, 0-9 and a-f). To assign a value from such a string, an explicit C++ style cast of the string to the appropriate type must be made.

Following are examples of initialization and assignments, including for values greater than 64-bit, are:

```
ap_int<42> a_42b_var(-1424692392255LL); // long long decimal format
a_42b_var = 0x14BB648B13FLL; // hexadecimal format

a_42b_var = -1; // negative int literal sign-extended to full width

ap_uint<96> wide_var(“76543210fedcba9876543210”, 16); // Greater than 64-bit
wide_var = ap_int<96>(“0123456789abcdef01234567”, 16);
```

Starting with the 2023.2 release, there is a change in the initialization behavior of the `ap_[u]int/ap_[u]fixed` data type support, so that they are more compatible with the C built-in types (such as int and float).

```
ap_int<N> A[N]; // Such a declaration would result in uninitialized array
elements in all releases.

ap_int<N> B[N] = {0}; // Starting with the 2023.2 release, the whole array
will be initialized to zero. With previous releases, only the first element
is initialized to zero, other elements remained uninitialized.

ap_int<N> B[N] = { }; // With 23.2, the whole array is initialized to zero.
With previous releases, the array elements remained uninitialized.
```

Note: Due to this change, this might result in designs now failing dataflow checks as the initialization of the array element will be inferred as an addition write access to the array.

The `ap_[u]<>` constructor may be explicitly instructed to interpret the string as representing the number in radix 2, 8, 10, or 16 formats. This is accomplished by adding the appropriate radix value as a second parameter to the constructor call.

A compilation error occurs if the string literal contains any characters that are invalid as digits for the radix specified.

The following examples use different radix formats:

```
ap_int<6> a_6bit_var("101010", 2); // 42d in binary format
a_6bit_var = ap_int<6>("40", 8); // 32d in octal format
a_6bit_var = ap_int<6>("55", 10); // decimal format
a_6bit_var = ap_int<6>("2A", 16); // 42d in hexadecimal format

a_6bit_var = ap_int<6>("42", 2);   // COMPILE-TIME ERROR! "42" is not binary
```

The radix of the number encoded in the string can also be inferred by the constructor, when it is prefixed with a zero (`0`) followed by one of the following characters: "`b`", "`o`" or "`x`". The prefixes "`0b`", "`0o`" and "`0x`" correspond to binary, octal and hexadecimal formats respectively.

The following examples use alternate initializer string formats:

```
ap_int<6> a_6bit_var("0b101010", 2); // 42d in binary format
a_6bit_var = ap_int<6>("0o40", 8); // 32d in octal format
a_6bit_var = ap_int<6>("0x2A", 16); // 42d in hexidecimal format

a_6bit_var = ap_int<6>("0b42", 2); // COMPILE-TIME ERROR! "42" is not binary
```

If the bit-width is greater than 53-bits, the `ap_[u]int<>` value must be initialized with a string, for example:

```
ap_uint<72 Val("2460508560057040035.375");
```

Support for Console I/O (Printing)

As with initialization and assignment to `ap_[u]fixed<>` variables, Vitis HLS supports printing values that require more than 64-bits to represent.

Using the C++ Standard Output Stream

The easiest way to output any value stored in an `ap_[u]int` variable is to use the C++ standard output stream:

`std::cout` (`#include <iostream>` or `<iostream.h>`)

The stream insertion operator (<<) is overloaded to correctly output the full range of values possible for any given `ap_[u]fixed` variable. The following stream manipulators are also supported:

- dec (decimal)

- hex (hexadecimal)
- oct (octal)

These allow formatting of the value as indicated.

The following example uses `cout` to print values:

```
#include <iostream.h>
// Alternative: #include <iostream>

ap_ufixed<72> Val("10fedcba9876543210");

cout << Val << endl; // Yields: "313512663723845890576"
cout << hex << val << endl; // Yields: "10fedcba9876543210"
cout << oct << val << endl; // Yields: "417733456514166250310 20"
```

Using the Standard C Library

You can also use the standard C library (`#include <stdio.h>`) to print out values larger than 64-bits:

1. Convert the value to a C++ `std::string` using the `ap_[u]fixed` classes method `to_string()`.
2. Convert the result to a null-terminated C character string using the `std::string` class method `c_str()`.

Optional Argument One (Specifying the Radix)

You can pass the `ap[u]int::to_string()` method an optional argument specifying the radix of the numerical format desired. The valid radix argument values are:

- 2 (binary) (default)
- 8 (octal)
- 10 (decimal)
- 16 (hexadecimal)

Optional Argument Two (Printing as Signed Values)

A second optional argument to `ap_[u]int::to_string()` specifies whether to print the non-decimal formats as signed values. This argument is boolean. The default value is false, causing the non-decimal formats to be printed as unsigned values.

The following examples use `printf` to print values:

```
ap_int<72> Val("80fedcba9876543210");

printf("%s\n", Val.to_string().c_str()); // => "80FEDCBA9876543210"
printf("%s\n", Val.to_string(10).c_str()); // => "-2342818482890329542128"
printf("%s\n", Val.to_string(8).c_str()); // => "401773345651416625031020"
printf("%s\n", Val.to_string(16, true).c_str()); // => "-7F0123456789ABCDF0"
```

Expressions Involving ap_[u]<> types

Variables of `ap_[u]<>` types may generally be used freely in expressions involving C/C++ operators. Some behaviors may be unexpected. These are discussed in detail below.

Zero- and Sign-Extension on Assignment From Narrower to Wider Variables

When assigning the value of a narrower bit-width signed (`ap_int<>`) variable to a wider one, the value is sign-extended to the width of the destination variable, regardless of its signedness.

Similarly, an unsigned source variable is zero-extended before assignment.

Explicit casting of the source variable may be necessary to ensure expected behavior on assignment. See the following example:

```
ap_uint<10> Result;

ap_int<7> Val1 = 0x7f;
ap_uint<6> Val2 = 0x3f;

Result = Val1; // Yields: 0x3ff (sign-extended)
Result = Val2; // Yields: 0x03f (zero-padded)

Result = ap_uint<7>(Val1); // Yields: 0x07f (zero-padded)
Result = ap_int<6>(Val2); // Yields: 0x3ff (sign-extended)
```

Truncation on Assignment of Wider to Narrower Variables

Assigning the value of a wider source variable to a narrower one leads to truncation of the value. All bits beyond the most significant bit (MSB) position of the destination variable are lost.

There is no special handling of the sign information during truncation. This may lead to unexpected behavior. Explicit casting may help avoid this unexpected behavior.

Class Methods and Operators

The `ap_[u]int` types do not support implicit conversion from `wide ap_[u]int` (>64bits) to builtin C/C++ integer types. For example, the following code example return s1, because the implicit cast from `ap_int[65]` to `bool` in the if-statement returns a 0.

```
bool nonzero(ap_uint<65> data) {
   return data; // This leads to implicit truncation to 64b int
 }

int main() {
  if (nonzero((ap_uint<65>)1 << 64)) {
     return 0;
  }
  printf(FAIL\n);
  return 1;
}
```

To convert wide `ap_[u]int` types to built-in integers, use the explicit conversion functions included with the `ap_[u]int` types:

- `to_int()`
- `to_long()`
- `to_bool()`

In general, any valid operation that can be done on a native C/C++ integer data type is supported using operator overloading for `ap_[u]int` types.

In addition to these overloaded operators, some class specific operators and methods are included to ease bit-level operations.

Binary Arithmetic Operators

Standard binary integer arithmetic operators are overloaded to provide arbitrary precision arithmetic. These operators take either:

- Two operands of `ap_[u]int`, or
- One `ap_[u]int` type and one C/C++ fundamental integer data type

For example:

- char
- short
- int

The width and signedness of the resulting value is determined by the width and signedness of the operands, before sign-extension, zero-padding or truncation are applied based on the width of the destination variable (or expression). Details of the return value are described for each operator.

When expressions contain a mix of `ap_[u]int` and C/C++ fundamental integer types, the C++ types assume the following widths:

- `char` (8-bits)
- `short` (16-bits)
- `int` (32-bits)
- `long` (32-bits)
- `long long` (64-bits)

Addition

```
ap_(u)int::RType ap_(u)int::operator + (ap_(u)int op)
```

Returns the sum of:

- Two `ap_[u]int`, or
- One `ap_[u]int` and a C/C++ integer type

The width of the sum value is:

- One bit more than the wider of the two operands, or
- Two bits if and only if the wider is unsigned and the narrower is signed

The sum is treated as signed if either (or both) of the operands is of a signed type.

Subtraction

```
ap_(u)int::RType ap_(u)int::operator - (ap_(u)int op)
```

Returns the difference of two integers.

The width of the difference value is:

- One bit more than the wider of the two operands, or
- Two bits if and only if the wider is unsigned and the narrower signed

This is true before assignment, at which point it is sign-extended, zero-padded, or truncated based on the width of the destination variable.

The difference is treated as signed regardless of the signedness of the operands.

Multiplication

```
ap_(u)int::RType ap_(u)int::operator * (ap_(u)int op)
```

Returns the product of two integer values.

The width of the product is the sum of the widths of the operands.

The product is treated as a signed type if either of the operands is of a signed type.

Division

```
ap_(u)int::RType ap_(u)int::operator / (ap_(u)int op)
```

Returns the quotient of two integer values.

The width of the quotient is the width of the dividend if the divisor is an unsigned type. Otherwise, it is the width of the dividend plus one.

The quotient is treated as a signed type if either of the operands is of a signed type.

Modulus

```
ap_(u)int::RType ap_(u)int::operator % (ap_(u)int op)
```

Returns the modulus, or remainder of integer division, for two integer values.

The width of the modulus is the minimum of the widths of the operands, if they are both of the same signedness.

If the divisor is an unsigned type and the dividend is signed, then the width is that of the divisor plus one.

The quotient is treated as having the same signedness as the dividend.

IMPORTANT! *Vitis HLS synthesis of the modulus (%) operator will lead to instantiation of appropriately parameterized AMD LogiCORE divider cores in the generated RTL.*

Following are examples of arithmetic operators:

```
ap_uint<71> Rslt;

ap_uint<42> Val1 = 5;
ap_int<23> Val2 = -8;

Rslt = Val1 + Val2; // Yields: -3 (43 bits) sign-extended to 71 bits
Rslt = Val1 - Val2; // Yields: +3 sign extended to 71 bits
Rslt = Val1 * Val2; // Yields: -40 (65 bits) sign extended to 71 bits
Rslt = 50 / Val2; // Yields: -6 (33 bits) sign extended to 71 bits
Rslt = 50 % Val2; // Yields: +2 (23 bits) sign extended to 71 bits
```

Bitwise Logical Operators

The bitwise logical operators all return a value with a width that is the maximum of the widths of the two operands. It is treated as unsigned if and only if both operands are unsigned. Otherwise, it is of a signed type.

Sign-extension (or zero-padding) may occur, based on the signedness of the expression, not the destination variable.

Bitwise OR

```
ap_(u)int::RType ap_(u)int::operator | (ap_(u)int op)
```

Returns the bitwise `OR` of the two operands.

Bitwise AND

```
ap_(u)int::RType ap_(u)int::operator & (ap_(u)int op)
```

Returns the bitwise `AND` of the two operands.

Bitwise XOR

```
ap_(u)int::RType ap_(u)int::operator ^ (ap_(u)int op)
```

Returns the bitwise `XOR` of the two operands.

Unary Operators

Addition

```
ap_(u)int ap_(u)int::operator + ()
```

Returns the self copy of the `ap_[u]int` operand.

Subtraction

```
ap_(u)int::RType ap_(u)int::operator - ()
```

Returns the following:

- The negated value of the operand with the same width if it is a signed type, or
- Its width plus one if it is unsigned.

The return value is always a signed type.

Bitwise Inverse

```
ap_(u)int::RType ap_(u)int::operator ~ ()
```

Returns the bitwise-`NOT` of the operand with the same width and signedness.

Logical Invert

```
bool ap_(u)int::operator ! ()
```

Returns a Boolean `false` value if and only if the operand is *not* equal to zero (`0`).

Returns a Boolean `true` value if the operand is equal to zero (`0`).

Ternary Operators

When you use the ternary operator with the standard C `int` type, you must explicitly cast from one type to the other to ensure that both results have the same type. For example:

```
// Integer type is cast to ap_int type
ap_int<32> testc3(int a, ap_int<32> b, ap_int<32> c, bool d) {
 return d?ap_int<32>(a):b;
}
// ap_int type is cast to an integer type
ap_int<32> testc4(int a, ap_int<32> b, ap_int<32> c, bool d) {
 return d?a+1:(int)b;
}
// Integer type is cast to ap_int type
ap_int<32> testc5(int a, ap_int<32> b, ap_int<32> c, bool d) {
 return d?ap_int<33>(a):b+1;
}
```

Shift Operators

Each shift operator comes in two versions:

- One version for *unsigned* right-hand side (RHS) operands
- One version for *signed* right-hand side (RHS) operands

A negative value supplied to the signed RHS versions reverses the shift operations direction. That is, a shift by the absolute value of the RHS operand in the opposite direction occurs.

The shift operators return a value with the same width as the left-hand side (LHS) operand. As with C/C++, if the LHS operand of a shift-right is a signed type, the sign bit is copied into the most significant bit positions, maintaining the sign of the LHS operand.

Unsigned Integer Shift Right

```
ap_(u)int ap_(u)int::operator >> (ap_uint<int_W2> op)
```

Integer Shift Right

```
ap_(u)int ap_(u)int::operator >> (ap_int<int_W2> op)
```

Unsigned Integer Shift Left

```
ap_(u)int ap_(u)int::operator << (ap_uint<int_W2> op)
```

Integer Shift Left

```
ap_(u)int ap_(u)int::operator << (ap_int<int_W2> op)
```

CAUTION! *When assigning the result of a shift-left operator to a wider destination variable, some or all information may be lost. AMD recommends that you explicitly cast the shift expression to the destination type to avoid unexpected behavior.*

Following are examples of shift operations:

```
ap_uint<13> Rslt;

ap_uint<7> Val1 = 0x41;

Rslt = Val1 << 6;  // Yields: 0x0040, i.e. msb of Val1 is lost
Rslt = ap_uint<13>(Val1) << 6;  // Yields: 0x1040, no info lost

ap_int<7> Val2 = -63;
Rslt = Val2 >> 4;  //Yields: 0x1ffc, sign is maintained and extended
```

Compound Assignment Operators

Vitis HLS supports compound assignment operators:

*=
/=
%=
+=
-=
<<=
>>=
&=
^=
|=

The `RHS` expression is first evaluated then supplied as the `RHS` operand to the base operator, the result of which is assigned back to the `LHS` variable. The expression sizing, signedness, and potential sign-extension or truncation rules apply as discussed above for the relevant operations.

```
ap_uint<10> Val1 = 630;
ap_int<3> Val2 = -3;
ap_uint<5> Val3 = 27;

Val1 += Val2 - Val3; // Yields: 600 and is equivalent to:

// Val1 = ap_uint<10>(ap_int<11>(Val1) +
// ap_int<11>((ap_int<6>(Val2) -
// ap_int<6>(Val3))));
```

Increment and Decrement Operators

The increment and decrement operators are provided. All return a value of the same width as the operand and which is unsigned if and only if both operands are of unsigned types and signed otherwise.

Pre-Increment

```
ap_(u)int& ap_(u)int::operator ++ ()
```

Returns the incremented value of the operand.

Assigns the incremented value to the operand.

Post-Increment

```
const ap_(u)int ap_(u)int::operator ++ (int)
```

Returns the value of the operand before assignment of the incremented value to the operand variable.

Pre-Decrement

```
ap_(u)int& ap_(u)int::operator -- ()
```

Returns the decremented value of, as well as assigning the decremented value to, the operand.

Post-Decrement

```
const ap_(u)int ap_(u)int::operator -- (int)
```

Returns the value of the operand before assignment of the decremented value to the operand variable.

Relational Operators

Vitis HLS supports all relational operators. They return a Boolean value based on the result of the comparison. You can compare variables of `ap_[u]int` types to C/C++ fundamental integer types with these operators.

Equality

```
bool ap_(u)int::operator == (ap_(u)int op)
```

Inequality

```
bool ap_(u)int::operator != (ap_(u)int op)
```

Less than

```
bool ap_(u)int::operator < (ap_(u)int op)
```

Greater than

```
bool ap_(u)int::operator > (ap_(u)int op)
```

Less than or equal to

```
bool ap_(u)int::operator <= (ap_(u)int op)
```

Greater than or equal to

```
bool ap_(u)int::operator >= (ap_(u)int op)
```

Other Class Methods, Operators, and Data Members

The following sections discuss other class methods, operators, and data members.

Bit-Level Operations

The following methods facilitate common bit-level operations on the value stored in `ap_[u]int` type variables.

Length

```
int ap_(u)int::length ()
```

Returns an integer value providing the total number of bits in the `ap_[u]int` variable.

Concatenation

```
ap_concat_ref ap_(u)int::concat (ap_(u)int low)
ap_concat_ref ap_(u)int::operator , (ap_(u)int high, ap_(u)int low)
```

Concatenates two `ap_[u]int` variables, the width of the returned value is the sum of the widths of the operands.

The High and Low arguments are placed in the higher and lower order bits of the result respectively; the `concat()` method places the argument in the lower order bits.

When using the overloaded comma operator, the parentheses are required. The comma operator version may also appear on the LHS of assignment.

RECOMMENDED: *To avoid unexpected results, explicitly cast C/C++ native types (including integer literals) to an appropriate `ap_[u]int` type before concatenating.*

```
ap_uint<10> Rslt;

ap_int<3> Val1 = -3;
ap_int<7> Val2 = 54;

Rslt = (Val2, Val1); // Yields: 0x1B5
Rslt = Val1.concat(Val2); // Yields: 0x2B6
(Val1, Val2) = 0xAB; // Yields: Val1 == 1, Val2 == 43
```

Bit Selection

```
ap_bit_ref ap_(u)int::operator [] (int bit)
```

Selects one bit from an arbitrary precision integer value and returns it.

The returned value is a reference value that can set or clear the corresponding bit in this `ap_[u]int`.

The bit argument must be an `int` value. It specifies the index of the bit to select. The least significant bit has index 0. The highest permissible index is one less than the bit-width of this `ap_[u]int`.

The result type `ap_bit_ref` represents the reference to one bit of this `ap_[u]int` instance specified by bit.

Range Selection

```
ap_range_ref ap_(u)int::range (unsigned Hi, unsigned Lo)
ap_range_ref ap_(u)int::operator () (unsigned Hi, unsigned Lo)
```

Returns the value represented by the range of bits specified by the arguments.

The `Hi` argument specifies the most significant bit (MSB) position of the range, and `Lo` specifies the least significant bit (LSB).

The LSB of the source variable is in position 0. If the `Hi` argument has a value less than `Lo`, the bits are returned in reverse order.

```
ap_uint<4> Rslt;

ap_uint<8> Val1 = 0x5f;
ap_uint<8> Val2 = 0xaa;

Rslt = Val1.range(3, 0); // Yields: 0xF
Val1(3,0) = Val2(3, 0); // Yields: 0x5A
Val1(3,0) = Val2(4, 1); // Yields: 0x55
Rslt = Val1.range(4, 7); // Yields: 0xA; bit-reversed!
```

***Note*:** The object returned by range select is not an `ap_(u)int` object and lacks operators, but can be used for assignment. To use the range select result in a chained expression with `ap_(u)int` methods, add an explicit constructor like below.

```
ap_uint<32> v = 0x8fff0000;
bool r = ap_uint<16>(v.range(23, 8)).xor_reduce();
```

AND reduce

```
bool ap_(u)int::and_reduce ()
```

- Applies the `AND` operation on all bits in this `ap_(u)int`.
- Returns the resulting single bit.
- Equivalent to comparing this value against `-1` (all ones) and returning `true` if it matches, `false` otherwise.

OR reduce

```
bool ap_(u)int::or_reduce ()
```

- Applies the `OR` operation on all bits in this `ap_(u)int`.
- Returns the resulting single bit.
- Equivalent to comparing this value against `0` (all zeros) and returning `false` if it matches, `true` otherwise.

XOR reduce

```
bool ap_(u)int::xor_reduce ()
```

- Applies the `XOR` operation on all bits in this `ap_int`.
- Returns the resulting single bit.
- Equivalent to counting the number of `1` bits in this value and returning `false` if the count is even or `true` if the count is odd.

NAND reduce

```
bool ap_(u)int::nand_reduce ()
```

- Applies the `NAND` operation on all bits in this `ap_int`.
- Returns the resulting single bit.
- Equivalent to comparing this value against `-1` (all ones) and returning `false` if it matches, `true` otherwise.

NOR reduce

```
bool ap_int::nor_reduce ()
```

- Applies the `NOR` operation on all bits in this `ap_int`.
- Returns the resulting single bit.
- Equivalent to comparing this value against `0` (all zeros) and returning `true` if it matches, `false` otherwise.

XNOR reduce

```
bool ap_(u)int::xnor_reduce ()
```

- Applies the `XNOR` operation on all bits in this `ap_(u)int`.
- Returns the resulting single bit.
- Equivalent to counting the number of `1` bits in this value and returning `true` if the count is even or `false` if the count is odd.

Bit Reduction Method Examples

```
ap_uint<8> Val = 0xaa;

bool t = Val.and_reduce(); // Yields: false
t = Val.or_reduce();       // Yields: true
t = Val.xor_reduce();      // Yields: false
t = Val.nand_reduce();     // Yields: true
t = Val.nor_reduce();      // Yields: false
t = Val.xnor_reduce();     // Yields: true
```

Bit Reverse

```
void ap_(u)int::reverse ()
```

Reverses the contents of `ap_[u]int` instance:

- The LSB becomes the MSB.
- The MSB becomes the LSB.

Reverse Method Example

```
ap_uint<8> Val = 0x12;

Val.reverse(); // Yields: 0x48
```

Test Bit Value

```
bool ap_(u)int::test (unsigned i)
```

Checks whether specified bit of `ap_(u)int` instance is `1`.

Returns true if Yes, false if No.

Test Method Example

```
ap_uint<8> Val = 0x12;
bool t = Val.test(5); // Yields: true
```

Set Bit Value

```
void ap_(u)int::set (unsigned i, bool v)
void ap_(u)int::set_bit (unsigned i, bool v)
```

Sets the specified bit of the `ap_(u)int` instance to the value of integer `V`.

Set Bit (to 1)

```
void ap_(u)int::set (unsigned i)
```

Sets the specified bit of the `ap_(u)int` instance to the value `1` (one).

Clear Bit (to 0)

```
void ap_(u)int:: clear(unsigned i)
```

Sets the specified bit of the `ap_(u)int` instance to the value `0` (zero).

Invert Bit

```
void ap_(u)int:: invert(unsigned i)
```

Inverts the bit specified in the function argument of the `ap_(u)int` instance. The specified bit becomes `0` if its original value is `1` and vice versa.

Example of bit set, clear and invert bit methods:

```
ap_uint<8> Val = 0x12;
Val.set(0, 1); // Yields: 0x13
Val.set_bit(4, false); // Yields: 0x03
Val.set(7); // Yields: 0x83
Val.clear(1); // Yields: 0x81
Val.invert(4); // Yields: 0x91
```

Rotate Right

```
void ap_(u)int:: rrotate(unsigned n)
```

Rotates the `ap_(u)int` instance *n* places to right.

Rotate Left

```
void ap_(u)int:: lrotate(unsigned n)
```

Rotates the `ap_(u)int` instance *n* places to left.

```
ap_uint<8> Val = 0x12;

Val.rrotate(3); // Yields: 0x42
Val.lrotate(6); // Yields: 0x90
```

Bitwise NOT

```
void ap_(u)int:: b_not()
```

- Complements every bit of the `ap_(u)int` instance.

```
ap_uint<8> Val = 0x12;

Val.b_not(); // Yields: 0xED
```

Bitwise NOT Example

Test Sign

```
bool ap_int:: sign()
```

- Checks whether the `ap_(u)int` instance is negative.
- Returns `true` if negative.
- Returns `false` if positive.

Explicit Conversion Methods

To C/C++ "(u)int"

```
int ap_(u)int::to_int ()
unsigned ap_(u)int::to_uint ()
```

- Returns native C/C++ (32-bit on most systems) integers with the value contained in the `ap_[u]int`.
- Truncation occurs if the value is greater than can be represented by an `[unsigned] int`.

To C/C++ 64-bit "(u)int"

```
long long ap_(u)int::to_int64 ()
unsigned long long ap_(u)int::to_uint64 ()
```

- Returns native C/C++ 64-bit integers with the value contained in the `ap_[u]int`.
- Truncation occurs if the value is greater than can be represented by an `[unsigned] int`.

To C/C++ "double"

```
double ap_(u)int::to_double ()
```

- Returns a native C/C++ `double` 64-bit floating point representation of the value contained in the `ap_[u]int`.
- If the `ap_[u]int` is wider than 53 bits (the number of bits in the mantissa of a `double`), the resulting `double` may not have the exact value expected.

RECOMMENDED: *AMD recommends that you explicitly call member functions instead of using C-style cast to convert `ap_[u]int` to other data types.*

Sizeof

The standard C++ `sizeof()` function should not be used with `ap_[u]int` or other classes or instance of object. The `ap_int<>` data type is a class and `sizeof` returns the storage used by that class or instance object. `sizeof(ap_int<N>)` always returns the number of bytes used. For example:

```
sizeof(ap_int<127>)=16
sizeof(ap_int<128>)=16
sizeof(ap_int<129>)=24
sizeof(ap_int<130>)=24
```

Compile Time Access to Data Type Attributes

The `ap_[u]int<>` types are provided with a static member that allows the size of the variables to be determined at compile time. The data type is provided with the static const member `width`, which is automatically assigned the width of the data type:

```
static const int width = _AP_W;
```

You can use the `width` data member to extract the data width of an existing `ap_[u]int<>` data type to create another `ap_[u]int<>` data type at compile time. The following example shows how the size of variable `Res` is defined as 1-bit greater than variables `Val1` and `Val2`:

```
// Definition of basic data type
#define INPUT_DATA_WIDTH 8
typedef ap_int<INPUT_DATA_WIDTH> data_t;
// Definition of variables
data_t Val1, Val2;
// Res is automatically sized at compile-time to be 1-bit greater than data
type
data_t
ap_int<data_t::width+1> Res = Val1 + Val2;
```

This ensures that Vitis HLS correctly models the bit-growth caused by the addition even if you update the value of INPUT_DATA_WIDTH for `data_t`.

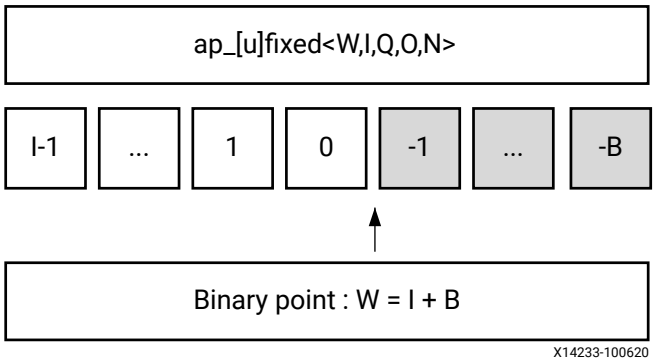
C++ Arbitrary Precision Fixed-Point Types

IMPORTANT! *The arbitrary precision fixed-point types require the header file `ap_fixed.h` to be included in the code.*

C++ functions can take advantage of the arbitrary precision fixed-point types included with Vitis HLS. The following figure summarizes the basic features of these fixed-point types:

- The word can be signed (`ap_fixed`) or unsigned (`ap_ufixed`).
- A word with of any arbitrary size `W` can be defined.
- The number of places above the decimal point I, also defines the number of decimal places in the word, `W-I` (represented by `B` in the following figure).
- The type of rounding or quantization (`Q`) can be selected.
- The overflow behavior (`O` and `N`) can be selected.

Figure 145: **Arbitrary Precision Fixed-Point Types**



TIP: *In the preceding figure the integer value (`I`) specifies the number of integer bits to the left of the binary point, including the sign bit.*

Arbitrary precision fixed-point types use more memory during C simulation and if you are using very large arrays of `ap_[u]fixed` types.

The advantages of using fixed-point types are:

- They allow fractional number to be easily represented.
- When variables have a different number of integer and decimal place bits, the alignment of the decimal point is handled.
- There are numerous options to handle how rounding should happen: when there are too few decimal bits to represent the precision of the result.
- There are numerous options to handle how variables should overflow: when the result is greater than the number of integer bits can represent.

These attributes are summarized by examining the code in the example below. First, the header file `ap_fixed.h` is included. The `ap_fixed` types are then defined using the `typedef` statement:

- A 10-bit input: 8-bit integer value with 2 decimal places.
- A 6-bit input: 3-bit integer value with 3 decimal places.
- A 22-bit variable for the accumulation: 17-bit integer value with 5 decimal places.
- A 36-bit variable for the result: 30-bit integer value with 6 decimal places.

The function contains no code to manage the alignment of the decimal point after operations are performed. The alignment is done automatically.

The following code sample shows `ap_fixed` type.

```
#include "ap_fixed.h"

typedef ap_ufixed<10,8, AP_RND, AP_SAT> din1_t;
typedef ap_fixed<6,3, AP_RND, AP_WRAP> din2_t;
typedef ap_fixed<22,17, AP_TRN, AP_SAT> dint_t;
typedef ap_fixed<36,30> dout_t;

dout_t cpp_ap_fixed(din1_t d_in1, din2_t d_in2) {

 static dint_t sum;
 sum += d_in1;
 return sum * d_in2;
}
```

Using `ap_(u)fixed` types, the C++ simulation is bit accurate. Fast simulation can validate the algorithm and its accuracy. After synthesis, the RTL exhibits the identical bit-accurate behavior.

Arbitrary precision fixed-point types can be freely assigned literal values in the code. This is shown in the test bench (see the example below) used with the example above, in which the values of `in1` and `in2` are declared and assigned constant values.

When assigning literal values involving operators, the literal values must first be cast to `ap_(u)fixed` types. Otherwise, the C compiler and Vitis HLS interpret the literal as an integer or `float/double` type and might fail to find a suitable operator. As shown in the following example, in the assignment of `in1 = in1 + din1_t(0.25)`, the literal 0.25 is cast to an `ap_fixed` type.

```
#include <cmath>
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;
#include "ap_fixed.h"

typedef ap_ufixed<10,8, AP_RND, AP_SAT> din1_t;
typedef ap_fixed<6,3, AP_RND, AP_WRAP> din2_t;
typedef ap_fixed<22,17, AP_TRN, AP_SAT> dint_t;
typedef ap_fixed<36,30> dout_t;

dout_t cpp_ap_fixed(din1_t d_in1, din2_t d_in2);
int main()
 {
 ofstream result;
 din1_t in1 = 0.25;
 din2_t in2 = 2.125;
 dout_t output;
 int retval=0;


 result.open(result.dat);
 // Persistent manipulators
 result << right << fixed << setbase(10) << setprecision(15);

 for (int i = 0; i <= 250; i++)
 {
```

```
 output = cpp_ap_fixed(in1,in2);

 result << setw(10) << i;
 result << setw(20) << in1;
 result << setw(20) << in2;
 result << setw(20) << output;
 result << endl;

 in1 = in1 + din1_t(0.25);
 in2 = in2 - din2_t(0.125);
 }
 result.close();

 // Compare the results file with the golden results
 retval = system(diff --brief -w result.dat result.golden.dat);
 if (retval != 0) {
 printf(Test failed  !!!\n);
 retval=1;
 } else {
 printf(Test passed !\n);
 }

 // Return 0 if the test passes
 return retval;
}
```

Fixed-Point Identifier Summary

The following table shows the quantization and overflow modes.



TIP: *Quantization and overflow modes that do more than the default behavior of standard hardware arithmetic (wrap and truncate) result in operators with more associated hardware. It costs logic (LUTs) to implement the more advanced modes, such as round to minus infinity or saturate symmetrically.*

Table 52: **Fixed-Point Identifier Summary**

Identifier	Description
W	Word length in bits

I	The number of bits used to represent the integer value, that is, the number of integer bits to the *left* of the binary point, including the sign bit.
	When I is negative, as shown in the example below, it represents the number of *implicit* sign bits (for signed representation), or the number of *implicit* zero bits (for unsigned representation) to the *right* of the binary point. For example,
	`ap_fixed<2, 0> a = -0.5;  // a can be -0.5,`
	`ap_ufixed<1, 0> x = 0.5;  // 1-bit representation. x can be 0 or 0.5`
	`ap_ufixed<1, -1> y = 0.25;  // 1-bit representation. y can be 0 or 0.25`
	`const ap_fixed<1, -7> z = 1.0/256;  // 1-bit representation for z = 2^-8`

Table 52: **Fixed-Point Identifier Summary** *(cont'd)*

Identifier	Description	
Q	Quantization mode: This dictates the behavior when greater precision is generated than can be defined by smallest fractional bit in the variable used to store the result.	
	ap_fixed Types	Description
	AP_RND	Round to plus infinity
	AP_RND_ZERO	Round to zero
	AP_RND_MIN_INF	Round to minus infinity
	AP_RND_INF	Round to infinity
	AP_RND_CONV	Convergent rounding
	AP_TRN	Truncation to minus infinity (default)
	AP_TRN_ZERO	Truncation to zero
O	Overflow mode: This dictates the behavior when the result of an operation exceeds the maximum (or minimum in the case of negative numbers) possible value that can be stored in the variable used to store the result.	
	ap_fixed Types	Description
	AP_SAT[1]	Saturation
	AP_SAT_ZERO[1]	Saturation to zero
	AP_SAT_SYM[1]	Symmetrical saturation
	AP_WRAP	Wrap around (default)
	AP_WRAP_SM	Sign magnitude wrap around
N	This defines the number of saturation bits in overflow wrap modes.	

Notes:

1. Using the AP_SAT* modes can result in higher resource usage as extra logic will be needed to perform saturation and this extra cost can be as high as 20% additional LUT usage.
2. Fixed-point math functions from the `hls_math` library do not support the `ap_[u]fixed` template parameters Q,O, and N, for quantization mode, overflow mode, and the number of saturation bits, respectively. The quantization and overflow modes are only effective when an `ap_[u]fixed` variable is on the left hand of assignment or being initialized, but not during the calculation.

C++ Arbitrary Precision Fixed-Point Types: Reference Information

IMPORTANT! *The arbitrary precision fixed-point types require the header file `ap_fixed.h` to be included in the code.*

This section includes comprehensive information on the methods, synthesis behavior, and all aspects of using the `ap_(u)fixed<N>` arbitrary precision fixed-point data types.

- Techniques for assigning constant and initialization values to arbitrary precision integers (including values greater than 1024-bit).
- A detailed description of the overflow and saturation modes.
- A description of Vitis HLS helper methods, such as printing, concatenating, bit-slicing and range selection functions.

- A description of operator behavior, including a description of shift operations (a negative shift values, results in a shift in the opposite direction).

Vitis HLS supports fixed-point types that allow fractional arithmetic to be easily handled. The advantage of fixed-point arithmetic is shown in the following example.

```
ap_fixed<11, 6> Var1 = 22.96875; // 11-bit signed word, 5 fractional bits
ap_ufixed<12,11> Var2 = 512.5; // 12-bit word, 1 fractional bit
ap_fixed<16,11> Res1; // 16-bit signed word, 5 fractional bits

Res1 = Var1 + Var2; // Result is 535.46875
```

Even though `Var1` and `Var2` have different precisions, the fixed-point type ensures that the decimal point is correctly aligned before the operation (an addition in this case), is performed. You are not required to perform any operations in the C code to align the decimal point.

The type used to store the result of any fixed-point arithmetic operation must be large enough (in both the integer and fractional bits) to store the full result.

If this is not the case, the `ap_fixed` type performs:

- overflow handling (when the result has more MSBs than the assigned type supports)
- quantization (or rounding, when the result has fewer LSBs than the assigned type supports)

The `ap_[u]fixed` type provides various options on how the overflow and quantization are performed. The options are discussed below.

ap_[u]fixed Representation

In `ap[u]fixed` types, a fixed-point value is represented as a sequence of bits with a specified position for the binary point.

- Bits to the left of the binary point represent the integer part of the value, including the sign bit.
- Bits to the right of the binary point represent the fractional part of the value.

`ap_[u]fixed type` is defined as follows:

```
ap_[u]fixed<int W,
 int I,
 ap_q_mode Q,
 ap_o_mode O,
 ap_sat_bits N>;
```

Quantization Modes

Rounding to plus infinity	AP_RND
Rounding to zero	AP_RND_ZERO
Rounding to minus infinity	AP_RND_MIN_INF

Rounding to infinity	AP_RND_INF
Convergent rounding	AP_RND_CONV
Truncation	AP_TRN
Truncation to zero	AP_TRN_ZERO

AP_RND

- Round the value to the nearest representable value for the specific `ap_[u]fixed` type.

```
ap_fixed<3, 2, AP_RND, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.5
ap_fixed<3, 2, AP_RND, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.0
```

AP_RND_ZERO

- Round the value to the nearest representable value.
- Round towards zero.
  - For positive values, delete the redundant bits.
  - For negative values, add the least significant bits to get the nearest representable value.

```
ap_fixed<3, 2, AP_RND_ZERO, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_RND_ZERO, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.0
```

AP_RND_MIN_INF

- Round the value to the nearest representable value.
- Round towards minus infinity.
  - For positive values, delete the redundant bits.
  - For negative values, add the least significant bits.

```
ap_fixed<3, 2, AP_RND_MIN_INF, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_RND_MIN_INF, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.5
```

AP_RND_INF

- Round the value to the nearest representable value.
- The rounding depends on the least significant bit.
  - For positive values, if the least significant bit is set, round towards plus infinity. Otherwise, round towards minus infinity.
  - For negative values, if the least significant bit is set, round towards minus infinity. Otherwise, round towards plus infinity.

```
ap_fixed<3, 2, AP_RND_INF, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.5
ap_fixed<3, 2, AP_RND_INF, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.5
```

AP_RND_CONV

- Round to the nearest representable value with "ties" rounding to even, that is, the least significant bit (after rounding) is forced to zero.
- A "tie" is the midpoint of two representable values and occurs when the bit following the least significant bit (after rounding) is 1 and all the bits below it are zero.

```
// For the following examples, bit3 of the 8-bit value becomes the
// LSB of the final 5-bit value (after rounding).
// Notes:
//   * bit7 of the 8-bit value is the MSB (sign bit)
//   * the 3 LSBs of the 8-bit value (bit2, bit1, bit0) are treated as
//     guard, round and sticky bits.
//   * See http://pages.cs.wisc.edu/~david/courses/cs552/S12/handouts/
guardbits.pdf

ap_fixed<8,3> p1 = 1.59375; // p1 = 001.10011
ap_fixed<5,3,AP_RND_CONV> rconv1 = p1; // rconv1 = 1.5 (001.10)

ap_fixed<8,3> p2 = 1.625; // p2 = 001.10100 => tie with bit3 (LSB-to-be)
= 0
ap_fixed<5,3,AP_RND_CONV> rconv2 = p2; // rconv2 = 1.5 (001.10) => lsb is
already zero, just truncate

ap_fixed<8,3> p3 = 1.375; // p3 = 001.01100 => tie with bit3 (LSB-to-be)
= 1
ap_fixed<5,3,AP_RND_CONV> rconv3 = p3; // rconv3 = 1.5 (001.10) => lsb is
made zero by rounding up

ap_fixed<8,3> p3 = 1.65625; // p3 = 001.10101
ap_fixed<5,3,AP_RND_CONV> rconv3 = p3; // rconv3 = 1.75 (001.11) => round
up
```

AP_TRN

- Always round the value towards minus infinity.

```
ap_fixed<3, 2, AP_TRN, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_TRN, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.5
```

AP_TRN_ZERO

Round the value to:

- For positive values, the rounding is the same as mode `AP_TRN`.
- For negative values, round towards zero.

```
ap_fixed<3, 2, AP_TRN_ZERO, AP_SAT> UAPFixed4 = 1.25; // Yields: 1.0
ap_fixed<3, 2, AP_TRN_ZERO, AP_SAT> UAPFixed4 = -1.25; // Yields: -1.0
```

Overflow Modes

Saturation	AP_SAT
Saturation to zero	AP_SAT_ZERO

Symmetrical saturation	AP_SAT_SYM
Wrap-around	AP_WRAP
Sign magnitude wrap-around	AP_WRAP_SM

AP_SAT

Saturate the value.

- To the maximum value in case of overflow.
- To the negative maximum value in case of negative overflow.

```
ap_fixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = 19.0; // Yields: 7.0
ap_fixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = -19.0; // Yields: -8.0
ap_ufixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = 19.0; // Yields: 15.0
ap_ufixed<4, 4, AP_RND, AP_SAT> UAPFixed4 = -19.0; // Yields: 0.0
```

AP_SAT_ZERO

Force the value to zero in case of overflow, or negative overflow.

```
ap_fixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = 19.0; // Yields: 0.0
ap_fixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = -19.0; // Yields: 0.0
ap_ufixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = 19.0; // Yields: 0.0
ap_ufixed<4, 4, AP_RND, AP_SAT_ZERO> UAPFixed4 = -19.0; // Yields: 0.0
```

AP_SAT_SYM

Saturate the value:

- To the maximum value in case of overflow.
- To the minimum value in case of negative overflow.
  - Negative maximum for signed `ap_fixed` types
  - Zero for unsigned `ap_ufixed` types

```
ap_fixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = 19.0; // Yields: 7.0
ap_fixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = -19.0; // Yields: -7.0
ap_ufixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = 19.0; // Yields: 15.0
ap_ufixed<4, 4, AP_RND, AP_SAT_SYM> UAPFixed4 = -19.0; // Yields: 0.0
```

AP_WRAP

Wrap the value around in case of overflow.

```
ap_fixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = 31.0; // Yields: -1.0
ap_fixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = -19.0; // Yields: -3.0
ap_ufixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = 19.0; // Yields: 3.0
ap_ufixed<4, 4, AP_RND, AP_WRAP> UAPFixed4 = -19.0; // Yields: 13.0
```

If the value of N is set to zero (the default overflow mode):

- All MSB bits outside the range are deleted.
- For unsigned numbers. After the maximum it wraps around to zero.
- For signed numbers. After the maximum, it wraps to the minimum values.

If N>0:

- When N > 0, N MSB bits are saturated or set to 1.
- The sign bit is retained, so positive numbers remain positive and negative numbers remain negative.
- The bits that are not saturated are copied starting from the LSB side.

AP_WRAP_SM

The value should be sign-magnitude wrapped around.

```
ap_fixed<4, 4, AP_RND, AP_WRAP_SM> UAPFixed4 = 19.0; // Yields: -4.0
ap_fixed<4, 4, AP_RND, AP_WRAP_SM> UAPFixed4 = -19.0; // Yields: 2.0
```

If the value of N is set to zero (the default overflow mode):

- This mode uses sign magnitude wrapping.
- Sign bit set to the value of the least significant deleted bit.
- If the most significant remaining bit is different from the original MSB, all the remaining bits are inverted.
- If MSBs are same, the other bits are copied over.
  1. Delete redundant MSBs.
  2. The new sign bit is the least significant bit of the deleted bits. 0 in this case.
  3. Compare the new sign bit with the sign of the new value.
- If different, invert all the numbers. They are different in this case.

If N>0:

- Uses sign magnitude saturation
- N MSBs are saturated to 1.
- Behaves similar to a case in which N = 0, except that positive numbers stay positive and negative numbers stay negative.

Compiling ap_[u]fixed<> Types

To use the `ap_[u]fixed<>` classes, you must include the `ap_fixed.h` header file in all source files that reference `ap_[u]fixed<> variables`.

When compiling software models that use these classes, it may be necessary to specify the location of the Vitis HLS header files, for example by adding the "`-I/<HLS_HOME>/include`" option for `g++` compilation.

Declaring and Defining ap_[u]fixed<> Variables

There are separate signed and unsigned classes:

- `ap_fixed<W,I>` (signed)
- `ap_ufixed<W,I>` (unsigned)

You can create user-defined types with the C/C++ `typedef` statement:

```
#include "ap_fixed.h" // use ap_[u]fixed<> types

typedef ap_ufixed<128,32> uint128_t; // 128-bit user defined type,
 //  32 integer bits
```

Initialization and Assignment from Constants (Literals)

You can initialize `ap_[u]fixed` variable with normal floating point constants of the usual C/C++ width:

- 32 bits for type `float`
- 64 bits for type `double`

That is, typically, a floating point value that is single precision type or in the form of double precision. Note that the value assigned to the fixed-point variable will be limited by the precision of the constant.

```
#include <ap_fixed.h>

ap_ufixed<30, 15> my15BitInt = 3.1415;
ap_fixed<42, 23> my42BitInt = -1158.987;
ap_ufixed<99, 40> = 287432.0382911;
ap_fixed<36,30> = -0x123.456p-1;
```

You can also use string initialization to ensure that all bits of the fixed-point variable are populated according to the precision described by the string.

```
ap_ufixed<2,  0> x     = "0b0.01";      // 0.25 in "dot" format
ap_ufixed<2,  0> y     = "0b01p-2";     // 0.25 in binary "scientific"
format
ap_ufixed<2,  0> z     = "0x4p-4";     // 0.25 in hex "scientific" format
ap_ufixed<62, 2> my_pi =
"0b11.0010010000111111011010101000100010000101101000110000100011101"; // pi
with 60 fractional bits
```

The `ap_[u]fixed` types do not support initialization if they are used in an array of `std::complex` types.

```
typedef ap_fixed<DIN_W, 1, AP_TRN, AP_SAT> coeff_t; // MUST have IW >= 1
std::complex<coeff_t> twid_rom[REAL_SZ/2] = {{ 1, -0 },{ 0.9,-0.006 }, etc.}
```

The initialization values must first be cast to `std::complex`:

```
typedef ap_fixed<DIN_W, 1, AP_TRN, AP_SAT> coeff_t; // MUST have IW >= 1
std::complex<coeff_t> twid_rom[REAL_SZ/2] = {std::complex<coeff_t>( 1,
-0 ),
std::complex<coeff_t>(0.9,-0.006 ),etc.}
```

Support for Console I/O (Printing)

As with initialization and assignment to `ap_[u]fixed<>` variables, Vitis HLS supports printing values that require more than 64 bits to represent.

The easiest way to output any value stored in an `ap_[u]fixed` variable is to use the C++ standard output stream, `std::cout (#include <iostream> or <iostream.h>)`. The stream insertion operator, "<<", is overloaded to correctly output the full range of values possible for any given `ap_[u]fixed` variable. The following stream manipulators are also supported, allowing formatting of the value as shown.

- `dec` (decimal)
- `hex` (hexadecimal)
- `oct` (octal)

```
#include <iostream.h>
// Alternative: #include <iostream>

ap_fixed<6,3, AP_RND, AP_WRAP> Val = 3.25;

cout << Val << endl;     // Yields: 3.25
```

Using the Standard C Library

You can also use the standard C library (`#include <stdio.h>`) to print out values larger than 64-bits:

1. Convert the value to a C++ `std::string` using the `ap_[u]fixed` classes method `to_string()`.
2. Convert the result to a null-terminated C character string using the `std::string` class method `c_str()`.

Optional Argument One (Specifying the Radix)

You can pass the `ap[u]int::to_string()` method an optional argument specifying the radix of the numerical format desired. The valid radix argument values are:

- 2 (binary)
- 8 (octal
- 10 (decimal)
- 16 (hexadecimal) (default)

Optional Argument Two (Printing as Signed Values)

A second optional argument to `ap_[u]int::to_string()` specifies whether to print the non-decimal formats as signed values. This argument is boolean. The default value is false, causing the non-decimal formats to be printed as unsigned values.

```
ap_fixed<6,3, AP_RND, AP_WRAP> Val = 3.25;

printf("%s \n", in2.to_string().c_str()); // Yields: 0b011.010
printf("%s \n", in2.to_string(10).c_str()); //Yields: 3.25
```

The `ap_[u]fixed` types are supported by the following C++ manipulator functions:

- setprecision
- setw
- setfill

The setprecision manipulator sets the decimal precision to be used. It takes one parameter `f` as the value of decimal precision, where `n` specifies the maximum number of meaningful digits to display in total (counting both those before and those after the decimal point).

The default value of `f` is 6, which is consistent with native C float type.

```
ap_fixed<64, 32> f =3.14159;
cout << setprecision (5) << f << endl;
cout << setprecision (9) << f << endl;
f = 123456;
cout << setprecision (5) << f << endl;
```

The example above displays the following results where the printed results are rounded when the actual precision exceeds the specified precision:

```
   3.1416
   3.14159
   1.2346e+05
```

The `setw` manipulator:

- Sets the number of characters to be used for the field width.

- Takes one parameter `w` as the value of the width

  where

  - `w` determines the minimum number of characters to be written in some output representation.

If the standard width of the representation is shorter than the field width, the representation is padded with fill characters. Fill characters are controlled by the setfill manipulator which takes one parameter `f` as the padding character.

For example, given:

```
ap_fixed<65,32> aa = 123456;
int precision = 5;
cout<<setprecision(precision)<<setw(13)<<setfill('T')<<a<<endl;
```

The output is:

```
 TTT1.2346e+05
```

Expressions Involving ap_[u]fixed<> types

Arbitrary precision fixed-point values can participate in expressions that use any operators supported by C/C++. After an arbitrary precision fixed-point type or variable is defined, their usage is the same as for any floating point type or variable in the C/C++ languages.

Observe the following caveats:

- Zero and Sign Extensions

  All values of smaller bit-width are zero or sign-extended depending on the sign of the source value. You may need to insert casts to obtain alternative signs when assigning smaller bit-widths to larger.

- Truncations

  Truncation occurs when you assign an arbitrary precision fixed-point of larger bit-width than the destination variable.

Class Methods, Operators, and Data Members

In general, any valid operation that can be done on a native C/C++ integer data type is supported (using operator overloading) for `ap_[u]fixed` types. In addition to these overloaded operators, some class specific operators and methods are included to ease bit-level operations.

Binary Arithmetic Operators

Addition

```
ap_[u]fixed::RType ap_[u]fixed::operator + (ap_[u]fixed op)
```

Adds an arbitrary precision fixed-point with a given operand `op`.

The operands can be any of the following integer types:

- ap_[u]fixed
- ap_[u]int
- C/C++

The result type `ap_[u]fixed::RType` depends on the type information of the two operands.

```
ap_fixed<76, 63> Result;

ap_fixed<5, 2> Val1 = 1.125;
ap_fixed<75, 62> Val2 = 6721.35595703125;

Result = Val1 + Val2; //Yields 6722.480957
```

Because `Val2` has the larger bit-width on both integer part and fraction part, the result type has the same bit-width and plus one to be able to store all possible result values.

Specifying the data's width controls resources by using the power functions, as shown below. In similar cases, AMD recommends specifying the width of the stored result instead of specifying the width of fixed point operations.

```
ap_ufixed<16,6> x=5;
ap_ufixed<16,7>y=hls::rsqrt<16,6>(x+x);
```

Subtraction

```
ap_[u]fixed::RType ap_[u]fixed::operator - (ap_[u]fixed op)
```

Subtracts an arbitrary precision fixed-point with a given operand `op`.

The result type `ap_[u]fixed::RType` depends on the type information of the two operands.

```
ap_fixed<76, 63> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val2 - Val1; // Yields 6720.23057
```

Because `Val2` has the larger bit-width on both integer part and fraction part, the result type has the same bit-width and plus one to be able to store all possible result values.

Multiplication

```
ap_[u]fixed::RType ap_[u]fixed::operator * (ap_[u]fixed op)
```

Multiplies an arbitrary precision fixed-point with a given operand `op`.

```
ap_fixed<80, 64> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 * Val2; // Yields 7561.525452
```

This shows the multiplication of `Val1` and `Val2`. The result type is the sum of their integer part bit-width and their fraction part bit width.

Division

```
ap_[u]fixed::RType ap_[u]fixed::operator / (ap_[u]fixed op)
```

Divides an arbitrary precision fixed-point by a given operand `op`.

```
ap_fixed<84, 66> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Val2 / Val1; // Yields 5974.538628
```

This shows the division of `Val2` and `Val1`. To preserve enough precision:

- The integer bit-width of the result type is sum of the integer bit-width of `Val2` and the fraction bit-width of `Val1`.
- The fraction bit-width of the result type is equal to the fraction bit-width of `Val2`.

Bitwise Logical Operators

Bitwise OR

```
ap_[u]fixed::RType ap_[u]fixed::operator | (ap_[u]fixed op)
```

Applies a bitwise operation on an arbitrary precision fixed-point and a given operand `op`.

```
ap_fixed<75, 62> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 | Val2; // Yields 6271.480957
```

Bitwise AND

```
ap_[u]fixed::RType ap_[u]fixed::operator & (ap_[u]fixed op)
```

Applies a bitwise operation on an arbitrary precision fixed-point and a given operand `op`.

```
ap_fixed<75, 62> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 & Val2;  // Yields 1.00000
```

Bitwise XOR

```
ap_[u]fixed::RType ap_[u]fixed::operator ^ (ap_[u]fixed op)
```

Applies an `xor` bitwise operation on an arbitrary precision fixed-point and a given operand `op`.

```
ap_fixed<75, 62> Result;

ap_fixed<5, 2> Val1 = 1625.153;
ap_fixed<75, 62> Val2 = 6721.355992351;

Result = Val1 ^ Val2; // Yields 6720.480957
```

Increment and Decrement Operators

Pre-Increment

```
ap_[u]fixed ap_[u]fixed::operator ++ ()
```

This operator function prefix increases an arbitrary precision fixed-point variable by `1`.

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = ++Val1; // Yields 6.125000
```

Post-Increment

```
ap_[u]fixed ap_[u]fixed::operator ++ (int)
```

This operator function postfix:

- Increases an arbitrary precision fixed-point variable by `1`.
- Returns the original val of this arbitrary precision fixed-point.

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = Val1++; // Yields 5.125000
```

Pre-Decrement

```
ap_[u]fixed ap_[u]fixed::operator -- ()
```

This operator function prefix decreases this arbitrary precision fixed-point variable by `1`.

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = --Val1; // Yields 4.125000
```

Post-Decrement

```
ap_[u]fixed ap_[u]fixed::operator -- (int)
```

This operator function postfix:

- Decreases this arbitrary precision fixed-point variable by `1`.
- Returns the original val of this arbitrary precision fixed-point.

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = Val1--; // Yields 5.125000
```

Unary Operators

Addition

```
ap_[u]fixed ap_[u]fixed::operator + ()
```

Returns a self copy of an arbitrary precision fixed-point variable.

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = +Val1;  // Yields 5.125000
```

Subtraction

```
ap_[u]fixed::RType ap_[u]fixed::operator - ()
```

Returns a negative value of an arbitrary precision fixed-point variable.

```
ap_fixed<25, 8> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = -Val1; // Yields -5.125000
```

Equality Zero

```
bool ap_[u]fixed::operator ! ()
```

This operator function:

- Compares an arbitrary precision fixed-point variable with `0`,
- Returns the result.

```
bool  Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = !Val1; // Yields false
```

Bitwise Inverse

```
ap_[u]fixed::RType ap_[u]fixed::operator ~ ()
```

Returns a bitwise complement of an arbitrary precision fixed-point variable.

```
ap_fixed<25, 15> Result;
ap_fixed<8, 5> Val1 = 5.125;

Result = ~Val1; // Yields -5.25
```

Shift Operators

Unsigned Shift Left

```
ap_[u]fixed ap_[u]fixed::operator << (ap_uint<_W2> op)
```

This operator function:

- Shifts left by a given integer operand.
- Returns the result.

The operand can be a C/C++ integer type:

- `char`
- `short`
- `int`
- `long`

The return type of the shift left operation is the same width as the type being shifted.

***Note*:** Shift does not support overflow or quantization modes.

```
ap_fixed<25, 15> Result;
ap_fixed<8, 5> Val = 5.375;

ap_uint<4> sh = 2;

Result = Val << sh; // Yields -10.5
```

The bit-width of the result is (`W = 25`, `I = 15`). Because the shift left operation result type is same as the type of `Val`:

- The high order two bits of `Val` are shifted out.
- The result is -10.5.

If a result of 21.5 is required, `Val` must be cast to `ap_fixed<10, 7>` first -- for example, `ap_ufixed<10, 7>(Val)`.

Signed Shift Left

```
ap_[u]fixed ap_[u]fixed::operator << (ap_int<_W2> op)
```

This operator:

- Shifts left by a given integer operand.
- Returns the result.

The shift direction depends on whether the operand is positive or negative.

- If the operand is positive, a shift right is performed.
- If the operand is negative, a shift left (opposite direction) is performed.

The operand can be a C/C++ integer type:

- `char`
- `short`
- `int`
- `long`

The return type of the shift right operation is the same width as the type being shifted.

```
ap_fixed<25, 15,  false> Result;
ap_uint<8, 5> Val = 5.375;

ap_int<4> Sh = 2;
Result = Val << sh; // Shift left, yields -10.25

Sh = -2;
Result = Val << sh; // Shift right, yields 1.25
```

Unsigned Shift Right

```
ap_[u]fixed ap_[u]fixed::operator >> (ap_uint<_W2> op)
```

This operator function:

- Shifts right by a given integer operand.
- Returns the result.

The operand can be a C/C++ integer type:

- `char`
- `short`
- `int`
- `long`

The return type of the shift right operation is the same width as the type being shifted.

```
ap_fixed<25, 15> Result;
ap_fixed<8, 5> Val = 5.375;

ap_uint<4> sh = 2;

Result = Val >> sh; // Yields 1.25
```

If it is necessary to preserve all significant bits, extend fraction part bit-width of the `Val` first, for example `ap_fixed<10, 5>(Val)`.

Signed Shift Right

```
ap_[u]fixed ap_[u]fixed::operator >> (ap_int<_W2> op)
```

This operator:

- Shifts right by a given integer operand.
- Returns the result.

The shift direction depends on whether operand is positive or negative.

- If the operand is positive, a shift right performed.
- If operand is negative, a shift left (opposite direction) is performed.

The operand can be a C/C++ integer type (`char`, `short`, `int`, or `long`).

The return type of the shift right operation is the same width as type being shifted. For example:

```
ap_fixed<25, 15,  false> Result;
ap_uint<8, 5> Val = 5.375;

ap_int<4> Sh = 2;
Result = Val >> sh; // Shift right, yields 1.25

Sh = -2;
Result = Val >> sh; // Shift left,  yields -10.5

1.25
```

Relational Operators

Equality

```
bool ap_[u]fixed::operator == (ap_[u]fixed op)
```

This operator compares the arbitrary precision fixed-point variable with a given operand.

Returns `true` if they are equal and `false` if they are *not* equal.

The type of operand `op` can be `ap_[u]fixed`, `ap_int` or C/C++ integer types. For example:

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 == Val2; // Yields  true
Result = Val1 == Val3; // Yields  false
```

Inequality

```
bool ap_[u]fixed::operator != (ap_[u]fixed op)
```

This operator compares this arbitrary precision fixed-point variable with a given operand.

Returns `true` if they are *not* equal and `false` if they are equal.

The type of operand `op` can be:

- ap_[u]fixed
- ap_int
- C or C++ integer types

For example:

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 != Val2; // Yields false
Result = Val1 != Val3; // Yields true
```

Greater than or equal to

```
bool ap_[u]fixed::operator >= (ap_[u]fixed op)
```

This operator compares a variable with a given operand.

Returns `true` if they are equal or if the variable is greater than the operator and `false` otherwise.

The type of operand `op` can be `ap_[u]fixed`, `ap_int` or C/C++ integer types.

For example:

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 >= Val2; // Yields true
Result = Val1 >= Val3; // Yields false
```

Less than or equal to

```
bool ap_[u]fixed::operator <= (ap_[u]fixed op)
```

This operator compares a variable with a given operand, and return `true` if it is equal to or less than the operand and `false` if not.

The type of operand op can be `ap_[u]fixed`, `ap_int` or C/C++ integer types.

For example:

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 <= Val2; // Yields true
Result = Val1 <= Val3; // Yields true
```

Greater than

```
bool ap_[u]fixed::operator > (ap_[u]fixed op)
```

This operator compares a variable with a given operand, and return `true` if it is greater than the operand and `false` if not.

The type of operand `op` can be `ap_[u]fixed`, `ap_int`, or C/C++ integer types.

For example:

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 > Val2; // Yields false
Result = Val1 > Val3; // Yields false
```

Less than

```
bool ap_[u]fixed::operator < (ap_[u]fixed op)
```

This operator compares a variable with a given operand, and return `true` if it is less than the operand and `false` if not.

The type of operand op can be `ap_[u]fixed`, `ap_int`, or C/C++ integer types. For example:

```
bool Result;

ap_ufixed<8, 5> Val1 = 1.25;
ap_fixed<9, 4> Val2 = 17.25;
ap_fixed<10, 5> Val3 = 3.25;

Result = Val1 < Val2; // Yields false
Result = Val1 < Val3; // Yields true
```

Bit Operator

Bit-Select and Set

```
af_bit_ref ap_[u]fixed::operator [] (int bit)
```

This operator selects one bit from an arbitrary precision fixed-point value and returns it.

The returned value is a reference value that can set or clear the corresponding bit in the `ap_[u]fixed` variable. The bit argument must be an integer value and it specifies the index of the bit to select. The least significant bit has index `0`. The highest permissible index is one less than the bit-width of this `ap_[u]fixed` variable.

The result type is `af_bit_ref` with a value of either `0` or `1`. For example:

```
ap_int<8, 5> Value = 1.375;

Value[3]; // Yields  1
Value[4]; // Yields  0

Value[2] = 1; // Yields 1.875
Value[3] = 0; // Yields 0.875
```

Bit Range

```
af_range_ref af_(u)fixed::range (unsigned Hi, unsigned Lo)
af_range_ref af_(u)fixed::operator [] (unsigned Hi, unsigned Lo)
```

This operation is similar to bit-select operator [] except that it operates on a range of bits instead of a single bit.

It selects a group of bits from the arbitrary precision fixed-point variable. The `Hi` argument provides the upper range of bits to be selected. The `Lo` argument provides the lowest bit to be selected. If `Lo` is larger than `Hi` the bits selected are returned in the reverse order.

The return type `af_range_ref` represents a reference in the range of the `ap_[u]fixed` variable specified by `Hi` and `Lo`. For example:

```
ap_uint<4> Result = 0;
ap_ufixed<4, 2> Value = 1.25;
ap_uint<8> Repl = 0xAA;

Result = Value.range(3, 0); // Yields: 0x5
Value(3, 0) = Repl(3, 0); // Yields: -1.5

// when Lo > Hi, return the reverse bits string
Result = Value.range(0, 3); // Yields: 0xA
```

Range Select

```
af_range_ref af_(u)fixed::range ()
af_range_ref af_(u)fixed::operator []
```

This operation is the special case of the range select operator `[]`. It selects all bits from this arbitrary precision fixed-point value in the normal order.

The return type `af_range_ref` represents a reference to the range specified by Hi = W - 1 and Lo = 0. For example:

```
ap_uint<4> Result = 0;

ap_ufixed<4, 2> Value = 1.25;
ap_uint<8> Repl = 0xAA;

Result = Value.range(); // Yields: 0x5
Value() = Repl(3, 0); // Yields: -1.5
```

Length

```
int ap_[u]fixed::length ()
```

This function returns an integer value that provides the number of bits in an arbitrary precision fixed-point value. It can be used with a type or a value. For example:

```
ap_ufixed<128, 64> My128APFixed;

int bitwidth = My128APFixed.length(); // Yields 128
```

Explicit Conversion Methods

Fixed to Double

```
double ap_[u]fixed::to_double ()
```

This member function returns this fixed-point value in form of IEEE double precision format. For example:

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
double Result;

Result = MyAPFixed.to_double(); // Yields 333.789
```

Fixed to Float

```
float ap_[u]fixed::to_float()
```

This member function returns this fixed-point value in form of IEEE float precision format. For example:

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
float Result;

Result = MyAPFixed.to_float();  // Yields 333.789
```

Fixed to Half-Precision Floating Point

```
half ap_[u]fixed::to_half()
```

This member function return this fixed-point value in form of HLS half-precision (16-bit) float precision format. For example:

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
half Result;

Result = MyAPFixed.to_half();  // Yields 333.789
```

Fixed to ap_int

```
ap_int ap_[u]fixed::to_ap_int ()
```

This member function explicitly converts this fixed-point value to `ap_int` that captures all integer bits (fraction bits are truncated). For example:

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
ap_uint<77> Result;

Result = MyAPFixed.to_ap_int(); //Yields 333
```

Fixed to Integer

```
int ap_[u]fixed::to_int ()
unsigned ap_[u]fixed::to_uint ()
ap_slong ap_[u]fixed::to_int64 ()
ap_ulong ap_[u]fixed::to_uint64 ()
```

This member function explicitly converts this fixed-point value to C built-in integer types. For example:

```
ap_ufixed<256, 77> MyAPFixed = 333.789;
unsigned int  Result;

Result = MyAPFixed.to_uint(); //Yields 333

unsigned long long Result;
Result = MyAPFixed.to_uint64(); //Yields 333
```

RECOMMENDED: *AMD recommends that you explicitly call member functions instead of using C-style cast to convert `ap_[u]fixed` to other data types.*

Compile Time Access to Data Type Attributes

The `ap_[u]fixed<>` types are provided with several static members that allow the size and configuration of data types to be determined at compile time. The data type is provided with the `static const` members: `width`, `iwidth`, `qmode` and `omode`:

```
static const int width = _AP_W;
static const int iwidth = _AP_I;
static const ap_q_mode qmode = _AP_Q;
static const ap_o_mode omode = _AP_O;
```

You can use these data members to extract the following information from any existing `ap_[u]fixed<>` data type:

- **`width`:** The width of the data type.
- **`iwidth`:** The width of the integer part of the data type.
- **`qmode`:** The quantization mode of the data type.

- `omode`: The overflow mode of the data type.

For example, you can use these data members to extract the data width of an existing `ap_[u]fixed<>` data type to create another `ap_[u]fixed<>` data type at compile time.

The following example shows how the size of variable `Res` is automatically defined as 1-bit greater than variables `Val1` and `Val2` with the same quantization modes:

```
// Definition of basic data type
#define INPUT_DATA_WIDTH 12
#define IN_INTG_WIDTH 6
#define IN_QMODE AP_RND_ZERO
#define IN_OMODE AP_WRAP
typedef ap_fixed<INPUT_DATA_WIDTH, IN_INTG_WIDTH, IN_QMODE, IN_OMODE>
data_t;
// Definition of variables
data_t Val1, Val2;
// Res is automatically sized at run-time to be 1-bit greater than
INPUT_DATA_WIDTH
// The bit growth in Res will be in the integer bits
ap_int<data_t::width+1, data_t::iwidth+1, data_t::qmode, data_t::omode> Res
= Val1 +
Val2;
```

This ensures that Vitis HLS correctly models the bit-growth caused by the addition even if you update the value of INPUT_DATA_WIDTH, IN_INTG_WIDTH, or the quantization modes for `data_t`.

HLS Print Function

The `hls::print` function is similar to `printf` in C, because it prints a format string and a single optional `int` or `double` argument to standard output, and to the simulation log in C simulation, RTL co-simulation and HW emulation.

However, it is limited to printing at most one argument, with a restricted set of datatypes, as mentioned below. It can also change the initiation interval and latency of a pipeline, so it must used very sparingly.

The `hls::print` function can be used to:

- Trace the values of selected variables
- Trace the order in which code blocks are executed across complex control and concurrent execution (for example in dataflow)



TIP: *It cannot be used to trace the order in which individual statements are scheduled within a basic block of code, because the scheduler can significantly change that order.*

When used in this simple example:

```
#include "hls_print.h"
...
  for (int i=0; i<N; i++) {
#pragma HLS pipeline ii=1
   hls::print("loop %d\n", i);
...
```

It prints the value of "i" at each iteration of the loop in both C simulation, SW emulation, RTL co-simulation, and HW emulation (it is currently ignored when the target is a HW implementation).

Note the following:

- `hls::print` is supported only in Verilog RTL.
- The only supported format specifiers are:
  - %d for integer or unsigned
  - %f for float or double
- Values of type `long` and `long long`, and the unsigned variants, must be cast to `int` or `uint` (due to the argument promotion rules of C++).

- By adding an "observation" point, insertion of `hls::print` can alter the optimizations performed by HLS. Thus it can change the behavior of the RTL (just like a `printf` in SW can alter the behavior of the binary, but much more dramatically due to the nature of HLS).
- Only a single `int` or `double` value can be passed, to minimize the above mentioned impact.
- The order of execution of different `hls::print` functions within a code block can change due to optimizations and scheduling.
- In RTL the current simulation time is printed out as well, to ease debugging.
- The amount of data that it produces can be significant, thus it should not be used to dump large arrays.

HLS Math Library

The AMD Vitis™ HLS Math Library (`hls_math.h`) provides coverage of math functions from C++ (`cmath`) libraries, and can be used in both C simulation and synthesis. It offers floating-point (single-precision, double-precision, and half-precision) for all functions and fixed-point support for the majority of the functions. The functions in `hls_math.h` is grouped in `hls` namespace, and can be used as in-place replacement of function of `std` namespace from the standard C++ math library (`cmath`).

IMPORTANT! *Using `hls_math.h` header in C code is not supported.*

HLS Math Library Accuracy

The HLS math functions are implemented as synthesizable bit-approximate functions from the `hls_math.h` library. Bit-approximate HLS math library functions do not provide the same accuracy as the standard C function. To achieve the desired result, the bit-approximate implementation might use a different underlying algorithm than the standard C math library version. The accuracy of the function is specified in terms of ULP (Unit of Least Precision). This difference in accuracy has implications for both C simulation and C/RTL co-simulation.

The ULP difference is typically in the range of 1-4 ULP.

- If the standard C math library is used in the C source code, there may be a difference between the C simulation and the C/RTL co-simulation due to the fact that some functions exhibit a ULP difference from the standard C math library.
- If the HLS math library is used in the C source code, there will be no difference between the C simulation and the C/RTL co-simulation. A C simulation using the HLS math library, may however differ from a C simulation using the standard C math library.

In addition, the following seven functions might show some differences, depending on the C standard used to compile and run the C simulation:

- copysign
- fpclassify
- isinf

- isfinite
- isnan
- isnormal
- signbit

C90 mode

Only `isinf`, `isnan`, and `copysign` are usually provided by the system header files, and they operate on doubles. In particular, `copysign` always returns a double result. This might result in unexpected results after synthesis if it must be returned to a float, because a double-to-float conversion block is introduced into the hardware.

C99 mode (-std=c99)

All seven functions are usually provided under the expectation that the system header files will redirect them to `__isnan(double)` and `__isnan(float)`. The usual GCC header files do not redirect `isnormal`, but implement it in terms of `fpclassify`.

C++ Using math.h

All seven are provided by the system header files, and they operate on doubles.

`copysign` always returns a double result. This might cause unexpected results after synthesis if it must be returned to a float, because a double-to-float conversion block is introduced into the hardware.

C++ Using cmath

Similar to C99 `mode(-std=c99)`, except that:

- The system header files are usually different.
- The functions are properly overloaded for:
  - `float(). snan(double)`
  - `isinf(double)`

`copysign` and `copysignf` are handled as built-ins even when using `namespace std;`.

C++ Using cmath and namespace std

No issues. AMD recommends using the following for best results:

- `-std=c99` for C
- `-fno-builtin` for C and C++

***Note*:** To specify the C compile options, such as `-std=c99`, use the Tcl command `add_files` with the `-cflags` option. Alternatively, use the **Edit CFLAGs** button in the Project Settings dialog box.

HLS Math Library

The following functions are provided in the HLS math library. Each function supports half-precision (type `half`), single-precision (type `float`) and double precision (type `double`).

IMPORTANT! *For each function `func` listed below, there is also an associated half-precision only function named `half_func` and single-precision only function named `funcf` provided in the library.*

When mixing half-precision, single-precision and double-precision data types, check for common synthesis errors to prevent introducing type-conversion hardware in the final FPGA implementation.

Trigonometric Functions

acos	acospi	asin	asinpi
atan	atan2	atan2pi	cos
cospi	sin	sincos	sinpi
tan	tanpi		

Hyperbolic Functions

acosh	asinh	atanh	cosh
sinh	tanh		

Exponential Functions

exp	exp10	exp2	expm1
frexp	ldexp	modf	

Logarithmic Functions

ilogb	log	log10	log1p

Power Functions

cbrt	hypot	pow	rsqrt
sqrt			

Error Functions

erf	erfc

Rounding Functions

ceil	floor	llrint	llround
lrint	lround	nearbyint	rint
round	trunc		

Remainder Functions

fmod	remainder	remquo

Floating-point

copysign	nan	nextafter	nexttoward

Difference Functions

fdim	fmax	fmin	maxmag
minmag			

IMPORTANT! *Vitis HLS does not support Technical Specification 18661-1 sNaN handling in `fmin`/`fmax` yet, This can lead to result differences when compared with `glibc 2.25` and later results.*

Other Functions

abs	divide	fabs	fma
fract	mad	recip	

Classification Functions

fpclassify	isfinite	isinf	isnan
isnormal	signbit		

Comparison Functions

isgreater	isgreaterequal	isless	islessequal
islessgreater	isunordered		

Relational Functions

all	any	bitselect	isequal
isnotequal	isordered	select	

Fixed-Point Math Functions

Fixed-point implementations are also provided for the following math functions.

All fixed-point math functions support `ap_[u]fixed` and `ap_[u]int` data types with the following bit-width specification,

1. `ap_fixed<W,I>` where I<=33 and W-I<=32
2. `ap_ufixed<W,I>` where I<=32 and W-I<=32
3. `ap_int<I>` where I<=33
4. `ap_uint<I>` where I<=32

IMPORTANT! *Fixed-point math functions from the `hls_math` library do not support the `ap_[u]fixed` template parameters Q,O, and N, for quantization mode, overflow mode, and the number of saturation bits, respectively. The quantization and overflow modes are only effective when an `ap_[u]fixed` variable is on the left hand of assignment or being initialized, but not during the calculation.*

Trigonometric Functions

cos	sin	tan	acos	asin	atan	atan2	sincos
cospi	sinpi						

Hyperbolic Functions

cosh	sinh	tanh	acosh	asinh	atanh

Exponential Functions

exp	frexp	modf	exp2	expm1

Logarithmic Functions

log	log10	ilogb	log1p

Power Functions

pow	sqrt	rsqrt	cbrt	hypot

Error Functions

erf	erfc

Rounding Functions

ceil	floor	trunc	round	rint	nearbyint

Floating Point

nextafter	nexttoward

Difference Functions

erf	erfc	fdim	fmax	fmin	maxmag	minmag

Other Functions

fabs	recip	abs	fract	divide

Classification Functions

signbit

Comparison Functions

isgreater	isgreaterequal	isless	islessequal	islessgreater

Relational Functions

isequal	isnotequal	any	all	bitselect

The fixed-point type provides a slightly-less accurate version of the function value, but a smaller and faster RTL implementation.

The methodology for implementing a math function with a fixed-point data types is:

1. Determine if a fixed-point implementation is supported.
2. Update the math functions to use `ap_fixed` types.
3. Perform C simulation to validate the design still operates with the required precision. The C simulation is performed using the same bit-accurate types as the RTL implementation.
4. Synthesize the design.

For example, a fixed-point implementation of the function `sin` is specified by using fixed-point types with the math function as follows:

```
#include "hls_math.h"
#include "ap_fixed.h"

ap_fixed<26,6> my_input, my_output;

my_input = 24.675;
my_output = sin(my_input);
```

Verification and Math Functions

If the standard C math library is used in the C source code, the C simulation results and the C/RTL co-simulation results may be different: if any of the math functions in the source code have an ULP difference from the standard C math library it may result in differences when the RTL is simulated.

If the `hls_math.h` library is used in the C source code, the C simulation and C/RTL co-simulation results are identical. However, the results of C simulation using `hls_math.h` are not the same as those using the standard C libraries. The `hls_math.h` library simply ensures the C simulation matches the C/RTL co-simulation results. In both cases, the same RTL implementation is created. The following explains each of the possible options which are used to perform verification when using math functions.

Verification Option 1: Standard Math Library and Verify Differences

In this option, the standard C math libraries are used in the source code. If any of the functions synthesized do have exact accuracy the C/RTL co-simulation is different than the C simulation. The following example highlights this approach.

```
#include <cmath>
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

typedef float data_t;

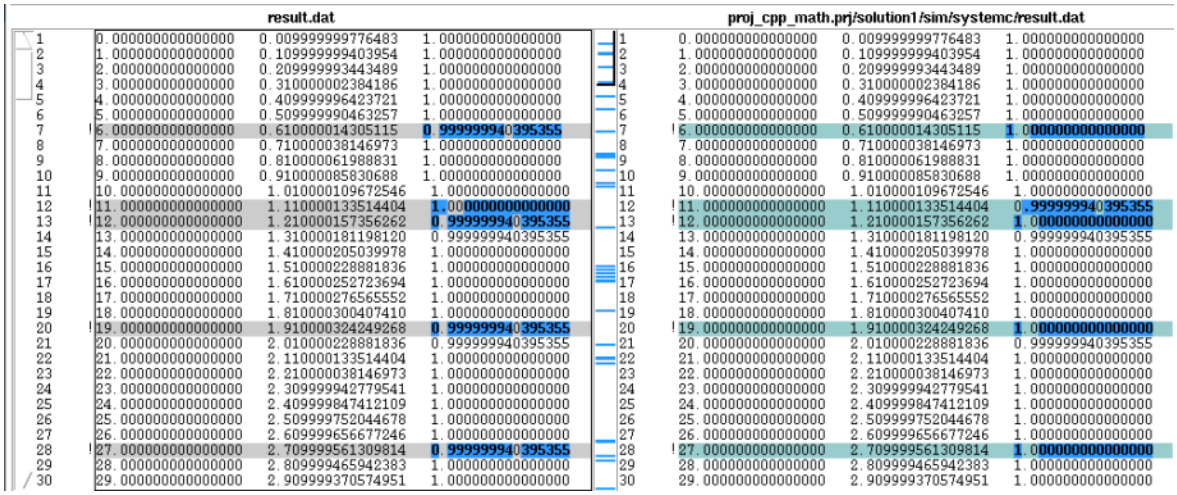
data_t cpp_math(data_t angle) {
     data_t s = sinf(angle);
     data_t c = cosf(angle);
     return sqrtf(s*s+c*c);
}
```

In this case, the results between C simulation and C/RTL co-simulation are different. Keep in mind when comparing the outputs of simulation, any results written from the test bench are written to the working directory where the simulation executes:

- C simulation: Folder `<project>/<solution>/csim/build`
- C/RTL co-simulation: Folder `<project>/<solution>/sim/<RTL>`

where `<project>` is the project folder, `<solution>` is the name of the solution folder and `<RTL>` is the type of RTL verified (Verilog or VHDL). The following figure shows a typical comparison of the pre-synthesis results file on the left-hand side and the post-synthesis RTL results file on the right-hand side. The output is shown in the third column.

Figure 146: **Pre-Synthesis and Post-Synthesis Simulation Differences**



The results of pre-synthesis simulation and post-synthesis simulation differ by fractional amounts. You must decide whether these fractional amounts are acceptable in the final RTL implementation.

The recommended flow for handling these differences is using a test bench that checks the results to ensure that they lie within an acceptable error range. This can be accomplished by creating two versions of the same function, one for synthesis and one as a reference version. In this example, only function `cpp_math` is synthesized.

```
#include <cmath>
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

typedef float data_t;

data_t cpp_math(data_t angle) {
 data_t s = sinf(angle);
 data_t c = cosf(angle);
```

```
 return sqrtf(s*s+c*c);
}

data_t cpp_math_sw(data_t angle) {
 data_t s = sinf(angle);
 data_t c = cosf(angle);
 return sqrtf(s*s+c*c);
}
```

The test bench to verify the design compares the outputs of both functions to determine the difference, using variable `diff` in the following example. During C simulation both functions produce identical outputs. During C/RTL co-simulation function `cpp_math` produces different results and the difference in results are checked.

```
int main() {
 data_t angle = 0.01;
 data_t output, exp_output, diff;
 int retval=0;

 for (data_t i = 0; i <= 250; i++) {
 output = cpp_math(angle);
 exp_output = cpp_math_sw(angle);

 // Check for differences
 diff = ( (exp_output > output) ? exp_output - output : output -
exp_output);
 if (diff > 0.0000005) {
 printf("Difference %.10f exceeds tolerance at angle %.10f \n", diff,
angle);
 retval=1;
 }

 angle = angle + .1;
 }

 if (retval != 0) {
 printf("Test failed  !!!\n");
 retval=1;
 } else {
 printf("Test passed !\n");
  }
 // Return 0 if the test passes
  return retval;
}
```

If the margin of difference is lowered to 0.00000005, this test bench highlights the margin of error during C/RTL co-simulation:

```
Difference 0.0000000596 at angle 1.1100001335
Difference 0.0000000596 at angle 1.2100001574
Difference 0.0000000596 at angle 1.5100002289
Difference 0.0000000596 at angle 1.6100002527
etc..
```

When using the standard C math libraries (`math.h` and `cmath.h`) create a “smart” test bench to verify any differences in accuracy are acceptable.

Verification Option 2: HLS Math Library and Validate Differences

An alternative verification option is to convert the source code to use the HLS math library. With this option, there are no differences between the C simulation and C/RTL co-simulation results. The following example shows how the code above is modified to use the `hls_math.h` library.

***Note*:** This option is only available in C++.

- Include the `hls_math.h` header file.
- Replace the math functions with the equivalent `hls::` function.

```
#include <cmath>
#include "hls_math.h"
#include <fstream>
#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

typedef float data_t;

data_t cpp_math(data_t angle) {
 data_t s = hls::sinf(angle);
 data_t c = hls::cosf(angle);
 return hls::sqrtf(s*s+c*c);
}
```

Verification Option 3: HLS Math Library File and Validate Differences

Including the HLS math library file `lib_hlsm.cpp` as a design file ensures Vitis HLS uses the HLS math library for C simulation. This option is identical to option2 however it does not require the C code to be modified.

The HLS math library file is located in the `src` directory in the Vitis HLS installation area. Simply copy the file to your local folder and add the file as a standard design file.

***Note*:** This option is only available in C++.

As with option 2, with this option there is now a difference between the C simulation results using the HLS math library file and those previously obtained without adding this file. These difference should be validated with C simulation using a “smart” test bench similar to option 1.

Common Synthesis Errors

The following are common use errors when synthesizing math functions. These are often (but not exclusively) caused by converting C functions to C++ to take advantage of synthesis for math functions.

C++ cmath.h

If the C++ `cmath.h` header file is used, the floating point functions (for example, `sinf` and `cosf`) can be used. These result in 32-bit operations in hardware. The `cmath.h` header file also overloads the standard functions (for example, `sin` and `cos`) so they can be used for float and double types.

C math.h

If the C `math.h` library is used, the single-precision functions (for example, `sinf` and `cosf`) are required to synthesize 32-bit floating point operations. All standard function calls (for example, `sin` and `cos`) result in doubles and 64-bit double-precision operations being synthesized.

Cautions

When converting C functions to C++ to take advantage of `math.h` support, be sure that the new C++ code compiles correctly before synthesizing with Vitis HLS. For example, if `sqrtf()` is used in the code with `math.h`, it requires the following code extern added to the C++ code to support it:

```
#include <math.h>
extern “C” float sqrtf(float);
```

To avoid unnecessary hardware caused by type conversion, follow the warnings on mixing double and float types discussed in Floats and Doubles.

HLS Stream Library

Streaming data is a type of data transfer in which data samples are sent in sequential order starting from the first sample. Streaming requires no address management.

Modeling designs that use streaming data can be difficult in C. The approach of using pointers to perform multiple read and/or write accesses can introduce issues, because there are implications for the type qualifier and how the test bench is constructed.

IMPORTANT! *The `hls::stream` class is only used in C++ designs.*

Vitis HLS provides a C++ template class `hls::stream<>` for modeling streaming data structures. The streams implemented with the `hls::stream<>` class have the following attributes.

- In the C code, an `hls::stream<>` behaves like a FIFO of infinite depth. There is no requirement to define the size of an `hls::stream<>`.
- They are read from and written to sequentially. That is, after data is read from an `hls::stream<>`, it cannot be read again.
- An `hls::stream<>` on the top-level interface is by default implemented with an `ap_fifo` interface for the AMD Vivado™ IP flow, or as an axis interface for the AMD Vitis™ kernel flow.
- Streams can be defined either locally or globally and are always implemented as internal FIFOs. Streams defined in the global scope follow the same rules as any other global variables.
- There are two possible stream declarations:
  - `hls::stream<Type>`: specify the data type for the stream.

    An `hls::stream<>` internal to the design is implemented as a FIFO with a default depth of 2. The STREAM pragma or directive can be used to change the depth.
  - `hls::stream<Type, Depth>` : specify the data type for the stream, and the FIFO depth.

    Set the depth to prevent stalls. If any task in the design can produce or consume samples at a greater rate than the specified depth, the FIFOs might become empty (or full) resulting in stalls, because it is unable to read (or write).

This section shows how the `hls::stream<>` class can more easily model designs with streaming data. The topics in this section provide:

- An overview of modeling with streams and the RTL implementation of streams.
- How to use streams.
- Blocking reads and writes.
- Non-Blocking Reads and writes.
- Controlling the FIFO depth.

***Note*:** The `hls::stream` class should always be passed between functions as a C++ reference argument. For example, `&my_stream`.

C Modeling and RTL Implementation

Streams are modeled as an infinite queue in software (and in the test bench during RTL co-simulation). There is no need to specify any depth to simulate streams in C++. Streams can be used inside functions and on the interface to functions. Internal streams may be passed as function parameters.

Streams can be used only in C++ based designs. Each `hls::stream<>` object must be written by a single process and read by a single process.

If an `hls::stream` is used on the top-level interface in the Vivado IP flow it is implemented in the RTL as a FIFO interface (`ap_fifo`) by default, but can be optionally implemented as an AXI4-Stream interface (`axis`). In the Vitis kernel flow it is by default implemented as an AXI4-Stream interface (`axis`).

If an `hls::stream` is used inside the design function and synthesized into hardware, it is implemented as a FIFO with a default depth of 2. In some cases, such as when interpolation is used, the depth of the FIFO might have to be increased to ensure the FIFO can hold all the elements produced by the hardware. Failure to ensure the FIFO is large enough to hold all the data samples generated by the hardware can result in a stall in the design (seen in C/RTL co-simulation and in the hardware implementation). The depth of the FIFO can be adjusted using the STREAM directive with the `depth` option.

IMPORTANT! *Ensure `hls::stream` variables are correctly sized when used in the default non-DATAFLOW regions.*

If an `hls::stream` is used to transfer data between tasks (sub-functions or loops), you should immediately consider implementing the tasks in a DATAFLOW region where data streams from one task to the next. The default (non-DATAFLOW) behavior is to complete each task before starting the next task, in which case the FIFOs used to implement the `hls::stream` variables must be sized to ensure they are large enough to hold all the data samples generated by the producer task. Failure to increase the size of the `hls::stream` variables results in the error below:

```
ERROR: [XFORM 203-733] An internal stream xxxx.xxxx.V.user.V' with default
size is
used in a non-dataflow region, which may result in deadlock. Please
consider to
resize the stream using the directive 'set_directive_stream' or the 'HLS
stream'
pragma.
```

This error informs you that in a non-DATAFLOW region, the default FIFOs depth of 2 may not be large enough to hold all the data samples written to the FIFO by the producer task, and deadlock may occur.

Using HLS Streams

To use `hls::stream<>` objects in your code include the header file `hls_stream.h` as shown below. Streaming data objects are defined by specifying the type and variable name. In this example, a 128-bit unsigned integer type is defined and used to create a stream variable called `my_wide_stream`.

```
#include "ap_int.h"
#include "hls_stream.h"

typedef ap_uint<128> uint128_t;  // 128-bit user defined type
hls::stream<uint128_t> my_wide_stream;  // A stream declaration
```

Streams must use scoped naming. AMD recommends using the scoped `hls::` naming shown in the example above. However, if you want to use the `hls` namespace, you can rewrite the preceding example as:

```
#include <ap_int.h>
#include <hls_stream.h>
using namespace hls;

typedef ap_uint<128> uint128_t;  // 128-bit user defined type
stream<uint128_t> my_wide_stream;  // hls:: no longer required
```

Given a stream specified as `hls::stream<T>`, the type T can be:

- Any C++ native data type
- A Vitis HLS arbitrary precision type (for example, `ap_int<>`, `ap_ufixed<>`)

- A user-defined struct containing either of the above types

Note: General user-defined classes (or structures) that contain methods (member functions) should not be used as the type (T) for a stream variable.

A stream can also be specified as `hls::stream<Type, Depth>`, where Depth indicates the depth of the FIFO needed in the verification adapter that the HLS tool creates for RTL co-simulation.

Streams can be optionally named. Providing a name for the stream allows the name to be used in reporting. For example, Vitis HLS automatically checks to ensure all elements from an input stream are read during simulation. Given the following two streams:

```
hls::stream<uint8_t> bytestr_in1;
hls::stream<uint8_t> bytestr_in2("input_stream2");
```

Any warning on elements of the streams are reported as follows, where it is clear that `input_stream2` refers to `bytestr_in2`:

```
WARNING: Hls::stream 'hls::stream<unsigned char>.1' contains leftover data,
which
may result in RTL simulation hanging.
WARNING: Hls::stream 'input_stream2' contains leftover data, which may
result in RTL
simulation hanging.
```

When streams are passed into and out of functions, they must be passed-by-reference as in the following example:

```
void stream_function (
      hls::stream<uint8_t> &strm_out,
      hls::stream<uint8_t> &strm_in,
     uint16_t strm_len
    )
```

Streaming examples are provided in the Vitis-HLS-Introductory-Examples repository on GitHub, under Interface/Streaming. Additional design examples using streams are provided in the Vitis_Accel_Examples also on GitHub.

Vitis HLS also supports both blocking and non-blocking access methods for `hls::stream` objects, as described in the following sections.

Blocking API

The term blocking means that operation stalls until fresh data is available on the streaming channels. The blocking API automatically checks if the FIFO is full or empty and performs the read/write operation on the FIFO. There is no need to manually check for full and empty conditions of the FIFO. The blocking API is fully deterministic, which means that C-simulation and RTL/Co-simulation will behave exactly the same way. This blocking API should only be used in the scenarios described in the following sections. Other uses could lead to non-deterministic behavior.

TIP: *When used in blocking mode the `hls::stream` can deadlock due to insufficiently sized data streams.*

Blocking Write Methods

In this example, the value of variable `src_var` is pushed into the stream.

```
// Usage of void write(const T & wdata)

hls::stream<int> my_stream;
int src_var = 42;

my_stream.write(src_var);
```

The << operator is overloaded such that it can be used in a similar fashion to the stream insertion operators for C++ stream (for example, iostreams and filestreams). The `hls::stream<>` object to be written to is supplied as the left-hand side argument and the value to be written as the right-hand side.

```
// Usage of void operator << (T & wdata)

hls::stream<int> my_stream;
int src_var = 42;

my_stream << src_var;
```

Blocking Read Methods

This method reads from the head of the stream and assigns the values to the variable `dst_var`.

```
// Usage of void read(T &rdata)

hls::stream<int> my_stream;
int dst_var;

my_stream.read(dst_var);
```

Alternatively, the next object in the stream can be read by assigning (using for example =, +=) the stream to an object on the left-hand side:

```
// Usage of T read(void)

hls::stream<int> my_stream;

int dst_var = my_stream.read();
```

The >> operator is overloaded to allow use similar to the stream extraction operator for C++ stream (for example, iostreams and filestreams). The `hls::stream` is supplied as the LHS argument and the destination variable the RHS.

```
// Usage of void operator >> (T & rdata)

hls::stream<int> my_stream;
int dst_var;

my_stream >> dst_var;
```

Deterministic Behavior

As discussed in Blocking API, the blocking API can have both deterministic and non-deterministic behavior as shown in the following code example:

```
func1()
{
   while(!s.empty()) {
   s.read();
   }
}
```

During C-simulation the data to the stream is always available so the `while` loop runs through to completion. However, if the data stream has a single bubble when running in hardware the `while` loop will exit, and `func1` will return prematurely. This will lead to non-deterministic behavior between C-simulation and RTL execution.

The proper way to implement this loop is to use the side-channel signal `tlast`, as shown in the following example. Refer to AXI4-Stream Interfaces with Side-Channels for more information.

```
func1()
{
   while(!tlast) {
      s1.read()
   }
}
```

The blocking API can be deterministic when used in one of the following ways.

- **Case 1 - Simple read/write to the FIFO:**

```
int data = in.read();
  if (data >= 10)
    out1.write(data);
  else
    out2.write(data);
```

- **Case 2 - FULL and EMPTY check using blocking API:**

  This case does not perform any computation with side effects, i.e. read or write a stream, memory, or static variable is still considered to use only blocking stream accesses.

```
void df(hls::stream<...> &s1, hls::stream<...> &s2, ...) {
#pragma HLS dataflow

  p1(s1, ...);
  p2(s2, ...);
...
}
void p1(hls::stream<...> &s1, ...) {
  if (s1.empty())
    return;
  ... = s1.read();
  ...
}
void p2(hls::stream<...> &s2, ...) {
  if (s2.full())
    return;
  ...
  s2.write(...);
}
```

Non-Blocking API

IMPORTANT! *Non-blocking API is fully supported on interfaces using the `ap_fifo` protocol. However, the AXI4-Stream (`axis`) interface only supports non-blocking read accesses, but not write.*

The term non-blocking means that lack of data (or too much data) on the stream does not block the operation of a function or the iteration of a loop. Non-blocking methods return a Boolean value indicating the status of a read or write: `true` if successful, `false` otherwise. However, using non-blocking APIs can lead to non-deterministic behavior which cannot be fully validated during either C-simulation or RTL/Co-simulation, and requires an RTL test bench to test it exhaustively. Non-deterministic behavior can occur when reading from an empty FIFO or writing to a full FIFO.

During C simulation, streams have an infinite size. It is therefore not possible to validate with C simulation if the stream is full. These methods can be verified only during RTL simulation when the FIFO sizes are defined (either the default size of 1, or an arbitrary size defined with the STREAM directive).

IMPORTANT! *If the design is specified to use the block-level I/O protocol* `ap_ctrl_none` *and the design contains any* `hls::stream` *variables that employ non-blocking behavior, C/RTL co-simulation is not guaranteed to complete.*

Non-Blocking Write

The `hls::stream.write_nb()` method attempts to push data into the stream, returning a boolean `true` if successful. Otherwise, `false` is returned and the queue is unaffected.

```
// Usage of bool write_nb(const T & wdata)
hls::stream<int> my_stream;
int src_var = 42;

if (my_stream.write_nb(src_var)) {
 // Perform standard operations
 ...
} else {
 // Write did not occur
 return;
}
```

Non-blocking behavior can also be modeled using blocking writes with full checking conditions, as explained in hls::stream.full() Method. This can lead to non-deterministic behavior and should be verified in RTL simulation with a sophisticated test bench.

```
hls::stream<int> my_stream;
int src_var = 42;
bool stream_full;

stream_full = my_stream.full();
if(!stream_full)
   my_stream.write_nb(src_var);
```

Non-Blocking Read

The `hls::stream.read_nb()` method attempts to read a value from the stream, returning `true` if successful. Otherwise, `false` is returned and the queue is unaffected.

```
// Usage of bool read_nb(const T & wdata)

hls::stream<int> my_stream;
int dst_var;

if (my_stream.read_nb(dst_var)) {
 // Perform standard operations
 ...
} else {
 // Read did not occur
 return;
}
```

Non-blocking behavior can also be modeled using blocking read with a check for an empty stream, as described in hls::stream.empty() Method. This can lead to non-deterministic behavior and should be verified in RTL simulation with a sophisticated test bench.

```
READ_ONLY_LOOP:
while (check != 0) {
   if ( !addr_strm.empty() )
   {
      addr_strm.read_nb(addr_for_HBM);
      hbm[addr_for_HBM] = some_data;
      check[0] = 1;
      ...
   }
   ...
   ...
   check = (check << 1);
   }
```

hls::stream.capacity() Method

The `hls::stream.capacity()` method returns an unsigned integer which defines the maximum number of elements the stream can hold, or the total depth of the stream This lets you check how much space is available in the channel prior to writing.

```
hls::stream<int> my_stream;
int var = 16;

if ((my_stream.capacity() - my_stream.size()) >= N) {
  for (int=0; i<N;i++)
    my_stream.write(...); // this will not block
}
```

TIP: *In C-simulation `hls::stream.capacity()` returns MAXINT as a theoretical limit. In synthesis and C/RTL Co-simulation it returns the computed capacity of the stream.*

The `hls::stream.capacity()` and `hls::stream.size()` methods enable advanced flow control to write or read the stream only when there is specific amounts of space or data available for the transaction. For example, before starting a read burst of size N, you can check if there are N data items in the input stream so that the burst can complete without stalling. For write bursts you can also check if there is enough space available in an output stream to store the burst data without stalling.

hls::stream.empty() Method

The `hls::stream.empty()` method returns `true` if the stream is empty, as shown below.

```
hls::stream<int> my_stream;
int dst_var;
bool stream_empty;

stream_empty = my_stream.empty();
```

The following example shows how a combination of non-blocking accesses and full/empty tests can provide error handling functionality when the RTL FIFOs are full or empty:

```
#include "hls_stream.h"
using namespace hls;

typedef struct {
   short    data;
   bool     valid;
   bool     invert;
} input_interface;

bool invert(stream<input_interface>& in_data_1,
            stream<input_interface>& in_data_2,
            stream<short>& output
  ) {
  input_interface in;
  bool full_n;

// Read an input value or return
  if (!in_data_1.read_nb(in))
      if (!in_data_2.read_nb(in))
          return false;

// If the valid data is written, return not-full (full_n) as true
  if (in.valid) {
    if (in.invert)
      full_n = output.write_nb(~in.data);
    else
      full_n = output.write_nb(in.data);
  }
  return full_n;
}
```

hls::stream.full() Method

The `hls::stream.full()` method returns `true` if and only if the object is full, as shown below:

```
hls::stream<int> my_stream;
int src_var = 42;
bool stream_full;

stream_full = my_stream.full();
```

hls::stream.size() Method

The `hls::stream.size()` method returns an unsigned integer specifying the current number of elements the stream is holding. This lets you check if there is data or how much data is in the channel prior to reading to prevent stalls or deadlocks.

***Note*:** `hls::stream.size()` can be used to check if an input FIFO is full, and an output FIFO is empty. While `hls::stream.empty()` can only be applied to an input FIFO and `hls::stream.full()` to an output FIFO.

```
hls::stream<int> my_stream;
var = 16;

if (my_stream.size()>0) {
   my_stream.read(var);
}
```

The `hls::stream.size()` and `hls::stream.capacity()` methods enable advanced flow control to write or read the stream only when there is specific amounts of space or data available for the transaction. For example, before starting a read burst of size N, you can check if there are N data items in the input stream so that the burst can complete without stalling. For write bursts you can also check if there is enough space available in an output stream to store the burst data without stalling.

```
if (instream.size() > N)
  for(i=0, i<N; i++)
    ... = instream.read(); // will not block
```

To read only when the stream is full:

```
if (instream.size() == instream.capacity())
  ... = instream.read(); // read only if full
```



TIP: *You cannot use `hls::stream.full()` to check for this condition. The following will not work:*

```
if (instream.full())
  ... = instream.read();
```

Controlling the RTL FIFO Depth

For most designs using streaming data, the default RTL FIFO depth of 2 is sufficient. Streaming data is generally processed one sample at a time.

For multirate designs in which the implementation requires a FIFO with a depth greater than 2, you must determine (and set using the STREAM directive) the depth necessary for the RTL simulation to complete. If the FIFO depth is insufficient, RTL co-simulation stalls.

Because stream objects cannot be viewed in the GUI directives pane, the STREAM directive cannot be applied directly in that pane.

Right-click the function in which an `hls::stream<>` object is declared (or is used, or exists in the argument list) to:

- Select the STREAM directive.
- Populate the `variable` field manually with name of the stream variable.

Alternatively, you can:

- Specify the STREAM directive manually in the `directives.tcl` file, or
- Add it as a pragma in `source`.

HLS Direct I/O

Continuously running kernels are a type of hardware kernel designed to run indefinitely until a reset event occurs. Unlike traditional software acceleration kernels that are called, execute, and return.

In SW acceleration kernels parameters can only be changed at the time the kernel is invoked or when the kernel is idle. However, with continuously running kernels, you cannot modify their arguments during execution in the usual way because they are always running. To dynamically modify kernel arguments (scalars and memory offsets) for continuously running kernels, AMD Vitis™ HLS provides the Direct I/O class, offering a convenient solution for updating these arguments on the fly.

Dynamic Parameter Adjustment using Direct I/O

AMD Vitis™ HLS offers a C++ template class `hls::direct_io` specifically for modifying the kernel's scalar/ Memory offset arguments during execution.

Features of hls::direct_io

- `hls::direct_io` streams can be used only on the top-level functions
- This class supports all the port-level protocols. The class supports the following port protocols:`ap_none`, `ap_ack`, `ap_vld`, and `ap_hs`.
- Direct I/O streams use blocking and non-blocking calls for read and writes.
- Direct I/O streams are coded similar to HLS Streams. However, they are implemented as wires based on the port protocols. Note: They are not implemented/synthesized as FIFO.

Using Direct I/O Streams

To utilize Direct I/O in your High-Level Synthesis (HLS) code, you must first include the necessary header file that provides the Direct I/O functionality. Begin by adding the following line at the top of your source file.

```
#include "hls_directio.h"
```

Once the header file is included, you can define Direct I/O objects by selecting an appropriate port level protocol. These protocols determine how the Direct I/O streams will interact with other components in the system design. You can then use blocking or non-blocking calls to read from or write to these streams. The Blocking and Non-blocking APIs are explained in detail in the next section.

Example Definition and Usage

In the following example, two Direct I/O stream variables are defined, `reset_value` and `reset_my_counter`, using the `ap_vld` handshake protocol with an integer data type:

```
#include "hls_directio.h"
hls::ap_vld<int> &reset_value,
hls::ap_vld<int> &reset_myCounter
```

Given a Direct I/O stream specified as `hls::ap_vld<T>`, the type T can be:

- Any C++ native data type
- A Vitis HLS arbitrary precision type (for example, ap_int<>, ap_ufixed<>)
- A user-defined struct containing either of the above types

Direct I/O Streaming examples are provided in the Vitis-HLS-Introductory-Examples repository on GitHub.

Mapping Direct I/O Streams to a Wire Interface

Direct I/O streams offer a flexible interface with the hardware (HW) wire interface. By choosing the appropriate protocol Port-Level I/O: Wire Handshakes — ap_ack, ap_vld, ap_none, or ap_hs, you can manage data flow, validation, and acknowledgment signals effectively.

Protocol Options for Direct I/O Streams

AP_HS

Interface mode `ap_hs` includes a two-way handshake signal with the data port, adhering to the industry-standard valid and acknowledge handshake. The signals associated with the `ap_hs` protocol are:

- **Data Port:** Access data with `<var>.read()`.
- **Valid Signal:** Check data validity with `<var>.valid()`, which returns true when data is ready for reading.

- **Acknowledge Signal:** Confirm data reception with `<var>.ready()`, which returns true after the data has been read.

AP_VLD

The `ap_vld` protocol, a simpler version of `ap_hs`, provides:

- **Data port:** *"<var>.read()"*
- **Valid signal:** *"<var>.valid()"* to indicate when the data signal is valid and can be read.

Example kernel function utilizing `ap_vld`:

```
void krnl_stream_vdatamover(hls::stream<pkt> &in,
                        hls::stream<pkt> &out,
                                  int mem[DATA],
                        hls::ap_vld<int> &reset_value,
                        hls::ap_vld<int> &reset_myCounter
                        )


{for(int i=0;i<DATA;i++)
{
// ...
  if(reset_myCounter.valid())    {
      int reset = reset_myCounter.read();
    //...
}
 // ..
}
}
```

AP_ACK

The `ap_ack` port-level I/O protocol is a subset of the `ap_hs` interface type. The `ap_ack` port-level I/O protocol provides the following signals:

- **Data Port:** Access data with `<var>.read()`.
- **Acknowledge Signal:** Indicate data consumption with `<var>.ready()`.

AP_NONE

The `ap_none` port-level I/O protocol is the simplest interface type and has no other signals associated with it. Neither the input nor output data signals have associated control ports that indicate when data is read or written. The only ports in the RTL design are those specified in the source code.

Note: AP_NONE protocol does not have non-blocking APIs.

```
typedef hls::ap_none<int> mystream_in;
typedef hls::ap_none<int> mystream_out;
typedef hls::ap_none<int> stream_vld_in;
typedef hls::ap_none<int> stream_vld_out;
```

```
void addInputs(mystream_in &in, mystream_in
&dinB,stream_vld_in&vldIn,mystream_out &dout_Y,stream_vld_out & vldOut)
{
    #pragma HLS pipeline II=1
    int var = vldIn.read();
    if(var ==0)
    {
        vldOut.write(var);
        return;
    }
    int tmp1 = in.read(); // Blocking_read
    int tmp2 = dinB.read(); // Blocking read
    int out = tmp1+ tmp2;
    dout_Y.write(out);
    vldOut.write(1);
}
```

Mapping Direct I/O Streams to SAXI Lite Interface

Mapping Direct I/O variables to AXI4-Lite requires special consideration. This is achieved by using the interface attribute direct_io=true. The generated RTL will contain identical logic to the Direct I/O streams. However, this interface attribute has a simulation limitation, which is explained in the following section.

Limitations and Solutions

The direct_io=true attribute has a limitation: the testbench can only provide one variable per top-level call. This means that during csim/co-simulation, the testbench cannot read or write multiple values to the scalar per top-level call. However, in Vitis, since software emulation (sw_emu) and hardware emulation (hw_emu) work on different threads, the host can read and write multiple values to the Direct I/O variable.

In this example, the `base` variable is mapped to a Direct I/O interface using the interface attribute `direct_io=true`.

```
#include "hls_print.h"
#include "hls_stream.h"
#include "hls_directio.h"
struct pkt {int size, bucket; };
typedef int scalar_t


void pkt_proc(int *buf, scalar_t &base, hls::stream<pkt> &in,
hls::stream<pkt> &out) {

#pragma HLS interface ap_none port=base direct_io=true
#pragma HLS interface s_axilite port=base
#pragma HLS interface s_axilite port=buf
#pragma HLS interface ap_memory port=buf depth=32
```

```
  for (int i = 0; i < 16; i++) {
#pragma HLS pipeline II=1
    pkt t = in.read();
    int tmp = base;
    if (tmp && tmp + i + t.bucket < 32) { // do something only if you have
buffer, otherwise pass through
      buf[tmp + i + t.bucket] = buf[tmp + i + t.bucket] + t.size;
    }
    out.write(t);
  }
}
```

Blocking API

The term blocking means that operation stalls until fresh data is available on the Direct I/O channels. The blocking API automatically checks if the Direct I/O variable has any input data.

Blocking Writes

This method *.write()* writes the value "counter" to the Direct I/O variable "reset_value".

```
// usage of void write( const T &din)

dut_top(hls::ap_vld<int> &reset_value,..)

...
    reset_value.write(counter);

..
```

The << operator is overloaded such that it can be used in a similar fashion to the stream insertion operators for C++ stream (for example, iostreams and filestreams). The `hls::ap_vld/ap_hs/ap_none<>` object to be written to is supplied as the left-hand side argument and the value to be written as the right-hand side.

```
// usage of void operator << (_const T & din) {

dut_top(hls::ap_vld<int> &reset_value,..)


...
    reset_value << counter;

..
```

Blocking Reads

This <var>`.read()` method reads from the "reset_myCounter" and assigns the values to the variable `reset`.

```
// usage of void read(const T & dout)


void krnl_stream_vdatamover(...
                      hls::ap_hs<int> &reset_myCounter
```

```
                        ) {
...
      int reset = reset_myCounter.read();
...
}
```

The >> operator is overloaded such that it can be used in a similar fashion to the stream insertion operators for C++ stream (for example, iostreams and filestreams). The `hls::ap_vld/ap_hs/ap_none<>` object to be read is supplied as the LHS argument, and the destination variable is the RHS.

```
// Usage of void operator >> (const T& rdata) void
krnl_stream_vdatamover(..., hls::ap_hs<int> &reset_myCounter ){
...
 reset_myCounter >> reset;
...
}
```

Non-blocking API

The term non-blocking means that lack of data (or too much data) on the stream does not block the operation of a function or the iteration of a loop. Non-blocking methods return a Boolean value indicating the status of a read or write: `true` if successful, `false` otherwise.

Non-blocking Write

This method is supported for ap_hs/ap_vld which attempts to push data into the stream, returning a Boolean `true` if successful. Otherwise, `false` is returned.

```
// usage of bool write_nb(const T_& din)

dut_top(hls::ap_vld<int> &reset_value,..)

int counter =1;
reset_value.write_nb(counter);
```

The << operator is overloaded such that it can be used in a similar fashion to the stream insertion operators for C++ stream (for example, iostreams and filestreams). The `hls::ap_vld/ap_hs/ap_none<>` object to be written to is supplied as the left-hand side argument and the value to be written as the right-hand side.

```
// usage of void operator << (_const T & din) {

dut_top(hls::ap_vld<int> &reset_value,..)


...
    reset_value << counter;

..
```

Non-blocking Read

This method is supported for ap_hs/ap_vld which attempts to push data into the stream, returning a Boolean `true` if successful. Otherwise, `false` is returned.

```
// Usage of  bool read_nb(__STREAM_T__& dout)


hls::ap_vld<int> &reset_myCounterint reset =1;
reset_myCounter.read_nb(reset);
```

The >> operator is overloaded such that it can be used in a similar fashion to the stream insertion operators for C++ stream (for example, iostreams and filestreams). The `hls::ap_vld/ap_hs/ap_none<>` object to be read is supplied as the LHS argument, and the destination variable is the RHS.

```
// Usage of void operator >> (const T& rdata)


void krnl_stream_vdatamover(..., hls::ap_hs<int> &reset_myCounter ){
...
 reset_myCounter >> reset;
...
}
```

Chapter 25

HLS Fence Library

The Vitis HLS compiler can reorder unrelated accesses to improve performance. However, to ensure logic correctness (For example; preserve the program semantics), it preserves the sequential semantics of the program, for both "control dependences" and "data dependences."

When the user wants, in addition, to force some order between unrelated accesses, the Vitis HLS fence can be used to guarantee two (or more) accesses to streams, arrays, and scalar function arguments passed by reference are scheduled following the sequential order implied by the source code.

IMPORTANT! *To use fence, include the header file `hls_fence.h` into your source. An example of fence usage can be found in `Vitis-HLS-Introductory-Examples/Modeling` on GitHub.*

Full and Half Fence

The library offers two different types of fences.

Full Fence

```
hls::fence(obj_1, obj_2, ..., obj_n);
```

The full fence constrains the scheduling of the accesses to obj_1, obj_2, ..., obj_n. Any access that is, from a sequential point of view, before (resp. after) the fence will remain scheduled before (resp. after) the fence.

Half Fence

The half fence is less restrictive and is useful in pipeline loops.

```
hls::fence({obj_1, obj_2, ..., obj_n}, {obj'_1, obj'_2, ..., obj'_p});
```

The fence above constrains the access schedule as follows: objects in the first set of curly braces can be scheduled earlier than the fence. But if they are before the fence in sequential order, they cannot be scheduled after. Similarly, accesses to objects in the second set (right curly braces) can be scheduled later than the fence. But if they are after the fence in sequential order, they cannot be scheduled before.

Note: The full fence `hls::fence(a,b)` is equivalent to the half fence `hls::fence({a, b}, {a, b});`.

Practical Examples

Here are two typical scenarios where hls::fence is useful.

Avoiding Deadlocks

Suppose two communicating dataflow processes.

```
Producer(...) {
    int bound = ...;
    strm1.write(bound);
    for (int i=0; i<bound; i++) {
        strm2.write(...);
    }
}

Consumer(...) {
    int bound = strm1.read();
    for (int i=0; i<bound; i++) {
        ... = strm2.read();
    }
}
```

The second process (Consumer) needs to read the stream strm1 to access the loop bound and start iterating on reading the stream strm2. This loop bound is sent by the first process (Producer) before writing in strm2.

In that arrangement, both processes can overlap and only a small FIFO is needed to store the elements in strm2. In principle, you would not expect the compiler and/or the scheduler to move the write of strm1 after the loop. In this case, the consumer cannot start reading before all elements are written in strm2 and a stream of depth "bound" is needed to avoid a deadlock.

The use of the fence will guarantee that the write of strm1 happens before the write of strm2, as follows.

```
Producer(...) {
    int bound = ...;
    strm1.write(bound);
    hls::fence({strm1}, {strm2});
    for (int i=0; i<bound; i++) {
        strm2.write(...);
    }
}

Consumer(...) {
    int bound = strm1.read();
    for (int i=0; i<bound; i++) {
        ... = strm2.read();
    }
}
```

Configuring the Vivado LogiCORE FFT

The initialization for the configuration of the FFT and the loop to send the data can be written as follows:

```
void inputdatamover(... input_parameters, config_t *fft_config, cmpxDataIn
in[FFT_LENGTH], cmpxDataIn xn[FFT_LENGTH]) {
   config_t fft_config_tmp;
   fft_config_tmp.setDir(...);
   fft_config_tmp.setSch(...);
   *fft_config = fft_config_tmp;

   for(int i=0; i<FFT_LENGTH; i++) {
   #pragma HLS pipeline rewind
      xn[i] = in[i];
   }
}
```

For correctness it is imperative that the configuration of the FFT gets captured prior to the actual processing of the data.

As written above, there is no guarantee, and a fence might be needed:

```
void inputdatamover(... input_parameters, config_t *fft_config, cmpxDataIn
in[FFT_LENGTH], cmpxDataIn xn[FFT_LENGTH]) {
   config_t fft_config_tmp;
   fft_config_tmp.setDir(...);
   fft_config_tmp.setSch(...);
   *fft_config = fft_config_tmp;
   hls::fence({fft_config}, {xn});
   for(int i=0; i<FFT_LENGTH; i++) {
   #pragma HLS pipeline rewind style=flp
      xn[i] = in[i];
   }
}
```

Known Limitations

The fence cannot be used as a barrier involving all variables by default. All variables that need to be constrained should be explicitly arguments of the fence.

Fences can be applied in a scheduling region (pipeline or sequential) but not into a dataflow region.

Chapter 26

HLS Vector Library

IMPORTANT! *To use `hls::vector` objects in your code include the header file `hls_vector.h`. An example vector application can be found in Vitis-HLS-Introductory-Examples/Modeling on GitHub.*

Vector Data Type Usage

The vector data type is provided to easily model and synthesize SIMD-type vector operations. AMD Vitis™ HLS vector data type can be defined as follows, where `T` is a primitive or user-defined type with most of the arithmetic operations defined on it. `N` is an integer greater than zero. Once a vector type variable is declared it can be used like any other primitive type variable to perform arithmetic and logic operations.

```
#include <hls_vector.h>
hls::vector<T,N>  aVec;
```

TIP: *The best performance is achieved when both the bit-width of `T` and `N` are integer powers of 2.*

Memory Layout

For any vector type defined as `hls::vector<T,N>`, the storage is guaranteed to be contiguous of size `sizeof(T)*N` and aligned to the greatest power of 2 such that the allocated size is at least `sizeof(T)*N`. In particular, when `N` is a power of 2 and `sizeof(T)` is a power of 2, `vector<T, N>` is aligned to its total size. This matches vector implementation on most architectures.

TIP: *When `sizeof(T)*N` is an integer power of 2, the allocated size will be exactly `sizeof(T)*N`, otherwise the allocated size will be larger to make alignment possible.*

The following example shows the definition of a vector class that aligns itself as described above.

```
constexpr size_t gp2(size_t N)
{
    return (N > 0 && N % 2 == 0) ? 2 * gp2(N / 2) : 1;
}

template<typename T, size_t N> class alignas(gp2(sizeof(T) * N)) vector
{
    std::array<T, N> data;
};
```

Following are different examples of alignment:

```
hls::vector<char,8> char8Vec; // aligns on 8 Bytes boundary
hls::vector<int,8> int8Vec; // aligns on 32 byte boundary
```

Requirements and Dependencies

Vitis HLS vector types requires support for C++ 14 or later. It has the following dependencies on the standard headers:

- `<array>`
  - `std::array<T, N>`
- `<cassert>`
  - `assert`
- `<initializer_list>`
  - `std::initializer_list<T>`

Supported Operations

- Initialization:

```
hls::vector<int, 4> x; // uninitialized
hls::vector<int, 4> y = 10; // scalar initialized: all elements set to 10
hls::vector<int, 4> z = {0, 1, 2, 3}; // initializer list (must have 4
elements)
hls::vector<ap_int, 4> a; // uninitialized arbitrary precision data type
```

- Access:

  The operator[] enables access to individual elements of the vector, similar to a standard array:

```
x[i] = ...; // set the element at index i
... = x[i]; // value of the element at index i
```

- Arithmetic:

  They are defined recursively, relying on the matching operation on `T`.

Table 53: **Arithmetic Operation**

Operation	In Place	Expression	Reduction (Left Fold)
Addition	`+=`	`+`	`reduce_add`
Subtraction	`-=`	`-`	*non-associative*
Multiplication	`*=`	`*`	`reduce_mult`
Division	`/=`	`/`	*non-associative*
Remainder	`%=`	`%`	*non-associative*
Bitwise AND	`&=`	`&`	`reduce_and`

Table 53: **Arithmetic Operation** *(cont'd)*

Operation	In Place	Expression	Reduction (Left Fold)
Bitwise OR	`|=`	`|`	`reduce_or`
Bitwise XOR	`^=`	`^`	`reduce_xor`
Shift Left	`<<=`	`<<`	*non-associative*
Shift Right	`>>=`	`>>`	*non-associative*
Pre-increment	`++x`	*none*	*unary operator*
Pre-decrement	`--x`	*none*	*unary operator*
Post-increment	`x++`	*none*	*unary operator*
Post-decrement	`x--`	*none*	*unary operator*

- Comparison:

  Lexicographic order on vectors (returns bool):

Table 54: **Operation**

Operation	Expression
Less than	`<`
Less or equal	`<=`
Equal	`==`
Different	`!=`
Greater or equal	`>=`
Greater than	`>`

HLS Task Library

IMPORTANT! *To use `hls::task` objects in your code include the header file `hls_task.h`.*

The `hls::task` library provides a simple way of modeling data-driven processes, allowing static instantiation of tasks with only streaming I/O (`hls::stream` or `hls::stream_of_blocks`) to eliminate the need to check for empty stream needed to model concurrent processes in C++.

TIP: *`hls::task` also supports stable inputs for scalar values on `s_axilite` interfaces, or pointers to arrays on `m_axi` interfaces as described in Support of Scalars/Memory Interfaces in DTLP.*

The following is a simple example that can be found at simple_data_driven on GitHub:

```
void odds_and_evens(hls::stream<int> &in, hls::stream<int> &out1,
hls::stream<int> &out2) {
    hls_thread_local hls::stream<int> s1; // channel connecting t1 and
t2
    hls_thread_local hls::stream<int> s2; // channel connecting t1 and t3

    // t1 infinitely runs splitter, with input in and outputs s1 and s2
    hls_thread_local hls::task t1(splitter, in, s1, s2);
    // t2 infinitely runs function odds, with input s1 and output out1
    hls_thread_local hls::task t2(odds, s1, out1);
    // t3 infinitely runs function evens, with input s2 and output
    hls_thread_local hls::task t3(evens, s2, out2);
}
```

Notice the top-level function, `odds_and_evens` uses streaming input and output interfaces. This is a purely streaming kernel. The top-level function includes the following:

- s1 and s2 are thread-local streams (`hls_thread_local`) and are used to connect the task-channel tasks t1 and t2. These streams need to be thread-local so that they can be kept alive across top-level calls.
- t1, t2, and t3 are the thread-local `hls::task` that execute the functions (`splitter`, `odds`, and `evens` respectively). The tasks run infinitely and process data on their input streams. No synchronization is needed.

However, this type of stream-only model does have some restrictions such as:

- You cannot access non-local memory
- Non-stream data, such as scalars, arrays and pointers, can be passed in as arguments, provided these ports are declared stable via the STABLE pragma. Currently top pointers with `m_axi` interface can be passed only with the `offset=off` option

> **IMPORTANT!** *Scalars, arrays and pointers will be read at unknown intervals as there is no synchronization with `hls::task`. So the code needs to be written such that it does not depend on when these ports are read.*

- You must explicitly describe the parallelism in the design by the specification of parallel tasks
- An `hls::task` must always be instantiated in a parallel context and can not be nested in a sequential context
  - If you use `hls::task` in your top function, then the top function becomes a parallel context
  - You can instantiate an `hls::task` inside a dataflow region as it is a parallel context
  - In a sequential context, if you call a function that instantiates a task:
    - The function must be a parallel context such as a dataflow region
    - The task inputs and output streams must be produced and consumed by regular dataflow processes in the parallel context
  - Control-driven TLP can be at top, inside another control-driven, or inside a sequential region
  - Data-driven can be at top, inside another data-driven, or nested between control-driven tasks
  - Control-driven TLP cannot be inside a pipeline or DIRECTLY inside data-driven. In the latter case, it must be inside a sequential region executed by a data-driven task
  - Data-driven cannot be directly inside a sequential, pipeline or control driven
  - A sequential region, or a pipeline can only be inside the body of a control or data-driven TASK (not a control or data-driven region)

The `hls::task` objects can be mixed freely with standard dataflow-style function calls, which can move data in and out of memories. Tasks also support splitting channels (`hls::split`) to support one-to-many data distributions to build pools of workers that process streams of data, and merging channels (`hls::merge`) to support many-to-one data aggregation.

Support of Scalars/Memory Interfaces in DTLP

Control-Driven Task-Level Parallelism (CDTLP), follows standard C semantics for example: function parameters must be available before the function is called. This means that non-stream I/O parameters are provided to the kernel at the start of its execution and remain constant until the kernel completes. Synchronization points, such as the function start and end, allow for predictable updates of these parameters.

In contrast, Data-Driven Task-Level Parallelism (DTLP) designs operate similarly to RTL designs. These kernels run continuously from the moment the FPGA is programmed until a reset occurs. There are no function starts or completion points where non-stream I/O parameters can be updated in the traditional sense because the kernel is always active.

Despite the lack of synchronization points in DTLP designs, non-stream I/O can still be supported with a key consideration: the kernel designer does not require precise timing for when these non-stream inputs change. The important aspect is that the kernel can access the updated data at some point during its continuous execution.

There are two types of behaviors for non-stream I/O:

- Case (a): Can change at any time, regardless of whether the kernel is executing or idle. Kernels whose arguments are modeled using the hls::direct I/O class can change the value at any point in the kernel execution. The kernel designer is not particular about the timing of the change if the kernel can see the updated values at some point in future time.
- Case (b): The I/O never changes during the whole execution of the kernel. The inputs that only change when the kernel is idle are, by default, treated as stable by the compiler. This means the compiler assumes these inputs will remain constant during the active execution of the kernel.

The Stable Memory I/O should be marked as `stable` in the KPN context as shown in the following code example.

```
void write_process(hls::stream<int>& in,
        hls::stream<int>& out,
        int* mem)
{
#pragma HLS PIPELINE off
    int val;
    static int addr = 0;

    in.read(val);
    if (addr >= 32)
        addr = 0;
    //hls::print("writing %d\n", addr);
    mem[addr] = val;
    addr++;
    val = mem[addr-1];
    out.write(val);
}
...
...
...
...
void stable_pointer(int* mem,
        hls::stream<int>& in,
        hls::stream<int>& out)
{
#pragma HLS DATAFLOW
#pragma HLS INTERFACE mode=s_axilite port=mem offset=30
#pragma HLS INTERFACE mode=m_axi bundle=gmem depth=256
max_read_burst_length=16 \
max_widen_bitwidth=512 max_write_burst_length=16 num_read_outstanding=16 \
num_write_outstanding=16 port=mem
```

```
#pragma HLS stable variable=mem


    hls_thread_local hls::stream<int> int_fifo("int_fifo");
#pragma HLS STREAM depth=512 type=fifo variable=int_fifo
    hls_thread_local hls::stream<int> int_fifo2("int_fifo2");
#pragma HLS STREAM depth=512 type=fifo variable=int_fifo2


    hls_thread_local hls::task t1(process_23, in, int_fifo);
    hls_thread_local hls::task t2(process_11, int_fifo, int_fifo2);
    hls_thread_local hls::task t3(write_process, int_fifo2, out, mem);

}
```

For both C/RTL Co-simulation the un-synchronized access needs to be enabled for `hls::task` and M_AXI interfaces via the `cosim.enable_tasks_with_m_axi` command as described in Co-Simulation Configuration.

Tasks and Channels

The original DATAFLOW model lets you write sequential functions, and then requires the AMD Vitis™ HLS tool to identify dataflow processes (tasks) and make them parallel, analyze and manage dependencies, perform scalar propagation and optimizations such as array-to-stream. Alternatively, the use of `hls::task` objects requires you to explicitly instantiate tasks and channels, managing parallelization yourself in your algorithm design. The purpose of `hls::task` is to define a programming model that supports parallel tasks using only streaming data channels. Tasks are not controlled by function call/return, but run whenever data is present in the input streams.



TIP: *The `hls::task` library provides concurrent semantics so that the C-simulation will be consistent with the RTL. This eliminates some of the problems with the sequential dataflow model.*

The following is an example of tasks and channels. You can see that only streaming interfaces (`hls::stream` or `hls::stream_of_blocks`) are used. You can also see that the top-level function defines the tasks and stream channels using the `hls_thread_local` keyword.

```
void func1(hls::stream<int> &in, hls::stream<int> &out1, hls::stream<int>
&out2) {
  int data = in.read();
  if (data >= 10)
    out1.write(data);
  else
    out2.write(data);
}
void func2(hls::stream<int> &in, hls::stream<int> &out) {
  out.write(in.read() + 1);
}
void func3(hls::stream<int> &in, hls::stream<int> &out) {
  out.write(in.read() + 2);
}
```

```
void top-func(hls::stream<int> &in, hls::stream<int> &out1,
hls::stream<int> &out2) {
  hls_thread_local hls::stream<int> s1; // channel connecting t1 and t2
  hls_thread_local hls::stream<int> s2; // channel connecting t1 and t3

  hls_thread_local hls::task t1(func1, in, s1, s2); // t1 infinitely runs
func1, with input in and outputs s1 and s2
  hls_thread_local hls::task t2(func2, s1, out1);   // t2 infinitely runs
func2, with input s1 and output out1
  hls_thread_local hls::task t3(func3, s2, out2);   // t3 infinitely runs
func3, with input s2 and output out2
}
```

The `hls::task` objects are variables that should be declared as `hls_thread_local` to keep the variable and the underlying thread alive across multiple calls of the instantiating function (`top_func`) in the example above. The task objects implicitly manage a thread that runs a function continuously, such as `func1`, `func2`, or `func3` in the example above. The function is the task body, and has an implicit infinite loop around it.

Each `hls::task` must be passed a set of arguments that include the function name, input and output channels `hls::streams` or `hls::stream_of_blocks`. `hls::task` objects typically only read and write streaming channels `hls::stream` and `hls::stream_of_blocks`.

Both `hls::task` and the channels that connect to them must be declared as `hls_thread_local`. The channels must be declared `hls_thread_local` to keep them alive across calls of the top-level function. Non-stream data, such as scalar and array variables, must all be local to the task functions and cannot be passed as arguments, except as noted in *Stable M_AXI and S_AXILITE Accesses* below.

IMPORTANT! *Inclusion of `hls_task.h` makes `hls::stream` and `hls::stream_of_blocks` read calls blocking in C-simulation. This means that code that previously relied on reading an empty stream will now result in deadlock during simulation.*

Use of Flushing Pipelines

In general, `hls::task` designs must use flushing pipelines (`flp`) or free-running pipelines (`frp`), as described in Flushing Pipelines and Pipeline Types. Non-flushing pipelines introduce dependencies between process executions and can result in unexpected deadlocks.

Note: You can configure the default flushing behavior in `hls::tasks` using the `syn.compile.pipeline_flush_in_task` as described in Compile Options.

Nested Tasks

In the following example, there are two instances of task1 used in task2, both also instantiated as `hls::task` instances. This demonstrates that in addition to sequential functions the body of an `hls::task` can be functions containing only `hls::task` objects.

```
void task1(hls::stream<int> &in, hls::stream<int> &out) {
  hls_thread_local hls::stream<int> s1;

  hls_thread_local hls::task t1(func2, in, s1);
  hls_thread_local hls::task t2(func3, s1, out);
}
void task2(hls::stream<int> &in1, hls::stream<int> &in2, hls::stream<int>
&out1, hls::stream<int> &out2) {
  hls_thread_local hls::task tA(task1, in1, out1);
  hls_thread_local hls::task tB(task1, in2, out2);
}
```

The use of `hls_thread_local` is still required to ensure safe multiple instantiation of the intermediate network (`tA` and `tB`, both instances of `task1` in this example; and safe instances of the leaf-level processes `t1` inside `tA` and `tB`, both executing different copies of `func2`, and `t2` inside `tA` and `tB`.

Simulation and Co-simulation

C-simulation behavior for tasks and channels model will be the same as in C/RTL Co-simulation. Reading from an empty stream was previously allowed with only a warning informing that this condition can cause hangs during simulation. In Vitis HLS 2022.2 reading from an empty stream can cause deadlock even in C-simulation and therefore is now an error condition with the following messages:

- In designs containing `hls::task` objects:

  ```
  ERROR [HLS SIM]: deadlock detected when simulating hls::tasks.
  Execute C-simulation in debug mode in the GUI and examine the source code
  location of all the blocked hls::stream::read() calls
  ```

- In designs that do not use `hls::task`:

  ```
  ERROR [HLS SIM]: an hls::stream is read while empty, which may result in
  RTL simulation hanging. If this is not expected, execute C simulation in
  debug mode
  in the GUI and examine the source code location of the blocked
  hls::stream::read()
  call to debug. If this is expected, add
  -DHLS_STREAM_READ_EMPTY_RETURNS_GARBAGE to
  -cflags to turn this error into a warning and allow empty hls::stream
  reads to return
   the default value for the data type.
  ```



TIP: *add `-DHLS_STREAM_READ_EMPTY_RETURNS_GARBAGE` to `-cflags` to turn this error into a warning*

Tasks and Dataflow

`hls::task` also supports the definition of tasks inside of dataflow regions. The dataflow region allows the definition of processes that access external arrays mapped to M_AXI, scalar, or PIPO arguments from upper levels of the design hierarchy. This requires dataflow processes identified by the `#pragma HLS dataflow` statement, synchronized via `ap_ctrl_chain`, that read data from any non-streamed C++ data structure and output it as `hls::stream` or `hls::stream_of_blocks` channels for connection to `hls::tasks`. Tasks can then output streams that are read by other dataflow processes and written to M_AXI, scalar, or PIPO arguments. Tasks must be declared after the processes or regions that produce their input streams, and before the processes or regions that consume their output streams.

IMPORTANT! *Because `hls::task` objects cannot read or write M_AXI, scalar, or PIPO arguments dataflow processes must read or write these interfaces and write or read stream channels to `hls::tasks` as shown in the example below.*

The following example illustrates tasks and dataflow processes together. The top-level function (`top-func`) is a dataflow region that defines sequential functions `write_out()` and `read_in()`, as well as `hls::task` objects and `hls::stream` channels.

```
#include "hls_task.h"

// This is an I/O dataflow process
void write_out(int* out, int n, hls::stream<int> &s2) {
  for (int i=0; i<n; i++)
    out[i] = s2.read();
}
// This is an I/O dataflow process
void read_in(int* in, int n, hls::stream<int> &s1) {
  for (int i=0; i<n; i++)
    s1.write(in[i]);
}
// This is an hls::task body
void func1(hls::stream<int> &s1, hls::stream<int> &s3) {
  // No while(1) needed! This will be a task
  s3.write(... + s1.read());
}
// This is an hls::task body
void func2(hls::stream<int> &s3, hls::stream<int> &s2) {
  // No while(1) needed! This will be a task
  s2.write(... * s3.read());
}
// This could legally be at the top of the design hierarchy
void top-func(int *in, int *out, int n) {
#pragma HLS dataflow
  hls_thread_local hls::stream<int> sk3;
  hls_thread_local hls::stream<int> sk1;
  hls_thread_local hls::stream<int> sk2;

  read_in(in, n, sk1);                              // can access stream,
scalar or array; calling order matters
  hls_thread_local hls::task t2(func2, sk3, sk2);   // can access only
stream; instance order does not matter
```

```
  hls_thread_local hls::task t1(func1, sk1, sk3);   // can access only
stream; instance order does not matter
  write_out(out, n, sk2);                           // can access stream,
scalar or array; calling order matters
}
```

`#pragma HLS DATAFLOW` is required for the two sequential functions, but the `hls::task` objects do not require it. Internally, Vitis HLS will automatically split `top-func`, including both regular dataflow processes and `hls::task` processes into two dataflow regions:

1. One dataflow region using `ap_ctrl_chain` that contains regular dataflow processes, like `read_in()` and `write_out()`, in the order in which they appear in the C++ code, and a call to the `ap_ctrl_none` region below
2. A second dataflow region using `ap_ctrl_none` containing the task and channels.

As a result of this, you can expect to see two levels of hierarchy in the Dataflow viewer in the Vitis HLS GUI.

HLS Split/Merge Library

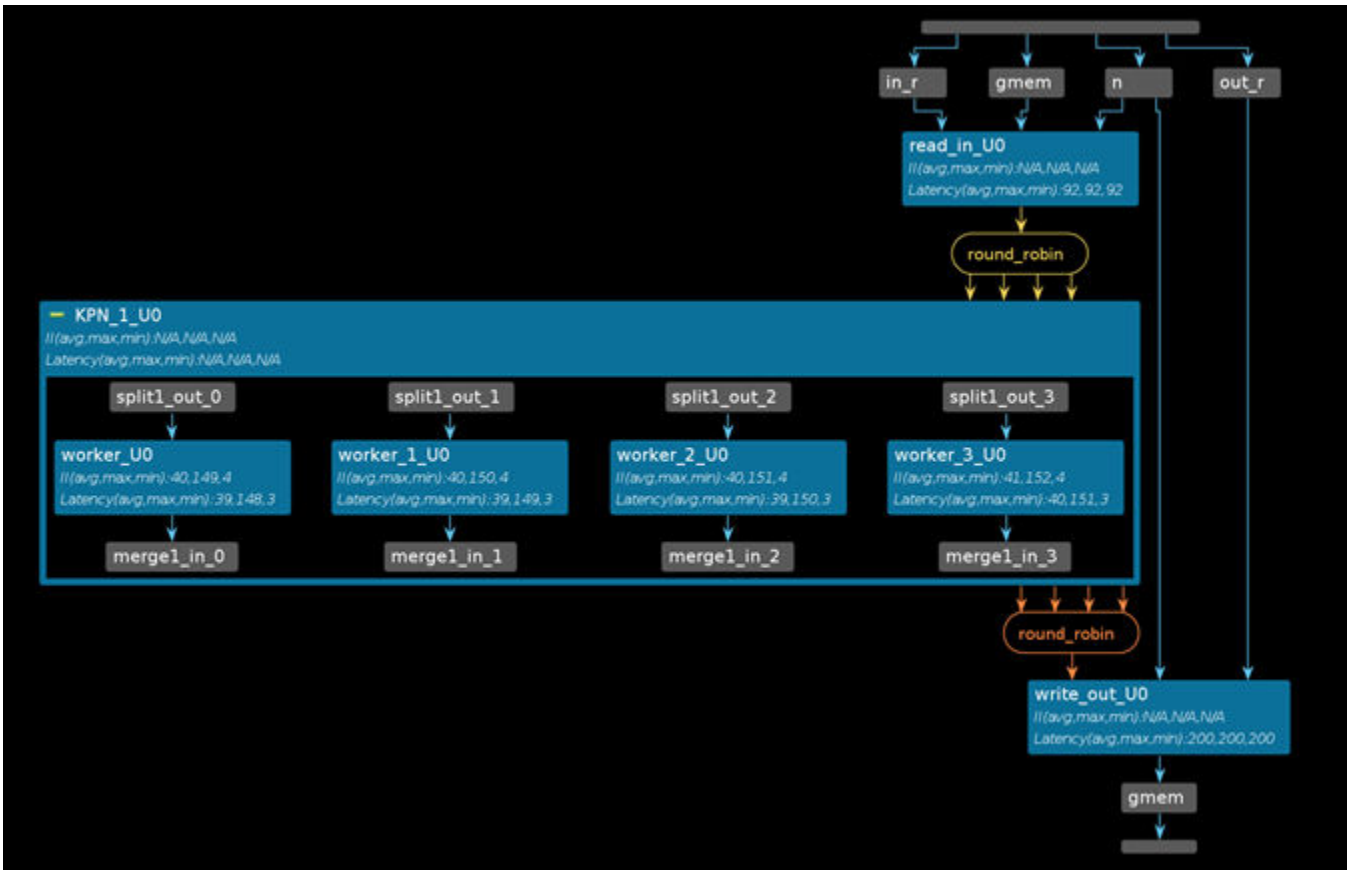
IMPORTANT! *To use `hls::split<>` or `hls::merge<>` objects in your code include the header file `hls_np_channel.h` as shown in the example below.*

For use in Dataflow processes, split/merge channels let you create one-to-many or many-to-one type channels to distribute data to multiple tasks, or aggregate data from multiple tasks. These channels have a built-in job scheduler using either a round-robin approach in which data are sequentially distributed or gathered across the channels, or a load balancing approach that is determined based on channel availability.

TIP: *Load balancing can lead to non-deterministic results in RTL/Co-simulation. In this case, you will need to write a test bench that is agnostic as to the order of results.*

As shown in the figure below, data is read from an input stream and split through the round-robin scheduler mechanism, and distributed to associated worker tasks. After a worker completes the task, it writes the output which is merged also using the round-robin scheduler, into a single stream.

Figure 147: **Split/Merge Dataflow**



A split channel has one producer and many consumers, and can be typically used to distribute tasks to a set of workers, abstracting and implementing in RTL the distribution logic, and thus leading to both better performance and fewer resources. The distribution of an input to one of the N outputs can be:

- Round-robin, where the consumers read the input data in a fixed rotating order, thus ensuring deterministic behavior, but not allowing load sharing with dynamically varying computational loads for the workers.
- Load balancing, where the first consumer to attempt a read will read the first input data, thus ensuring good load balancing, but with non-deterministic results.

A merge channel has many producers and a single consumer, and operates based on the reverse logic:

- Round-robin, where the producer output data is merged using a fixed rotating order, thus ensuring deterministic behavior, but not allowing load sharing with dynamically varying computational loads for the workers.
- The load balancing merge channel, where the first producer that completes the work will write first into the channel with non-deterministic results.

The general idea of split and merge is that with the round_robin scheduler data are distributed around to workers for the split, and read from workers for the merge, in a deterministic fashion. So if all workers compute the same function the result is the same as with a single worker, but the performance is better.

If the workers perform different functions, then your design must ensure that the correct data item is sent to the correct function in the round-robin order of workers, starting from out[0] or in[0] respectively.

Specification

Specification of split/merge channels is as follows:

```
hls::split::load_balancing<DATATYPE, NUM_PORTS[, DEPTH, N_PORT_DEPTH]>
name;
hls::split::round_robin<DATATYPE, NUM_PORTS[, DEPTH]> name
hls::merge::load_balancing<DATATYPE, NUM_PORTS[, DEPTH]> name
hls::merge::round_robin<DATATYPE, NUM_PORTS[, DEPTH]> name
```

Where:

- `round_robin`/`load_balancing`: Specifies the type of scheduler mechanism used for the channel.
- *DATATYPE*: Specifies the data type on the channel. This has the same restrictions as standard `hls::stream`. The DATATYPE can be:
  - Any C++ native data type
  - A Vitis HLS arbitrary precision type (for example, `ap_int<>`, `ap_ufixed<>`)
  - A user-defined struct containing either of the above types
- *NUM_PORTS*: Indicates the number of write ports required for split (`1:num`) or read-ports required for merge (`num:1`) operation.
- *DEPTH*: Optional argument is the depth of the main buffer, located before the split or after the merge. This is optional, and the default depth is 2 when not specified.
- *N_PORT_DEPTH*: Optional field for round-robin to specify the depth of output buffers applied after split, or before merge. This is optional and the default depth is 0 when not specified.

  TIP: *To specify the optional `N_PORT_DEPTH` value, you must also specify `DEPTH`.*

- *name*: Indicates the name of the created channel object

Following is an example which can be found at mixed_control_and_data_driven available on GitHub:

```
#include "hls_np_channel.h"

const int N = 16;
const int NP = 4;

void dut(int in[N], int out[N], int n) {
#pragma HLS dataflow
  hls::split::round_robin<int, NP> split1;
  hls::merge::round_robin<int, NP> merge1;

  read_in(in, n, split1.in);

  // Task-Channels
  hls_thread_local hls::task t[NP];
  for (int i=0; i<NP; i++) {
#pragma HLS unroll
    t[i](worker, split1.out[i], merge1.in[i]);
  }

  write_out(merge1.out, out, n);
}
```

TIP: *The example above shows the workers implemented as `hls::task` objects. However, this is simply a feature of the example and not a requirement of `split/merge` channels.*

Application of Split/Merge

The main use of split and merge is to support multiple compute engine instantiation to fully exploit the bandwidth of a DDR or HBM port. In this case, the producer is a load process that reads a burst of data from MAXI, and then passes the individual packets of data to be processed to a number of workers via the split channel. Use the round-robin protocol if the workers take similar amounts of time, or load balancing the execution time per input if variable. The consumer performs the reverse, writing data back into DRAM.

TIP: *The write back address can be passed through the split and the merge, along with the data, in the case of load balancing.*

These channels are modeled as implementing `hls::stream` objects at both ends of the split or the merge channel. This means that a split or merge channel end can be connected to any process that takes an `hls::stream` as an input or an output. The process does not need to be aware of the type of channel connection. Therefore, they can be used both for standard dataflow and for `hls::task` objects.

The following example shows how split can be used by a single produce and multiple consumers:

```
#include "hls_np_channel.h"

void producer(hls::stream<int> &s) {
  s.write(xxx);
}
```

```
void consumer1(hls::stream<int> &s) {
  ... = s.read();
}

void consumer2(hls::stream<int> &s) {
  ... = s.read();
}

void top-func() {
#pragma HLS dataflow
  hls::split::load_balancing<int, 4, 6> s; // NUM_PORTS=4, DEPTH=6

  producer(s.in, ...);
  consumer1(s.out[0], ...);
  consumer2(s.out[1], ...);
  consumer3(s.out[2], ...);
  consumer4(s.out[3], ...);
}
```

HLS Stream of Blocks Library

IMPORTANT! *To use `hls::stream_of_blocks` objects in your code include the header file `hls_streamofblocks.h`.*

The `hls::stream_of_blocks` type provides a user-synchronized stream that supports streaming blocks of data for process-level interfaces in a dataflow context, where each block is an array or multidimensional array. The intended use of stream-of-blocks is to replace array-based communication between a pair of processes within a dataflow region. Refer to the using_stream_of_blocks example on Github.

Currently, AMD Vitis™ HLS implements arrays written by a producer process and read by a consumer process in a dataflow region by mapping them to ping pong buffers (PIPOs). The buffer exchange for a PIPO buffer occurs at the return of the producer function and the calling of the consumer function in C++.

While this ensures a concurrent communication semantic that is fully compliant with the sequential C++ execution semantics, it also implies that the consumer cannot start until the producer is done, as shown in the following code example.

```
void producer (int b[M][N], ...) {
  for (int i = 0; i < M; i++)
    for (int j = 0; j < N; j++)
      b[i][f(j)] = ...;
}

void consumer(int b[M][N], ...) {
   for (int i = 0; i < M; i++)
     for (int j = 0; j < N; j++)
       ... = b[i][g(j)] ...;;
}

void top(...) {
#pragma HLS dataflow
  int b[M][N];
#pragma HLS stream off variable=b

  producer(b, ...);
  consumer(b, ...);
}
```

This can unnecessarily limit throughput and/or increase resources if the producer generates data for the consumer in smaller blocks, for example by writing one row of the buffer output inside a nested loop, and the consumer uses the data in smaller blocks by reading one row of the buffer input inside a nested loop, as the example above does. In this example, due to the non-sequential buffer column access in the inner loop, you cannot simply stream the array `b`. However, the row access in the outer loop is sequential thus supporting `hls::stream_of_blocks` communication where each block is a 1-dimensional array of size N.

The main purpose of the `hls::stream_of_blocks` feature is to provide PIPO-like functionality, but with user-managed explicit synchronization, accesses, and a better coding style. Stream-of-blocks lets you avoid the use of dataflow in a loop containing the producer and consumer, which would have been a way to optimize the example above. However, in this case, the use of the dataflow loop containing the producer and consumer requires the use of a PIPO buffer (`2xN`) as shown in the following example:

```
void producer (int b[N], ...) {
  for (int j = 0; j < N; j++)
    b[f(j)] = ...;
}

void consumer(int b[N], ...) {
   for (int j = 0; j < N; j++)
     ... = b[g(j)];
}

void top(...) {
// The loop below is very constrained in terms of how it must be written
  for (int i = 0; i < M; i++) {
#pragma HLS dataflow
    int b[N];
#pragma HLS stream off variable=b

    producer(b, ...); // writes b
    consumer(b, ...); // reads b
  }
}
```

The dataflow-in-a-loop code above is also not desirable because this structure has several limitations in Vitis HLS, such as the loop structure must be very constrained (single induction variable, starting from 0 and compared with a constant or a function argument and incremented by 1).

Stream-of-Blocks Modeling Style

On the other hand, for a stream-of-blocks the communication between the producer and the consumer is modeled as a stream of array-like objects, providing several advantages over array transfer through PIPO.

The use of stream-of-blocks in your code requires the following include file:

```
#include "hls_streamofblocks.h"
```

The stream-of-blocks object template is: `hls::stream_of_blocks<block_type, depth> v`

Where:

- `<block_type>` specifies the datatype of the array or multidimensional array held by the stream-of-blocks
- `<depth>` is an optional argument that provides depth control just like `hls::stream` or PIPOs, and specifies the total number of blocks, including the one acquired by the producer and the one acquired by the consumer at any given time. The default value is 2
- `v` specifies the variable name for the stream-of-blocks object

Use the following steps to access a block in a stream of blocks:

1. The producer or consumer process that wants to access the stream first needs to acquire access to it, using a `hls::write_lock` or `hls::read_lock` object.
2. After the producer has acquired the lock it can start writing (or reading) the acquired block. Once the block has been fully initialized, it can be released by the producer, when the `write_lock` object goes out of scope.

   Note: The producer process with a `write_lock` can also read the block as long as it only reads from already written locations, because the newly acquired buffer must be assumed to contain uninitialized data. The ability to write and read the block is unique to the producer process, and is not supported for the consumer.
3. Then the block is queued in the stream-of-blocks in a FIFO fashion, and when the consumer acquires a `read_lock` object, the block can be read by the consumer process.

The main difference between `hls::stream_of_blocks` and the PIPO mechanism seen in the prior examples is that the block becomes available to the consumer as soon as the `write_lock` goes out of scope, rather than only at the return of the producer process. Hence the size of storage required to manage the original example (without the dataflow loop) is much less with stream-of-blocks than with just PIPOs: namely 2N instead of 2xMxN in the example.

Rewriting the prior example to use `hls::stream_of_blocks` is shown in the example below. The producer acquires the block by constructing an `hls::write_lock` object called `b`, and passing it the reference to the stream-of-blocks object, called `s`. The `write_lock` object provides an overloaded array access operator, letting it be accessed as an array to access underlying storage in random order as shown in the example below.

The acquisition of the lock is performed by constructing the `write_lock`/`read_lock` object, and the release occurs automatically when that object is destructed as it goes out of scope. This approach uses the common *Resource Acquisition Is Initialization* (RAII) style of locking and unlocking.

```
#include "hls_streamofblocks.h"
typedef int buf[N];
void producer (hls::stream_of_blocks<buf> &s, ...) {
  for (int i = 0; i < M; i++) {
    // Allocation of hls::write_lock acquires the block for the producer
    hls::write_lock<buf> b(s);
    for (int j = 0; j < N; j++)
      b[f(j)] = ...;
    // Deallocation of hls::write_lock releases the block for the consumer
  }
}

void consumer(hls::stream_of_blocks<buf> &s, ...) {
  for (int i = 0; i < M; i++) {
    // Allocation of hls::read_lock acquires the block for the consumer
    hls::read_lock<buf> b(s);
    for (int j = 0; j < N; j++)
       ... = b[g(j)] ...;
    // Deallocation of hls::write_lock releases the block to be reused by
the producer
  }
}

void top(...) {
#pragma HLS dataflow
  hls::stream_of_blocks<buf> s;

  producer(b, ...);
  consumer(b, ...);
}
```

The key features of this approach include:

- The expected performance of the outer loop in the producer above is to achieve an overall Initiation Interval (II) of 1
- A locked block can be used as though it were private to the producer or the consumer process until it is released.
- The initial state of the array object for the producer is undefined, whereas it contains the values written by the producer for the consumer.
- The principal advantage of stream-of-blocks is to provide overlapped execution of multiple iterations of the consumer and the producer to increase throughput.

Resource Usage

The resource cost when increasing the depth beyond the default value of 2 is similar to the resource cost of PIPOs. Namely, each increment of 1 will require enough memory for a block, for example N * 32-bit words.

The stream of blocks object can be bound to a specific RAM type, by placing the `BIND_STORAGE` pragma where the stream-of-blocks is declared, for example in the top-level function. The stream of blocks uses 2-port block RAM (`type=RAM_2P`) by default.

Checking for Full and Empty Blocks

The `read_lock` and `write_lock` are like `while(empty) or while (full)` loops - they keep trying to acquire the resource until they get the resource - so the code execution will stall until the lock is acquired. You can use the `empty()` and `full()` methods as shown in the following example to determine if a call to `read_lock` or `write_lock` will stall due to the lack of available blocks to be acquired.

```
#include "hls_streamofblocks.h"

void reader(hls::stream_of_blocks<buf> &in1, hls::stream_of_blocks<buf>
&in2, int out[M][N], int c) {
    for(unsigned j = 0; j < M;) {
        if (!in1.empty()) {
            hls::read_lock<ppbuf> arr1(in1);
            for(unsigned i = 0; i < N; ++i) {
                out[j][i] = arr1[N-1-i];
            }
            j++;
        } else if (!in2.empty()) {
            hls::read_lock<ppbuf> arr2(in2);
            for(unsigned i = 0; i < N; ++i) {
                out[j][i] = arr2[N-1-i];
            }
            j++;
        }
    }
}

void writer(hls::stream_of_blocks<buf> &out1, hls::stream_of_blocks<buf>
&out2, int in[M][N], int d) {
    for(unsigned j = 0; j < M; ++j) {
        if (d < 2) {
            if (!out1.full()) {
                hls::write_lock<ppbuf> arr(out1);
                for(unsigned i = 0; i < N; ++i) {
                    arr[N-1-i] = in[j][i];
                }
            }
        } else {
           if (!out2.full()) {
               hls::write_lock<ppbuf> arr(out2);
               for(unsigned i = 0; i < N; ++i) {
                   arr[N-1-i] = in[j][i];
               }
           }
        }
    }
}
```

```
void top(int in[M][N], int out[M][N], int c, int d) {
#pragma HLS dataflow
    hls::stream_of_blocks<buf, 3> strm1, strm2; // Depth=3
    writer(strm1, strm2, in, d);
    reader(strm1, strm2, out, c);
}
```

The producer and the consumer processes can perform the following actions within any scope in their body. As shown in the various examples, the scope will typically be a loop, but this is not required. Other scopes such as conditionals are also supported. Supported actions include:

- Acquire a block, i.e. an array of any supported data type.
  - In the case of the producer, the array will be empty, i.e. initialized according to the constructor (if any) of the underlying data type.
  - In the case of the consumer, the array will be full (of course in as much as the producer has filled it; the same requirements as for PIPO buffers, namely full writing if needed apply).
- Use the block for both reading and writing as if it were private local memory, up to its maximum allocated number of ports based on a `BIND_STORAGE` pragma or directive specified for the stream of blocks, which specifies what ports each side can see:
  - 1 port means that each side can access only one port, and the final stream-of-blocks can use a single dual-port memory for implementation.
  - 2 ports means that each side can use 1 or 2 ports depending on the schedule:
    - If the scheduler uses 2 ports on at least one side, merging will not happen
    - If the scheduler uses 1 port, merging can happen
  - If the pragma is not specified, the scheduler will decide, based on the same criteria currently used for local arrays. Moreover:
    - The producer can both write and read the block it has acquired
    - The consumer can only read the block it has acquired
- Automatically release the block when exiting the scope in which it was acquired. A released block:
  - If released by the producer, can be acquired by the consumer.
  - If released by the consumer, can be acquired to be reused by the producer, after being re-initialized by the constructor, if any. This initialization may slow down the design, hence often it is not desired. You may use the __no_ctor__ attribute (explained earlier for std::complex) to prevent calling the constructor for the array elements.

A stream-of-blocks is very similar in spirit to a PIPO buffer. In the case of a PIPO, acquire is the same as calling the producer or consumer process function, while the release is the same as returning from it. This means that:

- the handshakes for a PIPO are
  - ap_start/ap_ready on the consumer side and

    - ap_done/ap_continue on the producer side.
- the handshakes of a stream of blocks are
    - its own read/empty_n on the consumer side and
    - write/full_n on the producer side.

Modeling Feedback in Dataflow Regions

One main limitation of PIPO buffers is that they can flow only forward with respect to the function call sequence in C++. In other words, the following connection is not supported with PIPOs, while it can be supported with `hls::stream_of_blocks`:

```
void top(...) {
  int b[N];
  for (int i = 0; i < M; i++) {
#pragma HLS dataflow
#pragma HLS stream off variable=b
    consumer(b, ...); // reads b
    producer(b, ...); // writes b
  }
}
```

The following code example is contrived to demonstrate the concept:

```
#include "hls_streamofblocks.h"
typedef int buf[N];
void producer (hls::stream_of_blocks<buf> &out, ...) {
  for (int i = 0; i < M; i++) {
    hls::write_lock<buf> arr(out);
    for (int j = 0; j < N; j++)
      arr[f(j)] = ...;
  }
}

void consumer(hls::stream_of_blocks<buf> &in, ...) {
  if (in.empty()) // execute only when producer has already generated some
meaningful data
    return;

  for (int i = 0; i < M; i++) {
    hls::read_lock<buf> arr(in);
    for (int j = 0; j < N; j++)
      ... = arr[g(j)];
    ...
  }
}

void top(...) {
  // Note the large non-default depth.
  // The producer must complete execution before the consumer can start
again, due to ap_ctrl_chain.
  // A smaller depth would require ap_ctrl_none
  hls::stream_of_blocks<buf, M+2> backward;
```

```
  for (int i = 0; i < M; i++) {
#pragma HLS dataflow
    consumer(backward, ...); // reads backward
    producer(backward, ...); // writes backward
}
```

Limitations

There are some limitations with the use of `hls::stream_of_blocks` that you should be aware of:

- Each `hls::stream_of_blocks` object must have a single producer and consumer process, and each process must be different. In other words, local streams-of-blocks within a single process are not supported.
- You cannot use `hls::stream_of_blocks` within a sequential region. The producer and consumer must be separate concurrent processes in a dataflow region.
- You cannot use multiple nested acquire/release statements (`write_lock`/`read_lock`), for example in the same or nested scopes, as shown in the following example:

```
using ppbuf = int[N];
void readerImplicitNested(hls::stream_of_blocks<ppbuf>& in, ...) {
    for(unsigned j = 0; j < M; ++j) {
       hls::read_lock<ppbuf> arrA(in); // constructor would acquire A
first
       hls::read_lock<ppbuf> arrB(in); // constructor would acquire B
second
       for(unsigned i = 0; i < N; ++i)
           ... = arrA[f(i)] + arrB[g(i)];
       // destructor would release B first
       // destructor would release A second
    }
}
```

  However, you can use multiple sequential or mutually exclusive acquire/release statements (`write_lock`/`read_lock`), for example inside IF/ELSE branches or in two subsequent code blocks. This is shown in the following example:

```
void readerImplicitNested(hls::stream_of_blocks<ppbuf>& in, ...) {
 for(unsigned j = 0; j < M; ++j) {
  {
    hls::read_lock<ppbuf> arrA(in); // constructor acquires A
    for(unsigned i = 0; i < N; ++i)
      ... = arrA[f(i)];
    // destructor releases A
  }
  {
    hls::read_lock<ppbuf> arrB(in); // constructor acquires B
    for(unsigned i = 0; i < N; ++i)
```

```
      ... = arrB[g(i)];
    // destructor releases B
  }
 }
}
```

- Explicit release of locks in producer and consumer processes are not recommended, as they are automatically released when they go out of scope. However, you can use these by adding `#define EXPLICIT_ACQUIRE_RELEASE` before `#include "hls_streamofblocks.h` in your source code.

HLS IP Libraries

AMD Vitis™ HLS provides C++ libraries to implement a number of AMD IP blocks. The C libraries allow the following AMD IP blocks to be directly inferred from the C++ source code ensuring a high-quality implementation in the FPGA.

Table 55: **HLS IP Libraries**

Library Header File	Description
hls_dsp_builtins.h	Provides a set of C++ intrinsic functions and classes to simulate and directly implement the integer functions of the AMD DSP slices primitives.
hls_fft.h	Allows the AMD LogiCORE IP FFT to be simulated in C and implemented using the AMD LogiCORE block.
hls_fir.h	Allows the AMD LogiCORE IP FIR to be simulated in C and implemented using the AMD LogiCORE block.
hls_dds.h	Allows the AMD LogiCORE IP DDS to be simulated in C and implemented using the AMD LogiCORE block.
ap_shift_reg.h	Provides a C++ class to implement a shift register which is implemented directly using an AMD SRL primitive.

DSP Intrinsics

Integer arithmetic is automatically mapped to DSP resources by the VHLS compiler, however, an experienced designer, familiar with the internal architecture of the DSP48E1, DSP48E2 or DSP58 units, can now use DSP intrinsic functions to create DSP instances with:

- A user-specified opcode via an intrinsic function name, e.g. add_mul_add or mul_add (note those are contained in specific namespaces detailed later).
- A given set of registers via a template argument of the intrinsic that encodes the DSP register names as a set of values OR-ed together to get multiple registers.
- A given use of DSP cascading ports, if allowed by the underlying placement tool.

By including `hls_dsp_builtins.h` and using the functions and classes described below.

There is currently no support for simple add and mul operations mapped to a DSP, because regular automated DSP inferencing and latency specification, via bind_op, works well in those cases.

Using DSP Intrinsics

DSP Intrinsics can implement a variety of operators, register configurations and cascading connections using intrinsic functions. When either cascades or accumulators are used, those corresponding opcodes are implemented using classes. Those are discussed below, and lists of supported opcodes and their functions or classes are given.

The template argument defines which registers are used inside the DSP.

These functions can be called in user code, with appropriate datatype conversions to and from user code variables, following the C++ and ap_int conversion rules.

For example, a DSP:

- Computing the add-mul-add function (P = (A + D) * B + C).
- With latency 2, due to the use of the A1 (A1 is one of the two registers on the A input of the DSP) and P registers (as specified using REG_A1 | REG_P as the template argument).
- Using the DSP58 bitwidths (as specified by using namespace hls::dsp58;), can be instantiated as:

```
#include "hls_dsp_builtins.h"
using namespace hls::dsp58;
...
long test(long d, long a, long b, long c)
{
    return add_mul_add<REG_A1 | REG_P>(d, a, b, c);
}
```

***Note*:** That an attempt to use the design above for a pre-Versal platform would result in an error since it does not support a 58-bit DSP.

mul_add	P = A * B + C	template<int64_t flags> C_t mul_add(A_t a, B_t b, C_t c);
mul_sub	P = A * B - C	template<int64_t flags> C_t mul_sub(A_t a, B_t b, C_t c);
mul_rev_sub	P = C - A * B	template<int64_t flags> C_t mul_rev_sub(A_t a, B_t b, C_t c);
add_mul_add	P = (A + D) * B + C	template<int64_t flags> C_t add_mul_add(D_t d, A_t a, B_t b, C_t c);
add_mul_sub	P = (A + D) * B - C	template<int64_t flags> C_t add_mul_sub(D_t d, A_t a, B_t b, C_t c);
add_mul_rev_sub	P = C - (A + D) * B	template<int64_t flags> C_t add_mul_rev_sub(D_t d, A_t a, B_t b, C_t c);
sub_mul_add	P = (A - D) * B + C	template<int64_t flags> C_t sub_mul_add(D_t d, A_t a, B_t b, C_t c);
sub_mul_sub	P = (A - D) * B - C	template<int64_t flags> C_t sub_mul_sub(D_t d, A_t a, B_t b, C_t c);
sub_mul_rev_sub	P = C - (A + D) * B	template<int64_t flags> C_t sub_mul_rev_sub(D_t d, A_t a, B_t b, C_t c);

This is the list of registers that can be used to configure each intrinsic template to use the corresponding register in the RTL model:

- REG_A1, REG_A2 on input A
- REG_B1, REG_B2 on input B

- REG_C
- REG_D on input D
- REG_AD at the output of the pre-adder
- REG_M at the output of the multiplier
- REG_P on the output of the post-adder (also used as accumulator)

The datatypes used for the arguments of the functions above differ depending on the platform. The different typedef defined in the header file, depending on the platform, are listed below for easy reference.

The specific version for a desired DSP variant can be specified in the C++ source after #includie "hls_dsp_builtins.h" by writing: using hls::dsp48e1; using hls::dsp48e2; using hls::dsp58; respectively.

Datatype	DSP48E1	DSP48E2	DSP58
A_t	ap_int<25>	ap_int<27>	ap_int<27>
B_t	ap_int<18>	ap_int<18>	ap_int<24>
C_t	ap_int<48>	ap_int<48>	ap_int<58>
D_t	ap_int<25>	ap_int<27>	ap_int<27>

Unsigned Inputs

The DSP48 and DSP58 both contain **signed multipliers**. This implies that when using DSP intrinsics to multiply unsigned values, one **must explicitly create them by setting the MSB to 0**, otherwise an unsigned input with the MSB set to 1 would be interpreted as a negative number. The easiest mechanism is to explicitly use unsigned types with one less bit.

For Example:

```
#include "hls_dsp_builtins.h"
using namespace hls::dsp58;
...
ap_uint<57> test(ap_uint<26> d, ap_uint<26> a, ap_uint<23> b, ap_uint<57>
c) {
    return add_mul_add<...>((ap_uint<27>(d), ap_uint<27>(a),
ap_uint<24>(b), ap_uint<58>(c));
}
```

Accumulation

The accumulation functionality, on REG_P, is supported via an `hls::dsp(48e1|48e2|58)::acc<>` class.

- That can be instantiated to infer different DSP accumulator instances (just one in the example below).

- Has two methods mul_acc (P = A * B + P) and add_mul_acc (P = (A + D) * B + P).
  - Performs an accumulation and returns its result (i.e. register P).
  - If the initialization condition is true (in the case below when iteration variable is 0), resets register P to 0.

mul_acc	P += A * B	template <int64_t flags> C_t mul_acc(A_t a, B_t b, bool init);
add_mul_acc	P += (A + D) * B	template <int64_t flags> C_t mul_acc(D_t D, A_t a, B_t b, bool init);

This is a simple example computing a FIR filter, using the namespace `hls::dsp48e2` to expose the `acc<>` class and the REG_P and REG_M symbols:

```
#include "hls_dsp_builtins.h"
using namespace hls::dsp48e2;
...
long test_mul_acc(long delay[N], long coeff[N])
{
    acc<REG_P|REG_M> my_acc;
    C_t res;
    for (int i=0; i<N; i++) {
       res = my_acc.mul_acc(delay[i], coeff[i], /* init condition */ i==0);
    }
    return res;
}
```

DSP Cascading

It is also possible to generate an RTL that Vivado **can** use (if allowed by the placement) to cascade DSPs via the direct ACIN, BCIN and PCIN inputs and ACOUT, BCOUT and PCOUT outputs

This can be achieved by instantiating the `hls::dsp(48e1|48e2|58)::cascade` class.

The class has a public method called mul_add (more methods will be added in the future) to perform the required operation with the cascaded inputs from the previous stage and return the cascaded outputs.

Return Struct

```
struct R_t {
  A_t acout;
  B_t bcout;
  C_t pcout;
};
R_t mul_add(A_t a, B_t b, C_t c);
```

The result of `mul_add` is a struct that allows the designer to chain the DSP for example, from `BCOUT` output port to `BCIN` input port as shown in this example, which models a templated FIR with `size` taps and using the systolic cascaded architecture.

```
#include "hls_dsp_builtins.h"
using namespace hls::dsp48e2;
...
template <int size>
class FIR {
     private:
        const long* coeff_;
        long bias_;
        cascade<REG_A1 | REG_B1 | REG_M | REG_P > dsp0;
        cascade<REG_A1 | REG_B1 | REG_B2 | REG_M | REG_P > dspN[size - 1];

     public:
        FIR(const long* coeff, long bias)
            : coeff_(coeff), bias_(bias) {
#pragma HLS ARRAY_PARTITION variable=dspN complete dim=1
            };

        long fir(B_t input){

            auto out = dsp0.mul_add(coeff_[0], input, bias_);
            for(int j=1; j < size ; j++){
#pragma HLS unroll
                out = dspN[j - 1].mul_add(coeff_[j], out.bcout, out.pcout);
            }
            return out.pcout;
        };
};

void test_cascaded_systolic_fir(long b[N], long c, long hw[N])
{
    long a[4] = {1, 12, 3, 4}; // coefficients
#pragma HLS ARRAY_PARTITION variable=a complete dim=1
#pragma HLS ARRAY_PARTITION variable=hw complete dim=1
    FIR<4> my_fir(a, c);

    LOOP_FIR:
    for(int i = 0 ; i< N ; i++){
        hw[i] = my_fir.fir(b[i]);
    }
}
```

Note the following details:

- Two different configurations of the cascaded DSPs are needed for the first stage of the cascade and the rest of the stages.
- Passing out.bcout and out.pcout as the b and c inputs of each mul_add call allows Vivado to use the bcin and pcin inputs of the next FIR stage.
- The array of dsp(48e1|48e2|58)::cascade objects must be fully partitioned to create `size` independent DSP instances by using the array_partition pragma.
- Vitis HLS generates synthesizable RTL from the code above, which means that:

    - Cascading is selected by Vivado only at placement time, if the corresponding DSP units can be placed close to each other.
    - Users should always check the Vivado report to see if cascading happened and use Vivado to determine why it could not happen (For example, insufficient neighboring DSP resources).
- For correct cascading in Vivado, the following registers must always be used.
    - REG_A1 and/or REG_A2
    - REG_B1 and/or REG_B2
    - REG_P
- For the mul_add and mul_sub to be used.
    - REG_AD and REG_D cannot be used (this restriction can be lifted in future releases).
    - REG_C cannot be used in the cascade version (this restriction can be lifted in future releases).

FFT IP Library

The AMD FFT IP block can be called within a C++ design using the library `hls_fft.h`. This section explains how the FFT can be configured in your C++ code. An example FFT application can be found in Vitis-HLS-Introductory-Examples/Misc on GitHub.



RECOMMENDED: *AMD highly recommends that you review the Fast Fourier Transform LogiCORE IP Product Guide (PG109) for information on how to implement and use the features of the IP.*

To use the FFT in your C++ code:

1. Include the `hls_fft.h` library in the code
2. Set the default parameters using the predefined struct `hls::ip_fft::params_t`
3. Define the runtime configuration
4. Call the FFT function



TIP: *Because it uses the DATAFLOW pragma or directive, the pipelined execution of an FFT must be in a dataflow loop and not a pipelined loop.*

5. Optionally, check the runtime status

The following code examples provide a summary of how each of these steps is performed. Each step is discussed in more detail below.

First, include the FFT library in the source code. This header file resides in the include directory in the Vitis HLS installation area which is automatically searched when Vitis HLS executes.

```
#include "hls_fft.h"
```

Define the static parameters of the FFT. This includes such things as input width, number of channels, type of architecture. which do not change dynamically. The FFT library includes a parameterization struct `hls::ip_fft::params_t`, which can be used to initialize all static parameters with default values.

In this example, the default values for output ordering and the widths of the configuration and status ports are over-ridden using a user-defined struct `param1` based on the predefined struct.

```
struct param1 : hls::ip_fft::params_t {
    static const unsigned ordering_opt = hls::ip_fft::natural_order;
    static const unsigned config_width = FFT_CONFIG_WIDTH;
    static const unsigned status_width = FFT_STATUS_WIDTH;
};
```

Define types and variables for both the runtime configuration and runtime status. These values can be dynamic and are therefore defined as variables in the C code which can change and are accessed through APIs.

```
typedef hls::ip_fft::config_t<param1> config_t;
typedef hls::ip_fft::status_t<param1> status_t;
config_t fft_config1;
status_t fft_status1;
```

Next, set the runtime configuration. This example sets the direction of the FFT (Forward or Inverse) based on the value of variable “direction” and also set the value of the scaling schedule.

```
fft_config1.setDir(direction);
fft_config1.setSch(0x2AB);
```

Call the FFT function using the HLS namespace with the defined static configuration (`param1` in this example). The function parameters are, in order, input data, output data, output status and input configuration.

```
hls::fft<param1> (xn1, xk1, &fft_status1, &fft_config1);
```

Finally, check the output status. This example checks the overflow flag and stores the results in variable “ovflo”

```
    *ovflo = fft_status1->getOvflo();
```

TIP: *The example above shows the use of scalar values and arrays, but the FFT function also supports the use of `hls::stream` for arguments. Refer to Using FFT Function with Streaming Interface for more information.*

FFT Static Parameters

The static parameters of the FFT define how the FFT is configured and specifies the fixed parameters such as the size of the FFT, whether the size can be changed dynamically, whether the implementation is pipelined or `radix_4_burst_io`.

The `hls_fft.h` header file defines a struct `hls::ip_fft::params_t` which can be used to set default values for the static parameters. If the default values are to be used, the parameterization struct can be used directly with the FFT function.

```
hls::fft<hls::ip_fft::params_t >
    (xn1, xk1, &fft_status1, &fft_config1);
```

A more typical use is to change some of the parameters to non-default values. This is performed by creating a new user-defined parameterization struct based on the default parameterization struct and changing some of the default values.

In the following example, a new user struct `my_fft_config` is defined with a new value for the output ordering (changed to `natural_order`). All other static parameters to the FFT use the default values.

```
struct my_fft_config : hls::ip_fft::params_t {
    static const unsigned ordering_opt = hls::ip_fft::natural_order;
};

hls::fft<my_fft_config >
     (xn1, xk1, &fft_status1, &fft_config1);
```

The parameters used for the FFT struct `hls::ip_fft::params_t` are explained below. The default values for the parameters and a list of possible values are provided.



TIP: *The FFT LogiCORE IP uses the `data_format` parameter to specify the values as `fixed_point` or `floating_point`, with the default being `fixed_point`. However, with the `hls_fft.h` library the input variables passed to the FFT function determine the `data_format` rather than a parameter.*

Table 58: **FFT Struct Parameters**

Parameter	Description	C Type	Default Value	Valid Values
input_width	Data input port width.	unsigned	16	8-34
output_width	Data output port width.	unsigned	16	input_width to (input_width + max_nfft + 1)
status_width	Output status port width.	unsigned	8	Depends on FFT configuration
config_width	Input configuration port width.	unsigned	16	Depends on FFT configuration
max_nfft	The size of the FFT data set is specified as 1 << max_nfft.	unsigned	10	3-16

Table 58: **FFT Struct Parameters** *(cont'd)*

Parameter	Description	C Type	Default Value	Valid Values
has_nfft	Determines if the size of the FFT can be runtime configurable.	bool	false	True, False
channels	Number of channels.	unsigned	1	1-12
arch_opt	The implementation architecture.	unsigned	pipelined_streaming_io	automatically_select pipelined_streaming_io radix_4_burst_io radix_2_burst_io radix_2_lite_burst_io
phase_factor_width	Configure the internal phase factor precision.	unsigned	16	8-34
ordering_opt	The output ordering mode.	unsigned	bit_reversed_order	bit_reversed_order natural_order
ovflo	Enable overflow mode.	bool	true	true false
scaling_opt	Define the scaling options.	unsigned	scaled	scaled unscaled block_floating_point
rounding_opt	Define the rounding modes.	unsigned	truncation	truncation convergent_rounding
mem_data	Specify using block or distributed RAM for data memory.	unsigned	block_ram	block_ram distributed_ram
mem_phase_factors	Specify using block or distributed RAM for phase factors memory.	unsigned	block_ram	block_ram distributed_ram
mem_reorder	Specify using block or distributed RAM for output reorder memory.	unsigned	block_ram	block_ram distributed_ram
stages_block_ram	Defines the number of block RAM stages used in the implementation.	unsigned	(max_nfft < 10) ? 0 : (max_nfft - 9)	0-11
mem_hybrid	When block RAMs are specified for data, phase factor, or reorder buffer, mem_hybrid specifies where or not to use a hybrid of block and distributed RAMs to reduce block RAM count in certain configurations.	bool	false	true false
complex_mult_type	Defines the types of multiplier to use for complex multiplications.	unsigned	use_mults_resources	use_luts use_mults_resources use_mults_performance
butterfly_type	Defines the implementation used for the FFT butterfly.	unsigned	use_luts	use_luts use_xtremedsp_slices

IMPORTANT! *When specifying parameter values which are not integer or boolean, the HLS FFT namespace should be used. For example, the possible values for parameter* `butterfly_type` *in the following table are* `use_luts` *and* `use_xtremedsp_slices`*. The values used in the C program should be* `butterfly_type = hls::ip_fft::use_luts` *and* `butterfly_type = hls::ip_fft::use_xtremedsp_slices`*.*

FFT Runtime Configuration and Status

The FFT supports runtime configuration and runtime status monitoring through the configuration and status ports. These ports are defined as arguments to the FFT function, shown here as variables `fft_status1` and `fft_config1`:

```
hls::fft<param1> (xn1, xk1, &fft_status1, &fft_config1);
```

The runtime configuration and status can be accessed using the predefined structs from the FFT C library:

- `hls::ip_fft::config_t<param1>`
- `hls::ip_fft::status_t<param1>`

***Note*:** In both cases, the struct requires the name of the static parameterization struct, shown in these examples as param1. Refer to the previous section for details on defining the static parameterization struct.

The runtime configuration struct allows the following actions to be performed in the C code:

- Set the FFT length, if runtime configuration is enabled
- Set the FFT direction as forward or inverse
- Set the scaling schedule

The FFT length can be set as follows:

```
typedef hls::ip_fft::config_t<param1> config_t;
config_t fft_config1;
// Set FFT length to 512 => log2(512) =>9
fft_config1.setNfft(9);
```



IMPORTANT! *The length specified during runtime cannot exceed the size defined by `max_nfft` in the static configuration.*

The FFT direction can be set as follows:

```
typedef hls::ip_fft::config_t<param1> config_t;
config_t fft_config1;
// Forward FFT
fft_config1.setDir(1);
// Inverse FFT
fft_config1.setDir(0);
```

The FFT scaling schedule can be set as follows:

```
typedef hls::ip_fft::config_t<param1> config_t;
config_t fft_config1;
fft_config1.setSch(0x2AB);
```

The output status port can be accessed using the pre-defined struct to determine:

- If any overflow occurred during the FFT
- The value of the block exponent

The FFT overflow mode can be checked as follows:

```
typedef hls::ip_fft::status_t<param1> status_t;
status_t fft_status1;
// Check the overflow flag
bool *ovflo = fft_status1.getOvflo();
```

IMPORTANT! *After each transaction completes, check the overflow status to confirm the correct operation of the FFT.*

And the block exponent value can be obtained using:

```
typedef hls::ip_fft::status_t<param1> status_t;
status_t fft_status1;
// Obtain the block exponent
unsigned int *blk_exp = fft_status1.getBlkExp();
```

Using the FFT Function with Array Interface

The FFT function with array arguments is defined in the HLS namespace and can be called as follows:

```
hls::fft<STATIC_PARAM> (
INPUT_DATA_ARRAY,
OUTPUT_DATA_ARRAY,
OUTPUT_STATUS,
INPUT_RUN_TIME_CONFIGURATION);
```

The `STATIC_PARAM` is the static parameterization struct that defines the static parameters for the FFT.

Both the input and output data are supplied to the function as arrays (`INPUT_DATA_ARRAY` and `OUTPUT_DATA_ARRAY`). In the final implementation, the ports on the FFT RTL block will be implemented as AXI4-Stream ports. AMD recommends

IMPORTANT! *You are required to always using the FFT function in a dataflow region using (`set_directive_dataflow`), and to specify both arrays as streaming using the `set_directive_stream` command. The FFT cannot be used in a region which is pipelined. If high-performance operation is required, in particular by avoiding clock cycles in which the FFT is stalled without processing data, then:*

- The entire design hierarchy including and above the FFT must be either dataflow regions or functions containing only dataflow-in-loop.
- The depth of all FIFOs must be enough to avoid blocking in cosimulation, as described in Cosim Deadlock Detection.

- All dataflow processes throughout the hierarchy must contain only:
  - Pipelines with the rewind option
  - Or pipelined loops with the rewind option

The data types for the arrays can be `float` or `ap_fixed`.

```
typedef float data_t;
complex<data_t> in_fft[FFT_LENGTH];
complex<data_t> out_fft[FFT_LENGTH];
```

To use fixed-point data types, the Vitis HLS arbitrary precision type `ap_fixed` should be used.

```
#include "ap_fixed.h"
typedef ap_fixed<FFT_INPUT_WIDTH,1> data_in_t;
typedef ap_fixed<FFT_OUTPUT_WIDTH,FFT_OUTPUT_WIDTH-FFT_INPUT_WIDTH+1>
data_out_t;
#include <complex>
typedef hls::x_complex<data_in_t> cmpxData;
typedef hls::x_complex<data_out_t> cmpxDataOut;
```

In both cases, the FFT should be parameterized with the same correct data sizes. In the case of floating point data, the data widths will always be 32-bit and any other specified size will be considered invalid.

IMPORTANT! *The input and output width of the FFT can be configured to any arbitrary value within the supported range. The variables which connect to the input and output parameters must be defined in increments of 8-bit. For example, if the output width is configured as 33-bit, the output variable must be defined as a 40-bit variable.*

Multi-Channel FFT

The multichannel functionality of the FFT can be used by using two-dimensional arrays for the input and output data. In this case, the array data should be configured with the first dimension representing each channel and the second dimension representing the FFT data.

```
typedef float data_t;
static complex<data_t> in_fft[FFT_CHANNELS][FFT_LENGTH];
static complex<data_t> out_fft[FFT_CHANNELS][FFT_LENGTH];
```

The FFT core consumes and produces data as interleaved channels (for example, ch0-data0, ch1-data0, ch2-data0, etc, ch0-data1, ch1-data1, ch2-data1, etc.). Therefore, to stream the input or output arrays of the FFT using the same sequential order that the data was read or written, you must fill or empty the two-dimensional arrays for multiple channels by iterating through the channel index first, as shown in the following example:

```
cmpxData   in_fft[FFT_CHANNELS][FFT_LENGTH];
cmpxData  out_fft[FFT_CHANNELS][FFT_LENGTH];

// Read data into FFT Input Array
for (unsigned i = 0; i < FFT_LENGTH; i++) {
 for (unsigned j = 0; j < FFT_CHANNELS; ++j) {
```

```
 in_fft[j][i] = in.read();
 }
}

// Write data out from FFT Output Array
for (unsigned i = 0; i < FFT_LENGTH; i++) {
 for (unsigned j = 0; j < FFT_CHANNELS; ++j) {
 out.write(out_fft[j][i]);
 }
}
```

Using FFT Function with Streaming Interface

The FFT function with streaming interfaces is defined in the HLS namespace similarly to this:

```
  template <typename PARAM_T>
  void fft(hls::stream<complex<float or ap_fixed>> &xn_s,
           hls::stream<complex<float or ap_fixed >> &xk_s,
           hls::stream<ip_fft::status_t<CONFIG_T>> &status_s,
           hls::stream<ip_fft::config_t<CONFIG_T>> &config_s);
```

and can be called as follows:

```
hls::fft<STATIC_PARAM> (
INPUT_DATA_STREAM,
OUTPUT_DATA_STREAM,
OUTPUT_STATUS_STREAM,
INPUT_RUN_TIME_CONFIGURATION_STREAM);
```

The `STATIC_PARAM` is the static parameterization struct that defines the static parameters for the FFT.

All input and outputs are supplied to the function as a `hls::stream<>`. In the final implementation, the ports on the FFT RTL block will be implemented as AXI4-Stream ports.

IMPORTANT! *You are required to use the streaming FFT function in a dataflow region using `set_directive_dataflow` or `#pragma HLS dataflow`. The FFT cannot be used in a region which is pipelined. If high-performance operation is required, in particular by avoiding clock cycles in which the FFT is stalled without processing data, then:*

- The entire design hierarchy including and above the FFT must be either dataflow regions or functions containing only dataflow-in-loop.
- The depth of all FIFOs must be enough to avoid blocking in cosimulation, as described in Cosim Deadlock Detection.
- All dataflow processes throughout the hierarchy must contain only:
  - Pipelines with the rewind option
  - Or pipelined loops with the rewind option

The data types for input data and output data streams can be `float` or `ap_fixed`.

```
void fft_top(
    bool direction,
    complex<data_in_t> in[FFT_LENGTH],
    complex<data_out_t> out[FFT_LENGTH],
    bool &ovflo)
{
    #pragma HLS dataflow
    hls::stream<complex<data_in_t>> in_fft;
    hls::stream<complex<data_out_t>> out_fft;
    hls::stream<config_t> fft_config;
    hls::stream<status_t> fft_status;

    // convert inputs to hls::stream<> and generates fft_config stream
based on input arguments
    proc_fe(direction, fft_config, in, in_fft);
    // FFT IP
    hls::fft<config1>(in_fft, out_fft, fft_status, fft_config);
    // convert fft result to outputs data type and sets output ovflo
according to fft_status stream
    proc_be(fft_status, ovflo, out_fft, out);
}
```

To use fixed-point data types, the Vitis HLS arbitrary precision type `ap_fixed` should be used.

```
#include "ap_fixed.h"
typedef ap_fixed<FFT_INPUT_WIDTH,1> data_in_t;
typedef ap_fixed<FFT_OUTPUT_WIDTH,FFT_OUTPUT_WIDTH-FFT_INPUT_WIDTH+1>
data_out_t;
#include <complex>
typedef complex<data_in_t> cmpxData;
typedef complex<data_out_t> cmpxDataOut;
```

In both cases, the FFT should be parameterized with the same correct data sizes. In the case of floating point data, the data widths will always be 32-bit and any other specified size will be considered invalid.

Comparing Stream and Array

The following table shows the array example and stream example usage of the FFT IP. In both cases the dataflow hierarchy extends all the way to the top, without any sequential code (code that is not a dataflow region or a dataflow-in-loop function) and that copy-in and copy-out loops are pipelined with rewind.



TIP: *The top functions in each example are using array interfaces for consistency, but could be changed to use `hls::stream<>` or use the STREAM pragma or directive to the same effect.*

Table 59: **FFT Arguments**

FFT with Array Arguments

```
#include "fft_top.h"

void proc_fe(
 bool direction,
 config_t* config,
 cmpxDataIn in[FFT_LENGTH],
 cmpxDataIn out[FFT_LENGTH])
{
 int i;
 config->setDir(direction);
 config->setSch(0x2AB);
 for (i=0; i< FFT_LENGTH; i++)
 #pragma HLS pipeline II=1 rewind
 out[i] = in[i];
}

void proc_be(
 status_t* status_in,
 bool* ovflo,
 cmpxDataOut in[FFT_LENGTH],
 cmpxDataOut out[FFT_LENGTH])
{
 int i;
 for (i=0; i< FFT_LENGTH; i++)
 #pragma HLS pipeline II=1 rewind
 out[i] = in[i];
 *ovflo = status_in->getOvflo() & 0x1;
}

// TOP function
void fft_top(
 bool direction,
 complex<data_in_t> in[FFT_LENGTH],
 complex<data_out_t> out[FFT_LENGTH],
 bool* ovflo)
{
#pragma HLS interface ap_hs port=direction
#pragma HLS interface ap_fifo depth=1
port=ovflo
#pragma HLS interface ap_fifo
depth=FFT_LENGTH port=in,out
#pragma HLS data_pack variable=in
#pragma HLS data_pack variable=out
#pragma HLS dataflow
 complex<data_in_t> xn[FFT_LENGTH];
 complex<data_out_t> xk[FFT_LENGTH];
 config_t fft_config;
 status_t fft_status;

 // convert inputs to arrays and generates
fft_config
 // based on input arguments
 proc_fe(direction, &fft_config, in, xn);
 // FFT IP
 hls::fft<config1>(xn, xk, &fft_status,
&fft_config);
 // convert fft results to outputs data type
and
 // sets output ovflo according to fft_status
 proc_be(&fft_status, ovflo, xk, out);
}
```

FFT with `hls::stream<>` Arguments

```
#include "fft_top.h"

void proc_fe(
 bool direction,
 hls::stream<config_t> &config_s,
 cmpxDataIn in[FFT_LENGTH],
 hls::stream<cmpxDataIn> &out_s)
{
 int i;
 config_t config;
 config.setDir(direction);
 config.setSch(0x2AB);
 config_s.write(config);
 for (i=0; i< FFT_LENGTH; i++) {
 #pragma HLS pipeline II=1 rewind
 out_s.write(in[i]);
 }
}

void proc_be(
 hls::stream<status_t> &status_in_s,
 bool &ovflo,
 hls::stream<cmpxDataOut> &in_s,
 cmpxDataOut out[FFT_LENGTH])
{
 int i;
 for (i=0; i< FFT_LENGTH; i++) {
 #pragma HLS pipeline II=1 rewind
 out[i] = in_s.read();
 }
 status_t status_in = status_in_s.read();
 ovflo = status_in.getOvflo() & 0x1;
}

// TOP function
void fft_top(
 bool direction,
 complex<data_in_t> in[FFT_LENGTH],
 complex<data_out_t> out[FFT_LENGTH],
 bool &ovflo)
{
#pragma HLS dataflow
 hls::stream<complex<data_in_t>> xn;
 hls::stream<complex<data_out_t>> xk;
 hls::stream<config_t> fft_config;
 hls::stream<status_t> fft_status;
 // convert inputs to hls::stream<>
 // and generates fft_config stream based on
input arguments
 proc_fe(direction, fft_config, in, xn);
 // FFT IP
 hls::fft<config1>(xn, xk, fft_status,
fft_config);
 // convert fft result to outputs data type
 // and sets output ovflo according to
fft_status stream
 proc_be(fft_status, ovflo, xk, out);
}
```

FIR Filter IP Library

The AMD FIR IP block can be called within a C++ design using the library `hls_fir.h`. This section explains how the FIR can be configured in your C++ code.

RECOMMENDED: *AMD highly recommends that you review the FIR Compiler LogiCORE IP Product Guide (PG149) for information on how to implement and use the features of the IP.*

To use the FIR in your C++ code:

1. Include the `hls_fir.h` library in the code.
2. Set the static parameters using the predefined struct `hls::ip_fir::params_t`.
3. Call the FIR function.
4. Optionally, define a runtime input configuration to modify some parameters dynamically.

The following code examples provide a summary of how each of these steps is performed. Each step is discussed in more detail below.

First, include the FIR library in the source code. This header file resides in the include directory in the Vitis HLS installation area. This directory is automatically searched when Vitis HLS executes. There is no need to specify the path to this directory if compiling inside Vitis HLS.

```
#include "hls_fir.h"
```

Define the static parameters of the FIR. This includes such static attributes such as the input width, the coefficients, the filter rate (`single`, `decimation`, `hilbert`). The FIR library includes a parameterization struct `hls::ip_fir::params_t` which can be used to initialize all static parameters with default values.

In this example, the coefficients are defined as residing in array `coeff_vec` and the default values for the number of coefficients, the input width and the quantization mode are over-ridden using a user a user-defined struct `myconfig` based on the predefined struct.

```
struct myconfig : hls::ip_fir::params_t {
    static const double coeff_vec[sg_fir_srrc_coeffs_len];
    static const unsigned num_coeffs = sg_fir_srrc_coeffs_len;
    static const unsigned input_width = INPUT_WIDTH;
    static const unsigned quantization = hls::ip_fir::quantize_only;
};
```

Create an instance of the FIR function using the HLS namespace with the defined static parameters (`myconfig` in this example) and then call the function with the `run` method to execute the function. The function arguments are, in order, input data and output data.

```
static hls::FIR<param1> fir1;
fir1.run(fir_in, fir_out);
```

Optionally, a runtime input configuration can be used. In some modes of the FIR, the data on this input determines how the coefficients are used during interleaved channels or when coefficient reloading is required. This configuration can be dynamic and is therefore defined as a variable. For a complete description of which modes require this input configuration, refer to the *FIR Compiler LogiCORE IP Product Guide* (PG149).

When the runtime input configuration is used, the FIR function is called with three arguments: input data, output data and input configuration.

```
// Define the configuration type
typedef ap_uint<8> config_t;
// Define the configuration variable
config_t fir_config = 8;
// Use the configuration in the FFT
static hls::FIR<param1> fir1;
fir1.run(fir_in, fir_out, &fir_config);
```

TIP: *The example above shows the use of scalar values and arrays, but the FIR function also supports the use of `hls::stream` for arguments. Refer to Vitis HLS Introductory Examples for more information.*

FIR Static Parameters

The static parameters of the FIR define how the FIR IP is parameterized and specifies non-dynamic items such as the input and output widths, the number of fractional bits, the coefficient values, the interpolation and decimation rates. Most of these configurations have default values: there are no default values for the coefficients.

The `hls_fir.h` header file defines a `struct hls::ip_fir::params_t` that can be used to set the default values for most of the static parameters.

IMPORTANT! *There are no defaults defined for the coefficients. Therefore, AMD does not recommend using the pre-defined struct to directly initialize the FIR. A new user defined struct which specifies the coefficients should always be used to perform the static parameterization.*

In this example, a new user `struct my_config` is defined and with a new value for the coefficients. The coefficients are specified as residing in array `coeff_vec`. All other parameters to the FIR use the default values.

```
struct myconfig : hls::ip_fir::params_t {
    static const double coeff_vec[sg_fir_srrc_coeffs_len];
};
static hls::FIR<myconfig> fir1;
fir1.run(fir_in, fir_out);
```

Fir Struct Parameters

The following table describes the parameters used for the parametrization struct `hls::ip_fir::params_t`and lists the default values for the parameters as well as the possible values.

Table 60: **FIR Struct Parameter Values**

Parameter	Description	C Type	Default Value	Valid Values
input_width	Data input port width	unsigned	16	No limitation
input_fractional_bits	Number of fractional bits on the input port	unsigned	0	Limited by size of input_width
output_width	Data output port width	unsigned	24	No limitation
output_fractional_bits	Number of fractional bits on the output port	unsigned	0	Limited by size of output_width
coeff_width	Bit-width of the coefficients	unsigned	16	No limitation
coeff_fractional_bits	Number of fractional bits in the coefficients	unsigned	0	Limited by size of coeff_width
num_coeffs	Number of coefficients	bool	21	Full
coeff_sets	Number of coefficient sets	unsigned	1	1-1024
input_length	Number of samples in the input data	unsigned	21	No limitation
output_length	Number of samples in the output data	unsigned	21	No limitation
num_channels	Specify the number of channels of data to process	unsigned	1	1-1024
total_num_coeff	Total number of coefficients	unsigned	21	num_coeffs * coeff_sets
coeff_vec[total_num_coeff]	The coefficient array	double array	None	Not applicable
filter_type	The type implementation used for the filter	unsigned	single_rate	single_rate, interpolation, decimation, hilbert_filter, interpolated
rate_change	Specifies integer or fractional rate changes	unsigned	integer	integer, fixed_fractional
interp_rate	The interpolation rate	unsigned	1	1-1024
decim_rate	The decimation rate	unsigned	1	1-1024
zero_pack_factor	Number of zero coefficients used in interpolation	unsigned	1	1-8
rate_specification	Specify the rate as frequency or period	unsigned	period	frequency, period
hardware_oversampling_rate	Specify the rate of over-sampling	unsigned	1	No Limitation
sample_period	The hardware oversample period	bool	1	No Limitation
sample_frequency	The hardware oversample frequency	unsigned	0.001	No Limitation
quantization	The quantization method to be used	unsigned	integer_coefficients	integer_coefficients, quantize_only, maximize_dynamic_range
best_precision	Enable or disable the best precision	unsigned	false	false true

Table 60: **FIR Struct Parameter Values** *(cont'd)*

Parameter	Description	C Type	Default Value	Valid Values
coeff_structure	The type of coefficient structure to be used	unsigned	non_symmetric	inferred, non_symmetric, symmetric, negative_symmetric, half_band, hilbert
output_rounding_mode	Type of rounding used on the output	unsigned	full_precision	full_precision, truncate_lsbs, non_symmetric_rounding_down, non_symmetric_rounding_up, symmetric_rounding_to_zero, symmetric_rounding_to_infinity, convergent_rounding_to_even, convergent_rounding_to_odd
filter_arch	Selects a systolic or transposed architecture	unsigned	systolic_multiply_accumulate	systolic_multiply_accumulate, transpose_multiply_accumulate
optimization_goal	Specify a speed or area goal for optimization	unsigned	area	area, speed
inter_column_pipe_length	The pipeline length required between DSP columns	unsigned	4	1-16
column_config	Specifies the number of DSP module columns	unsigned	1	Limited by number of DSP macrocells used
config_method	Specifies how the DSP module columns are configured	unsigned	single	single, by_channel
coeff_padding	Specifies if zero padding is added to the front of the filter	bool	false	false true



IMPORTANT! *When specifying parameter values which are not integer or boolean, the HLS FIR namespace should be used. For example the possible values for* `rate_change` *are shown in the table to be* `integer` *and* `fixed_fractional`*. The values used in the C program should be* `rate_change = hls::ip_fir::integer` *and* `rate_change = hls::ip_fir::fixed_fractional`*.*

Using the FIR Function with Array Interface

The FIR function is defined in the HLS namespace and can be called as follows:

```
// Create an instance of the FIR
static hls::FIR<STATIC_PARAM> fir1;
// Execute the FIR instance fir1
fir1.run(INPUT_DATA_ARRAY, OUTPUT_DATA_ARRAY);
```

The `STATIC_PARAM` is the static parameterization struct that defines most static parameters for the FIR.

Both the input and output data are supplied to the function as arrays (`INPUT_DATA_ARRAY` and `OUTPUT_DATA_ARRAY`). In the final implementation, these ports on the FIR IP will be implemented as AXI4-Stream ports. AMD recommends always using the FIR function in a region using the dataflow optimization (`set_directive_dataflow`), because this ensures the arrays are implemented as streaming arrays. An alternative is to specify both arrays as streaming using the `set_directive_stream` command.

IMPORTANT! *The FIR cannot be used in a region which is pipelined. If high-performance operation is required, pipeline the loops or functions before and after the FIR then use dataflow optimization on all loops and functions in the region.*

The multichannel functionality of the FIR is supported through interleaving the data in a single input and single output array.

- The size of the input array should be large enough to accommodate all samples: `num_channels * input_length`.
- The output array size should be specified to contain all output samples: `num_channels * output_length`.

The following code example demonstrates, for two channels, how the data is interleaved. In this example, the top-level function has two channels of input data (`din_i`, `din_q`) and two channels of output data (`dout_i`, `dout_q`). Two functions, at the front-end (fe) and back-end (be) are used to correctly order the data in the FIR input array and extract it from the FIR output array.

```
void dummy_fe(din_t din_i[LENGTH], din_t din_q[LENGTH], din_t
out[FIR_LENGTH]) {
    for (unsigned i = 0; i < LENGTH; ++i) {
        out[2*i] = din_i[i];
        out[2*i + 1] = din_q[i];
    }
}
void dummy_be(dout_t in[FIR_LENGTH], dout_t dout_i[LENGTH], dout_t
dout_q[LENGTH]) {
    for(unsigned i = 0; i < LENGTH; ++i) {
        dout_i[i] = in[2*i];
        dout_q[i] = in[2*i+1];
    }
}
void fir_top(din_t din_i[LENGTH], din_t din_q[LENGTH],
             dout_t dout_i[LENGTH], dout_t dout_q[LENGTH]) {

 din_t fir_in[FIR_LENGTH];
    dout_t fir_out[FIR_LENGTH];
    static hls::FIR<myconfig> fir1;

    dummy_fe(din_i, din_q, fir_in);
    fir1.run(fir_in, fir_out);
    dummy_be(fir_out, dout_i, dout_q);
}
```

Optional FIR Runtime Configuration

In some modes of operation, the FIR requires an additional input to configure how the coefficients are used. For a complete description of which modes require this input configuration, refer to the *FIR Compiler LogiCORE IP Product Guide* (PG149).

This input configuration can be performed in the C code using a standard ap_int.h 8-bit data type. In this example, the header file `fir_top.h` specifies the use of the FIR and `ap_fixed` libraries, defines a number of the design parameter values and then defines some fixed-point types based on these:

```
#include "ap_fixed.h"
#include "hls_fir.h"

const unsigned FIR_LENGTH   = 21;
const unsigned INPUT_WIDTH = 16;
const unsigned INPUT_FRACTIONAL_BITS = 0;
const unsigned OUTPUT_WIDTH = 24;
const unsigned OUTPUT_FRACTIONAL_BITS = 0;
const unsigned COEFF_WIDTH = 16;
const unsigned COEFF_FRACTIONAL_BITS = 0;
const unsigned COEFF_NUM = 7;
const unsigned COEFF_SETS = 3;
const unsigned INPUT_LENGTH = FIR_LENGTH;
const unsigned OUTPUT_LENGTH = FIR_LENGTH;
const unsigned CHAN_NUM = 1;
typedef ap_fixed<INPUT_WIDTH, INPUT_WIDTH - INPUT_FRACTIONAL_BITS> s_data_t;
typedef ap_fixed<OUTPUT_WIDTH, OUTPUT_WIDTH - OUTPUT_FRACTIONAL_BITS>
m_data_t;
typedef ap_uint<8> config_t;
```

In the top-level code, the information in the header file is included, the static parameterization struct is created using the same constant values used to specify the bit-widths, ensuring the C code and FIR configuration match, and the coefficients are specified. At the top-level, an input configuration, defined in the header file as 8-bit data, is passed into the FIR.

```
#include "fir_top.h"

struct param1 : hls::ip_fir::params_t {
    static const double coeff_vec[total_num_coeff];
    static const unsigned input_length = INPUT_LENGTH;
    static const unsigned output_length = OUTPUT_LENGTH;
    static const unsigned num_coeffs = COEFF_NUM;
    static const unsigned coeff_sets = COEFF_SETS;
};
const double param1::coeff_vec[total_num_coeff] =
    {6,0,-4,-3,5,6,-6,-13,7,44,64,44,7,-13,-6,6,5,-3,-4,0,6};

void dummy_fe(s_data_t in[INPUT_LENGTH], s_data_t out[INPUT_LENGTH],
                config_t* config_in, config_t* config_out)
{
    *config_out = *config_in;
    for(unsigned i = 0; i < INPUT_LENGTH; ++i)
        out[i] = in[i];
}

void dummy_be(m_data_t in[OUTPUT_LENGTH], m_data_t out[OUTPUT_LENGTH])
```

```
{
    for(unsigned i = 0; i < OUTPUT_LENGTH; ++i)
        out[i] = in[i];
}

// DUT
void fir_top(s_data_t in[INPUT_LENGTH],
             m_data_t out[OUTPUT_LENGTH],
             config_t* config)
{

    s_data_t fir_in[INPUT_LENGTH];
    m_data_t fir_out[OUTPUT_LENGTH];
    config_t fir_config;
    // Create struct for config
    static hls::FIR<param1> fir1;

    //================================================
// Dataflow process
    dummy_fe(in, fir_in, config, &fir_config);
    fir1.run(fir_in, fir_out, &fir_config);
    dummy_be(fir_out, out);
    //================================================
}
```

Using FIR Function with Streaming Interface

The `run()` function with streaming interfaces and without config input is defined in the HLS namespace similar to this:

```
 void run(
 hls::stream<in_data_t> &in_V,
 hls::stream<out_data_t> &out_V);
```

With config input, it is defined similar to this:

```
 void run(
 hls::stream<in_data_t> &in_V,
 hls::stream<out_data_t> &out_V,
 hls::stream<config_t> &config_V);
```

The FIR function is defined in the HLS namespace and can be called as follows:

```
// Create an instance of the FIR
static hls::FIR<STATIC_PARAM> fir1;
// Execute the FIR instance fir1
fir1.run(INPUT_DATA_STREAM, OUTPUT_DATA_STREAM);
```

The STATIC_PARAM is the static parameterization struct that defines most static parameters for the FIR. Both the input and output data are supplied to the function as `hls::stream<>`. These ports on the FIR IP will be implemented as AXI4-Stream ports.

AMD recommends always using the FIR function in a dataflow region using `set_directive_dataflow` or `#pragma HLS dataflow`.

IMPORTANT! *The FIR cannot be used in a region which is pipelined. If high-performance operation is required, pipeline the loops or functions before and after the FIR, and use the DATAFLOW pragma or directive on all loops and functions in the region, as shown in the exampe below.*

The multichannel functionality of the FIR is supported through interleaving the data in a single input and single output stream.

- The size of the input stream should be large enough to accommodate all samples: `num_channels * input_length`
- The output stream size should be specified to contain all output samples: `num_channels * output_length`

The following code example demonstrates how the FIR IP function can be used.

```
template<typename data_t, int LENGTH>
void process_fe(data_t in[LENGTH], hls::stream<data_t> &out)
{
    for(unsigned i = 0; i < LENGTH; ++i)
        out.write(in[i]);
}

template<typename data_t, int LENGTH>
void process_be(hls::stream<data_t> &in, data_t out[LENGTH])
{
    for(unsigned i = 0; i < LENGTH; ++i)
        out[i] = in.read();
}

// TOP function
void fir_top(
    data_t in[FIR1_LENGTH],
    data_out_t out[FIR2_LENGTH])
{

    #pragma HLS dataflow

    hls::stream<data_t> fir1_in;
    hls::stream<data_intern_t> fir1_out;
    hls::stream<data_out_t> fir2_out;

    // Create FIR instance
    static hls::FIR<config1> fir1;
    static hls::FIR<config2> fir2;

    //================================================
    // Dataflow process
    process_fe<data_t, FIR1_LENGTH>(in, fir1_in);
    fir1.run(fir1_in, fir1_out);
    fir2.run(fir1_out, fir2_out);
    process_be<data_out_t, FIR2_LENGTH>(fir2_out, out);
    //================================================
}
```

TIP: *To avoid bubbles in the execution of the FIR you need to ensure the FIFO depth is sufficient for the data throughput in the dataflow region using the `config_dataflow -fifo_depth` command, and might need to apply loop rewind to the loops in `process_fe` and `process_be` as described in the PIPELINE pragma or directive.*

DDS IP Library

You can use the AMD Direct Digital Synthesizer (DDS) IP block within a C++ design using the `hls_dds.h` library. This section explains how to configure DDS IP in your C++ code.

RECOMMENDED: *AMD highly recommends that you review the DDS Compiler LogiCORE IP Product Guide (PG141) for information on how to implement and use the features of the IP.*

IMPORTANT! *The C IP implementation of the DDS IP core supports the fixed mode for the Phase_Increment and Phase_Offset parameters and supports the `none` mode for Phase_Offset, but it does not support `programmable` and `streaming` modes for these parameters.*

To use the DDS in the C++ code:

1. Include the `hls_dds.h` library in the code.
2. Set the default parameters using the pre-defined struct `hls::ip_dds::params_t`.
3. Call the DDS function.

First, include the DDS library in the source code. This header file resides in the include directory in the Vitis HLS installation area, which is automatically searched when Vitis HLS executes.

```
#include "hls_dds.h"
```

Define the static parameters of the DDS. For example, define the phase width, clock rate, and phase and increment offsets. The DDS C library includes a parameterization struct `hls::ip_dds::params_t`, which is used to initialize all static parameters with default values. By redefining any of the values in this struct, you can customize the implementation.

The following example shows how to override the default values for the phase width, clock rate, phase offset, and the number of channels using a user-defined struct `param1`, which is based on the existing predefined struct `hls::ip_dds::params_t`:

```
struct param1 : hls::ip_dds::params_t {
 static const unsigned Phase_Width = PHASEWIDTH;
 static const double   DDS_Clock_Rate = 25.0;
 static const double PINC[16];
 static const double POFF[16];
};
```

Create an instance of the DDS function using the HLS namespace with the defined static parameters (for example, `param1`). Then, call the function with the run method to execute the function. Following are the data and phase function arguments shown in order:

```
static hls::DDS<config1> dds1;
dds1.run(data_channel, phase_channel);
```

DDS Static Parameters

The static parameters of the DDS define how to configure the DDS, such as the clock rate, phase interval, and modes. The `hls_dds.h` header file defines an `hls::ip_dds::params_t` struct, which sets the default values for the static parameters. To use the default values, you can use the parameterization struct directly with the DDS function.

```
static hls::DDS< hls::ip_dds::params_t > dds1;
dds1.run(data_channel, phase_channel);
```

The following table describes the parameters for the `hls::ip_dds::params_t` parameterization struct.

RECOMMENDED: *AMD highly recommends that you review the DDS Compiler LogiCORE IP Product Guide (PG141) for details on the parameters and values.*

Table 61: **DDS Struct Parameters**

Parameter	Description
`DDS_Clock_Rate`	Specifies the clock rate for the DDS output.
`Channels`	Specifies the number of channels. The DDS and phase generator can support up to 16 channels. The channels are time-multiplexed, which reduces the effective clock frequency per channel.
`Mode_of_Operation`	Specifies one of the following operation modes: Standard mode for use when the accumulated phase can be truncated before it is used to access the SIN/COS LUT. Rasterized mode for use when the desired frequencies and system clock are related by a rational fraction.
`Modulus`	Describes the relationship between the system clock frequency and the desired frequencies. Use this parameter in rasterized mode only.
`Spurious_Free_Dynamic_Range`	Specifies the targeted purity of the tone produced by the DDS.
`Frequency_Resolution`	Specifies the minimum frequency resolution in Hz and determines the Phase Width used by the phase accumulator, including associated phase increment (PINC) and phase offset (POFF) values.
`Noise_Shaping`	Controls whether to use phase truncation, dithering, or Taylor series correction.
`Phase_Width`	Sets the width of the following: • **PHASE_OUT** field within `m_axis_phase_tdata` • **Phase** field within `s_axis_phase_tdata` when the DDS is configured to be a SIN/COS LUT only • Phase accumulator • Associated phase increment and offset registers • **Phase** field in `s_axis_config_tdata` For rasterized mode, the phase width is fixed as the number of bits required to describe the valid input range `[0, Modulus-1]`, that is, `log2 (Modulus-1)` rounded up.
`Output_Width`	Sets the width of **SINE** and **COSINE** fields within `m_axis_data_tdata`. The SFDR provided by this parameter depends on the selected **Noise Shaping** option.

Table 61: **DDS Struct Parameters** *(cont'd)*

Parameter	Description
`Phase_Increment`	Selects the phase increment value.
`Phase_Offset`	Selects the phase offset value.
`Output_Selection`	Sets the output selection to **SINE**, **COSINE**, or both in the `m_axis_data_tdata` bus.
`Negative_Sine`	Negates the **SINE** field at runtime.
`Negative_Cosine`	Negates the **COSINE** field at runtime.
`Amplitude_Mode`	Sets the amplitude to full range or unit circle.
`Memory_Type`	Controls the implementation of the SIN/COS LUT.
`Optimization_Goal`	Controls whether the implementation decisions target highest speed or lowest resource.
`DSP48_Use`	Controls the implementation of the phase accumulator and addition stages for phase offset, dither noise addition, or both.
`Latency_Configuration`	Sets the latency of the core to the optimum value based upon the Optimization Goal.
`Latency`	Specifies the manual latency value.
`Output_Form`	Sets the output form to two's complement or to sign and magnitude. In general, the output of SINE and COSINE is in two's complement form. However, when quadrant symmetry is used, the output form can be changed to sign and magnitude.
`PINC[XIP_DDS_CHANNELS_MAX]`	Sets the values for the phase increment for each output channel.
`POFF[XIP_DDS_CHANNELS_MAX]`	Sets the values for the phase offset for each output channel.

DDS Struct Parameter Values

The following table shows the possible values for the `hls::ip_dds::params_t` parameterization struct parameters.

Table 62: **DDS Struct Parameter Values**

Parameter	C Type	Default Value	Valid Values
DDS_Clock_Rate	double	20.0	Any double value
Channels	unsigned	1	1 to 16
Mode_of_Operation	unsigned	XIP_DDS_MOO_CONVENTIONAL	XIP_DDS_MOO_CONVENTIONAL truncates the accumulated phase. XIP_DDS_MOO_RASTERIZED selects rasterized mode.
Modulus	unsigned	200	129 to 256
Spurious_Free_Dynamic_Range	double	20.0	18.0 to 150.0
Frequency_Resolution	double	10.0	0.000000001 to 125000000

Table 62: **DDS Struct Parameter Values** *(cont'd)*

Parameter	C Type	Default Value	Valid Values
Noise_Shaping	unsigned	XIP_DDS_NS_NONE	XIP_DDS_NS_NONE produces phase truncation DDS. XIP_DDS_NS_DITHER uses phase dither to improve SFDR at the expense of increased noise floor. XIP_DDS_NS_TAYLOR interpolates sine/cosine values using the otherwise discarded bits from phase truncation XIP_DDS_NS_AUTO automatically determines noise-shaping.
Phase_Width	unsigned	16	Must be an integer multiple of 8
Output_Width	unsigned	16	Must be an integer multiple of 8
Phase_Increment	unsigned	XIP_DDS_PINCPOFF_FIXED	XIP_DDS_PINCPOFF_FIXED fixes PINC at generation time, and PINC cannot be changed at runtime. This is the only value supported.
Phase_Offset	unsigned	XIP_DDS_PINCPOFF_NONE	XIP_DDS_PINCPOFF_NONE does not generate phase offset. XIP_DDS_PINCPOFF_FIXED fixes POFF at generation time, and POFF cannot be changed at runtime.
Output_Selection	unsigned	XIP_DDS_OUT_SIN_AND_COS	XIP_DDS_OUT_SIN_ONLY produces sine output only. XIP_DDS_OUT_COS_ONLY produces cosine output only. XIP_DDS_OUT_SIN_AND_COS produces both sin and cosine output.
Negative_Sine	unsigned	XIP_DDS_ABSENT	XIP_DDS_ABSENT produces standard sine wave. XIP_DDS_PRESENT negates sine wave.
Negative_Cosine	bool	XIP_DDS_ABSENT	XIP_DDS_ABSENT produces standard sine wave. XIP_DDS_PRESENT negates sine wave.
Amplitude_Mode	unsigned	XIP_DDS_FULL_RANGE	XIP_DDS_FULL_RANGE normalizes amplitude to the output width with the binary point in the first place. For example, an 8-bit output has a binary amplitude of 100000000 - 10 giving values between 01111110 and 11111110, which corresponds to just less than 1 and just more than -1, respectively. XIP_DDS_UNIT_CIRCLE normalizes amplitude to half full range, that is, values range from 01000 .. (+0.5). to 110000 .. (-0.5).

Table 62: **DDS Struct Parameter Values** *(cont'd)*

Parameter	C Type	Default Value	Valid Values
Memory_Type	unsigned	XIP_DDS_MEM_AUTO	XIP_DDS_MEM_AUTO selects distributed ROM for small cases where the table can be contained in a single layer of memory and selects block ROM for larger cases. XIP_DDS_MEM_BLOCK always uses block RAM. XIP_DDS_MEM_DIST always uses distributed RAM.
Optimization_Goal	unsigned	XIP_DDS_OPTGOAL_AUTO	XIP_DDS_OPTGOAL_AUTO automatically selects the optimization goal. XIP_DDS_OPTGOAL_AREA optimizes for area. XIP_DDS_OPTGOAL_SPEED optimizes for performance.
DSP48_Use	unsigned	XIP_DDS_DSP_MIN	XIP_DDS_DSP_MIN implements the phase accumulator and the stages for phase offset, dither noise addition, or both in FPGA logic. XIP_DDS_DSP_MAX implements the phase accumulator and the phase offset, dither noise addition, or both using DSP slices. In the case of single channel, the DSP slice can also provide the register to store programmable phase increment, phase offset, or both and thereby, save further fabric resources.
Latency_Configuration	unsigned	XIP_DDS_LATENCY_AUTO	XIP_DDS_LATENCY_AUTO automatically determines he latency. XIP_DDS_LATENCY_MANUAL manually specifies the latency using the Latency option.
Latency	unsigned	5	Any value
Output_Form	unsigned	XIP_DDS_OUTPUT_TWOS	XIP_DDS_OUTPUT_TWOS outputs two's complement. XIP_DDS_OUTPUT_SIGN_MAG outputs signed magnitude.
PINC[XIP_DDS_CHANNELS_MAX]	unsigned array	{0}	Any value for the phase increment for each channel
POFF[XIP_DDS_CHANNELS_MAX]	unsigned array	{0}	Any value for the phase offset for each channel

SRL IP Library

C code is written to satisfy several different requirements: reuse, readability, and performance. Until now, it is unlikely that the C code was written to result in the most ideal hardware after high-level synthesis.

Like the requirements for reuse, readability, and performance, certain coding techniques or pre-defined constructs can ensure that the synthesis output results in more optimal hardware or to better model hardware in C for easier validation of the algorithm.

Mapping Directly into SRL Resources

Many C algorithms sequentially shift data through arrays. They add a new value to the start of the array, shift the existing data through array, and drop the oldest data value. This operation is implemented in hardware as a shift register.

This most common way to implement a shift register from C into hardware is to completely partition the array into individual elements, and allow the data dependencies between the elements in the RTL to imply a shift register.

Logic synthesis typically implements the RTL shift register into an AMD SRL resource, which efficiently implements shift registers. The issue is that sometimes logic synthesis does not implement the RTL shift register using an SRL component:

- When data is accessed in the middle of the shift register, logic synthesis cannot directly infer an SRL.
- Sometimes, even when the SRL is ideal, logic synthesis may implement the shift-resister in flip-flops, due to other factors. (Logic synthesis is also a complex process).

Vitis HLS provides a C++ class (`ap_shift_reg`) to ensure that the shift register defined in the C code is always implemented using an SRL resource. The `ap_shift_reg` class has two methods to perform the various read and write accesses supported by an SRL component.

Read from the Shifter

The read method allows a specified location to be read from the shifter register.

The `ap_shift_reg.h` header file that defines the `ap_shift_reg` class is also included with Vitis HLS as a standalone package. You have the right to use it in your own source code. The package `xilinx_hls_lib_<release_number>.tgz` is located in the `include` directory in the Vitis HLS installation area.

```
// Include the Class
#include "ap_shift_reg.h"

// Define a variable of type ap_shift_reg<type, depth>
// - Sreg must use the static qualifier
// - Sreg will hold integer data types
// - Sreg will hold 4 data values
static ap_shift_reg<int, 4> Sreg;
int var1;

// Read location 2 of Sreg into var1
var1 = Sreg.read(2);
```

Read, Write, and Shift Data

A `shift` method allows a read, write, and shift operation to be performed.

```
// Include the Class
#include "ap_shift_reg.h"

// Define a variable of type ap_shift_reg<type, depth>
// - Sreg must use the static qualifier
// - Sreg will hold integer data types
// - Sreg will hold 4 data values
static ap_shift_reg<int, 4> Sreg;
int var1;

// Read location 3 of Sreg into var1
// THEN shift all values up one and load In1 into location 0
var1 = Sreg.shift(In1,3);
```

Read, Write, and Enable-Shift

The `shift` method also supports an enabled input, allowing the shift process to be controlled and enabled by a variable.

```
// Include the Class
#include "ap_shift_reg.h"

// Define a variable of type ap_shift_reg<type, depth>
// - Sreg must use the static qualifier
// - Sreg will hold integer data types
// - Sreg will hold 4 data values
static ap_shift_reg<int, 4> Sreg;
int var1, In1;
bool En;
```

```
// Read location 3 of Sreg into var1
// THEN if En=1
// Shift all values up one and load In1 into location 0
var1 = Sreg.shift(In1,3,En);
```

When using the `ap_shift_reg` class, Vitis HLS creates a unique RTL component for each shifter. When logic synthesis is performed, this component is synthesized into an SRL resource.

Working with OpenCV

As described at https://github.com/Xilinx/Vitis_Libraries/tree/main/vision, to work with the Vitis Vision Library you must also have installed the OpenCV computer vision library. Vitis HLS does not come with OpenCV pre-installed, but the following link describes how to find and install it: Vitis Libraries - Compiling and Installing OpenCV libraries for use with Vision library.

AMD

Section VII

Vitis HLS Migration Guide

This section contains the following chapters:

- Migrating from Vitis HLS to the Vitis Unified IDE
- Migrating from Vivado HLS to Vitis HLS
- Deprecated and Unsupported Features
- Unsupported Features

Chapter 32

Migrating from Vitis HLS to the Vitis Unified IDE

The Vitis Unified IDE provides a common tool for creating the different components of a heterogeneous system design, or a Data Center acceleration application. These components include the HLS component generated from C/C++ source code. It includes the `v++` and `vitis-run` common command-line for compiling and linking the system design. Vitis Unified IDE provides multiple ways to migrate an existing Vitis HLS classic project into the new tool. These include the following:

1. Create an HLS Component from an existing Vitis HLS classic project
2. Continue using an existing Tcl script
3. Convert to using Python script

1. Create an HLS Component from an Exisiting Vitis HLS Project

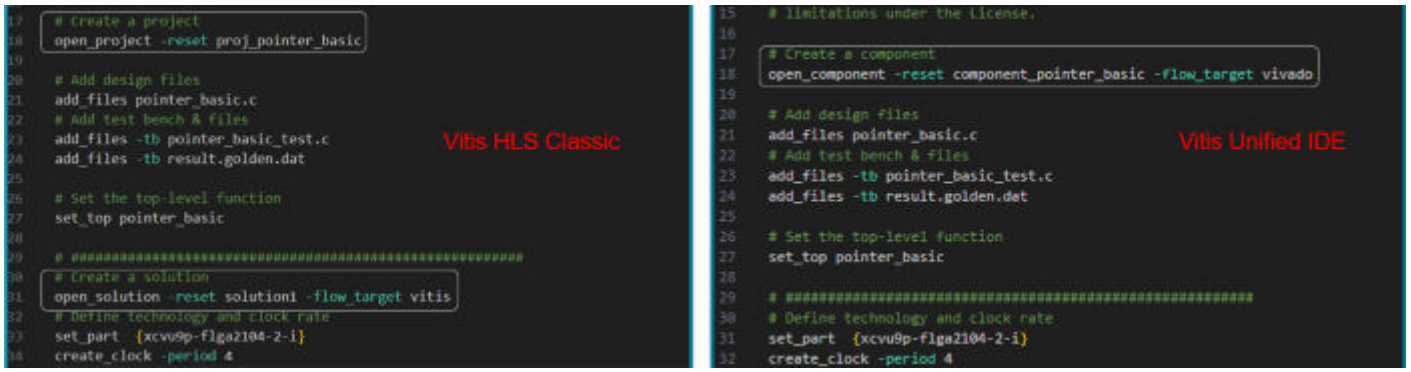
As described in Creating an HLS Component you can specify the `hls.app` file from an existing Vitis HLS classic project to generate a new HLS config file. The tool lets you identify the `hls.app` file inside the project, and enter a New configuration file name for the config file generated from the `hls.app` file.



TIP: *If the project contains multiple solutions, the `hls.app` file identifies the solutions, and the HLS Component Creation wizard will prompt you to specify which of the available solutions should be imported. This allows creation of separate HLS components for each solution in the project.*

2. Continue Using an Existing Tcl Script

To continue to use the existing Tcl script, replace`open_project` with `open_component`and specify `-flow_target vivado|vitis`. Remove `open_solution` Tcl command as it is not required in Vitis Unified IDE as shown below.



By using `open_component` the output structure created is compatible with the Vitis Unified IDE and the directory from where the Tcl script is run is the workspace that can be opened in the Vitis Unified IDE. The hls reports and generated RTL files are now under the component directory. Tcl script can be run using `vitis-run` as shown below.

```
vitis-run --mode hls --tcl <tcl_script>.tcl
```

3. Convert to Using Python Script

Vitis Unified IDE supports Python APIs to create and manage various components including HLS component. A python script can be used containing these APIs for HLS component development and execution. HLS component configuration can be specified either in a config file (generated using `write_ini` as described above) or inside the python script as key-value pairs. The python script can be run using `vitis` as shown below.

```
vitis -s <python_script>.py
```

Use write_ini to Create an HLS Config File

The `vitis_hls` executable has the ability to create a config file from an existing Tcl script using the write_ini command. This can be done by either including the `write_ini` command in the Tcl script or by launching `vitis_hls` in interactive mode (shown below) and specifying the `write_ini` command to export the Tcl commands used to create a project, or a specific solution. After creating the HLS config files from the existing project or solutions, HLS components can be created in the Vitis Unified IDE using these config files along with the specified source files.

```
$ vitis_hls -i
vitis_hls> open_project <project_name>
vitis_hls> write_ini <config_file_name>
```

Additionally, for users who prefer scripted flows for HLS development with the Vitis Unified IDE there are a couple of ways as described next along with examples provided in the Migration directory of the Vitis-HLS-Introductory-Examples repository.

Migrating from Vivado HLS to Vitis HLS

AMD Vivado™ HLS was the older generation HLS solution that has its last official release in 2020.1. AMD Vitis™ HLS represents the next-generation HLS solution from AMD and has several improvements over the older Vivado HLS technology. First and foremost, AMD Vitis™ HLS has a new compiler that uses an updated version of the LLVM compiler standard and supports C/C++ 11/14 for compilation/simulation. When migrating a kernel module or IP implemented with one version of Vivado HLS, it is essential to understand the difference between the versions of HLS, and the impact that these differences have on the design.

The key differences between these version of HLS versions can be categorized as follows:

- Key Behavioral Differences
- Deprecated Commands
- Unsupported Features

In addition, Vitis HLS now supports the following new and stricter syntax checking for both source code and pragma usage:

- Syntax Checker: Vitis HLS will error out on unconnected ports (either in dataflow regions or during RTL generation).
- Pragma Conflict Checker: New warnings and errors are reported for pragma conflicts and/or typos in pragma options.
- New and Improved Messages when user-specified pragmas are ignored by the tool.

Key Behavioral Differences

This section highlights some of the key behavioral differences between HLS and AMD Vitis™ HLS. These key differences can be the source of big quality of results (QoR) differences and therefore, it is necessary to understand these key differences when comparing QoR between different versions of HLS tools. AMD recommends reviewing this section before using Vitis HLS.

TIP: *Due to the behavioral differences between Vitis HLS and Vivado HLS, you might need to differentiate your code for use in the Vitis tool. To enable the same source code to be used in both tools, Vitis HLS supports the `__VITIS_HLS__` predefined macro to encapsulate source code written specifically for use in that tool. Use `#if defined( __VITIS_HLS__)` type pre-processor declarations to encapsulate tool specific code.*

Default User Control Settings

The default global option configures the solution for either Vitis kernel acceleration development flow or Vivado IP development flow.

```
open_solution -flow_target [vitis | vivado]
```

This global option is replacing the previous config option (`config_sdx`).

Vivado IP Development Flow

Configures the solution to run in support of the Vivado IP generation flow, requiring strict use of pragmas and directives, and exporting the results as Vivado IP. The command to set up the project solution for the Vivado IP flow is:

```
open_solution -flow_target vivado
```

The table below shows the original default settings of command options in the Vivado HLS tool, and the new defaults applied in the Vitis HLS tool.

Table 63: **Default Control Settings**

Default Control Settings	Vivado HLS	Vitis HLS
`config_compile -pipeline_loops`	0	64
`config_export -vivado_optimization_level`	2	0
`set_clock_uncertainty`	12.5%	27%
`config_interface -m_axi_alignment_byte_size`	N/A	0
`config_interface -m_axi_max_widen_bitwidth`	N/A	0
`config_export -vivado_phys_opt`	place	none
`config_interface -m_axi_addr64`	false	true
`config_schedule -enable_dsp_full_reg`	false	true
`config_rtl -module_auto_prefix`	false	true
interface pragma defaults	ip mode	ip mode

Vitis Application Acceleration Development Flow (Kernel Mode)

Configures the solution to run in support of the Vivado IP generation flow, requiring strict use of pragmas and directives, and exporting the results as Vivado IP. The command to set up the project solution for the Vivado IP flow is:

```
open_solution -flow_target vivado
```

The table below shows the original default settings of command options in the Vivado HLS tool, and the new defaults applied in the Vitis HLS tool.

Table 64: **Default Control Settings**

Default Control Settings	Vivado HLS	Vitis HLS
`config_compile -pipeline_loops`	0	64
`config_export -vivado_optimization_level`	2	0
`set_clock_uncertainty`	12.5%	27%
`config_interface -m_axi_alignment_byte_size`	N/A	0
`config_interface -m_axi_max_widen_bitwidth`	N/A	0
`config_export -vivado_phys_opt`	place	none
`config_interface -m_axi_addr64`	false	true
`config_schedule -enable_dsp_full_reg`	false	true
`config_rtl -module_auto_prefix`	false	true
interface pragma defaults	ip mode	ip mode

Default Interfaces

Both AMD Vivado™ HLS and AMD Vitis™ HLS allow the user to customize the interfaces to the top-level function using pragma/directions. Vitis HLS additionally supports the ability to automatically infer the right interface. The type of interfaces that Vitis HLS creates depends on:

- The data type of the C arguments in the top-level function.
- The target flow.
- The default interface mode.
- Any user-specified INTERFACE pragmas or directives.

This is fully described in the Defining Interfaces section of the user guide (UG1399) and should be reviewed before proceeding further.

Interface Bundle Rules

By default, AMD Vitis™ HLS groups function arguments with compatible options into a single `m_axi/s_axilite` interface adapter. Bundling ports into a single interface helps save device resources by eliminating AXI4 logic, which can be necessary when working in congested designs. However, a single interface bundle can limit the performance of the kernel because all the memory transfers have to go through a single interface. For example, the `m_axi interface` has independent READ and WRITE channels, so a single interface can read and write simultaneously, though only at one location. Using multiple bundles lets you increase the bandwidth and throughput of the kernel by creating multiple interfaces to connect to memory banks.

Bundle rules for AXI interfaces are fully described in M_AXI Bundles/S_AXILITE Bundles section in the user guide (UG1399) and should be reviewed before proceeding further.

Memory Property on Interface

The `storage_type` option on the interface pragma or directive lets the user explicitly define which type of RAM is used, and which RAM ports are created (single-port or dual-port). If no `storage_type` is specified, Vitis HLS uses:

- A single-port RAM by default.
- A dual-port RAM if it reduces the initiation interval or latency.

For the Vivado flow, the user can specify a RAM storage type on the specified interface, replacing the deprecated RESOURCE pragma with the `storage_type`.

```
#pragma HLS INTERFACE ap_memory port = in1 storage_type=RAM_2P
#pragma HLS INTERFACE ap_memory port = out storage_type=RAM_1P latency=3
```

AXI4-Stream Interfaces with Side-Channels

Side-channels are optional signals, which are part of the AXI4-Stream standard. The side-channel signals can be directly referenced and controlled in the C/C++ code using a struct, provided that the member elements of the struct match the names of the AXI4-Stream side-channel signals. The AXI4-Stream side-channel signals are considered data signals and are registered whenever `TDATA` is registered. There are significant differences in how this is handled in Vivado HLS vs Vitis HLS. The AXI4-Stream Interfaces with Side-Channels section in the user guide (UG1399) details the limitations and restrictions in Vitis HLS.

Memory Model

One key difference between AMD Vivado™ HLS and AMD Vitis™ HLS is how the internal memory model is implemented. The memory model defines the way data is arranged and accessed in computer memory. It consists of two separate but related issues: data alignment and data structure padding. Vitis HLS uses a different memory model from Vivado HLS and the key differences are as follows:

- Vitis HLS follows the GCC Compiler standard for data alignment
- Vitis HLS differs significantly from Vivado HLS in how it aggregates and disaggregates structs/classes in the interface
  - Vivado HLS used to disaggregate structs in the interface by default. Vitis HLS instead keeps structs aggregated. You can use the AGGREGATE/pragma HLS disaggregate pragma/directive to match the Vivado HLS behavior.
    - The exception is when there is a hls::stream object in the struct or an array inside the struct is partitioned, Vitis HLS will disaggregate the struct in both of these cases.
  - Structs in the code, both internal and global variables, are disaggregated by default and decomposed into their member elements, as described in Structs. The number and type of elements created are determined by the contents of the struct itself. Arrays of structs are implemented as multiple arrays, with a separate array for each member of the struct.
- Vitis HLS also differs in how data alignment is handled for interface protocols (such as AXI).
- Changes also include deprecated pragmas such as DATA_PACK.

These changes are fully described in the Vitis HLS Memory Layout Model section of the user guide (UG1399) and should be reviewed before proceeding further.

Unconnected Ports

Vivado HLS accepted unconnected output signals for streamed single and multidimensional arrays.

Vitis HLS allows the following types of unconnected outputs:

- Unconnected output scalars
- Unconnected output single and multidimensional arrays implemented as PIPOs when used inside a dataflow region

Vitis HLS does not allow:

- Unconnected output streams
- Unconnected output streamed single or multidimensional arrays

Global Variables on the Interface

Both Vivado HLS and AMD Vitis™ HLS allow global variables to be freely used in the code and are fully synthesizable. However, while Vivado HLS supported exporting global variables in the top-level function interface, AMD Vitis™ HLS does not.

Global variables can not be used as arguments to the top-level function, or exposed as ports on the RTL interface in AMD Vitis™ HLS. Variables needed on the interface of the top-level function must be explicitly declared in the interface.

Behavior Changes to Module Names and Module Prefix

In Vivado HLS, when `config_rtl -module_auto_prefix` was enabled the top RTL module would have its name prefixed with its own name. SinceVitis HLS 2020.1, this auto prefix will only be applied to sub-modules.

There is no change to the `-module_prefix` behavior. If this option is used, the specified prefix value will be prepended to all modules including the top module. The `-module_prefix` option also still takes precedence over `-module_auto_prefix`.

```
# vivado HLS 2020.1 generated module names (top module is "top")
top_top.v
top_submodule1.v
top_submodule2.v

# Vitis HLS 2020.1 generated module names
top.v             <-- top module no longer has prefix
top_submodule1.v
top_submodule2.v
```

Updated Memory (RAM/ROM) Module Name and RTL File Name

The new naming of the module is appended with the type of memory as shown below.

Table 65: **Module and RTL File Names**

OLD Module and RTL File Names	NEW Module and RTL File Names
ncp_encoder_func_parbits_memcore_ram	ncp_encoder_func_parbits_**RAM_1P_LUTRAM_1R1W**
test_A_V_ROM	test_A_V_ROM_**1P_BRAM_1R, test_A_V_ROM_1P_BRAM_1.v**

Behavior Differences in Inlining of Functions

Automatic inlining of functions in Vivado HLS was governed by the size of the function. However, in Vitis HLS the automatic inlining optimization is driven by user pragma preferences or by a cost model. For example, consider the case of a loop that is being pipelined, and calls to sub-functions are being made inside this loop body. In Vivado HLS, these sub-functions may have been automatically inlined due to their small size. In Vitis HLS they will only be inlined if the user requests a pipeline of II=1. So the automatic inlining will only occur if the Vitis HLS scheduler determines that there is a benefit to inlining these sub-functions to achieve II=1 in this pipelined loop.

Due to this change you may see differences in the QoR achieved by Vitis HLS and Vivado HLS. You can overcome these differences in Vitis HLS by manually inlining the sub-functions to match the prior behavior of Vivado HLS.

Dataflow

Support of std::complex:

In Vivado HLS, std::complex data type could not be used directly inside the DATAFLOW, because of multiple writers issue. This multiple reader and writer issue is coming from the std class constructor being called to initialize the value. When this variable is also used inside the dataflow as a channel, it leads to the above issue. However, Vitis supports the use of `std::complex` with support of an attribute `no_ctor` as shown below.

```
// Nothing to do here.
void proc_1(std::complex<float> (&buffer)[50], const std::complex<float>
*in);
void proc_2(hls::stream<std::complex<float>> &fifo, const
std::complex<float> (&buffer)[50], std::complex<float> &acc);
void proc_3(std::complex<float> *out, hls::stream<std::complex<float>>
&fifo, const std::complex<float> acc);

void top(std::complex<float> *out, const std::complex<float> *in) {
#pragma HLS DATAFLOW
  std::complex<float> acc __attribute((no_ctor)); // here
  std::complex<float> buffer[50] __attribute__((no_ctor)); // here
  hls::stream<std::complex<float>, 5> fifo; // no need here (hls::stream
has it internally)

  proc_1(buffer, in);
  proc_2(fifo, buffer, acc);
  porc_3(out, fifo, acc);
}
```

Migration from 2023.2 to 2024.2 (New Host Compiler)

As you migrate to 2024.2, there will be changes in HLS behavior during C-simulation and C/RTL co-simulation. The synthesized hardware is NOT affected.

It is possible for the affected code to produce a different result during C simulation or C/RTL co-simulation compared to previous versions of Vitis HLS. The following code constructs could result in a different behavior.

1. Undefined behaviors in the test bench or design.

   Undefined behaviors occur when the C/C++ language specifications mandate that the result of an operation has unpredictable effects. Typical examples of undefined behaviors include integer overflows, divisions by zero, array accesses out of bounds, missing return statements, or shifts by a negative value or by a value greater or equal than the number of bits used to store the shifted value. The semantics of C/C++ with undefined behavior is unpredictable and can change due to many factors, including the migration to Vitis HLS 2024.2.

   Vitis HLS 2024.2 now provides address and undefined-behavior sanitizers which should be enabled during C Simulation to find these issues.

2. Unspecified and Implementation-defined behaviors in the test bench.

   Unspecified and implementation-defined behaviors happen when the C/C++ language specifications leave it to the compiler to decide which semantics to implement for some operations. Typical examples include the evaluation order of some operators like arithmetic operators or the `,` operator (including for arguments of function calls), use of out-of-range enum values, comparison of string litterals, the size of fundamental types, the rounding mode for un-representable float litterals, of the value of pow(0, 0).

   Because clang is now used instead of gcc to create the host executables during C simulation and for the test bench of C/RTL co-simulation, a different behavior can occur compared to previous releases of Vitis HLS. When the new behavior affects the design in C simulation, the resulting semantics should be closer to that of the synthesized designed compared to previous Vitis HLS releases.

3. gcc-specific C simulation or C/RTL co-simulation options.

   These options are not supported and need to be adjusted to their equivalent in Clang.

Deprecated and Unsupported Features

These deprecated commands will be discontinued in a future release, and are not recommended for use:

Table 66: **Vivado HLS Commands Deprecated in Vitis HLS**

Type	Command	Option	Vitis HLS	Details
config	config_interface	-m_axi_max_data_size	Deprecated	
config	config_interface	-m_axi_min_data_size	Deprecated	
config	config_interface	-m_axi_alignment_byte_size	Deprecated	
config	config_interface	-m_axi_offset=slave	Unsupported	This is now handled through a combination of -m_axi_offset=direct and -default_slave_interface=s_axilite
config	config_interface	-expose_global	Unsupported	Global variables are not supported in Vitis HLS for exposure as top-level ports in the IP or kernel.
config	config_interface	-trim_dangling_port	Unsupported	
config	config_array_partition	-auto_promotion_threshold	Deprecated	
config	config_array_partition	-auto_partition_threshold	Deprecated	Has been renamed to -complete_threshold
config	config_array_partition	-scalarize_all	Unsupported	
config	config_array_partition	-throughput_driven	Unsupported	
config	config_array_partition	-maximum_size	Unsupported	
config	config_array_partition	-include_extern_globals	Unsupported	
config	config_array_partition	-include_ports	Unsupported	

Table 66: **Vivado HLS Commands Deprecated in Vitis HLS** *(cont'd)*

Type	Command	Option	Vitis HLS	Details
config	config_schedule	All options but -enable_dsp_fill_reg	Deprecated	
config	config_bind	* (all options)	Deprecated	
config	config_rtl	-encoding	Deprecated	The FSM encoding is one-hot.
config	config_sdx	* (all options)	Deprecated	
config	config_flow	* (all options)	Deprecated	
config	config_dataflow	-disable_start_propagation	Deprecated	
config	config_rtl	-auto_prefix	Deprecated	Replaced by config_rtl -module_prefix.
config	config_rtl	-prefix	Deprecated	Replaced by config_rtl -module_prefix.
config	config_rtl	-m_axi_conservative_mode	Deprecated	Use config_interface -m_axi_conservative_mode
directive/pragma	set_directive_pipeline	-enable_flush	Deprecated	
directive/pragma	CLOCK	*	Unsupported	
directive/pragma	DATA_PACK	*	Unsupported	Use AGGREGATE pragma or directive, and __attribute__(packed(X)) if needed.
directive/pragma	INLINE	-region	Deprecated	
directive/pragma	INTERFACE	-mode ap_bus	Unsupported	Use m_axi instead.
directive/pragma	INTERFACE	-mode ap_stable	Deprecated	Use STABLE pragma or directive instead.
directive/pragma	ARRAY_MAP	*	Unsupported	
directive/pragma	RESOURCE	*	Deprecated	Replaced by BIND_OP and BIND_STORAGE pragmas and directives. Use INTERFACE pragma or directive with storage_type option for top function arguments.
directive/pragma	SHARED	*	Deprecated	The SHARED pragma or directive has been moved to the type=shared option of the STREAM pragma or directive.
directive/pragma	STREAM	-dim	Unsupported	

Table 66: **Vivado HLS Commands Deprecated in Vitis HLS** *(cont'd)*

Type	Command	Option	Vitis HLS	Details
directive/pragma	`STREAM`	`-off`	Deprecated	`STREAM off` has become `STREAM type=pipo`
project	`csim_design`	`-clang_sanitizer`	Add/Rename	
project	`export_design`	`-use_netlist`	Deprecated	Replaced by: `export_design -format ip_catalog`
project	`export_design`	`-xo`	Deprecated	Replaced by: `export_design -format xo`
project	`add_files`		Unsupported	System-C files are not supported by Vitis HLS.
config	`config_export`	`-disable_deadlock_detection`	Deprecated	Replaced by: `config_export -deadlock_detection_sim`

Notes:

1. Deprecated: A warning message for discontinuity of the pragma in a future release will be issued.
2. Unsupported: Vitis HLS errors out with a valid message.
3. *: All the options in the command.

The following libraries are deprecated.

Table 67: **Deprecated Libraries**

Library	API	Deprecated	Suggested Replacement / Action
Linear Algebra Lib hls_linear_algebra.h hls/hls_axi_io.h hls/hls_linear_algebra_io.h hls/linear_algebra |-- deprecated | |-- x_complex_back_substitute.h | |-- x_complex_cholesky.h | |-- x_complex_cholesky_inverse.h | |-- x_complex_matrix_multiply.h | |-- x_complex_matrix_tb_utils.h | |-- x_complex_matrix_utils.h | |-- x_complex_qr_inverse.h | |-- x_complex_qrf.h | `-- x_complex_svd.h |-- hls_back_substitute.h |-- hls_cholesky.h |-- hls_cholesky_inverse.h |-- hls_matrix_multiply.h |-- hls_qr_inverse.h |-- hls_qrf.h |-- hls_svd.h `-- utils |-- std_complex_utils.h |-- x_hls_complex.h |-- x_hls_matrix_tb_utils.h `-- x_hls_matrix_utils.h	**cholesky** float, ap_fixed, x_complex<float>, x_complex<ap_fixed>	deprecated	Vitis Solver **potrf** float, double
	cholesky_inverse float, ap_fixed, x_complex<float>, x_complex<ap_fixed>		Vitis Solver **pomatrixinverse** float, double
	matrix_multiply float, ap_fixed, x_complex<float>, x_complex<ap_fixed>		
	qrf float, x_complex<float>		Vitis Solver **geqrf** float, double
	qr_inverse float, x_complex<float>		
	svd float, x_complex<float>		Vitis Solver **gesvdj** float, double

Table 67: **Deprecated Libraries** *(cont'd)*

Library	API	Deprecated	Suggested Replacement / Action
DSP Lib hls_dsp.h hls/dsp |-- hls_atan2_cordic.h |-- hls_awgn.h |-- hls_cmpy.h |-- hls_convolution_encoder.h |-- hls_nco.h |-- hls_qam_demod.h |-- hls_qam_mod.h |-- hls_sqrt_cordic.h |-- hls_viterbi_decoder.h `-- utils |-- hls_cordic.h |-- hls_cordic_functions.h `-- hls_dsp_common_utils.h	**atan2** input: std::complex<ap_fixed> output: ap_ufixed	deprecated	HLS Math **atan2** ap_fixed/ap_ufixed/float/double
	sqrt unsigned binary fraction with 1 bit integer, unsigned int		HLS Math **sqrt** ap_fixed/ap_ufixed/float/double
	awgn(Additive white Gaussian noise) input: ap_ufixed output: ap_int		
	cmpy(complex number multiply) input: std::complex< ap_fixed > output: std::complex< ap_fixed >		
	convolution_encoder (for data transferring in channel with error correcting, used in pair with viterbi decoder) input: ap_uint output: ap_uint		
	viterbi_decoder(for data transferring in channel with error correcting, used in pair with convolution encoder) input: ap_uint output: ap_uint		
	nco (Numerically-Controlled Oscillator) input: ap_uint output: std::complex< ap_int >		

Unsupported Features

The following features are not supported in this release.

IMPORTANT! *HLS either issues a warning or error for all the unsupported features mentioned in this section.*

Pragmas

- Pragma DEPENDENCE on an argument that also has an `m_axi` INTERFACE pragma specifying a `bundle` with two or more ports is not supported.

```
void top(int *a, int *b) { // both a and b are bundled to m_axi port gmem
#prgama HLS interface m_axi port=a offset=slave bundle=gmem
#prgama HLS interface m_axi port=b offset=slave bundle=gmem
#pragma HLS dependence variable=a false
}
```

- Pragma INTERFACE no longer supports the `ap_bus` mode that was supported in AMD Vivado™ HLS. You should use the`m_axi` interface instead.

Top-Level Function Argument

IMPORTANT! *AMD Vitis™ HLS does not support the INTERFACE pragma inside sub-functions.*

Top-level argument with C data type of:

- enum or any use of enum (struct, array pointer of enum)
- ap_int<N> (where N is 1-32768)
- _Complex
- _Half, __fp16

HLS Video Library

The `hls_video.h` for video utilities and functions has been deprecated and replaced by the Vitis vision library. See the Migrating HLS Video Library to Vitis vision on GitHub for more details.

C Arbitrary Precision Types

Vitis HLS does not support C arbitrary precision types. AMD recommends using C++ types with arbitrary precision.

In addition, C++ arbitrary precision types in Vitis HLS are limited to a maximum width of 4096 bits, instead of 32K bits supported by Vivado HLS.

C Constructs

- Pointer cast are not supported.
- Virtual functions are not supported.

AMD

Appendix A

Tcl to Config File Command Map

The following table provides a map to Vitis HLS Tcl commands to the `v++ -c --mode hls` and `vitis-run` configuration file commands.

Table 68: **Tcl Project Commands**

Tcl Command	Option	Default	Value Type	Config File Command
add_files	appendflags	false	bool	syn.file_cflags syn.file_csimflags tb.file_cflags
add_files	blackbox	false	bool	syn.blackbox.file
add_files	cflags		string	syn.cflags syn.file_cflags tb.cflags tb.file_cflags
add_files	csimflags		string	syn.csimflags syn.file_csimflags
add_files	src_files		string	syn.file
add_files	tb	false	bool	tb.file
cosim_design	enable_tasks_with_m_axi		bool	
cosim_design	hwemu_trace_dir		string	
cosim_design	disable_binary_tv		bool	
cosim_design	stable_axilite_update		bool	
cosim_design	disable_dependency_check		bool	
cosim_design	enable_dataflow_profiling		bool	
cosim_design	enable_fifo_sizing		bool	
cosim_design	random_stall		bool	
cosim_design	user_stall		string	
cosim_design	disable_deadlock_detection		bool	
cosim_design	wave_debug		bool	
cosim_design	argv		string	
cosim_design	compiled_library_dir		string	
cosim_design	coverage		bool	
cosim_design	ldflags		string	
cosim_design	mflags		string	
cosim_design	O		bool	
cosim_design	rtl	verilog	enum	

Table 68: **Tcl Project Commands** *(cont'd)*

Tcl Command	Option	Default	Value Type	Config File Command
cosim_design	setup		bool	
cosim_design	tool	xsim	enum	
cosim_design	trace_level	none	enum	
cosim_stall	generate_burst_default		string	
cosim_stall	check		string	
cosim_stall	generate		string	
cosim_stall	list		bool	
create_clock	name		string	
create_clock	period	10ns	string	clock
csim_design	code_analyzer	0	bool	
csim_design	argv		string	
csim_design	clean		bool	
csim_design	ldflags		string	
csim_design	mflags		string	
csim_design	O		bool	
csim_design	setup		bool	
csynth_design	dump_post_cfg	0	bool	
csynth_design	synthesis_check	0	bool	
csynth_design	dump_cfg	0	bool	
delete_project	project		string	
delete_solution	solution		string	
enable_beta_device	pattern	*	string	
export_design	output		string	
export_design	description		string	
export_design	display_name		string	
export_design	flow	none	enum	
export_design	format	ip_catalog	enum	
export_design	ipname		string	
export_design	library		string	
export_design	rtl	verilog	enum	
export_design	taxonomy		string	
export_design	vendor		string	
export_design	version		string	
get_clock_period	units		enum	
get_clock_period	default	0	bool	
get_clock_period	name		string	
get_clock_uncertainty	default	1	bool	
get_clock_uncertainty	clock_name	default	string	
get_files	appfile		string	

Table 68: **Tcl Project Commands** *(cont'd)*

Tcl Command	Option	Default	Value Type	Config File Command
get_files	blackbox	false	bool	
get_files	filepaths	none	enum	
get_files	fileroot		string	
get_files	quiet	false	bool	
get_files	csimflags	false	bool	
get_files	cflags	false	bool	
get_files	tb	false	bool	
get_part	family	false	bool	
get_project	solutions	0	bool	
get_project	directory	0	bool	
get_project	name	0	bool	
get_solution	json	0	bool	
get_solution	directory	0	bool	
get_solution	flow_target	0	bool	
get_solution	name	0	bool	
help	ini	0	bool	
help	width	120	int	
help	cmd		string	
list_part	family	false	bool	
list_part	clock_regions	false	bool	
list_part	slr_pblocks	false	bool	
list_part	board	false	bool	
list_part	name		string	
open_project	diskless	0	bool	
open_project	upgrade	0	bool	
open_project	name		string	
open_project	reset	0	bool	
open_solution	flow_target		enum	flow_target
open_solution	name		string	
open_solution	reset	0	bool	
open_tcl_project	tclfile		string	
set_clock_uncertainty	clock_list		string	
set_clock_uncertainty	uncertainty		string	clock_uncertainty
set_part	board		string	board
set_part	value		string	part
set_top	name		string	syn.top

Table 69: **Tcl Configuration Commands**

Tcl Command	Option	Default	Value Type	Config File Command
config_array_partition	complete_threshold	4	uint	syn.array_partition.complete_threshold
config_array_partition	throughput_driven	auto	enum	syn.array_partition.throughput_driven
config_array_stencil	throughput_driven	off	enum	syn.array_stencil.throughput_driven
config_compile	design_size_maximum_warning	100000	int	syn.compile.design_size_maximum_warning
config_compile	pipeline_flush_in_task	ii1	enum	syn.compile.pipeline_flush_in_task
config_compile	ignore_long_run_time	0	bool	syn.compile.ignore_long_run_time
config_compile	pre_tcl		string	
config_compile	enable_auto_rewind	1	bool	syn.compile.enable_auto_rewind
config_compile	pipeline_style	frp	enum	syn.compile.pipeline_style
config_compile	pragma_strict_mode	0	bool	syn.compile.pragma_strict_mode
config_compile	name_max_length	80	uint	syn.compile.name_max_length
config_compile	no_signed_zeros	0	bool	syn.compile.no_signed_zeros
config_compile	pipeline_loops	64	uint	syn.compile.pipeline_loops
config_compile	unsafe_math_optimizations	0	bool	syn.compile.unsafe_math_optimizations
config_cosim	enable_tasks_with_m_axi		bool	cosim.enable_tasks_with_m_axi
config_cosim	argv		string	cosim.argv
config_cosim	compiled_library_dir		string	cosim.compiled_library_dir
config_cosim	coverage		bool	cosim.coverage
config_cosim	disable_binary_tv		bool	cosim.disable_binary_tv
config_cosim	disable_deadlock_detection		bool	cosim.disable_deadlock_detection
config_cosim	disable_dependency_check		bool	cosim.disable_dependency_check
config_cosim	enable_dataflow_profiling		bool	cosim.enable_dataflow_profiling
config_cosim	enable_fifo_sizing		bool	cosim.enable_fifo_sizing
config_cosim	hwemu_trace_dir		string	cosim.hwemu_trace_dir
config_cosim	ldflags		string	cosim.ldflags

Table 69: **Tcl Configuration Commands** *(cont'd)*

Tcl Command	Option	Default	Value Type	Config File Command
config_cosim	mflags		string	cosim.mflags
config_cosim	O		bool	cosim.O
config_cosim	random_stall		bool	cosim.random_stall
config_cosim	rtl	verilog	enum	cosim.rtl
config_cosim	setup		bool	cosim.setup
config_cosim	stable_axilite_update		bool	cosim.stable_axilite_update
config_cosim	tool	xsim	enum	cosim.tool
config_cosim	trace_level	none	enum	cosim.trace_level
config_cosim	user_stall		string	cosim.user_stall
config_cosim	wave_debug		bool	cosim.wave_debug
config_csim	code_analyzer	0	bool	csim.code_analyzer
config_csim	argv		string	csim.argv
config_csim	clean		bool	csim.clean
config_csim	ldflags		string	csim.ldflags
config_csim	mflags		string	csim.mflags
config_csim	O		bool	csim.O
config_csim	setup		bool	csim.setup
config_dataflow	override_user_fifo_depth	0	uint	syn.dataflow.override_user_fifo_depth
config_dataflow	disable_fifo_sizing_opt	0	bool	syn.dataflow.disable_fifo_sizing_opt
config_dataflow	task_level_fifo_depth	2	uint	syn.dataflow.task_level_fifo_depth
config_dataflow	strict_stable_sync	0	bool	syn.dataflow.strict_stable_sync
config_dataflow	default_channel	pingpong	enum	syn.dataflow.default_channel
config_dataflow	fifo_depth	2	uint	syn.dataflow.fifo_depth
config_dataflow	scalar_fifo_depth	2	uint	syn.dataflow.scalar_fifo_depth
config_dataflow	start_fifo_depth	2	uint	syn.dataflow.start_fifo_depth
config_dataflow	strict_mode	warning	enum	syn.dataflow.strict_mode
config_debug	directory	.debug	string	syn.debug.directory
config_debug	enable	0	bool	syn.debug.enable
config_export	xo_type	rtl	enum	package.output.xo_type
config_export	cosim_trace_generation	0	bool	syn.rtl.cosim_trace_generation
config_export	flow	none	enum	vivado.flow

Table 69: **Tcl Configuration Commands** *(cont'd)*

Tcl Command	Option	Default	Value Type	Config File Command
config_export	ip_xdc_file		string	package.ip.xdc_file
config_export	ip_xdc_ooc_file		string	package.ip.xdc_ooc_file
config_export	vivado_clock		string	vivado.clock
config_export	vivado_max_timing_paths	10	uint	vivado.max_timing_paths
config_export	vivado_pblock		string	vivado.pblock
config_export	output		string	package.output.file
config_export	description		string	package.ip.description
config_export	display_name		string	package.ip.display_name
config_export	format	ip_catalog	enum	package.output.format
config_export	ipname		string	package.ip.name
config_export	library		string	package.ip.library
config_export	rtl	verilog	enum	vivado.rtl
config_export	taxonomy		string	package.ip.taxonomy
config_export	vendor		string	package.ip.vendor
config_export	version		string	package.ip.version
config_export	vivado_impl_strategy	default	string	vivado.impl_strategy
config_export	vivado_optimization_level	0	enum	vivado.optimization_level
config_export	vivado_phys_opt	none	enum	vivado.phys_opt
config_export	vivado_report_level	2	enum	vivado.report_level
config_export	vivado_synth_design_args	-directive sdx_optimization_effort_high	string	vivado.synth_design_args
config_export	vivado_synth_strategy	default	string	vivado.synth_strategy
config_interface	m_axi_cache_impl	auto	enum	syn.interface.m_axi_cache_impl
config_interface	m_axi_auto_id_channel	0	bool	syn.interface.m_axi_auto_id_channel
config_interface	s_axilite_auto_restart_counter	0	enum	syn.interface.s_axilite_auto_restart_counter
config_interface	s_axilite_interrupt_mode	tow	enum	syn.interface.s_axilite_interrupt_mode
config_interface	s_axilite_mailbox	none	enum	syn.interface.s_axilite_mailbox
config_interface	s_axilite_status_regs	off	enum	syn.interface.s_axilite_status_regs
config_interface	s_axilite_sw_reset	0	bool	syn.interface.s_axilite_sw_reset
config_interface	m_axi_buffer_impl	bram	enum	syn.interface.m_axi_buffer_impl

Table 69: **Tcl Configuration Commands** *(cont'd)*

Tcl Command	Option	Default	Value Type	Config File Command
config_interface	m_axi_conservative_mode	1	bool	syn.interface.m_axi_conservative_mode
config_interface	m_axi_flush_mode	0	bool	syn.interface.m_axi_flush_mode
config_interface	m_axi_alignment_byte_size	1	uint	syn.interface.m_axi_alignment_byte_size
config_interface	m_axi_alignment_size	0	uint	syn.interface.m_axi_alignment_size
config_interface	m_axi_auto_max_ports	0	bool	syn.interface.m_axi_auto_max_ports
config_interface	m_axi_latency	0	uint	syn.interface.m_axi_latency
config_interface	m_axi_max_bitwidth	1024	uint	syn.interface.m_axi_max_bitwidth
config_interface	m_axi_max_read_burst_length	16	uint	syn.interface.m_axi_max_read_burst_length
config_interface	m_axi_max_widen_bitwidth	0	uint	syn.interface.m_axi_max_widen_bitwidth
config_interface	m_axi_max_write_burst_length	16	uint	syn.interface.m_axi_max_write_burst_length
config_interface	m_axi_min_bitwidth	8	uint	syn.interface.m_axi_min_bitwidth
config_interface	m_axi_num_read_outstanding	16	uint	syn.interface.m_axi_num_read_outstanding
config_interface	m_axi_num_write_outstanding	16	uint	syn.interface.m_axi_num_write_outstanding
config_interface	default_slave_interface	s_axilite	enum	syn.interface.default_slave_interface
config_interface	s_axilite_data64	0	bool	syn.interface.s_axilite_data64
config_interface	clock_enable	0	bool	syn.interface.clock_enable
config_interface	m_axi_addr64	1	bool	syn.interface.m_axi_addr64
config_interface	m_axi_offset	slave	enum	syn.interface.m_axi_offset
config_interface	register_io	off	enum	syn.interface.register_io
config_op	precision	standard	enum	syn.op
config_op	impl	all	string	syn.op
config_op	latency	-1	int	syn.op
config_op	op		string	syn.op
config_rtl	deadlock_detection	sim	enum	syn.rtl.deadlock_detection
config_rtl	register_all_io	0	bool	syn.rtl.register_all_io

Table 69: **Tcl Configuration Commands** *(cont'd)*

Tcl Command	Option	Default	Value Type	Config File Command
config_rtl	register_reset_num	0	int	syn.rtl.register_reset_num
config_rtl	header		string	syn.rtl.header
config_rtl	kernel_profile	0	bool	syn.rtl.kernel_profile
config_rtl	module_auto_prefix	1	bool	syn.rtl.module_auto_prefix
config_rtl	module_prefix		string	syn.rtl.module_prefix
config_rtl	mult_keep_attribute	0	bool	syn.rtl.mult_keep_attribute
config_rtl	reset	control	enum	syn.rtl.reset
config_rtl	reset_async	0	bool	syn.rtl.reset_async
config_rtl	reset_level	high	enum	syn.rtl.reset_level
config_schedule	enable_dsp_full_reg	1	bool	syn.schedule.enable_dsp_full_reg
config_storage	auto_srl_max_bits	1024	uint	syn.storage
config_storage	auto_srl_max_depth	2	uint	syn.storage
config_storage	impl	autosrl	string	syn.storage
config_storage	type		string	syn.storage
config_unroll	tripcount_threshold	0	uint	syn.unroll.tripcount_threshold

Table 70: **Tcl Optimization Directives**

Tcl Command	Config File Command
set_directive_aggregate	syn.directive.aggregate
set_directive_alias	syn.directive.alias
set_directive_allocation	syn.directive.allocation
set_directive_array_partition	syn.directive.array_partition
set_directive_array_reshape	syn.directive.array_reshape
set_directive_bind_op	syn.directive.bind_op
set_directive_bind_storage	syn.directive.bind_storage
set_directive_cache	syn.directive.cache
set_directive_dataflow	syn.directive.dataflow
set_directive_dependence	syn.directive.dependence
set_directive_disaggregate	syn.directive.disaggregate
set_directive_expression_balance	syn.directive.expression_balance
set_directive_function_instantiate	syn.directive.function_instantiate
set_directive_inline	syn.directive.inline
set_directive_interface	syn.directive.interface
set_directive_latency	syn.directive.latency

Table 70: **Tcl Optimization Directives** *(cont'd)*

Tcl Command	Config File Command
set_directive_loop_flatten	syn.directive.loop_flatten
set_directive_loop_merge	syn.directive.loop_merge
set_directive_loop_tripcount	syn.directive.loop_tripcount
set_directive_occurrence	syn.directive.occurrence
set_directive_performance	syn.directive.performance
set_directive_pipeline	syn.directive.pipeline
set_directive_protocol	syn.directive.protocol
set_directive_reset	syn.directive.reset
set_directive_resource	syn.directive.resource
set_directive_shared	syn.directive.shared
set_directive_stable	syn.directive.stable
set_directive_stream	syn.directive.stream
set_directive_top	syn.top
set_directive_unroll	syn.directive.unroll

AMD

Appendix B

Instruction/Operator Explanation

The following table offers a definition of some of the operators used in HLS compiler. Standard llvm instruction can be found here: https://llvm.org/docs/LangRef.html#instruction-reference.

Name	Description
FSqrt	Compute the square root of float
FRSqrt	Compute the reciprocal of square root of float
FRecip	Compute the reciprocal of float
FLog	Compute the Logarithmic of float
FExp	Compute the exponential of float
DSqrt	Compute the square root of double
DRSqrt	Compute the reciprocal of square root of double
DRecip	Compute the reciprocal of double
DLog	Compute the Logarithmic of double
DExp	Compute the exponential of double
BlackBox	Instantiate a blackbox function
BitSelect	Select a single bit of a variable
BitSet	Set a single bit of a variable
PartSelect	Select a range of bits of a variable
PartSet	Set a range of bits of a variable
BitConcatenate	Concatenate two variables
XorReduce	Compute XOR reduce of a variable defined by xor_reduce() in ap_(u)int types
XnorReduce	Compute XNOR reduce of a variable defined by xnor_reduce() in ap_(u)int types
AndReduce	Compute AND reduce of a variable defined by and_reduce() in ap_(u)int types
NandReduce	Compute NAND reduce of a variable defined by nand_reduce() in ap_(u)int types
OrReduce	Compute OR reduce of a variable defined by or_reduce() in ap_(u)int types
NorReduce	Compute NOR reduce of a variable defined by nor_reduce() in ap_(u)int types
Read	Read a value from an interface
Write	Write a value to an interface
NbRead	Non-block read a value from an interface
NbWrite	non-block write a value to an interface

Name	Description
ReadReq	Send a read request to a bus. Only available for m_axi.
WriteReq	Send a write request to a bus. Only available for m_axi.
WriteResp	Receive a write respond from a bus. Only for m_axi.
MemShiftRead	Shift the registers of SRL primitive
Mux	Select a value from multiple values

AMD

Appendix C

Additional Resources and Legal Notices

Finding Additional Documentation

Technical Information Portal

The AMD Technical Information Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Technical Information Portal, go to https://docs.amd.com.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help → Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools → DocNav**.
- At the Linux command prompt, enter `docnav`.

***Note*:** For more information on DocNav, refer to the *Documentation Navigator User Guide* (UG968).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- Go to the Design Hubs web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see Support.

References

These documents provide supplemental material useful with this guide:

1. *UltraScale Architecture DSP Slice* (UG579)
2. *Vivado Design Suite Tcl Command Reference Guide* (UG835)
3. *Introduction to FPGA Design with Vivado High-Level Synthesis* (UG998)
4. *Vivado Design Suite Tutorial: High-Level Synthesis* (UG871)
5. *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* (UG973)
6. *Floating-Point Operator LogiCORE IP Product Guide* (PG060)
7. *Fast Fourier Transform LogiCORE IP Product Guide* (PG109)
8. *FIR Compiler LogiCORE IP Product Guide* (PG149)
9. *DDS Compiler LogiCORE IP Product Guide* (PG141)
10. *Vivado Design Suite: AXI Reference Guide* (UG1037)
11. *Vitis Unified Software Platform Documentation: Embedded Software Development* (UG1400)
12. *Data Center Acceleration using Vitis* (UG1700)
13. *Embedded Design Development Using Vitis* (UG1701)
14. *Vitis Reference Guide* (UG1702)
15. *Versal Adaptive SoC DSP Engine Architecture Manual* (AM004)
16. *Floating-Point Design with Vivado HLS* (XAPP599)
17. Option Summary page on the GCC website (gcc.gnu.org/onlinedocs/gcc/Option-Summary.html)
18. Accellera website (http://www.accellera.org/)
19. AWGN page on the MathWorks website (http://www.mathworks.com/help/comm/ug/awgn-channel.html)
20. AMD Vivado™ Design Suite Documentation

Revision History

The following table shows the revision history for Section II: HLS Programmers Guide.

Section	Revision Summary
03/31/2025 Version 2024.2	
Entire section	No changes to this section.
11/18/2024 Version 2024.2	
Entire section	No changes to this section.
07/03/2024 Version 2024.1	
Entire section	No changes to this section.
05/30/2024 Version 2024.1	
Mixing Data-Driven and Control-Driven Models Execution Modes of HLS Designs	Updated Sections.

The following table shows the revision history for Section III: Creating Vitis HLS Components.

Section	Revision Summary
11/13/2024 Version 2024.2	
Component Comparison Feature	Updated Sections.
07/03/2024 Version 2024.1	
Component Comparison Feature L1 Library Wizard Flow	Updated Sections.
05/30/2024 Version 2024.1	
Introduction to Vitis HLS Components Refactoring C++ Source Code for HLS Adding RTL Blackbox Functions Using the RTL Blackbox Wizard C++ Model and Header Files RTL IP Definition JSON File for RTL Blackbox Component Comparison Feature L1 Library Wizard Flow	Updated Sections and New Section.

The following table shows the revision history for Section IV: Vitis HLS Command Reference.

Section	Revision Summary
11/13/2024 Version 2024.2	
Entire section	No changes to this section.
07/03/2024 Version 2024.1	
Entire section	No changes to this section.

Section	Revision Summary
05/30/2024 Version 2024.1	
set_directive_inline set_directive_interface pragma HLS inline	Updated Sections.

The following table shows the revision history for Section VI: Vitis HLS Libraries Reference.

Section	Revision Summary
11/13/2024 Version 2024.2	
Entire section	No changes to this section.
07/03/2024 Version 2024.1	
Entire section	No changes to this section.
05/30/2024 Version 2024.1	
Vitis HLS Libraries Reference: C++ Arbitrary Precision Fixed-Point Types HLS Stream Library FFT IP Library	Minor Updates.

The following table shows the revision history for Section VII: Vitis HLS Migration Guide.

Section	Revision Summary
11/13/2024 Version 2024.2	
Entire section	No changes to this section.
07/03/2024 Version 2024.1	
Entire section	No changes to this section.
05/30/2024 Version 2024.1	
Entire section	No changes to this section.

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to

revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2020-2025 Advanced Micro Devices, Inc. AMD, the AMD Arrow logo, Alveo, UltraScale, UltraScale+, Versal, Virtex, Vitis, Vivado, Zynq and combinations thereof are trademarks of Advanced Micro Devices, Inc. OpenCL and the OpenCL logo are trademarks of Apple Inc. used by permission by Khronos. PCI, PCIe, and PCI Express are trademarks of PCI-SIG and used under license. AMBA, AMBA Designer, Arm, ARM1176JZ-S, CoreSight, Cortex, PrimeCell, Mali, and MPCore are trademarks of Arm Limited in the US and/or elsewhere. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.